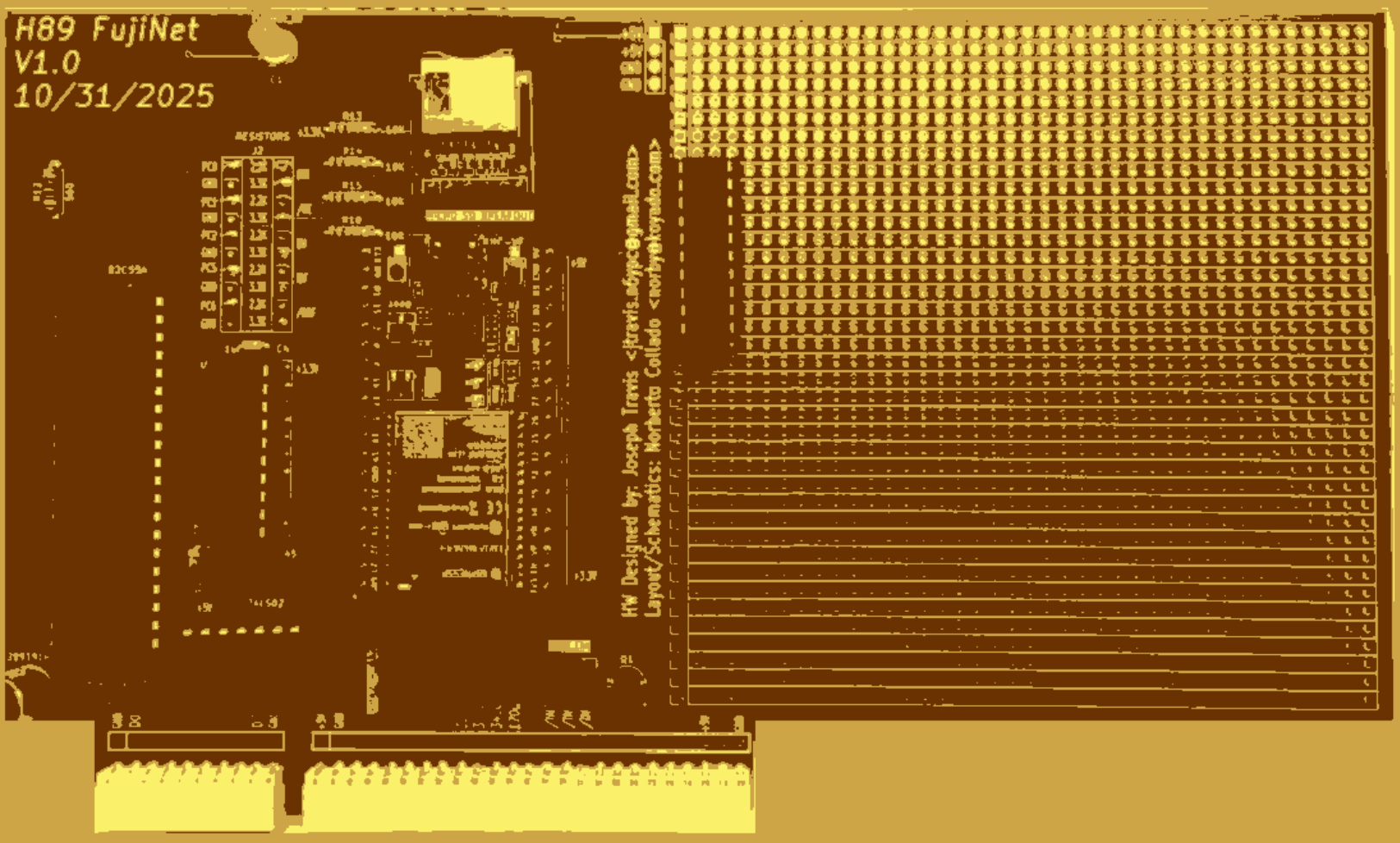
FUJINET MANUAL

for the

NETWORK ADAPTER

Model H89 FujiNet

PROGRAMMING FN-89-02



FUJINET PROJECT DIRECTORY

The following addresses go straight to the people and things listed:

The FujiNet project, news and downloads	fujinet.online
FujiNet-Flasher (firmware updates)	fujinet.online/download
Help from other owners, any hour (Discord)	discord.gg/7MfFTvD
Source code for everything in this manual	github.com/FujiNetWIFI
The FujiNet firmware	github.com/FujiNetWIFI/fujinet-firmware
fujinet-lib, the C library	github.com/FujiNetWIFI/fujinet-lib
z88dk, the C compiler and Z80 assembler	z88dk.org
The FNDISK drivers for HDOS and CP/M	github.com/FujiNetWIFI/fujinet-h89
The FujiNet Wiki	github.com/FujiNetWIFI/fujinet-firmware/wiki
Corrections to this manual	github.com/FujiNetWIFI/fujinet-manuals

YOUR FUJINET GUARANTEE

If you are not satisfied with your FujiNet – the card, its programs, or this manual – tell us. There is no Director of Customer Services, but there are a great many owners who will make certain your problem receives immediate, personal attention, usually within the hour, and usually from someone who has already met it on their own bench.

Our guarantee is a short one, and needs no attorney. Every part of the FujiNet is yours to see: the schematics, the firmware, fujinet-lib, CONFIG, the FNDISK drivers and every program printed in this manual are free software, published under the GNU General Public License, version 3. You may read them, build them, change them and give them away, as long as you pass the same freedom along.

The FujiNet is provided as it is, without warranty of any kind. We are not responsible for incidental or consequential damages, for a cold solder joint you made yourself, or for the hours you will lose browsing disk images you had forgotten you owned. We are, however, always glad to hear what you did with it.

Heath, Heathkit, HDOS and the model numbers of Heath equipment are trademarks of their owners. Zenith and Z-89 are trademarks of their owners. CP/M is a trademark of its owner. FujiNet is a community project, and is not affiliated with, endorsed by, or sponsored by Heath Company, Zenith Data Systems, or their successors. This manual borrows the look of Heath's 1979 Operation Manual for the H88 out of affection for it.

FujiNet Manual

for the

NETWORK ADAPTER

Model H89 FujiNet

PROGRAMMING FN-89-02

FUJINET PROJECT
FUJINET.ONLINE

Copyright © 2026
Thomas Cherryhomes
Free software: GNU GPL, version 3
Printed wherever you are

TABLE OF CONTENTS

INTRODUCTION 1-5
What You Need 1-5
How This Manual Is Arranged 1-6
Conventions 1-7
Reading a Command Card 1-7
The Software This Manual Describes 1-8
How the Examples Were Checked 1-8

PROGRAMMING MODEL 2-1
General 2-1
The Ports 2-1
Port C 2-2
Starting the FujiNet 2-2
Sending a Byte 2-3
Receiving a Byte 2-4
Time 2-5
No Interrupts 2-6
Sharing the Card 2-6
Memory 2-6

THE FUJIBUS PROTOCOL 3-1
General 3-1
Devices 3-1
Frames 3-1
The Header 3-2
The Checksum 3-2
Fields 3-3
The Payload 3-3
The Reply 3-4
A Complete Exchange 3-4
The Rules 3-5

ASSEMBLY LANGUAGE 4-1
General 4-1
Setting Up z88dk 4-1
A First Program 4-1
Assembling 4-3
Carrying It to the H89 4-3
Running FNHELLO 4-3
The Program on HDOS 4-3
The Program on CP/M 4-4
The Library 4-4
Making a Request 4-5
Debugging 4-6
Listing of the Library 4-6

ASSEMBLY ON THE H89 ITSELF 5-1
General 5-1
Heath's ASM, Under HDOS 5-1
FNONH 5-2
Digital Research's ASM, Under CP/M 5-3
A FujiBus Exchange in 8080 Code 5-3
The Listings 5-4

C WITH Z88DK 6-1
General 6-1
What You Need 6-1
A First Program 6-1
Compiling for CP/M 6-2
Compiling for HDOS 6-2
Where Everything Ends Up 6-4
The Console 6-4
Files 6-6
Mixing C and Assembly 6-6
fujinet-lib's Own Tests 6-6

FUJINET-LIB REFERENCE 7-1
General 7-1
The Header Files 7-1
What Functions Return 7-2
Structures 7-2
The Bus Call 7-3
Functions That Will Not Link 7-4
The FujiNet Device 7-4
App Keys 7-11
Network Channels 7-12
The Clock 7-18
QR Codes 7-19
The Bus 7-21
Helpers 7-22

THE FUJINET DEVICE 8-1
General 8-1
The Examples in This Chapter 8-1
Adapter and WiFi 8-2
Host Slots 8-5
Directories 8-7
Drive Slots 8-9
Booting 8-12
App Keys 8-13
Base64 8-15
Hashes 8-16
QR Codes 8-19
Odds and Ends 8-20
Commands the H89 Does Not Have 8-22

THE DISK DRIVES 9-1
General 9-1
Sectors and Images 9-1
The Commands 9-1
When There Is No Disk 9-5

THE NETWORK DEVICE 10-1
General 10-1
The Examples in This Chapter 10-1
Opening and Closing 10-2
Status 10-4
Reading and Writing 10-5
JSON and Other Parsers 10-7
HTTP 10-9
TCP Servers 10-14
UDP 10-16
Files and Directories 10-17
Channel Settings 10-22
Commands the Network Device Does Not Have 10-24

NETWORK PROTOCOLS 11-1
General 11-1
The Devicespec 11-1
The Families 11-1
File Systems 11-2
Streams 11-3
Utilities 11-3
Mail and Calendars 11-4
Which Were Tried 11-4

CLOCK, PRINTER AND MODEM 12-1
General 12-1
The Clock 12-1
The Printer 12-6
The Modem 12-8

NETCAT 13-1
General 13-1
Using NETCAT 13-1
How It Works 13-2
The Assembly Version 13-2
The C Version 13-3
Listings 13-3

NGET AND NPUT 14-1
General 14-1
Using NGET and NPUT 14-1
How Long Is a File? 14-2
How NGET Works 14-2
How NPUT Works 14-3
Getting the Names 14-4
Listings 14-4

ISS TRACKER 15-1
General 15-1
Running ISS 15-1
The FujiNet's JSON Parser 15-2
The Assembly Version 15-2
The C Version 15-4
Listings 15-5

THE NETWORK GAMES 16-1
General 16-1
The Game System 16-1
How the Code Is Arranged 16-2
fujinet-lib in the Games 16-3
Building the Games 16-3
Licences 16-4

TEXAS HOLD'EM 17-1
General 17-1
Playing 17-1
How It Works 17-2
The Files 17-3
Listing 17-3

BATTLESHIP 18-1
General 18-1
Playing 18-1
How It Works 18-2
The Files 18-3
Listing 18-3

FUJITZEE 19-1
General 19-1
Playing 19-1
How It Works 19-2
The Files 19-3
Listing 19-3

APPENDIX 20-1
Network Error Codes 20-1
Command Quick Reference 20-2
The H19's Escape Sequences 20-3
The H19's Graphics Characters 20-3
CP/M 2.2: The BDOS and the BIOS 20-4
Firmware Behaviour and Errata 20-4
Glossary 20-9

INTRODUCTION

This Manual is for the H89 owner who wants to write programs that use the FujiNet. Its companion, the Operation Manual, shows how to install the card, join a network and put disk images in the FujiNet drives. This one goes underneath. It shows how a program talks to the card, every command the FujiNet answers, and the programs that use them.

Everything in it is shown three ways, so you can work the way you like best:

- In **Z80 assembly language**, assembled on another computer with z88dk's assembler and carried to the H89 on a disk image. One source file makes both an HDOS program and a CP/M program.
- In **C**, compiled on another computer with z88dk and the FujiNet project's library, fujinet-lib.
- As the **bytes** that went between the H89 and the FujiNet when the example ran, so you can write the program in any language you please, BASIC included.

If you would rather stay at the H89′s own keyboard, Chapter 5 shows how to assemble a FujiNet program with the 8080 assemblers that come with HDOS and with CP/M.

The Manual ends with whole programs, fully listed and explained:

- NETCAT, a terminal for bulletin boards and other computers on the network;
- NGET and NPUT, which copy files between your disks and the network;
- an ISS tracker that reads the space station's position from the Internet and plots it on the H19;
- three network games, **Texas Hold'em**, **Battleship** and **Fujitzee**, which you play against other people on other kinds of computers.

WHAT YOU NEED

- An H89 (or H88, or Z-89) with the FujiNet card installed and working, as the Operation Manual describes. FujiNet Go H89, the H89 emulator, will do as well. Every example in this Manual was run on it.
- HDOS 2.0, or Heath/Zenith CP/M 2.2.04. The programs were run on a full-memory H89 (one HDOS reports as 56K). All but Fujitzee should fit a 48K machine, but were not tried in one; Fujitzee needs the full memory under HDOS.
- For Z80 assembly and C: a modern computer with z88dk (Linux, macOS or Windows). Chapter 4 shows how to set it up. For C, also fujinet-lib, which Chapter 6 shows how to build.

- A way to carry programs to the H89: the FujiNet itself (put the disk image on its microSD card or on a TNFS server, and mount it), or FujiNet Go H89's disk drives.
- For Chapter 5 only: the HDOS 2.0 Software Tools diskette, or the CP/M 2.2.04 distribution diskettes, which carry the 8080 assemblers.

HOW THIS MANUAL IS ARRANGED

2 **Programming Model**. The card as the H89 sees it: four ports, a handshake, a clock.

3 **The FujiBus Protocol**. How a request and its reply are framed, byte by byte.

4 **Assembly Language**. The book's Z80 library, and building programs for HDOS and CP/M.

5 **Assembly on the H89**. The same, with Heath's assembler and Digital Research's.

6 **C with z88dk**. The compiler, fujinet-lib, and getting a C program onto the H89.

7 **fujinet-lib Reference**. Every function in the library.

8 **The FujiNet Device**. Device 70H: WiFi, hosts, directories, drives, app keys and more.

9 **The Disk Drives**. Devices 31H to 38H, sector by sector.

10 **The Network Device**. Devices 71H to 78H: eight channels to the network.

11 **Network Protocols**. The 28 kinds of place a channel can open.

12 **Clock, Printer, Modem**. Devices 45H, 40H and 50H.

13 **NETCAT**. A terminal program, in assembly and in C.

14 **NGET and NPUT**. Copying files to and from the network.

15 **ISS Tracker**. Reading JSON from a web service.

16 **The Network Games**. How the three games work, and their server.

17-19 **The Games**. Texas Hold'em, Battleship and Fujitzee, complete.

20 **Appendix**. Error codes, a quick reference, the H19, HDOS and CP/M, errata.

CONVENTIONS

NUMBERS

Heath wrote its numbers in octal, and the H89's front panel and monitor still do. The FujiNet's firmware, and nearly everything written about it since, uses hexadecimal. This Manual leads with hexadecimal and gives the octal beside it wherever an H89 owner would look for it:

- **70H** is hexadecimal: 7 sixteens and no units, 112 decimal.
- **160Q** is the same number in octal, as Heath's manuals write it.
- **042.200** is an address in Heath's **split octal**: the high byte and the low byte, each in octal. It is 2280H, where HDOS loads programs.

Numbers with no letter after them are decimal.

NAMES

Every device and command is called by the name the FujiNet firmware gives it, such as DEV_FUJINET and FUJI_GET_ADAPTERCONFIG. The same names are defined for you in the book's assembly library (Chapter 4), so a program can say what it means. fujinet-lib has names of its own for the same things; Chapter 7 gives both.

WHAT YOU TYPE

In the sessions, what you type is underlined, and ↵ means press the RETURN key. The screens are drawn in the H19's own characters, taken from its character generator ROM.

H89

"H89" means the H88 and the Z-89 as well. They are the same computer to the FujiNet.

READING A COMMAND CARD

Chapters 8 through 12 give each FujiNet command a card, followed by what you need to know to use it. In order, a card shows:

1. The command's name, and its number at the right.
2. The device it goes to and the command number, in hexadecimal and in octal, and the name the firmware and the book's library give it.
3. Any tags (see below).
4. FIELDS: the small numbers that go in the request's header (Chapter 3), in order.
5. PAYLOAD: the data that follows them, if any.
6. REPLY: what comes back.

Then come the command's description, and any NOTES and WARNINGS. Three examples close the entry:

- the command in Z80 assembly;
- the same in C;
- the bytes that went each way when the example ran, shaded to show which part of the frame each one is (the key is in Chapter 3).

All three are cut from the exerciser programs (Chapter 8), which were built and run for this Manual under both HDOS and CP/M. Here is a short one, from Chapter 8:

GET WIFI STATUS FAH

DEVICE 70H (160Q) COMMAND FAH (372Q)

STAND-IN WIFI

FIELDS	none
PAYLOAD	none
REPLY	1 byte: 3, connected; 6, not

Whether the FujiNet is on its network.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_wifist            ; is it connected?
        call    fn_call
        jr      nz,wifist_x
        ld      a,(wifist)              ; 3 = connected, 6 = not
        call    pdec8
wifist_x:
...
rq_wifist:
        FNREQ   DEV_FUJINET, FUJI_GET_WIFISTATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, wifist, 1
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_wifi_status(&status);
if (ok)                             /* 3 = connected, 6 = not */
    printf("%u\n", status);
```

H89 → FUJINET

C0	70	FA	06	00	71	00	C0

FUJINET → H89 · ACK · 1 BYTE

C0	70	06	07	00	80	00	03	C0

TAGS

A few commands cannot be shown the ordinary way. Their cards say so with a tag:

TAG	MEANS
STAND-IN WIFI	The example ran on fujinet-pc, which has no radio of its own. The networks it reports stand in for real ones.

NOT SENT, COMPUTED	The command would stop the example program, so its bytes were worked out rather than sent. What the FujiNet does with it is cited from the firmware's source.
UNFRAMED	The device answers with bare bytes rather than a FujiBus reply.
OUTPUT CHECKED ON HOST	The command's effect is outside the H89 (paper, a file), and was checked there.
NAK	The FujiNet refuses the command on the H89.

THE SOFTWARE THIS MANUAL DESCRIBES

The FujiNet for the H89 is new, and its software is still changing. This Manual describes these versions, which are the ones its examples ran against:

PROGRAM	VERSION
Firmware	fujinet-firmware, branch h89-disk-fixes, 7ab84ae64 (reports itself as v1.6)
fujinet-lib	branch h89-setup-after-reset, d08f584 (the H89 is not yet in a release). The programs of Chapters 13 to 19 and the library's own tests ran with 9e6d4dd, which differs only in how it starts the card (L22).
z88dk	v2.4-1231 (3bd06cad56)
FujiNet Go H89	51f411d, which resets the card's 82C55A as a real H89 does. The programs of Chapters 13 to 19 and the library's tests ran on v0.1.0-3 (8dc059d).
FNDISK, FN.DVD	fujinet-h89, branch card-setup-mark, 4cb0024
FNON	FujiNet Go H89, 51f411d

Where the firmware or the library does something it should not, the entry says so in a WARNING, with what to do instead. The examples all work as printed. The Appendix lists every such case in one place, with the line of source it comes from, so it can be checked against later versions.

HOW THE EXAMPLES WERE CHECKED

Every program in this Manual was built from the source printed here and run on FujiNet Go H89 under both HDOS 2.0 and CP/M 2.2.04. The emulator's card was connected to a FujiNet built from the firmware above (fujinet-pc, the FujiNet that runs on a personal computer), through a small program that recorded every byte in both directions. The byte strips printed with each command are those recordings, unedited.

The programs that use the network talked to small servers on the same computer, so they give the same answers every time. The screens in Chapters 13 through 19 were taken the same way.

Nothing in this Manual was run on a real H89 FujiNet card. The card and fujinet-pc run the same firmware, built for the same bus, so they should agree; where they could differ (WiFi, the microSD card, timing), the entry says so.

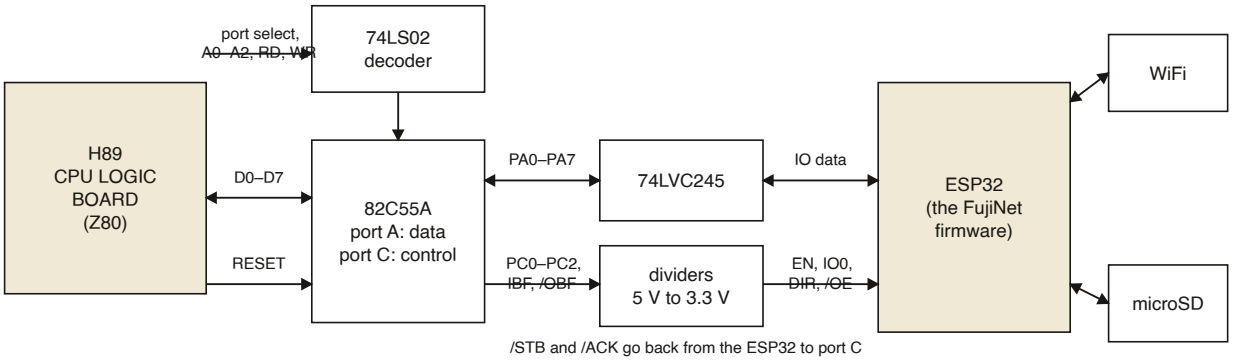
PROGRAMMING MODEL

GENERAL

To a program, the FujiNet card is four I/O ports and a clock. There is no memory on the card for the H89 to see, no interrupt, and no ROM. A program writes bytes to one port and reads bytes from it, watches a second port to know when it may, and uses the H89′s 2-millisecond clock to give up if the FujiNet does not answer.

Those bytes make **requests** and **replies**, in the FujiBus protocol (Chapter 3). Everything the FujiNet can do — join a network, mount a disk image, fetch a web page — is a request from the H89 and a reply from the FujiNet, one at a time. The FujiNet never speaks first.

This chapter covers the ports and the handshake. Most programs never deal with them directly: they call `fn_call` in the book's assembly library (Chapter 4), or a function in fujinet-lib (Chapter 6). But it is worth knowing what those do, and you will need it if you write for the card in another language.



PICTORIAL 2-1
The path from the H89 to the FujiNet.

THE PORTS

The card answers at four consecutive ports, set by its PORT SEL jumper. This Manual, the book's library and fujinet-lib all assume the setting the Operation Manual recommends, 320Q:

PORT	OCTAL	USE
D0H	320Q	PORTA. The data: every byte to and from the FujiNet goes through it.
D1H	321Q	PORTB. Wired to nothing. Holds a mark once the card is set up (below).
D2H	322Q	PORTC. Status (read) and control (written).
D3H	323Q	PCTRL. The 82C55A's control register.

If your card is set to another group (330Q, 340Q or 170Q), change the four equates at the top of `fujinet.inc` in the book's library. For C, change `IOPORT` in fujinet-lib's `bus/h89/portio.c` and rebuild the library (Chapter 6).

Port D0H is the 82C55A's port A in Mode 2: one byte in each direction can wait there. A byte the H89 writes stays in port A's output latch until the ESP32 takes it, and a byte the ESP32 sends stays in its input latch until the H89 reads it. Port C tells the H89 which latch is full.

PORT C

Reading port D2H gives the 82C55A's port C. The top five bits are the handshake; the bottom three are the outputs that start the ESP32 and steer the bus transceiver, read back:

BIT	NAME	MASK	MEANING
PC7	/OBF	80H	0 while a byte the H89 wrote waits for the ESP32. `PC_OBF`.
PC6	/ACK	40H	From the ESP32: 0 while it takes the byte. `PC_ACK`.
PC5	IBF	20H	1 while a byte from the ESP32 waits for the H89. `PC_IBF`.
PC4	/STB	10H	From the ESP32: 0 while it puts a byte in. `PC_STB`.
PC3	INTR	08H	Interrupt request. Not connected (J3 open).
PC2	EN	04H	1 = the ESP32 may run. Trust it only with the mark in port B. `PC_EN`.
PC1	/OE	02H	0 = the bus transceiver is on. `PC_OE`.
PC0	DIR	01H	1 = toward the ESP32, 0 = toward the H89. `PC_DIR`.

A program changes PC0, PC1 and PC2 one at a time with the 82C55A's bit set/reset command: writing 00000bbs (binary) to PCTRL sets bit **bbb** of port C to **s**. So `DIR_TO_ESP` (01H) sets DIR, `DIR_TO_H89` (00H) clears it, and `OE_ON` (02H) clears /OE.

NOTE: Writing a mode word to PCTRL clears all of port C's outputs, and EN with them. Write one only when the card has not been set up, and raise EN again with the next instruction, as `card_on` does.

STARTING THE FUJINET

When the H89 is reset, the 82C55A's ports become inputs and port A leaves Mode 2. Nothing can be sent until a program sets the card up again. FN.DVD, FNDISK, FNON and CONFIG all do it, and so does every program in this Manual, with `card_on`:

CARD.INC — START THE FUJINET IF NOTHING HAS

```
        in      a,(PORTB)               ; set up since the last reset?
        cp      SETUP_MARK
        jr      nz,card_start
        in      a,(PORTC)
        and     PC_EN
        jr      nz,card_ready           ; yes, and running: leave it
card_start:
        ld      a,MODE2                 ; mode 2; PC0-PC2 out, all 0
        out     (PCTRL),a
        ld      a,0x06                  ; EN=1, IO0=1 at once
        out     (PORTC),a
        ld      b,100                   ; about 200 ms with IO0 high
card_wait:
        call    tick
        djnz    card_wait
        ld      a,SETUP_MARK            ; the card is set up
        out     (PORTB),a
card_ready:
        in      a,(PORTA)               ; drop a byte latched at start
        ld      a,OE_ON
        out     (PCTRL),a
        ld      a,DIR_TO_H89
        out     (PCTRL),a
        ret
```

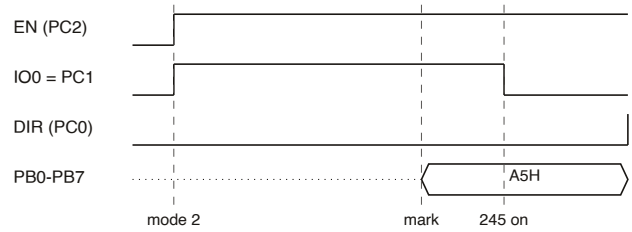
A program must not set the card up again when another already has: the mode word drops EN, and a FujiNet that restarts throws away the drives FNDISK has mounted and the network connections other programs have open. EN cannot tell which it is. After a reset the 82C55A holds its inputs high with weak pull-ups, and PC2 can read 1 before anything has set the card up. That is how the first HDOS CONFIG, which trusted EN, hung on a real H89.

So whatever sets the card up leaves a mark, A5H, in port B, which is wired to nothing. A reset makes port B an input that reads FFH, and any other mode word clears it or makes it an input. `card_on` leaves the card alone when it finds the mark and EN = 1. Otherwise it sets Mode 2 and, with the next instruction, brings EN up with PC1 (and so the ESP32's IO0 strap) high, which tells the ESP32 to run the firmware in its flash. A running ESP32 sees EN low for only a few microseconds; one held in reset starts. `card_on` waits 200 ms with IO0 still high while the ESP32 reads its straps, leaves the mark, throws away any byte latched while the ESP32 woke up, and turns the bus transceiver on facing the H89.

fujinet-lib does the same the first time a program calls it (from d08f584 on: L22).

NOTE: Heath's CP/M BIOS sends a mode word to 323Q at every cold boot, for the serial port that is usually at 320Q. It leaves port A in Mode 0, so under CP/M too nothing works until a program sets the card up.

NOTE: The firmware needs a little while after EN rises before it answers. A request sent sooner waits in port A until it does; `fn_call` allows 15 seconds for a reply to begin.



PICTORIAL 2-2

Starting the ESP32 after a reset. EN and PC1 rise with the instruction after the mode word. PC1 falls when the transceiver is turned on.

IS IT THERE?

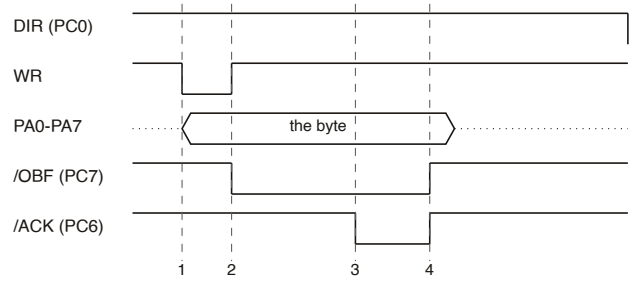
With no card at the selected ports, every port reads FFH. A program that waits for IBF would see a byte at once, and another, forever. `card_present` checks the three output bits instead, which `card_on` has left at EN = 1, /OE = 0, DIR = 0:

CARD.INC — Z IF THE CARD IS THERE AND RUNNING

```
card_present:
        in      a,(PORTC)
        and     0x07                    ; EN, /OE, DIR
        cp      PC_EN
        ret
```

SENDING A BYTE

To send, the program sets DIR to 1, so the transceiver drives toward the ESP32. Then, for each byte, it waits until /OBF is high, which means port A's output latch is empty, and writes the byte to port A:



PICTORIAL 2-3

Sending a byte. (1) The H89 writes port A. (2) /OBF falls. (3) The ESP32 takes the byte. (4) /OBF rises: the next byte may be written.

FUJIBUS.INC — SEND THE BYTE IN A

```
; wait until the ESP32 has taken the last byte (/OBF high). Keeps BC DE HL.
fn_txwait:
        in      a,(PORTC)
        and     PC_OBF
        ret     nz
        push    hl
        push    de
        ld      hl,(TICKS)
        ld      (fn_t0),hl
fn_txw1:
        in      a,(PORTC)
        and     PC_OBF
        jr      nz,fn_txw2
        call    fn_expired
        jr      nc,fn_txw1
        jp      fn_abort
fn_txw2:
        pop     de
        pop     hl
        ret

; send A. Keeps BC, DE, HL.
fn_putc:
        push    af
```

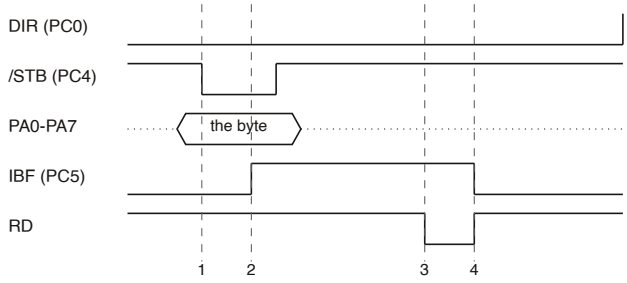
```
        call    fn_txwait
        pop     af
        out     (PORTA),a
        ret
```

After the last byte of a request, the program waits once more for /OBF to rise, so it knows the ESP32 has the whole request, and sets DIR back to 0.

WARNING: The firmware waits at most 100 ms for each byte of a request after the first. A program that pauses longer in the middle of a request — to compute a checksum the slow way, say, or because it is interpreted — loses the whole request: the FujiNet sends no reply, and treats the rest of the bytes as data for its modem (Chapter 12). Build the request in memory first, then send it in one go.

RECEIVING A BYTE

To receive, DIR must be 0. The ESP32 puts a byte in port A's input latch and IBF goes high; the program reads port A, which clears IBF and lets the ESP32 send the next:



PICTORIAL 2-4

Receiving a byte. (1) The ESP32 strobes the byte in. (2) IBF rises. (3) The H89 reads port A. (4) IBF falls.

FUJIBUS.INC — THE NEXT BYTE INTO A, OR GIVE UP

```
; the next byte from the ESP32 into A. Keeps BC, DE, HL.
fn_getc:
        in      a,(PORTC)
        and     PC_IBF
        jr      nz,fn_take
        push    hl
        push    de
        ld      hl,(TICKS)
        ld      (fn_t0),hl
fn_getc1:
        in      a,(PORTC)
        and     PC_IBF
        jr      nz,fn_getc2
        call    fn_expired
        jr      nc,fn_getc1
        jp      fn_abort
fn_getc2:
        pop     de
        pop     hl
fn_take:
        in      a,(PORTA)
        ret
```

The ESP32 waits up to a second for each byte to be taken. A program that reads its reply slowly loses the rest of it.

TURNING THE BUS AROUND

Both sides share port A's eight lines, so only one may drive them. The rules are simple:

- DIR is 0 except while the H89 is sending a request.
- The H89 never sends while a reply is coming. If it does, the ESP32 stops sending at once: the firmware sees /OBF fall and cuts its reply short.
- A reply always comes after a request, never before. The FujiNet does not speak unless spoken to.

TIME

The H89's clock interrupt counts 2-millisecond ticks into a 16-bit word in memory. HDOS keeps it at 201BH (040.033); Heath's CP/M BIOS at 000BH. The book's library calls it `TICKS`, and fujinet-lib reads the same word:

CARD.INC — WAIT FOR THE NEXT TICK

```
tick:
        ld      a,(TICKS)
        ld      c,a
tick_wait:
        ld      a,(TICKS)
        cp      c
        jr      z,tick_wait
        ret
```

To time something, a program saves `TICKS` at the start and subtracts. The subtraction works across the moment the count wraps from FFFFH to 0:

FUJIBUS.INC — HAS `FN_TLIM` TICKS PASSED SINCE `FN_T0`?

```
; carry once (TICKS - fn_t0) has reached fn_tlim. Uses HL, DE.
fn_expired:
        ld      hl,(TICKS)
        ld      de,(fn_t0)
        or      a
        sbc     hl,de
        ld      de,(fn_tlim)
        or      a
        sbc     hl,de
        ccf
        ret
```

Both sides of the card give up when the other stops. These are the limits:

WHO WAITS	HOW LONG
Firmware, between request bytes	100 ms; then the request is lost (see above)
Firmware, for the H89 to take a reply byte	1 second; then the rest of the reply is lost
Book library, for a reply to begin	15 seconds (`RX_FIRST`)
Book library, between reply bytes	1 second (`RX_TIMEOUT`)
Book library, for a request byte to be taken	1 second (`TX_TIMEOUT`)
fujinet-lib, for any reply byte	15 seconds
fujinet-lib, for a request byte to be taken	forever

WARNING: fujinet-lib never gives up while sending. If the FujiNet is not running — the card was never started, or the ESP32 has crashed — a C program hangs in its first call, and only the H89's RESET will free it. Check with `card_present` first if that matters to you.

NO INTERRUPTS

The card's interrupt line is not connected (jumper J3 is open), and nothing in the firmware would use it. A program learns that something has happened only by asking. That matters most for the network device: data arriving from the far end of a connection waits in the FujiNet until the program asks how much there is, with the network STATUS command (Chapter 10), and then reads it. A program that talks to the network therefore loops: check the keyboard, ask for STATUS, read what has come, and around again. NETCAT (Chapter 13) is the pattern.

SHARING THE CARD

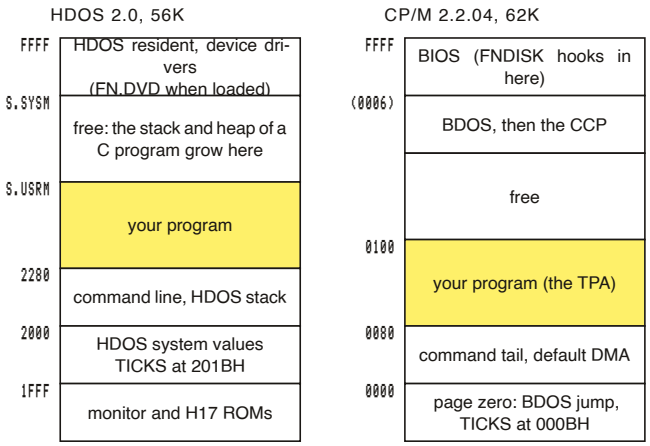
Several programs may use the card in one session: FN.DVD or FNDISK, for the FujiNet drives; CONFIG; and yours. This works because every transaction is complete before the code that made it returns. A disk read by HDOS's FN.DVD in the middle of your program's work starts and finishes between two of your requests, and leaves the card as it found it: running, transceiver on, DIR = 0.

Keep to that, and your programs will get on with the drivers too:

- Start the card with `card_on`, which leaves a running FujiNet alone.
- Leave DIR at 0 and /OE at 0 between requests.
- Never leave a request half sent, or a reply half read. If your program gives up on a reply, take and discard whatever is still coming before the next request. `fn_call` does this for you (it waits until 100 ms pass with nothing arriving).
- Don't write the 82C55A's mode word when port B holds the mark. If you set the card up yourself, leave the mark, as `card_on` does.

The FujiNet's state is shared, too. If your program mounts a different image in drive slot 2, the drive FNDISK calls FN2: (HDOS) or F: (CP/M) changes under it. Programs that change the drives should say so.

MEMORY



HDOS loads a program at 2280H (042.200) and starts it with SP pointing at the command line's arguments, just below. S.USRM and S.SYSM, at 20D2H and 20D0H, give the end of the loaded program and the start of the memory HDOS keeps for itself. CP/M loads a program at 0100H; the word at 0006H gives the start of the BDOS, and the space between is the program's to use.

Chapter 4 shows how the book's library deals with both, and Chapter 6 how z88dk's C does.

THE FUJIBUS PROTOCOL

GENERAL

Every exchange between the H89 and the FujiNet is a **request** and a **reply**. The H89 sends a request naming a device and a command, with any numbers and data the command needs. The device answers ACK, with whatever data it has to give, or NAK, which means no. Then the next request may go.

The format is FujiBus, the FujiNet project's protocol for computers that reach it over a plain byte stream, as the H89 does through port A. It is the same on every such computer; only the wires differ. This chapter describes it a byte at a time, so you can read the byte strips under every command in this Manual, and write a FujiBus program in any language.

DEVICES

A request goes to one **device**, by number. The FujiNet on the H89 has these:

DEVICE	NAME	WHAT IT IS
70H	DEV_FUJINET	The FujiNet itself: WiFi, hosts, drives, app keys, utilities (Chapter 8)
31H–38H	DEV_DISK + n	Disk drive slots 1 to 8 (Chapter 9)
71H–78H	DEV_NETWORK + n	Network channels 1 to 8 (Chapter 10)
45H	DEV_CLOCK	The clock (Chapter 12)
40H	DEV_PRINTER	The printer (Chapter 12)
50H	DEV_SERIAL	The modem (Chapter 12)

Other device numbers appear in the firmware's list, for other computers. On the H89 a request to any of them is answered with a NAK, as for a device that does not exist.

FRAMES

A request or a reply travels as one **frame**: a byte C0H, the packet, and another C0H. The C0H bytes are called END. They mark where a frame starts and stops, so the receiver needs no count to find them.

The packet itself may contain any byte, including C0H. So before it is sent, two bytes are replaced by pairs:

IN THE PACKET	IS SENT AS
C0H (END)	DBH DCH
DBH (ESC)	DBH DDH

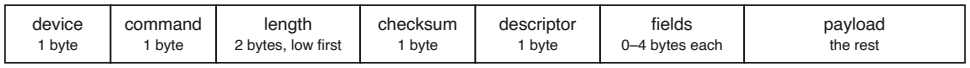
The receiver turns each pair back into one byte. This is SLIP framing, from the early days of the Internet; the names END and ESC are SLIP's. Escapes are commoner than you might think: the command FUJI_CLOSE_APPKEY **is** DBH, so it always goes as DB DD (page 8-14), and any checksum can come out as C0H (the reply on page 8-19 does). Here is how the book's library sends a packet:

FUJIBUS.INC — SEND BC BYTES, ESCAPED

```
; BC bytes from HL, SLIP-escaped
fn_slip:
        ld      a,b
        or      c
        ret     z
        ld      a,(hl)
        cp      SLIP_END
        jr      z,fn_slipe
        cp      SLIP_ESC
        jr      z,fn_slipx
```

```
        call    fn_putc
        jr      fn_slipn
fn_slipe:
        ld      a,SLIP_ESC
        call    fn_putc
        ld      a,SLIP_ESC_END
        call    fn_putc
        jr      fn_slipn
fn_slipx:
        ld      a,SLIP_ESC
        call    fn_putc
        ld      a,SLIP_ESC_ESC
        call    fn_putc
fn_slipn:
        inc     hl
        dec     bc
        jr      fn_slip
```

WARNING: Send exactly one END before the packet. Some SLIP programs send END twice, to flush the line; the FujiNet takes the second END as the end of an empty frame, throws it away, and reads the packet that follows as data for its modem. The request is lost, with no reply.



PICTORIAL 3-1
A FujiBus packet, before it is framed.

THE HEADER

Every packet starts with a six-byte header:

BYTE	NAME	HOLDS
0	device	The device the request is for; in a reply, the device answering.
1	command	The command. In a reply, ACK (06H) or NAK (15H).
2, 3	length	The length of the whole packet, header included, before escaping. Low byte first.
4	checksum	See below.
5	descriptor	What fields follow. See “Fields.”

The length counts bytes as they are in the packet, not as they go down the wire: an escaped C0H counts as one byte. The smallest packet is six bytes, a header alone.

THE CHECKSUM

The checksum is an eight-bit sum of every byte of the packet, with the carry out of each addition added back in at the bottom (an **end-around carry**). The checksum byte itself counts as 0 while the sum is made.

For example, the request for the WiFi status is device 70H, command FAH, length 6, and no fields. With the checksum byte as 0, its bytes are 70 FA 06 00 00 00:

ADD	SUM	CARRY BACK
70H	70H	
FAH	16AH	6AH + 1 = 6BH
06H	71H	
00H, 00H, 00H	71H	

So the checksum is 71H, and the packet is 70 FA 06 00 71 00. The library’s routine is short:

FUJIBUS.INC — ADD BC BYTES INTO THE SUM IN D

```
; D += BC bytes from HL: an 8-bit sum with end-around carry
fn_cksum:
        ld      a,b
        or      c
        ret     z
        ld      a,d
        add     a,(hl)
        adc     a,0
        ld      d,a
        inc     hl
        dec     bc
        jr      fn_cksum
```

The receiver adds up the packet the same way, skipping byte 4, and compares. A packet whose sum is wrong is thrown away.

FIELDS

Most commands need a few small numbers: a slot number, a length, a mode. These go in **fields**, just after the header, and byte 5, the **descriptor**, says how many there are and how big:

DESCRIPTOR	LIBRARY	FIELDS
0	F_NONE	none
1	F_A1	one byte
2	F_A1_A2	two bytes
3	F_A1_A2_A3	three bytes
4	F_A1_A2_A3_A4	four bytes
5	F_B12	one 16-bit word
6	F_B12_B34	two 16-bit words
7	F_C1234	one 32-bit number

Words and longer numbers are sent low byte first, like the length. (The names come from the AUX1 to AUX4 bytes of the Atari's SIO protocol, which FujiBus grew from: B12 is AUX1 and AUX2 taken as a word.)

If a command needs more fields than one descriptor can describe, bit 7 of the descriptor is set and another descriptor byte follows it, before any field. No command on the H89 needs this.

The FujiNet does not care how wide a field is sent: a command that takes a number takes it from a one-byte field, a word or a long alike. Each card in Chapters 8 to 12 shows the size the command uses, which is what this Manual's examples send.

NOTE: The FujiNet looks only at the descriptor's low three bits (and bit 7). fujinet-lib puts two flags of its own in bits 3 and 4 (08H, "payload follows," and 10H, "reply expected"), so its requests carry descriptors like 10H and 12H where the book's library sends 00H and 02H. The FujiNet treats them the same.

WARNING: Always send every field a command takes. A command that finds a field missing does not NAK: the firmware stops. On the card the ESP32 restarts, and every drive and network connection is lost; fujinet-pc exits.

THE PAYLOAD

Whatever follows the fields is the **payload**: a file name, a block of data to write, a structure of settings. Its length is not stated separately; it is what the header's length leaves after the header, the extra descriptors and the fields.

Commands are not strict about payloads. A payload shorter than the command expects is accepted (the rest of the command's buffer is whatever was there before), and a longer one is cut off. Each card says what the command expects.

THE REPLY

For every request it accepts, the device sends one reply:

BYTE	HOLDS
0	The device answering. For a request to a device that does not exist, 00H.
1	ACK (06H): done. NAK (15H): refused, or failed.
2, 3	The length, as in a request.
4	The checksum, as in a request.
5	00H: a reply has no fields.
6 …	The reply's data, if any. Only an ACK carries data.

The reply carries no count of its own other than the length. A program learns how much data came by subtracting six from the length: `fn_call` returns it in BC.

WHEN THERE IS NO REPLY

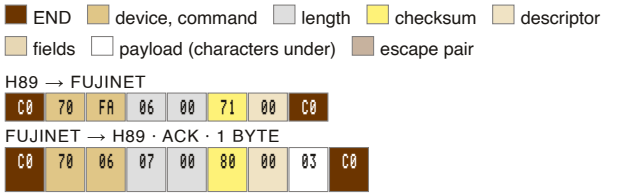
Some requests are never answered:

- A frame with a wrong checksum or length, or a gap of more than 100 ms in the middle (Chapter 2). The FujiNet cannot tell what it was, so it says nothing.
- FUJI_RESET, which restarts the FujiNet instead (Chapter 8).
- Any request to the printer while it is turned off in the FujiNet's settings.
- The modem's commands. The modem answers with bare bytes, not a frame (Chapter 12).

A program must therefore give up on a reply after a while. The book's library waits 15 seconds for the first byte, and 1 second for each after.

A COMPLETE EXCHANGE

Here is the WiFi status request, as it went over the bus, and the FujiNet's reply. The bytes are shaded by their part of the frame:



The request is the packet worked out above, framed in ENDs. In the reply, the FujiNet has answered as device 70H, with ACK, in a packet seven bytes long: the header and one byte of data, 03H, which means the FujiNet is connected to its network.

THE RULES

These hold for every exchange in this Manual:

1. The H89 speaks first. The FujiNet only answers.
2. One request, one reply, and then the next request. Requests do not overlap, even to different devices.
3. Send a request in one go: no pause of 100 ms or more between its bytes.
4. Send exactly one END before the packet and one after.
5. Send every field the command takes.
6. Take the whole reply, to its last END, before the next request, even if you do not want it all.
7. If no reply comes, wait until the line is quiet, then go on. Expect that the request did nothing, or that it did everything; you cannot tell which.

ASSEMBLY LANGUAGE

GENERAL

The H89's processor is a Z80, and Z80 assembly language is the most direct way to drive the FujiNet. This chapter shows how to write a FujiNet program in it, assemble it on a modern computer with z88dk's assembler, carry it to the H89, and run it under HDOS or CP/M — the same source file for both.

It also introduces the book's assembly library: a handful of include files that hide the differences between HDOS and CP/M, start the card, and make a complete FujiBus exchange with one call. Every assembly program in this Manual uses it, and it is listed in full at the end of the chapter.

Why assemble somewhere else? Neither HDOS nor CP/M came with a Z80 assembler: Heath's ASM and Digital Research's ASM know only the 8080's instructions, which the Z80 runs but which cannot use its extra registers and instructions. Chapter 5 shows how to use them anyway. A modern computer also makes editing, and keeping your work, much easier, and the FujiNet makes moving the result to the H89 a matter of seconds.

SETTING UP Z88DK

z88dk is a free development kit for Z80 computers, with a C compiler (Chapter 6), an assembler and linker (z88dk-z80asm), and tools for making disk images. It knows both HDOS and CP/M.

( ) Get z88dk from z88dk.org: a nightly build for Windows or macOS, or the source for Linux. This Manual used v2.4-1231.

( ) Put z88dk's `bin` directory on your `PATH`, and point `ZCCCFG` at its `lib/config` directory. On Linux, for example:

```
export PATH=$HOME/z88dk/bin:$PATH
export ZCCCFG=$HOME/z88dk/lib/config
```

( ) Check it: `z88dk-z80asm` with no arguments prints its options.

( ) Get the book's sources, which include the library in `src/asm/inc`, from the fujinet-manuals repository (see the inside front cover), in `h89/programmers-handbook`.

A FIRST PROGRAM

Here is FNHELLO, the shortest useful FujiNet program. It starts the card, asks the FujiNet for its adapter configuration (Chapter 8), and prints four things from the reply:

SRC/ASM/HELLO/FNHELLO.ASM 85 LINES

```
;-------------------------------------------------------------------------
; FNHELLO -- first contact. Starts the FujiNet card, asks it for its adapter
; configuration, and prints the network it has joined, its name, its address
; and its firmware version. One source, two systems:
;   z88dk-z80asm -b -DHDOS -I=../inc fnhello.asm   ->  FNHELLO.ABS (HDOS)
;   z88dk-z80asm -b -DCPM  -I=../inc fnhello.asm   ->  FNHELLO.COM (CP/M)
;
; Copyright (C) 2026 Thomas Cherryhomes
; SPDX-License-Identifier: GPL-3.0-or-later
;-------------------------------------------------------------------------
INCLUDE "os.inc"
INCLUDE "fujinet.inc"
```

```
13
14 main:
15        ld      hl,t_banner
16        call    print
17        call    card_on               ; start the card (if nothing has)
18        call    card_present
19        jr      nz,no_card
20        ld      hl,rq_config          ; ask for the adapter configuration
21        call    fn_call
22        jr      nz,failed
23        ld      hl,t_ssid
24        call    print
25        ld      hl,cfg_ssid
26        call    print
27        ld      hl,t_host
28        call    print
29        ld      hl,cfg_host
30        call    print
31        ld      hl,t_ip
32        call    print
33        ld      hl,cfg_ip
34        call    pip
35        ld      hl,t_ver
36        call    print
37        ld      hl,cfg_ver
38        call    print
39        jp      crlf
40
41 no_card:
42        ld      hl,t_nocard
43        jp      print
44 failed:
45        ld      hl,t_failed
46        jp      print
47
48 rq_config:
49        FNREQ   DEV_FUJINET, FUJI_GET_ADAPTERCONFIG, F_NONE, 0,0,0,0
50        FNBUF   0, 0, cfg, 140        ; no payload; 140 bytes back to cfg
51
52 ; the reply: 140 bytes
53 cfg:
54 cfg_ssid:      defs    33            ; the WiFi network's name
55 cfg_host:      defs    64            ; the FujiNet's own name
56 cfg_ip:        defs    4             ; its address,
57 cfg_gw:        defs    4             ;   the router's,
58 cfg_mask:      defs    4             ;   the netmask
59 cfg_dns:       defs    4             ;   and the name server
60 cfg_mac:       defs    6
61 cfg_bssid:     defs    6
62 cfg_ver:       defs    15            ; the firmware version
63
64 t_banner:      defm    "FNHELLO - first contact with the FujiNet"
65                defb    13, 10, 0
66 t_ssid:        defm    "Network:  "
67                defb    0
68 t_host:        defb    13, 10
69                defm    "Name:     "
70                defb    0
71 t_ip:          defb    13, 10
72                defm    "Address:  "
73                defb    0
74 t_ver:         defb    13, 10
75                defm    "Firmware: "
76                defb    0
77 t_nocard:      defm    "No FujiNet card at 0D0H."
78                defb    13, 10, 0
79 t_failed:      defm    "The FujiNet did not answer."
80                defb    13, 10, 0
81
82 INCLUDE "card.inc"
83 INCLUDE "fujibus.inc"
84 INCLUDE "con.inc"
85 INCLUDE "tail.inc"
```

Notice its shape, which every program in this Manual shares:

1. `os.inc` comes first. It sets where the program loads, writes the .ABS header for HDOS, and supplies the start-up code, which calls `main`.
2. `fujinet.inc` names the ports, the protocol's codes, and every device and command.
3. `main` is the program. When it returns, the program ends and the console is put back as it was.
4. The request is a block of data, `rq_config`, made with the `FNREQ` and `FNBUF` macros. `fn_call` sends it and takes the reply.
5. The library's code comes last: `card.inc`, `fujibus.inc`, `con.inc`, and `tail.inc`, which must be the very last, because it marks where the program ends.

ASSEMBLING

One command makes the HDOS program, and another the CP/M one. From the `programmers-handbook` directory:

```
z88dk-z80asm -b -DHDOS -Isrc/asm/inc -oFNHELLO.ABS src/asm/hello/fnhello.asm
z88dk-z80asm -b -DCPM  -Isrc/asm/inc -oFNHELLO.COM src/asm/hello/fnhello.asm
```

- `-b` makes a binary file, ready to run, rather than an object file to link.
- `-DHDOS` or `-DCPM` defines the symbol that `os.inc` tests, to choose the system.
- `-I` says where the include files are.
- `-o` names the output.

Add `-l` for a listing, which shows the address and bytes of every line; it is a great help when something goes wrong. The book's `tools/asm.sh hdos FNHELLO src/asm/hello/fnhello.asm` does all of this and puts the program in `build/bin/hdos/asm`.

NOTE: z88dk-z80asm writes numbers the modern way, 0x70 for hexadecimal, but it also accepts 70H and the octal 160Q, so a line copied from a Heath listing assembles as it stands.

CARRYING IT TO THE H89

The program needs to be on a disk the H89 can read. There are two easy ways.

ON A FUJINET DRIVE

Put the program on a disk image, put the image on the FujiNet's microSD card (or a TNFS server you run), and mount it in a FujiNet drive with CONFIG. The book's tools make the images:

```
tools/hdosdisk.sh FNHELLO.H8D "BOOK PROGRAMS" FNHELLO.ABS     (HDOS)
tools/cpmdisk.sh  FNHELLO.H8D FNHELLO.COM                      (CP/M)
```

The HDOS image comes from fujinet-h89's `hdosfs.py`; the CP/M one from z88dk's `z88dk-appmake +cpmdisk -f hz89`, with fujinet-h89's `cpm-dir-fixup.py` to mark the empty directory entries the way Heath's CP/M expects. Both make a 100K, 40-track H17 image.

ON FUJINET GO H89

The emulator takes the same images in its drives, or reads a host directory. Either way, the program runs exactly as it will on the real machine.

RUNNING FNHELLO

( ) Boot HDOS. Mount the disk with FNHELLO on it: here it is in the H17's second drive.

( ) Type MOUNT SY1: ↵, then RUN SY1:FNHELLO.ABS ↵.

```
>MOUNT SY1:
VOLUME 000, MOUNTED ON SY1:
LABEL: BOOK PROGRAMS
>RUN SY1:FNHELLO.ABS
FNHELLO - first contact with the FujiNet
Network:  Workshop
Name:     FujiNet
Address:  192.168.1.89
Firmware: v1.6-7d7a3ba
```

( ) Under CP/M, with the disk in drive B, type B:FNHELLO ↵.

```
A>B:FNHELLO
FNHELLO - first contact with the FujiNet
Network:  Workshop
Name:     FujiNet
Address:  192.168.1.89
Firmware: v1.6-7d7a3ba
```

The network's name, the FujiNet's name and address, and the version of its firmware: the FujiNet is answering. If FNHELLO says "No FujiNet card at 0D0H," the card is missing, or set to other ports; if it says "The FujiNet did not answer," it is there but not talking. The Operation Manual's troubleshooting chart covers both.

THE PROGRAM ON HDOS

For HDOS, `os.inc` begins the program at 2278H with the eight-byte header HDOS expects of an .ABS file, so that the code itself starts at 2280H, where HDOS loads programs:

```
        org     0x2278              ; HDOS loads programs at 2280H:
        defb    0xFF, 0x00          ; the .ABS header, 8 bytes, comes first
        defw    start               ; load address
        defw    prog_end - start    ; length
        defw    start               ; entry point
```

HDOS's services are **system calls**: a `RST 38H` instruction followed by a byte that says which service. The library's `SCALL` macro writes both. It uses these:

CALL	CODE	DOES
.EXIT	00H	Back to HDOS. A = 0.
.SCIN	01H	A key from the console into A; carry set if none is waiting.
.SCOUT	02H	The character in A to the console.
.READ	04H	Read BC bytes (whole sectors) from channel A into DE; carry at the end.
.WRITE	05H	Write BC bytes (whole sectors) from DE to channel A.
.CONSL	06H	Set console bits: A = which byte, B = the bits, C = which bits to change.
.OPENR	22H	Open file HL for reading on channel A; DE = default name parts.
.OPENW	23H	Open file HL for writing on channel A.
.CLOSE	26H	Close channel A.

The Appendix lists the rest. At the start, `os.inc` saves the stack pointer, which HDOS leaves pointing at the command line's arguments, and uses `.CONSL` to put the console in raw character mode: no echo, no CTRL-C handling, no wrapping at 80 columns and no mapping of lower case to upper. That lets a program send the H19 escape sequences and graphics characters untouched. `exit` puts the console back as it found it.

HDOS files are read and written a 256-byte sector at a time; `f_read` and `f_write` move any number of bytes through a sector buffer, and `f_close` pads the last sector with zeros.

THE PROGRAM ON CP/M

For CP/M, `os.inc` begins the program at 0100H, as CP/M requires, and gives it its own 256-byte stack: the one the CCP leaves is only a few words deep. The console goes through BDOS function 6, "direct console I/O," which neither echoes nor stops for CTRL-S. `exit` jumps to 0000H, which reloads the CCP.

CP/M files are read and written a 128-byte record at a time, through a file control block. `f_open` and `f_create` fill one in from a name such as `B:FILE.TXT`; `f_close` pads the last record with CTRL-Z (1AH), CP/M's end-of-text mark.

THE LIBRARY

These are the routines a program can call. Unless it says otherwise, a routine may change A and the flags, and keeps BC, DE and HL.

ROUTINE	DOES
`main`	Yours: the program. Return to end it.
`exit`	End the program now.
`conout`	A to the console, raw.
`conin`	A key into A, with carry clear; carry set if no key is waiting.
`getkey`	Wait for a key; A = the key.
`print`	HL points at NUL-terminated text: print it. HL is changed.
`crlf`	Carriage return and line feed.
`getline`	Read a line into HL (B = room), shown as typed, ended by RETURN, stored with a NUL.
`args`	HL points at the command line's arguments, NUL-terminated.
`f_create`, `f_open`	Open file HL (`SY1:NAME.EXT` or `B:NAME.EXT`) to write or read; carry if it failed.
`f_write`	Write BC bytes from DE.
`f_read`	Read up to BC bytes into DE; BC = how many came, 0 at the end of the file.
`f_close`	Finish and close the file.
`phex8`, `phex16`	Print A, or HL, in hexadecimal.
`pdec8`, `pdec16`	Print A, or HL, in decimal.
`pip`	Print the 4 bytes at HL as an IP address.
`ptext`, `phexs`, `pdump`	Print BC bytes at HL as text, as hex on one line, or as a dump.

strlen	BC = the length of the string at HL.
card_on	Set up the card unless a program already has (Chapter 2); face the bus toward the H89.
card_present	Z if the card is there and running.
fn_call	Make one FujiBus exchange (see below).
fn_send	Send a request and wait for no reply (the modem's, which are not framed).
fn_raw	Take unframed bytes (the modem's) into DE, BC = room, until 200 ms of quiet.

MAKING A REQUEST

fn_call takes HL pointing at a **request block**, sends the request, and takes the whole reply. A request block is two lines, which follow the order of the request's header:

```
rq_name:
        FNREQ   device, command, descriptor, aux1, aux2, aux3, aux4
        FNBUF   payload, payload length, reply buffer, reply size
```

- **descriptor** is one of F_NONE through F_C1234 (Chapter 3), and says how many of aux1 to aux4 are sent. A word goes in aux1 and aux2, low byte first, as F_B12 sends it.
- **payload** and **payload length** give data to send after the fields: 0, 0 for none.
- **reply buffer** and **reply size** say where the reply's data goes and how much room there is: 0, 0 if no data is expected.

fn_call returns:

	ON RETURN
A	FN_OK (0) for ACK, FN_NAK (1), FN_TIMEOUT (2) or FN_BAD (3), a reply that was garbled or from the wrong device.
Z	Set for ACK, so jr nz,failed follows a call.
BC	How many data bytes the reply carried, even if that was more than the buffer.
Carry	Set if the reply was longer than the buffer. The rest was read and dropped.

fn_res and fn_len keep A and BC, and fn_rep the reply's six-byte header. Other registers are changed.

A program that needs a different field or length each time stores it into the block before the call. The offsets are named RQ_DEV through RQ_RMAX in fujinet.inc. NETCAT does this for every read, with the length the FujiNet says is waiting:

NETCAT.ASM — THE LENGTH GOES INTO THE REQUEST BLOCK

```
loop:
        call    conin                   ; a key struck?
        jr      c,poll
        cp      QUIT_KEY
        jr      z,hangup
        ld      (outch),a               ; send it: one byte
        ld      hl,rq_write
        call    fn_call
poll:
        ld      hl,rq_status            ; anything come in?
        call    fn_call
        jr      nz,loop
        ld      hl,(nstat+NS_AVAIL)
        ld      a,h
        or      l
        jr      nz,fetch
        ld      a,(nstat+NS_CONN)       ; nothing: still connected?
        or      a
        jr      nz,loop
        jr      hungup                  ; the far end has gone
fetch:
        ld      de,BUFSZ                ; take what waits, a buffer at most
        or      a
        sbc     hl,de
        add     hl,de
        jr      c,fetch1
        ex      de,hl
fetch1:
        ld      (rq_read+RQ_AUX1),hl    ; the read's length field
        ld      hl,rq_read
        call    fn_call
        jr      nz,loop
        ld      hl,rxbuf                ; BC bytes came: type them
show:
        ld      a,b
        or      c
        jr      z,loop
        ld      a,(hl)
        call    conout
        inc     hl
        dec     bc
        jr      show
```

If no reply comes, fn_call gives up (Chapter 2's table), and the next call first takes and throws away anything that comes late. The FujiNet is never left half way through an exchange.

DEBUGGING

When a FujiNet program does not work, the first question is: what went over the bus? On FujiNet Go H89 the answer is easy, because the emulator can log every byte. On a real H89 the FujiNet's own log, over its USB port at 115,200 baud (Chapter 9 of the Operation Manual), shows every request the firmware received and what it did.

For the program itself, HDOS has DBUG and CP/M has DDT. Both are 8080 debuggers: they will step through Z80 code, but show the Z80's extra instructions as data. A listing from `-l` tells you where to set breakpoints.

CONDITION	POSSIBLE CAUSE
The program hangs at its first request.	The card was never started (call `card_on`), or it is set to other ports.
Every request times out.	A request is malformed — usually the length — so the FujiNet says nothing (Chapter 3).
The FujiNet restarts (its drives disappear).	A request was missing a field the command needs.
Replies are shifted, or from the wrong command.	A reply was not read to its end. Use `fn_call`, which always reads to the end.
Text comes out in capitals under HDOS.	The console is mapping lower case to upper. `os.inc` turns this off; a program that does its own start-up must too.
Under CP/M, a program's first keys are lost.	Heath's BIOS keeps no type-ahead while a program is busy. Poll the keyboard more often.

LISTING OF THE LIBRARY

SRC/ASM/INC/OS.INC 566 LINES

```
;-------------------------------------------------------------------------
; os.inc -- one program, two operating systems.
;
; Include this FIRST. Assemble with -DHDOS for an HDOS 2.0 .ABS file or
; -DCPM for a Heath CP/M 2.2.04 .COM file. Your program supplies 'main'
; (it may return, or call exit) and ends by including tail.inc.
;
; Both systems get the same routines:
;   conout   A = character out. Keeps BC, DE, HL.
;   conin    A = a key, NC; carry if no key is waiting. Keeps BC, DE, HL.
;   getkey   waits for a key; A = it.
;   print    HL -> NUL-terminated text out.
;   crlf     carriage return, line feed.
;   exit     restore the console and return to the system.
;   args     HL -> the command line's arguments, NUL-terminated.
;   f_create HL -> "SY1:NAME.EXT" / "B:NAME.EXT". Carry = failed.
;   f_open   the same, for reading.
;   f_write  DE -> data, BC = length (any). Carry = failed.
;   f_read   DE -> buffer, BC = size; BC = bytes read, 0 at the end.
;   f_close  write what is buffered, close. Carry = failed.
;   TICKS    the word the system's 2 ms clock counts up.
;
; Copyright (C) 2026 Thomas Cherryhomes
; SPDX-License-Identifier: GPL-3.0-or-later
;-------------------------------------------------------------------------

IFDEF HDOS
;------------------------------------------------------------ HDOS 2.0 ----
TICKS           equ     0x201B          ; HDOS's tick counter (S.TIME area)
S_CSLMD         equ     0x20D6          ; the five console mode bytes
HDOS_EOS        equ     0x227F          ; the command line ends here (a NUL)

; system calls: RST 38H followed by the function number
macro SCALL fn
        rst     0x38
        defb    fn
endm
_EXIT           equ     0x00
_SCIN           equ     0x01
_SCOUT          equ     0x02
_READ           equ     0x04
_WRITE          equ     0x05
_CONSL          equ     0x06
_OPENR          equ     0x22
_OPENW          equ     0x23
_CLOSE          equ     0x26

        org     0x2278                  ; HDOS loads programs at 2280H:
        defb    0xFF, 0x00              ; the .ABS header, 8 bytes, comes first
        defw    start                   ; load address
        defw    prog_end - start        ; length
        defw    start                   ; entry point
start:
        ld      (os_argp),sp            ; HDOS leaves SP on the command line
        call    os_init
        call    main
        jp      exit

; raw character mode, no echo, no case mapping, no width: escape sequences
; and graphics characters pass untouched
os_init:
        ld      hl,S_CSLMD              ; keep the console as we found it
```

```
63          ld      de,os_consave
64          ld      bc,5
65          ldir
66          ld      a,0                     ; S.CSLMD: echo off, raw, character mode
67          ld      b,0x85
68          ld      c,0x87
69          SCALL   _CONSL
70          ld      a,1                     ; S.CONTY: no case mapping
71          ld      b,0
72          ld      c,0x32
73          SCALL   _CONSL
74          ld      a,3                     ; S.CONWI: width 255
75          ld      b,255
76          ld      c,0xFF
77          SCALL   _CONSL
78          ret
79
80  exit:
81          ld      a,(os_consave)          ; put the console back as it was
82          ld      b,a
83          xor     a
84          ld      c,0xFF
85          SCALL   _CONSL
86          ld      a,(os_consave+1)
87          ld      b,a
88          ld      a,1
89          ld      c,0xFF
90          SCALL   _CONSL
91          ld      a,(os_consave+3)
92          ld      b,a
93          ld      a,3
94          ld      c,0xFF
95          SCALL   _CONSL
96          xor     a
97          SCALL   _EXIT
98
99  conout:
100         push    bc
101         push    de
102         push    hl
103         SCALL   _SCOUT
104         pop     hl
105         pop     de
106         pop     bc
107         ret
108
109 conin:
110         push    bc
111         push    de
112         push    hl
113         SCALL   _SCIN                   ; carry: nothing waiting
114         pop     hl
115         pop     de
116         pop     bc
117         ret
118
119 ; HL -> the arguments: from where SP pointed at entry to 227FH
120 args:
121         ld      hl,(os_argp)
122 args_skip:
123         ld      a,h                     ; SP was 2280H: no arguments
124         cp      0x22
125         jr      nz,args_sp
126         ld      a,l
127         cp      0x80
128         jr      nz,args_sp
129         ld      hl,os_nul               ; none: an empty string (227FH
130         ret                             ;   holds a NUL only if there were)
131 args_sp:
132         ld      a,(hl)
133         cp      ' '
134         ret     nz
135         inc     hl
136         jr      args_sp
137
138 ; files: channel 0, read or written through a 256-byte sector buffer
139 f_create:
140         ld      a,_OPENW
141         jr      f_openx
142 f_open:
143         ld      a,_OPENR
144 f_openx:
145         ld      (f_scall+1),a           ; the SCALL number, patched in
146         xor     a
147         ld      (os_fcnt),a
148         ld      (os_fcnt+1),a
149         ld      (os_feof),a
150         ld      de,os_hdefault
151 f_scall:
152         SCALL   0                       ; OPENW or OPENR, A = channel 0
153         ret
154
155 f_write:
156         ld      a,b
157         or      c
158         ret     z
159         ld      a,(de)
160         inc     de
161         push    de
162         push    bc
163         ld      hl,(os_fcnt)
164         push    hl
165         ld      de,os_fbuf
166         add     hl,de
167         ld      (hl),a
168         pop     hl
169         inc     hl
170         ld      (os_fcnt),hl
171         ld      a,h                     ; a whole sector: write it
172         cp      1
173         jr      nz,f_write1
174         call    f_wsector
175         jr      c,f_wfail
176 f_write1:
177         pop     bc
178         pop     de
179         dec     bc
180         jr      f_write
181 f_wfail:
182         pop     bc
183         pop     de
184         scf
185         ret
186 f_wsector:
187         xor     a                       ; channel 0
188         ld      de,os_fbuf
189         ld      bc,256
190         SCALL   _WRITE
191         ld      hl,0
192         ld      (os_fcnt),hl
193         ret
194
195 f_close:
196         ld      hl,(os_fcnt)            ; pad the last sector with zeros
197         ld      a,h
198         or      l
199         jr      z,f_close1
200 f_pad:
201         ld      de,os_fbuf
202         push    hl
203         add     hl,de
204         ld      (hl),0
205         pop     hl
206         inc     hl
207         ld      a,h
208         cp      1
209         jr      nz,f_pad
210         call    f_wsector
211         ret     c
212 f_close1:
213         xor     a
214         SCALL   _CLOSE
215         ret
216
217 f_read:
218         push    bc                      ; BC = size wanted
219         ld      hl,0
220         ld      (os_rcount),hl
221 f_read1:
222         pop     bc
223         ld      a,b
224         or      c
225         jr      z,f_readx
226         dec     bc
227         push    bc
228         ld      hl,(os_fcnt)            ; buffer empty? read a sector
229         ld      a,h
230         or      l
231         jr      nz,f_read2
232         ld      a,(os_feof)
233         or      a
234         jr      nz,f_reof
235         push    de
236         xor     a
237         ld      de,os_fbuf
238         ld      bc,256
239         SCALL   _READ                   ; carry: the end of the file
240         pop     de
241         jr      c,f_rend
242         ld      hl,256
243         ld      (os_fcnt),hl
244         ld      hl,os_fbuf
245         ld      (os_fptr),hl
246 f_read2:
247         ld      hl,(os_fptr)
248         ld      a,(hl)
```

```
249         inc     hl
250         ld      (os_fptr),hl
251         ld      (de),a
252         inc     de
253         ld      hl,(os_fcnt)
254         dec     hl
255         ld      (os_fcnt),hl
256         ld      hl,(os_rcount)
257         inc     hl
258         ld      (os_rcount),hl
259         jr      f_read1
260 f_rend:
261         ld      a,1
262         ld      (os_feof),a
263 f_reof:
264         pop     bc
265 f_readx:
266         ld      bc,(os_rcount)
267         ret
268
269 ELSE
270 ;------------------------------------------------ Heath CP/M 2.2.04 ----
271 TICKS           equ     0x000B          ; the Heath BIOS's tick counter
272 BDOS            equ     0x0005
273 CPM_TAIL        equ     0x0080          ; the command tail: length, text
274
275         org     0x0100
276 start:
277         ld      sp,os_stack             ; our own stack (the CCP's is tiny)
278         call    os_init
279         call    main
280         jp      exit
281
282 os_init:
283         ret
284
285 exit:
286         jp      0                       ; warm boot back to the CCP
287
288 ; console through BDOS function 6, direct: no echo, no ^S or ^P
289 conout:
290         push    af
291         push    bc
292         push    de
293         push    hl
294         ld      e,a
295         ld      c,6
296         call    BDOS
297         pop     hl
298         pop     de
299         pop     bc
300         pop     af
301         ret
302
303 conin:
304         push    bc
305         push    de
306         push    hl
307         ld      e,0xFF
308         ld      c,6
309         call    BDOS                    ; A = 0: nothing waiting
310         pop     hl
311         pop     de
312         pop     bc
313         or      a
314         scf
315         ret     z
316         ccf
317         ret
318
319 ; HL -> the arguments: the CCP's command tail at 80H, made NUL-terminated
320 args:
321         ld      hl,CPM_TAIL
322         ld      e,(hl)
323         ld      d,0
324         inc     hl
325         push    hl
326         add     hl,de
327         ld      (hl),0
328         pop     hl
329         jr      args_sp
330 args_sp:
331         ld      a,(hl)
332         cp      ' '
333         ret     nz
334         inc     hl
335         jr      args_sp
336
337 ; files: one FCB, read or written through a 128-byte record buffer
338 f_create:
339         call    f_fcb
340         ld      de,os_fcb
341         ld      c,19                    ; delete any old one
342         call    BDOS
343         ld      de,os_fcb
344         ld      c,22                    ; make
345         jr      f_openx
346 f_open:
347         call    f_fcb
348         ld      de,os_fcb
349         ld      c,15                    ; open
350 f_openx:
351         call    BDOS
352         inc     a                       ; FFH: failed
353         scf
354         ret     z
355         xor     a
356         ld      (os_fcnt),a
357         ld      (os_fcnt+1),a
358         ld      (os_feof),a
359         ret
360
361 ; HL -> "D:NAME.EXT" into os_fcb
362 f_fcb:
363         push    hl
364         ld      hl,os_fcb               ; clear it: drive 0, blank name
365         ld      (hl),0
366         inc     hl
367         ld      b,11
368 f_fcbsp:
369         ld      (hl),' '
370         inc     hl
371         djnz    f_fcbsp
372         ld      b,24
373 f_fcbz:
374         ld      (hl),0
375         inc     hl
376         djnz    f_fcbz
377         pop     hl
378         inc     hl                      ; "D:" names a drive
379         ld      a,(hl)
380         dec     hl
381         cp      ':'
382         jr      nz,f_fcbn
383         ld      a,(hl)
384         and     0x5F
385         sub     'A' - 1
386         ld      (os_fcb),a
387         inc     hl
388         inc     hl
389 f_fcbn:
390         ld      de,os_fcb + 1
391         ld      b,8
392         call    f_fcbpart
393         ld      a,(hl)
394         cp      '.'
395         jr      nz,f_fcbx
396         inc     hl
397         ld      de,os_fcb + 9
398         ld      b,3
399         call    f_fcbpart
400 f_fcbx:
401         ret
402 f_fcbpart:
403         ld      a,(hl)
404         or      a
405         ret     z
406         cp      '.'
407         ret     z
408         cp      ' '
409         ret     z
410         cp      'a'
411         jr      c,f_fcbup
412         and     0x5F
413 f_fcbup:
414         ld      (de),a
415         inc     de
416         inc     hl
417         djnz    f_fcbpart
418         ret
419
420 f_write:
421         ld      a,b
422         or      c
423         ret     z
424         ld      a,(de)
425         inc     de
426         push    de
427         push    bc
428         ld      hl,(os_fcnt)
429         push    hl
430         ld      de,os_fbuf
431         add     hl,de
432         ld      (hl),a
433         pop     hl
434         inc     hl
```

```
435         ld      (os_fcnt),hl
436         ld      a,1                     ; a whole record: write it
437         cp      128
438         jr      nz,f_write1
439         call    f_wrecord
440         jr      c,f_wfail
441 f_write1:
442         pop     bc
443         pop     de
444         dec     bc
445         jr      f_write
446 f_wfail:
447         pop     bc
448         pop     de
449         scf
450         ret
451 f_wrecord:
452         ld      de,os_fbuf
453         ld      c,26                    ; set DMA
454         call    BDOS
455         ld      de,os_fcb
456         ld      c,21                    ; write sequential
457         call    BDOS
458         ld      hl,0
459         ld      (os_fcnt),hl
460         or      a
461         ret     z
462         scf
463         ret
464
465 f_close:
466         ld      hl,(os_fcnt)            ; pad the last record with ^Z
467         ld      a,1
468         or      a
469         jr      z,f_close1
470 f_pad:
471         ld      de,os_fbuf
472         push    hl
473         add     hl,de
474         ld      (hl),0x1A
475         pop     hl
476         inc     hl
477         ld      a,1
478         cp      128
479         jr      nz,f_pad
480         call    f_wrecord
481         ret     c
482 f_close1:
483         ld      de,os_fcb
484         ld      c,16                    ; close
485         call    BDOS
486         inc     a
487         scf
488         ret     z
489         or      a
490         ret
491
492 f_read:
493         push    bc
494         ld      hl,0
495         ld      (os_rcount),hl
496 f_read1:
497         pop     bc
498         ld      a,b
499         or      c
500         jr      z,f_readx
501         dec     bc
502         push    bc
503         ld      hl,(os_fcnt)            ; buffer empty? read a record
504         ld      a,h
505         or      l
506         jr      nz,f_read2
507         ld      a,(os_feof)
508         or      a
509         jr      nz,f_reof
510         push    de
511         ld      de,os_fbuf
512         ld      c,26
513         call    BDOS
514         ld      de,os_fcb
515         ld      c,20                    ; read sequential: not 0 = the end
516         call    BDOS
517         pop     de
518         or      a
519         jr      nz,f_rend
520         ld      hl,128
521         ld      (os_fcnt),hl
522         ld      hl,os_fbuf
523         ld      (os_fptr),hl
524 f_read2:
525         ld      hl,(os_fptr)
526         ld      a,(hl)
527         inc     hl
528         ld      (os_fptr),hl
529         ld      (de),a
530         inc     de
531         ld      hl,(os_fcnt)
532         dec     hl
533         ld      (os_fcnt),hl
534         ld      hl,(os_rcount)
535         inc     hl
536         ld      (os_rcount),hl
537         jr      f_read1
538 f_rend:
539         ld      a,1
540         ld      (os_feof),a
541 f_reof:
542         pop     bc
543 f_readx:
544         ld      bc,(os_rcount)
545         ret
546 ENDIF
547
548 ;------------------------------------------------------------ both systems ----
549 getkey:
550         call    conin
551         jr      c,getkey
552         ret
553
554 print:
555         ld      a,(hl)
556         or      a
557         ret     z
558         call    conout
559         inc     hl
560         jr      print
561
562 crlf:
563         ld      a,13
564         call    conout
565         ld      a,10
566         jp      conout
```

SRC/ASM/INC/FUJINET.INC — 93 LINES

```
1 ;-----------------------------------------------------------------------------
2 ; fujinet.inc -- the H89 FujiNet card and the FujiBus protocol, as names.
3 ;
4 ; The card is an 82C55A at 0D0H (320Q): port A carries the bytes, port C
5 ; the handshake and the lines that start the ESP32, and 0D3H is the
6 ; control register. fncmd.inc (made from the firmware) names every device
7 ; and command.
8 ;
9 ; Copyright (C) 2026 Thomas Cherryhomes
10 ; SPDX-License-Identifier: GPL-3.0-or-later
11 ;-----------------------------------------------------------------------------
12
13 PORTA           equ     0xD0    ; 82C55A port A: the byte pipe (mode 2)
14 PORTB           equ     0xD1    ; port B: wired to nothing; holds SETUP_MARK
15 PORTC           equ     0xD2    ; port C: status in, control out
16 PCTRL           equ     0xD3    ; control word / port C bit set-reset
17
18 ; port C, read
19 PC_OBF          equ     0x80    ; PC7 /OBF: 1 = the byte we wrote was taken
20 PC_ACK          equ     0x40    ; PC6 /ACK (driven by the ESP32)
21 PC_IBF          equ     0x20    ; PC5 IBF: 1 = a byte from the ESP32 waits
22 PC_STB          equ     0x10    ; PC4 /STB (driven by the ESP32)
23 PC_EN           equ     0x04    ; PC2: ESP32 enable (1 = running)
24 PC_OE           equ     0x02    ; PC1: 74LVC245 /OE (0 = on); also ESP32 IO0
25 PC_DIR          equ     0x01    ; PC0: 74LVC245 direction (1 = toward ESP32)
26
27 ; control register values
28 MODE2           equ     0xC0    ; port A mode 2; PC0-PC2 outputs, all 0
29 DIR_TO_H89      equ     0x00    ; bit set/reset: PC0 = 0
30 DIR_TO_ESP      equ     0x01    ; PC0 = 1
31 OE_ON           equ     0x02    ; PC1 = 0: the 245 enabled
32
33 ; Left in port B by whatever sets up the card (card_on, fujinet-lib, FNON,
34 ; FN.DVD, FNDISK). A reset makes port B an input that reads FF, and any
35 ; other mode word clears it or makes it an input.
36 SETUP_MARK      equ     0xA5
37
38 ; SLIP framing
39 SLIP_END        equ     0xC0
40 SLIP_ESC        equ     0xDB
41 SLIP_ESC_END    equ     0xDC
42 SLIP_ESC_ESC    equ     0xDD
43
44 ; FujiBus field descriptors: what follows the 6-byte header
45 F_NONE          equ     0       ; no fields
46 F_A1            equ     1       ; one byte
47 F_A1_A2         equ     2       ; two bytes
48 F_A1_A2_A3      equ     3       ; three bytes
49 F_A1_A2_A3_A4   equ     4       ; four bytes
50 F_B12           equ     5       ; one 16-bit word
```

```
F_B12_B34       equ     6       ; two 16-bit words
F_C1234         equ     7       ; one 32-bit long

; replies
ACK             equ     0x06
NAK             equ     0x15

; fn_call results (in A)
FN_OK           equ     0       ; ACK
FN_NAK          equ     1       ; NAK
FN_TIMEOUT      equ     2       ; no answer in time
FN_BAD          equ     3       ; bad length, checksum or device

; the network units' STATUS reply
NS_AVAIL        equ     0       ; word: bytes waiting
NS_CONN         equ     2       ; byte: 1 = connected
NS_ERR          equ     3       ; byte: last error (1 = OK, 136 = EOF)

INCLUDE "fncmd.inc"

; A request block, read by fn_call, is two lines: what to send, then where
; the payload is and where the reply goes.
;   FNREQ  device, command, descriptor, aux1, aux2, aux3, aux4
;   FNBUF  payload, payload length, reply buffer, reply buffer size
;
; A program changes a block in place before a call (a length, a buffer):
RQ_DEV          equ     0       ; device
RQ_CMD          equ     1       ; command
RQ_DESC         equ     2       ; field descriptor
RQ_AUX1         equ     3       ; aux1-aux4: the fields (a word at aux1-aux2,
RQ_AUX2         equ     4       ;   a long at aux1-aux4, low byte first)
RQ_AUX3         equ     5
RQ_AUX4         equ     6
RQ_PAY          equ     7       ; payload address
RQ_PLEN         equ     9       ; payload length
RQ_REP          equ     11      ; reply buffer
RQ_RMAX         equ     13      ; reply buffer size
macro FNREQ dev, cmd, desc, a1, a2, a3, a4
        defb    dev, cmd, desc, a1, a2, a3, a4
endm
macro FNBUF pay, plen, rep, rmax
        defw    pay, plen, rep, rmax
endm
```

SRC/ASM/INC/FUJIBUS.INC 501 LINES

```
;-----------------------------------------------------------------------------
; fujibus.inc -- FujiBus transactions over the H89 card's 82C55A.
;
; fn_call   HL -> a request block (FNREQ in fujinet.inc):
;             +0 device   +1 command   +2 field descriptor   +3..+6 aux1-aux4
;             +7 payload address   +9 payload length
;             +11 reply buffer     +13 reply buffer size
;           Sends the request in a SLIP frame, then takes the whole reply,
;           to its closing END, so the next request starts clean.
;   Exit:   A  = FN_OK (ACK), FN_NAK, FN_TIMEOUT or FN_BAD; Z set on FN_OK
;           BC = the reply's payload length (all of it, even past the buffer)
;           carry set if the payload was longer than the buffer (the rest
;           was dropped); fn_rep = the reply's 6-byte header;
;           fn_res and fn_len keep A and BC for later
;           All other registers are changed.
;
; fn_raw    DE -> buffer, BC = its size. Takes whatever bytes come until
;           200 ms pass with none (the modem's replies are not framed).
;   Exit:   BC = bytes taken.
;
; fn_send   HL -> a request block: sends it as fn_call does, but does not
;           wait for a reply. For the modem (50H), which answers in bare
;           bytes or not at all: take what comes with fn_raw.
;   Exit:   A = FN_OK, Z set; FN_TIMEOUT if the ESP32 did not take it.
;
; Forked from fujinet-h89's common/fujibus.inc: a request block instead of
; variables, replies of any length, NAK told apart from a bad frame.
;
; Copyright (C) 2026 Thomas Cherryhomes
; SPDX-License-Identifier: GPL-3.0-or-later
;-----------------------------------------------------------------------------

TX_TIMEOUT      equ     500             ; 1 s for the ESP32 to take a byte
RX_FIRST        equ     7500            ; 15 s for the reply to begin
RX_TIMEOUT      equ     500             ; 1 s between reply bytes
RAW_FIRST       equ     1000            ; fn_raw: 2 s for the first byte
RAW_QUIET       equ     100             ; fn_raw: 200 ms of quiet ends it

fn_send:
        ld      a,1                     ; send only: no reply to wait for
        jr      fn_call0
fn_call:
        xor     a
fn_call0:
        ld      (fn_only),a
        ld      (fn_sp),sp              ; a timeout returns from here
        ld      (fn_rq),hl
        ld      a,(fn_dirty)            ; a call given up before: drain it
        or      a
        call    nz,fn_drain

        ; the header: device, command, length, checksum 0, descriptor, aux
        ld      a,(hl)
        ld      (fn_out),a              ; device
        inc     hl
        ld      a,(hl)
        ld      (fn_out+1),a            ; command
        inc     hl
        ld      a,(hl)
        and     7
        ld      (fn_out+5),a            ; descriptor
        ld      e,a                     ; how many aux bytes it carries
        ld      d,0
        push    hl
        ld      hl,fn_auxn
        add     hl,de
        ld      a,(hl)
        pop     hl
        ld      (fn_nout),a
        inc     hl
        ld      de,fn_out+6             ; aux1-aux4 (only n of them go)
        ld      bc,4
        ldir
        ld      e,(hl)                  ; payload address and length
        inc     hl
        ld      d,(hl)
        inc     hl
        ld      (fn_pay),de
        ld      e,(hl)
        inc     hl
        ld      d,(hl)
        inc     hl
        ld      (fn_plen),de
        ld      e,(hl)                  ; reply buffer and its size
        inc     hl
        ld      d,(hl)
        inc     hl
        ld      (fn_rbuf),de
        ld      e,(hl)
        inc     hl
        ld      d,(hl)
        ld      (fn_rmax),de

        ld      a,(fn_nout)             ; length = 6 + aux + payload
        add     a,6
        ld      (fn_nout),a
        ld      l,a
        ld      h,0
        ld      de,(fn_plen)
        add     hl,de
        ld      (fn_out+2),hl
        xor     a                       ; checksum over header, aux, payload
        ld      (fn_out+4),a
        ld      d,a
        ld      hl,fn_out
        ld      a,(fn_nout)
        ld      c,a
        ld      b,0
        call    fn_cksum
        ld      hl,(fn_pay)
        ld      bc,(fn_plen)
        call    fn_cksum
        ld      a,d
        ld      (fn_out+4),a

        ; send: one END, the packet, one END
        ld      a,OE_ON
        out     (PCTRL),a
        ld      a,DIR_TO_ESP
        out     (PCTRL),a
        ld      hl,TX_TIMEOUT
        ld      (fn_tlim),hl
        ld      a,SLIP_END
        call    fn_putc
        ld      hl,fn_out
        ld      a,(fn_nout)
        ld      c,a
        ld      b,0
        call    fn_slip
        ld      hl,(fn_pay)
        ld      bc,(fn_plen)
        call    fn_slip
        ld      a,SLIP_END
        call    fn_putc
        call    fn_txwait
        ld      a,DIR_TO_H89
        out     (PCTRL),a
        ld      a,(fn_only)             ; fn_send: that is all
        or      a
```

```
        jr      z,fn_recv
        ld      bc,0
        xor     a                       ; A = FN_OK, Z set, carry clear
        jp      fn_done

        ; receive: wait for an END, skip any more, decode up to the next
fn_recv:
        ld      hl,RX_FIRST
        ld      (fn_tlim),hl
fn_sync:
        call    fn_getc
        cp      SLIP_END
        jr      nz,fn_sync
        ld      hl,RX_TIMEOUT
        ld      (fn_tlim),hl
fn_skip:
        call    fn_getc
        cp      SLIP_END
        jr      z,fn_skip
        ld      hl,0                    ; the reply's first byte is in A
        ld      (fn_idx),hl
        ld      l,a
        xor     a
        ld      (fn_cks),a
        ld      a,l
fn_rx:
        cp      SLIP_END
        jr      z,fn_rxend
        cp      SLIP_ESC
        jr      nz,fn_rxput
        call    fn_getc
        cp      SLIP_ESC_END
        jr      nz,fn_rxesc
        ld      a,SLIP_END
        jr      fn_rxput
fn_rxesc:
        cp      SLIP_ESC_ESC
        jr      nz,fn_rxput
        ld      a,SLIP_ESC
fn_rxput:
        call    fn_store
        call    fn_getc
        jr      fn_rx

fn_rxend:
        ld      hl,(fn_idx)             ; at least a header,
        ld      de,6
        or      a
        sbc     hl,de
        jr      c,fn_bad
        ld      hl,(fn_idx)             ; as long as its header says,
        ld      de,(fn_rep+2)
        or      a
        sbc     hl,de
        jr      nz,fn_bad
        ld      a,(fn_cks)              ; with its checksum right
        ld      hl,fn_rep+4
        cp      (hl)
        jr      nz,fn_bad
        ld      a,(fn_rep+1)
        cp      NAK
        jr      z,fn_nak
        cp      ACK                     ; an ACK, from the device asked
        jr      nz,fn_bad
        ld      a,(fn_rep)
        ld      hl,fn_out
        cp      (hl)
        jr      nz,fn_bad
        call    fn_rlen
        xor     a                       ; A = FN_OK, Z set, carry clear
        ld      a,(fn_trunc)
        rra                             ; carry = truncated (Z kept)
        ld      a,FN_OK
        jr      fn_done
fn_nak:
        call    fn_rlen
        ld      a,FN_NAK
        or      a                       ; NZ, carry clear
        ld      a,(fn_trunc)
        rra
        ld      a,FN_NAK
        jr      fn_done
fn_bad:
        ld      bc,0
        ld      a,FN_BAD
        or      a
; every way out: keep the result for later (fn_res, fn_len); flags kept
fn_done:
        ld      (fn_res),a
        ld      (fn_len),bc
        ret

; BC = payload bytes received; fn_trunc = 1 if more than the buffer held
fn_rlen:
        ld      hl,(fn_idx)
        ld      de,6
        or      a
        sbc     hl,de
        ld      b,h
        ld      c,l
        ld      de,(fn_rmax)
        ex      de,hl                   ; HL = size, DE = received
        or      a
        sbc     hl,de                   ; size - received: borrow if short
        ld      a,0
        rla
        ld      (fn_trunc),a
        ret

; A reply byte: the first 6 to fn_rep, then into the reply buffer while it
; has room; every byte but the checksum's own goes into fn_cks
fn_store:
        ld      c,a
        ld      hl,(fn_idx)
        ld      a,h                     ; byte 4 is the checksum itself
        or      a
        jr      nz,fn_sum
        ld      a,l
        cp      4
        jr      z,fn_nosum
fn_sum:
        ld      a,(fn_cks)
        add     a,c
        adc     a,0
        ld      (fn_cks),a
fn_nosum:
        ld      hl,(fn_idx)
        inc     hl
        ld      (fn_idx),hl
        dec     hl
        ld      de,6
        or      a
        sbc     hl,de
        jr      nc,fn_spay
        add     hl,de                   ; header byte
        ld      de,fn_rep
        add     hl,de
        ld      (hl),c
        ret
fn_spay:
        ld      de,(fn_rmax)            ; payload byte, if there is room
        ld      a,l
        sub     e
        ld      a,h
        sbc     a,d
        ret     nc
        ld      de,(fn_rbuf)
        add     hl,de
        ld      (hl),c
        ret

; a timeout anywhere: give the call up (its reply may still be coming)
fn_abort:
        ld      a,1
        ld      (fn_dirty),a
        ld      a,DIR_TO_H89
        out     (PCTRL),a
        ld      sp,(fn_sp)
        ld      bc,0
        ld      a,FN_TIMEOUT
        or      a
        jr      fn_done

; D += BC bytes from HL: an 8-bit sum with end-around carry
fn_cksum:
        ld      a,b
        or      c
        ret     z
        ld      a,d
        add     a,(hl)
        adc     a,0
        ld      d,a
        inc     hl
        dec     bc
        jr      fn_cksum

; BC bytes from HL, SLIP-escaped
fn_slip:
        ld      a,b
        or      c
        ret     z
        ld      a,(hl)
        cp      SLIP_END
        jr      z,fn_slipe
        cp      SLIP_ESC
        jr      z,fn_slipx
```

```
        call    fn_putc
        jr      fn_slipn
fn_slipe:
        ld      a,SLIP_ESC
        call    fn_putc
        ld      a,SLIP_ESC_END
        call    fn_putc
        jr      fn_slipn
fn_slipx:
        ld      a,SLIP_ESC
        call    fn_putc
        ld      a,SLIP_ESC_ESC
        call    fn_putc
fn_slipn:
        inc     hl
        dec     bc
        jr      fn_slip

; carry once (TICKS - fn_t0) has reached fn_tlim. Uses HL, DE.
fn_expired:
        ld      hl,(TICKS)
        ld      de,(fn_t0)
        or      a
        sbc     hl,de
        ld      de,(fn_tlim)
        or      a
        sbc     hl,de
        ccf
        ret

; wait until the ESP32 has taken the last byte (/OBF high). Keeps BC DE HL.
fn_txwait:
        in      a,(PORTC)
        and     PC_OBF
        ret     nz
        push    hl
        push    de
        ld      hl,(TICKS)
        ld      (fn_t0),hl
fn_txw1:
        in      a,(PORTC)
        and     PC_OBF
        jr      nz,fn_txw2
        call    fn_expired
        jr      nc,fn_txw1
        jp      fn_abort
fn_txw2:
        pop     de
        pop     hl
        ret

; send A. Keeps BC, DE, HL.
fn_putc:
        push    af
        call    fn_txwait
        pop     af
        out     (PORTA),a
        ret

; the next byte from the ESP32 into A. Keeps BC, DE, HL.
fn_getc:
        in      a,(PORTC)
        and     PC_IBF
        jr      nz,fn_take
        push    hl
        push    de
        ld      hl,(TICKS)
        ld      (fn_t0),hl
fn_getc1:
        in      a,(PORTC)
        and     PC_IBF
        jr      nz,fn_getc2
        call    fn_expired
        jr      nc,fn_getc1
        jp      fn_abort
fn_getc2:
        pop     de
        pop     hl
fn_take:
        in      a,(PORTA)
        ret

; after a call given up part way: take bytes until 100 ms pass with none
; (and at most 2048, in case a missing card reads as an endless stream)
fn_drain:
        xor     a
        ld      (fn_dirty),a
        ld      a,DIR_TO_H89
        out     (PCTRL),a
        ld      hl,50
        ld      (fn_tlim),hl
        ld      bc,2048
fn_drain1:
        ld      hl,(TICKS)
        ld      (fn_t0),hl
fn_drain2:
        in      a,(PORTC)
        and     PC_IBF
        jr      z,fn_drain3
        in      a,(PORTA)
        dec     bc
        ld      a,b
        or      c
        ret     z
        jr      fn_drain1
fn_drain3:
        call    fn_expired
        jr      nc,fn_drain2
        ret

; take unframed bytes into DE (BC = room) until RAW_QUIET ticks of quiet
fn_raw:
        ld      (fn_rbuf),de
        ld      (fn_rmax),bc
        ld      hl,0
        ld      (fn_idx),hl
        ld      a,OE_ON
        out     (PCTRL),a
        ld      a,DIR_TO_H89
        out     (PCTRL),a
        ld      hl,RAW_FIRST
        ld      (fn_tlim),hl
fn_raw1:
        ld      hl,(TICKS)
        ld      (fn_t0),hl
fn_raw2:
        in      a,(PORTC)
        and     PC_IBF
        jr      nz,fn_raw3
        call    fn_expired
        jr      nc,fn_raw2
        ld      bc,(fn_idx)             ; quiet: done
        ret
fn_raw3:
        in      a,(PORTA)
        ld      c,a
        ld      hl,(fn_idx)
        ld      de,(fn_rmax)
        ld      a,l
        sub     e
        ld      a,h
        sbc     a,d
        jr      nc,fn_raw4              ; no room: count, drop
        ld      de,(fn_rbuf)
        add     hl,de
        ld      (hl),c
fn_raw4:
        ld      hl,(fn_idx)
        inc     hl
        ld      (fn_idx),hl
        ld      hl,RAW_QUIET
        ld      (fn_tlim),hl
        jr      fn_raw1

; aux bytes carried by each field descriptor, 0-7
fn_auxn:        defb    0, 1, 2, 3, 4, 2, 4, 4

; state
fn_rq:          defw    0
fn_sp:          defw    0
fn_out:         defs    10              ; header (6) and aux (up to 4)
fn_nout:        defb    0               ; header + aux bytes
fn_pay:         defw    0
fn_plen:        defw    0
fn_rbuf:        defw    0
fn_rmax:        defw    0
fn_rep:         defs    6               ; the reply's header
fn_idx:         defw    0
fn_cks:         defb    0
fn_t0:          defw    0
fn_tlim:        defw    0
fn_dirty:       defb    0
fn_only:        defb    0               ; 1: fn_send, no reply
fn_trunc:       defb    0
fn_res:         defb    0               ; the last call's result (A)
fn_len:         defw    0               ;   and reply length (BC)
```

SRC/ASM/INC/CARD.INC 55 LINES

```
;-----------------------------------------------------------------------
; card.inc -- start the H89 FujiNet card if nothing has yet (as FNON does),
; and face it toward us. From fujinet-h89's common/card.inc. Uses TICKS.
;
; The mark in port B and EN high mean something set the card up and nothing
; has reset the 82C55A since. EN alone can't tell: after a reset the 82C55A's
```

```
; bus hold can pull PC2 high while port A isn't in mode 2.
;
; Copyright (C) 2026 Thomas Cherryhomes
; SPDX-License-Identifier: GPL-3.0-or-later
;-------------------------------------------------------------------------

card_on:
        in      a,(PORTB)               ; set up since the last reset?
        cp      SETUP_MARK
        jr      nz,card_start
        in      a,(PORTC)
        and     PC_EN
        jr      nz,card_ready           ; yes, and running: leave it
card_start:
        ld      a,MODE2                 ; mode 2; PC0-PC2 out, all 0
        out     (PCTRL),a
        ld      a,0x06                  ; EN=1, IO0=1 at once
        out     (PORTC),a
        ld      b,100                   ; about 200 ms with IO0 high
card_wait:
        call    tick
        djnz    card_wait
        ld      a,SETUP_MARK            ; the card is set up
        out     (PORTB),a
card_ready:
        in      a,(PORTA)               ; drop a byte latched at start
        ld      a,OE_ON
        out     (PCTRL),a
        ld      a,DIR_TO_H89
        out     (PCTRL),a
        ret

; Is the card there, running and facing us? Z if so. Without a card the
; ports read FF, and every wait for a byte would see one at once, forever.
card_present:
        in      a,(PORTC)
        and     0x07                    ; EN, /OE, DIR
        cp      PC_EN
        ret

; Wait for the next 2 ms tick
tick:
        ld      a,(TICKS)
        ld      c,a
tick_wait:
        ld      a,(TICKS)
        cp      c
        jr      z,tick_wait
        ret
```

SRC/ASM/INC/CON.INC 247 LINES

```
;-------------------------------------------------------------------------
; con.inc -- printing numbers and bytes on the console (built on conout).
;
;   phex8   A in hex          phex16  HL in hex
;   pdec8   A in decimal      pdec16  HL in decimal
;   pspace  one space         pip     HL -> 4 bytes as a dotted IP address
;   ptext   HL -> BC bytes as text, unprintable ones as '.'
;   pdump   HL -> BC bytes in hex, 16 to a line
;   phexs   HL -> BC bytes in hex on one line (at most 24, then "...")
;   getline HL -> buffer, B = its size: a line typed at the keyboard, shown
;           as it is typed (BACKSPACE and DELETE rub out), ended by RETURN;
;           stored NUL-terminated without the RETURN. Keeps HL.
;   strlen  HL -> a NUL-terminated string: BC = its length. Keeps HL.
;
; All keep BC, DE and HL unless they say otherwise.
;
; Copyright (C) 2026 Thomas Cherryhomes
; SPDX-License-Identifier: GPL-3.0-or-later
;-------------------------------------------------------------------------

phex16:
        ld      a,h
        call    phex8
        ld      a,l
phex8:
        push    af
        rrca
        rrca
        rrca
        rrca
        call    phex4
        pop     af
phex4:
        and     0x0F
        add     a,'0'
        cp      '9'+1
        jp      c,conout
        add     a,'A'-'9'-1
        jp      conout

pspace:
        ld      a,' '
        jp      conout

pdec8:
        push    hl
        ld      l,a
        ld      h,0
        call    pdec16
        pop     hl
        ret

pdec16:
        push    bc
        push    de
        push    hl
        ld      b,0                     ; digits printed so far
        ld      de,-10000
        call    pdec1
        ld      de,-1000
        call    pdec1
        ld      de,-100
        call    pdec1
        ld      de,-10
        call    pdec1
        ld      a,l
        add     a,'0'
        call    conout
        pop     hl
        pop     de
        pop     bc
        ret
pdec1:
        ld      a,'0'-1
pdec2:
        inc     a
        add     hl,de
        jr      c,pdec2
        sbc     hl,de                   ; one too many: put it back
        cp      '0'                     ; no leading zeros
        jr      nz,pdec3
        inc     b
        dec     b
        ret     z
pdec3:
        ld      b,1
        jp      conout

pip:
        push    bc
        push    hl
        ld      b,4
pip1:
        ld      a,(hl)
        call    pdec8
        inc     hl
        dec     b
        jr      z,pip2
        ld      a,'.'
        call    conout
        jr      pip1
pip2:
        pop     hl
        pop     bc
        ret

ptext:
        push    bc
        push    hl
ptext1:
        ld      a,b
        or      c
        jr      z,ptext3
        ld      a,(hl)
        cp      ' '
        jr      c,ptext2
        cp      0x7F
        jr      c,ptext4
ptext2:
        ld      a,'.'
ptext4:
        call    conout
        inc     hl
        dec     bc
        jr      ptext1
ptext3:
        pop     hl
        pop     bc
        ret

phexs:
        push    bc
        push    de
```

```
134         push    hl
135         ld      d,24
136 phexs1:
137         ld      a,b
138         or      c
139         jr      z,phexs3
140         ld      a,d
141         or      a
142         jr      z,phexs2
143         ld      a,(hl)
144         call    phex8
145         call    pspace
146         inc     hl
147         dec     bc
148         dec     d
149         jr      phexs1
150 phexs2:
151         ld      a,'.'
152         call    conout
153         call    conout
154         call    conout
155 phexs3:
156         pop     hl
157         pop     de
158         pop     bc
159         ret
160
161 pdump:
162         push    bc
163         push    de
164         push    hl
165 pdump1:
166         ld      a,b
167         or      c
168         jr      z,pdump3
169         ld      e,16
170 pdump2:
171         ld      a,(hl)
172         call    phex8
173         call    pspace
174         inc     hl
175         dec     bc
176         ld      a,b
177         or      c
178         jr      z,pdump3
179         dec     e
180         jr      nz,pdump2
181         call    crlf
182         jr      pdump1
183 pdump3:
184         call    crlf
185         pop     hl
186         pop     de
187         pop     bc
188         ret
189
190 getline:
191         push    hl
192         push    de
193         dec     b                   ; room for the NUL
194         ld      c,0                 ; characters so far
195 getl1:
196         call    getkey
197         cp      13
198         jr      z,getl4
199         cp      8
200         jr      z,getl3
201         cp      0x7F
202         jr      z,getl3
203         cp      ' '
204         jr      c,getl1             ; other control characters: ignored
205         ld      e,a
206         ld      a,c
207         cp      b
208         jr      nc,getl1            ; full
209         ld      a,e
210         ld      (hl),a
211         inc     hl
212         inc     c
213         call    conout
214         jr      getl1
215 getl3:
216         ld      a,c                 ; rub out the last one, if any
217         or      a
218         jr      z,getl1
219         dec     hl
220         dec     c
221         ld      a,8
222         call    conout
223         ld      a,' '
224         call    conout
225         ld      a,8
226         call    conout
227         jr      getl1
228 getl4:
229         ld      (hl),0
230         call    crlf
231         pop     de
232         pop     hl
233         ret
234
235 strlen:
236         push    hl
237         ld      bc,0
238 strl1:
239         ld      a,(hl)
240         or      a
241         jr      z,strl2
242         inc     hl
243         inc     bc
244         jr      strl1
245 strl2:
246         pop     hl
247         ret
```

SRC/ASM/INC/TAIL.INC 25 LINES

```
 1 ;-----------------------------------------------------------------------
 2 ; tail.inc -- the library's data, the CP/M stack, and prog_end. Include it
 3 ; LAST, after your own code and data.
 4 ;
 5 ; Copyright (C) 2026 Thomas Cherryhomes
 6 ; SPDX-License-Identifier: GPL-3.0-or-later
 7 ;-----------------------------------------------------------------------
 8 os_fcnt:      defw    0           ; bytes in os_fbuf
 9 os_fptr:      defw    0           ; next byte to read from os_fbuf
10 os_feof:      defb    0           ; 1 = the file has ended
11 os_rcount:    defw    0
12 IFDEF HDOS
13 os_argp:      defw    0           ; SP at entry: the command line
14 os_nul:       defb    0           ; "no arguments"
15 os_consave:   defs    5           ; the console mode bytes at entry
16 os_hdefault:  defm    "SY0"       ; the default device, no extension
17               defb    0, 0, 0, 0
18 os_fbuf:      defs    256         ; one HDOS sector
19 ELSE
20 os_fcb:       defs    36
21 os_fbuf:      defs    128         ; one CP/M record
22               defs    256         ; our stack
23 os_stack:
24 ENDIF
25 prog_end:
```

ASSEMBLY ON THE H89 ITSELF

GENERAL

Both systems came with an assembler. HDOS's Software Tools diskette has Heath's ASM, and CP/M has Digital Research's ASM and LOAD. With them a FujiNet program can be written, assembled and run without leaving the H89, as programs were in 1980.

Both assemblers know only the 8080's instructions: the Z80 runs every one of them, but the Z80's own additions — relative jumps, DJNZ, the IX and IY registers, block moves, the two-byte `LD` instructions — are not available. So the programs in this chapter are written in 8080 code, which is longer, and neither assembler has macros. But they talk to the FujiNet exactly as the Z80 programs do, and their output is byte for byte what the book's cross-assembler makes from the same source.

This chapter shows four small programs, two for each system:

- FNONH (HDOS) and FNON (CP/M) start the card if nothing has, as `card_on` does (Chapter 2);
- FNVERH (HDOS) and FNVER (CP/M) make one complete FujiBus exchange in 8080 code — the request FUJI_GET_ADAPTERCONFIG, built, checksummed, framed and sent, and the reply taken, unescaped and checked — and print the FujiNet's network and firmware version.

HEATH'S ASM, UNDER HDOS

WHAT YOU NEED

- HDOS 2.0, and from the **Software Tools** diskette: ASM.ABS (issue #104.06.00), HOSDEF.ACM, which names HDOS's system calls, and ASCII.ACM, which names the control characters.
- A diskette for your work, in SY1:, with room for the program, its listing, and those three files if SY0: has no room for them (the book's boot disk had none).

ASSEMBLING

ASM takes an HDOS command line: the program to make, the listing, an equals sign, and the source:

```
SY1:ASM SY1:FNONH,SY1:FNONH=SY1:FNONH
```

Each name takes its usual extension (.ABS, .LST, .ASM). With no listing named, ASM shows any errors on the console; with one, they go in the listing, and the console shows only the totals. ASM makes the .ABS file itself, header and all: there is no separate load step. Run with nothing after its name, it shows its banner and asks with *.

( ) Mount the work diskette: MOUNT SY1: ↵.

( ) Assemble: SY1:ASM SY1:FNONH,SY1:FNONH=SY1:FNONH ↵. ASM takes about 50 seconds; FNVERH, a larger program, about 90.

( ) Run it: SY1:FNONH ↵.

```
>SY1:ASM SY1:FNONH,SY1:FNONH=SY1:FNONH
00158 STATEMENTS ASSEMBLED
40192 BYTES FREE
 NO ERRORS DETECTED
>SY1:FNONH
FUJINET ENABLED
>SY1:FNONH
FUJINET ALREADY ON
>
```

The second run finds the FujiNet running, and leaves it alone.

HEATH'S ASSEMBLY LANGUAGE

The book's programs keep to these rules, which differ from both z88dk's and Digital Research's:

- A label starts in column 1, and needs no colon. Labels may be up to 7 characters long, and may hold . and $ (as HDOS's own `.PRINT` and `S.TIME` do). Use capitals: lower case is allowed only in quotes.
- A whole-line comment starts with * in column 1. After the operand, everything past the first blank

is a comment, with no ; — so an operand may contain no blanks at all. `MVI A,HEX AND 0FH` quietly assembles as `MVI A,HEX`.

- Numbers are decimal unless they end in Q or O (octal), A (split octal: `42200A` is 042.200, which is 2280H), B (binary) or H (hexadecimal, starting with a digit). The location counter is *.
- Expressions use `+ - * /`, worked strictly from left to right, with no parentheses.
- Strings are in single quotes. `DB 'X'+200Q` sets the top bit of the last character, which ends a string for HDOS's `.PRINT`.
- `SCALL` is built in: `SCALL .PRINT` assembles as RST 7 (377Q) and the call's number (003Q). HOSDEF.ACM only names the numbers, and it sets the location counter itself (`ORG 0`), so `XTEXT HOSDEF` must come **before** the program's own `ORG`.
- `END` must name the entry point.

CAUTION: Heath's `IF` assembles the lines that follow when its expression is **zero**, the opposite of DRI's. The book's programs use no `IF`.

FNONH

FNONH is FNON (below) in Heath's language: 65 lines. Notice `XTEXT` before `ORG`, the octal numbers, and `.PRINT` with its strings ending in `'...'+200Q` or `ENL` (a new line with the top bit set).

SRC/NATIVE/FNONH.ASM 65 LINES

```
 1 *      FNONH.ASM - ENABLE THE FUJINET CARD IN AN H89 (82C55 AT 320Q)
 2 *
 3 *      THE HDOS TWIN OF FNON.ASM, FOR HEATH'S ASM.  SETS THE 82C55
 4 *      TO MODE 2, BOOTS THE ESP32 FROM FLASH AND TURNS THE 74LVC245
 5 *      TOWARD THE H89.  LEAVES A CARD THAT IS ALREADY ON ALONE:
 6 *      WHATEVER SETS THE CARD UP LEAVES 245Q IN PORT B, WHICH A RESET
 7 *      OR ANOTHER MODE WORD CLEARS.  EN ALONE CAN'T TELL, AS AFTER A
 8 *      RESET THE 82C55'S BUS HOLD CAN PULL PC2 HIGH.
 9 *
10 *      >SY1:ASM SY1:FNONH,SY1:FNONH=SY1:FNONH
11 *      >SY1:FNONH
12 *
13 *      ASM.ABS, HOSDEF.ACM AND ASCII.ACM (FROM THE SOFTWARE TOOLS
14 *      DISK) ARE ON SY1: WITH THIS FILE.
15 *
16 *      Copyright (C) 2026 Thomas Cherryhomes
17 *      SPDX-License-Identifier: GPL-3.0-or-later
18 *
19        XTEXT   HOSDEF          HDOS SYSTEM CALLS: .PRINT, .EXIT
20        XTEXT   ASCII           ENL: NEW LINE + 200Q, ENDS A LINE
21 *
22 PORTA  EQU     320Q            D0H: 82C55 PORT A, THE BYTE PIPE
23 PORTB  EQU     321Q            UNWIRED: HOLDS MARK ONCE SET UP
24 PORTC  EQU     322Q            PC0 DIR, PC1 /OE (ESP32 IO0), PC2 EN
25 PCTRL  EQU     323Q            CONTROL WORD, PORT C BIT SET/RESET
26 MARK   EQU     245Q            0A5H, AS FUJINET-LIB LEAVES IT
27 *
28        ORG     42200A          WHERE HDOS LOADS A PROGRAM
29 *
30 START  IN      PORTB
31        CPI     MARK            MARK AND EN HIGH: ALREADY RUNNING
32        JNZ     SETUP
33        IN      PORTC
34        ANI     004Q
35        JNZ     ISON
36 SETUP  MVI     A,300Q          PORT A MODE 2, PC0-PC2 OUTPUTS
37        OUT     PCTRL           (CLEARS PORT C, SO EN GOES LOW)
38        MVI     A,006Q          EN=1, IO0=1 AT ONCE: A RUNNING ESP32
39        OUT     PORTC           STAYS UP, ONE IN RESET BOOTS
40        LXI     B,17000         KEEP IO0 HIGH WHILE IT STARTS, ~200 MS
41        CALL    WAIT
42        MVI     A,MARK          THE CARD IS SET UP
43        OUT     PORTB
44        IN      PORTA           DROP ANY BYTE LATCHED DURING RESET
45        MVI     A,002Q          PC1=0: ENABLE THE 245
46        OUT     PCTRL
47        MVI     A,000Q          PC0=0: 245 DRIVES TOWARD THE H89
48        OUT     PCTRL
49        LXI     H,MSGON
50        JMP     DONE
51 ISON   LXI     H,MSGRUN
52 DONE   SCALL   .PRINT          PRINT TO THE BYTE WITH BIT 7 SET
53        XRA     A
54        SCALL   .EXIT           BACK TO HDOS
55 *
56 WAIT   DCX     B               ABOUT 12 US A PASS AT 2 MHZ
57        MOV     A,B
58        ORA     C
59        JNZ     WAIT
60        RET
61 *
62 MSGON  DB      'FUJINET ENABLED',ENL
63 MSGRUN DB      'FUJINET ALREADY ON',ENL
64 *
65        END     START
```

DIGITAL RESEARCH'S ASM, UNDER CP/M

Heath's CP/M 2.2.04 has ASM and LOAD on its system diskette. ASM reads the source and writes a .HEX file and a .PRN listing; LOAD turns the .HEX into a .COM file to run.

( ) With the source in drive B, type ASM B:FNVER ↵. ASM says what it is, the next free address after the program, how full its symbol table got, and END OF ASSEMBLY, with any errors above, a line each.

( ) Type LOAD B:FNVER ↵. LOAD reports the program's first and last addresses and how many 128-byte records it wrote.

( ) Run it: B:FNVER ↵.

```
A>ASM B:FNVER
CP/M ASSEMBLER - VER 2.0
03E1
002H USE FACTOR
END OF ASSEMBLY
A>LOAD B:FNVER
FIRST ADDRESS 0100
LAST  ADDRESS 02D6
BYTES READ    01D7
RECORDS WRITTEN 04
A>B:FNVER
Network:  Workshop
Firmware: v1.6-7d7a3ba
A>
```

The assembly took 32 seconds and the LOAD 8. DRI's language is the one most 8080 books use: labels as long as you like, with a colon after an instruction label; `;` for comments, anywhere; `$` the location counter; operators with the usual precedence and parentheses; `IF` and `ENDIF`.

CAUTION: DRI's ASM accepts `ELSE` without a word and does nothing with it: both branches are assembled.

A FUJIBUS EXCHANGE IN 8080 CODE

FNVER and FNVERH do the whole of `fn_call`'s work, for one request, in about 240 lines. In order:

1. Start the card, as FNON does, unless port B holds the mark and EN is 1.
2. Build the packet `70 E8 06 00 00 00` and work out its checksum (an 8-bit sum with end-around carry: `ADD M`, then `ACI 0`), giving `70 E8 06 00 5F 00`.
3. Send it between two C0H bytes, escaping any C0H or DBH inside, a byte at a time, each when /OBF says the last was taken.
4. Take the reply up to its closing C0H, turning DB DC and DB DD back into C0H and DBH, giving up after 15 seconds of the 2 ms clock (7,500 ticks).
5. Check the reply: its length word against the bytes received, its checksum (with byte 4 counted as 0), ACK rather than NAK, and at least 146 bytes.
6. Print the network's name (from offset 0 of the data) and the firmware's version (offset 125).

The reply the FujiNet sent contained the escape pair DB DC (a C0H in a MAC address), so the unescaping was exercised.

```
>SY1:ASM SY1:FNVERH,SY1:FNVERH=SY1:FNVERH
00330 STATEMENTS ASSEMBLED
39909 BYTES FREE
 NO ERRORS DETECTED
>SY1:FNVERH
NETWORK:  WORKSHOP
FIRMWARE: V1.6-7D7A3BA
>
```

FROM Z80 TO 8080

These are the substitutions the 8080 programs make for what the book's Z80 library does:

Z80	8080	NOTE
`JR, JR Z …`	`JMP, JZ …`	no relative jumps; 3 bytes each
`DJNZ loop`	`DCR B` / `JNZ loop`	
`IN A,(PORTC)`	`IN PORTC`	A and a fixed port only
`LD (nn),SP`	`LXI H,0` / `DAD SP` / `SHLD nn`	
`LD SP,(nn)`	`LHLD nn` / `SPHL`	
`LD DE,(nn)`	`XCHG` / `LHLD nn` / `XCHG`	only HL loads a word directly
`SBC HL,DE`	six byte instructions	or count down instead
`LDIR`	a loop	
`IX, IY, BIT n,r`	memory; `ANI mask`	
`LD A,n, LD A,(HL)`	`MVI A,n, MOV A,M`	
`CP n`	`CPI n`	8080 `CP` is **call if plus**
`JP nn`	`JMP nn`	8080 `JP` is **jump if plus**
`ADC A,0`	`ACI 0`	the checksum's carry
`PUSH AF`	`PUSH PSW`	
`EX DE,HL`	`XCHG`	
SCALL macro	`SCALL fn` (Heath only)	neither ASM has macros

WARNING: The 8080's `JP` and `CP` are **jump if plus** and **call if plus**. A Z80 habit, `JP label` or `CP 3`, assembles without a word of complaint and does something quite different.

THE SAME BYTES

Each of the four programs was also assembled by z88dk-z80asm in 8080 mode (`-m8080`), from the same source with only the dialect changed. The results are identical, byte for byte: every byte of ASM's .HEX files and LOAD's .COM files, and every code byte of Heath ASM's .ABS files. (Heath's ASM also writes the space reserved by `DS` into the .ABS file, filled with whatever was in its buffer, where the others leave it out.) FNON.COM is also identical to the copy that comes with FujiNet Go H89.

THE LISTINGS

These are the assemblers' own listings, as they came off the work diskettes: FNVERH from Heath's ASM, with its split-octal addresses, octal bytes and statement numbers (the lines of HOSDEF.ACM and ASCII.ACM are not listed, so the numbers jump); FNVER from DRI's, in hexadecimal. FNON's source follows them.

DATA/NATIVE/FNVERH.LST 258 LINES

```
                                HEATH ASM #104.06.00
                                12-Oct-80  Page    1

00001   *       FNVERH.ASM - ASK THE H89 FUJINET FOR ITS FIRMWARE VERSION
00002   *
00003   *       THE HDOS TWIN OF FNVER.ASM, FOR HEATH'S ASM.  STARTS THE
    CARD
00004   *       IF NOTHING HAS, SENDS FUJI_GET_ADAPTERCONFIG (DEVICE 160Q,
00005   *       COMMAND 350Q) IN A SLIP FRAME, CHECKS THE REPLY AND PRINTS
00006   *       THE NETWORK NAME AND FIRMWARE VERSION.  PURE 8080.
00007   *
00008   *       )SY1:ASM SY1:FNVERH,SY1:FNVERH=SY1:FNVERH
00009   *       )SY1:FNVERH
00010   *
00011   *       ASM.ABS, HOSDEF.ACM AND ASCII.ACM (FROM THE SOFTWARE TOOLS
00012   *       DISK) ARE ON SY1: WITH THIS FILE.
00013   *
00014   *       Copyright (C) 2026 Thomas Cherryhomes
00015   *       SPDX-License-Identifier: GPL-3.0-or-later
00016   *
042.200 00017           XTEXT   HOSDEF          HDOS SYSTEM CALLS
000.207 00083           XTEXT   ASCII           NL, ENL
00112   *
000.320 00113   PORTA   EQU     320Q            D8H: 82C55 PORT A, THE BYTE PIPE
000.321 00114   PORTB   EQU     321Q            UNWIRED: HOLDS MARK ONCE SET UP
000.322 00115   PORTC   EQU     322Q            STATUS IN, PC0-PC2 OUT
000.323 00116   PCTRL   EQU     323Q            CONTROL WORD, PORT C BIT SET/RESET
000.245 00117   MARK    EQU     245Q            0A5H, AS FUJINET-LIB LEAVES IT
000.200 00118   OBF     EQU     200Q            PC7 /OBF: 1 = OUR LAST BYTE WAS
```

```
   TAKEN
000.040             00119  IBF    EQU   040Q          PC5 IBF: 1 = A BYTE WAITS FOR US
040.033             00120  TICKS  EQU   40033A        HDOS 2 MS CLOCK (.TICCNT IN MTR.ACM)
035.114             00121  TMAX   EQU   7500          GIVE UP AFTER 15 S (7500 TICKS)
000.300             00122  SLEND  EQU   300Q          SLIP END: FRAMES A PACKET
000.333             00123  SLESC  EQU   333Q          SLIP ESC
000.334             00124  ESCEND EQU   334Q          333 334 STANDS FOR A 300
000.335             00125  ESCESC EQU   335Q          333 335 STANDS FOR A 333
000.310             00126  RMAX   EQU   200           REPLY BUFFER SIZE
                    00127  *
042.200             00128         ORG   42200H       WHERE HDOS LOADS A PROGRAM
                    00129  *
042.200 315 273 042 00130  START  CALL  CARDON        START THE CARD IF IT IS OFF
042.203 315 355 042 00131         CALL  SENDRQ        SEND THE REQUEST
042.206 315 060 043 00132         CALL  RECV          TAKE THE REPLY
042.211 315 151 043 00133         CALL  CHECK         CARRY: HL -> WHAT WAS WRONG
042.214 332 254 042 00134         JC    EXIT
042.217 041 372 043 00135         LXI   H,MNET
042.222 377 003     00136         SCALL .PRINT
042.224 041 072 044 00137         LXI   H,REPLY+6     SSID: OFFSET 0, 33 BYTES
042.227 006 041     00138         MVI   B,33
042.231 315 345 043 00139         CALL  PRFLD
042.234 041 004 044 00140         LXI   H,MVER
042.237 377 003     00141         SCALL .PRINT
042.241 041 267 044 00142         LXI   H,REPLY+6+125 FN_VERSION: OFFSET 125, 15 BYTES
042.244 006 017     00143         MVI   B,15
042.246 315 345 043 00144         CALL  PRFLD
042.251 041 017 044 00145         LXI   H,MEND
042.254 377 003     00146  EXIT   SCALL .PRINT       PRINT TO THE BYTE WITH BIT 7 SET
042.256 257         00147         XRA   A
042.257 377 000     00148         SCALL .EXIT         BACK TO HDOS
042.261 076 000     00149  TMOUT  MVI   A,000Q       NO ANSWER: PC0=0, BUS TO THE H89
042.263 323 323     00150         OUT   PCTRL
                                          HEATH ASM #104.06.00
                                          12-Oct-80  Page    2


042.265 041 053 044 00151         LXI   H,MTIME
042.270 303 254 042 00152         JMP   EXIT
                    00153  *
042.273 333 321     00154  CARDON IN    PORTB
042.275 376 245     00155         CPI   MARK          MARK AND EN HIGH: ALREADY RUNNING
042.277 302 311 042 00156         JNZ   CSTART
042.302 333 322     00157         IN    PORTC
042.304 346 004     00158         ANI   004Q
042.306 302 333 042 00159         JNZ   CREADY
042.311 076 300     00160  CSTART MVI   A,300Q       PORT A MODE 2, PC0-PC2 OUTPUTS
042.313 323 323     00161         OUT   PCTRL         (CLEARS PORT C, SO EN GOES LOW)
042.315 076 006     00162         MVI   A,006Q       EN=1, IO0=1 AT ONCE: A RUNNING ESP32
042.317 323 322     00163         OUT   PORTC         STAYS UP, ONE IN RESET BOOTS
042.321 001 150 102 00164         LXI   B,17000       KEEP IO0 HIGH WHILE IT STARTS, ~200
   MS
042.324 315 346 042 00165         CALL  WAIT
042.327 076 245     00166         MVI   A,MARK        THE CARD IS SET UP
042.331 323 321     00167         OUT   PORTB
042.333 333 320     00168  CREADY IN    PORTA         DROP ANY BYTE LATCHED DURING RESET
042.335 076 002     00169         MVI   A,002Q       PC1=0: ENABLE THE 245
042.337 323 323     00170         OUT   PCTRL
042.341 076 000     00171         MVI   A,000Q       PC0=0: 245 DRIVES TOWARD THE H89
042.343 323 323     00172         OUT   PCTRL
042.345 311         00173         RET
042.346 013         00174  WAIT   DCX   B             ABOUT 12 US A PASS AT 2 MHZ
042.347 170         00175         MOV   A,B
042.350 261         00176         ORA   C
042.351 302 346 042 00177         JNZ   WAIT
042.354 311         00178         RET
                    00179  *
042.355 041 364 043 00180  SENDRQ LXI   H,PKT         CHECKSUM THE PACKET, BYTE 4 AS 0
042.360 006 006     00181         MVI   B,6
042.362 315 245 043 00182         CALL  CKSUM
042.365 062 370 043 00183         STA   PKT+4
042.370 076 001     00184         MVI   A,001Q       PC0=1: 245 DRIVES TOWARD THE ESP32
042.372 323 323     00185         OUT   PCTRL
042.374 076 300     00186         MVI   A,SLEND       ONE END BEFORE THE PACKET
042.376 315 257 043 00187         CALL  PUTC
043.001 041 364 043 00188         LXI   H,PKT
043.004 006 006     00189         MVI   B,6
043.006 176         00190  SEND1  MOV   A,M
043.007 376 300     00191         CPI   SLEND         300 GOES AS 333 334,
043.011 016 334     00192         MVI   C,ESCEND      (MVI LEAVES THE FLAGS ALONE)
043.013 312 025 043 00193         JZ    SEND2
043.016 376 333     00194         CPI   SLESC         333 AS 333 335
043.020 016 335     00195         MVI   C,ESCESC
043.022 302 033 043 00196         JNZ   SEND3
043.025 076 333     00197  SEND2  MVI   A,SLESC
043.027 315 257 043 00198         CALL  PUTC
043.032 171         00199         MOV   A,C
043.033 315 257 043 00200  SEND3  CALL  PUTC
043.036 043         00201         INX   H
043.037 005         00202         DCR   B
043.040 302 006 043 00203         JNZ   SEND1
043.043 076 300     00204         MVI   A,SLEND       ONE END AFTER IT
043.045 315 257 043 00205         CALL  PUTC
043.050 315 267 043 00206         CALL  TXWAIT        UNTIL THE ESP32 HAS IT ALL
043.053 076 000     00207         MVI   A,000Q       PC0=0: BACK TOWARD THE H89
                                          HEATH ASM #104.06.00
                                          12-Oct-80  Page    3


043.055 323 323     00208         OUT   PCTRL
043.057 311         00209         RET
                    00210  *
043.060 315 274 043 00211  RECV   CALL  GETC          WAIT FOR AN END,
043.063 376 300     00212         CPI   SLEND
043.065 302 060 043 00213         JNZ   RECV
043.070 315 274 043 00214  RECV1  CALL  GETC          SKIP ANY MORE
043.073 376 300     00215         CPI   SLEND
043.075 312 070 043 00216         JZ    RECV1
043.100 041 064 044 00217         LXI   H,REPLY       HL -> BUFFER, C = BYTES KEPT
043.103 016 000     00218         MVI   C,0
043.105 376 300     00219  RECV2  CPI   SLEND         THE CLOSING END: DONE
043.107 310         00220         RZ
043.110 376 333     00221         CPI   SLESC         ESC: THE NEXT BYTE SAYS WHICH
043.112 302 131 043 00222         JNZ   RECV3
043.115 315 274 043 00223         CALL  GETC
043.120 376 334     00224         CPI   ESCEND        333 334 WAS A 300,
043.122 076 300     00225         MVI   A,SLEND
043.124 312 131 043 00226         JZ    RECV3
043.127 076 333     00227         MVI   A,SLESC       333 335 A 333
043.131 107         00228  RECV3  MOV   B,A           KEEP IT IF THERE IS ROOM
043.132 171         00229         MOV   A,C
043.133 376 310     00230         CPI   RMAX
043.135 322 143 043 00231         JNC   RECV4
043.140 160         00232         MOV   M,B
043.141 043         00233         INX   H
043.142 014         00234         INR   C
043.143 315 274 043 00235  RECV4  CALL  GETC
043.146 303 105 043 00236         JMP   RECV2
                    00237  *
043.151 171         00238  CHECK  MOV   A,C           A HEADER AT LEAST
043.152 376 006     00239         CPI   6
043.154 332 240 043 00240         JC    BAD
043.157 052 066 044 00241         LHLD  REPLY+2       LENGTH WORD = BYTES RECEIVED
043.162 174         00242         MOV   A,H
043.163 267         00243         ORA   A
043.164 302 240 043 00244         JNZ   BAD
043.167 175         00245         MOV   A,L
043.170 271         00246         CMP   C
043.171 302 240 043 00247         JNZ   BAD
043.174 041 070 044 00248         LXI   H,REPLY+4     CHECKSUM: ITS OWN BYTE COUNTS AS 0
043.177 136         00249         MOV   E,M
043.200 066 000     00250         MVI   M,0
043.202 041 064 044 00251         LXI   H,REPLY
043.205 101         00252         MOV   B,C
043.206 315 245 043 00253         CALL  CKSUM
043.211 273         00254         CMP   E
043.212 302 240 043 00255         JNZ   BAD
043.215 041 032 044 00256         LXI   H,MMRK
043.220 072 065 044 00257         LDA   REPLY+1
043.223 376 025     00258         CPI   025Q          NAK: THE FUJINET SAID NO
043.225 067         00259         STC
043.226 310         00260         RZ
043.227 376 006     00261         CPI   006Q          ACK, WITH ALL 140 BYTES
043.231 302 240 043 00262         JNZ   BAD
043.234 171         00263         MOV   A,C
043.235 376 222     00264         CPI   6+140
                                          HEATH ASM #104.06.00
                                          12-Oct-80  Page    4


043.237 320         00265         RNC
043.240 041 020 044 00266  BAD    LXI   H,MBAD
043.243 067         00267         STC
043.244 311         00268         RET
                    00269  *
043.245 257         00270  CKSUM  XRA   A             A = SUM OF B BYTES AT HL,
043.246 206         00271  CKS1   ADD   M             WITH EACH CARRY ADDED BACK IN
043.247 316 000     00272         ACI   0
043.251 043         00273         INX   H
043.252 005         00274         DCR   B
043.253 302 246 043 00275         JNZ   CKS1
043.256 311         00276         RET
                    00277  *
043.257 365         00278  PUTC   PUSH  PSW           SEND A WHEN THE LATCH IS EMPTY
043.260 315 267 043 00279         CALL  TXWAIT
043.263 361         00280         POP   PSW
043.264 323 320     00281         OUT   PORTA
043.266 311         00282         RET
043.267 036 200     00283  TXWAIT  MVI   E,OBF        WAIT FOR /OBF HIGH
043.271 303 304 043 00284         JMP   PCWAIT
043.274 036 040     00285  GETC   MVI   E,IBF         A = THE NEXT BYTE FROM THE ESP32
043.276 315 304 043 00286         CALL  PCWAIT
043.301 333 320     00287         IN    PORTA
043.303 311         00288         RET
043.304 345         00289  PCWAIT  PUSH  H             WAIT UNTIL PORT C HAS BIT E SET,
043.305 041 114 035 00290         LXI   H,TMAX        FOR UP TO TMAX TICKS. KEEPS BC HL
043.310 072 033 040 00291         LDA   TICKS
043.313 127         00292         MOV   D,A           D = THE TICK WE ARE IN
043.314 333 322     00293  PCW1   IN    PORTC
043.316 243         00294         ANA   E
043.317 302 343 043 00295         JNZ   PCW2
043.322 072 033 040 00296         LDA   TICKS         HAS THE CLOCK TICKED?
043.325 272         00297         CMP   D
043.326 312 314 043 00298         JZ    PCW1
043.331 127         00299         MOV   D,A           YES: ONE TICK FEWER LEFT
```

```
 043.332  053        00300          DCX     H
 043.333  174        00301          MOV     A,H
 043.334  265        00302          ORA     L
 043.335  302 314 043 00303          JNZ     PCW1
 043.340  303 261 042 00304          JMP     TMOUT           NONE LEFT: GIVE UP
 043.343  341        00305  PCW2     POP     H
 043.344  311        00306          RET
                     00307  *
 043.345  176        00308  PRFLD    MOV     A,M             PRINT UP TO B CHARACTERS AT HL,
 043.346  267        00309          ORA     A               STOPPING AT A NUL
 043.347  310        00310          RZ
 043.350  345        00311          PUSH    H
 043.351  305        00312          PUSH    B
 043.352  377 002    00313          SCALL   .SCOUT          CONSOLE OUTPUT
 043.354  301        00314          POP     B
 043.355  341        00315          POP     H
 043.356  043        00316          INX     H
 043.357  005        00317          DCR     B
 043.360  302 345 043 00318          JNZ     PRFLD
 043.363  311        00319          RET
                     00320  *
 043.364  160 350 006 00321  PKT     DB      160Q,350Q,6,0,0,0  DEV, CMD, LENGTH, CHECKSUM, DESC
                                                HEATH ASM #104.06.00
                                                12-Oct-80  Page    5


 043.372  116 105 124 00322  MNET    DB      'NETWORK: ',' '+200Q
 044.004  012 106 111 00323  MVER    DB      NL,'FIRMWARE:',' '+200Q
 044.017  212        00324  MEND    DB      ENL
 044.020  102 101 104 00325  MBAD    DB      'BAD REPLY',ENL
 044.032  106 125 112 00326  MNAK    DB      'FUJINET SAID NAK',ENL
 044.053  116 117 040 00327  MTIME   DB      'NO REPLY',ENL
 044.064              00328  REPLY   DS      RMAX            THE REPLY PACKET, UNESCAPED
                      00329  *
 044.374  000         00330          END     START


00330 Statements Assembled
39909 Bytes Free
 No Errors Detected
```

DATA/NATIVE/FNVER.PRN 245 LINES

```
                ;       FNVER.ASM - ask the H89 FujiNet for its firmware version
                ;
                ;       Starts the card if nothing has, sends FUJI_GET_ADAPTERCONFIG
                ;       (device 70H, command 0E8H) in a SLIP frame, checks the reply
                ;       and prints the network name and firmware version. Pure 8080.
                ;
                ;       A>ASM B:FNVER
                ;       A>LOAD B:FNVER
                ;       A>B:FNVER
                ;
                ;       Copyright (C) 2026 Thomas Cherryhomes
                ;       SPDX-License-Identifier: GPL-3.0-or-later
                ;
00D0 =          PORTA   EQU     0D0H            ;82C55 PORT A, THE BYTE PIPE
00D1 =          PORTB   EQU     0D1H            ;WIRED TO NOTHING: HOLDS MARK ONCE SET UP
00D2 =          PORTC   EQU     0D2H            ;STATUS IN, PC0-PC2 OUT
00D3 =          PCTRL   EQU     0D3H            ;CONTROL WORD, PORT C BIT SET/RESET
00A5 =          MARK    EQU     0A5H            ;AS FUJINET-LIB'S PORT_INIT LEAVES IT
0080 =          OBF     EQU     80H             ;PC7 /OBF: 1 = OUR LAST BYTE WAS TAKEN
0020 =          IBF     EQU     20H             ;PC5 IBF: 1 = A BYTE WAITS FOR US
000B =          TICKS   EQU     000BH           ;HEATH BIOS 2 MS CLOCK
1D4C =          TMAX    EQU     7500            ;GIVE UP AFTER 15 S (7500 TICKS)
0005 =          BDOS    EQU     0005H
00C0 =          SLEND   EQU     0C0H            ;SLIP END: FRAMES A PACKET
00DB =          SLESC   EQU     0DBH            ;SLIP ESC
00DC =          ESCEND  EQU     0DCH            ;DB DC STANDS FOR A C0H
00DD =          ESCESC  EQU     0DDH            ;DB DD STANDS FOR A DBH
00C8 =          RMAX    EQU     200             ;REPLY BUFFER SIZE
                ;
0100                    ORG     0100H
                ;
0100 210000     START:  LXI     H,0             ;KEEP THE CCP'S STACK POINTER
0103 39                 DAD     SP
0104 22D702             SHLD    OLDSP
0107 31E103             LXI     SP,STACK        ;AND USE OUR OWN
010A CD4A01             CALL    CARDON          ;START THE CARD IF IT IS OFF
010D CD7C01             CALL    SENDRQ          ;SEND THE REQUEST
0110 CDBF01             CALL    RECV            ;TAKE THE REPLY
0113 CDF801             CALL    CHECK           ;CARRY: DE -> WHAT WAS WRONG
0116 DA3801             JC      EXIT
0119 119202             LXI     D,MNET
011C CD8702             CALL    PRMSG
011F 21DF02             LXI     H,REPLY+6       ;SSID: OFFSET 0, 33 BYTES
0122 0621               MVI     B,33
0124 CD7402             CALL    PRFLD
0127 119D02             LXI     D,MVER
012A CD8702             CALL    PRMSG
012D 215C03             LXI     H,REPLY+6+125   ;FN_VERSION: OFFSET 125, 15 BYTES
0130 060F               MVI     B,15
0132 CD7402             CALL    PRFLD
0135 11AA02             LXI     D,MEND
0138 CD8702     EXIT:   CALL    PRMSG
013B 2AD702             LHLD    OLDSP           ;BACK TO THE CCP
013E F9                 SPHL
013F C9                 RET
0140 3E00       TMOUT:  MVI     A,00H           ;NO ANSWER: PC0=0, BUS TO THE H89
0142 D3D3               OUT     PCTRL
0144 11CC02             LXI     D,MTIME
0147 C33801             JMP     EXIT            ;(EXIT RESETS SP)
                ;
014A DBD1       CARDON: IN      PORTB
014C FEA5               CPI     MARK            ;MARK AND EN HIGH: CARD ALREADY RUNNING
014E C25801             JNZ     CSTART
0151 DBD2               IN      PORTC
0153 E604               ANI     04H
0155 C26A01             JNZ     CREADY
0158 3EC0       CSTART: MVI     A,0C0H          ;PORT A MODE 2, PC0-PC2 OUTPUTS
015A D3D3               OUT     PCTRL           ;(CLEARS PORT C, SO EN GOES LOW)
015C 3E06               MVI     A,06H           ;EN=1, IO0=1 AT ONCE: A RUNNING ESP32
015E D3D2               OUT     PORTC           ;STAYS UP, ONE IN RESET BOOTS FROM FLASH
0160 016842             LXI     B,17000         ;KEEP IO0 HIGH WHILE IT STARTS, ~200 MS
0163 CD7501             CALL    WAIT
0166 3EA5               MVI     A,MARK          ;THE CARD IS SET UP
0168 D3D1               OUT     PORTB
016A DBD0       CREADY: IN      PORTA           ;DROP ANY BYTE LATCHED DURING RESET
016C 3E02               MVI     A,02H           ;PC1=0: ENABLE THE 245
016E D3D3               OUT     PCTRL
0170 3E00               MVI     A,00H           ;PC0=0: 245 DRIVES TOWARD THE H89
0172 D3D3               OUT     PCTRL
0174 C9                 RET
0175 0B         WAIT:   DCX     B               ;ABOUT 12 US A PASS AT 2 MHZ
0176 78                 MOV     A,B
0177 B1                 ORA     C
0178 C27501             JNZ     WAIT
017B C9                 RET
                ;
017C 218C02     SENDRQ: LXI     H,PKT           ;CHECKSUM THE PACKET, BYTE 4 AS 0
017F 0606               MVI     B,6
0181 CD3402             CALL    CKSUM
0184 329002             STA     PKT+4
0187 3E01               MVI     A,01H           ;PC0=1: 245 DRIVES TOWARD THE ESP32
0189 D3D3               OUT     PCTRL
018B 3EC0               MVI     A,SLEND         ;ONE END BEFORE THE PACKET
018D CD3E02             CALL    PUTC
0190 218C02             LXI     H,PKT
0193 0606               MVI     B,6
0195 7E         SEND1:  MOV     A,M
0196 FEC0               CPI     SLEND           ;C0H GOES AS DB DC,
0198 0EDC               MVI     C,ESCEND        ;(MVI LEAVES THE FLAGS ALONE)
019A CAA401             JZ      SEND2
019D FEDB               CPI     SLESC           ;DBH AS DB DD
019F 0EDD               MVI     C,ESCESC
01A1 C2AA01             JNZ     SEND3
01A4 3EDB       SEND2:  MVI     A,SLESC
01A6 CD3E02             CALL    PUTC
01A9 79                 MOV     A,C
01AA CD3E02     SEND3:  CALL    PUTC
01AD 23                 INX     H
01AE 05                 DCR     B
01AF C29501             JNZ     SEND1
01B2 3EC0               MVI     A,SLEND         ;ONE END AFTER IT
01B4 CD3E02             CALL    PUTC
01B7 CD4602             CALL    TXWAIT          ;UNTIL THE ESP32 HAS IT ALL
01BA 3E00               MVI     A,00H           ;PC0=0: BACK TOWARD THE H89
01BC D3D3               OUT     PCTRL
01BE C9                 RET
                ;
01BF CD4B02     RECV:   CALL    GETC            ;WAIT FOR AN END,
01C2 FEC0               CPI     SLEND
01C4 C2BF01             JNZ     RECV
01C7 CD4B02     RECV1:  CALL    GETC            ;SKIP ANY MORE
01CA FEC0               CPI     SLEND
01CC CAC701             JZ      RECV1
01CF 21D902             LXI     H,REPLY         ;HL -> BUFFER, C = BYTES KEPT
01D2 0E00               MVI     C,0
01D4 FEC0       RECV2:  CPI     SLEND           ;THE CLOSING END: DONE
01D6 C8                 RZ
01D7 FEDB               CPI     SLESC           ;ESC: THE NEXT BYTE SAYS WHICH
01D9 C2E801             JNZ     RECV3
01DC CD4B02             CALL    GETC
01DF FEDC               CPI     ESCEND          ;DB DC WAS A C0H,
01E1 3EC0               MVI     A,SLEND
01E3 CAE801             JZ      RECV3
01E6 3EDB               MVI     A,SLESC         ;DB DD A DBH
01E8 47         RECV3:  MOV     B,A             ;KEEP IT IF THERE IS ROOM
01E9 79                 MOV     A,C
01EA FEC8               CPI     RMAX
```

```
01EC D2F201           JNC     RECV4
01EF 70               MOV     M,B
01F0 23               INX     H
01F1 0C               INR     C
01F2 CD4B02     RECV4:CALL    GETC
01F5 C3D401           JMP     RECV2
                ;
01F8 79         CHECK:MOV     A,C             ;A HEADER AT LEAST
01F9 FE06             CPI     6
01FB DA2F02           JC      BAD
01FE 2ADB02           LHLD    REPLY+2         ;LENGTH WORD = BYTES RECEIVED
0201 7C               MOV     A,H
0202 B7               ORA     A
0203 C22F02           JNZ     BAD
0206 7D               MOV     A,L
0207 B9               CMP     C
0208 C22F02           JNZ     BAD
020B 21DD02           LXI     H,REPLY+4       ;CHECKSUM: ITS OWN BYTE COUNTS AS 0
020E 5E               MOV     E,M
020F 3600             MVI     M,0
0211 21D902           LXI     H,REPLY
0214 41               MOV     B,C
0215 CD3402           CALL    CKSUM
0218 BB               CMP     E
0219 C22F02           JNZ     BAD
021C 11B902           LXI     D,MNAK
021F 3ADA02           LDA     REPLY+1
0222 FE15             CPI     15H             ;NAK: THE FUJINET SAID NO
0224 37               STC
0225 C8               RZ
0226 FE06             CPI     06H             ;ACK, WITH ALL 140 BYTES
0228 C22F02           JNZ     BAD
022B 79               MOV     A,C
022C FE92             CPI     6+140
022E D0               RNC
022F 11AD02     BAD:  LXI     D,MBAD
0232 37               STC
0233 C9               RET
                ;
0234 AF         CKSUM:XRA     A               ;A = SUM OF B BYTES AT HL,
0235 86         CKS1: ADD     M               ;WITH EACH CARRY ADDED BACK IN
0236 CE00             ACI     0
0238 23               INX     H
0239 05               DCR     B
023A C23502           JNZ     CKS1
023D C9               RET
                ;
023E F5         PUTC: PUSH    PSW             ;SEND A WHEN THE LATCH IS EMPTY
023F CD4602           CALL    TXWAIT
0242 F1               POP     PSW
0243 D3D0             OUT     PORTA
0245 C9               RET
0246 1E80       TXWAIT:MVI    E,OBF           ;WAIT FOR /OBF HIGH
0248 C35302           JMP     PCWAIT
024B 1E20       GETC: MVI     E,IBF           ;A = THE NEXT BYTE FROM THE ESP32
024D CD5302           CALL    PCWAIT
0250 DBD0             IN      PORTA
0252 C9               RET
0253 E5         PCWAIT:PUSH   H               ;WAIT UNTIL PORT C HAS BIT E SET,
0254 214C1D           LXI     H,TMAX          ;FOR UP TO TMAX TICKS. KEEPS BC HL
0257 3A0B00           LDA     TICKS
025A 57               MOV     D,A             ;D = THE TICK WE ARE IN
025B DBD2       PCW1: IN      PORTC
025D A3               ANA     E
025E C27202           JNZ     PCW2
0261 3A0B00           LDA     TICKS           ;HAS THE CLOCK TICKED?
0264 BA               CMP     D
0265 CA5B02           JZ      PCW1
0268 57               MOV     D,A             ;YES: ONE TICK FEWER LEFT
0269 2B               DCX     H
026A 7C               MOV     A,H
026B B5               ORA     L
026C C25B02           JNZ     PCW1
026F C34001           JMP     TMOUT           ;NONE LEFT: GIVE UP
0272 E1         PCW2: POP     H
0273 C9               RET
                ;
0274 7E         PRFLD:MOV     A,M             ;PRINT UP TO B CHARACTERS AT HL,
0275 B7               ORA     A               ;STOPPING AT A NUL
0276 C8               RZ
0277 E5               PUSH    H
0278 C5               PUSH    B
0279 5F               MOV     E,A
027A 0E02             MVI     C,2             ;BDOS CONSOLE OUTPUT
027C CD0500           CALL    BDOS
027F C1               POP     B
0280 E1               POP     H
0281 23               INX     H
0282 05               DCR     B
0283 C27402           JNZ     PRFLD
0286 C9               RET
0287 0E09       PRMSG:MVI     C,9             ;BDOS PRINT STRING, TO THE $
0289 C30500           JMP     BDOS
                ;
028C 70E8060000PKT:   DB      70H,0E8H,6,0,0,0 ;DEV, CMD, LENGTH, CHECKSUM, DESC
0292 4E6574776FMNET:  DB      'Network:  $'
029D 0D0A466972MVER:  DB      0DH,0AH,'Firmware: $'
02AA 0D0A24     MEND: DB      0DH,0AH,'$'
02AD 4261642072MBAD:  DB      'Bad reply',0DH,0AH,'$'
02B9 46756A694EMNAK:  DB      'FujiNet said NAK',0DH,0AH,'$'
02CC 4E6F207265MTIME: DB      'No reply',0DH,0AH,'$'
02D7            OLDSP:DS      2               ;THE CCP'S SP
02D9            REPLY:DS      RMAX            ;THE REPLY PACKET, UNESCAPED
03A1                  DS      64
                STACK:
03E1                  END     START
```

SRC/NATIVE/FNON.ASM 64 LINES

```
 1 ;       FNON.ASM - enable the FujiNet card in an H89 (82C55 at 0D0H)
 2 ;
 3 ;       Sets the 82C55 to Mode 2, boots the ESP32 from flash and turns
 4 ;       the 74LVC245 toward the H89. Leaves a card that is already on alone:
 5 ;       whatever sets the card up leaves 0A5H in port B, which a reset or
 6 ;       another mode word clears. EN alone can't tell, as after a reset the
 7 ;       82C55's bus hold can pull PC2 high while port A isn't in Mode 2.
 8 ;
 9 ;       A>ASM B:FNON
10 ;       A>LOAD B:FNON
11 ;       A>B:FNON
12 ;
13 ;       From FujiNet Go H89's CP/M disks: fujinet-go-h89-desktop
14 ;       tools/disks/fnon/FNON.ASM (51f411d), code unchanged.
15 ;
16 ;       Copyright (C) 2026 Thomas Cherryhomes
17 ;       SPDX-License-Identifier: GPL-3.0-or-later
18 ;
19 PORTA   EQU     0D0H            ;82C55 PORT A, THE BYTE PIPE
20 PORTB   EQU     0D1H            ;WIRED TO NOTHING: HOLDS MARK ONCE SET UP
21 PORTC   EQU     0D2H            ;PC0 DIR, PC1 /OE (ESP32 IO0), PC2 ESP32 EN
22 PCTRL   EQU     0D3H            ;CONTROL WORD, PORT C BIT SET/RESET
23 MARK    EQU     0A5H            ;AS FUJINET-LIB'S PORT_INIT LEAVES IT
24 BDOS    EQU     0005H
25 PRINT   EQU     9               ;BDOS PRINT STRING
26 ;
27         ORG     0100H
28 ;
29 START:  IN      PORTB
30         CPI     MARK            ;MARK AND EN HIGH: CARD ALREADY RUNNING
31         JNZ     SETUP
32         IN      PORTC
33         ANI     04H
34         JNZ     ISON
35 SETUP:  MVI     A,0C0H          ;PORT A MODE 2, PC0-PC2 OUTPUTS
36         OUT     PCTRL           ;(CLEARS PORT C, SO EN GOES LOW)
37         MVI     A,06H           ;EN=1, IO0=1 AT ONCE: A RUNNING ESP32
38         OUT     PORTC           ;STAYS UP, ONE IN RESET BOOTS FROM FLASH
39         LXI     B,17000         ;KEEP IO0 HIGH WHILE IT STARTS, ~200 MS
40         CALL    WAIT
41         MVI     A,MARK          ;THE CARD IS SET UP
42         OUT     PORTB
43         IN      PORTA           ;DROP ANY BYTE LATCHED DURING RESET
44         MVI     A,02H           ;PC1=0: ENABLE THE 245
45         OUT     PCTRL
46         MVI     A,00H           ;PC0=0: 245 DRIVES TOWARD THE H89
47         OUT     PCTRL
48         LXI     D,MSGON
49         JMP     DONE
50 ISON:   LXI     D,MSGRUN
51 DONE:   MVI     C,PRINT
52         CALL    BDOS
53         RET
54 ;
55 WAIT:   DCX     B               ;ABOUT 12 US A PASS AT 2 MHZ
56         MOV     A,B
57         ORA     C
58         JNZ     WAIT
59         RET
60 ;
61 MSGON:  DB      'FujiNet enabled',0DH,0AH,'$'
62 MSGRUN: DB      'FujiNet already on',0DH,0AH,'$'
63 ;
64         END     START
```

C WITH Z88DK

GENERAL

C is the easiest way to write a large FujiNet program for the H89. z88dk's C compiler knows both HDOS and CP/M, and the FujiNet project's library, fujinet-lib, wraps every FujiNet command in a C function: `fuji_get_adapter_config()` makes the request FNHELLO built by hand in Chapter 4, and `network_open()` opens a network channel. The three games in this Manual, and the C versions of its other programs, are written this way.

This chapter shows how to build fujinet-lib, how to compile a program for each system, and what to watch for. Two things need care on the H89: z88dk's start-up code for HDOS has two faults the book works around, and HDOS turns a C program's output into capital letters unless the program says otherwise.

WHAT YOU NEED

z88dk, set up as in Chapter 4, and fujinet-lib. No release of fujinet-lib includes the H89 yet, so it is built from its source:

( ) Get fujinet-lib from github.com/FujiNetWIFI/fujinet-lib, on the branch this Manual describes (Chapter 1).

( ) In its directory, type:
`make h89-hdos/r2r h89-cpm/r2r`

( ) The libraries are `r2r/h89-hdos/fujinet.h89-hdos.lib` and `r2r/h89-cpm/fujinet.h89-cpm.lib`; the header files are in `include/`.

NOTE: Use the `/r2r` targets. They build the library and copy it to `r2r/`; the plain `h89-hdos` target leaves a library a program can link with an older copy of, and the program then never reaches the FujiNet.

The library's test programs are built the same way, in its `testing` directory: `make -C testing h89-hdos/r2r h89-cpm/r2r`. They are described at the end of this chapter.

A FIRST PROGRAM

HELLO is FNHELLO again, in C:

SRC/C/HELLO/HELLO.C 30 LINES

```
 1 /*
 2  * HELLO -- first contact, in C. Asks the FujiNet for its adapter
 3  * configuration and prints the network it has joined, its name, its address
 4  * and its firmware version: FNHELLO's job, done with fujinet-lib.
 5  *
 6  *   tools/cc.sh hdos HELLO src/c/hello/hello.c   ->  HELLO.ABS (HDOS)
 7  *   tools/cc.sh cpm  HELLO src/c/hello/hello.c   ->  HELLO.COM (CP/M)
 8  *
 9  * Copyright (C) 2026 Thomas Cherryhomes
10  * SPDX-License-Identifier: GPL-3.0-or-later
11  */
12 #include <stdio.h>
13 #include <fujinet-fuji.h>
14
15 static AdapterConfig cfg;
16
17 int main(void)
18 {
19     printf("HELLO - first contact with the FujiNet\n");
20     if (!fuji_get_adapter_config(&cfg)) {
21         printf("The FujiNet did not answer.\n");
22         return 1;
23     }
24     printf("Network:  %s\n", cfg.ssid);
25     printf("Name:     %s\n", cfg.hostname);
26     printf("Address:  %u.%u.%u.%u\n", cfg.localIP[0], cfg.localIP[1],
27            cfg.localIP[2], cfg.localIP[3]);
28     printf("Firmware: %s\n", cfg.fn_version);
29     return 0;
30 }
```

`fuji_get_adapter_config()` fills in an `AdapterConfig` structure, which has the same layout as the reply (Chapter 8), and returns `true` for an ACK. That is all there is to it. fujinet-lib starts the card the first time

it is called, the way `card_on` does, and makes the whole exchange itself.

COMPILING FOR CP/M

```
zcc +cpm -create-app -D__H89__ -I$FNLIB/include \
    -L$FNLIB/r2r/h89-cpm -lfujinet.h89-cpm.lib \
    -o HELLO hello.c
```

- `+cpm` chooses CP/M.
- `-create-app` makes `HELLO.COM`, ready to run.
- `-D__H89__` tells fujinet-lib's headers it is the H89. Without it they do not know which computer this CP/M runs on, and the program will not compile.
- `-I` and `-L` say where fujinet-lib's headers and library are (`FNLIB` is its directory).
- `-l` names the library. Give it in full, `.lib` and all: the dot in the name stops z88dk adding `.lib` for you, and without it z88dk reports that it cannot find the file.

The program is loaded at 0100H, and z88dk sets the stack at the top of the TPA, under the BDOS.

COMPILING FOR HDOS

```
zcc +hdos -startup=3 -Cz--abs -create-app -D__H89__ \
    -pragma-define:CRT_ORG_CODE=0x2A80 \
    -pragma-define:REGISTER_SP=0x2A80 \
    -pragma-define:CRT_ENABLE_COMMANDLINE=0 \
    -pragma-define:CLIB_DISABLE_FGETS_CURSOR=1 \
    -I$FNLIB/include -L$FNLIB/r2r/h89-hdos -lfujinet.h89-hdos.lib \
    -o HELLO hello.c
```

- `+hdos -startup=3` chooses HDOS, and the start-up that takes its arguments from the command line rather than prompting for them.
- `-Cz--abs -create-app` makes an .ABS file.
- The three `-pragma-define` lines set where the program loads, where its stack is, and turn off z88dk's own command-line handling. They are explained below; use them as they stand.
- `CLIB_DISABLE_FGETS_CURSOR` stops `fgets()` drawing a cursor of its own, which the H19 does not need.

The book's `tools/cc.sh hdos NAME file.c ...` and `tools/cc.sh cpm NAME file.c ...` give exactly these commands.

WHERE THE PROGRAM LOADS

HDOS loads programs at 2280H, which Heath's documents write in split octal as 042.200. Left to itself, z88dk puts an HDOS program at 4480H: someone read 042200 as one octal number, which it is not. The program still runs — HDOS will load a program anywhere above 2280H — but the 8.6K below it is wasted, and z88dk's stack sits there, just under the program.

The book does what FujiNet CONFIG does: it loads the program at 2A80H and puts the stack just under it, at 2A80H, so that the stack has the 2K between 2280H and 2A80H to itself and can never run into the program, its data, or HDOS's device drivers above. A program that wants every byte can load at 2280H instead, and set `REGISTER_SP` to the top of the memory it knows it has.

WARNING: z88dk's `hdos.def` gives HDOS's system values (S_SYSM, S_USRM and the rest) 2000H too high, for the same reason. S.SYSM is at 20D0H, not 40D0H. The Appendix lists the right addresses.

THE COMMAND LINE

z88dk's HDOS start-up code saves the stack pointer HDOS gave it, moves the stack, and then takes the old value back off the **new** stack. So it splits up the command line with the wrong length, and `argv[]` comes out wrong: given `HELLO WORLD`, MEMPROBE's first argument printed as junk.

The cure is to turn z88dk's handling off (`CRT_ENABLE_COMMANDLINE=0`, above) and split the command line yourself, from where HDOS left it. `hdos_args()`, in the book's `src/c/common`, does that. Call it first thing:

```
int main(int argc, char *argv[])
{
    ARGS(argc, argv);       /* HDOS: argc and argv from the command line */
    ...
```

`ARGS` does nothing under CP/M, where z88dk's own handling is correct.

SRC/C/COMMON/HDOSARGS.H 26 LINES

```
 1 /*
 2  * hdosargs.h -- the command line under HDOS.
 3  *
 4  * z88dk's HDOS start-up code (hdos_crt0.asm, startup=3) picks the entry
 5  * stack pointer back up after it has moved the stack, so it parses the
 6  * command line with a wrong length and argv[] comes out damaged. Build with
 7  * -pragma-define:CRT_ENABLE_COMMANDLINE=0 and call hdos_args() first thing
 8  * in main(): it splits the command line HDOS left below 2280H into argv[].
```

```
 * Under CP/M it changes nothing.
 *
 * Copyright (C) 2026 Thomas Cherryhomes
 * SPDX-License-Identifier: GPL-3.0-or-later
 */
#ifndef HDOSARGS_H
#define HDOSARGS_H

#define HDOS_MAXARGS 16

#ifdef __HDOS__
int hdos_args(char ***argvp);
#define ARGS(argc, argv)  ((argc) = hdos_args(&(argv)))
#else
#define ARGS(argc, argv)
#endif

#endif
```

SRC/C/COMMON/HDOSARGS.C 43 LINES

```
/*
 * hdosargs.c -- the command line under HDOS: see hdosargs.h.
 *
 * HDOS starts a program with SP pointing at the first character of its
 * arguments; they run up to a NUL at 227FH. With no arguments SP is 2280H.
 * The start-up code keeps the entry SP inside its own exit code, as the
 * operand of "ld sp,nnnn" at __restore_sp_onexit, so it can be read back
 * from there.
 *
 * Copyright (C) 2026 Thomas Cherryhomes
 * SPDX-License-Identifier: GPL-3.0-or-later
 */
#ifdef __HDOS__
#include "hdosargs.h"

extern unsigned char _restore_sp_onexit[];   /* asm: __restore_sp_onexit */

static char *args_v[HDOS_MAXARGS + 2];
static char args_name[] = "";

int hdos_args(char ***argvp)
{
    char *p = *(char **)(_restore_sp_onexit + 1);   /* the entry SP */
    char *end = (char *)0x227F;
    int n = 0;

    args_v[n++] = args_name;
    while (p < end && n <= HDOS_MAXARGS) {
        while (p < end && *p == ' ')
            p++;
        if (p >= end || *p == 0)
            break;
        args_v[n++] = p;
        while (p < end && *p != ' ' && *p != 0)
            p++;
        if (p < end && *p == ' ')
            *p++ = 0;
    }
    args_v[n] = 0;
    *argvp = args_v;
    return n;
}
#endif
```

WHERE EVERYTHING ENDS UP

MEMPROBE (in `src/c/probe`) prints where its code, its data and its stack are, and what it was given on the command line. Here it is under each system, built as above:

```
S.SYSM   EDB1                    A>B:MEMPROBE HELLO WORLD
S.USRM   3B40                    MEMPROBE
ARGC     3                         main     0460
ARGV[0]  ''                        bss      1EA4
ARGV[1]  'HELLO'                   sp       D7B2
ARGV[2]  'WORLD'                   BDOS     D806
                                   argc     3
                                   argv[0]  ''
                                   argv[1]  'HELLO'
```

Under HDOS, the program starts at 2A80H (`main` is a little after it), its data ends near 3B09H, and the stack is just under 2A80H. S.SYSM shows that HDOS's resident part starts at EDB1H in this 56K machine, so everything from 3B40H up to there is free. Under CP/M the program starts at 0100H, the BDOS at D806H, and the stack is just under it.

THE CONSOLE

z88dk's `printf()` and `puts()` go through the system's ordinary console calls, which is fine for text. But HDOS normally maps lower case to upper case on the way out, so a C program that prints "Hello" shows HELLO, as the first run of HELLO above does.

For anything more than plain text — escape sequences for the H19, graphics characters, a key the moment it is struck — use the book's `h89con.c`. `con_init(0)` stops HDOS's case mapping; `con_init(1)` also turns off echo and CTRL-C handling and gives the program each key as it is struck. `con_getc()` returns a key or −1 at once, and `con_putc()` sends a byte exactly as it is. Under CP/M these go to the BIOS's console routines directly, which do the same. Call `con_exit()` before the program ends, to put the console back.

SRC/C/COMMON/H89CON.H — 34 LINES

```
 1 /*
 2  * h89con.h -- the H19 console, raw, under HDOS or CP/M.
 3  *
 4  * printf() and friends go through the OS's ordinary console calls, which
 5  * is fine for text. A program that sends escape sequences, reads keys as
 6  * they are struck or must not wait for a key uses these instead: HDOS's
 7  * .SCOUT/.SCIN with the console in raw character mode, or the CP/M BIOS's
 8  * CONOUT/CONST/CONIN straight from its jump table.
 9  *
10  * Copyright (C) 2026 Thomas Cherryhomes
11  * SPDX-License-Identifier: GPL-3.0-or-later
12  */
13 #ifndef H89CON_H
14 #define H89CON_H
15
16 #include <stdint.h>
17
18 /* The 2 ms clock both systems keep in low memory. */
19 #ifdef __HDOS__
20 #define TICKS (*(volatile uint16_t *)0x201B)
21 #else
22 #define TICKS (*(volatile uint16_t *)0x000B)
23 #endif
24 #define TICKS_PER_SECOND 500
25
26 /* raw = 0: just stop HDOS turning lower case into upper case.
27    raw = 1: also no echo, no CTRL-C/S/O handling, a character at a time. */
28 void con_init(uint8_t raw);
29 void con_exit(void);                  /* the console as it was */
30 void con_putc(uint8_t c) __z88dk_fastcall;
31 int  con_getc(void);                  /* a key, or -1 if none waits */
32 void con_puts(const char *s);
33
34 #endif
```

SRC/C/COMMON/H89CON.C — 158 LINES

```
 1 /*
 2  * h89con.c -- the H19 console, raw: see h89con.h.
 3  * After Battleship's src/h89/console.c.
 4  *
 5  * Copyright (C) 2026 Thomas Cherryhomes
 6  * SPDX-License-Identifier: GPL-3.0-or-later
 7  */
 8 #include <string.h>
 9 #include "h89con.h"
10
11 #ifdef __HDOS__
12
13 /* HDOS system calls are RST 38H and a function byte. */
14 void con_putc(uint8_t c) __z88dk_fastcall __naked
15 {
16 #asm
17     ld      a,l
18     rst     38h
19     defb    2               ; .SCOUT
20     ret
21 #endasm
22 }
23
24 int con_getc(void) __naked
25 {
26 #asm
27     rst     38h
28     defb    1               ; .SCIN: carry set when nothing waits
29     ld      hl,-1
30     ret     c
31     ld      l,a
32     ld      h,0
33     ret
34 #endasm
35 }
36
37 /* .CONSL: A = which console byte, B = the bits to set, C = the bits to
38    change. The five bytes, S.CSLMD to S.CONFL, sit at 20D6H. */
39 static uint8_t cs_index, cs_bits, cs_mask;
40
41 static void consl(void) __naked
42 {
43 #asm
44     ld      a,(_cs_bits)
45     ld      b,a
46     ld      a,(_cs_mask)
47     ld      c,a
48     ld      a,(_cs_index)
49     rst     38h
50     defb    6               ; .CONSL
51     ret
```

```
#endasm
}

static void consl_set(uint8_t index, uint8_t bits, uint8_t mask)
{
    cs_index = index;
    cs_bits = bits;
    cs_mask = mask;
    consl();
}

#define S_CSLMD ((uint8_t *)0x20D6)
#define I_CSLMD 0               /* mode */
#define I_CONTY 1               /* type */
#define I_CONWI 3               /* width */
#define CSL_ECH 0x80            /* no echo */
#define CSL_RAW 0x04            /* no CTRL-C, -S, -O, -P */
#define CSL_WRP 0x02            /* wrap at the width */
#define CSL_CHR 0x01            /* a character at a time */
#define CTP_MLI 0x20            /* lower to upper case, in */
#define CTP_MLO 0x10            /* lower to upper case, out */
#define CTP_BKM 0x02            /* BACKSPACE becomes RUBOUT, in */

static uint8_t saved[5];

void con_init(uint8_t raw)
{
    memcpy(saved, S_CSLMD, sizeof saved);
    consl_set(I_CONTY, 0, CTP_MLI | CTP_MLO | CTP_BKM);
    if (raw) {
        consl_set(I_CSLMD, CSL_ECH | CSL_RAW | CSL_CHR,
                CSL_ECH | CSL_RAW | CSL_WRP | CSL_CHR);
        consl_set(I_CONWI, 255, 0xFF);   /* escape sequences are not columns */
    }
}

void con_exit(void)
{
    consl_set(I_CSLMD, saved[I_CSLMD], 0xFF);
    consl_set(I_CONTY, saved[I_CONTY], 0xFF);
    consl_set(I_CONWI, saved[I_CONWI], 0xFF);
}

#else /* CP/M */

/* The BIOS jump table: (0001H) points at WBOOT, at its base + 3; CONST is
   3 bytes on from there, CONIN 6, CONOUT 9. The BIOS may use any register,
   so IX and IY are kept for the C code. */
void con_putc(uint8_t c) __z88dk_fastcall __naked
{
#asm
    EXTERN  l_dcal
    push    ix
    push    iy
    ld      c,l
    ld      hl,(1)
    ld      de,9
    add     hl,de
    call    l_dcal          ; CONOUT
    pop     iy
    pop     ix
    ret
#endasm
}

int con_getc(void) __naked
{
#asm
    EXTERN  l_dcal
    push    ix
    push    iy
    ld      hl,(1)
    ld      de,3
    add     hl,de
    call    l_dcal          ; CONST: A = FFH when a key waits
    or      a
    ld      hl,-1
    jr      z,con_getc_none
    ld      hl,(1)
    ld      de,6
    add     hl,de
    call    l_dcal          ; CONIN
    ld      l,a
    ld      h,0
con_getc_none:
    pop     iy
    pop     ix
    ret
#endasm
}

void con_init(uint8_t raw)
{
    (void)raw;              /* the BIOS calls are raw already */
}

void con_exit(void)
{
}

#endif

void con_puts(const char *s)
{
    while (*s)
        con_putc(*s++);
}
```

FILES

Under CP/M, z88dk's `fopen()`, `fread()`, `fwrite()` and `fclose()` work as on any C system, through the BDOS. Under HDOS they work through HDOS's channels, with a few limits: at most a handful of files open at once (z88dk's default is 3, including the console), and file names in HDOS's form, `SY1:NAME.EXT`. NGET and NPUT (Chapter 14) use them under both systems.

A text file written under CP/M ends with CTRL-Z (1AH) in the rest of its last 128-byte record; under HDOS, the rest of its last 256-byte sector is zeros. A program that reads files written on the other system, or fetched from the network, should stop at whichever it finds.

MIXING C AND ASSEMBLY

z88dk lets C call assembly in two ways. Short pieces go inside a C function between `#asm` and `#endasm`; `h89con.c` makes its HDOS system calls that way. Longer ones go in their own `.asm` file with a `PUBLIC` label, assembled by `zcc` along with the C files, and declared in C as an `extern` function.

The compiler fujinet-lib is built with, sccz80, passes arguments on the stack, last argument first, and returns a value in HL. A function marked `__z88dk_fastcall` takes its one argument in HL instead, as `con_putc()` does. Keep IX and IY: the C code uses them. HDOS's system calls leave them alone; a CP/M BIOS may not, which is why `h89con.c` saves them around each BIOS call.

WARNING: In the version of z88dk this Manual used, the remainder of an `unsigned long` by a constant came out wrong in ISS.C (Chapter 15): `t % 60` gave the remainder of `t`'s low 16 bits. Division was right. If a long remainder matters, check it, or do as ISS.C does: subtract the whole parts and take the remainder of what is left in 16 bits.

NOTE: fujinet-lib is built with sccz80. Compiling your own program with z88dk's other compiler, zsdcc (`-compiler=sdcc`), and linking it with fujinet-lib has not been tried.

FUJINET-LIB'S OWN TESTS

fujinet-lib comes with test programs that exercise the library against a running FujiNet. They are a good way to check a new build, and the source of each is a handy example. They are in fujinet-lib's `testing` directory: with z88dk on the PATH, `make h89-hdos h89-cpm` there builds them into `r2r/h89-hdos` and `r2r/h89-cpm`.

They need the FujiNet on the Internet, since they talk to public servers such as `httpbin.dev`, `tcpbin.com` and `tnfs.fujinet.online`. FUJIDEV and DISK change the FujiNet's host and drive slots as they go, and DISK copies a 160K disk image to the SD card, so run them on a FujiNet whose set-up you can put back.

The table that follows shows how they fared on this Manual's set-up, with fujinet-lib v5.0.1 and the tests from its commit 5fadd0d. Each program stops at its first failure, so "4 of 5" means that the fifth test failed and the rest were never run.

Three of the failures are mistakes in the test programs, not in fujinet-lib or the FujiNet. Each was corrected in a copy of the tests, which was then run again ("Corrected" in the table). The numbers are those of the errata list (Appendix).

- **NETWORK** takes the H89's line ending to be LF, but the FujiNet's is CR LF (Chapter 10), so the echo of its line comes back a byte longer (T1).
- **DISK** looks for a file's name 13 bytes into a directory entry, but on this bus the header is 12 bytes (Chapter 8) (T2).
- **FUJIDEV** puts back an empty drive slot with FUJI_SET_DEVICE_FULLPATH, which cannot empty a slot (Chapter 8), and leaves drive slot 0 named `/test_filename` (T3).

The fourth failure is real. FUJI_COPY_FILE took longer than the 15 seconds that fujinet-lib waits for a reply (Chapter 8), and after that every reply was one behind. DISK was run again, corrected and linked with a copy of fujinet-lib in which `TIMEOUT_SLOW` is 60 seconds.

PROGRAM	HDOS	CP/M	NOTES
FUJIDEV	23 of 24	23 of 24	The FujiNet device: adapter, host and drive slots, prefixes, directories, WiFi (fujinet-pc's stand-in, P2), GUIDs, base64, hashes. Stops at T3. Corrected: all 88 under both.
DEVLIST	all 12	all 12	Which devices answer.
APPKEY	all 18	all 18	Application keys.
CLOCK	all 48	all 48	Every time format, and time zones.
DISK	6 of 7	8 of 9	FUJI_COPY_FILE, TNFS to SD. HDOS: the copy outlasted the timeout (L21). CP/M: stops at T2. Corrected, with a 60-second timeout: all 10 under both.
NETWORK	4 of 5	4 of 5	HTTP methods, headers, JSON, TCP echo, several channels. Stops at T1. Corrected: all 82 under both.
NETFILE	all 62	all 62	File system commands, TNFS and FTP.
NET5PLUS	all 49	all 49	ACCEPT, SEEK and TELL, translation, log-in, UDP, channel modes.
HTML	all 14	all 14	The HTML parser.
PARSERR	all 57	all 57	The parsers' error codes.
QRCODE	all 20	all 20	QR codes.

FUJINET-LIB REFERENCE

GENERAL

fujinet-lib is the FujiNet project's C library. It runs on a dozen kinds of computer; on the H89 it is built with z88dk, for HDOS and for CP/M, as Chapter 6 describes. Nearly every function sends one FujiNet command and returns what the reply said, so this chapter is short on explanation: each entry names the command, and the command's card in Chapters 8 to 12 tells the rest.

The entries are in five groups, by header file. Each gives the function's declaration as the H89's build sees it, what it returns, the commands it sends with the page of their cards, and any WARNING for the H89. Many of the names are macros over the bus call rather than functions; that makes no difference to a program, except that a macro's arguments may be used more than once, so don't pass one an expression with side effects.

THE HEADER FILES

INCLUDE	FOR
`fujinet-fuji.h`	The FujiNet device: WiFi, hosts, directories, drives, app keys, base64, hashes. Includes the others it needs.
`fujinet-network.h`	Network channels: open, read, write, status, JSON, HTTP, file systems.
`fujinet-clock.h`	The clock.
`fujinet-qrcode.h`	QR codes.
`fujinet-bus.h`	The bus call itself, and the device numbers.
`fujinet-err.h`	`FN_ERR` codes and the network error numbers.
`fujinet-commands.h`	The command numbers, under fujinet-lib's names (`FUJICMD_...`).

fujinet-lib's command names differ a little from the firmware's, which this Manual uses: the library's `FUJICMD_GET_SSID` is the firmware's `FUJI_GET_SSID`, and `NETCMD_OPEN` is `NET_OPEN`.

WHAT FUNCTIONS RETURN

There are three kinds of answer:

- `bool`: `true` if the FujiNet answered ACK, `false` for a NAK, a garbled reply or no reply at all. Most `fuji_` functions and macros return this.
- `FN_ERR`: 0 (`FN_ERR_OK`) for success, otherwise a reason. Most `network_` and `clock_` functions return this. The reasons are few, because the H89's bus says little: `FN_ERR_IO_ERROR` (1) covers every failed exchange, `FN_ERR_BAD_CMD` (2) bad arguments.
- `int16_t`: a length, or a negative `FN_ERR`. The functions that read data return this: `network_read()`, `network_json_query()`.

For why a network request failed, ask for the channel's STATUS (`network_status()`): its error byte is one of the network error numbers in the Appendix, which `fujinet-err.h` names `NETWORK_ERROR_...`.

WARNING: A `bool` function cannot tell you how much data came back. If the FujiNet sends a shorter reply than the buffer you gave, the function still returns `true`, and the end of your buffer is whatever was there before. Clear buffers first when it matters.

STRUCTURES

The structures have the same layout as the replies, byte for byte, so a reply is read straight into one. On the H89 they are:

```
typedef struct {              /* FUJI_GET_ADAPTERCONFIG: 140 bytes */
    char ssid[33];            /* the network's name */
    char hostname[64];        /* the FujiNet's name */
    uint8_t localIP[4], gateway[4], netmask[4], dnsIP[4];
    uint8_t macAddress[6], bssid[6];
    char fn_version[15];      /* the firmware's version */
} AdapterConfig;

typedef struct {              /* ..._EXTENDED: 240 bytes */
    /* AdapterConfig's fields, then the addresses as text: */
    char sLocalIP[16], sGateway[16], sNetmask[16], sDnsIP[16];
    char sMacAddress[18], sBssid[18];
} AdapterConfigExtended;

typedef uint8_t HostSlot[32];   /* a host's name; 8 of them */

typedef struct {              /* 38 bytes; 8 of them */
    uint8_t hostSlot;         /* which host; FFH: empty */
    uint8_t mode;             /* 01H read, 02H write */
    uint8_t file[36];         /* the image's name */
} DeviceSlot;

typedef struct {              /* 34 bytes */
    char ssid[33];
    signed char rssi;         /* signal strength, dBm */
} SSIDInfo;

typedef struct {              /* 97 bytes */
    char ssid[33];
    char password[64];
} NetConfig;

typedef struct {              /* FUJI_NEW_DISK's payload */
    uint16_t numSectors, sectorSize;
    uint8_t hostSlot, deviceSlot;
    char filename[256];
} NewDisk;

typedef struct {              /* a directory entry with its details */
    FujiDirEntryTimestamp modified;   /* years since 1970, month,
                                         day, hour, minute, second */
    uint32_t size;
    uint8_t flags;            /* 01H a directory, 02H name cut short */
    uint8_t mediatype;
    char filename[256];
} FujiDirEntry;
```

WARNING: On the H89, `FujiDirEntry` has a `flags` byte where other computers have `is_dir` and `is_trunc`, but the macros `FUJI_DIR_ENTRY_IS_DIR()` and `FUJI_DIR_ENTRY_IS_TRUNC()` still use the other names, so a program that uses them will not compile. Test `flags & 1` and `flags & 2` instead. The header the firmware actually sends before the name is 12 bytes long (Chapter 8).

THE BUS CALL

Under all of it is one function:

```
bool fuji_bus_call(uint8_t device, uint8_t fuji_cmd, uint8_t fields,
                   uint8_t aux1, uint8_t aux2, uint8_t aux3, uint8_t aux4,
                   const void *buf, size_t buf_length);
```

It sends one request and takes its reply, exactly as the book's `fn_call` does (Chapter 4). `fields` is the descriptor, `FUJI_FIELD_NONE` to `FUJI_FIELD_C1234` (0 to 7), plus one of two flags of fujinet-lib's own:

- `FUJI_FIELD_DATA` (8): `buf` and `buf_length` are a payload to send.
- `FUJI_FIELD_REPLY` (16): `buf` and `buf_length` are where the reply's data goes.

Without either, `buf` is sent as a payload if `buf_length` is not 0. With `FUJI_FIELD_REPLY`, the call succeeds only if the reply is ACK from the right device with a good checksum; it does not check that the data filled the buffer.

A call can send a payload **or** take a reply's data, not both. No FujiNet command on the H89 needs both.

When fujinet-lib has no function for something — the disk drives, for one — call `fuji_bus_call()` yourself:

EXDISK.C — A DISK SECTOR, BY THE BUS CALL

```
/* fujinet-lib has no disk calls: fuji_bus_call sends the frame */
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_READ,
                   FUJI_FIELD_C1234 | FUJI_FIELD_REPLY,
                   3, 0, 0, 0, sector, 512);    /* sector 3 */
if (ok) {
    for (i = 0; i < 512 && sector[i] == 3; i++)
        ;                           /* sector n: 512 bytes of n */
    if (i == 512)
        printf("\nSector 3 is 512 bytes of 03H.");
}
```

THE SHORTHAND MACROS

fujinet-lib writes most of its functions as one-line macros over the bus call, named for what they send: `FUJICALL_A1_RV(cmd, a1, reply, len)` sends `cmd` to the FujiNet device with one byte field and takes a reply of `len` bytes. `NETCALL_...` does the same for a network channel (with the unit as an extra first argument), and `CLKCALL_...` for the clock. The suffixes are:

SUFFIX	MEANS
`A1` … `A4`	one to four byte fields
`B12`	a word field (aux1 and aux2)
`B12_B34`	two word fields
`C1234`	a 32-bit field
`D`	a payload follows: data and its length
`RV`	a reply comes back: a buffer and its length

They are handy in your own programs too: `FUJICALL_A1_RV(FUJICMD_GET_HOST_PREFIX, 0, buf, 256)` is the whole of `fuji_get_host_prefix(0, buf)`.

FUNCTIONS THAT WILL NOT LINK

These are declared in fujinet-lib's headers, but not built for the H89. A program that calls one compiles, and then fails to link with "undefined symbol":

fuji_enable_udpstream
```
bool fuji_enable_udpstream(uint16_t port, char *host);
```

fuji_get_device_enabled_status
```
bool fuji_get_device_enabled_status(uint8_t d);
```

network_get_dstats
```
FN_ERR network_get_dstats(const char *devicespec, uint8_t cmd, uint8_t *dstats);
```

network_has_proceed
```
bool network_has_proceed(void);
```

network_proceed
```
bool network_proceed(void);
```

Also declared and not built: `fn_error()`, and the variable `_fuji_status`.

THE FUJINET DEVICE

fuji_base64_decode_compute MACRO · FUJINET-FUJI.H
```
#define fuji_base64_decode_compute(void)
```
Decodes the Base64 text sent with fuji_base64_decode_input().
RETURNS true on ACK
SENDS FUJI_BASE64_DECODE_COMPUTE (page 8-16)

WARNING: Encoding and decoding share one buffer in the FujiNet and nothing clears it: read all of one result before you send the next input, or what is left is put in front of it.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_base64_decode_compute();
```

fuji_base64_decode_input MACRO · FUJINET-FUJI.H
```
#define fuji_base64_decode_input(s,len)
```
Sends len bytes of Base64 text to be decoded. Call it as often as you need to; each call adds to what is there.
RETURNS true on ACK
SENDS FUJI_BASE64_DECODE_INPUT (page 8-16)

WARNING: Encoding and decoding share one buffer in the FujiNet and nothing clears it: read all of one result before you send the next input, or what is left is put in front of it.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_base64_decode_input(b64, (uint16_t)b64len);   /* back */
```

fuji_base64_decode_length MACRO · FUJINET-FUJI.H
```
#define fuji_base64_decode_length(len)
```
Gets the length of the decoded result into an unsigned long.
RETURNS true on ACK
SENDS FUJI_BASE64_DECODE_LENGTH (page 8-16)

WARNING: len points to an unsigned long (4 bytes), not an int.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_base64_decode_length(&b64len);
if (ok)
    printf("%lu\n", b64len);
```

fuji_base64_decode_output MACRO · FUJINET-FUJI.H
```
#define fuji_base64_decode_output(s,len)
```
Reads the next len bytes of the decoded result into s. The FujiNet forgets the bytes it has sent.
RETURNS true on ACK; false if len is 0 or more than is left
SENDS FUJI_BASE64_DECODE_OUTPUT (page 8-16)

WARNING: No NUL is added: put one on yourself if the result is text.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_base64_decode_output(b64, (uint16_t)b64len);
if (ok) {
    b64[(uint16_t)b64len] = 0;
    printf("%s\n", b64);
}
```

fuji_base64_encode_compute MACRO · FUJINET-FUJI.H
```
#define fuji_base64_encode_compute(void)
```
Encodes the bytes sent with fuji_base64_encode_input().
RETURNS true on ACK
SENDS FUJI_BASE64_ENCODE_COMPUTE (page 8-15)

WARNING: The text has a newline after every 72 characters and one at the end, and fuji_base64_encode_length() counts them. Encoding and decoding share one buffer in the FujiNet and nothing clears it: read all of one result before you send the next input, or what is left is put in front of it.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_base64_encode_compute();
```

fuji_base64_encode_input MACRO · FUJINET-FUJI.H
```
#define fuji_base64_encode_input(s,len)
```
Sends len bytes to be encoded. Call it as often as you need to; each call adds to what is there.
RETURNS true on ACK
SENDS FUJI_BASE64_ENCODE_INPUT (page 8-15)

WARNING: Encoding and decoding share one buffer in the FujiNet and nothing clears it: read all of one result before you send the next input, or what is left is put in front of it.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_base64_encode_input("Heathkit H89", 12);
```

---

fuji_base64_encode_length MACRO · FUJINET-FUJI.H

```
#define fuji_base64_encode_length(len)
```

Gets the length of the encoded text, newlines included, into an unsigned long.

RETURNS true on ACK

SENDS FUJI_BASE64_ENCODE_LENGTH (page 8-15)

WARNING: len points to an unsigned long (4 bytes), not an int.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_base64_encode_length(&b64len);    /* an unsigned long */
if (ok)
    printf("%lu\n", b64len);
```

---

fuji_base64_encode_output MACRO · FUJINET-FUJI.H

```
#define fuji_base64_encode_output(s,len)
```

Reads the next len bytes of the encoded text into s. The FujiNet forgets the bytes it has sent.

RETURNS true on ACK; false if len is 0 or more than is left

SENDS FUJI_BASE64_ENCODE_OUTPUT (page 8-15)

WARNING: No NUL is added, and the text ends with a newline.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_base64_encode_output(b64, (uint16_t)b64len);  /* all */
if (ok) {
    b64[(uint16_t)b64len] = 0;      /* it ends in a newline */
    printf("%s", b64);
}
```

---

fuji_close_directory FUNCTION · FUJINET-FUJI.H

```
bool fuji_close_directory(void);
```

Closes the directory that fuji_open_directory() opened. The FujiNet keeps one directory open at a time, for all the host slots together.

RETURNS true on ACK

SENDS FUJI_CLOSE_DIRECTORY (page 8-8)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_close_directory();
```

---

fuji_copy_file MACRO · FUJINET-FUJI.H

```
#define fuji_copy_file(src_slot,dest_slot,copy_spec)
```

Copies a file from one host slot to another, or within one: copy_spec is "source|destination". The FujiNet does the copying; the H89 waits for the ACK.

RETURNS true on ACK (see fuji_copy_file_common() for what happens on failure)

SENDS FUJI_COPY_FILE (page 8-11)

WARNING: Host slots count from 1 here (1-8), not 0. All 256 bytes from copy_spec are sent: build it in a cleared 256-byte buffer. It fails whenever that host has a prefix longer than "/". A destination ending in '/' (meaning "the same name") is never seen as one, because of the padding. A large copy can take longer than the 15 seconds fuji_bus_call() waits: it then returns false while the copy goes on, and the late ACK puts every later reply one behind (Chapter 8).

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* host slots count from 1 here. fujinet-lib sends 256 bytes:
   keep the spec in a cleared 256-byte buffer */
memset(path, 0, sizeof path);
strcpy(path, "/test/readme.txt|/test/copy.txt");
ok = fuji_copy_file(1, 1, path);
```

---

fuji_copy_file_common FUNCTION · FUJINET-FUJI.H

```
bool fuji_copy_file_common(uint8_t src_slot, uint8_t dest_slot, const char *copy_spec);
```

What the fuji_copy_file() macro calls. If COPY_FILE fails it sends FUJI_STATUS until the FujiNet answers, up to 100 times; if the FujiNet was only busy, it then checks with fuji_file_stat() that both files are there and the same size.

RETURNS true if COPY_FILE was ACKed, or the sizes match afterwards

SENDS FUJI_COPY_FILE (page 8-11)

WARNING: Slots count from 1, as for fuji_copy_file(). The size check scans both directories, closing any directory you had open.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* host slots count from 1 here. fujinet-lib sends 256 bytes:
   keep the spec in a cleared 256-byte buffer */
memset(path, 0, sizeof path);
strcpy(path, "/test/readme.txt|/test/copy.txt");
ok = fuji_copy_file(1, 1, path);
```

---

fuji_create_new MACRO · FUJINET-FUJI.H

```
#define fuji_create_new(new_disk)
```

Asks the FujiNet to make a new, empty disk image from a NewDisk block: number of sectors, sector size, host slot, drive slot and path.

RETURNS false on the H89: it always NAKs

SENDS FUJI_NEW_DISK (page 8-11)

WARNING: Always fails on this firmware (making an image is not supported for the H89's disks), and leaves a 0-byte file behind.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
nd.numSectors = 400;              /* of 256 bytes */
nd.sectorSize = 256;
nd.hostSlot = 0;
nd.deviceSlot = 1;
strcpy(nd.filename, "/test/new.h8d");
ok = fuji_create_new(&nd);        /* this firmware always NAKs */
```

---

fuji_disable_device MACRO · FUJINET-FUJI.H

```
#define fuji_disable_device(d)
```

Asks the FujiNet to switch device d off.

RETURNS false on the H89: it always NAKs

SENDS FUJI_DISABLE_DEVICE (page --)

WARNING: This firmware has no DISABLE_DEVICE on 70H, so it always NAKs.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
ok = fuji_disable_device(FUJI_DEVICEID_PRINTER); /* NAK as well */
```

---

fuji_enable_device MACRO · FUJINET-FUJI.H

```
#define fuji_enable_device(d)
```

Asks the FujiNet to switch device d on.

RETURNS false on the H89: it always NAKs

SENDS FUJI_ENABLE_DEVICE (page --)

WARNING: This firmware has no ENABLE_DEVICE on 70H, so it always NAKs.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
/* the H89's firmware has no ENABLE_DEVICE: a NAK, so false */
ok = fuji_enable_device(FUJI_DEVICEID_PRINTER);
```

---

fuji_enable_udpstream FUNCTION · FUJINET-FUJI.H

```
bool fuji_enable_udpstream(uint16_t port, char *host);
```

Declared for streaming a serial port over UDP (an Atari feature).

RETURNS does not link
LINKS **no**: not built for the H89

WARNING: Does not link on the H89, under HDOS or CP/M: fujinet-lib declares it but has no H89 version. The firmware would NAK ENABLE_UDPSTREAM anyway.

---

fuji_file_stat FUNCTION · FUJINET-FUJI.H

```
FujiDirEntry * fuji_file_stat(uint8_t host, const char *path);
```

Looks a file up by path on host slot host (0-7) and gives its directory entry: date, size, flags and media type. It mounts the host, opens the file's directory, reads entries until the name matches, and closes the directory.

RETURNS a pointer to a FujiDirEntry, or NULL if the file is not there
SENDS FUJI_READ_DIR_ENTRY (page 8-7)

WARNING: The entry is the library's own static copy, overwritten by the next call. It closes any directory you had open. It finds files only (a directory's name comes back with '/' on the end). The year counts from 1970. fujinet-pc only: a file in a subdirectory of its SD card shows size 0 and a 1969 date (it stats the wrong path); one at the top is right.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
/* MOUNT_HOST, OPEN_DIRECTORY, READ_DIR_ENTRY until the name
   matches, CLOSE_DIRECTORY; NULL if the file is not there */
fe = fuji_file_stat(0, "/h89lib.txt");
if (fe)
    printf("%lu bytes, %u/%u/%u\n", fe->size, fe->modified.month,
           fe->modified.day, fe->modified.year + 1970);
```

---

fuji_generate_guid MACRO · FUJINET-FUJI.H

```
#define fuji_generate_guid(guid)
```

Gets a random GUID, 36 characters and a NUL, into a 37-byte buffer.

RETURNS true on ACK
SENDS FUJI_GENERATE_GUID (page 8-20)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_generate_guid(guid);      /* 36 characters and a NUL */
if (ok)
    printf("%s\n", guid);
```

---

fuji_get_adapter_config MACRO · FUJINET-FUJI.H

```
#define fuji_get_adapter_config(ac)
```

Gets the FujiNet's network settings as bytes: SSID, host name, IP address, gateway, netmask, DNS server, MAC address, BSSID and the firmware version.

RETURNS true on ACK
SENDS FUJI_GET_ADAPTERCONFIG (page 8-2)

WARNING: The version string is cut to 14 characters.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_adapter_config(&cfg);
if (ok)
    printf("%s %u.%u.%u.%u %s\n", cfg.ssid, cfg.localIP[0],
           cfg.localIP[1], cfg.localIP[2], cfg.localIP[3],
           cfg.fn_version);
```

---

fuji_get_adapter_config_extended MACRO · FUJINET-FUJI.H

```
#define fuji_get_adapter_config_extended(ac)
```

As fuji_get_adapter_config(), with the addresses as text too (sLocalIP, sGateway, sNetmask, sDnsIP, sMacAddress, sBssid).

RETURNS true on ACK
SENDS FUJI_GET_ADAPTERCONFIG_EXTENDED (page 8-2)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_adapter_config_extended(&cfgx);
if (ok)                             /* the numbers as text too */
    printf("%s %s\n", cfgx.sLocalIP, cfgx.sMacAddress);
```

---

fuji_get_device_enabled_status FUNCTION · FUJINET-FUJI.H

```
bool fuji_get_device_enabled_status(uint8_t d);
```

Declared to ask whether a device is enabled; the header itself calls it bogus.

RETURNS does not link
LINKS **no**: not built for the H89

WARNING: Does not link on the H89, under HDOS or CP/M: fujinet-lib declares it but has no H89 version. The firmware would NAK DEVICE_ENABLE_STATUS anyway.

---

fuji_get_device_filename MACRO · FUJINET-FUJI.H

```
#define fuji_get_device_filename(ds,buffer)
```

Gets the full path of the image in drive slot ds (0-7).

RETURNS true on ACK
SENDS FUJI_GET_DEVICE_FULLPATH (page 8-10)

WARNING: The buffer must hold 256 bytes.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_device_filename(0, path);     /* drive slot 0 */
if (ok)
    printf("%s\n", path);
```

---

fuji_get_device_slots MACRO · FUJINET-FUJI.H

```
#define fuji_get_device_slots(d,count)
```

Gets all eight drive slots: the host slot, mode and file name (36 characters) of each. Host slot FFH means the drive slot is empty.

RETURNS true on ACK
SENDS FUJI_READ_DEVICE_SLOTS (page 8-9)

WARNING: count must be 8: the FujiNet always sends all eight (304 bytes), and a smaller count makes the call fail with nothing copied. The name is not padded with NULs: the tail of the full path can follow the NUL.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_device_slots(slots, 8);   /* all eight, or it fails */
if (ok)                     /* host FFH: empty; mode 40H: mounted */
    printf("%u %02X %s\n", slots[0].hostSlot, slots[0].mode,
           (char *)slots[0].file);
```

---

fuji_get_directory_position MACRO · FUJINET-FUJI.H

```
#define fuji_get_directory_position(pos)
```

Gets the number of the entry fuji_read_directory() will give next.

RETURNS true on ACK
SENDS FUJI_GET_DIRECTORY_POSITION (page 8-8)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_directory_position(&pos);
if (ok)                             /* the next entry's number */
    printf("%u\n", pos);
```

fuji_get_host_prefix MACRO · FUJINET-FUJI.H
```
#define fuji_get_host_prefix(hs,prefix)
```
Gets host slot hs's prefix, the directory every path on that host starts from.
RETURNS true on ACK
SENDS FUJI_GET_HOST_PREFIX (page 8-6)

WARNING: 256 bytes come back; after the NUL they are whatever was in the FujiNet's memory.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_get_host_prefix(0, prefix);   /* 256 bytes come back */
if (ok)
    printf("%s\n", prefix);
```

fuji_get_host_slots MACRO · FUJINET-FUJI.H
```
#define fuji_get_host_slots(d,count)
```
Gets the names of all eight host slots, 32 bytes each; an empty slot is an empty string.
RETURNS true on ACK
SENDS FUJI_READ_HOST_SLOTS (page 8-5)

WARNING: count must be 8: the FujiNet always sends all 256 bytes, and a smaller count makes the call fail with nothing copied.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_get_host_slots(hosts, 8);     /* all eight, or it fails */
if (ok)
    for (i = 0; i < 8; i++)
        printf("%s ", (char *)hosts[i]);
```

fuji_get_scan_result MACRO · FUJINET-FUJI.H
```
#define fuji_get_scan_result(n,ssid_info)
```
Gets network n of the last scan: its SSID and signal strength (rssi, in dBm).
RETURNS true on ACK
SENDS FUJI_GET_SCAN_RESULT (page 8-3)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_get_scan_result(0, &scan);    /* the first found */
if (ok)                             /* (and scan.rssi, in dBm) */
    printf("%s\n", scan.ssid);
```

fuji_get_ssid MACRO · FUJINET-FUJI.H
```
#define fuji_get_ssid(net_config)
```
Gets the saved WiFi network name and passphrase.
RETURNS true on ACK
SENDS FUJI_GET_SSID (page 8-4)

WARNING: The passphrase comes back in plain text.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_get_ssid(&net);           /* and net.password, in */
if (ok)                             /* the clear */
    printf("%s\n", net.ssid);
```

fuji_get_time MACRO · FUJINET-FUJI.H
```
#define fuji_get_time(buffer)
```
Asks the Fuji device (70H) for the time.
RETURNS false on the H89: it always NAKs
SENDS FUJI_GET_TIME (page --)

WARNING: Always NAKs: the time lives on the clock device, 45H. Use clock_get_time().

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_get_time(buf);    /* NAK: the clock is device 45H */
```

fuji_get_wifi_enabled FUNCTION · FUJINET-FUJI.H
```
bool fuji_get_wifi_enabled(void);
```
Asks whether WiFi is switched on in the FujiNet's configuration.
RETURNS true if WiFi is enabled
SENDS FUJI_GET_WIFI_ENABLED (page 8-3)

WARNING: It does not look at whether the call worked: when the frame fails it gives the answer from the last call that did work (false if none has).

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
/* true if it is switched on; a call that fails returns the
   last answer, not false */
ok = fuji_get_wifi_enabled();
printf("WiFi %s\n", ok ? "on" : "off");
```

fuji_get_wifi_status MACRO · FUJINET-FUJI.H
```
#define fuji_get_wifi_status(status)
```
Gets the WiFi status byte: 3 connected, 6 not connected.
RETURNS true on ACK
SENDS FUJI_GET_WIFISTATUS (page 8-2)

WARNING: fujinet-lib's WifiStatus enum has no name for 6.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_get_wifi_status(&status);
if (ok)                             /* 3 = connected, 6 = not */
    printf("%u\n", status);
```

fuji_hash_add MACRO · FUJINET-FUJI.H
```
#define fuji_hash_add(data,length)
```
Adds length bytes to the data to be hashed (it is fuji_hash_input()). Call it as often as you like, then fuji_hash_calculate().
RETURNS true on ACK
SENDS FUJI_HASH_INPUT (page 8-17)

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C
```
fuji_hash_clear();                  /* start with nothing */
fuji_hash_add((uint8_t *)"ab", 2);  /* "abc", in two pieces */
ok = fuji_hash_add((uint8_t *)"c", 1);
```

fuji_hash_calculate FUNCTION · FUJINET-FUJI.H
```
bool fuji_hash_calculate(hash_alg_t hash_type, bool as_hex, bool discard_data, uint8_t
*output);
```
Computes a digest (MD5, SHA1, SHA256 or SHA512) of everything added since the last clear and reads it into output, as hex text or bytes. With discard_data false the data stays, for another kind of digest; with true it is cleared.
RETURNS true if the compute and the read worked

WARNING: Hex output has no NUL: make output fuji_hash_size() + 1 bytes and put one on.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C
```
/* SHA-1 of what was added, keeping it; then MD5, letting go */
if (fuji_hash_calculate(SHA1, true, false, digest)) {
    digest[40] = 0;
    printf("%s\n", (char *)digest);
}
ok = fuji_hash_calculate(MD5, true, true, digest);
if (ok) {
    digest[32] = 0;
    printf("%s\n", (char *)digest);
}
```

fuji_hash_clear MACRO · FUJINET-FUJI.H
```
#define fuji_hash_clear()
```
Throws away any data added for hashing.
RETURNS true on ACK

SENDS FUJI_HASH_CLEAR (page 8-18)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_clear();            /* start again */
```

---

fuji_hash_compute MACRO · FUJINET-FUJI.H

```
#define fuji_hash_compute(type)
```

Computes the digest of the added data with algorithm type (MD5 0, SHA1 1, SHA256 2, SHA512 3), then clears the data. The digest stays for fuji_hash_length() and fuji_hash_output().

RETURNS true on ACK

SENDS FUJI_HASH_COMPUTE (page 8-17)

WARNING: An unknown algorithm still ACKs, and leaves an empty digest. The header calls this a low-level call; fuji_hash_calculate() does it for you.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_compute(SHA1);      /* then let go */
```

---

fuji_hash_compute_no_clear MACRO · FUJINET-FUJI.H

```
#define fuji_hash_compute_no_clear(type)
```

As fuji_hash_compute(), but the data stays, so you can compute another kind of digest of it.

RETURNS true on ACK

SENDS FUJI_HASH_COMPUTE_NO_CLEAR (page 8-17)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_compute_no_clear(MD5);   /* "abc" is kept */
```

---

fuji_hash_data FUNCTION · FUJINET-FUJI.H

```
bool fuji_hash_data(hash_alg_t hash_type, uint8_t *input, uint16_t length, bool as_hex,
uint8_t *output);
```

Hashes one buffer in one call: clears, adds the input, computes, and reads the digest into output as hex text or as bytes. Five frames.

RETURNS true if all of them worked

WARNING: Hex output has no NUL: make output fuji_hash_size() + 1 bytes and put one on. Anything you added with fuji_hash_add() first is thrown away.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
/* CLEAR, INPUT, COMPUTE, LENGTH, OUTPUT: five frames */
ok = fuji_hash_data(MD5, (uint8_t *)"abc", 3, true, digest);
if (ok) {
    digest[fuji_hash_size(MD5, true)] = 0;  /* hex has no NUL */
    printf("%s\n", (char *)digest);
}
```

---

fuji_hash_input MACRO · FUJINET-FUJI.H

```
#define fuji_hash_input(data,len)
```

Sends len bytes to be hashed; the same as fuji_hash_add().

RETURNS true on ACK

SENDS FUJI_HASH_INPUT (page 8-17)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_input("abc", 3);    /* or fuji_hash_add() */
```

---

fuji_hash_length MACRO · FUJINET-FUJI.H

```
#define fuji_hash_length(mode,reply,len)
```

Gets the length of the last digest into reply: mode 1 counts hex characters, 0 counts bytes.

RETURNS true on ACK

SENDS FUJI_HASH_LENGTH (page 8-17)

WARNING: The answer is one byte (pass len 1), not an unsigned long like the Base64 and QR lengths. The header calls it deprecated; fuji_hash_size() works it out without asking.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_length(1, &hashlen, 1);  /* 1: as hex; one byte */
if (ok)
    printf("%u\n", hashlen);
```

---

fuji_hash_output MACRO · FUJINET-FUJI.H

```
#define fuji_hash_output(type,data,len)
```

Reads the last digest into data: type 1 as lower-case hex (no NUL), 0 as bytes.

RETURNS true on ACK

SENDS FUJI_HASH_OUTPUT (page 8-17)

WARNING: The FujiNet sends the whole digest whatever len says: a len smaller than the digest makes the call fail.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_output(1, digest, hashlen);  /* hex, no NUL */
if (ok) {
    digest[hashlen] = 0;
    printf("%s\n", (char *)digest);
}
```

---

fuji_mount_all MACRO · FUJINET-FUJI.H

```
#define fuji_mount_all()
```

Mounts every drive slot that names an image, mounting their hosts as it goes.

RETURNS true on ACK

SENDS FUJI_MOUNT_ALL (page 8-11)

WARNING: Hosts mounted this way are not marked as mounted, so fuji_unmount_host_slot() on them NAKs afterwards.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_mount_all();             /* whatever the slots name */
```

---

fuji_mount_disk_image MACRO · FUJINET-FUJI.H

```
#define fuji_mount_disk_image(ds,mode)
```

Mounts the image set for drive slot ds (with fuji_set_device_filename()) in mode 1 (read) or 2 (read and write). The host must be mounted already.

RETURNS true on ACK

SENDS FUJI_MOUNT_IMAGE (page 8-10)

WARNING: A file whose name ends in ROM, BIN, INT, ITV, CHF, NES, SMS, A78, A52 or ST2 is taken for a cartridge: the FujiNet sends it to the computer in frames of its own, which the H89 does not expect. Do not mount those. The mode shown by fuji_get_device_slots() is the one set with the path, not this one.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_mount_disk_image(0, 1);  /* drive slot 0, read-only */
```

---

fuji_mount_host_slot MACRO · FUJINET-FUJI.H

```
#define fuji_mount_host_slot(hs)
```

Mounts host slot hs (0-7), the SD card or a TNFS server, ready for directories and images.

RETURNS true on ACK

SENDS FUJI_MOUNT_HOST (page 8-5)

WARNING: Once a host is mounted it is never mounted again, even if the server has gone away since.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_mount_host_slot(0);       /* host slot 0, the SD card */
```

---

fuji_open_directory MACRO · FUJINET-FUJI.H
```
#define fuji_open_directory(hs,path_filter)
```
Opens a directory on host slot hs for fuji_read_directory(). path_filter is a 256-byte buffer: the path, a NUL, then a wildcard pattern and a NUL if you want one.

RETURNS true on ACK; false if the host is not mounted or the path is not there
SENDS FUJI_OPEN_DIRECTORY (page 8-7)

WARNING: All 256 bytes are always sent, so clear the buffer first: with no pattern, whatever follows the path's NUL becomes the pattern. Mount the host first (fuji_mount_host_slot()). Opening a directory closes any other.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
/* the path, a NUL, then a pattern if wanted: fujinet-lib sends
   all 256 bytes of the buffer, so clear it first */
memset(path, 0, sizeof path);
strcpy(path, "/test/");
ok = fuji_open_directory(0, path);  /* on host slot 0 */
```

---

fuji_open_directory2 MACRO · FUJINET-FUJI.H
```
#define fuji_open_directory2(hs,path,filter)
```
Another name for fuji_open_directory_filter().

RETURNS true on ACK

WARNING: As fuji_open_directory_filter(): with an empty pattern the 256 bytes go from your path, so keep it in a cleared 256-byte buffer.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C
```
/* no pattern: then the 256 bytes go from path itself */
memset(path, 0, sizeof path);
strcpy(path, "/test/");
ok = fuji_open_directory2(0, path, "");
for (n = 0; ok && fuji_read_directory(32, 0, name)
          && name[0] != FUJI_DIR_EOF; n++)
    ;
printf("%u entries\n", n);              /* 3 */
fuji_close_directory();
```

---

fuji_open_directory_filter FUNCTION · FUJINET-FUJI.H
```
bool fuji_open_directory_filter(uint8_t hostSlot, const char *path, const char *filter);
```
Opens a directory with the path and the pattern as two strings; the library joins them (path, NUL, pattern) in a 256-byte buffer of its own and calls fuji_open_directory().

RETURNS true on ACK
SENDS FUJI_OPEN_DIRECTORY (page 8-7)

WARNING: With an empty pattern it sends 256 bytes straight from your path, so whatever follows its NUL in memory becomes the pattern: keep the path in a cleared 256-byte buffer. A NULL pattern, which the header allows, reads address 0 on the H89 and sends what it finds there as the pattern: pass "".

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C
```
fuji_mount_host_slot(0);            /* the SD card, first */
ok = fuji_open_directory_filter(0, "/test/", "*.txt");
while (ok && fuji_read_directory(32, 0, name)
       && name[0] != FUJI_DIR_EOF)
    printf("%s ", name);            /* readme.txt */
fuji_close_directory();
```

---

fuji_put_device_slots MACRO · FUJINET-FUJI.H
```
#define fuji_put_device_slots(d,count)
```
Writes all eight drive slots back: the host slot and mode of each. The names are not used; set a path with fuji_set_device_filename().

RETURNS true on ACK
SENDS FUJI_WRITE_DEVICE_SLOTS (page 8-9)

WARNING: count must be 8: the FujiNet does not clear the rest, so fewer slots leave garbage in the others. A host slot of 8 or more is not checked, and makes fuji_mount_all() read past the end of the FujiNet's host table.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
slots[0].mode = 2;                  /* slot 0 read/write from now */
ok = fuji_put_device_slots(slots, 8);   /* (names are not used) */
```

---

fuji_put_host_slots MACRO · FUJINET-FUJI.H
```
#define fuji_put_host_slots(d,count)
```
Writes all eight host slot names back and saves them in the FujiNet's configuration. A slot whose name changed is unmounted.

RETURNS true on ACK
SENDS FUJI_WRITE_HOST_SLOTS (page 8-5)

WARNING: count must be 8: the FujiNet does not clear the rest, so fewer slots leave garbage in the others.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
strcpy((char *)hosts[7], "localhost");  /* a name in slot 8 */
ok = fuji_put_host_slots(hosts, 8);     /* and all eight go back */
```

---

fuji_read_directory FUNCTION · FUJINET-FUJI.H
```
bool fuji_read_directory(uint8_t maxlen, uint8_t aux2, void *buffer);
```
Reads the next entry of the open directory into buffer, padded with zeros to maxlen bytes. With aux2 = FUJI_DIR_FLAG_ADDITIONAL_DATA (80H) a 12-byte header (date, size, flags, media type: a FujiDirEntry) comes before the name. Directories end in '/'. At the end the first byte is FUJI_DIR_EOF (7FH).

RETURNS true on ACK; false if no directory is open
SENDS FUJI_READ_DIR_ENTRY (page 8-7)

WARNING: FUJI_DIR_ENTRY_IS_DIR() and FUJI_DIR_ENTRY_IS_TRUNC() do not compile for the H89, whose FujiDirEntry has flags instead of is_dir and is_trunc: test flags & 1 (a directory) and flags & 2 (name cut short). With 80H keep maxlen at 14 or more. Block mode (aux2 C0H + n) does not work through this call, and C0H alone crashes the firmware.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
for (;;) {
    ok = fuji_read_directory(32, 0, name);  /* padded to 32 */
    if (!ok || name[0] == FUJI_DIR_EOF)     /* 7FH: no more */
        break;
    printf("%s ", name);
}
```

---

fuji_reset MACRO · FUJINET-FUJI.H
```
#define fuji_reset()
```
Restarts the FujiNet.

RETURNS false, after 15 seconds: no answer ever comes
SENDS FUJI_RESET (page 8-21)

WARNING: The FujiNet restarts without answering, so the call waits out the whole 15-second timeout and gives false. Mounted disks are not unmounted first.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_reset();          /* no reply: false, after 15 seconds */
```

---

fuji_scan_for_networks MACRO · FUJINET-FUJI.H

```
#define fuji_scan_for_networks(count)
```

Scans for WiFi networks and gets how many it found, in one byte.

RETURNS true on ACK
SENDS FUJI_SCAN_NETWORKS (page 8-3)

WARNING: On the H89 card (an ESP32) the scan blocks for a while and drops the WiFi connection until it is over.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_scan_for_networks(&count);
if (ok)
    printf("%u networks\n", count);
```

---

fuji_set_boot_config MACRO · FUJINET-FUJI.H

```
#define fuji_set_boot_config(toggle)
```

Sends CONFIG_BOOT with toggle; 0 takes a boot disk that fuji_set_boot_mode() put in out again, if drive slot 0 is empty.

RETURNS true on ACK (always)
SENDS FUJI_CONFIG_BOOT (page 8-12)

WARNING: Despite its name it changes no saved setting on this firmware.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_set_boot_config(0);       /* and out again */
```

---

fuji_set_boot_mode MACRO · FUJINET-FUJI.H

```
#define fuji_set_boot_mode(mode)
```

Sets the boot mode; mode 0 puts the FujiNet's boot image on the first drive (31H), read-only.

RETURNS true on ACK (always)
SENDS FUJI_SET_BOOT_MODE (page 8-12)

WARNING: Always ACKs, even when the mode or its image does not exist. The image mode 0 mounts is an MS-DOS boot disk (the RS-232 build's autorun.img), not something an H89 boots.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_set_boot_mode(0);         /* autorun.img into drive 31H */
```

---

fuji_set_device_filename MACRO · FUJINET-FUJI.H

```
#define fuji_set_device_filename(mode,hs,ds,buffer)
```

Sets the image path, host slot and mode (1 read, 2 read and write) of drive slot ds. Mount it afterwards with fuji_mount_disk_image().

RETURNS true on ACK
SENDS FUJI_SET_DEVICE_FULLPATH (page 8-9)

WARNING: The arguments are (mode, hs, ds, buffer), but they go on the wire as drive, host, mode. All 256 bytes from buffer are sent: keep the path in a cleared 256-byte buffer. An empty path does not empty the slot (the old path stays), and host slot 0xFF is refused: use fuji_unmount_disk_image().

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
memset(path, 0, sizeof path);       /* 256 bytes go */
strcpy(path, "/test/disk.img");
/* mode 1 (read), host slot 0, drive slot 0: not the order
   they go on the wire (drive, host, mode) */
ok = fuji_set_device_filename(1, 0, 0, path);
```

---

fuji_set_directory_position FUNCTION · FUJINET-FUJI.H

```
bool fuji_set_directory_position(uint16_t pos);
```

Moves the open directory to entry pos, for paging through a long one.

RETURNS true on ACK
SENDS FUJI_SET_DIRECTORY_POSITION (page 8-8)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_set_directory_position(1);    /* on to the second */
```

---

fuji_set_host_prefix MACRO · FUJINET-FUJI.H

```
#define fuji_set_host_prefix(hs,prefix)
```

Sets host slot hs's prefix: paths on that host are taken from there. An empty string clears it.

RETURNS true on ACK
SENDS FUJI_SET_HOST_PREFIX (page 8-6)

WARNING: All 256 bytes are sent: use a cleared 256-byte buffer. With a prefix longer than "/", fuji_copy_file() fails on that host, and mounting the same image again puts the prefix on twice.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
strcpy(prefix, "/test");            /* host slot 0's paths start */
ok = fuji_set_host_prefix(0, prefix);   /* here (256 bytes go) */
```

---

fuji_set_ssid MACRO · FUJINET-FUJI.H

```
#define fuji_set_ssid(nc)
```

Sets the WiFi network name and passphrase, joins that network, and saves both.

RETURNS true on ACK
SENDS FUJI_SET_SSID (page 8-4)

WARNING: An ACK does not mean it joined: a wrong passphrase is ACKed and saved. A failure sends two NAKs, and fujinet-lib takes the second for the answer to your next call, so every answer after it is one behind.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
memset(&net, 0, sizeof net);        /* join a network; keep it */
strcpy(net.ssid, "Dummy Cafe");
strcpy(net.password, "swordfish");
ok = fuji_set_ssid(&net);
```

---

fuji_status MACRO · FUJINET-FUJI.H

```
#define fuji_status(status)
```

Gets the FujiNet's four status bytes.

RETURNS true on ACK
SENDS FUJI_STATUS (page 8-21)

WARNING: Always four zero bytes on this firmware.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_status(&fst);             /* four bytes, all 0 here */
if (ok)
    printf("%02X %02X %02X %02X\n", fst.value[0], fst.value[1],
           fst.value[2], fst.value[3]);
```

---

fuji_unmount_disk_image MACRO · FUJINET-FUJI.H

```
#define fuji_unmount_disk_image(ds)
```

Empties drive slot ds.

RETURNS true on ACK
SENDS FUJI_UNMOUNT_IMAGE (page 8-10)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_unmount_disk_image(0);    /* empty drive slot 0 */
```

---

fuji_unmount_host_slot MACRO · FUJINET-FUJI.H
```
#define fuji_unmount_host_slot(hs)
```
Unmounts host slot hs.
RETURNS true on ACK
SENDS FUJI_UNMOUNT_HOST (page 8-6)

WARNING: The SD card cannot be unmounted: it NAKs, but empties the drive slots that use it first. A host mounted by fuji_mount_all() or fuji_copy_file() NAKs too.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
ok = fuji_unmount_host_slot(0);     /* the SD card NAKs */
```

APP KEYS

---

fuji_bus_appkey_read FUNCTION · FUJINET-APPKEY.H
```
bool fuji_bus_appkey_read(void *string, uint16_t *length);
```
The second half of fuji_read_appkey(): sends READ_APPKEY to a key already opened for reading and copies the key's bytes (up to 64) and their count out of the reply.
RETURNS true on ACK
SENDS FUJI_READ_APPKEY (page 8-14)

WARNING: Library internal (fujinet-appkey.h); call fuji_read_appkey(). It does not open the key, and the FujiNet never NAKs a READ_APPKEY: with no key open it ACKs with a count of 0.

---

fuji_bus_appkey_write FUNCTION · FUJINET-APPKEY.H
```
bool fuji_bus_appkey_write(const void *string, uint16_t length);
```
The second half of fuji_write_appkey(): sends WRITE_APPKEY with length bytes to a key already opened for writing.
RETURNS true on ACK
SENDS FUJI_WRITE_APPKEY (page 8-13)

WARNING: Library internal (fujinet-appkey.h); call fuji_write_appkey(). A write of 0 bytes empties the key and still ACKs.

---

fuji_read_appkey MACRO · FUJINET-FUJI.H
```
#define fuji_read_appkey(k,c,d)
```
Reads an application key: opens key key_id of the creator and application set with fuji_set_appkey_details() for reading, then reads up to 64 bytes into data and their count into *count.
RETURNS true if both frames ACK
SENDS FUJI_OPEN_APPKEY (page 8-13), FUJI_READ_APPKEY (page 8-14)

WARNING: A key that does not exist (or no SD card in the FujiNet) still gives true, with *count 0. No NUL is added: data needs 65 bytes if you put one on.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
/* key 0: opens it to read, then reads up to 64 bytes and their
   count (0 if there is no such key) */
ok = fuji_read_appkey(0, &keylen, key);
if (ok) {
    key[keylen] = 0;
    printf("%s\n", (char *)key);
}
```

---

fuji_read_appkey_common FUNCTION · FUJINET-FUJI.H
```
bool fuji_read_appkey_common(uint8_t key_id, uint16_t *count, uint8_t *data);
```
What the fuji_read_appkey() macro calls; the same thing under its own name.
RETURNS true if both frames ACK
SENDS FUJI_OPEN_APPKEY (page 8-13), FUJI_READ_APPKEY (page 8-14)

WARNING: As fuji_read_appkey(): a missing key gives true and a count of 0.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
/* key 0: opens it to read, then reads up to 64 bytes and their
   count (0 if there is no such key) */
ok = fuji_read_appkey(0, &keylen, key);
if (ok) {
    key[keylen] = 0;
    printf("%s\n", (char *)key);
}
```

---

fuji_set_appkey_details MACRO · FUJINET-FUJI.H
```
#define fuji_set_appkey_details(creator,app,size)
```
Sets the creator ID, application ID and key size that fuji_read_appkey() and fuji_write_appkey() use from then on. It is a macro that stores three variables; nothing is sent.
RETURNS nothing useful (the value of an assignment)

WARNING: The size is ignored: keys are 64 bytes (DEFAULT) whatever you pass.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
fuji_set_appkey_details(0x4889, 1, DEFAULT);    /* creator, app */
/* key 0: opens it to write, then writes (which closes it) */
ok = fuji_write_appkey(0, 16, (uint8_t *)"HIGH SCORE 12345");
```

---

fuji_write_appkey FUNCTION · FUJINET-FUJI.H
```
bool fuji_write_appkey(uint8_t key_id, uint16_t count, uint8_t *data);
```
Writes count bytes (64 at most) from data to application key key_id: opens the key for writing, then writes it. Set the creator and application first with fuji_set_appkey_details().
RETURNS true if both frames ACK
SENDS FUJI_OPEN_APPKEY (page 8-13), FUJI_WRITE_APPKEY (page 8-13)

WARNING: A count of 0 empties the key and still gives true. Keys live on the FujiNet's own SD card (in /the FujiNet/): without one, nothing is kept.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C
```
fuji_set_appkey_details(0x4889, 1, DEFAULT);    /* creator, app */
/* key 0: opens it to write, then writes (which closes it) */
ok = fuji_write_appkey(0, 16, (uint8_t *)"HIGH SCORE 12345");
```

NETWORK CHANNELS

---

network_accept FUNCTION · FUJINET-NETWORK.H
```
FN_ERR network_accept(const char* devicespec);
```
Takes the waiting call on a TCP server channel (one opened with "N6:TCP://:port/"). Ask network_status() first: on a listener, connected = 1 means a caller is waiting.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_ACCEPT (page 10-14)

C (Z88DK, FUJINET-LIB) — FROM EXNET.C
```
rc = network_accept(srv);       /* take the call */
```

---

network_close FUNCTION · FUJINET-NETWORK.H
```
FN_ERR network_close(const char* devicespec);
```
Closes the channel on the devicespec's unit. For an HTTP PUT this is when the request goes out.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_CLOSE (page 10-3)

WARNING: An HTTP POST or DELETE that has seen no STATUS or READ is never sent: the close just drops it.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C
```
rc = network_close(hello);
```

---

network_close_client FUNCTION · FUJINET-NETWORK.H
```
FN_ERR network_close_client(const char *devicespec);
```
Hangs up the caller on a TCP server channel and goes on listening.

RETURNS FN_ERR_IO_ERROR when it did hang up; FN_ERR_OK when there was no one to hang up

SENDS NET_CLOSE_CLIENT (page 10-15)

WARNING: The answer is upside down: the firmware NAKs a successful hang-up and ACKs when there was nothing to do, and fujinet-lib passes that on.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C
```
wait_caller(srv);               /* the next caller: */
network_accept(srv);
rc = network_close_client(srv); /* FN_ERR_IO_ERROR means it */
                                /*   DID hang up: inverted */
```

---

network_fs_cd MACRO · FUJINET-NETWORK.H
```
#define network_fs_cd(devicespec)
```
Sets the unit's prefix: later opens on that unit with a relative path start there. Give the whole spec, "N8:SD:/test/".

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_CHDIR (page 10-20)

WARNING: Do not end it with CR LF: unlike OPEN, CHDIR keeps them, in the prefix and in every relative open after.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C
```
rc = network_fs_cd("N8:SD:/test/");     /* names start here now */
```

---

network_fs_command MACRO · FUJINET-NETWORK.H
```
#define network_fs_command(cmd,devicespec)
```
Sends one of the file commands (NETCMD_RENAME, _DELETE, _LOCK, _UNLOCK, _MKDIR, _RMDIR) with two zero fields and the devicespec as the payload. The network_fs_ macros are made of it.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

WARNING: It ends whatever channel was open on that unit (unsent writes are lost), and the reason for a failure is lost too: STATUS afterwards says 207.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C
```
/* what network_fs_mkdir() and the others are made of */
rc = network_fs_command(NETCMD_MKDIR, "N8:SD:/test/libdir");
```

---

network_fs_delete MACRO · FUJINET-NETWORK.H
```
#define network_fs_delete(devicespec)
```
Deletes the file the devicespec names.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_DELETE (page 10-19)

WARNING: It ends whatever channel was open on that unit (unsent writes are lost), and the reason for a failure is lost too: STATUS afterwards says 207. FTP ACKs and deletes nothing.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C
```
rc = network_fs_delete("N8:SD:/test/h89new.txt");
```

---

network_fs_lock MACRO · FUJINET-NETWORK.H
```
#define network_fs_lock(devicespec)
```
Makes a file read-only.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_LOCK (page 10-20)

WARNING: The SD card cannot, and NAKs; FTP, HTTP, SMB and others ACK and do nothing. It ends whatever channel was open on that unit (unsent writes are lost), and the reason for a failure is lost too: STATUS afterwards says 207.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C
```
rc = network_fs_lock("N8:SD:/test/readme.txt"); /* SD: cannot */
```

---

network_fs_mkdir MACRO · FUJINET-NETWORK.H
```
#define network_fs_mkdir(devicespec)
```
Makes a directory.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_MKDIR (page 10-20)

WARNING: It ends whatever channel was open on that unit (unsent writes are lost), and the reason for a failure

is lost too: STATUS afterwards says 207. FTP ACKs and makes nothing.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_fs_mkdir("N8:SD:/test/newdir");
```

---

network_fs_pwd MACRO · FUJINET-NETWORK.H

```
#define network_fs_pwd(devicespec,cwd)
```

Gets the unit's prefix, as network_fs_cd() set it but without the Nn:.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_GETCWD (page 10-21)

WARNING: No NUL is added: clear the 256-byte buffer first.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static char cwd[256];
...
memset(cwd, 0, sizeof cwd);     /* the answer has no NUL */
rc = network_fs_pwd("N8:", cwd);
printf("%s", cwd);
```

---

network_fs_rename MACRO · FUJINET-NETWORK.H

```
#define network_fs_rename(devicespec)
```

Renames a file; the new name follows a comma: "N8:SD:/test/old.txt,new.txt".

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_RENAME (page 10-19)

WARNING: It ends whatever channel was open on that unit (unsent writes are lost), and the reason for a failure is lost too: STATUS afterwards says 207. TNFS with no comma ACKs and does nothing.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_fs_rename("N8:SD:/test/h89.txt,h89new.txt");
```

---

network_fs_rmdir MACRO · FUJINET-NETWORK.H

```
#define network_fs_rmdir(devicespec)
```

Removes a directory, which must be empty.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_RMDIR (page 10-20)

WARNING: It ends whatever channel was open on that unit (unsent writes are lost), and the reason for a failure is lost too: STATUS afterwards says 207.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_fs_rmdir("N8:SD:/test/newdir");    /* it is empty */
```

---

network_fs_unlock MACRO · FUJINET-NETWORK.H

```
#define network_fs_unlock(devicespec)
```

Makes a read-only file writable again.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_UNLOCK (page 10-20)

WARNING: The SD card cannot, and NAKs. It ends whatever channel was open on that unit (unsent writes are lost), and the reason for a failure is lost too: STATUS afterwards says 207.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_fs_unlock("N8:SD:/test/readme.txt");
```

---

network_get_dstats FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_get_dstats(const char *devicespec, uint8_t cmd, uint8_t *dstats);
```

Declared to ask which way a command moves its data (Atari SIO only).

RETURNS does not link

LINKS **no**: not built for the H89

WARNING: Does not link on the H89, under HDOS or CP/M: fujinet-lib declares it but has no H89 version. The firmware would NAK the inquiry anyway.

---

network_has_proceed FUNCTION · FUJINET-NETWORK.H

```
bool network_has_proceed(void);
```

Declared to say whether the hardware has a data-ready (PROCEED) line.

RETURNS does not link

LINKS **no**: not built for the H89

WARNING: Does not link on the H89, under HDOS or CP/M: fujinet-lib declares it but has no H89 version. The H89 has no such line: poll network_status().

---

network_html_parse FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_html_parse(const char *devicespec);
```

Sets the HTML parser on an open channel and parses the page: the FujiNet reads all of it first (512K at most). Plain text parses too, as the page's body.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_PARSE (page 10-7), NET_SET_PARSER (page 10-7)

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
static const char page[] = "N2:HTTP://127.0.0.1:7802/hello.txt";
...
network_open(page, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
/* SET_PARSER 2, then PARSE: the page is read in whole; plain
   text parses too, as the page's body */
rc = network_html_parse(page);
```

---

network_html_query MACRO · FUJINET-NETWORK.H

```
#define network_html_query(devicespec,selector,buffer)
```

Runs a CSS selector ("div.content a", with "@href" on the end for an attribute) against the parsed page and reads the first match's text into buffer; the same selector again gives the next match. It is network_json_query() under another name.

RETURNS the length (0 when nothing more matches); a negative FN_ERR

SENDS NET_QUERY (page 10-8)

WARNING: 256 bytes are sent and the firmware keeps them all as the selector: build it in a cleared 256-byte buffer, since an @ among the stray bytes would make it an attribute query. The reply has no limit.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
/* a CSS selector. All 256 bytes go and the firmware keeps them
   all (an @ among them would ask for an attribute): clear first */
memset(sel, 0, sizeof sel);
strcpy(sel, "body");
len = network_html_query(page, sel, (char *)buf);
if (len > 0)
    printf("%s\n", (char *)buf);       /* the text, both lines */
```

---

network_http_add_header MACRO · FUJINET-NETWORK.H

```
#define network_http_add_header(devicespec,header)
```

Writes one request header, such as "Accept: application/json". It is network_write() with strlen().

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_WRITE (page 10-6)

WARNING: Exactly one space after the colon, and no CR LF on the end: the H89 card keeps a CR LF in the value. A header with no colon NAKs.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_http_add_header(geth, "X-Machine: H89");
```

---

network_http_delete MACRO · FUJINET-NETWORK.H

```
#define network_http_delete(devicespec,trans)
```

Opens an HTTP DELETE (mode 9) on the URL.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_OPEN (page 10-2)

WARNING: Nothing is sent until a network_status() or network_read(); network_close() alone never sends it.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char del[] = "N3:HTTP://127.0.0.1:7802/old.txt";
...
rc = network_http_delete(del, OPEN_TRANS_NONE);    /* mode 9 */
```

---

network_http_end_add_headers
MACRO · FUJINET-NETWORK.H

```
#define network_http_end_add_headers(devicespec)
```

Ends adding headers and puts the channel back in BODY mode (SET_HTTP_MODE 0).

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_SET_HTTP_MODE (page 10-10)

WARNING: As any mode switch, it throws away an answer already buffered.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
static const char geth[] = "N4:HTTP://127.0.0.1:7802/hello.txt";
...
network_open(geth, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE);
network_http_start_add_headers(geth);
network_http_add_header(geth, "X-Machine: H89");
rc = network_http_end_add_headers(geth);    /* back to BODY */
network_status(geth, &avail, &conn, &nerr); /* sends the GET */
len = network_read(geth, buf, avail);
printf("%d bytes\n", len);              /* 55: the body */
```

---

network_http_post FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_http_post(const char *devicespec, const char *data);
```

Sends a string as the body of an HTTP POST opened with OPEN_MODE_HTTP_POST: SET_HTTP_MODE 4, WRITE, SET_HTTP_MODE 0. The request goes out at the next network_status() or network_read().

RETURNS FN_ERR_OK, or FN_ERR_IO_ERROR from the first frame that fails

SENDS NET_WRITE (page 10-6)

WARNING: strlen() gives the length, so the body can hold no NUL (network_http_post_bin() can). A STATUS or READ sends it, never network_close(). An empty body fails at the STATUS with error 144.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
static const char post[] = "N3:HTTP://127.0.0.1:7802/post";
...
network_open(post, OPEN_MODE_HTTP_POST, OPEN_TRANS_NONE);
/* SET_HTTP_MODE 4, WRITE, SET_HTTP_MODE 0: a string, no NULs */
rc = network_http_post(post, "name=H89&cpu=Z80");
network_status(post, &avail, &conn, &nerr);     /* sends it */
len = network_read(post, buf, avail);
if (len > 0) {
    buf[len] = 0;
    printf("%s", (char *)buf);          /* GOT 16 BYTES */
}
```

---

network_http_post_bin FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_http_post_bin(const char *devicespec, const uint8_t *data, uint16_t len);
```

As network_http_post(), with len bytes of anything.

RETURNS FN_ERR_OK, or FN_ERR_IO_ERROR from the first frame that fails

SENDS NET_WRITE (page 10-6)

WARNING: A STATUS or READ sends the request, never network_close().

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
static const uint8_t word[4] = { 'H', 0, '8', '9' };   /* a NUL too */
...
network_open(post, OPEN_MODE_HTTP_POST, OPEN_TRANS_NONE);
rc = network_http_post_bin(post, word, sizeof word);
network_status(post, &avail, &conn, &nerr);     /* sends it */
len = network_read(post, buf, avail);
if (len > 0) {
    buf[len] = 0;
    printf("%s", (char *)buf);          /* GOT 4 BYTES */
}
```

---

network_http_put MACRO · FUJINET-NETWORK.H

```
#define network_http_put(devicespec,data)
```

Sends a string as the body of an HTTP PUT opened with OPEN_MODE_HTTP_PUT (8). It is network_http_post() under another name; the PUT goes out at network_close().

RETURNS FN_ERR_OK, or FN_ERR_IO_ERROR from the first frame that fails

SENDS NET_WRITE (page 10-6)

WARNING: With OPEN_MODE_HTTP_PUT_H (14) the firmware sends a POST instead. A network_status() before the close sends the PUT early, and the close PUTs again.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
/* the body goes in as POST data does; it waits for CLOSE */
rc = network_http_put(put, "PUT from an H89");
```

---

network_http_set_mode MACRO · FUJINET-NETWORK.H

```
#define network_http_set_mode(devicespec,mode)
```

Switches an HTTP channel's mode: HTTP_CHAN_MODE_BODY (0), COLLECT_HEADERS (1), GET_HEADERS (2), SET_HEADERS (3), POST_SET_DATA (4).

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_SET_HTTP_MODE (page 10-10)

WARNING: The switch throws away any answer still buffered, and any write not yet sent. COLLECT_HEADERS works only with OPEN_MODE_HTTP_GET_H (12).

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_http_set_mode(post, HTTP_CHAN_MODE_POST_SET_DATA);
```

---

network_http_start_add_headers
MACRO · FUJINET-NETWORK.H

```
#define network_http_start_add_headers(devicespec)
```

Starts adding request headers (SET_HTTP_MODE 3). Add them with network_http_add_header(), then call network_http_end_add_headers().

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise
SENDS NET_SET_HTTP_MODE (page 10-10)

WARNING: Before the request goes out: before any STATUS or READ.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_http_start_add_headers(geth);  /* mode 3 */
```

---

network_init FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_init(void);
```

Checks the network before you start. On the H89 it sends nothing and always gives FN_ERR_OK.

RETURNS FN_ERR_OK, always

WARNING: It proves nothing on the H89. To see that the card answers, call fuji_get_adapter_config().

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
rc = network_init();        /* nothing to set up on the H89 */
if (rc != FN_ERR_OK)
    printf("No network.\n");
```

---

network_json_parse FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_json_parse(const char *devicespec);
```

Sets the JSON parser on an open channel and parses what it reads (SET_PARSER, then PARSE).

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise
SENDS NET_PARSE (page 10-7), NET_SET_PARSER (page 10-7)

WARNING: On a connection that stays open (TCP) PARSE waits for the far end to close, and fujinet-lib gives up after 15 seconds. After it, STATUS says 136 (the body is used up) until you query.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_json_parse(json);  /* sets the parser itself too */
```

---

network_json_query FUNCTION · FUJINET-NETWORK.H

```
int16_t network_json_query(const char *devicespec, const char *query, char *buffer);
```

Asks for the value at a JSON Pointer ("/name", "/ports/2") and reads it into buffer with the line ending taken off and a NUL put on.

RETURNS the length (0 if the name is not there); a negative FN_ERR
SENDS NET_QUERY (page 10-8)

WARNING: 256 bytes are always sent from query (harmless for JSON). The reply has no limit: buffer must hold the longest value and a NUL. A query whose value is, or holds, an empty array [] crashes the firmware.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
len = network_json_query(json, "/name", (char *)buf);
printf("%s", buf);              /* "FujiNet", line end removed */
```

---

network_open FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_open(const char* devicespec, uint8_t mode, uint8_t trans);
```

Opens a channel. The devicespec says which unit and what to talk to ("N1:HTTP://host/path", "N5:TCP://host:port/", "N8:SD:/test/x"); mode is 4 read, 8 write, 12 both, 6 a directory, 13 an HTTP POST and so on; trans is the line-end translation (0 none, 1 CR, 2 LF, 3 CR LF).

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise
SENDS NET_OPEN (page 10-2)

WARNING: HTTP sends nothing until the first STATUS or READ. On SD: and TNFS: a mode other than 4, 6, 7, 8, 9 or 12 crashes the firmware. A * in a URL cuts it short there, and a URL ending in a dot loses the dot. Every failure is FN_ERR_IO_ERROR, here and in the other network calls: the reason is in the error byte network_status() gives.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char hello[] = "N1:HTTP://127.0.0.1:7802/hello.txt";
...
rc = network_open(hello, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
```

---

network_proceed FUNCTION · FUJINET-NETWORK.H

```
bool network_proceed(void);
```

Declared to say whether data-ready (PROCEED) is asserted.

RETURNS does not link
LINKS **no**: not built for the H89

WARNING: Does not link on the H89, under HDOS or CP/M: fujinet-lib declares it but has no H89 version. Poll network_status() instead.

---

network_read FUNCTION · FUJINET-NETWORK.H

```
int16_t network_read(const char* devicespec, void *buf, uint16_t len);
```

Reads len bytes, waiting for them: it asks STATUS how many are waiting and reads those, again and again, until it has len, reaches the end, or meets an error.

RETURNS the count read (0 at the end); a negative FN_ERR if nothing was read
SENDS NET_READ (page 10-5)

WARNING: It blocks: on a TCP or UDP channel that goes quiet it waits for ever (it is network_read_nb() in a loop). It never asks for more than STATUS says is waiting.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
/* type len bytes as they are, line ends and all */
static void type(const uint8_t *p, uint16_t len)
{
    while (len--)
        putchar(*p++);
}
...
n = avail < sizeof buf ? avail : sizeof buf;
len = network_read(hello, buf, n);  /* STATUS, then READ */
if (len > 0)
    type(buf, len);
```

---

network_read_nb FUNCTION · FUJINET-NETWORK.H

```
int16_t network_read_nb(const char* devicespec, void *buf, uint16_t len);
```

Reads what is waiting, up to len: STATUS until something is, then one READ.

RETURNS the count; 0 at the end, or on an error with nothing waiting; a negative FN_ERR if STATUS fails
SENDS NET_READ (page 10-5)

WARNING: Not as non-blocking as its name: while nothing is waiting and there is no error it keeps asking, for ever. An idle TCP server, or a UDP channel (whose STATUS does not count the first datagram), never returns. Poll network_status() yourself there.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char bin[] = "N1:HTTP://127.0.0.1:7802/data.bin";
...
network_open(bin, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
total = 0;
while ((len = network_read_nb(bin, buf, sizeof buf)) > 0)
    total += len;              /* what was there, up to 2048 */
printf("%u bytes", total);     /* 0 from it: the end */
rc = network_close(bin);
```

---

network_seek FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_seek(const char *devicespec, uint32_t pos);
```

Moves the read position of an open file (SD, TNFS, HTTP) to pos.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_SEEK (page 10-18)

WARNING: The firmware seeks twice; over HTTP that is two Range requests.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
network_seek(img, 1536);       /* sector 3 */
len = network_read(img, buf, 8);
for (n = 0; n < len; n++)
    printf("%02X ", buf[n]);   /* 03 03 03 ... */
```

---

network_set_eol FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_set_eol(const char *devicespec, const char *eol);
```

Sets the end of line the unit translates to and from ("\r" suits the H89); "" goes back to the default.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_SET_EOL (page 10-22)

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_eol("N1:", "\r");      /* our lines end CR */
```

---

network_set_interrupt_rate
FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_set_interrupt_rate(const char *devicespec, uint8_t rate);
```

Sets how often, in milliseconds, the unit raises its data-ready signal.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_SET_INT_RATE (page 10-23)

WARNING: No use on the H89: the signal goes nowhere. And 0 crashes the firmware once a channel is open on that unit.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_interrupt_rate("N1:", 100);    /* never 0 */
```

---

network_set_line_ending MACRO · FUJINET-NETWORK.H

```
#define network_set_line_ending(devicespec,ch)
```

Sets the byte the parser puts after every value (parameter 1; LF to start with).

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_SET_PARAMETER (page 10-9)

WARNING: Set it after the parse, which resets it. network_json_query() takes a CR, LF or 9BH off again; any other byte stays.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
/* the byte put after every value; CR, LF or 9BH would come off
   again in network_json_query(), anything else stays */
rc = network_set_line_ending(json, ',');
len = network_json_query(json, "/ports/0", (char *)buf);
printf("%s\n", (char *)buf);           /* 208, */
```

---

network_set_parameter FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_set_parameter(const char *devicespec, uint8_t param, uint8_t value);
```

Sets a parser parameter: 0 the query flags (see network_set_query_param()), 1 the line ending (see network_set_line_ending()).

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_SET_PARAMETER (page 10-9)

WARNING: Needs an open channel with a parser, and SET_PARSER resets both values: call it after network_json_parse().

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
network_set_parameter(json, 1, '\r');   /* 1: the line end */
len = network_json_query(json, "/machine", (char *)buf);
show(buf, len);                /* "H89" (the CR is stripped) */
```

---

network_set_parser FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_set_parser(const char *devicespec, uint8_t mode);
```

Chooses the parser for an open channel: PARSER_NONE, PARSER_JSON, PARSER_HTML or PARSER_XML.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_SET_PARSER (page 10-7)

WARNING: Only after network_open(): with no channel open it ACKs, and the next READ, QUERY, SEEK or TELL on that unit crashes the firmware. An open sets it back to NONE.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_parser(json, PARSER_JSON);     /* after OPEN */
```

---

network_set_password FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_set_password(const char *devicespec, const char *password);
```

Sets the password for the unit's next open.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_PASSWORD (page 10-22)

WARNING: Ignored unless a user name is set too.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_password("N1:", "swordfish");
```

---

network_set_query_param MACRO · FUJINET-NETWORK.H

```
#define network_set_query_param(devicespec,flags)
```

Sets the query flags (parameter 0): 10H gives values as plain ASCII (entities decoded, other characters folded), 00H as they are. Bits 0-2 remap characters for other machines.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_SET_PARAMETER (page 10-9)

WARNING: With 10H a string's line ending is folded with it: the LF (or CR) comes back as a space, which network_json_query() leaves on, so "the FujiNet" arrives as "the FujiNet ". Numbers are not affected. Anything but 00H-07H and 10H-17H NAKs. Never 04H when a value holds a < with no > after it: the firmware loops for ever. Set it after the parse, which resets it.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
static const char json[] = "N2:HTTP://127.0.0.1:7802/fuji.json";
...
network_open(json, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
network_json_parse(json);
/* 10H: plain ASCII out, entities decoded; set after the parse */
rc = network_set_query_param(json, 0x10);
len = network_json_query(json, "/name", (char *)buf);
printf("[%s]\n", (char *)buf);  /* [FujiNet ]: in this mode a */
                                /*   string's LF comes as a space */
```

---

network_set_translation FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_set_translation(const char *devicespec, uint8_t trans);
```

Sets a translation that is ORed into every later open on the unit (0 none, 1 CR, 2 LF, 3 CR LF).

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_TRANSLATION (page 10-22)

WARNING: It ORs: 1 here and 2 in network_open() make 3, CR LF.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_translation("N1:", OPEN_TRANS_CRLF);
```

---

network_set_username FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_set_username(const char *devicespec, const char *username);
```

Sets the user name for the unit's next open (FTP, SMB and the like).

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_USERNAME (page 10-22)

WARNING: With an empty user name the password is not passed either.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_username("N1:", "h89user");
```

---

network_status FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_status(const char *devicespec, uint16_t *bw, uint8_t *c, uint8_t *err);
```

Asks the unit how many bytes are waiting, whether it is connected, and its last error (1 all well, 136 the end, 207 nothing open, and so on). The answer also lands in nw_status.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_STATUS (page 10-4)

WARNING: Over HTTP the first STATUS is what sends the request. On UDP the count leaves out the datagram STATUS has just taken in: one datagram waiting shows 0.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_status(hello, &avail, &conn, &nerr);   /* sends GET */
printf("%u waiting", avail);
```

---

network_tell MACRO · FUJINET-NETWORK.H

```
#define network_tell(ds,pos)
```

Gets the read position of an open file.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_TELL (page 10-19)

WARNING: With no channel open the four bytes are garbage, and still ACKed. On SD and TNFS, bytes STATUS has read ahead are thrown away, and the position is past them.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_tell(img, &pos);
printf("%lu", pos);            /* 1544 */
```

---

network_tell_common FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_tell_common(const char *devicespec, uint32_t *pos);
```

What the network_tell() macro calls; the same thing under its own name.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_TELL (page 10-19)

WARNING: As network_tell(): with no channel open the answer is garbage.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_tell(img, &pos);
printf("%lu", pos);            /* 1544 */
```

---

network_udp_get_remote FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_udp_get_remote(const char *devicespec, char *buf, uint16_t len);
```

Gets "ip:port" of whoever sent the last UDP datagram.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_GET_REMOTE (page 10-16)

WARNING: buf must be 256 bytes, which is what always comes back, or the call fails. Before any datagram it says 255.255.255.255:0.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static char remote[256];       /* the answer is always 256 bytes */
...
/* the buffer must hold all 256 bytes, or the call fails */
rc = network_udp_get_remote(udp, remote, sizeof remote);
printf("%s", remote);          /* who sent it */
```

---

network_udp_set_destination
FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_udp_set_destination(const char *dest_spec);
```

Sets where an open UDP channel sends to, as "N7:host:port".

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_SET_DESTINATION (page 10-17)

WARNING: Does not work on the H89 as it stands: it sends the spec with no NUL, which fujinet-pc needs, and the card misreads the "N7:" form (only "N:host:port" comes out right), so the ACK means nothing. Send SET_DESTINATION with fuji_bus_call() and a NUL, as the example does. A datagram that arrives later makes its sender the destination anyway.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
/* not network_udp_set_destination(): it sends no NUL, which
   fujinet-pc needs, and wants "N7:", which the card misreads */
ok = fuji_bus_call(FUJI_DEVICEID_NETWORK + 6,
                   NETCMD_SET_DESTINATION, FUJI_FIELD_NONE,
                   0, 0, 0, 0, "N:127.0.0.1:7805", 17);  /* +NUL */
```

network_unit_status MACRO · FUJINET-NETWORK.H

```
#define network_unit_status(unit,nws)
```

STATUS by unit number, into a NetworkStatus (avail, status, errcode); network_status() is made of it.

RETURNS 0 when it worked, 1 when it did not

SENDS FUJI_STATUS (page 8-21), NET_STATUS (page 10-4)

WARNING: 0 means success here: the macro is ! fuji_bus_call(). Its first argument is a unit number (1-8), not a devicespec.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
static const char hello[] = "N1:HTTP://127.0.0.1:7802/hello.txt";
...
network_open(hello, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
/* STATUS by unit number, into fujinet-lib's own nw_status;
   0 means it worked (the macro is !fuji_bus_call) */
n = network_unit_status(1, &nw_status);
if (n == 0)
    printf("%u waiting\n", nw_status.avail);    /* 55 */
```

network_write FUNCTION · FUJINET-NETWORK.H

```
FN_ERR network_write(const char* devicespec, const void *buf, uint16_t len);
```

Writes len bytes to the channel, in one frame.

RETURNS FN_ERR_OK on ACK, FN_ERR_IO_ERROR otherwise

SENDS NET_WRITE (page 10-6)

WARNING: A write that fails stays in the FujiNet's buffer and goes again in front of the next one. Translation turns round only the whole native end of line (CR LF unless network_set_eol() changed it); a lone CR goes as it is.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_write(tcp, "hello from the h89\r", 19);
```

THE CLOCK

clock_get_time MACRO · FUJINET-CLOCK.H

```
#define clock_get_time(time_data,format)
```

Gets the time from the clock device (45H) in one of seven formats, in the FujiNet's system time zone. The format picks the command; the string formats come back NUL-terminated.

RETURNS FN_ERR_OK; FN_ERR_IO_ERROR on a NAK or no answer; FN_ERR_BAD_CMD for a format past the end of TimeFormat

SENDS APETIME_GETTIME (page 12-1), APETIME_GET_ISO_LOCAL (page 12-3), APETIME_GET_ISO_UTC (page 12-3), APETIME_GET_PRODOS (page 12-2), APETIME_GET_SIMPLE_HUNDREDTHS (page 12-2), APETIME_GET_SOS (page 12-2), APETIME_SETTZ_ALT2 (page 12-1)

WARNING: The library writes a NUL after the reply, binary formats too, so the buffer must be one byte longer than the reply: 8 bytes for SIMPLE_BINARY, 5 for PRODOS_BINARY, 7 for APETIME_BINARY, 26 for TZ_ISO_STRING and UTC_ISO_STRING, 20 for APPLE3_SOS_BINARY, 9 for SIMPLE_BINARY_WITH_HUNDREDTHS. SIMPLE_BINARY is command 54H, which the firmware calls APETIME_SETTZ_ALT2 but answers with the time.

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static uint8_t when[7];        /* 6 bytes + the NUL fujinet-lib adds */
...
ok = clock_get_time(when, APETIME_BINARY) == FN_ERR_OK;
if (ok)                    /* day, month, year - 2000, h, m, s */
    printf("\n%u/%u/%u %u:%02u:%02u", when[0], when[1],
           when[2] + 2000, when[3], when[4], when[5]);
```

clock_get_time_common FUNCTION · FUJINET-CLOCK.H

```
uint8_t clock_get_time_common(uint8_t *time_data, TimeFormat format, bool alt);
```

What clock_get_time() calls. With alt true it asks for the time in the alternate time zone, the one clock_get_time_tz() set, instead of the system zone (field 1 = 1).

RETURNS FN_ERR_OK, FN_ERR_IO_ERROR or FN_ERR_BAD_CMD, as clock_get_time()

WARNING: Buffer one byte longer than the reply, as for clock_get_time(). With no alternate zone set, alt true gives the system zone's time.

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
/* clock_get_time_common(..., true): field 1 = 1, the other zone */
ok = clock_get_time_common(when, APETIME_BINARY, true) == FN_ERR_OK;
if (ok)
    printf("\n%u:%02u there", when[3], when[4]);
```

clock_get_time_tz FUNCTION · FUJINET-CLOCK.H

```
uint8_t clock_get_time_tz(uint8_t *time_data, const char *tz, TimeFormat format);
```

Sets the alternate time zone (in the FujiNet's memory only), then gets the time there, as clock_get_time_common() with alt true. The system zone is left alone.

RETURNS FN_ERR_OK; FN_ERR_IO_ERROR if either frame fails; FN_ERR_BAD_CMD for a bad format
SENDS APETIME_SETTZ (page 12-4)

WARNING: The alternate zone stays set after the call and is used by every later request with alt true. The FujiNet ACKs the zone before it looks at it: a bad zone is dropped without a word, and the time comes back in the zone that was there before. Buffer one byte longer than the reply.

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static const char central[] = "CST6CDT,M3.2.0,M11.1.0";
...
/* sets the alternate zone (in RAM only), then asks the time
   there: GETTIME with field 1 = 1 */
ok = clock_get_time_tz(when, central, APETIME_BINARY) == FN_ERR_OK;
if (ok)
    printf("\n%u:%02u in Chicago", when[3], when[4]);
```

---

clock_get_tz FUNCTION · FUJINET-CLOCK.H

```
uint8_t clock_get_tz(char *tz);
```

Reads the FujiNet's system time zone, a POSIX TZ string such as "CST6CDT,M3.2.0,M11.1.0". It asks for the length first, then for the string, which comes with its NUL.

RETURNS FN_ERR_OK or FN_ERR_IO_ERROR
SENDS APETIME_GETTZ_LEN (page 12-3), APETIME_GET_GENERAL (page 12-3)

WARNING: The FujiNet decides how many bytes come back, not you: the buffer must hold the longest zone that could be set, NUL and all. 64 bytes is plenty for a real zone.

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static char tz[64];            /* clock_get_tz() trusts it is enough */
...
ok = clock_get_tz(tz) == FN_ERR_OK;     /* asks the length first */
if (ok)
    printf("\n%s", tz);
```

---

clock_set_tz FUNCTION · FUJINET-CLOCK.H

```
uint8_t clock_set_tz(const char *tz);
```

Sets the FujiNet's system time zone. It is saved in the FujiNet's configuration and is still there after a restart.

RETURNS FN_ERR_OK or FN_ERR_IO_ERROR
SENDS APETIME_SETTZ_ALT (page 12-5)

WARNING: ACKed before it is checked: a zone holding a control character (a CR on the end, say) is thrown away and FN_ERR_OK still comes back. Read it back with clock_get_tz() if it matters.

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static const char eastern[] = "EST5EDT,M3.2.0,M11.1.0";
...
ok = clock_set_tz(eastern) == FN_ERR_OK;    /* and it is saved */
```

QR CODES

---

fuji_qrcode_encode MACRO · FUJINET-QRCODE.H

```
#define fuji_qrcode_encode(version,ecc,shorten)
```

Makes a QR code of the data sent: version 1-40 (0, the smallest that fits), error correction QR_ECC_LOW to

QR_ECC_HIGH, and shorten (non-zero: a the FujiNet short URL is coded instead). The data is cleared.

RETURNS true on ACK
SENDS FUJI_QRCODE_ENCODE (page 8-19)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* version 0: the smallest that fits; not shortened */
ok = fuji_qrcode_encode(0, QR_ECC_LOW, 0);
```

---

fuji_qrcode_input MACRO · FUJINET-QRCODE.H

```
#define fuji_qrcode_input(data,len)
```

Adds len bytes to the data for the next QR code. Call it as often as you need to, then fuji_qrcode_encode().

RETURNS true on ACK; false for len 0
SENDS FUJI_QRCODE_INPUT (page 8-19)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* capitals: QR's alphanumeric mode makes a smaller code */
ok = fuji_qrcode_input("HTTPS://FUJINET.ONLINE/", 23);
```

---

fuji_qrcode_length MACRO · FUJINET-QRCODE.H

```
#define fuji_qrcode_length(mode,len)
```

Renders the code in an output mode (QR_OUTPUT_MODE_BINARY, _ANSI, _BITMAP, _SVG, _ATASCII or _PETSCII) and gets its length in bytes into an unsigned long.

RETURNS true on ACK
SENDS FUJI_QRCODE_LENGTH (page 8-19)

WARNING: BINARY and BITMAP start with one size byte (the modules on a side) before the rows, and the length counts it. Once fuji_qrcode_output() has taken the whole code this gives 0.

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_qrcode_length(QR_OUTPUT_MODE_BITMAP, &qrlen);
if (ok)
    printf("%lu bytes\n", qrlen);
```

---

fuji_qrcode_output MACRO · FUJINET-QRCODE.H

```
#define fuji_qrcode_output(buf,len)
```

Reads the next len bytes of the rendered code into buf; the FujiNet forgets what it has sent.

RETURNS true on ACK; false if len is 0 or more than is left
SENDS FUJI_QRCODE_OUTPUT (page 8-19)

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_qrcode_output(qr, (uint16_t)qrlen);
if (ok)                 /* the modules on a side, then the rows */
    printf("%u x %u\n", qr[0], qr[0]);
```

---

qrcode_create FUNCTION · FUJINET-QRCODE.H

```
int qrcode_create(uint8_t version, qr_ecc_t ecc, bool shorten, qr_output_mode_t mode, void
*input, size_t input_len, void *output, size_t output_max);
```

Makes a QR code in one call: fuji_qrcode_input(), fuji_qrcode_encode(), fuji_qrcode_length() and fuji_qrcode_output(), into output.

RETURNS the length; -162 if output_max is too small; -215 if a frame failed
SENDS FUJI_QRCODE_OUTPUT (page 8-19)

WARNING: The errors are negative NETWORK_ERROR values, not FN_ERR. When output_max is too small the code is left waiting in the FujiNet.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
/* INPUT, ENCODE, LENGTH, OUTPUT; the length, or minus an error */
r = qrcode_create(0, QR_ECC_LOW, false, QR_OUTPUT_MODE_BITMAP,
                  "HTTPS://FUJINET.ONLINE/", 23, qr, sizeof qr);
if (r > 0)                  /* a size byte, then the rows */
    printf("%d bytes, %u x %u\n", r, qr[0], qr[0]);
```

THE BUS

fuji_bus_call FUNCTION · FUJINET-BUS.H

```
bool fuji_bus_call(uint8_t device, uint8_t fuji_cmd, uint8_t fields, uint8_t aux1, uint8_t aux2, uint8_t aux3, uint8_t aux4, const void *buf, size_t buf_length);
```

Sends one frame to any device and waits for the answer: device, command, a field descriptor, up to four field bytes, and a buffer. With FUJI_FIELD_REPLY in the descriptor the buffer receives the reply; otherwise it is sent as the payload. Every other call in fujinet-lib comes down to this one.

RETURNS true on ACK; false on a NAK, a bad checksum, a reply longer than the buffer, or no answer in 15 seconds

WARNING: false does not say whether it was a NAK or no answer. A reply shorter than the buffer still gives true, and you cannot learn its length. Sending has no timeout at all: if the card is not running, the program hangs in port_putc() before the 15 seconds start. Field bytes beyond what the descriptor counts are not sent.

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
/* fujinet-lib has no disk calls: fuji_bus_call sends the frame */
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_READ,
                   FUJI_FIELD_C1234 | FUJI_FIELD_REPLY,
                   3, 0, 0, 0, sector, 512);    /* sector 3 */
if (ok) {
    for (i = 0; i < 512 && sector[i] == 3; i++)
        ;                           /* sector n: 512 bytes of n */
    if (i == 512)
        printf("\nSector 3 is 512 bytes of 03H.");
}
```

fuji_devcall_vld FUNCTION · FUJINET-BUS-EZCALL.H

```
bool fuji_devcall_vld(uint8_t device, uint8_t cmd, const void *data, uint16_t data_len);
```

Sends a payload and no fields to any device. The H89 has frames of any length, so this is fuji_bus_call() with FUJI_FIELD_DATA, or with no payload at all when data_len is 0. network_set_eol() and network_udp_set_destination() are made of it.

RETURNS true on ACK

WARNING: Exactly data_len bytes go: count the NUL in if the far end needs one.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
/* a payload and no fields, to any device: SET_EOL on unit 1 */
ok = fuji_devcall_vld(FUJI_DEVICEID_NETWORK, NETCMD_SET_EOL,
                      "\r\n", 2);
```

network_bus_read FUNCTION · FUJINET-BUS.H

```
size_t network_bus_read(uint8_t device, void *buffer, size_t length);
```

Sends one READ frame for length bytes to a network device (71H-78H) and puts the reply in buffer. network_read() calls it once STATUS has said how much is waiting.

RETURNS length on ACK, whatever arrived; 0 on a NAK or no answer

SENDS NET_READ (page 10-5)

WARNING: It gives back the length you asked for on any ACK, even when fewer bytes came: ask for no more than STATUS says is waiting. It takes a device number (FUJI_DEVICEID_NETWORK + unit - 1), not a devicespec.

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
network_open(bin, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
/* network_read asks STATUS first; the bus-level READ does not */
n = network_bus_read(FUJI_DEVICEID_NETWORK, buf, 2048);
printf("%u ", n);               /* 2048, all of them */
n = network_bus_read(FUJI_DEVICEID_NETWORK, buf, 2048);
printf("%u", n);                /* 2048 again, though 952 came */
```

network_bus_write FUNCTION · FUJINET-BUS.H

```
size_t network_bus_write(uint8_t device, const void *buffer, size_t length);
```

Sends one WRITE frame with length bytes to a network device (71H-78H). network_write() is made of it.

RETURNS length on ACK; 0 on a NAK or no answer

SENDS NET_WRITE (page 10-6)

WARNING: A device number, not a devicespec. It is one frame, whose length is a word: keep length well under 64K.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
network_open("N8:SD:/h89lib.txt", OPEN_MODE_WRITE, OPEN_TRANS_NONE);
/* one WRITE frame to unit 8 (78H): the length, or 0 on a NAK */
n = network_bus_write(FUJI_DEVICEID_NETWORK + 7,
                      "Written by EXLIB.\r\n", 19);
printf("%u\n", n);                      /* 19 */
network_close("N8:");
```

HELPERS

fuji_field_numbytes MACRO · FUJINET-BUS.H

```
#define fuji_field_numbytes(descr)
```

How many field bytes a descriptor stands for, from a table: 0-4 for FUJI_FIELD_NONE to FUJI_FIELD_C1234. The DATA and REPLY bits are left out.

RETURNS 0 to 4

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
/* how many field bytes a descriptor means: two words here */
n = fuji_field_numbytes(FUJI_FIELD_B12_B34);
printf("%u bytes\n", n);              /* 4 */
```

fuji_field_numfields MACRO · FUJINET-BUS.H

```
#define fuji_field_numfields(descr)
```

How many fields a descriptor stands for: a word counts as one field, and so does a long.

RETURNS 0 to 4

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
n = fuji_field_numfields(FUJI_FIELD_B12_B34);
printf("%u fields\n", n);             /* 2 */
```

fuji_hash_size FUNCTION · FUJINET-FUJI.H

```
uint16_t fuji_hash_size(hash_alg_t hash_type, bool as_hex);
```

The size of a digest, worked out in the library with no frame sent: 16, 20, 32 or 64 bytes for MD5, SHA1, SHA256 and SHA512, twice that as hex.

RETURNS the size; 0 for an unknown type

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
n = fuji_hash_size(SHA256, true);   /* 64: two hex digits a byte */
printf("%u\n", n);
```

network_json_strip_newlines
FUNCTION · FUJINET-NETWORK.H

```
size_t network_json_strip_newlines(char *buffer, size_t buflen);
```

Cuts CR, LF and 9BH (the Atari end of line) off the end of buflen bytes and writes a NUL after what is left. network_json_query() uses it.

RETURNS the new length

WARNING: It writes buffer[buflen] when there is nothing to cut: the buffer needs one byte more than buflen.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
strcpy((char *)buf, "H89\r\n");
/* drops CR, LF and 9BH from the end, then writes a NUL */
n = network_json_strip_newlines((char *)buf, 5);
printf("%u \"%s\"\n", n, (char *)buf);  /* 3 "H89" */
```

network_unit FUNCTION · FUJINET-NETWORK.H

```
uint8_t network_unit(const char *devicespec);
```

Works out the unit from a devicespec: the digit in "N3:", or 1 for "N:" and anything else. No frame is sent.

RETURNS the unit number, unchecked

SENDS NET_OPEN (page 10-2)

WARNING: No range check. "N0:" gives unit 0, which is the Fuji device (70H): network calls on it reach the Fuji commands with the same numbers (NET_SET_PARAMETER, FBH, is SET_SSID). "N9:" gives 79H, which NAKs; "N10:" gives 1.

C (Z88DK, FUJINET-LIB) — FROM EXLIB.C

```
/* the unit in a devicespec (N: is 1); no frame is sent */
printf("%u %u %u\n", network_unit("N:TCP://127.0.0.1:7801/"),
       network_unit("N3:HTTP://127.0.0.1:7802/"),
       network_unit("N8:SD:/test/"));       /* 1 3 8 */
```

THE FUJINET DEVICE

GENERAL

Device 70H is the FujiNet itself. Through it a program can ask how the FujiNet is connected, join another WiFi network, change the eight host slots and the eight drive slots, browse a host's directories, mount disk images, keep a few bytes of its own on the FujiNet (an **app key**), and use the FujiNet as a calculator for jobs too big for the H89: base64, cryptographic hashes and QR codes. It answers 57 commands on the H89, and this chapter gives each a card.

The FujiNet keeps its settings in its configuration file, on its own flash or the microSD card, and the commands that change them save the file at once. So a program that changes a host slot, or the WiFi network, changes it for CONFIG, for FNDISK and for every program after it. Say so to the user before doing it.

THE EXAMPLES IN THIS CHAPTER

The examples come from two programs, EXFUJI.ASM and EXFUJI.C, which send every command in this chapter in turn, in the order of the cards, and print what came back. Both were run under HDOS and CP/M on FujiNet Go H89, with fujinet-pc as the FujiNet, and the bytes under each card are the assembly version's, under HDOS.

A few things in those bytes come from the test set-up rather than from a real card:

- The WiFi network is fujinet-pc's stand-in: it is always "Dummy Cafe", always connected, and a scan finds "Dummy Cafe 0" to "Dummy Cafe 15". Cards whose replies show this are tagged STAND-IN WIFI.
- The FujiNet's name is the name of the computer fujinet-pc ran on (TMA-3), and its address is 127.0.0.1. On a real card, both come from your network.
- The host slots are those of the test configuration: SD, tnfs.fujinet.online, apps.irata.online. The SD card is a directory on that computer, holding a `test` directory with three files.

The examples run one after another, so some depend on those before them: the directory examples need the SD host mounted, and the drive examples use the `test` directory. Each card says what it needs.

ADAPTER AND WIFI

GET ADAPTER CONFIGURATION E8H

DEVICE 70H (160Q) COMMAND E8H (350Q) FUJI_GET_ADAPTERCONFIG

STAND-IN WIFI

FIELDS	none
PAYLOAD	none
REPLY	140 bytes: the network, the FujiNet's name and addresses, the firmware version

How the FujiNet is connected. FNHELLO (Chapter 4) is built round this command. The reply is fujinet-lib's `AdapterConfig`:

OFFSET	BYTES	HOLDS
0	33	The network's name (SSID), padded with zeros. "NOT CONNECTED" if there is none.
33	64	The FujiNet's host name. Empty if not connected.
97	4	Its IP address, a byte at a time, most significant first: 192, 168, 1, 89.
101	4	The gateway (router).
105	4	The netmask.
109	4	The name server (DNS).
113	6	The FujiNet's MAC address.
119	6	The access point's MAC address (BSSID).
125	15	The firmware's version, ending in a zero (at most 14 characters).

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_config            ; the adapter's configuration
        call    fn_call
        jr      nz,config_x
        ld      hl,cfg_ssid             ; the network it is on,
        call    print
        call    pspace
        ld      hl,cfg_ip               ; its address
        call    pip
        call    pspace
        ld      hl,cfg_ver              ; and its firmware
        call    print
config_x:
...
rq_config:
        FNREQ   DEV_FUJINET, FUJI_GET_ADAPTERCONFIG, F_NONE, 0,0,0,0
        FNBUF   0, 0, cfg, 140
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_adapter_config(&cfg);
if (ok)
    printf("%s %u.%u.%u.%u %s\n", cfg.ssid, cfg.localIP[0],
           cfg.localIP[1], cfg.localIP[2], cfg.localIP[3],
           cfg.fn_version);
```

H89 → FUJINET

C0	70	E8	06	00	5F	00	C0

FUJINET → H89 · ACK · 140 BYTES

C0	70	06	92	00	E8	00	44 D	75 u	6D m	6D m	79 y	20	43 C	61 a	66 f
65 e	00	00	00	00	00	00	00	00	00	00	122 MORE				C0

GET ADAPTER CONFIGURATION, EXTENDED C4H

DEVICE 70H (160Q) COMMAND C4H (304Q) FUJI_GET_ADAPTERCONFIG_EXTENDED

STAND-IN WIFI

FIELDS	none
PAYLOAD	none
REPLY	240 bytes: the 140 above, then the addresses as text

The same, followed by the addresses already written out, which saves an H89 program the trouble:

OFFSET	BYTES	HOLDS
0	140	As FUJI_GET_ADAPTERCONFIG.
140	16	The IP address as text, "192.168.1.89", ending in a zero.
156	16	The gateway, as text.
172	16	The netmask, as text.
188	16	The name server, as text.
204	18	The MAC address, "24:6F:28:1A:89:01".
222	18	The access point's MAC address.

The text fields are filled in even when the FujiNet is not connected (with zeros).

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_configx           ; the same, with the numbers
        call    fn_call                 ;   as text too, ready to print
        jr      nz,configx_x
        ld      hl,cfgx_sip
        call    print
        call    pspace
        ld      hl,cfgx_smac
        call    print
configx_x:
...
rq_configx:                             ; FNREQ in two lines (the name
        defb    DEV_FUJINET, FUJI_GET_ADAPTERCONFIG_EXTENDED
        defb    F_NONE, 0,0,0,0         ;   is too long for one)
        FNBUF   0, 0, cfgx, 240
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_adapter_config_extended(&cfgx);
if (ok)                         /* the numbers as text too */
    printf("%s %s\n", cfgx.sLocalIP, cfgx.sMacAddress);
```

H89 → FUJINET

C0	70	C4	06	00	3B	00	C0

FUJINET → H89 · ACK · 240 BYTES

C0	70	06	F6	00	96	00	44 D	75 u	6D m	6D m	79 y	20	43 C	61 a	66 f
65 e	00	00	00	00	00	00	00	00	00	00	222 MORE				C0

GET WIFI STATUS FAH

DEVICE 70H (160Q) COMMAND FAH (372Q) FUJI_GET_WIFISTATUS

STAND-IN WIFI

FIELDS	none
PAYLOAD	none
REPLY	1 byte: 3, connected; 6, not

Whether the FujiNet is on its network. The firmware only ever answers 3 (connected) or 6 (not connected). fujinet-lib's `enum WifiStatus` has names for 1, 3, 4 and 5, but not for 6, so compare with 3.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_wifist            ; is it connected?
        call    fn_call
        jr      nz,wifist_x
        ld      a,(wifist)              ; 3 = connected, 6 = not
        call    pdec8
wifist_x:
...
rq_wifist:
        FNREQ   DEV_FUJINET, FUJI_GET_WIFISTATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, wifist, 1
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_wifi_status(&status);
if (ok)                             /* 3 = connected, 6 = not */
    printf("%u\n", status);
```

H89 → FUJINET

C0	70	FA	06	00	71	00	C0

FUJINET → H89 · ACK · 1 BYTE

C0	70	06	07	00	80	00	03	C0

GET WIFI ENABLED — EAH

DEVICE 70H (160Q) COMMAND EAH (352Q) FUJI_GET_WIFI_ENABLED

FIELDS	none
PAYLOAD	none
REPLY	1 byte: 1 if WiFi is turned on

Whether WiFi is turned on in the FujiNet's settings (it can be turned off from its web page).

WARNING: fujinet-lib's `fuji_get_wifi_enabled()` returns the answer of the **last** successful call if this one fails, without saying it failed. Make the call with `fuji_bus_call()` if you need to know.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_wifien            ; is the WiFi switched on?
        call    fn_call
        jr      nz,wifien_x
        ld      a,(wifien)              ; 1 = yes, 0 = no
        call    pdec8
wifien_x:
...
rq_wifien:
        FNREQ   DEV_FUJINET, FUJI_GET_WIFI_ENABLED, F_NONE, 0,0,0,0
        FNBUF   0, 0, wifien, 1
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* true if it is switched on; a call that fails returns the
   last answer, not false */
ok = fuji_get_wifi_enabled();
printf("WiFi %s\n", ok ? "on" : "off");
```

H89 → FUJINET

C0	70	EA	06	00	61	00	C0

FUJINET → H89 · ACK · 1 BYTE

C0	70	06	07	00	7E	00	01	C0

SCAN FOR NETWORKS — FDH

DEVICE 70H (160Q) COMMAND FDH (375Q) FUJI_SCAN_NETWORKS

STAND-IN WIFI

FIELDS	none
PAYLOAD	none
REPLY	1 byte: how many networks were found

Looks for WiFi networks within reach, and answers how many it found. FUJI_GET_SCAN_RESULT then gives them one at a time. On the card, the scan takes a few seconds, and drops the FujiNet's own connection while it looks: don't scan while network channels are open.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_scan              ; look for networks
        call    fn_call
        jr      nz,scan_x
        ld      a,(nnets)               ; how many were found
        call    pdec8
scan_x:
...
rq_scan:
        FNREQ   DEV_FUJINET, FUJI_SCAN_NETWORKS, F_NONE, 0,0,0,0
        FNBUF   0, 0, nnets, 1
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_scan_for_networks(&count);
if (ok)
    printf("%u networks\n", count);
```

H89 → FUJINET

C0	70	FD	06	00	74	00	C0

FUJINET → H89 · ACK · 1 BYTE

C0	70	06	07	00	8D	00	10	C0

GET SCAN RESULT — FCH

DEVICE 70H (160Q) COMMAND FCH (374Q) FUJI_GET_SCAN_RESULT

STAND-IN WIFI

FIELDS	1 byte: which network, from 0
PAYLOAD	none
REPLY	34 bytes: the network's name, then its signal strength

One network from the last scan:

OFFSET	BYTES	HOLDS
0	33	The network's name, padded with zeros.
33	1	Its signal strength in dBm, a signed byte: –48 (D0H) is strong, –84 weak.

NAK if the number is not less than the count the last scan gave, or if there has been no scan.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_scanres           ; the first network found
        call    fn_call
        jr      nz,scanres_x
        ld      hl,net_ssid             ; its name (its signal follows,
        call    print                   ;   in dBm, a signed byte)
scanres_x:
...
rq_scanres:                             ; which one: 0 to the count - 1
        FNREQ   DEV_FUJINET, FUJI_GET_SCAN_RESULT, F_A1, 0,0,0,0
        FNBUF   0, 0, net, 34
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_scan_result(0, &scan);    /* the first found */
if (ok)                             /* (and scan.rssi, in dBm) */
    printf("%s\n", scan.ssid);
```

H89 → FUJINET

C0	70	FC	07	00	75	01	00	C0

FUJINET → H89 · ACK · 34 BYTES

C0	70	06	28	00	8D	00	44 D	75 u	6D m	6D m	79 y	20	43 C	61 a	66 f
65 e	20	30 0	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	FF	C0						

GET SSID — FEH

DEVICE 70H (160Q) COMMAND FEH (376Q) FUJI_GET_SSID

FIELDS none
PAYLOAD none
REPLY 97 bytes: the saved network name, then its password

The network the FujiNet is set to join, and its password, as saved:

OFFSET	BYTES	HOLDS
0	33	The network's name, padded with zeros.
33	64	Its password, in plain text, padded with zeros.

CAUTION: The password comes back as it is. Don't show it on the screen, or keep it in a file.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_getssid           ; the network it is set to join
        call    fn_call
        jr      nz,getssid_x
        ld      hl,ssid                 ; (the passphrase follows it, in
        call    print                   ;   the clear)
getssid_x:
...
rq_getssid:
        FNREQ   DEV_FUJINET, FUJI_GET_SSID, F_NONE, 0,0,0,0
        FNBUF   0, 0, ssid, 97
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_ssid(&net);          /* and net.password, in */
if (ok)                            /* the clear */
    printf("%s\n", net.ssid);
```

H89 → FUJINET

C0	70	FE	06	00	75	00	C0

FUJINET → H89 · ACK · 97 BYTES

C0	70	06	67	00	59	00	44 D	75 u	6D m	6D m	79 y	20	43 C	61 a	66 f

65 e	00	00	00	00	00	00	00	00	00	00	77 MORE	C0

SET SSID — FBH

DEVICE 70H (160Q) COMMAND FBH (373Q) FUJI_SET_SSID

STAND-IN WIFI

FIELDS 1 byte: 0 (not used, but send it)
PAYLOAD 97 bytes: network name (33), password (64), each ending in a zero
REPLY ACK

Saves a new network and password, and joins it. The payload has the layout of FUJI_GET_SSID's reply; send all 97 bytes, with zeros after each string, because a short payload leaves the rest of the FujiNet's copy as it was in memory.

ACK means the FujiNet has started to join, not that it has joined: a wrong password is accepted and saved. Wait a few seconds, then ask FUJI_GET_WIFISTATUS.

WARNING: If the card's radio refuses at once, the FujiNet sends **two** NAKs. Take the whole of the second one before the next request (the book's `fn_call` does). fujinet-pc's stand-in network always accepts, so the example shows an ACK.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_setssid           ; join a network, and keep it
        call    fn_call
...
rq_setssid:                             ; (the field is not used)
        FNREQ   DEV_FUJINET, FUJI_SET_SSID, F_A1, 0,0,0,0
        FNBUF   p_netcfg, 97, 0, 0
p_netcfg:
        defm    "Dummy Cafe"            ; the SSID, 33 bytes
        defs    33-10
        defm    "swordfish"             ; the passphrase, 64
        defs    64-9
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
memset(&net, 0, sizeof net);       /* join a network; keep it */
strcpy(net.ssid, "Dummy Cafe");
strcpy(net.password, "swordfish");
ok = fuji_set_ssid(&net);
```

H89 → FUJINET

C0	70	FB	68	00	51	01	00	44 D	75 u	6D m	6D m	79 y	20	43 C	61 a

66 f	65 e	00	00	00	00	00	00	00	00	00	78 MORE	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

HOST SLOTS

The FujiNet keeps eight **host slots**, numbered 0 to 7 on the bus (CONFIG shows them as 1 to 8). Each holds the name of a place disk images come from: `SD` for the microSD card, a TNFS server's name, or a URL such as `smb://den-pc/images`. A host is **mounted** before its directories can be read or its images used.

READ HOST SLOTS F4H

DEVICE 70H (160Q) COMMAND F4H (364Q) FUJI_READ_HOST_SLOTS

FIELDS	none
PAYLOAD	none
REPLY	256 bytes: 8 names of 32 bytes

All eight host names, each in 32 bytes padded with zeros, slot 0 first. An empty slot is all zeros.

WARNING: With fujinet-lib, always ask for all eight slots (`fuji_get_host_slots(slots, 8)`). The reply is 256 bytes whatever you ask for, and a smaller buffer makes the call fail.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_hosts             ; the eight host slots
        call    fn_call
        jr      nz,hosts_x
        ld      hl,hosts                ; 32 bytes each, NUL-padded
        ld      b,8
hosts_1:
        push    hl
        call    print                   ; (an empty one prints nothing)
        call    pspace
        pop     hl
        ld      de,32
        add     hl,de
        djnz    hosts_1
hosts_x:
...
rq_hosts:
        FNREQ   DEV_FUJINET, FUJI_READ_HOST_SLOTS, F_NONE, 0,0,0,0
        FNBUF   0, 0, hosts, 256
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_host_slots(hosts, 8);     /* all eight, or it fails */
if (ok)
    for (i = 0; i < 8; i++)
        printf("%s ", (char *)hosts[i]);
```

H89 → FUJINET

C0	70	F4	06	00	6B	00	C0

FUJINET → H89 · ACK · 256 BYTES

C0	70	06	06	01	5A	00	53 S	44 D	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	236 MORE				C0

WRITE HOST SLOTS F3H

DEVICE 70H (160Q) COMMAND F3H (363Q) FUJI_WRITE_HOST_SLOTS

FIELDS	none
PAYLOAD	256 bytes: 8 names of 32
REPLY	ACK

Replaces all eight host names and saves them. A slot whose name changes is unmounted, and so are the others. The usual way to change one slot is to read all eight, change one, and write them back, as the example does.

Send all 256 bytes: the FujiNet does not clear its buffer first, so a short payload fills the last slots with whatever was in its memory.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,p_host8              ; a name in slot 8
        ld      de,hosts+7*32
        ld      bc,p_host8n
        ldir
        ld      hl,rq_whosts            ; and all eight go back
        call    fn_call
...
rq_whosts:                              ; 256 bytes, all eight slots
        FNREQ   DEV_FUJINET, FUJI_WRITE_HOST_SLOTS, F_NONE, 0,0,0,0
        FNBUF   hosts, 256, 0, 0
p_host8:
        defm    "localhost", 0
p_host8n        equ     $-p_host8
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
strcpy((char *)hosts[7], "localhost");  /* a name in slot 8 */
ok = fuji_put_host_slots(hosts, 8);     /* and all eight go back */
```

H89 → FUJINET

C0	70	F3	06	01	15	00	53 S	44 D	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	236 MORE				C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

MOUNT HOST F9H

DEVICE 70H (160Q) COMMAND F9H (371Q) FUJI_MOUNT_HOST

FIELDS	1 byte: the host slot, 0 to 7
PAYLOAD	none
REPLY	ACK

Connects to the host in a slot, so its directories can be read and its images mounted. For a TNFS or other network host this reaches out over the network, and may take a moment; for `SD` it is immediate. NAK if the slot is empty, or the host cannot be reached.

Once a slot has been mounted, mounting it again does nothing, even if the server has gone away since.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_mounth            ; mount host slot 0, the SD card
        call    fn_call
...
rq_mounth:                              ; host slot 0
        FNREQ   DEV_FUJINET, FUJI_MOUNT_HOST, F_A1, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_mount_host_slot(0);       /* host slot 0, the SD card */
```

H89 → FUJINET

C0	70	F9	07	00	72	01	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

UNMOUNT HOST E6H

DEVICE 70H (160Q) COMMAND E6H (346Q) FUJI_UNMOUNT_HOST

NAK

FIELDS	1 byte: the host slot
PAYLOAD	none
REPLY	ACK, or NAK

Disconnects from a host, first unmounting every drive slot that uses it.

WARNING: The SD host cannot be unmounted: the FujiNet answers NAK — but only **after** it has emptied every drive slot that used the SD card. The example shows it. Only a host mounted with FUJI_MOUNT_HOST can be unmounted; one mounted by FUJI_MOUNT_ALL or FUJI_COPY_FILE NAKs.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_unmounth          ; let go of host slot 0 (the SD
        call    fn_call                 ;   card NAKs: it stays mounted)
...
rq_unmounth:                            ; host slot 0
        FNREQ   DEV_FUJINET, FUJI_UNMOUNT_HOST, F_A1, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_unmount_host_slot(0);     /* the SD card NAKs */
```

H89 → FUJINET

C0	70	E6	07	00	5F	01	00	C0

FUJINET → H89 · NAK

C0	70	15	06	00	8B	00	C0

SET HOST PREFIX E1H

DEVICE 70H (160Q) COMMAND E1H (341Q) FUJI_SET_HOST_PREFIX

FIELDS	1 byte: the host slot
PAYLOAD	a directory, ending in a zero (at most 256 bytes)
REPLY	ACK

Sets a host's **current directory**, which the FujiNet puts in front of every path used on that host afterwards (directory listings, image names). A path beginning with / replaces the prefix; any other is added to it, like `cd` on a modern computer; an empty one clears it. The prefix is not saved, and is lost when the FujiNet restarts.

Send the zero after the path: the FujiNet does not clear its buffer.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_setpfx            ; paths on host slot 0 now start
        call    fn_call                 ;   in /test
...
rq_setpfx:                              ; host slot 0
        FNREQ   DEV_FUJINET, FUJI_SET_HOST_PREFIX, F_A1, 0,0,0,0
        FNBUF   p_pfx, p_pfxn, 0, 0
p_pfx:  defm    "/test", 0              ; send the NUL too
p_pfxn          equ     $-p_pfx
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
strcpy(prefix, "/test");            /* host slot 0's paths start */
ok = fuji_set_host_prefix(0, prefix);   /* here (256 bytes go) */
```

H89 → FUJINET

C0	70	E1	0D	00	51	01	00	2F /	74 t	65 e	73 s	74 t	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

GET HOST PREFIX E0H

DEVICE 70H (160Q) COMMAND E0H (340Q) FUJI_GET_HOST_PREFIX

FIELDS	1 byte: the host slot
PAYLOAD	none
REPLY	256 bytes: the prefix, ending in a zero

A host's current prefix. Everything after the zero is left over from the FujiNet's memory, as the example's bytes show: stop at the zero.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_getpfx            ; host slot 0's prefix
        call    fn_call
        jr      nz,getpfx_x
        ld      hl,prefix               ; NUL-terminated
        call    print
getpfx_x:
...
rq_getpfx:                              ; host slot 0
        FNREQ   DEV_FUJINET, FUJI_GET_HOST_PREFIX, F_A1, 0,0,0,0
        FNBUF   0, 0, prefix, 256
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_host_prefix(0, prefix);   /* 256 bytes come back */
if (ok)
    printf("%s\n", prefix);
```

H89 → FUJINET

C0	70	E0	07	00	59	01	00	C0

FUJINET → H89 · ACK · 256 BYTES

C0	70	06	06	01	B6	00	2F /	74 t	65 e	73 s	74 t	00	00	00	35 5

00	00	00	00	00	00	00	B0	66 f	67 g	35 5	236 MORE	C0

SET HOST PREFIX (CLEARING IT) E1H

DEVICE 70H (160Q) COMMAND E1H (341Q)

An empty path clears the prefix. The example sends just the zero.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_clrpfx            ; no prefix: paths start at the
        call    fn_call                 ;   top again
...
rq_clrpfx:                              ; host slot 0: just a NUL
        FNREQ   DEV_FUJINET, FUJI_SET_HOST_PREFIX, F_A1, 0,0,0,0
        FNBUF   p_nopfx, 1, 0, 0
p_nopfx:
        defb    0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
memset(prefix, 0, sizeof prefix);   /* no prefix: from the top */
ok = fuji_set_host_prefix(0, prefix);
```

H89 → FUJINET

C0	70	E1	08	00	5B	01	00	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

DIRECTORIES

A program can list a mounted host's directories, a name at a time, as CONFIG does. One directory may be open at a time on the whole FujiNet; opening another closes the first.

OPEN DIRECTORY F7H

DEVICE 70H (160Q) COMMAND F7H (367Q) FUJI_OPEN_DIRECTORY

FIELDS	1 byte: the host slot
PAYLOAD	the path, a zero, and optionally a pattern and a zero
REPLY	ACK

Opens a directory on a mounted host for FUJI_READ_DIR_ENTRY. The path is relative to the host's prefix. After the path's zero may come a pattern such as `*.H8D`, which only names matching it are listed; without one, every name is. NAK if the host is not mounted (FUJI_MOUNT_HOST first), or the directory cannot be opened.

WARNING: The FujiNet reads whatever follows the path's zero as the pattern, up to the end of the payload. Send the path and its zero and no more (as the Z80 example does), or clear the rest of the buffer first: fujinet-lib's `fuji_open_directory()` always sends 256 bytes, so the C example clears a 256-byte buffer before copying the path into it. `fuji_open_directory_filter()` with an empty filter sends the caller's memory after the path, and the FujiNet takes that as the pattern.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_opendir           ; open /test/ on host slot 0
        call    fn_call
...
rq_opendir:                             ; host slot 0
        FNREQ   DEV_FUJINET, FUJI_OPEN_DIRECTORY, F_A1, 0,0,0,0
        FNBUF   p_dir, p_dirn, 0, 0
p_dir:  defm    "/test/", 0             ; the path; a wildcard pattern
p_dirn          equ     $-p_dir         ;   and a NUL may follow
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* the path, a NUL, then a pattern if wanted: fujinet-lib sends
   all 256 bytes of the buffer, so clear it first */
memset(path, 0, sizeof path);
strcpy(path, "/test/");
ok = fuji_open_directory(0, path);  /* on host slot 0 */
```

H89 → FUJINET

C0	70	F7	0E	00	97	01	00	2F /	74 t	65 e	73 s	74 t	2F /	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

READ DIRECTORY ENTRY F6H

DEVICE 70H (160Q) COMMAND F6H (366Q) FUJI_READ_DIR_ENTRY

FIELDS	2 bytes: the reply's length, 1 to 255; 0 (or 80H, below)
PAYLOAD	none
REPLY	that many bytes: the next name

The next name in the open directory, padded with zeros to the length asked for. A directory's name ends in / if there is room. At the end of the directory the reply starts with 7FH 7FH. The example reads the `test` directory's three names and then the end; here are the first and the last:

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
dir_1:
        ld      hl,rq_readdir           ; the next name, padded to 32
        call    fn_call
        jr      nz,dir_x
        ld      a,(dirent)              ; 7FH 7FH: there are no more
        cp      0x7F
        jr      z,dir_x
        ld      hl,dirent
        call    print
        call    pspace
        jr      dir_1
dir_x:
...
rq_readdir:                             ; 32 bytes; no details
        FNREQ   DEV_FUJINET, FUJI_READ_DIR_ENTRY, F_A1_A2, 32,0,0,0
        FNBUF   0, 0, dirent, 32
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
for (;;) {
    ok = fuji_read_directory(32, 0, name);  /* padded to 32 */
    if (!ok || name[0] == FUJI_DIR_EOF)     /* 7FH: no more */
        break;
    printf("%s ", name);
}
```

H89 → FUJINET

C0	70	F6	08	00	91	02	20	00	C0

FUJINET → H89 · ACK · 32 BYTES

C0	70	06	26	00	D9	00	62 b	6C l	61 a	6E n	6B k	2E .	68 h	38 8	64 d
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	C0								

H89 → FUJINET

C0	70	F6	08	00	91	02	20	00	C0

FUJINET → H89 · ACK · 32 BYTES

C0	70	06	26	00	9B	00	7F	7F	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	C0								

READ DIRECTORY ENTRY, WITH DETAILS F6H

DEVICE 70H (160Q) COMMAND F6H (366Q)

FIELDS	2 bytes: the length, at least 14; 80H
REPLY	a 12-byte header, then the name

With 80H in the second field, each name comes after a header of details:

OFFSET	BYTES	HOLDS
0	1	Year the file was changed, counting from 1970.
1	1	Month, 1 to 12.
2	1	Day, 1 to 31.
3	3	Hour, minute, second (the FujiNet's local time).
6	4	The file's size in bytes, low byte first.
10	1	Flags: 01H a directory, 02H the name was cut short.
11	1	The kind of image: 0 unknown (including .IMG), 2 a ROM, 3 an .IMD.
12		The name, ending in a zero.

WARNING: Ask for at least 14 bytes with 80H. The FujiNet takes 12 from the length for the header in an

8-bit sum, so a smaller length wraps round and the reply is cut off in the middle of the header. And never send C0H in the second field: it asks for a “block mode” the H89 cannot take, and with no entries per block the firmware divides by zero and stops.

NOTE: The example ran on fujinet-pc, which (unlike the card) cannot find the details of a file in a directory below the top of its SD card: its header reads 1969-12-31, size 0. On the card, the date and size are those of the file.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_readdirx          ; the next entry, 12 bytes of
        call    fn_call                 ;   details before its name
        jr      nz,dirx_x
        ld      hl,dirx_name
        call    print
        call    pspace
        ld      hl,(dirx_size)          ; its size, a long: low word
        call    pdec16
dirx_x:
...
rq_readdirx:                            ; 64 bytes; 80H: details first
        FNREQ   DEV_FUJINET, FUJI_READ_DIR_ENTRY, F_A1_A2, 64,0x80,0,0
        FNBUF   0, 0, dirx, 64
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* 12 bytes of details before the name. FUJI_DIR_ENTRY_IS_DIR()
   does not fit the H89's FujiDirEntry: test flags bit 0 */
ok = fuji_read_directory(64, FUJI_DIR_FLAG_ADDITIONAL_DATA, &ent);
if (ok)
    printf("%s %lu%s\n", ent.filename, ent.size,
           ent.flags & 0x01 ? " (directory)" : "");
```

H89 → FUJINET

C0	70	F6	08	00	32	02	40	80	C0

FUJINET → H89 · ACK · 64 BYTES

C0	70	06	46	00	13	00	FF	0C	1F	12	00	00	00	00	00

00	00	00	64 d	69 i	73 s	6B k	2E .	69 i	6D m	67 g	44 MORE	C0

GET DIRECTORY POSITION E5H

DEVICE 70H (160Q) COMMAND E5H (345Q) FUJI_GET_DIRECTORY_POSITION

FIELDS	none
PAYLOAD	none
REPLY	2 bytes: the entry number, low first

Where the next FUJI_READ_DIR_ENTRY will read from, counting from 0. CONFIG keeps it to go back a page. NAK if no directory is open.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_getpos            ; how far through are we?
        call    fn_call
        jr      nz,getpos_x
        ld      hl,(dirpos)             ; a word: the next entry
        call    pdec16
getpos_x:
...
rq_getpos:
        FNREQ   DEV_FUJINET,FUJI_GET_DIRECTORY_POSITION,F_NONE,0,0,0,0
        FNBUF   0, 0, dirpos, 2
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_directory_position(&pos);
if (ok)                         /* the next entry's number */
    printf("%u\n", pos);
```

H89 → FUJINET

C0	70	E5	06	00	5C	00	C0

FUJINET → H89 · ACK · 2 BYTES

C0	70	06	08	00	81	00	03	00	C0

SET DIRECTORY POSITION E4H

DEVICE 70H (160Q) COMMAND E4H (344Q) FUJI_SET_DIRECTORY_POSITION

FIELDS	a word: the entry number
PAYLOAD	none
REPLY	ACK

Moves to an entry, so the next read returns it. The example goes back to entry 1.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,1                    ; on to the second entry (they
        ld      (rq_setpos+RQ_AUX1),hl  ;   count from 0)
        ld      hl,rq_setpos
        call    fn_call
...
rq_setpos:                              ; the entry's number, a word
        FNREQ   DEV_FUJINET, FUJI_SET_DIRECTORY_POSITION, F_B12, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_set_directory_position(1);    /* on to the second */
```

H89 → FUJINET

C0	70	E4	08	00	63	05	01	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

CLOSE DIRECTORY F5H

DEVICE 70H (160Q) COMMAND F5H (365Q) FUJI_CLOSE_DIRECTORY

FIELDS	none
PAYLOAD	none
REPLY	ACK

Closes the open directory, if there is one.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_closedir          ; done with the directory
        call    fn_call
...
rq_closedir:
        FNREQ   DEV_FUJINET, FUJI_CLOSE_DIRECTORY, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_close_directory();
```

H89 → FUJINET

C0	70	F5	06	00	6C	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

DRIVE SLOTS

The FujiNet has eight **drive slots**, numbered 0 to 7 on the bus, which are the disk drives of Chapter 9: slot 0 is device 31H, slot 7 is 38H. Each names an image (a host slot and a path) and how it may be used (read only, or read and write). Putting an image in a drive is two steps: name it with FUJI_SET_DEVICE_FULLPATH, then **mount** it with FUJI_MOUNT_IMAGE. FNDISK and FN.DVD then see it as a drive.

CAUTION: Changing a drive slot that FNDISK or FN.DVD is using changes the drive under it. Unmount the drive in HDOS or CP/M first, or warn the user.

READ DRIVE SLOTS F2H

DEVICE 70H (160Q) COMMAND F2H (362Q) FUJI_READ_DEVICE_SLOTS

FIELDS	none
PAYLOAD	none
REPLY	304 bytes: 8 slots of 38

All eight drive slots, slot 0 first, each fujinet-lib's `DeviceSlot`:

OFFSET	BYTES	HOLDS
0	1	The host slot the image is on; FFH if the drive is empty.
1	1	Mode: 01H read only, 02H read and write; 40H added while the image is mounted.
2	36	The image's name, without its directory, ending in a zero.

NOTE: Stop at the zero in each name: the bytes after it may be the end of the full path, left over, as in slot 0 of the example (`disk.img`, a zero, then `k.img`). An empty slot reads mode 01H.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_dslots            ; the eight drive slots, of 38
        call    fn_call                 ;   bytes each
        jr      nz,dslots_x
        ld      a,(slots+DS_HOST)       ; slot 0's host (FFH: empty)
        call    pdec8
        call    pspace
        ld      a,(slots+DS_MODE)       ; its mode (40H: mounted)
        call    phex8
        call    pspace
        ld      hl,slots+DS_NAME        ; and its file's name
        call    print
dslots_x:
...
rq_dslots:
        FNREQ   DEV_FUJINET, FUJI_READ_DEVICE_SLOTS, F_NONE, 0,0,0,0
        FNBUF   0, 0, slots, 304
DS_HOST         equ     0               ; a slot: its host slot,
DS_MODE         equ     1               ;   its mode
DS_NAME         equ     2               ;   and 36 bytes of file name
```

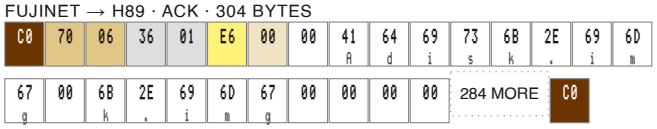
C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_device_slots(slots, 8);   /* all eight, or it fails */
if (ok)                     /* host FFH: empty; mode 40H: mounted */
    printf("%u %02X %s\n", slots[0].hostSlot, slots[0].mode,
           (char *)slots[0].file);
```

H89 → FUJINET



FUJINET → H89 · ACK · 304 BYTES



WRITE DRIVE SLOTS F1H

DEVICE 70H (160Q) COMMAND F1H (361Q) FUJI_WRITE_DEVICE_SLOTS

FIELDS	none
PAYLOAD	304 bytes, as Read Drive Slots
REPLY	ACK

Sets each slot's host and mode, and saves them. The names in the payload are **ignored**: only FUJI_SET_DEVICE_FULLPATH sets a path. The example reads the slots, changes slot 0 to read and write, and writes them back.

WARNING: Send all 304 bytes (the FujiNet does not clear its buffer), and only host slots 0 to 7 or FFH: the firmware does not check, and FUJI_MOUNT_ALL then reads past the end of its host table. A host of FFH also drops the slot's path from the saved settings.

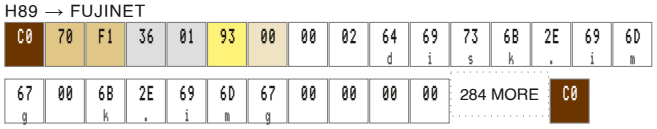
Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      a,2                     ; slot 0 read/write from now on
        ld      (slots+DS_MODE),a
        ld      hl,rq_wdslots           ; all eight go back (only host
        call    fn_call                 ;   and mode are used)
...
rq_wdslots:                             ; 304 bytes, all eight slots
        FNREQ   DEV_FUJINET, FUJI_WRITE_DEVICE_SLOTS, F_NONE, 0,0,0,0
        FNBUF   slots, 304, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
slots[0].mode = 2;                  /* slot 0 read/write from now */
ok = fuji_put_device_slots(slots, 8);   /* (names are not used) */
```

H89 → FUJINET



FUJINET → H89 · ACK



SET DRIVE SLOT PATH E2H

DEVICE 70H (160Q) COMMAND E2H (342Q) FUJI_SET_DEVICE_FULLPATH

FIELDS	3 bytes: drive slot, host slot, mode (01H read, 02H read/write)
PAYLOAD	the image's path, ending in a zero
REPLY	ACK

Names the image for a drive slot, and saves it. It does not mount it, or check that it exists.

WARNING: An empty path does not quite empty the slot. It sets the slot's host to FFH, but the old path stays, and FUJI_GET_DEVICE_FULLPATH still returns it. A host slot of FFH in the request — what FUJI_READ_DEVICE_SLOTS reports for an empty slot — is refused with NAK, even with an empty path.

To empty a slot, use FUJI_UNMOUNT_IMAGE (below). fujinet-lib's own test program puts an empty slot back with this command, and fails.

NOTE: fujinet-lib's `fuji_set_device_filename(mode, hs, ds, path)` takes its arguments in a different order from the fields: mode, host, drive. It sends 256 bytes; clear the buffer first.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_setpath           ; /test/disk.img into drive
        call    fn_call                 ;   slot 0 (not mounted yet)
...
rq_setpath:                             ; drive slot, host slot, mode
        FNREQ   DEV_FUJINET,FUJI_SET_DEVICE_FULLPATH,F_A1_A2_A3,0,0,1,0
        FNBUF   p_img, p_imgn, 0, 0     ; (mode 1: read, 2: read/write)
p_img:  defm    "/test/disk.img", 0
p_imgn          equ     $-p_img
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
memset(path, 0, sizeof path);       /* 256 bytes go */
strcpy(path, "/test/disk.img");
/* mode 1 (read), host slot 0, drive slot 0: not the order
   they go on the wire (drive, host, mode) */
ok = fuji_set_device_filename(1, 0, 0, path);
```

H89 → FUJINET

C0	70	E2	18	00	A8	03	00	00	01	2F /	74 t	65 e	73 s	74 t	2F /
64 d	69 i	73 s	6B k	2E .	69 i	6D m	67 g	00	C0						

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

GET DRIVE SLOT PATH — DAH

DEVICE 70H (160Q) COMMAND DAH (332Q) FUJI_GET_DEVICE_FULLPATH

FIELDS	1 byte: the drive slot
PAYLOAD	none
REPLY	256 bytes: the full path, padded with zeros

The whole path of a drive slot's image. After FUJI_MOUNT_IMAGE, it includes the host's prefix.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_getpath           ; what is in drive slot 0?
        call    fn_call
        jr      nz,getpath_x
        ld      hl,path
        call    print
getpath_x:
...
rq_getpath:                             ; drive slot 0
        FNREQ   DEV_FUJINET, FUJI_GET_DEVICE_FULLPATH, F_A1, 0,0,0,0
        FNBUF   0, 0, path, 256
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_device_filename(0, path);     /* drive slot 0 */
if (ok)
    printf("%s\n", path);
```

H89 → FUJINET

C0	70	DA	07	00	53	01	00	C0

FUJINET → H89 · ACK · 256 BYTES

C0	70	06	06	01	B6	00	2F /	74 t	65 e	73 s	74 t	2F /	64 d	69 i	73 s
6B k	2E .	69 i	6D m	67 g	00	00	00	00	00	00	236 MORE				C0

MOUNT IMAGE — F8H

DEVICE 70H (160Q) COMMAND F8H (370Q) FUJI_MOUNT_IMAGE

FIELDS	2 bytes: the drive slot; the mode (01H read, 02H read and write)
PAYLOAD	none
REPLY	ACK

Opens the image a drive slot names and puts it in the drive (device 31H plus the slot). The slot's host must be mounted first: FUJI_MOUNT_IMAGE does not do it. NAK if the slot is empty, the file cannot be opened, or read and write is asked of a file that cannot be written.

WARNING: Don't mount a file whose name ends in ROM or BIN (or INT, ITV, CHF, NES, SMS, A78, A52 or ST2). The FujiNet treats it as a cartridge for another computer, and before it answers it sends the whole file to device FFH, in frames the H89 does not expect, ahead of its reply. The H89's next replies are then out of step. An H89 disk image named with BIN at the end will meet this.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_mount             ; mount it, read-only: it is now
        call    fn_call                 ;   the disk on device 31H
...
rq_mount:                               ; drive slot 0; mode 1: read
        FNREQ   DEV_FUJINET, FUJI_MOUNT_IMAGE, F_A1_A2, 0,1,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_mount_disk_image(0, 1);   /* drive slot 0, read-only */
```

H89 → FUJINET

C0	70	F8	08	00	74	02	00	01	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

UNMOUNT IMAGE — E9H

DEVICE 70H (160Q) COMMAND E9H (351Q) FUJI_UNMOUNT_IMAGE

FIELDS	1 byte: the drive slot
PAYLOAD	none
REPLY	ACK

Takes the image out of a drive, and empties the slot: its path is forgotten, and the settings are saved without it. To use the image again, name it again.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_unmount           ; empty drive slot 0
        call    fn_call
...
rq_unmount:                             ; drive slot 0
        FNREQ   DEV_FUJINET, FUJI_UNMOUNT_IMAGE, F_A1, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_unmount_disk_image(0);    /* empty drive slot 0 */
```

H89 → FUJINET

C0	70	E9	07	00	62	01	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

MOUNT ALL D7H

DEVICE 70H (160Q) COMMAND D7H (327Q) FUJI_MOUNT_ALL

FIELDS none
PAYLOAD none
REPLY ACK, or NAK at the first failure

Mounts every drive slot that names an image, mounting its host first, as the FujiNet does at power on. It stops at the first slot that fails, and NAKs; the slots after it are not tried. In the example there is nothing left to mount (slot 0 was emptied above), so it simply answers ACK.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_mountall          ; mount whatever the drive slots
        call    fn_call                 ;   name
...
rq_mountall:
        FNREQ   DEV_FUJINET, FUJI_MOUNT_ALL, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_mount_all();              /* whatever the slots name */
```

H89 → FUJINET

C0	70	D7	06	00	4E	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

NEW DISK E7H

DEVICE 70H (160Q) COMMAND E7H (347Q) FUJI_NEW_DISK

NAK

FIELDS none
PAYLOAD 262 bytes: sectors (word), sector size (word), host slot, drive slot, path (256)
REPLY NAK, always

Meant to make a new, empty disk image and put it in a drive.

WARNING: On the H89 this always fails: the firmware's image code cannot create images for this computer, and answers NAK. Worse, before it does, it creates an **empty** file of that name on the host and names it in the drive slot. Delete the file, and empty the slot with FUJI_UNMOUNT_IMAGE, after trying. To make a blank disk, copy one: `test/blank.h8d` in the example is one (FUJI_COPY_FILE, next).

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_newdisk           ; make a new, blank disk image
        call    fn_call                 ;   (this firmware always NAKs)
...
rq_newdisk:
        FNREQ   DEV_FUJINET, FUJI_NEW_DISK, F_NONE, 0,0,0,0
        FNBUF   p_newdisk, 262, 0, 0
p_newdisk:
        defw    400                     ; sectors,
        defw    256                     ;   of 256 bytes
        defb    0                       ; host slot
        defb    1                       ; drive slot
        defm    "/test/new.h8d", 0      ; the name, in 256 bytes
        defs    256-14
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
nd.numSectors = 400;                /* of 256 bytes */
nd.sectorSize = 256;
nd.hostSlot = 0;
nd.deviceSlot = 1;
strcpy(nd.filename, "/test/new.h8d");
ok = fuji_create_new(&nd);          /* this firmware always NAKs */
```

H89 → FUJINET

C0	70	E7	0C	01	97	00	90	01	00	01	00	01	2F /	74 t	65 e

73 s	74 t	2F /	6E n	65 e	77 w	2E .	68 h	38 8	64 d	00	242 MORE	C0

FUJINET → H89 · NAK

C0	70	15	06	00	8B	00	C0

COPY FILE D8H

DEVICE 70H (160Q) COMMAND D8H (330Q) FUJI_COPY_FILE

FIELDS 2 bytes: source host slot, destination host slot, **counting from 1**
PAYLOAD `source|destination`
REPLY ACK, when the copy is done

Copies a file from one host to another, or within one, entirely inside the FujiNet. The H89 waits for the ACK until the copy is done. The payload is the two paths with a `|` between; if the destination ends in `/`, the source's name is added to it.

WARNING: The host slots count from 1 here, unlike every other command. And the copy fails if either host has a prefix longer than / (FUJI_SET_HOST_PREFIX): clear the prefixes first. The FujiNet does not check that everything was copied.

WARNING: A large copy can take longer than the 15 seconds that fujinet-lib and the book's library wait for a reply (Chapter 2). Copying a 161,280-byte disk image from `tnfs.fujinet.online` took from 14.4 to 18.2 seconds in six runs, and four of them took more than 15. Each time, `fuji_copy_file()` returned `false` although the copy went on and finished, and its late ACK was taken as the reply to the program's next request, so every reply after it was one behind. For large files, raise the limit: `RX_FIRST` in FUJIBUS.INC, or `TIMEOUT_SLOW` in fujinet-lib's `bus/h89/fuji_bus_call.c` (and rebuild the library). With `TIMEOUT_SLOW` at 60 seconds, fujinet-lib's own copy test passed under both systems (Chapter 6).

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_copy              ; copy a file on the SD card
        call    fn_call
...
rq_copy:                                ; host slots 1 and 1: COPY_FILE
        FNREQ   DEV_FUJINET, FUJI_COPY_FILE, F_A1_A2, 1,1,0,0
        FNBUF   p_copy, p_copyn, 0, 0   ;   counts them from 1, not 0
p_copy: defm    "/test/readme.txt!/test/copy.txt"
p_copyn         equ     $-p_copy        ; (no NUL)
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* host slots count from 1 here. fujinet-lib sends 256 bytes:
   keep the spec in a cleared 256-byte buffer */
memset(path, 0, sizeof path);
strcpy(path, "/test/readme.txt!/test/copy.txt");
ok = fuji_copy_file(1, 1, path);
```

H89 → FUJINET

C0	70	D8	27	00	7D	02	01	01	2F	74	65	73	74	2F	72
									/	t	e	s	t	/	r
65	61	64	6D	65	2E	74	78	74	7C	2F	74	65	73	74	2F
e	a	d	m	e	.	t	x	t	!	/	t	e	s	t	/
63	6F	70	79	2E	74	78	74	C0							
c	o	p	y	.	t	x	t								

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

BOOTING

Two commands concern starting a computer from the FujiNet. Other computers use them; the H89 starts from its own diskette, and has little use for either.

SET BOOT MODE D6H

DEVICE 70H (160Q) COMMAND D6H (326Q) FUJI_SET_BOOT_MODE

FIELDS	1 byte: 0 to 3
PAYLOAD	none
REPLY	ACK

Puts a boot image in the first drive (device 31H) at once, read only: 0 the FujiNet's own `autorun.img`, 1 `/mount-and-boot.img`, 2 the FujiNet project's lobby, 3 `/hisioboot-fujinet.img`.

WARNING: On the H89, mode 0's image is an MS-DOS boot disk (the firmware for this bus is shared with the IBM PC's), which an H89 cannot read — and it takes the place of whatever FNDISK was using as drive 31H. Modes 1 and 3 need files the FujiNet does not have, and do nothing. Every mode answers ACK. FUJI_CONFIG_BOOT with 0 takes the image out again, as the example does next.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_bootmode          ; put the boot disk in drive 31H
        call    fn_call
...
rq_bootmode:                            ; 0: the firmware's autorun.img
        FNREQ   DEV_FUJINET, FUJI_SET_BOOT_MODE, F_A1, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_set_boot_mode(0);         /* autorun.img into drive 31H */
```

H89 → FUJINET

C0	70	D6	07	00	4F	01	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

CONFIG BOOT D9H

DEVICE 70H (160Q) COMMAND D9H (331Q) FUJI_CONFIG_BOOT

FIELDS	1 byte: 0 or 1
PAYLOAD	none
REPLY	ACK

Meant to turn the FujiNet's boot-to-CONFIG feature on and off. On the H89 it does one thing: with 0, when drive slot 0 is empty, it takes a boot image put there by FUJI_SET_BOOT_MODE out of drive 31H. Otherwise it does nothing at all, and saves nothing.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_cfgboot           ; and take it out again
        call    fn_call
...
rq_cfgboot:                             ; 0: off
        FNREQ   DEV_FUJINET, FUJI_CONFIG_BOOT, F_A1, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_set_boot_config(0);       /* and out again */
```

H89 → FUJINET

C0	70	D9	07	00	52	01	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

APP KEYS

An **app key** is a small file, up to 64 bytes, that the FujiNet keeps for a program on its microSD card: a high score, a player's name, a setting. Any program on any computer can read it back, which is how the games remember who you are. A key is named by three numbers:

- a **creator** (16 bits, not 0), for the person or group who wrote the program — choose one of your own, and keep to it;
- an **application** (8 bits), for the program;
- a **key** (8 bits), for which of the program's keys.

The examples use creator 4889H, application 01H, key 00H, which the FujiNet keeps in `/FujiNet/48890100.key` on its SD card. Each read or write is three requests: OPEN, then READ or WRITE, then CLOSE.

NOTE: App keys need the microSD card. Without one, OPEN answers NAK.

OPEN APP KEY

DCH

DEVICE 70H (160Q) COMMAND DCH (334Q) FUJI_OPEN_APPKEY

FIELDS	none
PAYLOAD	6 bytes: creator (word), application, key, mode, 0
REPLY	ACK

Chooses the key for the next READ or WRITE, and whether to read (mode 0) or write (mode 1). NAK if the creator is 0 or there is no SD card. fujinet-lib has no function for this alone (its `fuji_read_appkey()` and `fuji_write_appkey()` send it themselves), so the C example uses the bus call.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_akopen            ; open a key, to write it
        call    fn_call
...
rq_akopen:
        FNREQ   DEV_FUJINET, FUJI_OPEN_APPKEY, F_NONE, 0,0,0,0
        FNBUF   p_akey, 6, 0, 0
p_akey: defw    0x4889                  ; creator ("H89"), not 0
        defb    0x01                    ; the creator's app
        defb    0x00                    ; the app's key
        defb    1                       ; mode: 0 read, 1 write
        defb    0                       ; (reserved)
AK_MODE         equ     4               ; where the mode is
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* OPEN_APPKEY's six bytes */
static struct {
    uint16_t creator;           /* who made the app: 4889H, "H89" */
    uint8_t app;                /* which of the creator's apps */
    uint8_t key;                /* which of the app's keys */
    uint8_t mode;               /* 0 read, 1 write */
    uint8_t reserved;
} ak = { 0x4889, 1, 0, 1, 0 };
...
/* fujinet-lib opens a key only inside fuji_write_appkey() and
   fuji_read_appkey(); to open one by itself, use the bus */
ok = fuji_bus_call(FUJI_DEVICEID_FUJINET, FUJICMD_OPEN_APPKEY,
                   FUJI_FIELD_NONE, 0, 0, 0, 0, &ak, sizeof ak);
```

H89 → FUJINET

C0	70	DC	0C	00	2D	00	89	48 H	01	00	01	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

WRITE APP KEY

DEH

DEVICE 70H (160Q) COMMAND DEH (336Q) FUJI_WRITE_APPKEY

FIELDS	none
PAYLOAD	the key's bytes, up to 64
REPLY	ACK

Writes the key opened for writing, replacing what was there. The payload is the whole of the key; bytes past 64 are dropped. The key is closed afterwards: open it again to write again.

WARNING: A WRITE with no payload empties the key, and answers ACK.

fujinet-lib's `fuji_write_appkey()` opens the key itself (with the creator and application set by `fuji_set_appkey_details()`), so the C example's bytes show two exchanges, OPEN and WRITE.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_akwrite           ; up to 64 bytes; this closes
        call    fn_call                 ;   the key too
...
rq_akwrite:                             ; the payload is the key
        FNREQ   DEV_FUJINET, FUJI_WRITE_APPKEY, F_NONE, 0,0,0,0
        FNBUF   p_akdata, p_akdatan, 0, 0
p_akdata:
        defm    "HIGH SCORE 12345"
p_akdatan       equ     $-p_akdata
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
fuji_set_appkey_details(0x4889, 1, DEFAULT);    /* creator, app */
/* key 0: opens it to write, then writes (which closes it) */
ok = fuji_write_appkey(0, 16, (uint8_t *)"HIGH SCORE 12345");
```

H89 → FUJINET

C0	70	DE	16	00	44	00	48 H	49 I	47 G	48 H	20	53 S	43 C	4F O	52 R

45 E	20	31 1	32 2	33 3	34 4	35 5	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

OPEN APP KEY, TO READ

DCH

DEVICE 70H (160Q) COMMAND DCH (334Q)

To read a key back, open it again, now with mode 0.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        xor     a                       ; the same key, mode 0: to read
        ld      (p_akey+AK_MODE),a
        ld      hl,rq_akopen
        call    fn_call
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ak.mode = 0;                        /* the same key, to read */
ok = fuji_bus_call(FUJI_DEVICEID_FUJINET, FUJICMD_OPEN_APPKEY,
                   FUJI_FIELD_NONE, 0, 0, 0, 0, &ak, sizeof ak);
```

H89 → FUJINET

C0	70	DC	0C	00	2C	00	89	48 H	01	00	00	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

READ APP KEY DDH

DEVICE 70H (160Q) COMMAND DDH (335Q) FUJI_READ_APPKEY

FIELDS	none
PAYLOAD	none
REPLY	2-byte length (low first), then that many bytes

Reads the key opened for reading. The reply is the key's length and the key. A key that does not exist reads as length 0: this command never NAKs, so check the length. The key stays open, and may be read again.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_akread            ; a word, the length, then the
        call    fn_call                 ;   key's bytes
        jr      nz,akread_x
        ld      bc,(akey)
        ld      hl,akey+2
        call    ptext
akread_x:
...
rq_akread:                              ; a word and up to 64 bytes
        FNREQ   DEV_FUJINET, FUJI_READ_APPKEY, F_NONE, 0,0,0,0
        FNBUF   0, 0, akey, 66
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* key 0: opens it to read, then reads up to 64 bytes and their
   count (0 if there is no such key) */
ok = fuji_read_appkey(0, &keylen, key);
if (ok) {
    key[keylen] = 0;
    printf("%s\n", (char *)key);
}
```

H89 → FUJINET

C0	70	DD	06	00	54	00	C0

FUJINET → H89 · ACK · 18 BYTES

C0	70	06	18	00	7D	00	10	00	48 H	49 I	47 G	48 H	20	53 S	43 C

4F O	52 R	45 E	20	31 1	32 2	33 3	34 4	35 5	C0

CLOSE APP KEY DBH

DEVICE 70H (160Q) COMMAND DBH (333Q) FUJI_CLOSE_APPKEY

FIELDS	none
PAYLOAD	none
REPLY	ACK

Forgets the open key. fujinet-lib has no function for it. Notice the command byte, DBH, which is SLIP's ESC: it goes over the bus as the pair DB DD (Chapter 3). The exerciser closes the key after writing it, and again after reading it.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_akclose           ; done with the key
        call    fn_call
...
rq_akclose:
        FNREQ   DEV_FUJINET, FUJI_CLOSE_APPKEY, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* no fujinet-lib call for CLOSE_APPKEY */
ok = fuji_bus_call(FUJI_DEVICEID_FUJINET, FUJICMD_CLOSE_APPKEY,
                   FUJI_FIELD_NONE, 0, 0, 0, 0, NULL, 0);
```

H89 → FUJINET

C0	70	DB	DD	06	00	52	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

BASE64

Base64 writes any bytes as letters, digits, + and /, four characters for every three bytes, so they can travel where only text may: in e-mail, JSON, URLs. The FujiNet does the arithmetic. It works in four steps — put the input in, compute, ask the length, take the output — in one buffer that encoding and decoding share.

WARNING: There is one buffer for both directions, and no command to empty it. Take **all** the output of one job before starting the next, or what is left is put in front of the next job's input.

BASE64 ENCODE: INPUT D0H

DEVICE 70H (160Q) COMMAND D0H (320Q) FUJI_BASE64_ENCODE_INPUT

FIELDS	a word: how many bytes
PAYLOAD	that many bytes
REPLY	ACK

Adds bytes to the input. May be sent several times, to encode more than one request can carry. The example encodes the 12 characters "Heathkit H89".

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_b64in             ; some bytes to encode
        call    fn_call
...
rq_b64in:                               ; the length goes in a word
        FNREQ   DEV_FUJINET, FUJI_BASE64_ENCODE_INPUT, F_B12, 12,0,0,0
        FNBUF   p_b64, 12, 0, 0
p_b64:  defm    "Heathkit H89"
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_base64_encode_input("Heathkit H89", 12);
```

H89 → FUJINET

C0	70	D0	14	00	75	05	0C	00	48 H	65 e	61 a	74 t	68 h	6B k	69 i

74 t	20	48 H	38 8	39 9	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

BASE64 ENCODE: COMPUTE CFH

DEVICE 70H (160Q) COMMAND CFH (317Q) FUJI_BASE64_ENCODE_COMPUTE

FIELDS	none
PAYLOAD	none
REPLY	ACK

Encodes the input, replacing it with the text. The FujiNet adds a line feed after every 72 characters, and one at the end.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_b64enc            ; encode them
        call    fn_call
...
rq_b64enc:
        FNREQ   DEV_FUJINET, FUJI_BASE64_ENCODE_COMPUTE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_base64_encode_compute();
```

H89 → FUJINET

C0	70	CF	06	00	46	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

BASE64 ENCODE: LENGTH CEH

DEVICE 70H (160Q) COMMAND CEH (316Q) FUJI_BASE64_ENCODE_LENGTH

FIELDS	none
PAYLOAD	none
REPLY	4 bytes: the length, low first

How long the encoded text is, line feeds included: here 17, sixteen characters and a line feed.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_b64len            ; how long is the text? a long
        call    fn_call
        jr      nz,b64len_x
        ld      hl,(b64len)             ; (the low word will do)
        call    pdec16
b64len_x:
...
rq_b64len:
        FNREQ   DEV_FUJINET, FUJI_BASE64_ENCODE_LENGTH, F_NONE, 0,0,0,0
        FNBUF   0, 0, b64len, 4
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_base64_encode_length(&b64len);    /* an unsigned long */
if (ok)
    printf("%lu\n", b64len);
```

H89 → FUJINET

C0	70	CE	06	00	45	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	70	06	0A	00	91	00	11	00	00	00	C0

BASE64 ENCODE: OUTPUT CDH

DEVICE 70H (160Q) COMMAND CDH (315Q) FUJI_BASE64_ENCODE_OUTPUT

FIELDS	a word: how many bytes
PAYLOAD	none
REPLY	that many bytes

Takes text from the front of the buffer. NAK if more is asked for than is left. Take a long result in pieces.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,(b64len)             ; take all of it
        ld      (rq_b64out+RQ_AUX1),hl
        ld      hl,rq_b64out
        call    fn_call
        jr      nz,b64out_x
        ld      hl,b64buf               ; BC bytes, a newline last
        call    ptext
b64out_x:
...
rq_b64out:                              ; how many: set before the call
        FNREQ   DEV_FUJINET, FUJI_BASE64_ENCODE_OUTPUT, F_B12, 0,0,0,0
        FNBUF   0, 0, b64buf, 64
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_base64_encode_output(b64, (uint16_t)b64len);  /* all */
if (ok) {
    b64[(uint16_t)b64len] = 0;      /* it ends in a newline */
    printf("%s", b64);
}
```

H89 → FUJINET

C0	70	CD	08	00	5C	05	11	00	C0

FUJINET → H89 · ACK · 17 BYTES

C0	70	06	17	00	1E	00	53 S	47 G	56 V	68 h	64 d	47 G	68 h	72 r	61 a

58 X	51 Q	67 g	53 S	44 D	67 g	35 5	0A	C0

BASE64 DECODE: INPUT — CCH

DEVICE 70H (160Q) COMMAND CCH (314Q) FUJI_BASE64_DECODE_INPUT

FIELDS	a word: how many characters
PAYLOAD	the text
REPLY	ACK

Adds base64 text to be decoded. The example gives back the text just made.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,(b64len)             ; give the text back to decode
        ld      (rq_b64din+RQ_AUX1),hl
        ld      (rq_b64din+RQ_PLEN),hl
        ld      hl,rq_b64din
        call    fn_call
...
rq_b64din:                              ; how many: set before the call
        FNREQ   DEV_FUJINET, FUJI_BASE64_DECODE_INPUT, F_B12, 0,0,0,0
        FNBUF   b64buf, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_base64_decode_input(b64, (uint16_t)b64len);   /* back */
```

H89 → FUJINET

C0	70	CC	19	00	FC	05	11	00	53 S	47 G	56 V	68 h	64 d	47 G	68 h

72 r	61 a	58 X	51 Q	67 g	53 S	44 D	67 g	35 5	0A	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

BASE64 DECODE: COMPUTE — CBH

DEVICE 70H (160Q) COMMAND CBH (313Q) FUJI_BASE64_DECODE_COMPUTE

FIELDS	none
PAYLOAD	none
REPLY	ACK

Decodes it, replacing it with the bytes.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_b64dec            ; decode it
        call    fn_call
...
rq_b64dec:
        FNREQ   DEV_FUJINET, FUJI_BASE64_DECODE_COMPUTE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_base64_decode_compute();
```

H89 → FUJINET

C0	70	CB	06	00	42	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

BASE64 DECODE: LENGTH — CAH

DEVICE 70H (160Q) COMMAND CAH (312Q) FUJI_BASE64_DECODE_LENGTH

FIELDS	none
PAYLOAD	none
REPLY	4 bytes: the length, low first

How many bytes the decoding made.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_b64dlen           ; how many bytes? a long
        call    fn_call
        jr      nz,b64dlen_x
        ld      hl,(b64len)
        call    pdec16
b64dlen_x:
...
rq_b64dlen:
        FNREQ   DEV_FUJINET, FUJI_BASE64_DECODE_LENGTH, F_NONE, 0,0,0,0
        FNBUF   0, 0, b64len, 4
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_base64_decode_length(&b64len);
if (ok)
    printf("%lu\n", b64len);
```

H89 → FUJINET

C0	70	CA	06	00	41	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	70	06	0A	00	8C	00	0C	00	00	00	C0

BASE64 DECODE: OUTPUT — C9H

DEVICE 70H (160Q) COMMAND C9H (311Q) FUJI_BASE64_DECODE_OUTPUT

FIELDS	a word: how many bytes
PAYLOAD	none
REPLY	that many bytes

Takes the decoded bytes: “Heathkit H89” again.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,(b64len)             ; take them all
        ld      (rq_b64dout+RQ_AUX1),hl
        ld      hl,rq_b64dout
        call    fn_call
        jr      nz,b64dout_x
        ld      hl,b64buf
        call    ptext
b64dout_x:
...
rq_b64dout:                             ; how many: set before the call
        FNREQ   DEV_FUJINET, FUJI_BASE64_DECODE_OUTPUT, F_B12, 0,0,0,0
        FNBUF   0, 0, b64buf, 64
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_base64_decode_output(b64, (uint16_t)b64len);
if (ok) {
    b64[(uint16_t)b64len] = 0;
    printf("%s\n", b64);
}
```

H89 → FUJINET

C0	70	C9	08	00	53	05	0C	00	C0

FUJINET → H89 · ACK · 12 BYTES

C0	70	06	12	00	97	00	48 H	65 e	61 a	74 t	68 h	6B k	69 i	74 t	20

48 H	38 8	39 9	C0

HASHES

A **hash** is a short fingerprint of any amount of data: change one bit of the data and the fingerprint changes completely. Programs use hashes to check that a file arrived intact, and to sign requests to web services. Working one out takes the H89 a long time; the FujiNet does it at once. Data goes in with FUJI_HASH_INPUT,

as many times as needed; FUJI_HASH_COMPUTE works out the hash; the result comes back as text or as bytes.

CODE	HASH	LENGTH (BYTES, OR HEX CHARACTERS)
0	MD5	16, or 32
1	SHA-1	20, or 40
2	SHA-256	32, or 64
3	SHA-512	64, or 128

HASH INPUT C8H

DEVICE 70H (160Q) COMMAND C8H (310Q) FUJI_HASH_INPUT

FIELDS a word: how many bytes
PAYLOAD that many bytes
REPLY ACK

Adds data to be hashed. The example hashes “abc”, whose hashes are published, so the answer can be checked.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_hashin            ; the bytes to hash
        call    fn_call
...
rq_hashin:                              ; the length goes in a word
        FNREQ   DEV_FUJINET, FUJI_HASH_INPUT, F_B12, 3,0,0,0
        FNBUF   p_abc, 3, 0, 0
p_abc:  defm    "abc"
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_input("abc", 3);     /* or fuji_hash_add() */
```

H89 → FUJINET

C0	70	C8	0B	00	73	05	03	00	61 a	62 b	63 c	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

HASH COMPUTE, KEEPING THE INPUT C3H

DEVICE 70H (160Q) COMMAND C3H (303Q) FUJI_HASH_COMPUTE_NO_CLEAR

FIELDS 1 byte: the hash, 0 to 3
PAYLOAD none
REPLY ACK

Works out a hash of the data so far, and keeps the data, so another kind of hash can be made from it. The example makes the MD5 hash, then (next) SHA-1 from the same data.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_md5               ; their MD5, keeping the bytes
        call    fn_call                 ;   for another digest
...
rq_md5:                                 ; 0 MD5, 1 SHA-1, 2 SHA-256,
        FNREQ   DEV_FUJINET, FUJI_HASH_COMPUTE_NO_CLEAR, F_A1, 0,0,0,0
        FNBUF   0, 0, 0, 0              ;   3 SHA-512
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_compute_no_clear(MD5);   /* "abc" is kept */
```

H89 → FUJINET

C0	70	C3	07	00	3C	01	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

HASH COMPUTE C7H

DEVICE 70H (160Q) COMMAND C7H (307Q) FUJI_HASH_COMPUTE

FIELDS 1 byte: the hash, 0 to 3
PAYLOAD none
REPLY ACK

Works out a hash, then empties the data ready for the next.

WARNING: Codes 4 and 5 (SHA-224 and SHA-384, which fujinet-lib names) are not available: the FujiNet answers ACK, and the result is empty.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_sha1              ; their SHA-1; the bytes are
        call    fn_call                 ;   then let go
...
rq_sha1:                                ; 1: SHA-1
        FNREQ   DEV_FUJINET, FUJI_HASH_COMPUTE, F_A1, 1,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_compute(SHA1);       /* then let go */
```

H89 → FUJINET

C0	70	C7	07	00	41	01	01	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

HASH LENGTH C6H

DEVICE 70H (160Q) COMMAND C6H (306Q) FUJI_HASH_LENGTH

FIELDS 1 byte: 1 for text, 0 for bytes
PAYLOAD none
REPLY 1 byte: the length

How long the result is: here 28H, the 40 hex digits of an SHA-1 hash. Unlike the base64 and QR lengths, this is one byte.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_hashlen           ; how long is it, as hex text?
        call    fn_call
        jr      nz,hashlen_x
        ld      a,(hashlen)             ; one byte
        call    pdec8
hashlen_x:
...
rq_hashlen:                             ; 1: as hex; 0: as bytes
        FNREQ   DEV_FUJINET, FUJI_HASH_LENGTH, F_A1, 1,0,0,0
        FNBUF   0, 0, hashlen, 1
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_length(1, &hashlen, 1);  /* 1: as hex; one byte */
if (ok)
    printf("%u\n", hashlen);
```

H89 → FUJINET

C0	70	C6	07	00	40	01	01	C0

FUJINET → H89 · ACK · 1 BYTE

C0	70	06	07	00	A5	00	28 (	C0

HASH OUTPUT C5H

DEVICE 70H (160Q) COMMAND C5H (305Q) FUJI_HASH_OUTPUT

FIELDS 1 byte: 1 for text, 0 for bytes
PAYLOAD none
REPLY the hash, as text or bytes

The hash itself, as lower-case hexadecimal text (no zero at the end) or as bytes. It can be fetched again, either

way. The text here, a9993e36..., is the published SHA-1 of "abc", which shows the data was kept by the compute before.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_hashhex           ; the digest as hex text, no NUL
        call    fn_call
        jr      nz,hashhex_x
        ld      hl,digest               ; BC characters
        call    ptext
hashhex_x:
...
rq_hashhex:                             ; 1: as hex
        FNREQ   DEV_FUJINET, FUJI_HASH_OUTPUT, F_A1, 1,0,0,0
        FNBUF   0, 0, digest, 128
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_output(1, digest, hashlen);  /* hex, no NUL */
if (ok) {
    digest[hashlen] = 0;
    printf("%s\n", (char *)digest);
}
```

H89 → FUJINET

C0	70	C5	07	00	3F	01	01	C0

FUJINET → H89 · ACK · 40 BYTES

C0	70	06	2E	00	22	00	61 a	39 9	39 9	39 9	33 3	65 e	33 3	36 6	34 4

37 7	30 0	36 6	38 8	31 1	36 6	61 a	62 b	61 a	33 3	65 e	20 MORE	C0

HASH OUTPUT, AS BYTES C5H

DEVICE 70H (160Q) COMMAND C5H (305Q)

The same hash as 20 bytes.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_hashbin           ; the same digest, as bytes
        call    fn_call
        jr      nz,hashbin_x
        ld      hl,digest               ; BC of them
        call    phexs
hashbin_x:
...
rq_hashbin:                             ; 0: as bytes
        FNREQ   DEV_FUJINET, FUJI_HASH_OUTPUT, F_A1, 0,0,0,0
        FNBUF   0, 0, digest, 128
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_output(0, digest, 20);   /* SHA-1: 20 bytes */
if (ok)
    for (i = 0; i < 20; i++)
        printf("%02X", digest[i]);
```

H89 → FUJINET

C0	70	C5	07	00	3E	01	00	C0

FUJINET → H89 · ACK · 20 BYTES

C0	70	06	1A	00	EC	00	A9	99	3E >	36 6	47 G	06	81	6A j	BA

3E >	25 %	71 q	78 x	50 P	C2	6C l	9C	D0	D8	9D	C0

HASH CLEAR C2H

DEVICE 70H (160Q) COMMAND C2H (302Q) FUJI_HASH_CLEAR

FIELDS	none
PAYLOAD	none
REPLY	ACK

Empties the data, keeping the last result.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_hashclr           ; start again: no bytes to hash
        call    fn_call
...
rq_hashclr:
        FNREQ   DEV_FUJINET, FUJI_HASH_CLEAR, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_hash_clear();             /* start again */
```

H89 → FUJINET

C0	70	C2	06	00	39	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

QR CODES

The FujiNet can turn text into a QR code, the square of dots a telephone's camera reads, so an H89 program can show a web address that a visitor can follow. The H19's graphics characters draw one nicely: a quarter-square character for each two-by-two block of dots.

QR CODE INPUT — BCH

DEVICE 70H (160Q) COMMAND BCH (274Q) FUJI_QRCODE_INPUT

FIELDS	a word: how many bytes
PAYLOAD	that many bytes
REPLY	ACK

Adds text to be encoded. The example uses HTTPS://FUJINET.ONLINE/. In capitals, it fits the QR code's compact alphanumeric mode, which gives the smallest code, 21 by 21.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_qrin              ; what the code is to say
        call    fn_call
...
rq_qrin:                                ; the length goes in a word
        FNREQ   DEV_FUJINET, FUJI_QRCODE_INPUT, F_B12, p_urln,0,0,0
        FNBUF   p_url, p_urln, 0, 0
p_url:                                  ; capitals: a smaller code
        defm    "HTTPS://FUJINET.ONLINE/"
p_urln          equ     $-p_url
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* capitals: QR's alphanumeric mode makes a smaller code */
ok = fuji_qrcode_input("HTTPS://FUJINET.ONLINE/", 23);
```

H89 → FUJINET

C0	70	BC	1F	00	D0	05	17	00	48 H	54 T	54 T	50 P	53 S	3A :	2F /
2F /	46 F	55 U	4A J	49 I	4E N	45 E	54 T	2E .	4F O	4E N	4C L	49 I	4E N	45 E	2F /
C0															

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

QR CODE ENCODE — BDH

DEVICE 70H (160Q) COMMAND BDH (275Q) FUJI_QRCODE_ENCODE

FIELDS	3 bytes: version (0 = smallest that fits), error correction (0 to 3), shorten (0)
PAYLOAD	none
REPLY	ACK

Makes the code, and empties the input. The error correction is 0 (L, 7% of the code can be lost) to 3 (H, 30%). With "shorten" not 0, the FujiNet first registers the text as a short web address on its own web server and encodes that instead.

WARNING: Send all three fields.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_qrenc             ; make the code
        call    fn_call
...
rq_qrenc:                               ; version 0: the smallest that
        FNREQ   DEV_FUJINET, FUJI_QRCODE_ENCODE, F_A1_A2_A3, 0,0,0,0
        FNBUF   0, 0, 0, 0              ;   fits; ECC low; no short URL
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* version 0: the smallest that fits; not shortened */
ok = fuji_qrcode_encode(0, QR_ECC_LOW, 0);
```

H89 → FUJINET

C0	70	BD	09	00	3A	03	00	00	00	C0

FUJINET → H89 · ACK

C0	70	06	06	00	7C	00	C0

QR CODE LENGTH — BEH

DEVICE 70H (160Q) COMMAND BEH (276Q) FUJI_QRCODE_LENGTH

FIELDS	1 byte: the form wanted
PAYLOAD	none
REPLY	4 bytes: the length, low first

How long the code is in the form asked for, which it is rendered in: 0, one bit per dot, rows run together; 2, a bitmap, each row in whole bytes, leftmost dot in the top bit; 1 ANSI text, 3 SVG, 4 and 5 the Atari's and Commodore's character sets.

For forms 0 and 2 the FujiNet puts a byte in front, the number of dots on a side. So the example's length is 64: that byte, then 21 rows of 3 bytes. (Notice the reply's checksum, C0H, sent as the escape pair DB DC.)

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_qrlen             ; as a bitmap: how many bytes?
        call    fn_call
        jr      nz,qrlen_x
        ld      hl,(qrlen)              ; a long; the low word will do
        call    pdec16
qrlen_x:
...
rq_qrlen:                               ; 2: a bitmap, 1 bit a module
        FNREQ   DEV_FUJINET, FUJI_QRCODE_LENGTH, F_A1, 2,0,0,0
        FNBUF   0, 0, qrlen, 4
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_qrcode_length(QR_OUTPUT_MODE_BITMAP, &qrlen);
if (ok)
    printf("%lu bytes\n", qrlen);
```

H89 → FUJINET

C0	70	BE	07	00	39	01	02	C0

FUJINET → H89 · ACK · 4 BYTES

C0	70	06	0A	00	DB	DC	00	40 @	00	00	00	C0

QR CODE OUTPUT — BFH

DEVICE 70H (160Q) COMMAND BFH (277Q) FUJI_QRCODE_OUTPUT

FIELDS	a word: how many bytes
PAYLOAD	none
REPLY	that many bytes

Takes the code, from the front: here 15H (21), then the rows. NAK if more is asked for than is left.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,(qrlen)              ; take them all
        ld      (rq_qrout+RQ_AUX1),hl
        ld      hl,rq_qrout
        call    fn_call
        jr      nz,qrout_x
        ld      a,(qr)                  ; first, the modules on a side;
        call    pdec8                   ;   then the rows, a bit each
qrout_x:
...
rq_qrout:                               ; how many: set before the call
        FNREQ   DEV_FUJINET, FUJI_QRCODE_OUTPUT, F_B12, 0,0,0,0
        FNBUF   0, 0, qr, 256
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_qrcode_output(qr, (uint16_t)qrlen);
if (ok)                 /* the modules on a side, then the rows */
    printf("%u x %u\n", qr[0], qr[0]);
```

H89 → FUJINET

C0	70	BF	08	00	7D	05	40	00	C0

FUJINET → H89 · ACK · 64 BYTES

C0	70	06	46	00	27	00	15	FE	A3	F8	82	FA	08	BA	FA
E8	BA	E2	E8	BA	6A j	E8	82	AA	08	FE	44 MORE				C0

ODDS AND ENDS

GENERATE GUID BBH

DEVICE 70H (160Q) COMMAND BBH (273Q) FUJI_GENERATE_GUID

FIELDS	none
PAYLOAD	none
REPLY	37 bytes: a new random identifier, as text, ending in a zero

A new universally unique identifier (a version 4 UUID), such as a program might use to name itself to a server.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_guid              ; a new GUID, as text
        call    fn_call
        jr      nz,guid_x
        ld      hl,guid
        call    print
guid_x:
...
rq_guid:                                ; 36 characters and a NUL
        FNREQ   DEV_FUJINET, FUJI_GENERATE_GUID, F_NONE, 0,0,0,0
        FNBUF   0, 0, guid, 37
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_generate_guid(guid);      /* 36 characters and a NUL */
if (ok)
    printf("%s\n", guid);
```

H89 → FUJINET

C0	70	BB	06	00	32	00	C0

FUJINET → H89 · ACK · 37 BYTES

C0	70	06	2B	00	15	00	64 d	33 3	31 1	39 9	63 c	64 d	66 f	61 a	2D -
61 a	63 c	38 8	30 0	2D -	34 4	64 d	66 f	39 9	2D -	61 a	34 4	36 6	38 8	2D -	31 1
32 2	33 3	31 1	61 a	37 7	38 8	37 7	61 a	30 0	31 1	31 1	00	C0			

RANDOM NUMBER D3H

DEVICE 70H (160Q) COMMAND D3H (323Q) FUJI_RANDOM_NUMBER

FIELDS	none
PAYLOAD	none
REPLY	4 bytes: a random number, low first

A random 32-bit number from the ESP32's hardware generator, far better than any an H89 program can make for itself. The top bit is always 0. fujinet-lib has no function for it.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_random            ; a random long
        call    fn_call
        jr      nz,random_x
        ld      hl,(rnd+2)              ; high word, then low
        call    phex16
        ld      hl,(rnd)
        call    phex16
random_x:
...
rq_random:
        FNREQ   DEV_FUJINET, FUJI_RANDOM_NUMBER, F_NONE, 0,0,0,0
        FNBUF   0, 0, rnd, 4
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* no fujinet-lib call for RANDOM_NUMBER */
ok = fuji_bus_call(FUJI_DEVICEID_FUJINET, FUJICMD_RANDOM_NUMBER,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0, &rnd, sizeof rnd);
if (ok)
    printf("%08lX\n", rnd);
```

H89 → FUJINET

C0	70	D3	06	00	4A	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	70	06	0A	00	1B	00	AD	8B	E2	7E ~	C0

STATUS 53H

DEVICE 70H (160Q) COMMAND 53H (123Q) FUJI_STATUS

FIELDS	1 byte (ignored)
PAYLOAD	none
REPLY	4 bytes, all zero

On other computers this reports the FujiNet's state. On the H89 it always answers four zero bytes.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_status            ; four status bytes (all 0 on
        call    fn_call                 ;   the H89)
        jr      nz,status_x
        ld      hl,fstat
        call    phexs
status_x:
...
rq_status:                              ; (the field is not used)
        FNREQ   DEV_FUJINET, FUJI_STATUS, F_A1, 0,0,0,0
        FNBUF   0, 0, fstat, 4
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_status(&fst);             /* four bytes, all 0 here */
if (ok)
    printf("%02X %02X %02X %02X\n", fst.value[0], fst.value[1],
           fst.value[2], fst.value[3]);
```

H89 → FUJINET

C0	70	53	07	00	CB	01	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	70	06	0A	00	80	00	00	00	00	00	C0

DEVICE READY 00H

DEVICE 70H (160Q) COMMAND 00H (000Q) FUJI_DEVICE_READY

FIELDS	none
PAYLOAD	none
REPLY	512 bytes of 41H

A test left in the firmware: it answers 512 letters A. A program can use it to check that long replies arrive intact.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_ready             ; a link test: 512 'A's back
        call    fn_call
        jr      nz,ready_x
        ld      h,b                     ; count them
        ld      l,c
        call    pdec16
ready_x:
...
rq_ready:
        FNREQ   DEV_FUJINET, FUJI_DEVICE_READY, F_NONE, 0,0,0,0
        FNBUF   0, 0, buf, 512
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
/* a link test: 512 'A's come back. No fujinet-lib call */
ok = fuji_bus_call(FUJI_DEVICEID_FUJINET, FUJICMD_DEVICE_READY,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0, buf, 512);
if (ok)
    printf("%c ... %c\n", buf[0], buf[511]);
```

H89 → FUJINET

C0	70	00	06	00	76	00	C0

FUJINET → H89 · ACK · 512 BYTES

C0	70	06	06	02	01	00	41 A	41 A	41 A	41 A	41 A	41 A	41 A	41 A	41 A

41 A	41 A	41 A	41 A	41 A	41 A	41 A	41 A	41 A	41 A	41 A	492 MORE	C0

RESET FFH

DEVICE 70H (160Q) COMMAND FFH (377Q) FUJI_RESET

FIELDS	none
PAYLOAD	none
REPLY	none: the FujiNet restarts

Restarts the FujiNet, as switching it off and on would: every drive is unmounted and every network channel closed, and the FujiNet starts again from its saved settings.

WARNING: There is no reply. A program sends RESET, and then waits for its own timeout (`fn_call` gives up after 15 seconds, and so does fujinet-lib's `fuji_reset()`, which then returns `false`). Wait a few seconds more before the next request, while the FujiNet starts.

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_reset             ; restart the FujiNet; no reply
        call    fn_call                 ;   comes (TIMEOUT, after 15 s)
...
rq_reset:
        FNREQ   DEV_FUJINET, FUJI_RESET, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_reset();          /* no reply: false, after 15 seconds */
```

H89 → FUJINET

C0	70	FF	06	00	76	00	C0

NO REPLY

COMMANDS THE H89 DOES NOT HAVE

The firmware's list of commands has others, for other computers; the table below lists them. Sent to the H89's FujiNet, each is answered with a NAK. Here is the first of them, sent as an example:

Z80 ASSEMBLY — FROM EXFUJI.ASM

```
        ld      hl,rq_ftime             ; not a command device 70H has:
        call    fn_call                 ;   NAK (the clock is 45H)
...
rq_ftime:
        FNREQ   DEV_FUJINET, FUJI_GET_TIME, F_NONE, 0,0,0,0
        FNBUF   0, 0, buf, 7
```

C (Z88DK, FUJINET-LIB) — FROM EXFUJI.C

```
ok = fuji_get_time(buf);    /* NAK: the clock is device 45H */
```

H89 → FUJINET

C0	70	D2	06	00	49	00	C0

FUJINET → H89 · NAK

C0	70	15	06	00	8B	00	C0

COMMAND	NO.	NOTE
FUJI_GET_TIME	D2H	fujinet-lib's `fuji_get_time()` sends it: use the clock (Chapter 12)
FUJI_ENABLE_DEVICE, FUJI_DISABLE_DEVICE	D5H, D4H	`fuji_enable_device()`, `fuji_disable_device()` send them
FUJI_DEVICE_ENABLE_STATUS	D1H	`fuji_get_device_enabled_status()` does not link
FUJI_ENABLE_UDPSTREAM	F0H	`fuji_enable_udpstream()` does not link
FUJI_SET_BAUDRATE	EBH	
FUJI_SET_HSIO_INDEX, FUJI_SET_SIO_EXTERNAL_CLOCK	E3H, DFH	Atari only
FUJI_GET_HEAP	C1H	
FUJI_GET_DEVICE1_FULLPATH … 10	A0H–A9H	use FUJI_GET_DEVICE_FULLPATH
FUJI_UPDATE_FIRMWARE	90H	update through the USB port (Operation Manual)

THE DISK DRIVES

GENERAL

Devices 31H to 38H are the FujiNet's eight disk drives, one for each drive slot (Chapter 8): slot 0 is 31H, slot 7 is 38H. Each reads and writes the image mounted in its slot, sector by sector. FN.DVD and FNDISK, the Operation Manual's drivers, use them to give HDOS and CP/M their FujiNet drives; a program can use them directly too, to read an image that is not a disk HDOS or CP/M knows, or to look at one byte by byte.

CAUTION: A drive FN.DVD or FNDISK is using belongs to the operating system. Writing its sectors behind the system's back will confuse it, and may ruin the disk. Read freely; write only to a drive you mounted yourself.

SECTORS AND IMAGES

The FujiNet knows an image's kind by the end of its name:

- **.IMD** is an ImageDisk image, a copy of a real diskette with its own layout; each sector is the size it had on the diskette (128 to 512 bytes), and they are numbered from 0 in order.
- **.ROM, .BIN** and the other cartridge names of Chapter 8's warning are not disks at all. Don't.
- Anything else — **.H8D**, **.IMG**, no extension — is a plain file of 512-byte sectors: sector **n** is the 512 bytes starting at byte **n** × 512. A 100K H17 image (.H8D) is 200 of them, each holding two of the H17's 256-byte sectors.

Sector numbers count from 0. The examples mount the test image `test/disk.img`, whose sector **n** is filled with the byte **n**, in drive slot 0, so the drive is 31H. Its setup, with three FujiNet commands from Chapter 8 (mount the SD host, name the image, mount it), is in EXDISK.ASM and EXDISK.C but not printed here.

fujinet-lib has no functions for the disk drives, so the C examples call `fuji_bus_call()`.

THE COMMANDS

READ SECTOR 52H

DEVICE 31H--38H COMMAND 52H (122Q) DISK_READ

FIELDS	4 bytes: the sector number, low first
PAYLOAD	none
REPLY	the sector: 512 bytes (an .IMD sector, its own size)

Reads one sector. NAK if no image is mounted, or the sector is past the end. The field may be any size; the examples send a long, as fujinet-lib's shorthand would.

NOTE: An .IMD sector that the original diskette could not read is still returned, with ACK; only DISK_STATUS (below) shows the error.

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_read              ; read sector 3
        call    fn_call
        jr      nz,rd_x
        ld      hl,sector               ; the test image's sector n is
        ld      bc,512                  ;   512 bytes of n: check it
        ld      a,3
rd_1:
        cpi                             ; A = (HL)? HL+1, BC-1
        jr      nz,rd_x
        jp      pe,rd_1                 ; on to the end of the sector
        ld      hl,t_all3
        call    print
rd_x:
...
rq_read:                                ; sector 3, counted from 0
        FNREQ   DEV_DISK, DISK_READ, F_C1234, 3,0,0,0
```

```
        FNBUF   0, 0, sector, 512
t_all3: defb    13, 10
        defm    "Sector 3 is 512 bytes of 03H."
        defb    0
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
/* fujinet-lib has no disk calls: fuji_bus_call sends the frame */
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_READ,
                   FUJI_FIELD_C1234 | FUJI_FIELD_REPLY,
                   3, 0, 0, 0, sector, 512);    /* sector 3 */
if (ok) {
    for (i = 0; i < 512 && sector[i] == 3; i++)
        ;                           /* sector n: 512 bytes of n */
    if (i == 512)
        printf("\nSector 3 is 512 bytes of 03H.");
}
```

H89 → FUJINET

C0	31	52	0A	00	97	07	03	00	00	00	C0

FUJINET → H89 · ACK · 512 BYTES

C0	31	06	06	02	45	00	03	03	03	03	03	03	03	03	03

03	03	03	03	03	03	03	03	03	03	03	492 MORE	C0

WRITE SECTOR 57H

DEVICE 31H--38H COMMAND 57H (127Q) DISK_WRITE

FIELDS	4 bytes: the sector number
PAYLOAD	the sector's bytes
REPLY	ACK

Writes one sector, at once (the FujiNet flushes it to the file). A short payload is filled out with zeros. NAK if the image was mounted read-only (FUJI_MOUNT_IMAGE's mode 01H), or the sector is past the end: the image never grows. The example fills sector 5 with E5H, the byte CP/M formats with, and reads it back.

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,sector               ; fill a sector with E5H, the
        ld      de,sector+1             ;   byte CP/M formats with
        ld      bc,511
        ld      (hl),0xE5
        ldir
        ld      hl,rq_write             ; and write it to sector 5
        call    fn_call
...
rq_write:                               ; sector 5, 512 bytes
        FNREQ   DEV_DISK, DISK_WRITE, F_C1234, 5,0,0,0
        FNBUF   sector, 512, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
memset(sector, 0xE5, 512);              /* CP/M's empty byte */
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_WRITE,
                   FUJI_FIELD_C1234 | FUJI_FIELD_DATA,
                   5, 0, 0, 0, sector, 512);    /* to sector 5 */
```

H89 → FUJINET

C0	31	57	0A	02	6C	07	05	00	00	00	E5	E5	E5	E5	E5

E5	E5	E5	E5	E5	E5	E5	E5	E5	E5	E5	496 MORE	C0

FUJINET → H89 · ACK

C0	31	06	06	00	3D	00	C0

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_back              ; read sector 5 back: E5H again
        call    fn_call
...
rq_back:
        FNREQ   DEV_DISK, DISK_READ, F_C1234, 5,0,0,0
        FNBUF   0, 0, sector, 512
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_READ,
                   FUJI_FIELD_C1234 | FUJI_FIELD_REPLY,
                   5, 0, 0, 0, sector, 512);    /* E5H again */
```

H89 → FUJINET

C0	31	52	0A	00	99	07	05	00	00	00	C0

FUJINET → H89 · ACK · 512 BYTES

C0	31	06	06	02	0B	00	E5	E5	E5	E5	E5	E5	E5	E5	E5

E5	E5	E5	E5	E5	E5	E5	E5	E5	E5	E5	492 MORE	C0

PUT SECTOR 50H

DEVICE 31H--38H COMMAND 50H (120Q) DISK_PUT

FIELDS	4 bytes: the sector number
PAYLOAD	the sector's bytes
REPLY	ACK

On other computers, a write without verifying. Here it is the same as DISK_WRITE.

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_put               ; PUT writes as WRITE does: the
        call    fn_call                 ;   verify it skips, nobody does
...
rq_put:                                 ; the same sector to sector 6
        FNREQ   DEV_DISK, DISK_PUT, F_C1234, 6,0,0,0
        FNBUF   sector, 512, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
/* PUT writes as WRITE does: the verify it skips, nobody does */
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_PUT,
                   FUJI_FIELD_C1234 | FUJI_FIELD_DATA,
                   6, 0, 0, 0, sector, 512);    /* to sector 6 */
```

H89 → FUJINET

C0	31	50	0A	02	66	07	06	00	00	00	E5	E5	E5	E5	E5

E5	E5	E5	E5	E5	E5	E5	E5	E5	E5	E5	496 MORE	C0

FUJINET → H89 · ACK

C0	31	06	06	00	3D	00	C0

STATUS 53H

DEVICE 31H--38H COMMAND 53H (123Q) DISK_STATUS

FIELDS	1 byte (ignored)
PAYLOAD	none
REPLY	4 bytes

The drive's status, in the Atari's form: a status byte, the controller's status (where an .IMD sector's error shows: 08H a CRC error, 10H missing, 20H deleted), a timeout, and 0. For a plain image it is normally 00 FF E0 00 — even with nothing mounted, so don't use it to see whether a drive has an image; read sector 0 instead.

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_dstat             ; drive, controller, format
        call    fn_call                 ;   timeout, and a 0
        jr      nz,ds_x
        call    crlf
        ld      hl,dstat
        ld      bc,4
        call    phexs
ds_x:
...
rq_dstat:
        FNREQ   DEV_DISK, DISK_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, dstat, 4
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_STATUS,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0, dstat, 4);
if (ok)                                 /* drive, controller, */
    printf("\n%02X %02X %02X %02X",     /* timeout, 0 */
           dstat[0], dstat[1], dstat[2], dstat[3]);
```

H89 → FUJINET

C0	31	53	06	00	8A	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	31	06	0A	00	22	00	00	FF	E0	00	C0

FORMAT 21H

DEVICE 31H--38H COMMAND 21H (041Q) DISK_FORMAT

FIELDS	none
PAYLOAD	none
REPLY	128 bytes: FFH FFH, then zeros

Meant to format the disk. It does nothing to the image: the FujiNet answers ACK with a 128-byte "bad sector list" that says none are bad, and every sector is as it was — sector 3 still reads 03H, as the example shows. To blank an image, write its sectors. DISK_FORMAT_MEDIUM (22H) is the same.

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_fmt               ; FORMAT formats nothing: it
        call    fn_call                 ;   ACKs, FF FF and 126 0s
...
rq_fmt:
        FNREQ   DEV_DISK, DISK_FORMAT, F_NONE, 0,0,0,0
        FNBUF   0, 0, sector, 512
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
/* FORMAT formats nothing: it ACKs, FF FF and 126 0s */
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_FORMAT,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0, sector, 128);
```

H89 → FUJINET

C0	31	21	06	00	58	00	C0

FUJINET → H89 · ACK · 128 BYTES

C0	31	06	86	00	BD	00	FF	FF	00	00	00	00	00	00	00

00	00	00	00	00	00	00	00	00	00	00	108 MORE	C0

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_read              ; sector 3 again: 03H as before
        call    fn_call
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_READ,
                   FUJI_FIELD_C1234 | FUJI_FIELD_REPLY,
                   3, 0, 0, 0, sector, 512);    /* 03H, as before */
```

H89 → FUJINET

C0	31	52	0A	00	97	07	03	00	00	00	C0

FUJINET → H89 · ACK · 512 BYTES

C0	31	06	06	02	45	00	03	03	03	03	03	03	03	03	03

03	03	03	03	03	03	03	03	03	03	03	492 MORE	C0

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_fmtm              ; the same as FORMAT
        call    fn_call
...
rq_fmtm:
        FNREQ   DEV_DISK, DISK_FORMAT_MEDIUM, F_NONE, 0,0,0,0
        FNBUF   0, 0, sector, 512
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_FORMAT_MEDIUM,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0, sector, 128);
```

H89 → FUJINET

C0	31	22	06	00	59	00	C0

FUJINET → H89 · ACK · 128 BYTES

C0	31	06	86	00	BD	00	FF	FF	00	00	00	00	00	00	00

00	00	00	00	00	00	00	00	00	00	00	108 MORE	C0

READ GEOMETRY 4EH

DEVICE 31H--38H COMMAND 4EH (116Q) DISK_PERCOM_READ

FIELDS	none
PAYLOAD	none
REPLY	12 bytes: the PERCOM block

The disk's geometry in the "PERCOM" form some disk controllers used: tracks, step rate, sectors per track (a word, high byte first), sides less one, density, sector size (a word, high first), drive present, and three spare bytes. For an .IMD it describes the diskette; for a plain image it is all zeros.

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_pcr               ; the geometry, 12 bytes: all
        call    fn_call                 ;   0 for a plain image
...
rq_pcr:
        FNREQ   DEV_DISK, DISK_PERCOM_READ, F_NONE, 0,0,0,0
        FNBUF   0, 0, percom, 12
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
/* the geometry, 12 bytes: all 0 for a plain image */
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_PERCOM_READ,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0, pblock, 12);
```

H89 → FUJINET

C0	31	4E	06	00	85	00	C0

FUJINET → H89 · ACK · 12 BYTES

C0	31	06	12	00	49	00	00	00	00	00	00	00	00	00	00

00	00	00	C0

WRITE GEOMETRY 4FH

DEVICE 31H--38H COMMAND 4FH (117Q) DISK_PERCOM_WRITE

FIELDS	none
PAYLOAD	12 bytes: a PERCOM block
REPLY	ACK

Replaces the geometry the FujiNet reports. It changes nothing else: the image's sectors are still 512 bytes.

Reading it back gives what was written, and a “sides” of 1 sets 40H in DISK_STATUS’s first byte.

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_pcw               ; set it: the firmware keeps the
        call    fn_call                 ;   block, not the image
...
rq_pcw:
        FNREQ   DEV_DISK, DISK_PERCOM_WRITE, F_NONE, 0,0,0,0
        FNBUF   p_percom, 12, 0, 0
p_percom:                               ; 80 tracks, step rate 1, 9
        defb    80, 1, 0, 9             ;   sectors a track (high byte
        defb    1, 4, 2, 0              ;   first), two sides, MFM, 512
        defb    0xFF, 0, 0, 0           ;   bytes a sector, present
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
static const uint8_t percom[12] = {
    80, 1, 0, 9,        /* 80 tracks, step 1, 9 sectors (high 1st) */
    1, 4, 2, 0,         /* two sides, MFM, 512 bytes a sector */
    0xFF, 0, 0, 0       /* drive present */
};
...
/* the firmware keeps the block; the image is not changed */
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_PERCOM_WRITE,
                   FUJI_FIELD_DATA, 0, 0, 0, 0, percom, 12);
```

H89 → FUJINET

C0	31	4F	12	00	F3	00	50 P	01	00	09	01	04	02	00	FF
00	00	00	C0												

FUJINET → H89 · ACK

C0	31	06	06	00	3D	00	C0

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_pcr               ; read it back
        call    fn_call
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_PERCOM_READ,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0, pblock, 12);
```

H89 → FUJINET

C0	31	4E	06	00	85	00	C0

FUJINET → H89 · ACK · 12 BYTES

C0	31	06	12	00	AA	00	50 P	01	00	09	01	04	02	00	FF
00	00	00	C0												

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_dstat             ; two sides in the PERCOM block:
        call    fn_call                 ;   the status says 40H now
        jr      nz,dp_x
        call    crlf
        ld      hl,dstat
        ld      bc,4
        call    phexs
dp_x:
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
/* two sides in the PERCOM block: the status says 40H now */
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_STATUS,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0, dstat, 4);
if (ok)
    printf("\n%02X %02X %02X %02X",
           dstat[0], dstat[1], dstat[2], dstat[3]);
```

H89 → FUJINET

C0	31	53	06	00	8A	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	31	06	0A	00	62	00	40 @	FF	E0	00	C0

WHEN THERE IS NO DISK

READING PAST THE END 52H

DEVICE 31H--38H COMMAND 52H (122Q)

The test image has 16 sectors, 0 to 15. Sector 16 is past the end:

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_rend              ; read past the end: NAK
        call    fn_call
...
rq_rend:
        FNREQ   DEV_DISK, DISK_READ, F_C1234, 16,0,0,0
        FNBUF   0, 0, sector, 512
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_READ,
                   FUJI_FIELD_C1234 | FUJI_FIELD_REPLY,
                   16, 0, 0, 0, sector, 512);   /* NAK */
```

H89 → FUJINET

C0	31	52	0A	00	A4	07	10	00	00	00	C0

FUJINET → H89 · NAK

C0	31	15	06	00	4C	00	C0

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_wend              ; sector 16 of 16 (0-15): NAK,
        call    fn_call                 ;   and the file is left alone
...
rq_wend:
        FNREQ   DEV_DISK, DISK_WRITE, F_C1234, 16,0,0,0
        FNBUF   sector, 512, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
/* sector 16 of a 16-sector image (0-15): NAK, file left alone */
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_WRITE,
                   FUJI_FIELD_C1234 | FUJI_FIELD_DATA,
                   16, 0, 0, 0, sector, 512);
```

H89 → FUJINET

C0	31	57	0A	02	77	07	10	00	00	00	E5	E5	E5	E5	E5

E5	E5	E5	E5	E5	E5	E5	E5	E5	E5	E5	496 MORE	C0

FUJINET → H89 · NAK

C0	31	15	06	00	4C	00	C0

READING AN EMPTY DRIVE 52H

DEVICE 31H--38H COMMAND 52H (122Q)

Drive 32H has never had an image:

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_rnone             ; drive 32H has no disk: NAK
        call    fn_call
...
rq_rnone:                               ; drive slot 1: drive 32H
        FNREQ   DEV_DISK+1, DISK_READ, F_C1234, 0,0,0,0
        FNBUF   0, 0, sector, 512
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
ok = fuji_bus_call(FUJI_DEVICEID_DISK + 1, DISKCMD_READ,
                   FUJI_FIELD_C1234 | FUJI_FIELD_REPLY,
                   0, 0, 0, 0, sector, 512);    /* 32H: NAK */
```

H89 → FUJINET

C0	32	52	0A	00	95	07	00	00	00	00	C0

FUJINET → H89 · NAK

C0	32	15	06	00	4D	00	C0

AFTER UNMOUNTING 52H

DEVICE 31H--38H COMMAND 52H (122Q)

After FUJI_UNMOUNT_IMAGE, the drive refuses to read — but STATUS still answers, as if the image were there.

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_read              ; drive 31H is empty now: NAK
        call    fn_call
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_READ,
                   FUJI_FIELD_C1234 | FUJI_FIELD_REPLY,
                   3, 0, 0, 0, sector, 512);    /* empty: NAK */
```

H89 → FUJINET

C0	31	52	0A	00	97	07	03	00	00	00	C0

FUJINET → H89 · NAK

C0	31	15	06	00	4C	00	C0

Z80 ASSEMBLY — FROM EXDISK.ASM

```
        ld      hl,rq_dstat             ; but STATUS still answers, from
        call    fn_call                 ;   the disk that was there
```

C (Z88DK, FUJINET-LIB) — FROM EXDISK.C

```
/* STATUS still answers, from the disk that was there */
ok = fuji_bus_call(FUJI_DEVICEID_DISK, DISKCMD_STATUS,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0, dstat, 4);
```

H89 → FUJINET

C0	31	53	06	00	8A	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	31	06	0A	00	62	00	40 @	FF	E0	00	C0

THE NETWORK DEVICE

GENERAL

The FujiNet keeps eight **network channels**, devices 71H to 78H. Each can be open on one thing at a time: a file on a web server, a TCP connection, a directory on the microSD card, a JSON document. Chapter 11 lists the kinds of thing; this chapter gives the commands.

Every channel lives the same way:

1. **OPEN** it on a devicespec, such as `N:HTTP://192.168.1.10/hello.txt`, with a mode (read, write, and others) and a line-ending translation.
2. Ask its **STATUS**: how many bytes are waiting, whether it is still connected, and what went wrong if anything did.
3. **READ** what is waiting — never more — or **WRITE** to it.
4. **CLOSE** it.

A channel never tells the H89 that data has arrived: there is no interrupt (Chapter 2). A program asks STATUS, and asks again. NETCAT's loop (Chapter 13) is the pattern.

WARNING: Every network command that takes fields must have all of them. A command that finds a field missing stops the firmware (Chapter 3): OPEN needs two, and so do SET_PARSER, TRANSLATION, SET_PARAMETER, SET_HTTP_MODE and the six file system commands, even where a field is not used.

THE EXAMPLES IN THIS CHAPTER

The examples come from EXNET.ASM and EXNET.C, which use the channels like this, all at once:

UNIT	OPEN ON
71H	a web page (`hello.txt`), then the channel settings and errors
72H	a JSON document
73H	web requests: POST, PUT, DELETE
74H	HTTP headers
75H	a TCP connection to the test board
76H	a TCP server, which the test client connects to
77H	UDP
78H	files on the microSD card

The servers were the book's test servers (`tools/helpers.py`) on the same computer as fujinet-pc, so the devicespecs in the bytes say 127.0.0.1. On a real card use the server's address on your network.

OPENING AND CLOSING

OPEN 4FH

DEVICE 71H--78H COMMAND 4FH (117Q) NET_OPEN

FIELDS	2 bytes: the mode; the translation
PAYLOAD	the devicespec (up to 256 bytes)
REPLY	ACK, or NAK

Opens the channel on a devicespec (Chapter 11). Any channel already open on the unit is closed first — even if this OPEN then fails. The payload is the devicespec; a zero or a CR LF ends it, and an `N:` or `Nn:` in front is optional (the device number chooses the unit).

MODE	MEANS
04H	Read. For HTTP, a GET.
05H	HTTP DELETE.
06H, 07H	A directory listing. The translation field then chooses the listing's form.
08H	Write: create or replace a file. For HTTP, a PUT.
09H	Append to a file. For HTTP, a DELETE that may send headers.
0CH	Read and write a file. For HTTP, a GET that can send and collect headers.
0DH	HTTP POST.
0EH	Meant as HTTP PUT with headers — but a POST is sent (see below).

TRANSLATION	MEANS
0	None: bytes pass as they are.
1	The far end uses CR: each CR read becomes CR LF; CR LF written becomes CR.
2	The far end uses LF: likewise with LF.
3	The far end uses CR LF: nothing changes, since the H89's line ending is CR LF.

Streams (TCP, UDP, Telnet) take any mode; mode 12 is the usual. For a file system, a mode not in the table is worse than refused: TNFS and SD stop the firmware.

WARNING: Mode 0EH sends a POST, not a PUT. Use mode 08H for a PUT. And an HTTP DELETE (05H or 09H) is sent only when the program next asks STATUS or READs; a CLOSE straight after the OPEN sends nothing at all.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_open              ; GET a file from the server:
        call    fn_call                 ;   ACK, but nothing is sent yet
...
rq_open:                                ; unit 1: mode 4 (GET), no
        FNREQ   DEV_NETWORK, NET_OPEN, F_A1_A2, 4,0,0,0
        FNBUF   p_hello, P_HELLO_LEN, 0, 0
p_hello:
        defm    "N1:HTTP://127.0.0.1:7802/hello.txt", 0
P_HELLO_LEN     equ     $ - p_hello
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char hello[] = "N1:HTTP://127.0.0.1:7802/hello.txt";
...
rc = network_open(hello, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
```

H89 → FUJINET

C0	71	4F	2B	00	1D	02	04	00	4E N	31 1	3A :	48 H	54 T	54 T	50 P
3A :	2F /	2F /	31 1	32 2	37 7	2E .	30 0	2E .	30 0	2E .	31 1	3A :	37 7	38 8	30 0
32 2	2F /	68 h	65 e	6C l	6C l	6F o	2E .	74 t	78 x	74 t	00	C0			

FUJINET → H89 · ACK

C0	71	06	06	00	7D	00	C0

OPEN: OTHER KINDS OF CHANNEL 4FH

DEVICE 71H--78H COMMAND 4FH (117Q)

The same command for the other examples' channels. A TCP connection, mode 12, no translation:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_optcp             ; connect to the test board
        call    fn_call
...
rq_optcp:                               ; mode 12: read and write
        FNREQ   DEV_NETWORK+4, NET_OPEN, F_A1_A2, 12,0,0,0
        FNBUF   p_tcp, P_TCP_LEN, 0, 0
p_tcp:
        defm    "N5:TCP://127.0.0.1:7801/", 0
P_TCP_LEN       equ     $ - p_tcp
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char tcp[] = "N5:TCP://127.0.0.1:7801/";
...
rc = network_open(tcp, OPEN_MODE_RW, OPEN_TRANS_NONE);
```

H89 → FUJINET

C0	75	4F	21	00	23	02	0C	00	4E N	35 5	3A :	54 T	43 C	50 P	3A :
2F /	2F /	31 1	32 2	37 7	2E .	30 0	2E .	30 0	2E .	31 1	3A :	37 7	38 8	30 0	31 1
2F /	00	C0													

FUJINET → H89 · ACK

C0	75	06	06	00	81	00	C0

A web page with translation 2, so its LF line endings arrive as CR LF:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_open              ; the same GET as before
        call    fn_call
        ld      hl,rq_status
        call    fn_call
        ld      hl,rq_read              ; 53 bytes now, not 55: each
        call    getwait                 ;   CR LF came as CR
        call    ptext
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
network_open(hello, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
network_status(hello, &avail, &conn, &nerr);
len = network_read(hello, buf, avail);  /* 53, not 55: each */
show(buf, len);                         /*   CR LF came as CR */
```

H89 → FUJINET

C0	71	4F	2B	00	1D	02	04	00	4E N	31 1	3A :	48 H	54 T	54 T	50 P
3A :	2F /	2F /	31 1	32 2	37 7	2E .	30 0	2E .	30 0	2E .	31 1	3A :	37 7	38 8	30 0
32 2	2F /	68 h	65 e	6C l	6C l	6F o	2E .	74 t	78 x	74 t	00	C0			

FUJINET → H89 · ACK

C0	71	06	06	00	7D	00	C0

A file on the microSD card, opened for reading, to seek in later:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opimg             ; a disk image: 512-byte
        call    fn_call                 ;   sectors, sector n all n
...
rq_opimg:                               ; mode 4: read
        FNREQ   DEV_NETWORK+7, NET_OPEN, F_A1_A2, 4,0,0,0
        FNBUF   p_img, P_IMG_LEN, 0, 0
p_img:
        defm    "N8:SD:/test/disk.img", 0
P_IMG_LEN       equ     $ - p_img
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char img[] = "N8:SD:/test/disk.img";
...
rc = network_open(img, OPEN_MODE_READ, OPEN_TRANS_NONE);
```

H89 → FUJINET

C0	78	4F	1D	00	B6	02	04	00	4E	38	3A	53	44	3A	2F
									N	8	:	S	D	:	/

74	65	73	74	2F	64	69	73	6B	2E	69	6D	67	00	C0
t	e	s	t	/	d	i	s	k	.	i	m	g		

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

And an OPEN that fails: nothing is listening on port 1, so the FujiNet answers NAK, and STATUS (later in the chapter) says why:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opbad             ; a port no one listens on:
        call    fn_call                 ;   NAK
...
rq_opbad:
        FNREQ   DEV_NETWORK, NET_OPEN, F_A1_A2, 12,0,0,0
        FNBUF   p_bad, P_BAD_LEN, 0, 0
p_bad:
        defm    "N1:TCP://127.0.0.1:1/", 0
P_BAD_LEN       equ     $ - p_bad
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_open("N1:TCP://127.0.0.1:1/", OPEN_MODE_RW,
                  OPEN_TRANS_NONE);     /* FN_ERR_IO_ERROR */
```

H89 → FUJINET

C0	71	4F	1E	00	78	02	0C	00	4E	31	3A	54	43	50	3A
									N	1	:	T	C	P	:

2F	2F	31	32	37	2E	30	2E	30	2E	31	3A	31	2F	00	C0
/	/	1	2	7	.	0	.	0	.	1	:	1	/		

FUJINET → H89 · NAK

C0	71	15	06	00	8C	00	C0

CLOSE 43H

DEVICE 71H--78H COMMAND 43H (103Q) NET_CLOSE

FIELDS	none
PAYLOAD	none
REPLY	ACK

Closes the channel. For a file being written, this is when the last of it is written; for an HTTP PUT, this is when the request is sent, with everything written as its body. Closing a closed channel does no harm.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_close             ; done: free the unit
        call    fn_call
...
rq_close:
        FNREQ   DEV_NETWORK, NET_CLOSE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_close(hello);
```

H89 → FUJINET

C0	71	43	06	00	BA	00	C0

FUJINET → H89 · ACK

C0	71	06	06	00	7D	00	C0

STATUS

STATUS 53H

DEVICE 71H--78H COMMAND 53H (123Q) NET_STATUS

FIELDS none
PAYLOAD none
REPLY 4 bytes: bytes waiting (word, low first), connected, error

How the channel stands:

OFFSET	BYTES	HOLDS
0	2	How many bytes the next READ can return.
2	1	1 while the connection is up, or the file or listing has more to give; 0 when it is done.
3	1	The last error: 1 means all is well; 136 the end of the data; others are in the Appendix.

For an HTTP channel the first STATUS is when the request is sent: it waits for the server's answer to begin. For TCP and UDP, STATUS collects what has arrived. A program that only reads after STATUS never asks for more than is there.

The example comes just after the OPEN of `hello.txt`: 55 bytes waiting (37H), connected 0 because the test server had already sent the whole page, error 1.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_status            ; the first STATUS sends the GET
        call    fn_call                 ;   and waits for the answer
        ld      hl,(nstat+NS_AVAIL)     ; bytes waiting to be read
        call    pdec16
...
rq_status:                              ; 4 bytes back: waiting (a
        FNREQ   DEV_NETWORK, NET_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, nstat, 4          ; word), connected, error
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_status(hello, &avail, &conn, &nerr);   /* sends GET */
printf("%u waiting", avail);
```

H89 → FUJINET

C0	71	53	06	00	CA	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	71	06	0A	00	B9	00	37	00	00	01	C0
							7				

STATUS OF A CLOSED CHANNEL 53H

DEVICE 71H--78H COMMAND 53H (123Q)

With nothing open, STATUS answers 00 00 00 CFH: error 207, **not connected**. The file system commands later in this chapter leave their unit like this.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_status            ; nothing is open on unit 1,
        call    fn_call
        ld      a,(nstat+NS_ERR)        ;   so the error is 207,
        call    pdec8                   ;   NOT CONNECTED
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_status(hello, &avail, &conn, &nerr);
printf("%u", nerr);             /* 207: nothing open on unit 1 */
```

H89 → FUJINET

C0	71	53	06	00	CA	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	71	06	0A	00	51	00	00	00	00	CF	C0

STATUS AT THE END 53H

DEVICE 71H--78H COMMAND 53H (123Q)

After the whole page has been read: nothing waiting, error 136, the end.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_status            ; all read: none waiting, not
        call    fn_call                 ;   connected, error 136 (end)
        ld      a,(nstat+NS_ERR)
        call    pdec8
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_status(hello, &avail, &conn, &nerr);
printf("%u", nerr);             /* 136: the end */
```

H89 → FUJINET

C0	71	53	06	00	CA	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	71	06	0A	00	0A	00	00	00	00	88	C0

STATUS AFTER A FAILED OPEN 53H

DEVICE 71H--78H COMMAND 53H (123Q)

After the refused OPEN above: error 200, **connection refused**.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_status            ; why: error 200, connection
        call    fn_call                 ;   refused
        ld      a,(nstat+NS_ERR)
        call    pdec8
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_status("N1:", &avail, &conn, &nerr);
printf("%u", nerr);             /* 200: connection refused */
```

H89 → FUJINET

C0	71	53	06	00	CA	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	71	06	0A	00	4A	00	00	00	00	C8	C0

READING AND WRITING

READ 52H

DEVICE 71H--78H COMMAND 52H (122Q) NET_READ

FIELDS	a word: the most bytes to return
PAYLOAD	none
REPLY	up to that many bytes

Takes bytes the channel has waiting. Ask for what STATUS said, or less: the reply may be shorter than asked, and the reply's length says how much came.

WARNING: A READ for more than is waiting can fail, and lose what **was** waiting. On a TCP channel, if nothing was waiting, the FujiNet asks the network for the whole amount at once, and when fewer arrive it answers NAK and throws away the bytes that did come. Always ask STATUS first, and read no more than it says.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_read              ; read what STATUS said waits
        call    getwait
        call    typeout                 ; and type it
...
; HL -> a READ request block. Read what the last STATUS said is
; waiting, a buffer full at most, into buf. HL = buf, BC = count.
getwait:
        push    hl
        pop     ix                      ; IX -> the block
        ld      hl,(nstat+NS_AVAIL)
        ld      de,BUFSZ
        or      a
        sbc     hl,de                   ; more than a buffer?
        add     hl,de
        jr      c,getw1
        ex      de,hl                   ; yes: a buffer full
getw1:
        ld      (ix+RQ_AUX1),l          ; the length to ask for
        ld      (ix+RQ_AUX2),h
        push    ix
        pop     hl
        call    fn_call                 ; BC = what really came
        ld      hl,buf
        ret

; type BC bytes from HL as they are, line ends and all
typeout:
        ld      a,b
        or      c
        ret     z
        ld      a,(hl)
        call    conout
        inc     hl
        dec     bc
        jr      typeout
...
rq_read:                                ; the length is set each time
        FNREQ   DEV_NETWORK, NET_READ, F_B12, 0,0,0,0
        FNBUF   0, 0, buf, BUFSZ
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
/* type len bytes as they are, line ends and all */
static void type(const uint8_t *p, uint16_t len)
{
    while (len--)
        putchar(*p++);
}
...
n = avail < sizeof buf ? avail : sizeof buf;
len = network_read(hello, buf, n);  /* STATUS, then READ */
if (len > 0)
    type(buf, len);
```

H89 → FUJINET

C0	71	52	08	00	08	05	37	00	C0

FUJINET → H89 · ACK · 55 BYTES

C0	71	06	3D	00	6A	00	48 H	65 e	6C l	6C l	6F o	20	66 f	72 r	6F o

6D m	20	74 t	68 h	65 e	20	74 t	65 e	73 s	74 t	20	35 MORE	C0

READ PAST THE END 52H

DEVICE 71H--78H COMMAND 52H (122Q)

With nothing left, READ answers NAK.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,64                   ; a READ past the end
        ld      (rq_read+RQ_AUX1),hl
        ld      hl,rq_read
        call    fn_call                 ; NAK
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
len = network_read(hello, buf, 64);
printf("%d", len);              /* 0: its STATUS saw the end */
```

H89 → FUJINET

C0	71	52	08	00	11	05	40	00	C0

FUJINET → H89 · NAK

C0	71	15	06	00	8C	00	C0

READING SOMETHING BIG 52H

DEVICE 71H--78H COMMAND 52H (122Q)

A file longer than one reply, read in pieces of 2048 bytes: STATUS, READ, and again until STATUS says the end. Here `data.bin`, 3000 bytes, comes as 2048 and 952:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opbin             ; GET a 3000-byte file
        call    fn_call
        ld      hl,0                    ; bytes so far
        ld      (total),hl
rl_next:
        ld      hl,rq_status            ; how many are waiting?
        call    fn_call
        ld      hl,(nstat+NS_AVAIL)
        ld      a,h
        or      l
        jr      nz,rl_read
        ld      a,(nstat+NS_CONN)       ; none: still coming?
        or      a
        jr      nz,rl_next
        jr      rl_done                 ; no: that was all
rl_read:
        ld      hl,rq_read              ; a buffer full at most
        call    getwait
        ld      hl,(total)              ; BC bytes came
        add     hl,bc
        ld      (total),hl
        jr      rl_next
rl_done:
        ld      hl,(total)
        call    pdec16
        ld      hl,rq_close
        call    fn_call
...
rq_opbin:
        FNREQ   DEV_NETWORK, NET_OPEN, F_A1_A2, 4,0,0,0
        FNBUF   p_bin, P_BIN_LEN, 0, 0
p_bin:
        defm    "N1:HTTP://127.0.0.1:7802/data.bin", 0
P_BIN_LEN       equ     $ - p_bin
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char bin[] = "N1:HTTP://127.0.0.1:7802/data.bin";
...
network_open(bin, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
total = 0;
while ((len = network_read_nb(bin, buf, sizeof buf)) > 0)
    total += len;              /* what was there, up to 2048 */
printf("%u bytes", total);     /* 0 from it: the end */
rc = network_close(bin);
```

H89 → FUJINET

C0	71	52	08	00	D8	05	00	08	C0

FUJINET → H89 · ACK · 2048 BYTES

C0	71	06	06	08	E4	00	00	01	02	03	04	05	06	07	08
09	0A	0B	0C	0D	0E	0F	10	11	12	13	2044 MORE				C0

H89 → FUJINET

C0	71	52	08	00	8C	05	B8	03	C0

FUJINET → H89 · ACK · 952 BYTES

C0	71	06	BE	03	D0	00	28 (	29)	2A *	2B +	2C ,	2D -	2E .	2F /	30 0
31 1	32 2	33 3	34 4	35 5	36 6	37 7	38 8	39 9	3A :	3B ;	940 MORE				C0

NOTE: Over HTTP, a READ larger than what is buffered (with no STATUS first) does work: the FujiNet returns what it has. But fujinet-lib's `network_bus_read()` then reports the length you asked for, not the length that came; use `network_read()`, which asks STATUS first.

WRITE 57H

DEVICE 71H--78H COMMAND 57H (127Q) NET_WRITE

FIELDS	a word: how many bytes
PAYLOAD	those bytes
REPLY	ACK

Sends bytes. The field and the payload's length must agree: a payload shorter than the field is made up with zeros. Here a line goes to the test board over TCP:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_wr5               ; send a line
        call    fn_call
...
rq_wr5:                                 ; the length is a word field
        FNREQ   DEV_NETWORK+4, NET_WRITE, F_B12, P_LINE_LEN,0,0,0
        FNBUF   p_line, P_LINE_LEN, 0, 0
p_line:
        defm    "hello from the h89", 13
P_LINE_LEN      equ     $ - p_line
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_write(tcp, "hello from the h89\r", 19);
```

H89 → FUJINET

C0	75	57	1B	00	55	05	13	00	68 h	65 e	6C l	6C l	6F o	20	66 f
72 r	6F o	6D m	20	74 t	68 h	65 e	20	68 h	38 8	39 9	0D	C0			

FUJINET → H89 · ACK

C0	75	06	06	00	81	00	C0

READING A TCP REPLY 52H

DEVICE 71H--78H COMMAND 52H (122Q)

The test board echoes each key as it comes, so its answer arrives in pieces. A program reads until nothing more comes for a while — here, half a second:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      b,100                   ; wait up to 10 s for it,
rt_next:
        ld      hl,rq_st5
        call    waitin
        jr      z,rt_done
        ld      hl,rq_rd5               ; take what has come
        call    getwait
        call    typeout
        ld      b,5                     ; and more, until 0.5 s
        jr      rt_next                 ;   pass with none
rt_done:
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
n = 100;                        /* up to 10 s for the answer, */
while (wait_avail(tcp, n)) {
    len = network_read(tcp, buf, avail);
    if (len > 0)
        type(buf, len);
    n = 5;                      /* then more, until 0.5 s */
}                               /*   pass with none */
```

H89 → FUJINET

C0	75	53	06	00	CE	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	75	06	0A	00	98	00	11	00	01	01	C0

H89 → FUJINET

C0	75	52	08	00	E5	05	11	00	C0

FUJINET → H89 · ACK · 17 BYTES

C0	75	06	17	00	A1	00	68 h	65 e	6C l	6C l	6F o	20	66 f	72 r	6F o
6D m	20	74 t	68 h	65 e	20	68 h	38 8	C0							

H89 → FUJINET

C0	75	53	06	00	CE	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	75	06	0A	00	A0	00	19	00	01	01	C0

H89 → FUJINET

C0	75	52	08	00	ED	05	19	00	C0

FUJINET → H89 · ACK · 25 BYTES

C0	75	06	1F	00	07	00	39 9	0D	0A	48 H	45 E	4C L	4C L	4F O	20
46 F	52 R	4F O	4D M	20	54 T	48 H	45 E	20	48 H	38 8	39 9	0D	0A	3E >	20
C0															

WHEN THE FAR END HANGS UP 53H

DEVICE 71H--78H COMMAND 53H (123Q)

After BYE, the board says goodbye and hangs up. STATUS then says not connected, error 136 — with the goodbye's 15 bytes still waiting to be read. The far end has gone only when **nothing** is waiting and it is not connected.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_wbye              ; BYE: the board hangs up
        call    fn_call
        ld      b,50                    ; watch for that, 5 s at most
rb_next:
        call    pause
        push    bc
        ld      hl,rq_st5
        call    fn_call
        pop     bc
        ld      a,(nstat+NS_CONN)       ; still connected?
        or      a
        jr      z,rb_gone
        djnz    rb_next
```

```
rb_gone:
        ld      a,(nstat+NS_ERR)        ; gone: error 136, but its
        call    pdec8                   ;   goodbye is still there
        call    pspace                  ;   to be read
        ld      hl,(nstat+NS_AVAIL)
        call    pdec16
...
rq_wbye:
        FNREQ   DEV_NETWORK+4, NET_WRITE, F_B12, P_BYE_LEN,0,0,0
        FNBUF   p_bye, P_BYE_LEN, 0, 0
p_bye:
        defm    "BYE", 13
P_BYE_LEN       equ     $ - p_bye
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
network_write(tcp, "BYE\r", 4); /* the board hangs up */
n = 0;
do {                            /* watch for that, 5 s at most */
    pause_tenth();
    network_status(tcp, &avail, &conn, &nerr);
} while (conn && ++n < 50);
printf("%u %u", nerr, avail);   /* 136, but the goodbye is */
                                /*   still there to be read */
```

H89 → FUJINET

C0	75	57	0C	00	CF	05	04	00	42 B	59 Y	45 E	0D	C0

FUJINET → H89 · ACK

C0	75	06	06	00	81	00	C0

H89 → FUJINET

C0	75	53	06	00	CE	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	75	06	0A	00	1D	00	0F	00	00	88	C0

JSON AND OTHER PARSERS

A channel can parse what it reads. With the parser set to JSON, the FujiNet reads the whole document into its own memory, and from then on READ returns only the values a program asks for, one at a time, by **JSON Pointer** (Chapter 15 explains them, and uses them). The parser can also be HTML, with CSS selectors, or XML.

SET PARSER FCH

DEVICE 71H--78H COMMAND FCH (374Q) NET_SET_PARSER

FIELDS	2 bytes: 0 (not used, but send it); the parser
PAYLOAD	none
REPLY	ACK

Chooses the channel's parser: 0 none, 1 JSON, 2 HTML, 3 XML. Send it after OPEN: an OPEN sets the parser back to none. With a parser set, WRITE always fails, so finish writing (an HTTP POST's body) first.

WARNING: Never send SET_PARSER to a unit with no channel open. It is accepted, but a later READ, QUERY, SEEK or TELL on that unit stops the firmware.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_parser            ; treat it as JSON (after OPEN:
        call    fn_call                 ;   OPEN resets the parser)
...
rq_parser:                              ; field 2: 1 = JSON
        FNREQ   DEV_NETWORK+1, NET_SET_PARSER, F_A1_A2, 0,1,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_parser(json, PARSER_JSON);     /* after OPEN */
```

H89 → FUJINET

C0	72	FC	08	00	7A	02	00	01	C0

FUJINET → H89 · ACK

C0	72	06	06	00	7E	00	C0

PARSE 50H

DEVICE 71H--78H COMMAND 50H (120Q) NET_PARSE

FIELDS	none
PAYLOAD	none
REPLY	ACK, or NAK if it will not parse

Reads the whole document and parses it. For HTTP, this is when the request goes. NAK, and error 213 in STATUS, if the document is not good JSON. Afterwards STATUS shows nothing waiting and error 136: the document has been read.

WARNING: On a connection that stays open (TCP), PARSE does not finish until the far end hangs up.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_parse             ; fetch it all and parse it
        call    fn_call
...
rq_parse:
        FNREQ   DEV_NETWORK+1, NET_PARSE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_json_parse(json);  /* sets the parser itself too */
```

H89 → FUJINET

C0	72	50	06	00	C8	00	C0

FUJINET → H89 · ACK

C0	72	06	06	00	7E	00	C0

QUERY 51H

DEVICE 71H--78H COMMAND 51H (121Q) NET_QUERY

FIELDS none
PAYLOAD a JSON Pointer, such as `/name`
REPLY ACK; then STATUS and READ for the value

Asks for one value. The answer is put in the channel's buffer as text, followed by a line feed (0AH): a string without its quotes, a number as digits, TRUE, FALSE or NULL. STATUS gives its length, and READ takes it. The payload is the pointer itself; a zero ends it.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_qname             ; the value of "name"
        call    fn_call
        call    answer                  ; "FujiNet", then 0AH
...
; STATUS, then READ the answer QUERY left waiting, and show it
answer:
        ld      hl,rq_st2
        call    fn_call
        ld      hl,rq_rd2
        call    getwait
        jp      ptext
...
rq_qname:                               ; the payload is the query
        FNREQ   DEV_NETWORK+1, NET_QUERY, F_NONE, 0,0,0,0
        FNBUF   p_qname, P_QNAME_LEN, 0, 0
p_qname:
        defm    "/name"
P_QNAME_LEN     equ     $ - p_qname
rq_rd2:
        FNREQ   DEV_NETWORK+1, NET_READ, F_B12, 0,0,0,0
        FNBUF   0, 0, buf, BUFSZ
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
len = network_json_query(json, "/name", (char *)buf);
printf("%s", buf);              /* "FujiNet", line end removed */
```

H89 → FUJINET

C0	72	51	0B	00	A0	00	2F /	6E n	61 a	6D m	65 e	C0

FUJINET → H89 · ACK

C0	72	06	06	00	7E	00	C0

H89 → FUJINET

C0	72	53	06	00	CB	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	72	06	0A	00	8B	00	08	00	00	01	C0

H89 → FUJINET

C0	72	52	08	00	D9	05	08	00	C0

FUJINET → H89 · ACK · 8 BYTES

C0	72	06	0E	00	48	00	46 F	75 u	6A j	69 i	4E N	65 e	74 t	0A	C0

QUERY AN ARRAY 51H

DEVICE 71H--78H COMMAND 51H (121Q)

Array elements count from 0: `/ports/2` is the third.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_qport             ; the third of the "ports"
        call    fn_call                 ;   (they count from 0)
        call    answer                  ; "210", then 0AH
...
rq_qport:
        FNREQ   DEV_NETWORK+1, NET_QUERY, F_NONE, 0,0,0,0
        FNBUF   p_qport, P_QPORT_LEN, 0, 0
p_qport:
        defm    "/ports/2"
P_QPORT_LEN     equ     $ - p_qport
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
/* show len bytes, anything unprintable as '.' */
static void show(const uint8_t *p, uint16_t len)
{
    for (; len; len--, p++)
        putchar(*p >= ' ' && *p < 0x7F ? *p : '.');
}
...
len = network_json_query(json, "/ports/2", (char *)buf);
show(buf, len);                 /* "210": they count from 0 */
```

H89 → FUJINET

C0	72	51	0E	00	9C	00	2F /	70 p	6F o	72 r	74 t	73 s	2F /	32 2	C0

FUJINET → H89 · ACK

C0	72	06	06	00	7E	00	C0

H89 → FUJINET

C0	72	53	06	00	CB	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	72	06	0A	00	87	00	04	00	00	01	C0

H89 → FUJINET

C0	72	52	08	00	D5	05	04	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	72	06	0A	00	20	00	32 2	31 1	30 0	0A	C0

QUERY SOMETHING MISSING 51H

DEVICE 71H--78H COMMAND 51H (121Q)

A pointer that names nothing still gets ACK, but STATUS then shows nothing waiting.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_qnone             ; a name that is not there:
        call    fn_call                 ;   still ACK,
        ld      hl,rq_st2
        call    fn_call
        ld      hl,(nstat+NS_AVAIL)     ;   but nothing to read
        call    pdec16
...
rq_qnone:
        FNREQ   DEV_NETWORK+1, NET_QUERY, F_NONE, 0,0,0,0
        FNBUF   p_qnone, P_QNONE_LEN, 0, 0
p_qnone:
        defm    "/serial"
P_QNONE_LEN     equ     $ - p_qnone
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
len = network_json_query(json, "/serial", (char *)buf);
printf("%d", len);              /* 0: not there, but no error */
```

H89 → FUJINET

C0	72	51	0D	00	82	00	2F /	73 s	65 e	72 r	69 i	61 a	6C l	C0

FUJINET → H89 · ACK

C0	72	06	06	00	7E	00	C0

H89 → FUJINET

C0	72	53	06	00	CB	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	72	06	0A	00	0B	00	00	00	00	88	C0

WARNING: A query whose answer contains an empty array, `[]`, stops the firmware.

SET PARAMETER FBH

DEVICE 71H--78H COMMAND FBH (373Q) NET_SET_PARAMETER

FIELDS	2 bytes: which; the value
PAYLOAD	none
REPLY	ACK

Tunes the parser: with 1, the value is the line ending put after each answer (normally 0AH); with 0, flags for HTML and XML text. Send it after SET_PARSER, which starts a fresh parser. The example makes the line ending CR, then asks for `/machine`.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_qeol              ; end answers with CR, not LF
        call    fn_call
        ld      hl,rq_qmach             ; the value of "machine"
        call    fn_call
        call    answer                  ; "H89", then 0DH
...
rq_qeol:                                ; parameter 1: the line end
        FNREQ   DEV_NETWORK+1, NET_SET_PARAMETER, F_A1_A2, 1,13,0,0
        FNBUF   0, 0, 0, 0
rq_qmach:
        FNREQ   DEV_NETWORK+1, NET_QUERY, F_NONE, 0,0,0,0
        FNBUF   p_qmach, P_QMACH_LEN, 0, 0
p_qmach:
        defm    "/machine"
P_QMACH_LEN     equ     $ - p_qmach
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
network_set_parameter(json, 1, '\r');   /* 1: the line end */
len = network_json_query(json, "/machine", (char *)buf);
show(buf, len);                 /* "H89" (the CR is stripped) */
```

H89 → FUJINET

C0	72	FB	08	00	86	02	01	0D	C0

FUJINET → H89 · ACK

C0	72	06	06	00	7E	00	C0

H89 → FUJINET

C0	72	51	0E	00	D8	00	2F /	6D m	61 a	63 c	68 h	69 i	6E n	65 e	C0

FUJINET → H89 · ACK

C0	72	06	06	00	7E	00	C0

H89 → FUJINET

C0	72	53	06	00	CB	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	72	06	0A	00	87	00	04	00	00	01	C0

H89 → FUJINET

C0	72	52	08	00	D5	05	04	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	72	06	0A	00	49	00	48 H	38 8	39 9	0D	C0

HTTP

An HTTP channel is opened with the method as its mode (OPEN's table, above). Reading the reply is ordinary: STATUS, then READ. Three things need more: sending a body (POST, PUT), sending request headers, and reading response headers. All three use SET_HTTP_MODE to say what the next WRITEs and READs mean.

MODE	WRITE AND READ THEN MEAN
0	Body: READ returns the reply's body. For PUT, WRITE adds to the body.
1	Collect: each WRITE names a response header to keep. Mode 0CH GETs only.
2	Get headers: each READ returns one kept header's value.
3	Set headers: each WRITE is a request header, `Name: value`.
4	Post data: WRITE adds to a POST's (or PUT's) body.

SET HTTP MODE 4DH

DEVICE 71H--78H COMMAND 4DH (115Q) NET_SET_HTTP_MODE

FIELDS	2 bytes: 0 (not used, but send it); the mode
PAYLOAD	none
REPLY	ACK

Changes what WRITE and READ mean on an HTTP channel. NAK on any other kind of channel. Changing mode throws away anything buffered.

A POST, step by step: open with mode 0DH, set mode 4, WRITE the body, set mode 0, then STATUS (which sends the request) and READ the answer.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_mpost             ; what is written now is the
        call    fn_call                 ;   body of the POST
...
rq_mpost:                               ; mode 4: the POST data
        FNREQ   DEV_NETWORK+2, NET_SET_HTTP_MODE, F_A1_A2, 0,4,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_http_set_mode(post, HTTP_CHAN_MODE_POST_SET_DATA);
```

H89 → FUJINET

C0	73	4D	08	00	CE	02	00	04	C0

FUJINET → H89 · ACK

C0	73	06	06	00	7F	00	C0

POST: THE BODY 57H

DEVICE 71H--78H COMMAND 57H (127Q)

The body, name=H89&cpu=Z80:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_wpost             ; the body
        call    fn_call
...
rq_wpost:
        FNREQ   DEV_NETWORK+2, NET_WRITE, F_B12, P_FORM_LEN,0,0,0
        FNBUF   p_form, P_FORM_LEN, 0, 0
p_form:
        defm    "name=H89&cpu=Z80"
P_FORM_LEN      equ     $ - p_form
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_write(post, "name=H89&cpu=Z80", 16);  /* the body */
```

H89 → FUJINET

C0	73	57	18	00	01	05	10	00	6E n	61 a	6D m	65 e	3D =	48 H	38 8

39 9	26 &	63 c	70 p	75 u	3D =	5A Z	38 8	30 0	C0

FUJINET → H89 · ACK

C0	73	06	06	00	7F	00	C0

POST: BACK TO THE BODY MODE 4DH

DEVICE 71H--78H COMMAND 4DH (115Q)

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_mbody             ; back to the body mode: READ
        call    fn_call                 ;   now gets the answer
...
rq_mbody:                               ; mode 0: the body
        FNREQ   DEV_NETWORK+2, NET_SET_HTTP_MODE, F_A1_A2, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_http_set_mode(post, HTTP_CHAN_MODE_BODY);
```

H89 → FUJINET

C0	73	4D	08	00	CA	02	00	00	C0

FUJINET → H89 · ACK

C0	73	06	06	00	7F	00	C0

POST: SENDING IT 53H

DEVICE 71H--78H COMMAND 53H (123Q)

This STATUS sends the request. The test server answered “GOT 16 BYTES”, 14 bytes with its CR LF:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_st3               ; this sends the POST
        call    fn_call
        ld      hl,(nstat+NS_AVAIL)     ; the answer's length
        call    pdec16
...
rq_st3:
        FNREQ   DEV_NETWORK+2, NET_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, nstat, 4
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_status(post, &avail, &conn, &nerr);   /* sends it */
printf("%u waiting", avail);
```

H89 → FUJINET

C0	73	53	06	00	CC	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	73	06	0A	00	92	00	0E	00	00	01	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_rd3               ; the server's answer
        call    getwait
        call    typeout
...
rq_rd3:
        FNREQ   DEV_NETWORK+2, NET_READ, F_B12, 0,0,0,0
        FNBUF   0, 0, buf, BUFSZ
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
len = network_read(post, buf, avail);   /* the server's answer */
if (len > 0)
    type(buf, len);
```

H89 → FUJINET

C0	73	52	08	00	E0	05	0E	00	C0

FUJINET → H89 · ACK · 14 BYTES

C0	73	06	14	00	BF	00	47 G	4F O	54 T	20	31 1	36 6	20	42 B	59 Y

54 T	45 E	53 S	0D	0A	C0

PUT 57H

DEVICE 71H--78H COMMAND 57H (127Q)

A PUT (mode 08H) is simpler: WRITE the body in mode 0, then CLOSE, which sends it.

WARNING: Don’t ask a PUT channel for STATUS. The first STATUS sends the PUT; CLOSE then sends it again.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opput             ; PUT a file on the server
        call    fn_call
...
rq_opput:                               ; mode 8: PUT
        FNREQ   DEV_NETWORK+2, NET_OPEN, F_A1_A2, 8,0,0,0
        FNBUF   p_put, P_PUT_LEN, 0, 0
p_put:
        defm    "N3:HTTP://127.0.0.1:7802/put.txt", 0
P_PUT_LEN       equ     $ - p_put
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char put[] = "N3:HTTP://127.0.0.1:7802/put.txt";
...
rc = network_open(put, OPEN_MODE_HTTP_PUT, OPEN_TRANS_NONE);
```

H89 → FUJINET

C0	73	4F	29	00	67	02	08	00	4E N	33 3	3A :	48 H	54 T	54 T	50 P
3A :	2F /	2F /	31 1	32 2	37 7	2E .	30 0	2E .	30 0	2E .	31 1	3A :	37 7	38 8	30 0
32 2	2F /	70 p	75 u	74 t	2E .	74 t	78 x	74 t	00	C0					

FUJINET → H89 · ACK

C0	73	06	06	00	7F	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_wput              ; for PUT, plain writes are the
        call    fn_call                 ;   body; they wait for CLOSE
...
rq_wput:
        FNREQ   DEV_NETWORK+2, NET_WRITE, F_B12, P_PUTB_LEN,0,0,0
        FNBUF   p_putb, P_PUTB_LEN, 0, 0
p_putb:
        defm    "PUT from an H89"
P_PUTB_LEN      equ     $ - p_putb
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
/* the body goes in as POST data does; it waits for CLOSE */
rc = network_http_put(put, "PUT from an H89");
```

H89 → FUJINET

C0	73	57	17	00	8F	05	0F	00	50 P	55 U	54 T	20	66 f	72 r	6F o
6D m	20	61 a	6E n	20	48 H	38 8	39 9	C0							

FUJINET → H89 · ACK

C0	73	06	06	00	7F	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_close3            ; this sends the PUT
        call    fn_call
...
rq_close3:
        FNREQ   DEV_NETWORK+2, NET_CLOSE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_close(put);        /* this sends the PUT */
```

H89 → FUJINET

C0	73	43	06	00	BC	00	C0

FUJINET → H89 · ACK

C0	73	06	06	00	7F	00	C0

DELETE

4FH

DEVICE 71H--78H COMMAND 4FH (117Q)

A DELETE (mode 05H) goes when the program asks STATUS:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opdel             ; DELETE a file on the server
        call    fn_call
...
rq_opdel:                               ; mode 5: DELETE
        FNREQ   DEV_NETWORK+2, NET_OPEN, F_A1_A2, 5,0,0,0
        FNBUF   p_del, P_DEL_LEN, 0, 0
p_del:
        defm    "N3:HTTP://127.0.0.1:7802/old.txt", 0
P_DEL_LEN       equ     $ - p_del
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char del[] = "N3:HTTP://127.0.0.1:7802/old.txt";
...
rc = network_http_delete(del, OPEN_TRANS_NONE);    /* mode 9 */
```

H89 → FUJINET

C0	73	4F	29	00	4A	02	05	00	4E N	33 3	3A :	48 H	54 T	54 T	50 P
3A :	2F /	2F /	31 1	32 2	37 7	2E .	30 0	2E .	30 0	2E .	31 1	3A :	37 7	38 8	30 0
32 2	2F /	6F o	6C l	64 d	2E .	74 t	78 x	74 t	00	C0					

FUJINET → H89 · ACK

C0	73	06	06	00	7F	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_st3               ; this sends the DELETE
        call    fn_call
        ld      hl,(nstat+NS_AVAIL)     ; the answer's length
        call    pdec16
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_status(del, &avail, &conn, &nerr);    /* sends it */
printf("%u waiting", avail);
```

H89 → FUJINET

C0	73	53	06	00	CC	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	73	06	0A	00	8D	00	09	00	00	01	C0

DELETE THAT IS NEVER SENT

4FH

DEVICE 71H--78H COMMAND 4FH (117Q)

OPEN, then CLOSE: the test server never heard of `/never.txt`.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opdel2            ; DELETE another file, but
        call    fn_call                 ;   CLOSE at once; with no
        ld      hl,rq_close3            ;   STATUS or READ between,
        call    fn_call                 ;   the DELETE is never sent
...
rq_opdel2:
        FNREQ   DEV_NETWORK+2, NET_OPEN, F_A1_A2, 5,0,0,0
        FNBUF   p_del2, P_DEL2_LEN, 0, 0
p_del2:
        defm    "N3:HTTP://127.0.0.1:7802/never.txt", 0
P_DEL2_LEN      equ     $ - p_del2
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char del2[] = "N3:HTTP://127.0.0.1:7802/never.txt";
...
network_open(del2, OPEN_MODE_HTTP_DELETE, OPEN_TRANS_NONE);
rc = network_close(del2);       /* no STATUS or READ between: */
                                /*   the DELETE is never sent */
```

H89 → FUJINET

C0	73	4F	2B	00	2E	02	05	00	4E N	33 3	3A :	48 H	54 T	54 T	50 P
3A :	2F /	2F /	31 1	32 2	37 7	2E .	30 0	2E .	30 0	2E .	31 1	3A :	37 7	38 8	30 0
32 2	2F /	6E n	65 e	76 v	65 e	72 r	2E .	74 t	78 x	74 t	00	C0			

FUJINET → H89 · ACK

C0	73	06	06	00	7F	00	C0

H89 → FUJINET

C0	73	43	06	00	BC	00	C0

FUJINET → H89 · ACK

C0	73	06	06	00	7F	00	C0

MODE 0EH SENDS POST

4FH

DEVICE 71H--78H COMMAND 4FH (117Q)

“PUT with headers” (0EH) goes out as a POST: the test server’s log shows the method.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opputh            ; mode 14, "PUT with headers"
```

```
        call    fn_call
        ld      hl,rq_mpost             ; a body, as for POST
        call    fn_call
        ld      hl,rq_wputh
        call    fn_call
        ld      hl,rq_mbody
        call    fn_call
        ld      hl,rq_st3               ; sent -- but as a POST
        call    fn_call
...
rq_opputh:                              ; mode 14
        FNREQ   DEV_NETWORK+2, NET_OPEN, F_A1_A2, 14,0,0,0
        FNBUF   p_puth, P_PUTH_LEN, 0, 0
p_puth:
        defm    "N3:HTTP://127.0.0.1:7802/puth.txt", 0
P_PUTH_LEN      equ     $ - p_puth
rq_wputh:
        FNREQ   DEV_NETWORK+2, NET_WRITE, F_B12, P_PUTHB_LEN,0,0,0
        FNBUF   p_puthb, P_PUTHB_LEN, 0, 0
p_puthb:
        defm    "PUT or POST?"
P_PUTHB_LEN     equ     $ - p_puthb
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char puth[] = "N3:HTTP://127.0.0.1:7802/puth.txt";
...
network_open(puth, OPEN_MODE_HTTP_PUT_H, OPEN_TRANS_NONE);
network_http_put(puth, "PUT or POST?");
rc = network_status(puth, &avail, &conn, &nerr);   /* a POST! */
```

H89 → FUJINET

C0	73	4F	2A	00	D6	02	0E	00	4E N	33 3	3A :	48 H	54 T	54 T	50 P
3A :	2F /	2F /	31 1	32 2	37 7	2E .	30 0	2E .	30 0	2E .	31 1	3A :	37 7	38 8	30 0
32 2	2F /	70 p	75 u	74 t	68 h	2E .	74 t	78 x	74 t	00	C0				

FUJINET → H89 · ACK

C0	73	06	06	00	7F	00	C0

H89 → FUJINET

C0	73	53	06	00	CC	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	73	06	0A	00	92	00	0E	00	00	01	C0

REQUEST HEADERS 57H

DEVICE 71H--78H COMMAND 57H (127Q)

On a channel opened with mode 0CH (or 09H, 0DH), set mode 3 and WRITE each header as `Name: value`, with exactly one space after the colon. Here `X-Machine: H89`:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opgeth            ; mode 12: a GET whose headers
        call    fn_call                 ;   we can set and read
...
rq_opgeth:                              ; mode 12: GET, headers
        FNREQ   DEV_NETWORK+3, NET_OPEN, F_A1_A2, 12,0,0,0
        FNBUF   p_geth, P_GETH_LEN, 0, 0
p_geth:
        defm    "N4:HTTP://127.0.0.1:7802/hello.txt", 0
P_GETH_LEN      equ     $ - p_geth
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char geth[] = "N4:HTTP://127.0.0.1:7802/hello.txt";
...
rc = network_open(geth, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE);
```

H89 → FUJINET

C0	74	4F	2B	00	2B	02	0C	00	4E N	34 4	3A :	48 H	54 T	54 T	50 P
3A :	2F /	2F /	31 1	32 2	37 7	2E .	30 0	2E .	30 0	2E .	31 1	3A :	37 7	38 8	30 0
32 2	2F /	68 h	65 e	6C l	6C l	6F o	2E .	74 t	78 x	74 t	00	C0			

FUJINET → H89 · ACK

C0	74	06	06	00	80	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_mseth             ; writes are request headers
        call    fn_call
...
rq_mseth:                               ; mode 3: set headers
        FNREQ   DEV_NETWORK+3, NET_SET_HTTP_MODE, F_A1_A2, 0,3,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_http_start_add_headers(geth);  /* mode 3 */
```

H89 → FUJINET

C0	74	4D	08	00	CE	02	00	03	C0

FUJINET → H89 · ACK

C0	74	06	06	00	80	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_whead             ; one header, "Name: value"
        call    fn_call
...
rq_whead:                               ; one space after the colon
        FNREQ   DEV_NETWORK+3, NET_WRITE, F_B12, P_HEAD_LEN,0,0,0
        FNBUF   p_head, P_HEAD_LEN, 0, 0
p_head:
        defm    "X-Machine: H89"
P_HEAD_LEN      equ     $ - p_head
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_http_add_header(geth, "X-Machine: H89");
```

H89 → FUJINET

C0	74	57	16	00	46	05	0E	00	58 X	2D -	4D M	61 a	63 c	68 h	69 i
6E n	65 e	3A :	20	48 H	38 8	39 9	C0								

FUJINET → H89 · ACK

C0	74	06	06	00	80	00	C0

RESPONSE HEADERS 52H

DEVICE 71H--78H COMMAND 52H (122Q)

To read headers from the reply: set mode 1, WRITE the name of each header to keep (mode 0CH GETs only), then set mode 2. The request goes at the next STATUS. Each READ then gives one value, ending in 9BH (the Atari's end of line, which the FujiNet uses here on every computer), in alphabetical order of the headers' names — not the order they were asked for.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_mcoll             ; writes name answer headers
        call    fn_call                 ;   to keep
...
rq_mcoll:                               ; mode 1: collect headers
        FNREQ   DEV_NETWORK+3, NET_SET_HTTP_MODE, F_A1_A2, 0,1,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_http_set_mode(geth, HTTP_CHAN_MODE_COLLECT_HEADERS);
```

H89 → FUJINET

C0	74	4D	08	00	CC	02	00	01	C0

FUJINET → H89 · ACK

C0	74	06	06	00	80	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_wcoll1            ; keep these two
        call    fn_call
        ld      hl,rq_wcoll2
        call    fn_call
...
rq_wcoll1:
        FNREQ   DEV_NETWORK+3, NET_WRITE, F_B12, P_COLL1_LEN,0,0,0
        FNBUF   p_coll1, P_COLL1_LEN, 0, 0
p_coll1:
        defm    "X-Test-Server"
P_COLL1_LEN     equ     $ - p_coll1
rq_wcoll2:
        FNREQ   DEV_NETWORK+3, NET_WRITE, F_B12, P_COLL2_LEN,0,0,0
        FNBUF   p_coll2, P_COLL2_LEN, 0, 0
p_coll2:
        defm    "Content-Type"
P_COLL2_LEN     equ     $ - p_coll2
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
network_write(geth, "X-Test-Server", 13);   /* keep these two */
rc = network_write(geth, "Content-Type", 12);
```

H89 → FUJINET

C0	74	57	15	00	DB	DC	05	0D	00	58 X	2D -	54 T	65 e	73 s	74 t

2D -	53 S	65 e	72 r	76 v	65 e	72 r	C0

FUJINET → H89 · ACK

C0	74	06	06	00	80	00	C0

H89 → FUJINET

C0	74	57	14	00	9F	05	0C	00	43 C	6F o	6E n	74 t	65 e	6E n	74 t

2D -	54 T	79 y	70 p	65 e	C0

FUJINET → H89 · ACK

C0	74	06	06	00	80	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_mgeth             ; this sends the GET; reads
        call    fn_call                 ;   now give the kept headers
...
rq_mgeth:                               ; mode 2: get headers
        FNREQ   DEV_NETWORK+3, NET_SET_HTTP_MODE, F_A1_A2, 0,2,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_http_set_mode(geth, HTTP_CHAN_MODE_GET_HEADERS);
```

H89 → FUJINET

C0	74	4D	08	00	CD	02	00	02	C0

FUJINET → H89 · ACK

C0	74	06	06	00	80	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      b,2                     ; two were kept
rh_next:
        push    bc
        ld      hl,rq_st4               ; the next one's length
        call    fn_call
        ld      hl,rq_rd4               ; its value, ended by 9BH
        call    getwait
        call    ptext
        call    crlf
        pop     bc
        djnz    rh_next
...
rq_st4:
        FNREQ   DEV_NETWORK+3, NET_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, nstat, 4
rq_rd4:
        FNREQ   DEV_NETWORK+3, NET_READ, F_B12, 0,0,0,0
        FNBUF   0, 0, buf, BUFSZ
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
for (n = 0; n < 2; n++) {       /* exactly two: no more */
    network_status(geth, &avail, &conn, &nerr);
    len = network_read(geth, buf, avail);   /* ends with 9BH */
    show(buf, len);
    putchar('\n');
}
```

H89 → FUJINET

C0	74	53	06	00	CD	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	74	06	0A	00	90	00	0B	00	00	01	C0

H89 → FUJINET

C0	74	52	08	00	DE	05	0B	00	C0

FUJINET → H89 · ACK · 11 BYTES

C0	74	06	11	00	33	00	74 t	65 e	78 x	74 t	2F /	70 p	6C l	61 a	69 i

6E n	9B	C0

H89 → FUJINET

C0	74	53	06	00	CD	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	74	06	0A	00	8E	00	09	00	00	01	C0

H89 → FUJINET

C0	74	52	08	00	DC	05	09	00	C0

FUJINET → H89 · ACK · 9 BYTES

C0	74	06	0F	00	AB	00	48 H	38 8	39 9	20	62 b	6F o	6F o	6B k	9B

C0

TCP SERVERS

A TCP channel opened with no host, `N:TCP://:7803/`, **listens**: other computers can connect to the FujiNet on that port, and the H89 can serve them.

LISTENING 4FH

DEVICE 71H--78H COMMAND 4FH (117Q)

Open the server, then watch STATUS: on a listening channel, "connected" is 1 when a caller is waiting to be accepted.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opsrv             ; listen on port 7803
        call    fn_call
...
rq_opsrv:                               ; no host: listen
        FNREQ   DEV_NETWORK+5, NET_OPEN, F_A1_A2, 12,0,0,0
        FNBUF   p_srv, P_SRV_LEN, 0, 0
p_srv:
        defm    "N6:TCP://:7803/", 0
P_SRV_LEN       equ     $ - p_srv
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char srv[] = "N6:TCP://:7803/";
...
rc = network_open(srv, OPEN_MODE_RW, OPEN_TRANS_NONE);  /* listen */
```

H89 → FUJINET

C0	76	4F	18	00	67	02	0C	00	4E N	36 6	3A :	54 T	43 C	50 P	3A :
2F /	2F /	3A :	37 7	38 8	30 0	33 3	2F /	00	C0						

FUJINET → H89 · ACK

C0	76	06	06	00	82	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_st6               ; wait for a caller
        call    waitcall
...
; HL -> a listener's STATUS request block. Ask every 0.5 s, for
; up to 15 s, until a caller waits ("connected" = 1): NZ if one
; does, Z if no one came.
waitcall:
        ld      (wt_rq),hl
        ld      b,30
waitc1:
        push    bc
        ld      hl,(wt_rq)
        call    fn_call
        pop     bc
        ld      a,(nstat+NS_CONN)
        or      a
        ret     nz
        ld      c,5
waitc2:
        call    pause
        dec     c
        jr      nz,waitc2
        djnz    waitc1
        ret
...
rq_st6:
        FNREQ   DEV_NETWORK+5, NET_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, nstat, 4
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
/* Ask every 0.5 s, for up to 15 s, until a caller waits
   ("connected" = 1): 1 if one does, 0 if no one came. */
static uint8_t wait_caller(const char *spec)
{
    uint8_t n, i;

    for (n = 0; n < 30; n++) {
        network_status(spec, &avail, &conn, &nerr);
        if (conn)
            break;
        for (i = 0; i < 5; i++)
            pause_tenth();
    }
    return conn;
}
...
ok = wait_caller(srv);
```

H89 → FUJINET

C0	76	53	06	00	CF	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	76	06	0A	00	88	00	00	00	01	01	C0

ACCEPT 41H

DEVICE 71H--78H COMMAND 41H (101Q) NET_ACCEPT

FIELDS	none
PAYLOAD	none
REPLY	ACK, or NAK

Takes the waiting caller. From then on READ, WRITE and STATUS on the channel talk to it, and the channel goes on listening for the next. NAK if nobody is waiting (error 209) or the channel is not a server (208).

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_accept            ; take the call
        call    fn_call
...
rq_accept:
        FNREQ   DEV_NETWORK+5, NET_ACCEPT, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_accept(srv);       /* take the call */
```

H89 → FUJINET

C0	76	41	06	00	BD	00	C0

FUJINET → H89 · ACK

C0	76	06	06	00	82	00	C0

TALKING TO THE CALLER 52H

DEVICE 71H--78H COMMAND 52H (122Q)

The test client sends `PING`; the H89 answers `PONG`:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_st6               ; the caller says something
        ld      b,100
        call    waitin
        ld      hl,rq_rd6
        call    getwait
        call    typeout
...
rq_rd6:
        FNREQ   DEV_NETWORK+5, NET_READ, F_B12, 0,0,0,0
        FNBUF   0, 0, buf, BUFSZ
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
len = network_read(srv, buf, wait_avail(srv, 100));
if (len > 0)
    type(buf, len);
```

H89 → FUJINET

C0	76	53	06	00	CF	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	76	06	0A	00	8E	00	06	00	01	01	C0

H89 → FUJINET

C0	76	52	08	00	DB	DD	05	06	00	C0

FUJINET → H89 · ACK · 6 BYTES

C0	76	06	0C	00	CE	00	50 P	49 I	4E N	47 G	0D	0A	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_wr6               ; and is answered
        call    fn_call
...
rq_wr6:
        FNREQ   DEV_NETWORK+5, NET_WRITE, F_B12, P_PONG_LEN,0,0,0
        FNBUF   p_pong, P_PONG_LEN, 0, 0
p_pong:
        defm    "PONG", 13, 10
P_PONG_LEN      equ     $ - p_pong
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_write(srv, "PONG\r\n", 6);
```

H89 → FUJINET

C0	76	57	0E	00	33	05	06	00	50 P	4F O	4E N	47 G	0D	0A	C0

FUJINET → H89 · ACK

C0	76	06	06	00	82	00	C0

CLOSE CLIENT 63H

DEVICE 71H--78H COMMAND 63H (143Q) NET_CLOSE_CLIENT

FIELDS	none
PAYLOAD	none
REPLY	**NAK** if it hung up on a caller

Hangs up on the accepted caller, and goes on listening.

WARNING: The answer is backwards: NAK means the caller **was** hung up on, and ACK that there was no caller to hang up on. fujinet-lib's `network_close_client()` passes this on, so it reports failure when it succeeded.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_st6               ; the next caller:
        call    waitcall
        ld      hl,rq_accept            ;   take the call
        call    fn_call
        ld      hl,rq_cclient           ;   and hang up. NAK: it WAS
        call    fn_call                 ;   done (the sense is inverted)
...
rq_cclient:
        FNREQ   DEV_NETWORK+5, NET_CLOSE_CLIENT, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
wait_caller(srv);               /* the next caller: */
network_accept(srv);
rc = network_close_client(srv); /* FN_ERR_IO_ERROR means it */
                                /*   DID hang up: inverted */
```

H89 → FUJINET

C0	76	63	06	00	DF	00	C0

FUJINET → H89 · NAK

C0	76	15	06	00	91	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_cclient           ; no one to hang up on: ACK
        call    fn_call
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_close_client(srv); /* nothing to do: FN_ERR_OK */
```

H89 → FUJINET

C0	76	63	06	00	DF	00	C0

FUJINET → H89 · ACK

C0	76	06	06	00	82	00	C0

UDP

A UDP channel sends and receives **datagrams**, single messages, rather than a stream. Open it on a port, `N:UDP://:7804/`, to receive; SET_DESTINATION says where WRITE sends.

READING A DATAGRAM 52H

DEVICE 71H--78H COMMAND 52H (122Q)

WARNING: STATUS does not count the datagram that is waiting, only the one after it: with one datagram in, it says 0 waiting. So a UDP program asks STATUS, then READs anyway, with room for the largest datagram it expects: the reply is the datagram. (If nothing has come, the READ answers NAK; if the FujiNet's buffer is empty but a datagram is in, it is padded with zeros to the length asked.)

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opudp             ; take datagrams on port 7804
        call    fn_call
...
rq_opudp:                               ; no host: take datagrams
        FNREQ   DEV_NETWORK+6, NET_OPEN, F_A1_A2, 12,0,0,0
        FNBUF   p_udp, P_UDP_LEN, 0, 0
p_udp:
        defm    "N7:UDP://:7804/", 0
P_UDP_LEN       equ     $ - p_udp
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char udp[] = "N7:UDP://:7804/";
...
rc = network_open(udp, OPEN_MODE_RW, OPEN_TRANS_NONE);
```

H89 → FUJINET

C0	77	4F	18	00	6C	02	0C	00	4E N	37 7	3A :	55 U	44 D	50 P	3A :

2F /	2F /	3A :	37 7	38 8	30 0	34 4	2F /	00	C0

FUJINET → H89 · ACK

C0	77	06	06	00	83	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      b,30                    ; up to 3 s
ru_next:
        push    bc
        call    pause
        ld      hl,rq_st7               ; STATUS takes a datagram in
        call    fn_call                 ;   but counts only the NEXT
        ld      hl,rq_rd7               ;   one, so READ anyway: NAK
        call    fn_call                 ;   means none has come yet
        pop     bc
        jr      z,ru_got
        djnz    ru_next
        jr      ru_none
ru_got:
        ld      bc,(fn_len)             ; one datagram, BC bytes
        ld      hl,buf
        call    ptext
ru_none:
...
rq_st7:
        FNREQ   DEV_NETWORK+6, NET_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, nstat, 4
rq_rd7:                                 ; up to 512 bytes (0200H)
        FNREQ   DEV_NETWORK+6, NET_READ, F_B12, 0,2,0,0
        FNBUF   0, 0, buf, BUFSZ
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
/* STATUS counts only the datagram AFTER the one it took in, so
   wait until that one has come too, then read the first */
len = network_read(udp, buf, wait_avail(udp, 50));
show(buf, len);
```

H89 → FUJINET

C0	77	53	06	00	D0	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	77	06	0A	00	89	00	00	00	01	01	C0

H89 → FUJINET

C0	77	52	08	00	D8	05	00	02	C0

FUJINET → H89 · ACK · 4 BYTES

C0	77	06	0A	00	B6	00	50 P	49 I	4E N	47 G	C0

GET REMOTE 72H

DEVICE 71H--78H COMMAND 72H (162Q) NET_GET_REMOTE

FIELDS	none
PAYLOAD	none
REPLY	256 bytes: who sent the last datagram, as text

Where the last datagram came from, as `address:port`, then 9BH and a zero; the rest of the 256 bytes is left over. Before any datagram it reads `255.255.255.255:0`.

WARNING: GET_REMOTE writes its text over the channel's buffer: a datagram not yet read loses its first bytes. Read first, then ask.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_remote            ; who sent it?
        call    fn_call
        call    premote
...
; show GET_REMOTE's answer: "a.b.c.d:port", ended by 9BH
premote:
        ld      hl,remote
prem1:
        ld      a,(hl)
        cp      0x9B
        ret     z
        call    conout
        inc     hl
        jr      prem1
...
rq_remote:                              ; always 256 bytes back
        FNREQ   DEV_NETWORK+6, NET_GET_REMOTE, F_NONE, 0,0,0,0
        FNBUF   0, 0, remote, 256
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static char remote[256];        /* the answer is always 256 bytes */
...
/* the buffer must hold all 256 bytes, or the call fails */
rc = network_udp_get_remote(udp, remote, sizeof remote);
printf("%s", remote);           /* who sent it */
```

H89 → FUJINET

C0	77	72	06	00	EF	00	C0

FUJINET → H89 · ACK · 256 BYTES

C0	77	06	06	01	98	00	31 1	32 2	37 7	2E .	30 0	2E .	30 0	2E .	31 1

3A :	37 7	38 8	30 0	35 5	9B	00	30 0	9B	00	00	236 MORE	C0

SET DESTINATION 44H

DEVICE 71H--78H COMMAND 44H (104Q) NET_SET_DESTINATION

FIELDS	none
PAYLOAD	`N:address:port`, ending in a zero
REPLY	ACK

Where the next WRITE sends its datagram. Write it exactly `N:address:port`, with a zero after: on the card, `N7:...` gets the address wrong, and on fujinet-pc a payload without the zero is read past its end.

NOTE: Any STATUS after a datagram arrives sets the destination to that datagram's sender, so a server can simply answer whoever wrote.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_dest              ; where writes go
        call    fn_call
...
rq_dest:                                ; "N:host:port", with a NUL
        FNREQ   DEV_NETWORK+6, NET_SET_DESTINATION, F_NONE, 0,0,0,0
        FNBUF   p_dest, P_DEST_LEN, 0, 0
p_dest:
        defm    "N:127.0.0.1:7805", 0
P_DEST_LEN      equ     $ - p_dest
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
/* not network_udp_set_destination(): it sends no NUL, which
   fujinet-pc needs, and wants "N7:", which the card misreads */
ok = fuji_bus_call(FUJI_DEVICEID_NETWORK + 6,
                   NETCMD_SET_DESTINATION, FUJI_FIELD_NONE,
                   0, 0, 0, 0, "N:127.0.0.1:7805", 17);  /* +NUL */
```

H89 → FUJINET

C0	77	44	17	00	21	00	4E N	3A :	31 1	32 2	37 7	2E .	30 0	2E .	30 0

2E .	31 1	3A :	37 7	38 8	30 0	35 5	00	C0

FUJINET → H89 · ACK

C0	77	06	06	00	83	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_wr7               ; one datagram
        call    fn_call
...
rq_wr7:
        FNREQ   DEV_NETWORK+6, NET_WRITE, F_B12, P_UPONG_LEN,0,0,0
        FNBUF   p_upong, P_UPONG_LEN, 0, 0
p_upong:
        defm    "PONG"
P_UPONG_LEN     equ     $ - p_upong
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_write(udp, "PONG", 4);     /* one datagram */
```

H89 → FUJINET

C0	77	57	0C	00	19	05	04	00	50 P	4F O	4E N	47 G	C0

FUJINET → H89 · ACK

C0	77	06	06	00	83	00	C0

FILES AND DIRECTORIES

On a file system channel (TNFS, SD, SMB, NFS, FTP, SFTP, and HTTP for some), OPEN reads, writes or lists, and six commands work on files directly. The examples use the microSD card (unit 78H).

DIRECTORY LISTING 4FH

DEVICE 71H--78H COMMAND 4FH (117Q)

OPEN with mode 6 lists a directory, as text lines ending CR LF. The translation field chooses the form: 00H short lines, `NAME EXT size date`; 80H long lines, the whole name, with `/` after directories; 83H the bare names.

NOTE: In the short form the date is always 1-01-24 12:00p, a placeholder. On fujinet-pc every size showed 0, as here.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opdir             ; list a directory
        call    fn_call
...
rq_opdir:                               ; mode 6: a directory; 0:
        FNREQ   DEV_NETWORK+7, NET_OPEN, F_A1_A2, 6,0,0,0
        FNBUF   p_dir, P_DIR_LEN, 0, 0  ; the short form
p_dir:
        defm    "N8:SD:/test/", 0
P_DIR_LEN       equ     $ - p_dir
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static const char dir[] = "N8:SD:/test/";
...
rc = network_open(dir, 6, 0);   /* 6: a directory, short form */
```

H89 → FUJINET

C0	78	4F	15	00	97	02	06	00	4E N	38 8	3A :	53 S	44 D	3A :	2F /
74 t	65 e	73 s	74 t	2F /	00	C0									

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_st8               ; the listing's length
        call    fn_call
        ld      hl,rq_rd8               ; read it: a line a file
        call    getwait
        call    typeout
...
rq_st8:
        FNREQ   DEV_NETWORK+7, NET_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, nstat, 4
rq_rd8:
        FNREQ   DEV_NETWORK+7, NET_READ, F_B12, 0,0,0,0
        FNBUF   0, 0, buf, BUFSZ
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
network_status(dir, &avail, &conn, &nerr);
len = network_read(dir, buf, avail);    /* a line a file */
if (len > 0)
    type(buf, len);
```

H89 → FUJINET

C0	78	53	06	00	D1	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	78	06	0A	00	0C	00	81	00	01	01	C0

H89 → FUJINET

C0	78	52	08	00	59	05	81	00	C0

FUJINET → H89 · ACK · 129 BYTES

C0	78	06	87	00	31	00	42 B	4C L	41 A	4E N	4B K	20	20	20	20
48 H	38 8	44 D	20	20	20	20	20	20	20	20	109 MORE	C0			

WRITING A FILE 57H

DEVICE 71H--78H COMMAND 57H (127Q)

OPEN with mode 8, WRITE, CLOSE:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_opwr              ; make a new file
        call    fn_call
...
rq_opwr:                                ; mode 8: write (a new file)
        FNREQ   DEV_NETWORK+7, NET_OPEN, F_A1_A2, 8,0,0,0
        FNBUF   p_wr, P_WR_LEN, 0, 0
p_wr:
        defm    "N8:SD:/test/h89.txt", 0
P_WR_LEN        equ     $ - p_wr
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_open("N8:SD:/test/h89.txt", OPEN_MODE_WRITE,
                  OPEN_TRANS_NONE);     /* a new file */
```

H89 → FUJINET

C0	78	4F	1C	00	0A	02	08	00	4E N	38 8	3A :	53 S	44 D	3A :	2F /
74 t	65 e	73 s	74 t	2F /	68 h	38 8	39 9	2E .	74 t	78 x	74 t	00	C0		

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_wfile             ; a line of text into it
        call    fn_call
...
rq_wfile:
        FNREQ   DEV_NETWORK+7, NET_WRITE, F_B12, P_TEXT_LEN,0,0,0
        FNBUF   p_text, P_TEXT_LEN, 0, 0
p_text:
        defm    "Written by an H89.", 13, 10
P_TEXT_LEN      equ     $ - p_text
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_write("N8:", "Written by an H89.\r\n", 20);
```

H89 → FUJINET

C0	78	57	1C	00	FF	05	14	00	57 W	72 r	69 i	74 t	74 t	65 e	6E n
20	62 b	79 y	20	61 a	6E n	20	48 H	38 8	39 9	2E .	0D	0A	C0		

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_close8            ; finish the file
        call    fn_call
...
rq_close8:
        FNREQ   DEV_NETWORK+7, NET_CLOSE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_close("N8:");      /* finish the file */
```

H89 → FUJINET

C0	78	43	06	00	C1	00	C0

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

SEEK 25H

DEVICE 71H--78H COMMAND 25H (045Q) NET_SEEK

FIELDS	4 bytes: the position, from the start
PAYLOAD	none
REPLY	ACK

Moves to a byte in the open file. TNFS, SD, SMB, NFS, SFTP and HTTP (GET only) can seek; others answer NAK. The example opens `disk.img` and seeks to 1536, the start of sector 3, which is all 03H.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_seek              ; go to sector 3 (byte 1536)
        call    fn_call
        ld      hl,rq_rd8x              ; and read 8 bytes of it
        call    fn_call
        ld      hl,buf                  ; 03 03 03 ...
        call    phexs
...
rq_seek:                                ; a long: 1536 = 00000600H
        FNREQ   DEV_NETWORK+7, NET_SEEK, F_C1234, 0,6,0,0
        FNBUF   0, 0, 0, 0
rq_rd8x:
        FNREQ   DEV_NETWORK+7, NET_READ, F_B12, 8,0,0,0
        FNBUF   0, 0, buf, BUFSZ
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
network_seek(img, 1536);        /* sector 3 */
len = network_read(img, buf, 8);
for (n = 0; n < len; n++)
    printf("%02X ", buf[n]);    /* 03 03 03 ... */
```

H89 → FUJINET

C0	78	25	0A	00	B4	07	00	06	00	00	C0

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

H89 → FUJINET

C0	78	52	08	00	DF	05	08	00	C0

FUJINET → H89 · ACK · 8 BYTES

C0	78	06	0E	00	A4	00	03	03	03	03	03	03	03	03	C0

TELL 26H

DEVICE 71H--78H COMMAND 26H (046Q) NET_TELL

FIELDS	none
PAYLOAD	none
REPLY	4 bytes: the position, low first

Where the next READ will come from. FFFFFFFFH if the channel cannot seek.

WARNING: With no channel open, the four bytes are left over from the FujiNet's memory. On TNFS and SD, TELL throws away bytes STATUS had already read ahead.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_tell              ; where are we now?
        call    fn_call
        ld      hl,(pos)                ; 1544 (the low word of four)
        call    pdec16
...
rq_tell:                                ; a long back
        FNREQ   DEV_NETWORK+7, NET_TELL, F_NONE, 0,0,0,0
        FNBUF   0, 0, pos, 4
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_tell(img, &pos);
printf("%lu", pos);             /* 1544 */
```

H89 → FUJINET

C0	78	26	06	00	A4	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	78	06	0A	00	96	00	08	06	00	00	C0

RENAME 20H

DEVICE 71H--78H COMMAND 20H (040Q) NET_RENAME

FIELDS	2 bytes: 0, 0 (send them)
PAYLOAD	devicespec,newname
REPLY	ACK

Renames a file: the payload is its devicespec, a comma, and the new name (in the same directory).

WARNING: The six file commands — RENAME, DELETE, MKDIR, RMDIR, LOCK and UNLOCK — each use the unit as a scratch channel: whatever was open on it is thrown away, **without** being closed (a file being written loses its end), and the unit is left closed. Use a spare unit for them. And if one fails, STATUS afterwards says only 207 (not connected); the reason is lost.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_rename            ; H89.TXT is now H89NEW.TXT
        call    fn_call
...
rq_rename:                              ; old name, comma, new name
        FNREQ   DEV_NETWORK+7, NET_RENAME, F_A1_A2, 0,0,0,0
        FNBUF   p_ren, P_REN_LEN, 0, 0
p_ren:
        defm    "N8:SD:/test/h89.txt,h89new.txt", 0
P_REN_LEN       equ     $ - p_ren
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_fs_rename("N8:SD:/test/h89.txt,h89new.txt");
```

H89 → FUJINET

C0	78	20	27	00	BE	02	00	00	4E N	38 8	3A :	53 S	44 D	3A :	2F /
74 t	65 e	73 s	74 t	2F /	68 h	38 8	39 9	2E .	74 t	78 x	74 t	2C ,	68 h	38 8	39 9
6E n	65 e	77 w	2E .	74 t	78 x	74 t	00	C0							

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

DELETE 21H

DEVICE 71H--78H COMMAND 21H (041Q) NET_DELETE

FIELDS	2 bytes: 0, 0
PAYLOAD	the devicespec
REPLY	ACK

Deletes a file.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_delete            ; and now it is gone
        call    fn_call
...
rq_delete:
        FNREQ   DEV_NETWORK+7, NET_DELETE, F_A1_A2, 0,0,0,0
        FNBUF   p_del3, P_DEL3_LEN, 0, 0
p_del3:
        defm    "N8:SD:/test/h89new.txt", 0
P_DEL3_LEN      equ     $ - p_del3
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_fs_delete("N8:SD:/test/h89new.txt");
```

H89 → FUJINET

C0	78	21	1F	00	22	02	00	00	4E N	38 8	3A :	53 S	44 D	3A :	2F /
74 t	65 e	73 s	74 t	2F /	68 h	38 8	39 9	6E n	65 e	77 w	2E .	74 t	78 x	74 t	00
C0															

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

MAKE DIRECTORY 2AH

DEVICE 71H--78H COMMAND 2AH (052Q) NET_MKDIR

FIELDS 2 bytes: 0, 0
PAYLOAD the devicespec
REPLY ACK

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_mkdir             ; a new directory
        call    fn_call
...
rq_mkdir:
        FNREQ   DEV_NETWORK+7, NET_MKDIR, F_A1_A2, 0,0,0,0
        FNBUF   p_newdir, P_NEWDIR_LEN, 0, 0
p_newdir:
        defm    "N8:SD:/test/newdir", 0
P_NEWDIR_LEN    equ     $ - p_newdir
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_fs_mkdir("N8:SD:/test/newdir");
```

H89 → FUJINET

C0	78	2A	1B	00	FD	02	00	00	4E N	38 8	3A :	53 S	44 D	3A :	2F /
74 t	65 e	73 s	74 t	2F /	6E n	65 e	77 w	64 d	69 i	72 r	00	C0			

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

REMOVE DIRECTORY 2BH

DEVICE 71H--78H COMMAND 2BH (053Q) NET_RMDIR

FIELDS 2 bytes: 0, 0
PAYLOAD the devicespec
REPLY ACK

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_rmdir             ; removed again (it is empty)
        call    fn_call
...
rq_rmdir:
        FNREQ   DEV_NETWORK+7, NET_RMDIR, F_A1_A2, 0,0,0,0
        FNBUF   p_newdir, P_NEWDIR_LEN, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_fs_rmdir("N8:SD:/test/newdir");    /* it is empty */
```

H89 → FUJINET

C0	78	2B	1B	00	FE	02	00	00	4E N	38 8	3A :	53 S	44 D	3A :	2F /
74 t	65 e	73 s	74 t	2F /	6E n	65 e	77 w	64 d	69 i	72 r	00	C0			

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

LOCK AND UNLOCK 23H

DEVICE 71H--78H COMMAND 23H (043Q) NET_LOCK

FIELDS 2 bytes: 0, 0
PAYLOAD the devicespec
REPLY ACK, or NAK

Make a file read-only, and writable again (UNLOCK is 24H, NET_UNLOCK). TNFS can; the SD card cannot, and answers NAK, as here; FTP and NFS answer ACK and do nothing.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_lock              ; make a file read-only: the
        call    fn_call                 ;   SD card cannot (NAK)
...
rq_lock:
        FNREQ   DEV_NETWORK+7, NET_LOCK, F_A1_A2, 0,0,0,0
        FNBUF   p_readme, P_README_LEN, 0, 0
p_readme:
        defm    "N8:SD:/test/readme.txt", 0
P_README_LEN    equ     $ - p_readme
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_fs_lock("N8:SD:/test/readme.txt"); /* SD: cannot */
```

H89 → FUJINET

C0	78	23	1F	00	6F	02	00	00	4E N	38 8	3A :	53 S	44 D	3A :	2F /
74 t	65 e	73 s	74 t	2F /	72 r	65 e	61 a	64 d	6D m	65 e	2E .	74 t	78 x	74 t	00
C0															

FUJINET → H89 · NAK

C0	78	15	06	00	93	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_unlock            ; nor this (NAK)
        call    fn_call
...
rq_unlock:
        FNREQ   DEV_NETWORK+7, NET_UNLOCK, F_A1_A2, 0,0,0,0
        FNBUF   p_readme, P_README_LEN, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_fs_unlock("N8:SD:/test/readme.txt");
```

H89 → FUJINET

C0	78	24	1F	00	70	02	00	00	4E N	38 8	3A :	53 S	44 D	3A :	2F /
74 t	65 e	73 s	74 t	2F /	72 r	65 e	61 a	64 d	6D m	65 e	2E .	74 t	78 x	74 t	00
C0															

FUJINET → H89 · NAK

C0	78	15	06	00	93	00	C0

CHANGE DIRECTORY 2CH

DEVICE 71H--78H COMMAND 2CH (054Q) NET_CHDIR

FIELDS none
PAYLOAD a directory
REPLY ACK

Sets the unit's **prefix**: a later OPEN or file command on the unit whose devicespec has no scheme of its own is taken relative to it. A path with a scheme replaces the prefix; one beginning / replaces the path; anything else is added to it; an empty one clears it. Nothing is checked.

WARNING: Unlike OPEN, CHDIR keeps a CR or LF at the end of its payload, and puts it in the prefix, where it

spoils every later name. End the payload with a zero. A CHDIR with no payload at all clears the prefix.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_chdir             ; names without a scheme now
        call    fn_call                 ;   start here
...
rq_chdir:
        FNREQ   DEV_NETWORK+7, NET_CHDIR, F_NONE, 0,0,0,0
        FNBUF   p_test, P_TEST_LEN, 0, 0
p_test:
        defm    "N8:SD:/test/", 0
P_TEST_LEN      equ     $ - p_test
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_fs_cd("N8:SD:/test/");     /* names start here now */
```

H89 → FUJINET

C0	78	2C	13	00	6A	00	4E N	38 8	3A :	53 S	44 D	3A :	2F /	74 t	65 e
73 s	74 t	2F /	00	C0											

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

GET CURRENT DIRECTORY 30H

DEVICE 71H--78H COMMAND 30H (060Q) NET_GETCWD

FIELDS	none
PAYLOAD	none
REPLY	the prefix, with no zero

The unit's prefix, without the `N:` and without a zero at the end: use the reply's length. fujinet-lib's `network_fs_pwd()` does not add the zero either.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_getcwd            ; where is that?
        call    fn_call
        ld      hl,cwd                  ; BC bytes, no NUL
        call    ptext
...
rq_getcwd:
        FNREQ   DEV_NETWORK+7, NET_GETCWD, F_NONE, 0,0,0,0
        FNBUF   0, 0, cwd, 256
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
static char cwd[256];
...
memset(cwd, 0, sizeof cwd);     /* the answer has no NUL */
rc = network_fs_pwd("N8:", cwd);
printf("%s", cwd);
```

H89 → FUJINET

C0	78	30	06	00	AE	00	C0

FUJINET → H89 · ACK · 9 BYTES

C0	78	06	0F	00	7F	00	53 S	44 D	3A :	2F /	74 t	65 e	73 s	74 t	2F /
C0															

OPENING RELATIVE TO THE PREFIX 4FH

DEVICE 71H--78H COMMAND 4FH (117Q)

With the prefix `SD:/test/`, `N8:readme.txt` opens `SD:/test/readme.txt`:

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_oprel             ; just a name: the directory
        call    fn_call                 ;   set by CHDIR goes before it
        ld      hl,rq_st8
        call    fn_call
        ld      hl,rq_rd8
        call    getwait
        call    typeout
...
rq_oprel:
        FNREQ   DEV_NETWORK+7, NET_OPEN, F_A1_A2, 4,0,0,0
        FNBUF   p_rel, P_REL_LEN, 0, 0
p_rel:
        defm    "N8:readme.txt", 0
P_REL_LEN       equ     $ - p_rel
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
network_open("N8:readme.txt", OPEN_MODE_READ, OPEN_TRANS_NONE);
network_status("N8:", &avail, &conn, &nerr);
len = network_read("N8:", buf, avail);
if (len > 0)
    type(buf, len);
```

H89 → FUJINET

C0	78	4F	16	00	A4	02	04	00	4E N	38 8	3A :	72 r	65 e	61 a	64 d
6D m	65 e	2E .	74 t	78 x	74 t	00	C0								

FUJINET → H89 · ACK

C0	78	06	06	00	84	00	C0

H89 → FUJINET

C0	78	53	06	00	D1	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	78	06	0A	00	C5	00	3B ;	00	01	01	C0

H89 → FUJINET

C0	78	52	08	00	13	05	3B	00	C0

FUJINET → H89 · ACK · 59 BYTES

C0	78	06	41	00	6B	00	48 H	65 e	6C l	6C l	6F o	20	66 f	72 r	6F o
6D m	20	74 t	68 h	65 e	20	46 F	75 u	6A j	69 i	4E N	39 MORE				C0

CHANNEL SETTINGS

USER NAME — FDH

DEVICE 71H--78H COMMAND FDH (375Q) NET_USERNAME

FIELDS	none
PAYLOAD	the user name
REPLY	ACK

A user name for the unit's next OPEN, for servers that ask (FTP, SMB, SSH, IMAPS and others), as another way than putting it in the devicespec.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_user              ; for servers that ask
        call    fn_call
...
rq_user:                                ; just the name
        FNREQ   DEV_NETWORK, NET_USERNAME, F_NONE, 0,0,0,0
        FNBUF   p_user, P_USER_LEN, 0, 0
p_user:
        defm    "h89user"
P_USER_LEN      equ     $ - p_user
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_username("N1:", "h89user");
```

H89 → FUJINET

C0	71	FD	0D	00	17	00	68 h	38 8	39 9	75 u	73 s	65 e	72 r	C0

FUJINET → H89 · ACK

C0	71	06	06	00	7D	00	C0

PASSWORD — FEH

DEVICE 71H--78H COMMAND FEH (376Q) NET_PASSWORD

FIELDS	none
PAYLOAD	the password
REPLY	ACK

The password to go with it.

NOTE: The password is passed on only if a user name has been set.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_pass
        call    fn_call
...
rq_pass:
        FNREQ   DEV_NETWORK, NET_PASSWORD, F_NONE, 0,0,0,0
        FNBUF   p_pass, P_PASS_LEN, 0, 0
p_pass:
        defm    "swordfish"
P_PASS_LEN      equ     $ - p_pass
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_password("N1:", "swordfish");
```

H89 → FUJINET

C0	71	FE	0F	00	5C	00	73 s	77 w	6F o	72 r	64 d	66 f	69 i	73 s	68 h

C0

FUJINET → H89 · ACK

C0	71	06	06	00	7D	00	C0

SET LINE ENDING — 4CH

DEVICE 71H--78H COMMAND 4CH (114Q) NET_SET_EOL

FIELDS	none
PAYLOAD	the H89's line ending, 1 or 2 bytes
REPLY	ACK

What translation (OPEN's second field) turns the far end's line endings into, and back from: normally CR LF. Every byte of the payload counts — don't send a zero after it. An empty payload puts back CR LF.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_seteol            ; our lines end with CR alone
        call    fn_call
...
rq_seteol:                              ; the payload is the line end
        FNREQ   DEV_NETWORK, NET_SET_EOL, F_NONE, 0,0,0,0
        FNBUF   p_cr, 1, 0, 0
p_cr:
        defb    13
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_eol("N1:", "\r");      /* our lines end CR */
```

H89 → FUJINET

C0	71	4C	07	00	D1	00	0D	C0

FUJINET → H89 · ACK

C0	71	06	06	00	7D	00	C0

TRANSLATION — 54H

DEVICE 71H--78H COMMAND 54H (124Q) NET_TRANSLATION

FIELDS	2 bytes: 0 (not used); the translation, 0 to 3, or FFH
PAYLOAD	none
REPLY	ACK

Sets a translation for the unit's **later** OPENs (not the open channel), added to theirs: with 1 here and 2 in an OPEN, the OPEN gets 3. FFH turns translation off for later OPENs, whatever they ask.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_xlat              ; the network's end in CR LF:
        call    fn_call                 ;   later OPENs translate
...
rq_xlat:                                ; field 2: 3 = CR LF
        FNREQ   DEV_NETWORK, NET_TRANSLATION, F_A1_A2, 0,3,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_translation("N1:", OPEN_TRANS_CRLF);
```

H89 → FUJINET

C0	71	54	08	00	D2	02	00	03	C0

FUJINET → H89 · ACK

C0	71	06	06	00	7D	00	C0

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_open              ; the same GET as before
        call    fn_call
        ld      hl,rq_status
        call    fn_call
        ld      hl,rq_read              ; 53 bytes now, not 55: each
        call    getwait                 ;   CR LF came as CR
        call    ptext
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
network_open(hello, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
network_status(hello, &avail, &conn, &nerr);
len = network_read(hello, buf, avail);  /* 53, not 55: each */
show(buf, len);                         /*   CR LF came as CR */
```

H89 → FUJINET

C0	71	4F	2B	00	1D	02	04	00	4E N	31 1	3A :	48 H	54 T	54 T	50 P
3A :	2F /	2F /	31 1	32 2	37 7	2E .	30 0	2E .	30 0	2E .	31 1	3A :	37 7	38 8	30 0
32 2	2F /	68 h	65 e	6C l	6C l	6F o	2E .	74 t	78 x	74 t	00	C0			

FUJINET → H89 · ACK

C0	71	06	06	00	7D	00	C0

H89 → FUJINET

C0	71	53	06	00	CA	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	71	06	0A	00	B7	00	35 5	00	00	01	C0

H89 → FUJINET

C0	71	52	08	00	06	05	35	00	C0

FUJINET → H89 · ACK · 53 BYTES

C0	71	06	3B	00	54	00	48 H	65 e	6C l	6C l	6F o	20	66 f	72 r	6F o
6D m	20	74 t	68 h	65 e	20	74 t	65 e	73 s	74 t	20	33 MORE				C0

INTERRUPT RATE 5AH

DEVICE 71H--78H COMMAND 5AH (132Q) NET_SET_INT_RATE

FIELDS	1 byte: milliseconds
PAYLOAD	none
REPLY	ACK

How often the FujiNet signals “data waiting”. On the H89 the signal goes nowhere, so this does nothing.

WARNING: Never send 0: the firmware divides by it, and stops.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_irate             ; 100 ms (never 0: that
        call    fn_call                 ;   crashes the firmware)
...
rq_irate:
        FNREQ   DEV_NETWORK, NET_SET_INT_RATE, F_A1, 100,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
rc = network_set_interrupt_rate("N1:", 100);    /* never 0 */
```

H89 → FUJINET

C0	71	5A	07	00	38	01	64	C0

FUJINET → H89 · ACK

C0	71	06	06	00	7D	00	C0

COMMANDS THE NETWORK DEVICE DOES NOT HAVE

These answer NAK on the H89: NET_GET_ERROR (45H, shown below), NET_GET_DSTATS_VALUE (FFH), NET_SET_UNIT (FAH), NET_SET_HSIO_INDEX (E3H), NET_QUERY_ALT (81H), NET_PARSE_ALT (80H) and NET_HSIO_INDEX (3FH). For a channel's last error, ask STATUS.

Z80 ASSEMBLY — FROM EXNET.ASM

```
        ld      hl,rq_geterr            ; a command the network units
        call    fn_call                 ;   do not have: NAK
...
rq_geterr:
        FNREQ   DEV_NETWORK, NET_GET_ERROR, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXNET.C

```
/* fujinet-lib has no call for it: the units do not know it */
ok = fuji_bus_call(FUJI_DEVICEID_NETWORK, NETCMD_GET_ERROR,
                   FUJI_FIELD_NONE, 0, 0, 0, 0, NULL, 0);
```

H89 → FUJINET

C0	71	45	06	00	BC	00	C0

FUJINET → H89 · NAK

C0	71	15	06	00	8C	00	C0

NETWORK PROTOCOLS

GENERAL

A network channel (Chapter 10) opens whatever its **devicespec** names. The devicespec is a text string in the payload of the OPEN request, and its **scheme** — the word before the first colon after the `N:` — chooses one of the FujiNet's 28 protocols. Behind each is a piece of the firmware that knows how to talk to one kind of server, or to make something from nothing (an SSH key, a calendar). To the H89 they all look the same: OPEN, STATUS, READ, WRITE, CLOSE.

This chapter lists them, with the form of devicespec each takes, a word on what it does, and anything special on the H89. The FujiNet project's **Network Protocol Handbook** gives each a full chapter.

THE DEVICESPEC

```
N[n]:SCHEME://[user:password@]host[:port]/path[?query]
```

- **N:** or **Nn:** names the channel. Under the FujiBus the channel is really chosen by the device number the request goes to (71H to 78H), so `N:` will do; fujinet-lib reads the digit to pick the device, and `N3:` sends the request to 73H.
- The **scheme** is matched in capitals, so `n:http://` works as well as `N:HTTP://`.
- Host names and paths are passed on as they are. Paths on a server whose names have case need the case right.
- The devicespec may be up to 256 bytes long.

The firmware changes a few characters on the way in, for the Atari's sake: an asterisk becomes the end of the string (so `*` works as "everything" in directory listings), and a full stop at the very end is dropped.

WARNING: An OPEN of a scheme the FujiNet does not know is refused with NAK, and STATUS then reports error 144. (The **Network Protocol Handbook** gives 165, which is what other buses report.) An OPEN always closes whatever the channel had open first, even when it then fails.

THE FAMILIES

The protocols fall into four families, and the family tells you most of what a protocol can do:

- **File systems** know files from directories. They can list a directory (OPEN mode 6), read and write files (modes 4 and 8), and most can rename, delete, make and remove directories (Chapter 10's file system commands) and seek.
- **Streams** are a connection: bytes in, bytes out, until one end hangs up. Mode and translation hardly matter to them.
- **Utilities** do one job when opened, and READ tells you how it went.
- **Mail and calendars** turn a mailbox or a calendar into numbered items and text you can read line by line.

SCHEME	FAMILY	REACHES
TNFS	file	TNFS servers, the FujiNet's own file protocol
SD	file	the FujiNet's microSD card
SMB	file	Windows and Samba shares
NFS	file	UNIX NFS exports
FTP	file	FTP servers
SFTP	file	files over SSH
HTTP, HTTPS	file	web servers, web services, WebDAV
S3	file	Amazon S3 and compatible stores
GDRIVE	file	Google Drive

ONEDRIVE	file	Microsoft OneDrive
TCP	stream	any TCP service; or listen for one
UDP	stream	UDP datagrams
TELNET	stream	Telnet: bulletin boards, servers
SSH	stream	a shell on another computer
WS, WSS	stream	WebSocket services
SSH.KEYGEN	utility	makes the FujiNet's SSH key
SSH.COPYID	utility	installs that key on a server
CLIPBOARD	utility	the FujiNet's clipboard
CPM	utility	a CP/M computer inside the FujiNet
TEST	utility	a fixed test message
GMAIL	mail	a Gmail mailbox
IMAPS	mail	any IMAP mailbox
GCAL	calendar	Google Calendar
ICAL, WEBCAL, ICALH	calendar	published calendar feeds

FILE SYSTEMS

TNFS

```
N:TNFS://host[:port]/path
```

The Tiny Network File System, made for 8-bit computers: small requests, small replies, and servers that run on anything. The FujiNet's public disk image servers are TNFS servers, and the tnfsd server runs on Linux, Windows and macOS. Reading, writing, appending, directories, every file system command and SEEK all work. `N:TNFS://tnfs.fujinet.online/` lists the public server.

SD

```
N:SD:/path
```

The microSD card on the FujiNet card, with the same commands as TNFS except LOCK and UNLOCK. Note the single slash. On fujinet-pc it is the directory given with `-s`. The book's examples use it, because it needs no network at all: `N:SD:/test/` lists the examples' test directory.

SMB

```
N:SMB://[user:password@]server/share/path
```

Windows file sharing (SMB 2 and 3), and Samba on other systems.

NFS

```
N:NFS://host[:port]/export/path
```

UNIX file servers. LOCK and UNLOCK do nothing.

FTP

```
N:FTP://host[:port]/path
```

Files from FTP servers, anonymously unless a user name and password are set (Chapter 10). Reading files and directories works; the file system commands answer ACK and do nothing.

SFTP

```
N:SFTP://user[:password]@host[:port]/path
```

Files over SSH, from any computer that runs an SSH server, with every file system command. The password can be left out if the FujiNet's key is installed on the server (SSH.COPYID, below).

HTTP, HTTPS

```
N:HTTPS://[user:password@]host[:port]/path[?query]
```

The World Wide Web, and the web services that answer in JSON or XML. OPEN's mode chooses GET (4), PUT (8), POST (13), DELETE (5) and their variants with headers; Chapter 10 has them all, with the JSON parser, which most programs pair with HTTP. HTTPS uses TLS, done entirely by the FujiNet, so an H89 can talk to modern secure servers. WebDAV servers can be listed (mode 6) and have files renamed (MOVE) and directories made (MKCOL).

S3

```
N:S3://[key:secret@]endpoint[:port]/bucket/key?region=,,&tls=0|1
```

Amazon S3 and the many stores that speak its language, such as MinIO.

GDRIVE, ONEDRIVE

```
N:GDRIVE:///path      N:ONEDRIVE:///path
```

Google Drive and Microsoft OneDrive. Both need the FujiNet signed in to the account first, from its web page; after that a path is all an H89 program needs. Files written are sent when the channel is closed.

STREAMS

TCP
`N:TCP://host:port/` `N:TCP://:port/` (listen)
A plain TCP connection to any port: the H89 sends bytes, the far end's bytes wait in the FujiNet until READ takes them. With no host, the FujiNet **listens** on the port instead, and ACCEPT takes each computer that connects (Chapter 10). NETCAT (Chapter 13) is a TCP and Telnet client.

UDP
`N:UDP://host:port/` `N:UDP://:port/`
Datagrams, one at a time, to and from a port. A port is required. SET_DESTINATION and GET_REMOTE (Chapter 10) say where datagrams go and where the last one came from.

TELNET
`N:TELNET://host[:port]/?term=ansi&cols=80&rows=24`
TCP with the Telnet protocol's negotiation handled by the FujiNet, so bulletin boards and Telnet servers see a proper terminal. `term`, `cols` and `rows` say what terminal to claim. The H19 in ANSI mode is a good match for `term=ansi` (NETCAT switches it); in its own Heath mode it is close to a VT52.

SSH
`N:SSH://user[:password]@host[:port]/?term=vt100&cols=80&rows=24`
A shell on another computer over SSH, with all the encryption done by the FujiNet.

WS, WSS
`N:WSS://host[:port]/path?frame=text`
WebSocket connections, plain or over TLS: a message-based stream that many web services use for live data. `frame=text` sends text frames rather than binary ones.

UTILITIES

SSH.KEYGEN
`N:SSH.KEYGEN://ed25519/?overwrite=1`
Makes an SSH key pair for the FujiNet and keeps it on the microSD card. READ gives a line saying what happened.

SSH.COPYID
`N:SSH.COPYID://user:password@host[:port]/`
Logs in to a server once with a password and adds the FujiNet's key to the user's `authorized_keys`, so SSH and SFTP need no password after.

CLIPBOARD
`N:CLIPBOARD:///[0-9]`
The FujiNet's clipboard, shared by every computer that uses the FujiNet and by its web page: read the current text (0) or one of the nine before it, write to replace it (when the channel closes), or append with mode 9. A handy way to move text from a modern computer to the H89 and back.

CPM
`N:CPM://`
A whole CP/M 2.2 computer, emulated inside the FujiNet: READ takes its console output, and WRITE types at its keyboard. There is one, shared by every channel. (It is not the H89's CP/M, and has nothing to do with device 5AH, which the H89 does not have.)

TEST
`N:TEST://anything/`
READ gives a fixed sentence and WRITE is thrown away: a way to check a program's network code with no server at all.

MAIL AND CALENDARS

GMAIL
`N:GMAIL:///Folder[/n[/a]]`
A Gmail mailbox, signed in from the FujiNet's web page. Reading a folder gives how many messages it holds; a directory listing (mode 6) gives one line per message; `/Inbox/3` reads message 3, and a third number an attachment. In mode 8, `N:GMAIL:///` sends a new message and `N:GMAIL:///Inbox/3` a reply.

IMAPS
`N:IMAPS://user:password@host[:port]/FOLDER[/n[/a]]`
The same, read-only, for any mailbox that offers IMAP over TLS.

GCAL
`N:GCAL:///[calendar]/VIEW[/DATE[/n]]`
Google Calendar as text: VIEW is DAY, WEEK or MONTH. On this bus lines are formatted 80 columns wide, just right for the H19. Mode 8 adds an event, or edits one.

ICAL, WEBCAL, ICALH
`N:ICAL://feed-host/feed-path/VIEW[/DATE[/n]]`
Published calendar feeds (iCalendar files), read-only, with the same views. ICAL and WEBCAL fetch over HTTPS, ICALH over plain HTTP.

WHICH WERE TRIED

The examples in this Manual open SD, HTTP, TCP and UDP channels, against servers on the same computer as fujinet-pc, and NETCAT opens TCP and Telnet ones. The others are as the firmware has them; the **Network Protocol Handbook** describes each in full. All 28 are in the H89's firmware and in fujinet-pc alike.

CLOCK, PRINTER AND MODEM

GENERAL

Three more devices complete the FujiNet: a clock (45H), which knows the time from the Internet; a printer (40H), which turns what an H89 program prints into a file; and a modem (50H), which makes the FujiNet look like a Hayes modem to programs written for one.

The clock is the useful one. The H89 has no clock of its own that survives being switched off — HDOS asks for the date every time it starts — and the FujiNet's is set from the network. The printer and the modem are there because the firmware is shared with other computers; on the H89 both have serious faults, given below.

THE CLOCK

The FujiNet keeps the time in Universal Time (UTC), from the Internet, and gives it in seven forms, each as some other computer liked it. It also keeps two **time zones**: the **system** zone, saved in its settings and set from its web page or by APETIME_SETTZ_ALT, and an **alternate** zone, kept only until the FujiNet restarts, set by APETIME_SETTZ. Most of the commands take one field: 1 asks for the time in the alternate zone, anything else (or nothing) in the system zone.

Time zones are written the POSIX way: `CST6CDT,M3.2.0,M11.1.0` is Central time, six hours behind UTC, with summer time from the second Sunday of March to the first Sunday of November. `UTC0` is UTC.

The examples ran with the system zone UTC, and then set the alternate zone to Central time and the system zone to Eastern; the times in the bytes are when they ran.

GET TIME 93H

DEVICE 45H (105Q) COMMAND 93H (223Q) APETIME_GETTIME

FIELDS	1 byte: 1 for the alternate zone
PAYLOAD	none
REPLY	6 bytes: day, month, year since 2000, hour, minute, second

The time in six binary bytes, the simplest form for a Z80 program.

WARNING: fujinet-lib's `clock_get_time()` puts a zero after the reply, so its buffer must be one byte longer than the form's reply: 7 bytes here.

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_time              ; day, month, year - 2000, hour,
        call    fn_call                 ;   minute, second
        jr      nz,gt_x
        ld      hl,tbuf
        ld      b,6
gt_1:
        call    pspace                  ; each byte in decimal
        ld      a,(hl)
        call    pdec8
        inc     hl
        djnz    gt_1
gt_x:
...
rq_time:
        FNREQ   DEV_CLOCK, APETIME_GETTIME, F_A1, 0,0,0,0
        FNBUF   0, 0, tbuf, 6
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static uint8_t when[7];         /* 6 bytes + the NUL fujinet-lib adds */
...
ok = clock_get_time(when, APETIME_BINARY) == FN_ERR_OK;
if (ok)                         /* day, month, year - 2000, h, m, s */
    printf("\n%u/%u/%u %u:%02u:%02u", when[0], when[1],
           when[2] + 2000, when[3], when[4], when[5]);
```

H89 → FUJINET

C0	45	93	07	00	E0	01	00	C0

FUJINET → H89 · ACK · 6 BYTES

C0	45	06	0C	00	CD	00	0D	0A	1A	04	23 #	1E	C0

GET TIME, SIMPLE 54H

DEVICE 45H (105Q) COMMAND 54H (124Q) APETIME_SETTZ_ALT2

FIELDS	1 byte: 1 for the alternate zone
PAYLOAD	none
REPLY	7 bytes: century, year, month, day, hour, minute, second

The time with the century: 20, 26, 10, 13 and so on. The firmware's name for 54H says it sets a time zone, but on the H89 it is this form of the time.

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_simple            ; 54H gets the time (the name
        call    fn_call                 ;   lies): century, year, month,
        jr      nz,gs_x                 ;   day, hour, minute, second
        ld      hl,tbuf
        ld      b,7
gs_1:
        call    pspace
        ld      a,(hl)
        call    pdec8
        inc     hl
        djnz    gs_1
gs_x:
...
rq_simple:
        FNREQ   DEV_CLOCK, APETIME_SETTZ_ALT2, F_A1, 0,0,0,0
        FNBUF   0, 0, tbuf, 7
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static uint8_t simple[8];       /* 7 bytes + the NUL fujinet-lib adds */
...
/* SIMPLE_BINARY is command 54H, which gets the time, whatever
   its name says: century, year, month, day, hour, minute, second */
ok = clock_get_time(simple, SIMPLE_BINARY) == FN_ERR_OK;
if (ok)
    printf("\n%u%02u-%02u-%02u %02u:%02u:%02u", simple[0],
           simple[1], simple[2], simple[3], simple[4], simple[5],
           simple[6]);
```

H89 → FUJINET

C0	45	54	07	00	A1	01	00	C0

FUJINET → H89 · ACK · 7 BYTES

C0	45	06	0D	00	E2	00	14	1A	0A	0D	04	23 #	1E	C0

GET TIME, WITH HUNDREDTHS 4DH

DEVICE 45H (105Q) COMMAND 4DH (115Q) APETIME_GET_SIMPLE_HUNDREDTHS

FIELDS	1 byte
PAYLOAD	none
REPLY	8 bytes: as 54H, then hundredths

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_hund              ; as 54H, and hundredths of a
        call    fn_call                 ;   second in byte 7
        jr      nz,gh_x
        call    pspace
        ld      a,(tbuf+7)
        call    pdec8
gh_x:
...
rq_hund:
        FNREQ   DEV_CLOCK, APETIME_GET_SIMPLE_HUNDREDTHS, F_A1, 0,0,0,0
        FNBUF   0, 0, tbuf, 8
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static uint8_t hund[9];         /* 8 + 1 */
...
ok = clock_get_time(hund, SIMPLE_BINARY_WITH_HUNDREDTHS)
     == FN_ERR_OK;
if (ok)                         /* as SIMPLE_BINARY, and byte 7 */
    printf("\n%02u:%02u.%02u", hund[5], hund[6], hund[7]);
```

H89 → FUJINET

C0	45	4D	07	00	9A	01	00	C0

FUJINET → H89 · ACK · 8 BYTES

C0	45	06	0E	00	F1	00	14	1A	0A	0D	04	23 #	1F	0D	C0

GET TIME, PRODOS 50H

DEVICE 45H (105Q) COMMAND 50H (120Q) APETIME_GET_PRODOS

FIELDS	1 byte
PAYLOAD	none
REPLY	4 bytes: the date as a word, minute, hour

The Apple's ProDOS form: a word holding the year since 1900 (7 bits), month (4) and day (5), low byte first, then the minute and hour.

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_prodos            ; ProDOS's date word (yyyyyyym
        call    fn_call                 ;   mmmddddd), minute, hour
...
rq_prodos:
        FNREQ   DEV_CLOCK, APETIME_GET_PRODOS, F_A1, 0,0,0,0
        FNBUF   0, 0, tbuf, 4
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static uint8_t prodos[5];       /* 4 + 1 */
...
ok = clock_get_time(prodos, PRODOS_BINARY) == FN_ERR_OK;
if (ok)                         /* yyyyyyym mmmddddd, minute, hour */
    printf("\n%u/%u/%u %u:%02u", prodos[0] & 0x1F,
           ((prodos[1] & 1) << 3) | (prodos[0] >> 5),
           prodos[1] >> 1, prodos[3], prodos[2]);
```

H89 → FUJINET

C0	45	50	07	00	9D	01	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	45	06	0A	00	FE	00	4D M	35 5	23 #	04	C0

GET TIME, SOS 53H

DEVICE 45H (105Q) COMMAND 53H (123Q) APETIME_GET_SOS

FIELDS	1 byte
PAYLOAD	none
REPLY	19 bytes: `YYYYMMDD0HHMMSS000` and a zero

The Apple III's form, as digits. (The tenth digit is a weekday that is always 0.)

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_sos               ; Apple III SOS: 18 digits, and
        call    fn_call                 ;   a NUL to end them
        jr      nz,sos_x
        call    crlf
        ld      hl,tbuf
        call    print
sos_x:
...
rq_sos:
        FNREQ   DEV_CLOCK, APETIME_GET_SOS, F_A1, 0,0,0,0
        FNBUF   0, 0, tbuf, 19
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static char sos[20];            /* 19, its own NUL counted, + 1 */
...
ok = clock_get_time((uint8_t *)sos, APPLE3_SOS_BINARY) == FN_ERR_OK;
if (ok)                         /* Apple III SOS: 18 digits */
    printf("\n%s", sos);
```

H89 → FUJINET

C0	45	53	07	00	A0	01	00	C0

FUJINET → H89 · ACK · 19 BYTES

C0	45	06	19	00	E6	00	32 2	30 0	32 2	36 6	31 1	30 0	31 1	33 3	30 0

30 0	34 4	33 3	35 5	33 3	31 1	30 0	30 0	30 0	00	C0

GET TIME, ISO 49H

DEVICE 45H (105Q) COMMAND 49H (111Q) APETIME_GET_ISO_LOCAL

FIELDS	1 byte
PAYLOAD	none
REPLY	25 bytes: YYYY-MM-DDTHH:MM:SS+HHMM and a zero

The international standard form, with the zone's distance from UTC at the end: the easiest to print.

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_isol              ; ISO 8601, with the zone's
        call    fn_call                 ;   offset; NUL at the end
        jr      nz,iso_x
        call    crlf
        ld      hl,tbuf
        call    print
iso_x:
...
rq_isol:
        FNREQ   DEV_CLOCK, APETIME_GET_ISO_LOCAL, F_A1, 0,0,0,0
        FNBUF   0, 0, tbuf, 25
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static char iso[26];            /* 25, its own NUL counted, + 1 */
...
ok = clock_get_time((uint8_t *)iso, TZ_ISO_STRING) == FN_ERR_OK;
if (ok)                         /* ISO 8601, with the offset */
    printf("\n%s", iso);
```

H89 → FUJINET

C0	45	49	07	00	96	01	00	C0

FUJINET → H89 · ACK · 25 BYTES

C0	45	06	1F	00	3B	00	32	30	32	36	2D	31	30	2D	31
							2	0	2	6	-	1	0	-	1

33	54	30	34	3A	33	35	3A	33	31	2B	30	30	30	30	00
3	T	0	4	:	3	5	:	3	1	+	0	0	0	0	

C0

GET TIME, ISO, UTC 5AH

DEVICE 45H (105Q) COMMAND 5AH (132Q) APETIME_GET_ISO_UTC

FIELDS	1 byte (ignored)
PAYLOAD	none
REPLY	25 bytes: as 49H, always +0000

The same in UTC, whatever the zones say.

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_isou              ; ISO 8601 in UTC, always
        call    fn_call
        jr      nz,utc_x
        call    crlf
        ld      hl,tbuf
        call    print
utc_x:
...
rq_isou:
        FNREQ   DEV_CLOCK, APETIME_GET_ISO_UTC, F_A1, 0,0,0,0
        FNBUF   0, 0, tbuf, 25
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
ok = clock_get_time((uint8_t *)iso, UTC_ISO_STRING) == FN_ERR_OK;
if (ok)                         /* ISO 8601 in UTC, always */
    printf("\n%s", iso);
```

H89 → FUJINET

C0	45	5A	07	00	A7	01	00	C0

FUJINET → H89 · ACK · 25 BYTES

C0	45	06	1F	00	3B	00	32	30	32	36	2D	31	30	2D	31
							2	0	2	6	-	1	0	-	1

33	54	30	34	3A	33	35	3A	33	31	2B	30	30	30	30	00
3	T	0	4	:	3	5	:	3	1	+	0	0	0	0	

C0

GET TIME ZONE LENGTH 4CH

DEVICE 45H (105Q) COMMAND 4CH (114Q) APETIME_GETTZ_LEN

FIELDS	none
PAYLOAD	none
REPLY	1 byte: the system zone's length + 1

How long the system zone's name is, counting the zero after it: ask this first, then APETIME_GET_GENERAL for that many bytes. fujinet-lib has no function for it alone.

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_tzlen             ; how long the zone's name is,
        call    fn_call                 ;   its NUL counted
        jr      nz,len_x
        call    crlf
        ld      a,(tbuf)
        call    pdec8
len_x:
...
rq_tzlen:
        FNREQ   DEV_CLOCK, APETIME_GETTZ_LEN, F_NONE, 0,0,0,0
        FNBUF   0, 0, tbuf, 1
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
/* fujinet-lib asks this only inside clock_get_tz(): ask it here */
ok = fuji_bus_call(FUJI_DEVICEID_CLOCK, APETIMECMD_GETTZ_LEN,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0, &len, 1);
if (ok)                         /* its NUL counted */
    printf("\n%u", len);
```

H89 → FUJINET

C0	45	4C	06	00	97	00	C0

FUJINET → H89 · ACK · 1 BYTE

C0	45	06	07	00	56	00	04	C0

GET TIME ZONE 47H

DEVICE 45H (105Q) COMMAND 47H (107Q) APETIME_GET_GENERAL

FIELDS	none
PAYLOAD	none
REPLY	the system zone, and a zero

The system time zone, as text. fujinet-lib's `clock_get_tz()` asks the length first, then this.

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_tz                ; the system zone, with a NUL
        call    fn_call
        jr      nz,tz_x
        call    crlf
        ld      hl,tbuf
        call    print
tz_x:
...
rq_tz:
        FNREQ   DEV_CLOCK, APETIME_GET_GENERAL, F_NONE, 0,0,0,0
        FNBUF   0, 0, tbuf, 64
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static char tz[64];            /* clock_get_tz() trusts it is enough */
...
ok = clock_get_tz(tz) == FN_ERR_OK;     /* asks the length first */
if (ok)
    printf("\n%s", tz);
```

H89 → FUJINET

C0	45	47	06	00	92	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	45	06	0A	00	42	00	55 U	54 T	43 C	00	C0

SET ALTERNATE TIME ZONE 99H

DEVICE 45H (105Q) COMMAND 99H (231Q) APETIME_SETTZ

FIELDS	any (ignored)
PAYLOAD	the zone, as text
REPLY	ACK

Sets the alternate zone, until the FujiNet restarts. The time commands then give it with their field 1.

WARNING: The FujiNet answers ACK before it looks at the zone. One with a character it does not like — a CR, a zero at the start — is quietly ignored, and the zone stays as it was; one that is merely wrong is kept, and the times come out in UTC. Check with APETIME_GET_ISO_LOCAL.

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_settz             ; set the alternate zone (in RAM
        call    fn_call                 ;   only): US Central time
...
rq_settz:                               ; the zone, a POSIX TZ string
        FNREQ   DEV_CLOCK, APETIME_SETTZ, F_NONE, 0,0,0,0
        FNBUF   p_cst, p_cste-p_cst, 0, 0
p_cst:  defm    "CST6CDT,M3.2.0,M11.1.0"
p_cste:
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static const char central[] = "CST6CDT,M3.2.0,M11.1.0";
...
/* sets the alternate zone (in RAM only), then asks the time
   there: GETTIME with field 1 = 1 */
ok = clock_get_time_tz(when, central, APETIME_BINARY) == FN_ERR_OK;
if (ok)
    printf("\n%u:%02u in Chicago", when[3], when[4]);
```

H89 → FUJINET

C0	45	99	1C	00	FC	00	43 C	53 S	54 T	36 6	43 C	44 D	54 T	2C ,	4D M

33 3	2E .	32 2	2E .	30 0	2C ,	4D M	31 1	31 1	2E .	31 1	2E .	30 0	C0

FUJINET → H89 · ACK

C0	45	06	06	00	51	00	C0

GET TIME IN THE ALTERNATE ZONE 9AH

DEVICE 45H (105Q) COMMAND 9AH (232Q) APETIME_GETTZTIME

FIELDS	any (ignored)
PAYLOAD	none
REPLY	6 bytes, as 93H

APETIME_GETTIME in the alternate zone, whatever the field says. With no alternate zone set, the system zone. fujinet-lib has no function for it.

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_tztime            ; the time in the other zone,
        call    fn_call                 ;   laid out as GETTIME's
        jr      nz,tzt_x
        call    crlf
        ld      a,(tbuf+3)              ; the hour, there
        call    pdec8
tzt_x:
...
rq_tztime:
        FNREQ   DEV_CLOCK, APETIME_GETTZTIME, F_NONE, 0,0,0,0
        FNBUF   0, 0, tbuf, 6
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
/* no fujinet-lib call: the time in the alternate zone */
ok = fuji_bus_call(FUJI_DEVICEID_CLOCK, APETIMECMD_GETTZTIME,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0, when, 6);
if (ok)
    printf("\n%u:%02u there", when[3], when[4]);
```

H89 → FUJINET

C0	45	9A	06	00	E5	00	C0

FUJINET → H89 · ACK · 6 BYTES

C0	45	06	0C	00	E1	00	0C	0A	1A	17	23 #	20	C0

THE FORMS IN THE ALTERNATE ZONE 49H

DEVICE 45H (105Q) COMMAND 49H (111Q)

The same commands with the field 1. Central time is five hours behind UTC in October:

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_isola             ; the offset is -0500 now
        call    fn_call
        jr      nz,isoa_x
        call    crlf
        ld      hl,tbuf
        call    print
isoa_x:
...
rq_isola:
        FNREQ   DEV_CLOCK, APETIME_GET_ISO_LOCAL, F_A1, 1,0,0,0
        FNBUF   0, 0, tbuf, 25
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
ok = clock_get_time_common((uint8_t *)iso, TZ_ISO_STRING, true)
     == FN_ERR_OK;
if (ok)                         /* the offset is -0500 now */
    printf("\n%s", iso);
```

H89 → FUJINET

C0	45	49	07	00	97	01	01	C0

FUJINET → H89 · ACK · 25 BYTES

C0	45	06	1F	00	44	00	32 2	30 0	32 2	36 6	2D -	31 1	30 0	2D -	31 1

32 2	54 T	32 2	33 3	3A :	33 3	35 5	3A :	33 3	33 3	2D -	30 0	35 5	30 0	30 0	00

C0

The ISO UTC form ignores the field:

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_isoua             ; UTC takes the field and pays
        call    fn_call                 ;   it no mind
        jr      nz,utca_x
        call    crlf
        ld      hl,tbuf
        call    print
utca_x:
...
rq_isoua:
        FNREQ   DEV_CLOCK, APETIME_GET_ISO_UTC, F_A1, 1,0,0,0
        FNBUF   0, 0, tbuf, 25
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
ok = clock_get_time_common((uint8_t *)iso, UTC_ISO_STRING, true)
     == FN_ERR_OK;
if (ok)                         /* UTC pays the field no mind */
    printf("\n%s", iso);
```

H89 → FUJINET

C0	45	5A	07	00	A8	01	01	C0

FUJINET → H89 · ACK · 25 BYTES

C0	45	06	1F	00	3E	00	32	30	32	36	2D	31	30	2D	31
							2	0	2	6	-	1	0	-	1

33	54	30	34	3A	33	35	3A	33	34	2B	30	30	30	30	00
3	T	0	4	:	3	5	:	3	4	+	0	0	0	0	

C0

SET TIME ZONE 74H

DEVICE 45H (105Q) COMMAND 74H (164Q) APETIME_SETTZ_ALT

FIELDS	any (ignored)
PAYLOAD	the zone, as text
REPLY	ACK

Sets the **system** zone, and saves it in the FujiNet's settings, so it lasts. (The names of 99H and 74H are the wrong way round for what they do.) The example sets Eastern time, then reads it back.

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_setsys            ; set the system zone: US
        call    fn_call                 ;   Eastern; it is saved
...
rq_setsys:
        FNREQ   DEV_CLOCK, APETIME_SETTZ_ALT, F_NONE, 0,0,0,0
        FNBUF   p_est, p_este-p_est, 0, 0
p_est:  defm    "EST5EDT,M3.2.0,M11.1.0"
p_este:
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
static const char eastern[] = "EST5EDT,M3.2.0,M11.1.0";
...
ok = clock_set_tz(eastern) == FN_ERR_OK;    /* and it is saved */
```

H89 → FUJINET

C0	45	74	1C	00	DA	00	45	53	54	35	45	44	54	2C	4D
							E	S	T	5	E	D	T	,	M

33	2E	32	2E	30	2C	4D	31	31	2E	31	2E	30	C0
3	.	2	.	0	,	M	1	1	.	1	.	0	

FUJINET → H89 · ACK

C0	45	06	06	00	51	00	C0

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_tz                ; read it back
        call    fn_call
        jr      nz,tzn_x
        call    crlf
        ld      hl,tbuf
        call    print
tzn_x:
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
ok = clock_get_tz(tz) == FN_ERR_OK;     /* read it back */
if (ok)
    printf("\n%s", tz);
```

H89 → FUJINET

C0	45	47	06	00	92	00	C0

FUJINET → H89 · ACK · 23 BYTES

C0	45	06	1D	00	6D	00	45	53	54	35	45	44	54	2C	4D
							E	S	T	5	E	D	T	,	M

33	2E	32	2E	30	2C	4D	31	31	2E	31	2E	30	00	C0
3	.	2	.	0	,	M	1	1	.	1	.	0		

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_isol              ; field 0: the system zone,
        call    fn_call                 ;   -0400 now
        jr      nz,ison_x
        call    crlf
        ld      hl,tbuf
        call    print
ison_x:
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
ok = clock_get_time((uint8_t *)iso, TZ_ISO_STRING) == FN_ERR_OK;
if (ok)                         /* the system zone: -0400 */
    printf("\n%s", iso);
```

H89 → FUJINET

C0	45	49	07	00	96	01	00	C0

FUJINET → H89 · ACK · 25 BYTES

C0	45	06	1F	00	40	00	32	30	32	36	2D	31	30	2D	31
							2	0	2	6	-	1	0	-	1

33	54	30	30	3A	33	35	3A	33	34	2D	30	34	30	30	00
3	T	0	0	:	3	5	:	3	4	-	0	4	0	0	

C0

A BAD TIME ZONE 74H

DEVICE 45H (105Q) COMMAND 74H (164Q)

A zone with a CR in it is answered ACK, and ignored: the zone is still Eastern.

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_setbad            ; a zone with a RETURN on the
        call    fn_call                 ;   end: ACK, and thrown away
...
rq_setbad:                              ; only 20H-7EH will do
        FNREQ   DEV_CLOCK, APETIME_SETTZ_ALT, F_NONE, 0,0,0,0
        FNBUF   p_bad, 8, 0, 0
p_bad:  defm    "PST8PDT"
        defb    13
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
/* a RETURN on the end: ACK, and thrown away (20H-7EH only) */
ok = clock_set_tz("PST8PDT\r") == FN_ERR_OK;
```

H89 → FUJINET

C0	45	74	0E	00	ED	00	50	53	54	38	50	44	54	0D	C0
							P	S	T	8	P	D	T		

FUJINET → H89 · ACK

C0	45	06	06	00	51	00	C0

Z80 ASSEMBLY — FROM EXCLOCK.ASM

```
        ld      hl,rq_tz                ; still US Eastern
        call    fn_call
        jr      nz,tzk_x
        call    crlf
        ld      hl,tbuf
        call    print
tzk_x:
```

C (Z88DK, FUJINET-LIB) — FROM EXCLOCK.C

```
ok = clock_get_tz(tz) == FN_ERR_OK;     /* still US Eastern */
if (ok)
    printf("\n%s", tz);
```

H89 → FUJINET

C0	45	47	06	00	92	00	C0

FUJINET → H89 · ACK · 23 BYTES

C0	45	06	1D	00	6D	00	45	53	54	35	45	44	54	2C	4D
							E	S	T	5	E	D	T	,	M

33	2E	32	2E	30	2C	4D	31	31	2E	31	2E	30	00	C0
3	.	2	.	0	,	M	1	1	.	1	.	0		

THE PRINTER

The FujiNet's printer is not a machine: it turns what it is sent into a file called `paper`, on the microSD card (or the FujiNet's own flash), which you fetch from the FujiNet's web page. The FujiNet's settings choose a printer to imitate; the examples ran with type 1, which writes the bytes as they come. (fujinet-pc keeps `paper` in its SD directory; the examples' output was checked there.)

WARNING: If the printer is turned off in the FujiNet's settings, it does not answer at all: no ACK, no NAK, nothing, and a program waits out its timeout.

PRINT 57H

DEVICE 40H (100Q) COMMAND 57H (127Q) PRINTER_WRITE

OUTPUT CHECKED ON HOST

FIELDS	none needed
PAYLOAD	the bytes to print, 1 to 255
REPLY	ACK

Prints the payload. NAK if there is none. The example's line arrived in `paper` exactly as sent, CR LF and all.

WARNING: Keep each WRITE to 255 bytes or fewer. The firmware takes the length in eight bits: 256 bytes print nothing (and still ACK), 300 print only the first 44 — as the example below shows. And more than 320 bytes overruns the printer's buffer and corrupts the firmware's memory: never send that much.

Z80 ASSEMBLY — FROM EXPRINT.ASM

```
        ld      hl,rq_pline             ; one line, RETURN and LINE FEED
        call    fn_call                 ;   and all
...
rq_pline:
        FNREQ   DEV_PRINTER, PRINTER_WRITE, F_NONE, 0,0,0,0
        FNBUF   p_line, p_linee-p_line, 0, 0
p_line: defm    "Printed on the H89 through FujiNet."
        defb    13, 10
p_linee:
```

C (Z88DK, FUJINET-LIB) — FROM EXPRINT.C

```
static const char line[] = "Printed on the H89 through FujiNet.\r\n";
...
/* fujinet-lib has no printer calls: fuji_bus_call sends it */
ok = fuji_bus_call(FUJI_DEVICEID_PRINTER, PRINTERCMD_WRITE,
                   FUJI_FIELD_DATA, 0, 0, 0, 0,
                   line, sizeof line - 1);     /* not the NUL */
```

H89 → FUJINET

C0	40	57	2B	00	17	00	50 P	72 r	69 i	6E n	74 t	65 e	64 d	20	6F o
6E n	20	74 t	68 h	65 e	20	48 H	38 8	39 9	20	74 t	68 h	72 r	6F o	75 u	67 g
68 h	20	46 F	75 u	6A j	69 i	4E N	65 e	74 t	2E .	0D	0A	C0			

FUJINET → H89 · ACK

C0	40	06	06	00	4C	00	C0

PRINT (PUT) 50H

DEVICE 40H (100Q) COMMAND 50H (120Q) PRINTER_PUT

OUTPUT CHECKED ON HOST

FIELDS	none needed
PAYLOAD	the bytes to print
REPLY	ACK

The same as PRINTER_WRITE.

Z80 ASSEMBLY — FROM EXPRINT.ASM

```
        ld      hl,rq_pput              ; PUT is WRITE by another name
        call    fn_call
...
rq_pput:
        FNREQ   DEV_PRINTER, PRINTER_PUT, F_NONE, 0,0,0,0
        FNBUF   p_put, p_pute-p_put, 0, 0
p_put:  defm    "And this line went by PUT."
        defb    13, 10
p_pute:
```

C (Z88DK, FUJINET-LIB) — FROM EXPRINT.C

```
static const char line2[] = "And this line went by PUT.\r\n";
...
ok = fuji_bus_call(FUJI_DEVICEID_PRINTER, PRINTERCMD_PUT,
                   FUJI_FIELD_DATA, 0, 0, 0, 0,
                   line2, sizeof line2 - 1);   /* WRITE, renamed */
```

H89 → FUJINET

C0	40	50	22	00	A5	00	41 A	6E n	64 d	20	74 t	68 h	69 i	73 s	20
6C l	69 i	6E n	65 e	20	77 w	65 e	6E n	74 t	20	62 b	79 y	20	50 P	55 U	54 T
2E .	0D	0A	C0												

FUJINET → H89 · ACK

C0	40	06	06	00	4C	00	C0

PRINTING TOO MUCH 57H

DEVICE 40H (100Q) COMMAND 57H (127Q)

Three hundred bytes, answered ACK: only the first 44 reached the paper. And 256 bytes print nothing:

Z80 ASSEMBLY — FROM EXPRINT.ASM

```
        ld      hl,rq_p300              ; 300 bytes: ACK, and the first
        call    fn_call                 ;   44 printed (300 - 256)
...
rq_p300:
        FNREQ   DEV_PRINTER, PRINTER_WRITE, F_NONE, 0,0,0,0
        FNBUF   big, 300, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXPRINT.C

```
/* 300 bytes: ACK, and the first 44 printed (300 - 256) */
ok = fuji_bus_call(FUJI_DEVICEID_PRINTER, PRINTERCMD_WRITE,
                   FUJI_FIELD_DATA, 0, 0, 0, 0, big, 300);
```

H89 → FUJINET

C0	40	57	32	01	FA	00	4F O	66 f	20	33 3	30 0	30 0	20	62 b	79 y
74 t	65 e	73 s	20	73 s	65 e	6E n	74 t	2C ,	20	6A j	280 MORE				C0

FUJINET → H89 · ACK

C0	40	06	06	00	4C	00	C0

Z80 ASSEMBLY — FROM EXPRINT.ASM

```
        ld      hl,rq_p256              ; 256 bytes: ACK, and nothing
        call    fn_call                 ;   printed (256 is 0 in a byte)
...
rq_p256:
        FNREQ   DEV_PRINTER, PRINTER_WRITE, F_NONE, 0,0,0,0
        FNBUF   big, 256, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXPRINT.C

```
/* 256 bytes: ACK, and nothing printed (256 is 0 in a byte) */
ok = fuji_bus_call(FUJI_DEVICEID_PRINTER, PRINTERCMD_WRITE,
                   FUJI_FIELD_DATA, 0, 0, 0, 0, big, 256);
```

H89 → FUJINET

C0	40	57	06	01	01	00	54 T	68 h	69 i	73 s	20	32 2	35 5	36 6	2D -

62 b	79 y	74 t	65 e	20	6C l	69 i	6E n	65 e	20	6E n	236 MORE	C0

FUJINET → H89 · ACK

C0	40	06	06	00	4C	00	C0

PRINTER STATUS 53H

DEVICE 40H (100Q) COMMAND 53H (123Q) PRINTER_STATUS

FIELDS	1 byte (any value, but send it)
PAYLOAD	none
REPLY	4 bytes

Four bytes in the Atari's form; the third is a timeout (5). The second is never set by the firmware, and is whatever was in memory (00H on fujinet-pc). NAK if the field is missing — the one command in this chapter that says so rather than stopping.

Z80 ASSEMBLY — FROM EXPRINT.ASM

```
        ld      hl,rq_pstat             ; 4 bytes: 0, a byte nobody set,
        call    fn_call                 ;   the timeout (5), 0
        jr      nz,ps_x
        call    crlf
        ld      hl,pstat
        ld      bc,4
        call    phexs
ps_x:
...
rq_pstat:                               ; one field: any value, but
        FNREQ   DEV_PRINTER, PRINTER_STATUS, F_A1, 0,0,0,0
        FNBUF   0, 0, pstat, 4          ;   it must be there
```

C (Z88DK, FUJINET-LIB) — FROM EXPRINT.C

```
/* one field, any value, but it must be there */
ok = fuji_bus_call(FUJI_DEVICEID_PRINTER, PRINTERCMD_STATUS,
                   FUJI_FIELD_A1 | FUJI_FIELD_REPLY, 0, 0, 0, 0,
                   pstat, 4);
if (ok)             /* 0, a byte nobody set, the timeout (5), 0 */
    printf("\n%02X %02X %02X %02X",
           pstat[0], pstat[1], pstat[2], pstat[3]);
```

H89 → FUJINET

C0	40	53	07	00	9B	01	00	C0

FUJINET → H89 · ACK · 4 BYTES

C0	40	06	0A	00	55	00	00	00	05	00	C0

Z80 ASSEMBLY — FROM EXPRINT.ASM

```
        ld      hl,rq_pstat0            ; STATUS with no field: NAK
        call    fn_call
...
rq_pstat0:
        FNREQ   DEV_PRINTER, PRINTER_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, pstat, 4
```

C (Z88DK, FUJINET-LIB) — FROM EXPRINT.C

```
ok = fuji_bus_call(FUJI_DEVICEID_PRINTER, PRINTERCMD_STATUS,
                   FUJI_FIELD_REPLY, 0, 0, 0, 0,
                   pstat, 4);                /* no field: NAK */
```

H89 → FUJINET

C0	40	53	06	00	99	00	C0

FUJINET → H89 · NAK

C0	40	15	06	00	5B	00	C0

THE MODEM

The FujiNet's modem lets a program written for a Hayes modem — a terminal program, a bulletin board — reach the Internet: `ATDT host:port` dials a TCP connection, and the bytes then pass both ways, as over a telephone line. The FujiNet can also answer calls, listening on a port of its own.

On the H89, the modem is the strangest device of the six:

- Its replies are not FujiBus frames. Commands go as frames, but what comes back is bare bytes, with no header, checksum or END — and an ACK or NAK sends nothing at all. A program sends a command and then simply reads what bytes come, until they stop. The book's library has `fn_send` (a request that waits for no reply) and `fn_raw` (take bytes until 200 ms of quiet) for this; fujinet-lib cannot do it, so the C examples use the book's `rawbus.c`. Every card in this section is tagged UNFRAMED.
- Bytes the H89 sends **outside** any frame go to the modem too, as if typed at it. Another way to send AT commands; and the reason a request that loses its opening END is taken as modem input.
- The modem never sends its result text. `AT` gets no `OK`; `ATDT` gets no `CONNECT` until well after the far end's first data has arrived. Only RING, CONNECT and NO CARRIER come back, and data.
- `+++`, to get back to command mode during a call, does not work: the bytes go to the far end.

WARNING: Several things stop the firmware: a modem command with a field missing (SET_DUMP, LISTEN, BAUDRATELOCK, AUTOANSWER and STATUS all need theirs); LISTEN on port 0; `ATCPM`; and an `ATDT` whose port is not a number. None of these is sent in the examples.

WRITE 57H

DEVICE 50H (120Q) COMMAND 57H (127Q) MODEM_WRITE

UNFRAMED

FIELDS	none
PAYLOAD	bytes for the modem
REPLY	bare bytes, or none

Sends bytes to the modem: AT commands when it is idle, data during a call. Here `ATDT` dials the test board: no `OK`, and the board's greeting arrives at once, with `CONNECT 2400` a second behind it.

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_dial              ; dial the test board, and type
        call    fn_send                 ;   what comes: its greeting,
        call    mtype                   ;   and CONNECT a second later
...
; mtype: type the modem's bytes as they come, until 2 s pass with
; none. BC = how many came in all.
mtype:
        ld      hl,0
        ld      (mcount),hl
mtype1:
        ld      de,mbuf
        ld      bc,MBUFSZ
        call    fn_raw                  ; until 200 ms pass with none
        ld      a,b
        or      c
        jr      z,mtype2                ; 2 s and nothing: that is all
        ld      hl,(mcount)
        add     hl,bc
        ld      (mcount),hl
        ld      hl,mbuf                 ; type them, the unprintable
        call    ptext                   ;   ones as dots
        jr      mtype1
mtype2:
        ld      bc,(mcount)
        ret
mcount: defw    0
...
rq_dial:                                ; one AT command a WRITE
        FNREQ   DEV_SERIAL, MODEM_WRITE, F_NONE, 0,0,0,0
        FNBUF   p_dial, p_diale-p_dial, 0, 0
p_dial: defm    "ATDT127.0.0.1:7801"
        defb    13
p_diale:
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* type the modem's bytes as they come, until 2 s pass with none;
   returns how many came in all */
static uint16_t type_raw(void)
{
    uint16_t n, total = 0;

    while ((n = raw_get(mbuf, sizeof mbuf - 1)) != 0) {
        mbuf[n] = 0;
        printf("%s", (char *)mbuf);
        total += n;
    }
    return total;
}

static const char dial[] = "ATDT127.0.0.1:7801\r";
...
/* dial the test board, one AT command a WRITE; then type what
   comes: its greeting, and CONNECT a second later */
raw_frame(MODEM, MODEMCMD_WRITE, FUJI_FIELD_NONE, 0, 0, 0, 0,
          dial, sizeof dial - 1);
n = type_raw();
```

H89 → FUJINET

C0	50	57	19	00	BD	00	41 A	54 T	44 D	54 T	31 1	32 2	37 7	2E .	30 0
2E .	30 0	2E .	31 1	3A :	37 7	38 8	30 0	31 1	0D	C0					

FUJINET → H89 · NOT FRAMED · 120 BYTES

0D	0A	46 F	75 u	6A j	69 i	4E N	65 e	74 t	20	74 t	65 e	73 s	74 t	20	62 b
6F o	61 a	72 r	64 d	2C ,	20	31 1	32 2	37 7	2E .	30 0	92 MORE				0A

AT, UNANSWERED 57H

DEVICE 50H (120Q) COMMAND 57H (127Q)

`AT` alone, which on a real modem answers `OK`. Here, nothing comes back.

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mat               ; AT: the modem takes it, but
        call    fn_send                 ;   its OK is never sent
...
rq_mat:
        FNREQ   DEV_SERIAL, MODEM_WRITE, F_NONE, 0,0,0,0
        FNBUF   p_at, 3, 0, 0
p_at:   defm    "AT"
        defb    13
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* the modem takes AT, but its OK is never sent */
raw_frame(MODEM, MODEMCMD_WRITE, FUJI_FIELD_NONE, 0, 0, 0, 0,
          "AT\r", 3);
```

H89 → FUJINET

C0	50	57	09	00	53	00	41 A	54 T	0D	C0

NO REPLY

DURING A CALL 57H

DEVICE 50H (120Q) COMMAND 57H (127Q)

During the call, WRITE sends data, and what the far end sends comes back as bare bytes:

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mline             ; online, a WRITE goes to the
        call    fn_send                 ;   far end: the board echoes
        call    mtype                   ;   it, then shouts it back
...
rq_mline:
        FNREQ   DEV_SERIAL, MODEM_WRITE, F_NONE, 0,0,0,0
        FNBUF   p_line, 6, 0, 0
p_line: defm    "hello"
        defb    13
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* online, a WRITE goes to the far end: the board echoes it,
   then shouts it back */
raw_frame(MODEM, MODEMCMD_WRITE, FUJI_FIELD_NONE, 0, 0, 0, 0,
          "hello\r", 6);
n = type_raw();
```

H89 → FUJINET

C0	50	57	0C	00	D6	00	68 h	65 e	6C l	6C l	6F o	0D	C0

FUJINET → H89 · NOT FRAMED · 16 BYTES

68 h	65 e	6C l	6C l	6F o	0D	0A	48 H	45 E	4C L	4C L	4F O	0D	0A	3E >	20

+++ and ATH are not commands during a call, only data: the board echoes them.

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mplus             ; +++ and a second of quiet: no
        call    fn_send                 ;   OK, it went to the board
        call    mtype
...
rq_mplus:
        FNREQ   DEV_SERIAL, MODEM_WRITE, F_NONE, 0,0,0,0
        FNBUF   p_plus, 3, 0, 0
p_plus: defm    "+++"
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* +++ and a second of quiet: no OK; it went to the board */
raw_frame(MODEM, MODEMCMD_WRITE, FUJI_FIELD_NONE, 0, 0, 0, 0,
          "+++", 3);
n = type_raw();
```

H89 → FUJINET

C0	50	57	09	00	32	00	2B +	2B +	2B +	C0

FUJINET → H89 · NOT FRAMED · 3 BYTES

2B +	2B +	2B +

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_math              ; so ATH is data too: the board
        call    fn_send                 ;   gets it; the call stays up
        call    mtype
...
rq_math:
        FNREQ   DEV_SERIAL, MODEM_WRITE, F_NONE, 0,0,0,0
        FNBUF   p_ath, 4, 0, 0
p_ath:  defm    "ATH"
        defb    13
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* so ATH is data too: the board gets it; the call stays up */
raw_frame(MODEM, MODEMCMD_WRITE, FUJI_FIELD_NONE, 0, 0, 0, 0,
          "ATH\r", 4);
n = type_raw();
```

H89 → FUJINET

C0	50	57	0A	00	9C	00	41 A	54 T	48 H	0D	C0

FUJINET → H89 · NOT FRAMED · 15 BYTES

41 A	54 T	48 H	0D	0A	2B +	2B +	2B +	41 A	54 T	48 H	0D	0A	3E >	20

STATUS 53H

DEVICE 50H (120Q) COMMAND 53H (123Q) MODEM_STATUS

UNFRAMED

FIELDS	1 byte (send it)
PAYLOAD	none
REPLY	2 bare bytes

The modem's state in two bytes, the Atari 850's form: 00 00 when idle, 00 CCH connected, 00 CDH with a caller waiting.

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mstat             ; STATUS: two bare bytes come
        call    fn_send                 ;   back, not a frame
        ld      de,mstat
        ld      bc,2
        call    fn_raw                  ; BC = how many came
        call    crlf
        ld      hl,mstat                ; error bits (0), and C0H+0CH
        call    phexs                   ;   when online, +01H with data
...
rq_mstat:                               ; the field must be there
        FNREQ   DEV_SERIAL, MODEM_STATUS, F_A1, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* two bare bytes come back, not a frame; the field must be there */
raw_frame(MODEM, MODEMCMD_STATUS, FUJI_FIELD_A1, 0, 0, 0, 0,
          NULL, 0);
n = raw_get(mstat, 2);
if (n == 2)                 /* 0, and C0H+0CH online, +01H data */
    printf("\n%02X %02X", mstat[0], mstat[1]);
```

H89 → FUJINET

C0	50	53	07	00	AB	01	00	C0

FUJINET → H89 · NOT FRAMED · 2 BYTES

00	00

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mstat             ; online: CCH
        call    fn_send
        ld      de,mstat
        ld      bc,2
        call    fn_raw
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
raw_frame(MODEM, MODEMCMD_STATUS, FUJI_FIELD_A1, 0, 0, 0, 0,
          NULL, 0);
n = raw_get(mstat, 2);                  /* online: 00 CC */
```

H89 → FUJINET

C0	50	53	07	00	AB	01	00	C0

FUJINET → H89 · NOT FRAMED · 2 BYTES

00	CC

LISTEN AND ANSWER 4CH

DEVICE 50H (120Q) COMMAND 4CH (114Q) MODEM_LISTEN

UNFRAMED

FIELDS	a word: the port (not 0)
PAYLOAD	none
REPLY	RING, when a call comes

Listens for calls on a TCP port. When one comes, the modem sends RING (and again every few seconds), and STATUS shows a caller waiting. ATA (in a WRITE) answers: the caller's data arrives, then CONNECT. MODEM_UNLISTEN (4DH) stops listening.

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mlstn             ; take calls on port 7803: the
        call    fn_send                 ;   rig's helper calls, and the
        ld      de,mbuf                 ;   modem says RING every 3 s
        ld      bc,MBUFSZ
        call    fn_raw
        ld      hl,mbuf
        call    ptext
...
rq_mlstn:                               ; port 7803 (1E7BH), a word
        FNREQ   DEV_SERIAL, MODEM_LISTEN, F_B12, 0x7B,0x1E,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* take calls on port 7803 (a word): the rig's helper calls, and
   the modem says RING every 3 s */
raw_frame(MODEM, MODEMCMD_LISTEN, FUJI_FIELD_B12,
          7803 & 0xFF, 7803 >> 8, 0, 0, NULL, 0);
n = raw_get(mbuf, sizeof mbuf - 1);
mbuf[n] = 0;
printf("\n%s", (char *)mbuf);
```

H89 → FUJINET

C0	50	4C	08	00	43	05	7B	1E	C0

FUJINET → H89 · NOT FRAMED · 6 BYTES

52 R	49 I	4E N	47 G	0D	0A

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mstat             ; a caller waits: CDH
        call    fn_send
        ld      de,mstat
        ld      bc,2
        call    fn_raw
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
raw_frame(MODEM, MODEMCMD_STATUS, FUJI_FIELD_A1, 0, 0, 0, 0,
          NULL, 0);
n = raw_get(mstat, 2);                  /* a caller waits: 00 CD */
```

H89 → FUJINET

C0	50	53	07	00	AB	01	00	C0

FUJINET → H89 · NOT FRAMED · 2 BYTES

00	CD

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mata              ; ATA answers; what the caller
        call    fn_send                 ;   sent (PING) comes through,
        call    mtype                   ;   CONNECT a second later
...
rq_mata:                                ; just these 4 bytes: ATA, CR
        FNREQ   DEV_SERIAL, MODEM_WRITE, F_NONE, 0,0,0,0
        FNBUF   p_ata, 4, 0, 0
p_ata:  defm    "ATA"
        defb    13
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* ATA answers (just these 4 bytes); what the caller sent comes
   through, CONNECT a second later */
raw_frame(MODEM, MODEMCMD_WRITE, FUJI_FIELD_NONE, 0, 0, 0, 0,
          "ATA\r", 4);
n = type_raw();
```

H89 → FUJINET

C0	50	57	0A	00	95	00	41 A	54 T	41 A	0D	C0

FUJINET → H89 · NOT FRAMED · 22 BYTES

0D	0A	50 P	49 I	4E N	47 G	0D	0A	43 C	4F O	4E N	4E N	45 E	43 C	54 T	20

32 2	34 4	30 0	30 0	0D	0A

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_munl              ; no more calls on 7803
        call    fn_send
...
rq_munl:
        FNREQ   DEV_SERIAL, MODEM_UNLISTEN, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* no more calls on 7803 */
raw_frame(MODEM, MODEMCMD_UNLISTEN, FUJI_FIELD_NONE, 0, 0, 0, 0,
          NULL, 0);
```

H89 → FUJINET

C0	50	4D	06	00	A3	00	C0

NO REPLY

STREAM 58H

DEVICE 50H (120Q) COMMAND 58H (130Q) MODEM_STREAM

UNFRAMED

FIELDS	none
PAYLOAD	none
REPLY	9 bare bytes

For the Atari's 850 interface, a description of the serial line; meaningless on the H89, and on fujinet-pc on a real serial port it changes the port's speed. Shown for completeness.

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mstrm             ; STREAM: nine bare bytes, the
        call    fn_send                 ;   Atari POKEY's divisors for
        ld      de,mbuf                 ;   the modem's baud rate
        ld      bc,9
        call    fn_raw
...
rq_mstrm:
        FNREQ   DEV_SERIAL, MODEM_STREAM, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* nine bare bytes: an Atari POKEY's divisors for the baud rate */
raw_frame(MODEM, MODEMCMD_STREAM, FUJI_FIELD_NONE, 0, 0, 0, 0,
          NULL, 0);
n = raw_get(mbuf, 9);
```

H89 → FUJINET

C0	50	58	06	00	AE	00	C0

FUJINET → H89 · NOT FRAMED · 9 BYTES

28 (	A0	00	A0	28 (	A0	00	A0	78 x

SETTINGS 42H

DEVICE 50H (120Q) COMMAND 42H (102Q) MODEM_CONFIGURE

UNFRAMED

FIELDS	varies
PAYLOAD	none
REPLY	nothing

Four commands set things up, and answer nothing: MODEM_CONFIGURE (42H, the line speed — always 9600, whatever is asked), MODEM_CONTROL (41H, the handshake lines, which do nothing here), MODEM_BAUDRATELOCK (4EH, two words: lock on, and the speed CONNECT reports) and MODEM_SET_DUMP (44H, a byte: the traffic log on or off). MODEM_AUTOANSWER (4FH, a byte) answers calls without ATA.

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mconf             ; CONFIGURE: 9600 baud, whatever
        call    fn_send                 ;   is asked; no reply at all
...
rq_mconf:
        FNREQ   DEV_SERIAL, MODEM_CONFIGURE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* 9600 baud, whatever is asked; no reply at all */
raw_frame(MODEM, MODEMCMD_CONFIGURE, FUJI_FIELD_NONE, 0, 0, 0, 0,
          NULL, 0);
```

H89 → FUJINET

C0	50	42	06	00	98	00	C0

NO REPLY

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mlock             ; lock the rate at 2400: CONNECT
        call    fn_send                 ;   says 2400 from now on
...
rq_mlock:                               ; lock on, 2400 baud (0960H)
        FNREQ   DEV_SERIAL,MODEM_BAUDRATELOCK,F_B12_B34, 1,0,0x60,0x09
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* two fields: lock on, 2400 baud; CONNECT says 2400 from now on */
raw_frame(MODEM, MODEMCMD_BAUDRATELOCK, FUJI_FIELD_B12_B34,
          1, 0, 2400 & 0xFF, 2400 >> 8, NULL, 0);
```

H89 → FUJINET

C0	50	4E	0A	00	19	06	01	00	60	09	C0

NO REPLY

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,rq_mauto             ; field 0: we answer with ATA
        call    fn_send
...
rq_mauto:                               ; the field must be there
        FNREQ   DEV_SERIAL, MODEM_AUTOANSWER, F_A1, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* 0: we answer with ATA; the field must be there */
raw_frame(MODEM, MODEMCMD_AUTOANSWER, FUJI_FIELD_A1, 0, 0, 0, 0,
          NULL, 0);
```

H89 → FUJINET

C0	50	4F	07	00	A7	01	00	C0

NO REPLY

AT COMMANDS WITHOUT A FRAME

DEVICE 50H (120Q)

The same ATDT sent as bare bytes, outside any frame, works as well; the FujiNet acts on it half a second after the last byte, when it decides the bytes have stopped.

Z80 ASSEMBLY — FROM EXMODEM.ASM

```
        ld      hl,p_dial               ; the same ATDT as bare bytes:
        ld      bc,p_diale-p_dial       ;   whatever is not in a frame
        call    rawput                  ;   goes to the modem as typed
        call    mtype
...
; rawput: BC bytes from HL to the FujiNet, outside any frame
rawput:
        ld      a,OE_ON
        out     (PCTRL),a
        ld      a,DIR_TO_ESP            ; the 245 toward the ESP32
        out     (PCTRL),a
rawput1:
        in      a,(PORTC)               ; wait till the last byte is
        and     PC_OBF                  ;   taken (/OBF high)
        jr      z,rawput1
        ld      a,b
        or      c
        jr      z,rawput2
        ld      a,(hl)
        out     (PORTA),a
        inc     hl
        dec     bc
        jr      rawput1
rawput2:
        ld      a,DIR_TO_H89            ; and back toward us
        out     (PCTRL),a
        ret
```

C (Z88DK, FUJINET-LIB) — FROM EXMODEM.C

```
/* the same ATDT as bare bytes: what is not in a frame goes to
   the modem as typed */
raw_put(dial, sizeof dial - 1);
n = type_raw();
```

H89 → FUJINET · NOT FRAMED · 1 BYTE

41 A

NO REPLY

H89 → FUJINET · NOT FRAMED · 5 BYTES

54 T	44 D	54 T	31 1	32 2

NO REPLY

H89 → FUJINET · NOT FRAMED · 3 BYTES

37 7	2E .	30 0

NO REPLY

H89 → FUJINET · NOT FRAMED · 2 BYTES

2E .	30 0

NO REPLY

H89 → FUJINET · NOT FRAMED · 1 BYTE

2E .

NO REPLY

H89 → FUJINET · NOT FRAMED · 3 BYTES

31 1	3A :	37 7

NO REPLY

H89 → FUJINET · NOT FRAMED · 1 BYTE

38 8

NO REPLY

H89 → FUJINET · NOT FRAMED · 3 BYTES

30	31	0D
0	1	

FUJINET → H89 · NOT FRAMED · 120 BYTES

0D	0A	46	75	6A	69	4E	65	74	20	74	65	73	74	20	62
		F	u	j	i	N	e	t		t	e	s	t		b

6F	61	72	64	2C	20	31	32	37	2E	30	92 MORE	0A
o	a	r	d	,		1	2	7	.	0		

NETCAT

GENERAL

NETCAT turns the H89 into a terminal on the network. Give it the devicespec of a bulletin board, a Telnet server or any TCP service, and it connects, sends what you type, and shows what comes back, until the far end hangs up or you strike CTRL-]. It is the shortest program in this Manual that does something you will use, and the pattern of its main loop — keyboard, STATUS, READ, around again — is the pattern of every program that talks to the network.

There are two versions, NETCAT.ASM and NETCAT.C. They behave the same, and each makes a program called NETCAT for HDOS and for CP/M.

USING NETCAT

( ) Give NETCAT the devicespec on the command line:
NETCAT N:TELNET://BBS.FOZZTEXX.COM/ ↵

( ) Or run it with nothing after its name, and answer its question. Just ↵ connects to BBS.FOZZTEXX.COM, a bulletin board run by one of the FujiNet's developers.

( ) Talk to the far end. Everything you type goes to it; everything it sends is shown.

( ) To leave, log off the far end, or strike CTRL-] (hold CTRL and press the] key).

The devicespec may be any stream the FujiNet knows (Chapter 11): `N:TELNET://host/` for bulletin boards and Telnet servers, `N:TCP://host:port/` for anything else. NETCAT adds the `N:` if you leave it off.

NOTE: HDOS and CP/M both turn a typed command line into capital letters. Host names don't care, but if a devicespec must have small letters in it — a path, a password — run NETCAT with nothing after its name and type the devicespec at its question, which keeps what you type as you type it.

Here NETCAT is talking to the book's test board (`tools/helpers.py`, which you can run on any computer on your network) under HDOS, and then under CP/M:

```
>RUN SY1:NETCAT.ABS N:TCP://192.168.1.10:7801/
NETCAT - the H89 on the network.  CTRL-] hangs up.
Connecting to N:TCP://192.168.1.10:7801/
FujiNet test board, 192.168.1.10 port 7801.
Type a line and it comes back in capitals; BYE hangs up.
> hello there
HELLO THERE
> bye
Goodbye.
Disconnected.

A>B:NETCAT
NETCAT - the H89 on the network.  CTRL-] hangs up.
Connect to (RETURN for BBS.FOZZTEXX.COM): N:TCP://192.168.1.10:7801/
Connecting to N:TCP://192.168.1.10:7801/
FujiNet test board, 192.168.1.10 port 7801.
Type a line and it comes back in capitals; BYE hangs up.
> hi
HI
> bye
Goodbye.
Disconnected.
```

HOW IT WORKS

NETCAT does four things:

1. It gets the devicespec, from the command line or by asking.
2. It opens network channel 1 on it, in mode 12 (read and write) with no translation, so every byte passes untouched in both directions.
3. It loops: if a key has been struck, it sends it; then it asks the channel's STATUS; if bytes are waiting, it reads them, no more than its buffer holds, and types them on the screen. If nothing is waiting and STATUS says the connection is gone, the far end has hung up.
4. It closes the channel.

There is no waiting anywhere in the loop. NETCAT never asks the FujiNet for bytes it does not have (Chapter 10 explains why that matters: a READ for more than is waiting fails, and loses what was there), and never waits for a key. So keys go out the moment they are struck, and bytes appear the moment they arrive.

THE H19 AND ANSI

Bulletin boards send the escape sequences of the ANSI standard (VT100 terminals and the IBM PC's ANSI.SYS), not the H19's own. Fortunately the H19 understands them too, in its **ANSI mode**. NETCAT sends `ESC <` when the connection opens, which puts the H19 in ANSI mode, and `ESC [ ? 2 h` when it closes, which puts it back in Heath mode, so the screen is as you left it. In ANSI mode the H19's keypad and arrow keys send ANSI sequences too, which is what the far end expects.

For a Telnet server, tell it what you are with the devicespec's query: `N:TELNET://BBS.FOZZTEXX.COM/?term=ansi&cols=80&rows=24`.

LINE ENDINGS

NETCAT opens the channel with no translation, so the far end's line endings reach the H19 as they are: most servers send CR LF, which the H19 shows correctly. The RETURN key sends CR, as a terminal should.

THE ASSEMBLY VERSION

NETCAT.ASM opens the channel with a request block whose payload is the devicespec; its length is set just before the call, since it depends on what was typed:

NETCAT.ASM — OPEN THE CHANNEL

```
        ld      hl,spec                 ; open the connection: the payload
        call    strlen                  ;   is the devicespec
        ld      (rq_open+RQ_PLEN),bc
        ld      hl,rq_open
        call    fn_call
        jp      nz,open_failed
```

The main loop is the heart of the program. `conin` returns with carry set when no key is waiting; a key goes out in a one-byte WRITE. Then STATUS fills `nstat`: two bytes saying how much is waiting, a byte that is 1 while the connection is up, and an error code (Chapter 10). The READ's length field is stored into its request block before each call:

NETCAT.ASM — THE LOOP

```
loop:
        call    conin                   ; a key struck?
        jr      c,poll
        cp      QUIT_KEY
        jr      z,hangup
        ld      (outch),a               ; send it: one byte
        ld      hl,rq_write
        call    fn_call
poll:
        ld      hl,rq_status            ; anything come in?
        call    fn_call
        jr      nz,loop
        ld      hl,(nstat+NS_AVAIL)
        ld      a,h
        or      l
        jr      nz,fetch
        ld      a,(nstat+NS_CONN)       ; nothing: still connected?
        or      a
        jr      nz,loop
        jr      hungup                  ; the far end has gone
fetch:
        ld      de,BUFSZ                ; take what waits, a buffer at most
        or      a
        sbc     hl,de
        add     hl,de
        jr      c,fetch1
        ex      de,hl
fetch1:
        ld      (rq_read+RQ_AUX1),hl    ; the read's length field
        ld      hl,rq_read
        call    fn_call
        jr      nz,loop
        ld      hl,rxbuf                ; BC bytes came: type them
show:
        ld      a,b
        or      c
        jr      z,loop
        ld      a,(hl)
        call    conout
        inc     hl
        dec     bc
        jr      show
```

The requests themselves are five blocks:

NETCAT.ASM — THE REQUESTS

```
rq_open:                            ; mode 12: read and write; no
        FNREQ   DEV_NETWORK, NET_OPEN, F_A1_A2, 12,0,0,0    ;   translation
        FNBUF   spec, 0, 0, 0
rq_status:
        FNREQ   DEV_NETWORK, NET_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, nstat, 4
rq_read:                            ; length set before each read
        FNREQ   DEV_NETWORK, NET_READ, F_B12, 0,0,0,0
        FNBUF   0, 0, rxbuf, BUFSZ
rq_write:
        FNREQ   DEV_NETWORK, NET_WRITE, F_B12, 1,0,0,0
        FNBUF   outch, 1, 0, 0
rq_close:
        FNREQ   DEV_NETWORK, NET_CLOSE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
```

THE C VERSION

NETCAT.C does the same with fujinet-lib, and with the book's `h89con.c` for the keyboard and screen. `con_init(1)` gives it each key as it is struck, with no echo (the far end echoes) and no CTRL-C handling, so every key, CTRL-C included, goes to the far end.

`network_status()` returns the same three things the assembly version takes from the reply, and `network_read()` reads the bytes. Note `len`, which never asks for more than is waiting.

NETCAT.C — THE LOOP

```
for (;;) {
    c = con_getc();                         /* a key struck? */
    if (c == QUIT_KEY)
        break;
    if (c >= 0) {
        rxbuf[0] = c;                       /* send it */
        network_write(spec, rxbuf, 1);
    }
    if (network_status(spec, &avail, &connected, &err) != FN_ERR_OK)
        continue;
    if (!avail) {
        if (!connected)                     /* the far end has gone */
            break;
        continue;
    }
    len = avail > BUFSZ ? BUFSZ : avail;    /* take what waits */
    len = network_read(spec, rxbuf, len);
    for (i = 0; i < len; i++)               /* and type it */
        con_putc(rxbuf[i]);
}
```

LISTINGS

SRC/ASM/NETCAT/NETCAT.ASM — 199 LINES

```
;--------------------------------------------------------------------------
; NETCAT -- the H89 as a terminal on the network. Opens a connection on
; network unit 1 (a TELNET or TCP devicespec, typed on the command line or
; asked for), then passes keys out and bytes in until the far end hangs up
; or CTRL-] is struck. The H19 is put in ANSI mode for the session, since
; that is what bulletin boards speak, and back in Heath mode after.
;
;   NETCAT N:TELNET://BBS.FOZZTEXX.COM/     or just NETCAT, and answer
;
; One source, two systems:
;   z88dk-z80asm -b -DHDOS -Isrc/asm/inc netcat.asm  ->  NETCAT.ABS (HDOS)
;   z88dk-z80asm -b -DCPM  -Isrc/asm/inc netcat.asm  ->  NETCAT.COM (CP/M)
;
; Copyright (C) 2026 Thomas Cherryhomes
; SPDX-License-Identifier: GPL-3.0-or-later
;--------------------------------------------------------------------------
INCLUDE "os.inc"
INCLUDE "fujinet.inc"

QUIT_KEY        equ     0x1D            ; CTRL-] ends the session
BUFSZ           equ     512             ; bytes taken from the FujiNet at once
ESC             equ     0x1B

main:
        ld      hl,t_banner
        call    print
        call    card_on                 ; start the card (if nothing has)
        call    card_present
        jp      nz,no_card

        call    args                    ; a devicespec on the command line?
        ld      a,(hl)
        or      a
        jr      nz,have_spec
        ld      hl,t_ask                ; no: ask for one
        call    print
        ld      hl,line
        ld      b,120
        call    getline
        ld      a,(hl)
        or      a
        jr      nz,have_spec
        ld      hl,t_default            ; just RETURN: the default
have_spec:
        call    set_spec                ; to spec, with N: in front
        ld      hl,t_conn
        call    print
        ld      hl,spec
```

```
        call    print
        call    crlf

        ld      hl,spec                 ; open the connection: the payload
        call    strlen                  ;   is the devicespec
        ld      (rq_open+RQ_PLEN),bc
        ld      hl,rq_open
        call    fn_call
        jp      nz,open_failed
        ld      hl,t_ansi               ; the H19 in ANSI mode
        call    print

loop:
        call    conin                   ; a key struck?
        jr      c,poll
        cp      QUIT_KEY
        jr      z,hangup
        ld      (outch),a               ; send it: one byte
        ld      hl,rq_write
        call    fn_call
poll:
        ld      hl,rq_status            ; anything come in?
        call    fn_call
        jr      nz,loop
        ld      hl,(nstat+NS_AVAIL)
        ld      a,h
        or      l
        jr      nz,fetch
        ld      a,(nstat+NS_CONN)       ; nothing: still connected?
        or      a
        jr      nz,loop
        jr      hungup                  ; the far end has gone
fetch:
        ld      de,BUFSZ                ; take what waits, a buffer at most
        or      a
        sbc     hl,de
        add     hl,de
        jr      c,fetch1
        ex      de,hl
fetch1:
        ld      (rq_read+RQ_AUX1),hl    ; the read's length field
        ld      hl,rq_read
        call    fn_call
        jr      nz,loop
        ld      hl,rxbuf                ; BC bytes came: type them
show:
        ld      a,b
        or      c
        jr      z,loop
        ld      a,(hl)
        call    conout
        inc     hl
        dec     bc
        jr      show

hangup:
hungup:
        ld      hl,t_heath              ; the H19 back in Heath mode
        call    print
        ld      hl,rq_close             ; and the connection closed
        call    fn_call
        ld      hl,t_bye
        jp      print

no_card:
        ld      hl,t_nocard
        jp      print
open_failed:
        ld      hl,t_nopen
        jp      print

; HL -> what was typed: copy it to spec, putting "N:" in front unless it
; starts with N: or Nn: already
set_spec:
        ld      de,spec
        ld      a,(hl)
        and     0xDF                    ; upper case
        cp      'N'
        jr      nz,set_n
        inc     hl
        ld      a,(hl)
        dec     hl
        cp      ':'
        jr      z,set_copy
        inc     hl
        inc     hl
        ld      a,(hl)
        dec     hl
        dec     hl
        cp      ':'
        jr      z,set_copy
set_n:
        ld      a,'N'
        ld      (de),a
        inc     de
        ld      a,':'
        ld      (de),a
        inc     de
set_copy:
        ld      a,(hl)
        ld      (de),a
        inc     hl
        inc     de
        or      a
        jr      nz,set_copy
        ret

rq_open:                                ; mode 12: read and write; no
        FNREQ   DEV_NETWORK, NET_OPEN, F_A1_A2, 12,0,0,0    ;   translation
        FNBUF   spec, 0, 0, 0
rq_status:
        FNREQ   DEV_NETWORK, NET_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, nstat, 4
rq_read:                                ; length set before each read
        FNREQ   DEV_NETWORK, NET_READ, F_B12, 0,0,0,0
        FNBUF   0, 0, rxbuf, BUFSZ
rq_write:
        FNREQ   DEV_NETWORK, NET_WRITE, F_B12, 1,0,0,0
        FNBUF   outch, 1, 0, 0
rq_close:
        FNREQ   DEV_NETWORK, NET_CLOSE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0

t_banner:       defm    "NETCAT - the H89 on the network.  CTRL-] hangs up."
                defb    13, 10, 0
t_ask:          defm    "Connect to (RETURN for BBS.FOZZTEXX.COM): "
                defb    0
t_default:      defm    "N:TELNET://BBS.FOZZTEXX.COM/"
                defb    0
t_conn:         defm    "Connecting to "
                defb    0
t_ansi:         defb    ESC, '<', 0             ; H19: enter ANSI mode
t_heath:        defb    ESC, '[', '?', '2', 'h', 0  ; ANSI: back to Heath mode
t_bye:          defb    13, 10
                defm    "Disconnected."
                defb    13, 10, 0
t_nocard:       defm    "No FujiNet card at 0D0H."
                defb    13, 10, 0
t_nopen:        defm    "The FujiNet could not open that."
                defb    13, 10, 0

outch:          defb    0               ; the key to send
nstat:          defs    4               ; STATUS: waiting (2), connected, error
line:           defs    120             ; what was typed
spec:           defs    124             ; the devicespec
rxbuf:          defs    BUFSZ

INCLUDE "card.inc"
INCLUDE "fujibus.inc"
INCLUDE "con.inc"
INCLUDE "tail.inc"
```

SRC/C/NETCAT/NETCAT.C 120 LINES

```
/*
 * NETCAT -- the H89 as a terminal on the network, in C. Opens a connection
 * (a TELNET or TCP devicespec, from the command line or asked for), then
 * passes keys out and bytes in until the far end hangs up or CTRL-] is
 * struck. The H19 is put in ANSI mode for the session, since that is what
 * bulletin boards speak, and back in Heath mode after. NETCAT.ASM's twin.
 *
 *   NETCAT N:TELNET://BBS.FOZZTEXX.COM/     or just NETCAT, and answer
 *
 *   tools/cc.sh hdos NETCAT src/c/netcat/netcat.c src/c/common/h89con.c \
 *       src/c/common/hdosargs.c                     ->  NETCAT.ABS
 *   tools/cc.sh cpm NETCAT ...the same...           ->  NETCAT.COM
 *
 * Copyright (C) 2026 Thomas Cherryhomes
 * SPDX-License-Identifier: GPL-3.0-or-later
 */
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <fujinet-network.h>
#include "h89con.h"
#include "hdosargs.h"

#define QUIT_KEY 0x1D           /* CTRL-] ends the session */
#define BUFSZ    512            /* bytes taken from the FujiNet at once */
#define DEFAULT  "N:TELNET://BBS.FOZZTEXX.COM/"

static char spec[124];
static char line[120];
static uint8_t rxbuf[BUFSZ];

```

```
/* a line typed at the keyboard, shown as it is typed */
static void getline(char *buf, uint8_t size)
{
    uint8_t n = 0;
    int c;

    for (;;) {
        while ((c = con_getc()) < 0)
            ;
        if (c == '\r')
            break;
        if ((c == 8 || c == 0x7F) && n) {
            n--;
            con_puts("\b \b");
        } else if (c >= ' ' && n < size - 1) {
            buf[n++] = c;
            con_putc(c);
        }
    }
    buf[n] = 0;
    con_puts("\r\n");
}

/* what was typed, with N: in front unless it has N: or Nn: already */
static void set_spec(const char *s)
{
    if (toupper(s[0]) == 'N' && (s[1] == ':' || s[2] == ':'))
        strcpy(spec, s);
    else {
        strcpy(spec, "N:");
        strcat(spec, s);
    }
}

int main(int argc, char *argv[])
{
    uint16_t avail, len, i;
    uint8_t connected, err;
    int c;

    ARGS(argc, argv);
    con_init(1);
    con_puts("NETCAT - the H89 on the network.  CTRL-] hangs up.\r\n");

    if (argc > 1)
        set_spec(argv[1]);
    else {
        con_puts("Connect to (RETURN for BBS.FOZZTEXX.COM): ");
        getline(line, sizeof line);
        set_spec(line[0] ? line : DEFAULT);
    }
    con_puts("Connecting to ");
    con_puts(spec);
    con_puts("\r\n");

    if (network_open(spec, 12, 0) != FN_ERR_OK) {   /* read and write */
        con_puts("The FujiNet could not open that.\r\n");
        con_exit();
        return 1;
    }
    con_puts("\x1B(");                              /* the H19 in ANSI mode */

    for (;;) {
        c = con_getc();                             /* a key struck? */
        if (c == QUIT_KEY)
            break;
        if (c >= 0) {
            rxbuf[0] = c;                           /* send it */
            network_write(spec, rxbuf, 1);
        }
        if (network_status(spec, &avail, &connected, &err) != FN_ERR_OK)
            continue;
        if (!avail) {
            if (!connected)                         /* the far end has gone */
                break;
            continue;
        }
        len = avail > BUFSZ ? BUFSZ : avail;        /* take what waits */
        len = network_read(spec, rxbuf, len);
        for (i = 0; i < len; i++)                   /* and type it */
            con_putc(rxbuf[i]);
    }

    con_puts("\x1B[?2h");                           /* back to Heath mode */
    network_close(spec);
    con_puts("\r\nDisconnected.\r\n");
    con_exit();
    return 0;
}
```

NGET AND NPUT

GENERAL

NGET copies a file from the network to one of your disks; NPUT copies one the other way. “The network” means anything a channel can open: a web server, a TNFS or FTP server, a Windows share, the FujiNet’s own microSD card (Chapter 11). With the two of them, an H89 can fetch a program from the Internet, or send its work to a modern computer, without a floppy disk changing hands.

Each comes in assembly language and in C, and each makes a program for HDOS and for CP/M. Like NETCAT, they are written to be read: about 200 lines of assembly, 100 of C.

USING NGET AND NPUT

```
NGET devicespec file          NPUT file devicespec
```

( ) To fetch a web page into a file under HDOS:
NGET N:HTTP://FUJINET.ONLINE/ SY1:INDEX.HTM ↵

( ) To send a file to a TNFS server under CP/M:
NPUT B:NOTES.TXT N:TNFS://MYSERVER/NOTES.TXT ↵

( ) Or run either with nothing after its name, and it asks for the two names.

NOTE: HDOS and CP/M turn a typed command line into capitals. Many servers — web servers, and TNFS or SMB servers on UNIX — care about the case of a path, and `/DATA.BIN` is not `/data.bin` to them. When a name needs small letters, run the program with nothing after its name and type the names at its questions: what you type there is kept as you typed it.

Here are NGET under HDOS and NPUT under CP/M, the first fetching 3,000 bytes from the book’s test server, the second sending a short text file to it:

```
>RUN SY1:NGET
NGET - copy a file from the network
From (devicespec): N:HTTP://192.168.1.10:7802/data.bin
To (file): SY1:DATA.BIN
3000 bytes copied.

A>B:NPUT
NPUT - copy a file to the network
From (file): B:HELLO.TXT
To (devicespec): N:HTTP://192.168.1.10:7802/up.txt
128 bytes sent.
```

The test server (`tools/helpers.py`) logged what NPUT sent: a PUT of 128 bytes, the file’s two lines followed by whatever filled the rest of its 128-byte record. Here that was E5H, the byte of an empty diskette, because the book’s tools put the file on the disk; a file saved by a CP/M program would have ended in CTRL-Zs. Which brings us to file lengths.

HOW LONG IS A FILE?

Neither HDOS nor CP/M records how many bytes a file holds. HDOS knows how many 256-byte sectors it has, and CP/M how many 128-byte records. So:

- **NGET** writes every byte it receives, and the file is rounded up to a whole sector or record: NGET.ASM's library fills the rest of the last HDOS sector with zeros, and of the last CP/M record with CTRL-Zs; NGET.C, through z88dk's file routines, fills both with CTRL-Zs. For a text file that does no harm: CTRL-Z is CP/M's end-of-text mark, and HDOS fills the end of a text file's last sector with zeros itself. For a program or a disk image it does no harm either, since the extra bytes are never used.
- **NPUT** sends the whole of the last sector or record, because it cannot tell where the data ended. A text file sent from CP/M arrives with CTRL-Zs on its end; one from HDOS, with zeros. Most modern editors show these, and are happy to delete them.

A program that knows its files are text could stop at the first CTRL-Z, or the first zero. NGET and NPUT don't, because they cannot know.

HOW NGET WORKS

NGET opens the devicespec on channel 1 for reading (mode 4), with no translation, so a binary file arrives intact. Then it loops, much as NETCAT does: STATUS, and if bytes are waiting, READ them — never more than are waiting — and write them to the file.

When nothing is waiting, the error byte of STATUS says why. 136 means **end of data**: the whole file has come, and NGET is done. 1 means all is well, but nothing has come yet: a slow server, or a long way away. Anything else is a failure, and NGET reports its number (the Appendix lists them). NGET gives up if nothing at all comes for 30 seconds.

NGET.ASM — THE COPYING LOOP

```
        ld      hl,(TICKS)              ; when data last came
        ld      (t_last),hl
copy:
        ld      hl,rq_status            ; how much is waiting?
        call    fn_call
        jp      nz,net_error
        ld      hl,(nstat+NS_AVAIL)
        ld      a,h
        or      l
        jr      nz,fetch
        ld      a,(nstat+NS_ERR)        ; nothing: the end?
        cp      EOF_ERR
        jr      z,done
        cp      2                       ; an error (1 is all well)
        jp      nc,net_error
        ld      a,(nstat+NS_CONN)       ; the far end gone, nothing left
        or      a
        jr      z,done
        ld      hl,(TICKS)              ; still waiting: for too long?
        ld      de,(t_last)
        or      a
        sbc     hl,de
        ld      de,WAIT_SECS*500
        or      a
        sbc     hl,de
        jr      c,copy
        jp      timed_out
fetch:
        ld      de,BUFSZ                ; never ask for more than waits
        or      a
        sbc     hl,de
        add     hl,de
        jr      c,fetch1
        ex      de,hl
fetch1:
        ld      (rq_read+RQ_AUX1),hl
        ld      hl,rq_read
        call    fn_call
        jp      nz,net_error
        push    bc                      ; BC bytes came: to the file
        ld      de,rxbuf
        call    f_write
        pop     bc
        jp      c,write_error
        ld      hl,(total)              ; count them
        add     hl,bc
        ld      (total),hl
        ld      hl,(TICKS)
        ld      (t_last),hl
        jr      copy
```

NOTE: For an HTTP channel, the request is sent when the program first asks for STATUS (or READ), not when the channel is opened. So the first STATUS may take a moment: it waits for the server's answer to begin.

The C version is the same loop, with `fwrite()` for the file:

NGET.C — THE COPYING LOOP

```
last = TICKS;
for (;;) {
    if (network_status(spec, &avail, &connected, &err) != FN_ERR_OK
```

```
        || (avail == 0 && err > 1 && err != EOF_ERR)) {
        printf("Network error %u\n", err);
        result = 1;
        break;
    }
    if (avail == 0) {
        if (err == EOF_ERR || !connected)      /* the end */
            break;
        if ((uint16_t)(TICKS - last) > WAIT_SECS * TICKS_PER_SECOND) {
            printf("Nothing came for %u seconds; stopped.\n", WAIT_SECS);
            result = 1;
            break;
        }
        continue;
    }
    len = avail > BUFSZ ? BUFSZ : avail;       /* never more than waits */
    len = network_read(spec, buf, len);
    if (fwrite(buf, 1, len, f) != len) {
        printf("Cannot write the file (disk full?).\n");
        result = 1;
        break;
    }
    total += len;
    last = TICKS;
}
```

HOW NPUT WORKS

NPUT opens the devicespec for writing (mode 8) and sends the file in pieces of up to 512 bytes, each in one WRITE request: its length goes in the request's field **and** is the length of its payload.

NPUT.ASM — SENDING THE FILE

```
copy:
        ld      de,txbuf                ; a piece of the file
        ld      bc,BUFSZ
        call    f_read
        ld      a,b
        or      c
        jr      z,done                  ; the end of the file
        ld      (rq_write+RQ_AUX1),bc   ; the length, as a field
        ld      (rq_write+RQ_PLEN),bc   ;   and of the payload
        push    bc
        ld      hl,rq_write
        call    fn_call
        pop     bc
        jr      nz,net_error
        ld      hl,(total)
        add     hl,bc
        ld      (total),hl
        jr      copy
```

Then it closes the channel. For most protocols the file is complete once the last WRITE is answered; but HTTP sends a PUT request only when the channel closes, with everything written as its body. Until CLOSE, nothing has reached the web server at all.

WARNING: Don't ask a PUT channel for its STATUS before closing it. The FujiNet sends the PUT at the first STATUS, and then again at CLOSE: the server gets the file twice, or, if more was written in between, gets two different files. NPUT never asks.

NPUT.C — CLOSE IS WHEN AN HTTP PUT IS SENT

```
network_close(spec);                        /* an HTTP PUT goes now */
fclose(f);
```

GETTING THE NAMES

Both programs take their two names from the command line, or ask for them. The assembly versions use two small routines from the book's `words.inc`: `word`, which takes the next word from the command line, and `netspec`, which puts `N:` in front of a devicespec that lacks it. The C versions use the same two, in `words.c`.

NGET.ASM — THE NAMES

```
        call    args                    ; NGET devicespec file
        ld      de,spec
        call    word
        jr      nz,two_words
        ld      hl,t_from               ; none: ask, since CP/M turns a
        call    print                   ;   command line into capitals
        ld      hl,spec
        ld      b,128
        call    getline
        ld      hl,t_to
        call    print
        ld      hl,fname
        ld      b,30
        call    getline
        jr      got_words
two_words:
        ld      de,fname
        call    word
        jp      z,usage
got_words:
        ld      de,spec                 ; N: in front if it is missing
        call    netspec
```

LISTINGS

SRC/ASM/NGET/NGET.ASM — 208 LINES

```
;-----------------------------------------------------------------------
; NGET -- copy a file from the network to a disk. Opens the devicespec on
; network unit 1 for reading, and writes whatever comes to the file, until
; the FujiNet says the end has been reached.
;
;   NGET N:HTTP://192.168.1.10/hello.txt SY1:HELLO.TXT      (HDOS)
;   NGET N:TNFS://tnfs.fujinet.online/README.TXT B:README.TXT  (CP/M)
;
; Any scheme a channel can read will do (Chapter 11). HDOS and CP/M keep
; files in whole sectors (256 bytes) and records (128), so the copy ends
; with up to a sector of zeros, or a record of CTRL-Z, after the data.
;
; One source, two systems:
;   z88dk-z80asm -b -DHDOS -Isrc/asm/inc nget.asm   ->  NGET.ABS (HDOS)
;   z88dk-z80asm -b -DCPM  -Isrc/asm/inc nget.asm   ->  NGET.COM (CP/M)
;
; Copyright (C) 2026 Thomas Cherryhomes
; SPDX-License-Identifier: GPL-3.0-or-later
;-----------------------------------------------------------------------
INCLUDE "os.inc"
INCLUDE "fujinet.inc"

BUFSZ           equ     512             ; bytes taken from the FujiNet at once
EOF_ERR         equ     136             ; STATUS error: the end of the data
WAIT_SECS       equ     30              ; give up after this long with nothing

main:
        ld      hl,t_banner
        call    print
        call    card_on
        call    card_present
        jp      nz,no_card

        call    args                    ; NGET devicespec file
        ld      de,spec
        call    word
        jr      nz,two_words
        ld      hl,t_from               ; none: ask, since CP/M turns a
        call    print                   ;   command line into capitals
        ld      hl,spec
        ld      b,128
        call    getline
        ld      hl,t_to
        call    print
        ld      hl,fname
        ld      b,30
        call    getline
        jr      got_words
two_words:
        ld      de,fname
        call    word
        jp      z,usage
got_words:
        ld      de,spec                 ; N: in front if it is missing
        call    netspec

        ld      hl,spec                 ; open it for reading (mode 4),
        call    strlen                  ;   no translation
        ld      (rq_open+RQ_PLEN),bc
        ld      hl,rq_open
        call    fn_call
        jp      nz,no_open
        ld      hl,fname                ; and the file to write
        call    f_create
        jp      c,no_file

        ld      hl,(TICKS)              ; when data last came
        ld      (t_last),hl
copy:
        ld      hl,rq_status            ; how much is waiting?
        call    fn_call
        jp      nz,net_error
        ld      hl,(nstat+NS_AVAIL)
        ld      a,h
        or      l
        jr      nz,fetch
        ld      a,(nstat+NS_ERR)        ; nothing: the end?
        cp      EOF_ERR
        jr      z,done
        cp      2                       ; an error (1 is all well)
        jp      nc,net_error
        ld      a,(nstat+NS_CONN)       ; the far end gone, nothing left
        or      a
        jr      z,done
        ld      hl,(TICKS)              ; still waiting: for too long?
        ld      de,(t_last)
        or      a
        sbc     hl,de
        ld      de,WAIT_SECS*500
        or      a
        sbc     hl,de
        jr      c,copy
        jp      timed_out
fetch:
        ld      de,BUFSZ                ; never ask for more than waits
        or      a
        sbc     hl,de
        add     hl,de
        jr      c,fetch1
        ex      de,hl
fetch1:
        ld      (rq_read+RQ_AUX1),hl
```

```
        ld      hl,rq_read
        call    fn_call
        jp      nz,net_error
        push    bc                      ; BC bytes came: to the file
        ld      de,rxbuf
        call    f_write
        pop     bc
        jp      c,write_error
        ld      hl,(total)              ; count them
        add     hl,bc
        ld      (total),hl
        ld      hl,(TICKS)
        ld      (t_last),hl
        jr      copy

done:
        call    finish
        ld      hl,(total)
        call    pdec16
        ld      hl,t_done
        jp      print

; close the channel and the file, whatever happened
finish:
        ld      hl,rq_close
        call    fn_call
        jp      f_close

usage:
        ld      hl,t_usage
        jp      print
no_card:
        ld      hl,t_nocard
        jp      print
no_open:
        ld      hl,t_noopen
        jp      print
no_file:
        ld      hl,rq_close
        call    fn_call
        ld      hl,t_nofile
        jp      print
net_error:
        call    finish
        ld      hl,t_neterr
        call    print
        ld      a,(nstat+NS_ERR)
        call    pdec8
        jp      crlf
write_error:
        call    finish
        ld      hl,t_wrerr
        jp      print
timed_out:
        call    finish
        ld      hl,t_tmo
        jp      print

rq_open:
        FNREQ   DEV_NETWORK, NET_OPEN, F_A1_A2, 4,0,0,0
        FNBUF   spec, 0, 0, 0
rq_status:
        FNREQ   DEV_NETWORK, NET_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, nstat, 4
rq_read:
        FNREQ   DEV_NETWORK, NET_READ, F_B12, 0,0,0,0
        FNBUF   0, 0, rxbuf, BUFSZ
rq_close:
        FNREQ   DEV_NETWORK, NET_CLOSE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0

t_banner:       defm    "NGET - copy a file from the network"
                defb    13, 10, 0
t_from:         defm    "From (devicespec): "
                defb    0
t_to:           defm    "To (file): "
                defb    0
t_usage:        defm    "Usage: NGET devicespec file"
                defb    13, 10, 0
t_nocard:       defm    "No FujiNet card at 0D0H."
                defb    13, 10, 0
t_noopen:       defm    "The FujiNet could not open that."
                defb    13, 10, 0
t_nofile:       defm    "Cannot create the file."
                defb    13, 10, 0
t_neterr:       defm    "Network error "
                defb    0
t_wrerr:        defm    "Cannot write the file (disk full?)."
                defb    13, 10, 0
t_tmo:          defm    "Nothing came for 30 seconds; stopped."
                defb    13, 10, 0
t_done:         defm    " bytes copied."
                defb    13, 10, 0

total:          defw    0
t_last:         defw    0
nstat:          defs    4               ; STATUS: waiting (2), connected, error
spec:           defs    132             ; the devicespec, room for N:
fname:          defs    32              ; the file
rxbuf:          defs    BUFSZ

INCLUDE "words.inc"
INCLUDE "card.inc"
INCLUDE "fujibus.inc"
INCLUDE "con.inc"
INCLUDE "tail.inc"
```

SRC/ASM/NPUT/NPUT.ASM 154 LINES

```
;-------------------------------------------------------------------------
; NPUT -- copy a file from a disk to the network. Opens the devicespec on
; network unit 1 for writing, sends the file in pieces, and closes the
; channel, which is when an HTTP PUT is really sent.
;
;   NPUT SY1:HELLO.TXT N:HTTP://192.168.1.10/hello.txt      (HDOS)
;   NPUT B:NOTES.TXT N:TNFS://myserver/notes.txt            (CP/M)
;
; HDOS and CP/M know a file's length only in whole sectors (256 bytes) and
; records (128), so the copy is that long: a text file arrives with the
; zeros or CTRL-Zs that fill its last sector or record.
;
; One source, two systems:
;   z88dk-z80asm -b -DHDOS -Isrc/asm/inc nput.asm   ->  NPUT.ABS (HDOS)
;   z88dk-z80asm -b -DCPM  -Isrc/asm/inc nput.asm   ->  NPUT.COM (CP/M)
;
; Copyright (C) 2026 Thomas Cherryhomes
; SPDX-License-Identifier: GPL-3.0-or-later
;-------------------------------------------------------------------------
INCLUDE "os.inc"
INCLUDE "fujinet.inc"

BUFSZ           equ     512             ; bytes sent to the FujiNet at once

main:
        ld      hl,t_banner
        call    print
        call    card_on
        call    card_present
        jp      nz,no_card

        call    args                    ; NPUT file devicespec
        ld      de,fname
        call    word
        jr      nz,two_words
        ld      hl,t_from               ; none: ask, since CP/M turns a
        call    print                   ;   command line into capitals
        ld      hl,fname
        ld      b,30
        call    getline
        ld      hl,t_to
        call    print
        ld      hl,spec
        ld      b,128
        call    getline
        jr      got_words
two_words:
        ld      de,spec
        call    word
        jp      z,usage
got_words:
        ld      de,spec                 ; N: in front if it is missing
        call    netspec
        ld      hl,fname                ; the file to send
        call    f_open
        jp      c,no_file

        ld      hl,spec                 ; open it for writing (mode 8),
        call    strlen                  ;   no translation
        ld      (rq_open+RQ_PLEN),bc
        ld      hl,rq_open
        call    fn_call
        jp      nz,no_open

copy:
        ld      de,txbuf                ; a piece of the file
        ld      bc,BUFSZ
        call    f_read
        ld      a,b
        or      c
        jr      z,done                  ; the end of the file
        ld      (rq_write+RQ_AUX1),bc   ; the length, as a field
        ld      (rq_write+RQ_PLEN),bc   ;   and of the payload
        push    bc
        ld      hl,rq_write
        call    fn_call
```

```
        pop     bc
        jr      nz,net_error
        ld      hl,(total)
        add     hl,bc
        ld      (total),hl
        jr      copy

done:
        ld      hl,rq_close             ; CLOSE: an HTTP PUT goes now
        call    fn_call
        push    af
        call    f_close
        pop     af
        jr      nz,net_error
        ld      hl,(total)
        call    pdec16
        ld      hl,t_done
        jp      print

usage:
        ld      hl,t_usage
        jp      print
no_card:
        ld      hl,t_nocard
        jp      print
no_file:
        ld      hl,t_nofile
        jp      print
no_open:
        call    f_close
        ld      hl,t_noopen
        jp      print
net_error:
        ld      hl,rq_close
        call    fn_call
        call    f_close
        ld      hl,t_neterr
        jp      print

rq_open:
        FNREQ   DEV_NETWORK, NET_OPEN, F_A1_A2, 8,0,0,0
        FNBUF   spec, 0, 0, 0
rq_write:                               ; length set before each write
        FNREQ   DEV_NETWORK, NET_WRITE, F_B12, 0,0,0,0
        FNBUF   txbuf, 0, 0, 0
rq_close:
        FNREQ   DEV_NETWORK, NET_CLOSE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0

t_banner:       defm    "NPUT - copy a file to the network"
                defb    13, 10, 0
t_from:         defm    "From (file): "
                defb    0
t_to:           defm    "To (devicespec): "
                defb    0
t_usage:        defm    "Usage: NPUT file devicespec"
                defb    13, 10, 0
t_nocard:       defm    "No FujiNet card at 0D0H."
                defb    13, 10, 0
t_nofile:       defm    "Cannot open the file."
                defb    13, 10, 0
t_noopen:       defm    "The FujiNet could not open that."
                defb    13, 10, 0
t_neterr:       defm    "Network error; the copy may be incomplete."
                defb    13, 10, 0
t_done:         defm    " bytes sent."
                defb    13, 10, 0

total:          defw    0
spec:           defs    132             ; the devicespec, room for N:
fname:          defs    32              ; the file
txbuf:          defs    BUFSZ

INCLUDE "words.inc"
INCLUDE "card.inc"
INCLUDE "fujibus.inc"
INCLUDE "con.inc"
INCLUDE "tail.inc"
```

SRC/C/NGET/NGET.C — 98 LINES

```
/*
 * NGET -- copy a file from the network to a disk, in C. Opens the
 * devicespec on network unit 1 for reading and writes whatever comes to
 * the file, until the FujiNet says the end has been reached. NGET.ASM's
 * twin.
 *
 *   NGET N:HTTP://192.168.1.10/hello.txt SY1:HELLO.TXT      (HDOS)
 *   NGET                    (and answer; CP/M capitalises a command line)
 *
 * HDOS and CP/M keep files in whole sectors and records, so the copy ends
 * with up to a sector of zeros, or a record of CTRL-Z, after the data.
 *
 *  tools/cc.sh hdos NGET src/c/nget/nget.c src/c/common/words.c \
 *      src/c/common/hdosargs.c                     ->  NGET.ABS
 *  tools/cc.sh cpm NGET ...the same...             ->  NGET.COM
 *
 * Copyright (C) 2026 Thomas Cherryhomes
 * SPDX-License-Identifier: GPL-3.0-or-later
 */
#include <stdio.h>
#include <string.h>
#include <fujinet-network.h>
#include "h89con.h"
#include "hdosargs.h"
#include "words.h"

#define BUFSZ    512          /* bytes taken from the FujiNet at once */
#define EOF_ERR  136          /* STATUS error: the end of the data */
#define WAIT_SECS 30          /* give up after this long with nothing */

static char spec[132], fname[32];
static uint8_t buf[BUFSZ];

int main(int argc, char *argv[])
{
    FILE *f;
    uint16_t avail, len, last;
    uint8_t connected, err;
    unsigned long total = 0;
    int result = 0;

    ARGS(argc, argv);
    con_init(0);
    printf("NGET - copy a file from the network\n");
    if (two_words(argc, argv, "From (devicespec): ", spec, sizeof spec - 2,
                  "To (file): ", fname, sizeof fname) < 0) {
        printf("Usage: NGET devicespec file\n");
        goto out;
    }
    net_spec(spec);

    if (network_open(spec, 4, 0) != FN_ERR_OK) {    /* read, no translation */
        printf("The FujiNet could not open that.\n");
        goto out;
    }
    if ((f = fopen(fname, "wb")) == NULL) {
        printf("Cannot create the file.\n");
        network_close(spec);
        goto out;
    }

    last = TICKS;
    for (;;) {
        if (network_status(spec, &avail, &connected, &err) != FN_ERR_OK
            || (avail == 0 && err > 1 && err != EOF_ERR)) {
            printf("Network error %u\n", err);
            result = 1;
            break;
        }
        if (avail == 0) {
            if (err == EOF_ERR || !connected)       /* the end */
                break;
            if ((uint16_t)(TICKS - last) > WAIT_SECS * TICKS_PER_SECOND) {
                printf("Nothing came for %u seconds; stopped.\n", WAIT_SECS);
                result = 1;
                break;
            }
            continue;
        }
        len = avail > BUFSZ ? BUFSZ : avail;        /* never more than waits */
        len = network_read(spec, buf, len);
        if (fwrite(buf, 1, len, f) != len) {
            printf("Cannot write the file (disk full?).\n");
            result = 1;
            break;
        }
        total += len;
        last = TICKS;
    }

    network_close(spec);
    fclose(f);
    if (!result)
        printf("%lu bytes copied.\n", total);
out:
    con_exit();
    return result;
}
```

SRC/C/NPUT/NPUT.C — 75 LINES

```
/*
 * NPUT -- copy a file from a disk to the network, in C. Opens the
```

```
 3 * devicespec on network unit 1 for writing, sends the file in pieces, and
 4 * closes the channel, which is when an HTTP PUT is really sent. NPUT.ASM's
 5 * twin.
 6 *
 7 *   NPUT SY1:HELLO.TXT N:HTTP://192.168.1.10/hello.txt      (HDOS)
 8 *   NPUT                    (and answer; CP/M capitalises a command line)
 9 *
10 * HDOS and CP/M know a file's length only in whole sectors and records,
11 * so the copy is that long.
12 *
13 *   tools/cc.sh hdos NPUT src/c/nput/nput.c src/c/common/words.c \
14 *       src/c/common/hdosargs.c                  ->  NPUT.ABS
15 *   tools/cc.sh cpm NPUT ...the same...          ->  NPUT.COM
16 *
17 * Copyright (C) 2026 Thomas Cherryhomes
18 * SPDX-License-Identifier: GPL-3.0-or-later
19 */
20 #include <stdio.h>
21 #include <string.h>
22 #include <fujinet-network.h>
23 #include "h89con.h"
24 #include "hdosargs.h"
25 #include "words.h"
26
27 #define BUFSZ 512              /* bytes sent to the FujiNet at once */
28
29 static char spec[132], fname[32];
30 static uint8_t buf[BUFSZ];
31
32 int main(int argc, char *argv[])
33 {
34     FILE *f;
35     size_t len;
36     unsigned long total = 0;
37     int result = 0;
38
39     ARGS(argc, argv);
40     con_init(0);
41     printf("NPUT - copy a file to the network\n");
42     if (two_words(argc, argv, "From (file): ", fname, sizeof fname,
43                   "To (devicespec): ", spec, sizeof spec - 2) < 0) {
44         printf("Usage: NPUT file devicespec\n");
45         goto out;
46     }
47     net_spec(spec);
48     if ((f = fopen(fname, "rb")) == NULL) {
49         printf("Cannot open the file.\n");
50         goto out;
51     }
52
53     if (network_open(spec, 8, 0) != FN_ERR_OK) {    /* write, no translation */
54         printf("The FujiNet could not open that.\n");
55         fclose(f);
56         goto out;
57     }
58
59     while ((len = fread(buf, 1, BUFSZ, f)) > 0) {
60         if (network_write(spec, buf, len) != FN_ERR_OK) {
61             printf("Network error; the copy may be incomplete.\n");
62             result = 1;
63             break;
64         }
65         total += len;
66     }
67
68     network_close(spec);                          /* an HTTP PUT goes now */
69     fclose(f);
70     if (!result)
71         printf("%lu bytes sent.\n", total);
72 out:
73     con_exit();
74     return result;
75 }
```

SRC/ASM/INC/WORDS.INC — 80 LINES

```
 1 ;-------------------------------------------------------------------------
 2 ; words.inc -- picking a command line apart.
 3 ;
 4 ;   word     HL -> text: skip spaces, copy the next word (up to a space or
 5 ;            the end) to DE as a NUL-terminated string. HL is left after
 6 ;            it. Z if there was no word.
 7 ;   netspec  DE -> a word: if it does not start "N:" or "Nn:", put "N:" in
 8 ;            front of it, in place (the buffer needs 2 bytes of room).
 9 ;            Changes HL.
10 ;
11 ; Copyright (C) 2026 Thomas Cherryhomes
12 ; SPDX-License-Identifier: GPL-3.0-or-later
13 ;-------------------------------------------------------------------------
14
15 word:
16         ld      a,(hl)                ; skip spaces
17         cp      ' '
18         jr      nz,word1
19         inc     hl
20         jr      word
21 word1:
22         or      a                     ; nothing left?
23         jr      z,word3
24         push    de
25 word2:
26         ld      a,(hl)
27         or      a
28         jr      z,word4
29         cp      ' '
30         jr      z,word4
31         ld      (de),a
32         inc     hl
33         inc     de
34         jr      word2
35 word4:
36         xor     a
37         ld      (de),a
38         pop     de
39         inc     a                     ; NZ: a word
40         ret
41 word3:
42         ld      (de),a                ; an empty word; Z
43         ret
44
45 netspec:
46         ld      a,(de)
47         and     0xDF                  ; upper case
48         cp      'N'
49         jr      nz,nets1
50         inc     de
51         ld      a,(de)
52         dec     de
53         cp      ':'
54         ret     z                     ; N:...
55         inc     de
56         inc     de
57         ld      a,(de)
58         dec     de
59         dec     de
60         cp      ':'
61         ret     z                     ; Nn:...
62 nets1:
63         push    de                    ; shift right 2, then N: in front
64         ex      de,hl
65         call    strlen
66         add     hl,bc                 ; HL -> the NUL
67         ld      d,h
68         ld      e,l
69         inc     de
70         inc     de
71         inc     bc                    ; move the NUL too
72         lddr
73         pop     de
74         ld      a,'N'
75         ld      (de),a
76         inc     de
77         ld      a,':'
78         ld      (de),a
79         dec     de
80         ret
```

SRC/C/COMMON/WORDS.H — 22 LINES

```
 1 /*
 2  * words.h -- the command line, or questions if there is none.
 3  *
 4  *  two_words(argc, argv, q1, a1, size1, q2, a2, size2)
 5  *       Takes the first two arguments, or, if there are none, asks for
 6  *       them (CP/M turns a command line into capitals, which a web path
 7  *       may not survive). Returns 0 if it got two, -1 if not.
 8  *  net_spec(s)
 9  *       Puts "N:" in front of s unless it starts N: or Nn: already. s needs
10  *       two bytes of room.
11  *
12  * Copyright (C) 2026 Thomas Cherryhomes
13  * SPDX-License-Identifier: GPL-3.0-or-later
14  */
15 #ifndef WORDS_H
16 #define WORDS_H
17
18 int two_words(int argc, char **argv, const char *q1, char *a1, int size1,
19               const char *q2, char *a2, int size2);
20 void net_spec(char *s);
21
22 #endif
```

SRC/C/COMMON/WORDS.C 49 LINES

```c
/*
 * words.c -- the command line, or questions if there is none: see words.h.
 *
 * Copyright (C) 2026 Thomas Cherryhomes
 * SPDX-License-Identifier: GPL-3.0-or-later
 */
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include "words.h"

static void ask(const char *q, char *a, int size)
{
    char *nl;

    fputs(q, stdout);
    fflush(stdout);
    if (!fgets(a, size, stdin))
        a[0] = 0;
    if ((nl = strchr(a, '\n')) != NULL)
        *nl = 0;
    if ((nl = strchr(a, '\r')) != NULL)
        *nl = 0;
}

int two_words(int argc, char **argv, const char *q1, char *a1, int size1,
              const char *q2, char *a2, int size2)
{
    if (argc >= 3) {
        strncpy(a1, argv[1], size1 - 1);
        a1[size1 - 1] = 0;
        strncpy(a2, argv[2], size2 - 1);
        a2[size2 - 1] = 0;
    } else if (argc == 1) {
        ask(q1, a1, size1);
        ask(q2, a2, size2);
    } else
        return -1;
    return a1[0] && a2[0] ? 0 : -1;
}

void net_spec(char *s)
{
    if (toupper(s[0]) == 'N' && (s[1] == ':' || s[2] == ':'))
        return;
    memmove(s + 2, s, strlen(s) + 1);
    s[0] = 'N';
    s[1] = ':';
}
```

ISS TRACKER

GENERAL

The International Space Station goes round the Earth every hour and a half, and a free web service, api.open-notify.org, will tell anyone where it is. It answers in JSON, the text format nearly every web service uses today:

```
{"message": "success", "timestamp": 1760270400,
 "iss_position": {"latitude": "29.7604", "longitude": "-95.3698"}}
```

An H89 has no business taking that apart itself, and with the FujiNet it doesn't have to: the FujiNet's JSON parser does it, and hands the H89 just the values it asks for. ISS shows how. It lists the people in space, then plots the station on a map of the world, drawn on the H19, and moves it every 30 seconds.

ISS comes in assembly and in C, each for HDOS and for CP/M. It began as Justin Skists's ISS tracker for the RC2014, itself after Bill Kendrick's for the Atari; the map is theirs.

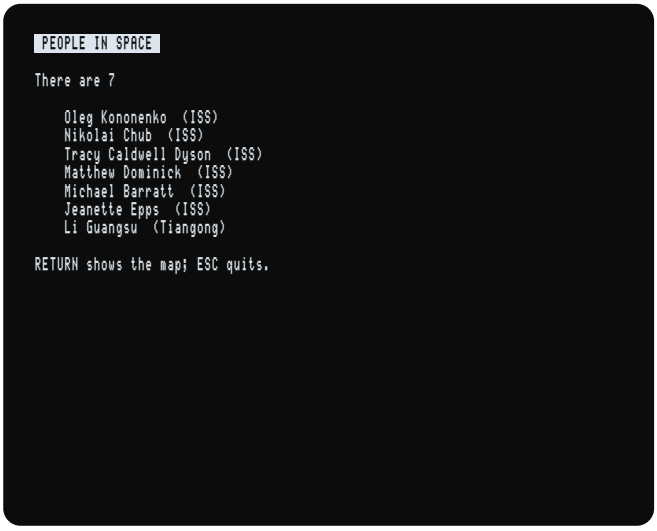
RUNNING ISS

( ) Type ISS ↵. ISS asks open-notify who is in space, and lists them.

( ) Press ↵ for the map. The station is the blinking star. Every 30 seconds ISS asks again and moves it; ↵ asks at once.

( ) Press ESC to leave.

To use a server of your own (one with files named `iss-now.json` and `astros.json`, such as the book's test server), give its address: `ISS N:HTTP://192.168.1.10/`. These screens were taken that way, with a fixed position, so the station is over Houston:





THE FUJINET'S JSON PARSER

Every network channel has a **parser**. It is normally off, and READ returns the bytes the server sent. Set to JSON, it reads the whole document into the FujiNet's memory, checks it, and from then on READ returns only what you ask for, a value at a time. Asking takes four requests:

1. OPEN the document on a channel, as usual.
2. SET_PARSER with its second field 1, for JSON (2 is HTML, 3 XML).
3. PARSE, which fetches and parses the whole document. For HTTP, this is when the request is sent.
4. Then, for each value: QUERY with a **JSON Pointer** as the payload, STATUS to learn how long the answer is, and READ to take it.

A JSON Pointer (the Internet standard RFC 6901) is a path through the document: `/timestamp` is the member **timestamp** of the top object; `/iss_position/latitude` the member **latitude** of the object **iss_position**; `/people/2/name` the **name** of the third element (counting from 0) of the array **people**. The FujiNet answers with the value as text — a string without its quotes, a number as digits — followed by a line feed.

If the path names nothing, QUERY still answers ACK, but STATUS then shows nothing waiting. That is how ISS knows a value is missing.

WARNING: The firmware is fragile here. A query whose answer contains an empty array (`[]`) crashes it; so does SET_PARSER on a channel that is not open, followed by a QUERY or READ. ISS opens the channel first, and asks only for single values.

THE ASSEMBLY VERSION

Two routines do all the JSON work. `open_json` opens a URL and parses it:

ISS.ASM — OPEN AND PARSE A JSON DOCUMENT

```
open_json:
        push    hl
        call    strlen
        pop     hl
        ld      (rq_open+RQ_PAY),hl
        ld      (rq_open+RQ_PLEN),bc
        ld      hl,rq_open              ; open: GET the document
        call    fn_call
        ret     nz
        ld      hl,rq_parser            ; the channel's parser: JSON
        call    fn_call
        ret     nz
        ld      hl,rq_parse             ; read and parse the whole reply
        jp      fn_call
```

`jget` asks for one value and reads it into `val`, dropping the line feed:

ISS.ASM — ONE VALUE, BY JSON POINTER

```
jget:
        push    hl
        call    strlen                  ; the query is the whole payload
        pop     hl
        ld      (rq_query+RQ_PAY),hl
        ld      (rq_query+RQ_PLEN),bc
        ld      hl,rq_query
        call    fn_call
        ret     nz
        ld      hl,rq_status            ; the value is waiting to be read
        call    fn_call
        ret     nz
        ld      hl,(nstat+NS_AVAIL)
        ld      a,h
        or      l
        jr      z,jget_none             ; not found: nothing waits
        ld      de,VALSZ-1
        or      a
        sbc     hl,de
        add     hl,de
        jr      c,jget1
        ex      de,hl
jget1:
        ld      (rq_read+RQ_AUX1),hl
        ld      hl,rq_read
        call    fn_call
        ret     nz
        ld      hl,val                  ; end it at BC, then drop the
        add     hl,bc                   ;   line feed (and any CR)
        ld      (hl),0
jget2:
        ld      a,b
        or      c
        jr      z,jget3
        dec     hl
        ld      a,(hl)
        cp      ' '
        jr      nc,jget3
        ld      (hl),0
        dec     bc
        jr      jget2
jget3:
        xor     a                       ; Z: found
        ret
jget_none:
        inc     a                       ; NZ
        ret
```

With those, the list of people is a loop that builds `/people/0/name`, `/people/0/craft`, `/people/1/name` and so on, up to the count `/number` gave:

ISS.ASM — THE PEOPLE IN SPACE

```
astros:
        ld      hl,t_astros
        call    make_url
        ld      hl,url
        call    open_json
        jr      nz,astros_fail
        ld      hl,q_number             ; how many?
        call    jget
        jr      nz,astros_fail
        ld      hl,val
        call    atou8
```

```
        ld      (people),a
        ld      hl,t_ptitle
        call    print
        ld      a,(people)
        call    pdec8
        call    crlf
        call    crlf
        xor     a                       ; then each by number
astros1:
        ld      (person),a
        ld      hl,people
        cp      (hl)
        jr      nc,astros2
        ld      hl,q_name               ; /people/n/name
        call    person_q
        call    jget
        ld      hl,t_indent
        call    print
        ld      hl,val
        call    print
        ld      hl,q_craft              ; /people/n/craft
        call    person_q
        call    jget
        ld      hl,t_open
        call    print
        ld      hl,val
        call    print
        ld      a,')'
        call    conout
        call    crlf
        ld      a,(person)
        inc     a
        jr      astros1
```

FROM DEGREES TO THE MAP

The map is 72 characters wide for 360 degrees of longitude, and 18 rows high for 180 degrees of latitude: 5 degrees a column and 10 a row. ISS works in tenths of a degree, so the arithmetic is all whole numbers: column = (longitude + 180.0) / 5, row = (90.0 - latitude) / 10.

ISS.ASM — WHERE ON THE MAP

```
place:
        ld      hl,lng
        call    tenths
        ld      de,1800
        add     hl,de                   ; 0 to 3600
        ld      c,50
        call    div16_8                 ; 0 to 72
        ld      a,l
        cp      MAP_W
        jr      c,place1
        ld      a,MAP_W-1
place1:
        ld      (iss_x),a
        ld      hl,lat
        call    tenths
        ex      de,hl
        ld      hl,900
        or      a
        sbc     hl,de                   ; 0 to 1800
        ld      c,100
        call    div16_8                 ; 0 to 18
        ld      a,l
        cp      MAP_H
        jr      c,place2
        ld      a,MAP_H-1
place2:
        ld      (iss_y),a
        ret
```

FROM SECONDS TO A DATE

`/timestamp` is the time of the position, in seconds since the start of 1970 (UNIX time), about 1.76 billion: too big for 16 bits. ISS reads it as a 32-bit number, and divides by 60, 60 and 24 a byte at a time, the way long division is done on paper; what is left is days. Then it takes away whole years, then whole months, to find the date.

ISS.ASM — SECONDS TO A DATE

```
print_stamp:
        ld      hl,stamp                ; seconds, as a 32-bit number
        call    atou32
        ld      c,60
        call    div32_8
        ld      (sec),a
        call    div32_8                 ; minutes
        ld      (min),a
        ld      c,24
        call    div32_8                 ; hours; days are left
        ld      (hour),a
        ld      hl,(num32)
        ld      de,1970                 ; take away whole years
ps1:
        ld      bc,365
        ld      a,e
        and     3
        jr      nz,ps2
        inc     bc                      ; leap year (1901-2099)
ps2:
        or      a
        sbc     hl,bc
        jr      c,ps3
        inc     de
        jr      ps1
ps3:
        add     hl,bc                   ; HL = day of the year, from 0
        ld      (year),de
        ld      ix,mdays                ; then whole months
        ld      b,1
ps4:
        ld      e,(ix+0)
        ld      d,0
        ld      a,b
        cp      2
        jr      nz,ps5
        ld      a,(year)
        and     3
        jr      nz,ps5
        inc     de                      ; 29 days in a leap February
ps5:
        or      a
        sbc     hl,de
        jr      c,ps6
        inc     ix
        inc     b
        jr      ps4
ps6:
        add     hl,de
        inc     l                       ; days from 1
        push    hl
        push    bc
        ld      hl,(year)
        call    pdec16
        ld      a,'-'
        call    conout
        pop     bc
        ld      a,b
        call    p2dig
        ld      a,'-'
        call    conout
        pop     hl
        ld      a,l
        call    p2dig
        call    pspace
```

```
        ld      a,(hour)
        call    p2dig
        ld      a,':'
        call    conout
        ld      a,(min)
        call    p2dig
        ld      a,':'
        call    conout
        ld      a,(sec)
        call    p2dig
        ld      hl,t_utc
        jp      print
```

THE SCREEN

ISS draws with the H19's escape sequences: `ESC E` clears the screen, `ESC Y` row column moves the cursor (each plus 32), `ESC p` and `ESC q` turn reverse video on and off, and `ESC x 5` and `ESC y 5` hide and show the cursor. The frame round the map is in the H19's graphics characters: after `ESC F`, the letters `a`, `f`, `c`, `e`, `d` and the grave accent are a horizontal line, the four corners and a vertical line, until `ESC G` turns them back into letters. The Appendix shows them all.

THE C VERSION

In C, fujinet-lib does the JSON requests. `network_json_parse()` sends SET_PARSER and PARSE, and `network_json_query()` sends the QUERY, then reads the answer, and strips the line feed:

ISS.C — ONE VALUE, BY JSON POINTER

```
/* the value at a JSON Pointer, into val; 0 if it is not there */
static uint8_t jget(const char *pointer)
{
    int16_t n;

    memset(q, 0, sizeof q);            /* every byte sent is defined */
    strncpy(q, pointer, sizeof q - 1);
    n = network_json_query(url, q, val);
    if (n <= 0)
        return 0;
    val[n] = 0;
    return 1;
}
```

WARNING: `network_json_query()` always sends 256 bytes of query, whatever the length of the string you give it, and reads the answer into your buffer with no limit on its length. Keep the query in a 256-byte buffer that is otherwise zero, as ISS.C does, and give it a buffer of 256 bytes for the answer.

The rest is the assembly version again, more briefly; the date, for one:

ISS.C — SECONDS TO A DATE

```
/* seconds since 1 January 1970 as YYYY-MM-DD hh:mm:ss */
static void print_stamp(void)
{
    static const uint8_t mdays[12] = {31, 28, 31, 30, 31, 30,
                                      31, 31, 30, 31, 30, 31};
    unsigned long t = strtoul(stamp, NULL, 10);
    uint16_t days = t / 86400UL, year = 1970, len, rest;
    uint8_t month = 0, sec, min, hour;

    t -= days * 86400UL;               /* seconds into the day: no long % */
    hour = t / 3600UL;
    rest = (uint16_t)(t - hour * 3600UL);
    min = rest / 60;
    sec = rest % 60;

    for (;;) {                         /* whole years */
        len = (year & 3) ? 365 : 366;
        if (days < len)
            break;
        days -= len;
        year++;
    }
    for (;;) {                         /* whole months */
        len = mdays[month] + (month == 1 && !(year & 3));
        if (days < len)
            break;
        days -= len;
        month++;
    }
    sprintf(line, "%u-%02u-%02u %02u:%02u:%02u UTC", year, month + 1,
            days + 1, hour, min, sec);
    con_puts(line);
}
```

LISTINGS

SRC/ASM/ISS/ISS.ASM 864 LINES

```
;-------------------------------------------------------------------------
; ISS -- where is the International Space Station? Asks a web service for
; the station's position and the people in space, both as JSON, has the
; FujiNet's JSON parser pick out the values, and plots the station on a map
; of the world drawn on the H19, every 30 seconds until ESC is struck.
;
;   ISS                              api.open-notify.org
;   ISS N:HTTP://192.168.1.10/       a server of your own, which has
;                                    iss-now.json and astros.json
;
; One source, two systems:
;   z88dk-z80asm -b -DHDOS -Isrc/asm/inc iss.asm   ->  ISS.ABS (HDOS)
;   z88dk-z80asm -b -DCPM  -Isrc/asm/inc iss.asm   ->  ISS.COM (CP/M)
;
; After Justin Skists's ISS tracker for the RC2014, itself after Bill
; Kendrick's for the Atari. The map is theirs (map.inc).
;
; Copyright (C) 2026 Thomas Cherryhomes
; SPDX-License-Identifier: GPL-3.0-or-later
;-------------------------------------------------------------------------
INCLUDE "os.inc"
INCLUDE "fujinet.inc"

ESC             equ     0x1B
MAP_X           equ     4               ; the map's top left on the screen
MAP_Y           equ     2
VALSZ           equ     64              ; the longest value we keep
REFRESH         equ     60              ; half-seconds between updates

main:
        call    card_on
        call    card_present
        jp      nz,no_card
        call    args                    ; a server of our own?
        ld      de,base
        call    word
        jr      nz,main1
        ld      hl,t_base               ; no: open-notify
        ld      de,base
        call    strcpyd
main1:
        ld      de,base
        call    netspec
        ld      hl,t_start              ; clear the screen, cursor off
        call    print

        call    astros                  ; who is up there
        jr      c,quit
map_loop:
        call    fetch_iss               ; where is the station?
        call    draw
        call    wait_blink              ; 30 seconds, or a key
        jr      nc,map_loop
quit:
        ld      hl,t_stop               ; clear the screen, cursor on
        jp      print

no_card:
        ld      hl,t_nocard
        jp      print

;------------------------------------------------------------ the JSON part
; open_json: HL -> a devicespec. Opens it on unit 1 (HTTP GET), sets the
; channel's parser to JSON and parses what comes. NZ if any step failed.
open_json:
        push    hl
        call    strlen
        pop     hl
        ld      (rq_open+RQ_PAY),hl
        ld      (rq_open+RQ_PLEN),bc
        ld      hl,rq_open              ; open: GET the document
        call    fn_call
        ret     nz
        ld      hl,rq_parser            ; the channel's parser: JSON
        call    fn_call
        ret     nz
        ld      hl,rq_parse             ; read and parse the whole reply
        jp      fn_call

; jget: HL -> a JSON Pointer, such as "/iss_position/latitude". Asks the
; parser for it and reads the value into val, NUL-terminated, without the
; line feed the FujiNet puts after it. NZ if it is not there.
jget:
        push    hl
        call    strlen                  ; the query is the whole payload
        pop     hl
        ld      (rq_query+RQ_PAY),hl
        ld      (rq_query+RQ_PLEN),bc
        ld      hl,rq_query
        call    fn_call
        ret     nz
        ld      hl,rq_status            ; the value is waiting to be read
        call    fn_call
        ret     nz
        ld      hl,(nstat+NS_AVAIL)
        ld      a,h
        or      l
        jr      z,jget_none             ; not found: nothing waits
        ld      de,VALSZ-1
        or      a
        sbc     hl,de
        add     hl,de
        jr      c,jget1
        ex      de,hl
jget1:
        ld      (rq_read+RQ_AUX1),hl
        ld      hl,rq_read
        call    fn_call
        ret     nz
        ld      hl,val                  ; end it at BC, then drop the
        add     hl,bc                   ;   line feed (and any CR)
        ld      (hl),0
jget2:
        ld      a,b
        or      c
        jr      z,jget3
        dec     hl
        ld      a,(hl)
        cp      ' '
        jr      nc,jget3
        ld      (hl),0
        dec     bc
        jr      jget2
jget3:
        xor     a                       ; Z: found
        ret
jget_none:
        inc     a                       ; NZ
        ret

close_ch:
        ld      hl,rq_close
        jp      fn_call

;------------------------------------------------------------ the people
; astros: list the people in space, then wait for a key. Carry if ESC.
astros:
        ld      hl,t_astros
        call    make_url
        ld      hl,url
        call    open_json
        jr      nz,astros_fail
        ld      hl,q_number             ; how many?
        call    jget
        jr      nz,astros_fail
        ld      hl,val
        call    atou8
        ld      (people),a
        ld      hl,t_ptitle
        call    print
        ld      a,(people)
        call    pdec8
        call    crlf
        call    crlf
        xor     a                       ; then each by number
astros1:
        ld      (person),a
        ld      hl,people
        cp      (hl)
        jr      nc,astros2
        ld      hl,q_name               ; /people/n/name
        call    person_q
        call    jget
        ld      hl,t_indent
        call    print
        ld      hl,val
        call    print
        ld      hl,q_craft              ; /people/n/craft
        call    person_q
        call    jget
        ld      hl,t_open
        call    print
        ld      hl,val
        call    print
        ld      a,')'
        call    conout
```

```
        call    crlf
        ld      a,(person)
        inc     a
        jr      astros1
astros2:
        call    close_ch
        ld      hl,t_anykey
        call    print
        ld      bc,REFRESH              ; wait up to 30 s for a key
astros3:
        push    bc
        call    half_sec
        pop     bc
        ret     c                       ; ESC
        ret     z                       ; another key
        dec     bc
        ld      a,b
        or      c
        jr      nz,astros3
        ret
astros_fail:
        call    close_ch
        ld      hl,t_nopeople
        call    print
        jr      astros2

; person_q: HL -> "/name" or "/craft". Builds "/people/<person><that>" in q
; and leaves HL -> q.
person_q:
        push    hl
        ld      hl,q_people
        ld      de,q
        call    strcpyd
        ld      a,(person)
        call    utoa8
        pop     hl
        call    strcpyd
        ld      hl,q
        ret

;------------------------------------------------------------- the station
; fetch_iss: the latitude, longitude and time of the station's last fix
; into lat, lng and stamp. Clears ok if anything failed.
fetch_iss:
        xor     a
        ld      (ok),a
        ld      hl,t_issnow
        call    make_url
        ld      hl,url
        call    open_json
        jr      nz,fetch_end
        ld      hl,q_lat
        call    jget
        jr      nz,fetch_end
        ld      hl,val
        ld      de,lat
        call    strcpyd
        ld      hl,q_lng
        call    jget
        jr      nz,fetch_end
        ld      hl,val
        ld      de,lng
        call    strcpyd
        ld      hl,q_time
        call    jget
        jr      nz,fetch_end
        ld      hl,val
        ld      de,stamp
        call    strcpyd
        ld      a,1
        ld      (ok),a
fetch_end:
        jp      close_ch

; make_url: HL -> a file name. url = base + it.
make_url:
        push    hl
        ld      hl,base
        ld      de,url
        call    strcpyd
        pop     hl
        jp      strcpyd

;------------------------------------------------------------- the screen
; draw: the frame and the map, the station's place on it, and the figures
; under it
draw:
        ld      hl,t_frame              ; clear; the title; graphics on
        call    print
        ld      bc,(MAP_Y-1)*256+MAP_X-1
        call    gotoxy
        ld      a,'f'                   ; the top: corner, line, corner
        call    conout
        call    hline
        ld      a,'c'
        call    conout
        ld      hl,map_rows             ; each row, between two sides
        ld      b,MAP_Y
draw1:
        push    bc
        ld      c,MAP_X-1
        call    gotoxy
        ld      a,'`'                   ; side (graphics on)
        call    conout
        ld      a,ESC                   ; graphics off for the map
        call    conout
        ld      a,'G'
        call    conout
        ld      b,MAP_W
draw2:
        ld      a,(hl)
        call    conout
        inc     hl
        djnz    draw2
        ld      a,ESC
        call    conout
        ld      a,'F'
        call    conout
        ld      a,'`'
        call    conout
        pop     bc
        inc     b
        ld      a,b
        cp      MAP_Y+MAP_H
        jr      nz,draw1
        ld      c,MAP_X-1               ; the bottom
        call    gotoxy
        ld      a,'e'
        call    conout
        call    hline
        ld      a,'d'
        call    conout
        ld      a,ESC
        call    conout
        ld      a,'G'                   ; graphics off
        call    conout
        ld      a,(ok)
        or      a
        jr      z,draw_fail
        call    place                   ; where on the map
        ld      bc,(MAP_Y+MAP_H+2)*256+MAP_X-1
        call    gotoxy
        ld      hl,t_time               ; the figures
        call    print
        call    print_stamp
        ld      hl,t_lat
        call    print
        ld      hl,lat
        call    print
        ld      hl,t_lng
        call    print
        ld      hl,lng
        call    print
        ld      bc,(MAP_Y+MAP_H+3)*256+MAP_X-1
        call    gotoxy
        ld      hl,t_esc
        jp      print
draw_fail:
        ld      bc,(MAP_Y+MAP_H+2)*256+MAP_X-1
        call    gotoxy
        ld      hl,t_noiss
        jp      print

hline:
        ld      b,MAP_W
hline1:
        ld      a,'a'
        call    conout
        djnz    hline1
        ret

; gotoxy: B = row, C = column, from 0. The H19's ESC Y, row + 32, col + 32.
gotoxy:
        ld      a,ESC
        call    conout
        ld      a,'Y'
        call    conout
        ld      a,b
        add     a,32
        call    conout
        ld      a,c
        add     a,32
        jp      conout

; place: the station's column and row on the map, from lat and lng.
; 72 columns for 360 degrees, 18 rows for 180: x = (lng + 180) / 5,
```

```
; y = (90 - lat) / 10, worked in tenths of a degree.
place:
        ld      hl,lng
        call    tenths
        ld      de,1800
        add     hl,de                   ; 0 to 3600
        ld      c,50
        call    div16_8                 ; 0 to 72
        ld      a,l
        cp      MAP_W
        jr      c,place1
        ld      a,MAP_W-1
place1:
        ld      (iss_x),a
        ld      hl,lat
        call    tenths
        ex      de,hl
        ld      hl,900
        or      a
        sbc     hl,de                   ; 0 to 1800
        ld      c,100
        call    div16_8                 ; 0 to 18
        ld      a,l
        cp      MAP_H
        jr      c,place2
        ld      a,MAP_H-1
place2:
        ld      (iss_y),a
        ret

; mark: A = 1 to show the station, 0 to put the map back
mark:
        push    af
        ld      a,(iss_y)
        add     a,MAP_Y
        ld      b,a
        ld      a,(iss_x)
        add     a,MAP_X
        ld      c,a
        call    gotoxy
        pop     af
        or      a
        jr      z,mark_off
        ld      hl,t_iss                ; reverse video *
        jp      print
mark_off:
        ld      a,(iss_y)               ; the map's own character
        ld      l,a
        ld      h,0
        add     hl,hl                   ; y * 72 = y * 8 + y * 64
        add     hl,hl
        add     hl,hl
        ld      d,h
        ld      e,l
        add     hl,hl
        add     hl,hl
        add     hl,hl
        add     hl,de
        ld      a,(iss_x)
        ld      e,a
        ld      d,0
        add     hl,de
        ld      de,map_rows
        add     hl,de
        ld      a,(hl)
        jp      conout

; wait_blink: blink the station for 30 seconds. Carry if ESC was struck;
; returns early if another key was.
wait_blink:
        ld      b,REFRESH
wb1:
        push    bc
        ld      a,(ok)
        or      a
        jr      z,wb2
        ld      a,b                     ; on, off, on ... from the start
        and     1
        xor     1
        call    mark
        ld      bc,0                    ; the cursor out of the way
        call    gotoxy
wb2:
        call    half_sec
        pop     bc
        ret     c                       ; ESC
        ret     z                       ; another key: update now
        djnz    wb1
        or      a
        ret

; half_sec: wait 0.5 s for a key. Carry if ESC, Z if another key, else NZ.
half_sec:
        ld      hl,(TICKS)
        ld      (t0),hl
hs1:
        call    conin
        jr      nc,hs2
        ld      hl,(TICKS)
        ld      de,(t0)
        or      a
        sbc     hl,de
        ld      de,250
        or      a
        sbc     hl,de
        jr      c,hs1
        or      1                       ; NZ, no carry: time is up
        ret
hs2:
        cp      ESC
        scf
        ret     z
        xor     a                       ; Z, no carry
        ret

;------------------------------------------------------------ the time
; print_stamp: stamp (seconds since 1 January 1970) as YYYY-MM-DD hh:mm:ss
print_stamp:
        ld      hl,stamp                ; seconds, as a 32-bit number
        call    atou32
        ld      c,60
        call    div32_8
        ld      (sec),a
        call    div32_8                 ; minutes
        ld      (min),a
        ld      c,24
        call    div32_8                 ; hours; days are left
        ld      (hour),a
        ld      hl,(num32)
        ld      de,1970                 ; take away whole years
ps1:
        ld      bc,365
        ld      a,e
        and     3
        jr      nz,ps2
        inc     bc                      ; leap year (1901-2099)
ps2:
        or      a
        sbc     hl,bc
        jr      c,ps3
        inc     de
        jr      ps1
ps3:
        add     hl,bc                   ; HL = day of the year, from 0
        ld      (year),de
        ld      ix,mdays                ; then whole months
        ld      b,1
ps4:
        ld      e,(ix+0)
        ld      d,0
        ld      a,b
        cp      2
        jr      nz,ps5
        ld      a,(year)
        and     3
        jr      nz,ps5
        inc     de                      ; 29 days in a leap February
ps5:
        or      a
        sbc     hl,de
        jr      c,ps6
        inc     ix
        inc     b
        jr      ps4
ps6:
        add     hl,de
        inc     l                       ; days from 1
        push    hl
        push    bc
        ld      hl,(year)
        call    pdec16
        ld      a,'-'
        call    conout
        pop     bc
        ld      a,b
        call    p2dig
        ld      a,'-'
        call    conout
        pop     hl
        ld      a,l
        call    p2dig
        call    pspace
        ld      a,(hour)
        call    p2dig
        ld      a,':'
        call    conout
```

```
549         ld      a,(min)
550         call    p2dig
551         ld      a,':'
552         call    conout
553         ld      a,(sec)
554         call    p2dig
555         ld      hl,t_utc
556         jp      print
557 mdays:          defb    31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31
558
559 ;---------------------------------------------------------------- arithmetic
560 ; div16_8: HL / C -> HL quotient, A remainder (C from 1 to 127)
561 div16_8:
562         xor     a
563         ld      b,16
564 d168a:
565         add     hl,hl
566         rla
567         cp      c
568         jr      c,d168b
569         sub     c
570         inc     l
571 d168b:
572         djnz    d168a
573         ret
574
575 ; div32_8: num32 / C -> num32 quotient, A remainder. A byte at a time,
576 ; from the top, the remainder carried down as in long division.
577 div32_8:
578         ld      hl,num32+3
579         ld      b,4
580         xor     a                       ; nothing carried in yet
581 d328:
582         push    bc
583         push    hl
584         ld      l,(hl)                  ; remainder * 256 + this byte
585         ld      h,a
586         call    div16_8                 ; the quotient fits in L
587         ld      e,l
588         pop     hl
589         ld      (hl),e
590         dec     hl
591         pop     bc
592         djnz    d328
593         ret
594
595 ; atou32: HL -> decimal digits. num32 = their value.
596 atou32:
597         xor     a
598         ld      (num32),a
599         ld      (num32+1),a
600         ld      (num32+2),a
601         ld      (num32+3),a
602 atou32a:
603         ld      a,(hl)
604         sub     '0'
605         ret     c
606         cp      10
607         ret     nc
608         push    hl
609         ld      b,4                     ; num32 = num32 * 10 + digit
610         ld      hl,num32
611         ld      c,a
612 atou32b:
613         ld      e,(hl)                  ; this byte * 10 + carry in C
614         ld      d,0
615         push    hl
616         ld      h,d
617         ld      l,e
618         add     hl,hl
619         add     hl,hl
620         add     hl,de
621         add     hl,hl
622         ld      e,c
623         add     hl,de
624         ld      c,h                     ; carry out
625         ld      a,l
626         pop     hl
627         ld      (hl),a
628         inc     hl
629         djnz    atou32b
630         pop     hl
631         inc     hl
632         jr      atou32a
633
634 ; atou8: HL -> decimal digits. A = their value (to 255).
635 atou8:
636         ld      b,0
637 atou8a:
638         ld      a,(hl)
639         sub     '0'
640         jr      c,atou8b
641         cp      10
642         jr      nc,atou8b
643         ld      c,a
644         ld      a,b
645         add     a,a
646         add     a,a
647         add     a,b
648         add     a,a
649         add     a,c
650         ld      b,a
651         inc     hl
652         jr      atou8a
653 atou8b:
654         ld      a,b
655         ret
656
657 ; tenths: HL -> a number like "-95.3698". HL = it in tenths (-953).
658 tenths:
659         ld      a,(hl)
660         cp      '-'
661         push    af                      ; Z: negative
662         jr      nz,ten1
663         inc     hl
664 ten1:
665         ld      de,0
666 ten2:
667         ld      a,(hl)
668         cp      '.'
669         jr      z,ten3
670         sub     '0'
671         jr      c,ten4
672         cp      10
673         jr      nc,ten4
674         call    de10a
675         inc     hl
676         jr      ten2
677 ten3:
678         inc     hl                      ; the first decimal
679         ld      a,(hl)
680         sub     '0'
681         jr      c,ten4
682         cp      10
683         jr      c,ten5
684 ten4:
685         xor     a                       ; no decimals: times 10
686 ten5:
687         call    de10a
688         ex      de,hl
689         pop     af
690         ret     nz
691         xor     a                       ; negative: HL = -HL
692         sub     l
693         ld      l,a
694         sbc     a,a
695         sub     h
696         ld      h,a
697         ret
698
699 ; de10a: DE = DE * 10 + A
700 de10a:
701         push    hl
702         ld      h,d
703         ld      l,e
704         add     hl,hl
705         add     hl,hl
706         add     hl,de
707         add     hl,hl
708         ld      e,a
709         ld      d,0
710         add     hl,de
711         ex      de,hl
712         pop     hl
713         ret
714
715 ; utoa8: A (0-99) as decimal at DE, NUL-terminated; DE left at the NUL
716 utoa8:
717         ld      b,'0'-1
718 utoa8a:
719         inc     b
720         sub     10
721         jr      nc,utoa8a
722         add     a,10+'0'
723         ld      c,a
724         ld      a,b
725         cp      '0'
726         jr      z,utoa8b
727         ld      (de),a
728         inc     de
729 utoa8b:
730         ld      a,c
731         ld      (de),a
732         inc     de
733         xor     a
734         ld      (de),a
```

```
        ret

; strcpyd: copy the string at HL to DE, with its NUL; DE left at the NUL
strcpyd:
        ld      a,(hl)
        ld      (de),a
        or      a
        ret     z
        inc     hl
        inc     de
        jr      strcpyd

; p2dig: A (0-99) as two digits
p2dig:
        cp      10
        jr      nc,p2dig1
        push    af
        ld      a,'0'
        call    conout
        pop     af
p2dig1:
        jp      pdec8

;---------------------------------------------------------------- data
rq_open:                                ; mode 4: HTTP GET
        FNREQ   DEV_NETWORK, NET_OPEN, F_A1_A2, 4,0,0,0
        FNBUF   0, 0, 0, 0
rq_parser:                              ; the second field: 1 = JSON
        FNREQ   DEV_NETWORK, NET_SET_PARSER, F_A1_A2, 0,1,0,0
        FNBUF   0, 0, 0, 0
rq_parse:
        FNREQ   DEV_NETWORK, NET_PARSE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
rq_query:                               ; payload: the JSON Pointer
        FNREQ   DEV_NETWORK, NET_QUERY, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0
rq_status:
        FNREQ   DEV_NETWORK, NET_STATUS, F_NONE, 0,0,0,0
        FNBUF   0, 0, nstat, 4
rq_read:
        FNREQ   DEV_NETWORK, NET_READ, F_B12, 0,0,0,0
        FNBUF   0, 0, val, VALSZ-1
rq_close:
        FNREQ   DEV_NETWORK, NET_CLOSE, F_NONE, 0,0,0,0
        FNBUF   0, 0, 0, 0

t_base:         defm    "N:HTTP://api.open-notify.org/"
                defb    0
t_issnow:       defm    "iss-now.json"
                defb    0
t_astros:       defm    "astros.json"
                defb    0
q_number:       defm    "/number"
                defb    0
q_people:       defm    "/people/"
                defb    0
q_name:         defm    "/name"
                defb    0
q_craft:        defm    "/craft"
                defb    0
q_lat:          defm    "/iss_position/latitude"
                defb    0
q_lng:          defm    "/iss_position/longitude"
                defb    0
q_time:         defm    "/timestamp"
                defb    0

t_start:        defb    ESC, 'E', ESC, 'x', '5', 0
t_stop:         defb    ESC, 'G', ESC, 'q', ESC, 'F', ESC, 'y', '5', 0
t_frame:        defb    ESC, 'E', ESC, 'Y', 32, 32+MAP_X+22, ESC, 'p'
                defm    " INTERNATIONAL SPACE STATION "
                defb    ESC, 'q', ESC, 'F', 0
t_iss:          defb    ESC, 'p', '*', ESC, 'q', 0
t_ptitle:       defb    ESC, 'E', ESC, 'p'
                defm    " PEOPLE IN SPACE "
                defb    ESC, 'q', 13, 10, 13, 10
                defm    "There are "
                defb    0
t_indent:       defm    "    "
                defb    0
t_open:         defm    "  ("
                defb    0
t_anykey:       defb    13, 10
                defm    "RETURN shows the map; ESC quits."
                defb    0
t_nopeople:     defb    ESC, 'E'
                defm    "Could not read the list of people in space."
                defb    13, 10, 0
t_time:         defm    "Time "
                defb    0
t_utc:          defm    " UTC"
                defb    0
t_lat:          defm    "   Lat "
                defb    0
t_lng:          defm    "   Long "
                defb    0
t_esc:          defm    "Updates every 30 seconds.  RETURN updates now; ESC quits."
                defb    0
t_noiss:        defm    "Could not read the station's position; trying again."
                defb    0
t_nocard:       defm    "No FujiNet card at 0D0H."
                defb    13, 10, 0

INCLUDE "map.inc"

ok:             defb    0               ; 1 = lat, lng and stamp are good
people:         defb    0
person:         defb    0
iss_x:          defb    0
iss_y:          defb    0
sec:            defb    0
min:            defb    0
hour:           defb    0
year:           defw    0
t0:             defw    0
num32:          defs    4
nstat:          defs    4
val:            defs    VALSZ
lat:            defs    VALSZ
lng:            defs    VALSZ
stamp:          defs    VALSZ
q:              defs    32
base:           defs    132
url:            defs    148

INCLUDE "words.inc"
INCLUDE "card.inc"
INCLUDE "fujibus.inc"
INCLUDE "con.inc"
INCLUDE "tail.inc"
```

SRC/C/ISS/ISS.C 286 LINES

```
/*
 * ISS -- where is the International Space Station? In C. Asks a web
 * service for the station's position and the people in space, both as
 * JSON, has the FujiNet's JSON parser pick out the values, and plots the
 * station on a map of the world drawn on the H19, every 30 seconds until
 * ESC is struck. ISS.ASM's twin.
 *
 *   ISS                        api.open-notify.org
 *   ISS N:HTTP://192.168.1.10/ a server of your own, with iss-now.json
 *                              and astros.json
 *
 *   tools/cc.sh hdos ISS src/c/iss/iss.c src/c/common/h89con.c \
 *       src/c/common/hdosargs.c src/c/common/words.c    ->  ISS.ABS
 *   tools/cc.sh cpm ISS ...the same...                  ->  ISS.COM
 *
 * After Justin Skists's ISS tracker for the RC2014, itself after Bill
 * Kendrick's for the Atari. The map is theirs (map.h).
 *
 * Copyright (C) 2026 Thomas Cherryhomes
 * SPDX-License-Identifier: GPL-3.0-or-later
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fujinet-network.h>
#include "h89con.h"
#include "hdosargs.h"
#include "words.h"
#include "map.h"

#define ESC     0x1B
#define MAP_X   4               /* the map's top left on the screen */
#define MAP_Y   2
#define REFRESH 60              /* half-seconds between updates */

static char base[132], url[148];
static char q[MAX_JSON_QUERY_LEN];  /* fujinet-lib sends all 256 bytes */
static char val[256];               /* ...and reads with no limit */
static char lat[32], lng[32], stamp[32];
static char path[32];               /* a JSON Pointer being built */
static char line[96];
static uint8_t iss_x, iss_y, ok;

static void gotoxy(uint8_t x, uint8_t y)
{
    con_putc(ESC);
    con_putc('Y');
    con_putc(32 + y);
    con_putc(32 + x);
}

/* open url on unit 1 (HTTP GET) and parse what comes as JSON */
```

```
static uint8_t open_json(const char *name)
{
    strcpy(url, base);
    strcat(url, name);
    if (network_open(url, 4, 0) != FN_ERR_OK)
        return 0;
    return network_json_parse(url) == FN_ERR_OK;
}

/* the value at a JSON Pointer, into val; 0 if it is not there */
static uint8_t jget(const char *pointer)
{
    int16_t n;

    memset(q, 0, sizeof q);            /* every byte sent is defined */
    strncpy(q, pointer, sizeof q - 1);
    n = network_json_query(url, q, val);
    if (n <= 0)
        return 0;
    val[n] = 0;
    return 1;
}

/* wait 0.5 s for a key: 2 = ESC, 1 = another key, 0 = none */
static uint8_t half_sec(void)
{
    uint16_t t0 = TICKS;
    int c;

    while ((uint16_t)(TICKS - t0) < TICKS_PER_SECOND / 2)
        if ((c = con_getc()) >= 0)
            return c == ESC ? 2 : 1;
    return 0;
}

/* list the people in space; 1 if ESC was struck after */
static uint8_t astros(void)
{
    uint8_t n, i, k;

    con_puts("\x1B" "E" "\x1B" "p PEOPLE IN SPACE \x1B" "q\r\n\r\n");
    if (open_json("astros.json") && jget("/number")) {
        n = atoi(val);
        sprintf(line, "There are %u\r\n\r\n", n);
        con_puts(line);
        for (i = 0; i < n; i++) {
            sprintf(path, "/people/%u/name", i);
            if (jget(path)) {
                con_puts("    ");
                con_puts(val);
            }
            sprintf(path, "/people/%u/craft", i);
            if (jget(path)) {
                con_puts("  (");
                con_puts(val);
                con_putc(')');
            }
            con_puts("\r\n");
        }
    } else
        con_puts("Could not read the list of people in space.\r\n");
    network_close(url);
    con_puts("\r\nRETURN shows the map; ESC quits.");
    for (i = 0; i < REFRESH; i++)
        if ((k = half_sec()) != 0)
            return k == 2;
    return 0;
}

/* the station's latitude, longitude and time into lat, lng, stamp */
static void fetch_iss(void)
{
    ok = 0;
    if (open_json("iss-now.json")) {
        if (jget("/iss_position/latitude")) {
            strcpy(lat, val);
            if (jget("/iss_position/longitude")) {
                strcpy(lng, val);
                if (jget("/timestamp")) {
                    strcpy(stamp, val);
                    ok = 1;
                }
            }
        }
    }
    network_close(url);
}

/* a number like "-95.3698" in tenths (-953) */
static int tenths(const char *s)
{
    int v = 0, neg = (*s == '-');

    if (neg)
        s++;
    while (*s >= '0' && *s <= '9')
        v = v * 10 + (*s++ - '0');
    v *= 10;
    if (*s == '.' && s[1] >= '0' && s[1] <= '9')
        v += s[1] - '0';
    return neg ? -v : v;
}

/* the station's column and row: 72 columns for 360 degrees of
   longitude, 18 rows for 180 of latitude */
static void place(void)
{
    int x = (tenths(lng) + 1800) / 50;
    int y = (900 - tenths(lat)) / 100;

    iss_x = x < 0 ? 0 : x >= MAP_W ? MAP_W - 1 : x;
    iss_y = y < 0 ? 0 : y >= MAP_H ? MAP_H - 1 : y;
}

/* seconds since 1 January 1970 as YYYY-MM-DD hh:mm:ss */
static void print_stamp(void)
{
    static const uint8_t mdays[12] = {31, 28, 31, 30, 31, 30,
                                      31, 31, 30, 31, 30, 31};
    unsigned long t = strtoul(stamp, NULL, 10);
    uint16_t days = t / 86400UL, year = 1970, len, rest;
    uint8_t month = 0, sec, min, hour;

    t -= days * 86400UL;               /* seconds into the day: no long % */
    hour = t / 3600UL;
    rest = (uint16_t)(t - hour * 3600UL);
    min = rest / 60;
    sec = rest % 60;

    for (;;) {                         /* whole years */
        len = (year & 3) ? 365 : 366;
        if (days < len)
            break;
        days -= len;
        year++;
    }
    for (;;) {                         /* whole months */
        len = mdays[month] + (month == 1 && !(year & 3));
        if (days < len)
            break;
        days -= len;
        month++;
    }
    sprintf(line, "%u-%02u-%02u %02u:%02u:%02u UTC", year, month + 1,
            days + 1, hour, min, sec);
    con_puts(line);
}

static void hline(void)
{
    uint8_t i;

    for (i = 0; i < MAP_W; i++)
        con_putc('a');                 /* graphics: a horizontal line */
}

static void draw(void)
{
    uint8_t y;

    con_puts("\x1B" "E");
    gotoxy(MAP_X + 22, 0);
    con_puts("\x1B" "p INTERNATIONAL SPACE STATION \x1B" "q");
    con_puts("\x1B" "F");              /* graphics on for the frame */
    gotoxy(MAP_X - 1, MAP_Y - 1);
    con_putc('f');
    hline();
    con_putc('c');
    for (y = 0; y < MAP_H; y++) {
        gotoxy(MAP_X - 1, MAP_Y + y);
        con_puts("`\x1B" "G");         /* a side; graphics off */
        con_puts(map[y]);
        con_puts("\x1B" "F`");
    }
    gotoxy(MAP_X - 1, MAP_Y + MAP_H);
    con_putc('e');
    hline();
    con_putc('d');
    con_puts("\x1B" "G");
    gotoxy(MAP_X - 1, MAP_Y + MAP_H + 2);
    if (!ok) {
        con_puts("Could not read the station's position; trying again.");
        return;
    }
    place();
    con_puts("Time ");
```

```
239     print_stamp();
240     con_puts("   Lat ");
241     con_puts(lat);
242     con_puts("   Long ");
243     con_puts(lng);
244     gotoxy(MAP_X - 1, MAP_Y + MAP_H + 3);
245     con_puts("Updates every 30 seconds.  RETURN updates now; ESC quits.");
246 }
247
248 /* show the station (reverse video *), or put the map back */
249 static void mark(uint8_t on)
250 {
251     gotoxy(MAP_X + iss_x, MAP_Y + iss_y);
252     if (on)
253         con_puts("\x1B" "p*\x1B" "q");
254     else
255         con_putc(map[iss_y][iss_x]);
256     gotoxy(0, 0);
257 }
258
259 int main(int argc, char *argv[])
260 {
261     uint8_t i, k = 0;
262
263     ARGS(argc, argv);
264     strcpy(base, argc > 1 ? argv[1] : "N:HTTP://api.open-notify.org/");
265     net_spec(base);
266     con_init(1);
267     con_puts("\x1B" "x5");              /* the cursor off */
268
269     if (!astros()) {
270         for (;;) {
271             fetch_iss();
272             draw();
273             for (i = REFRESH; i; i--) {
274                 if (ok)
275                     mark(!(i & 1));             /* on first */
276                 if ((k = half_sec()) != 0)
277                     break;
278             }
279             if (k == 2)
280                 break;
281         }
282     }
283     con_puts("\x1B" "G" "\x1B" "q" "\x1B" "E" "\x1B" "y5");
284     con_exit();
285     return 0;
286 }
```

THE NETWORK GAMES

GENERAL

Three games close this Manual: **Texas Hold'em**, **Battleship** and **Fujitzee**, a dice game. Each is played at a table on a server on the Internet, against other people — on H89s, Ataris, Apples, Commodores, IBM PCs, anything with a FujiNet — and against the server's own robot players when there are not enough people. They are Eric Carr's games, written once in C for every computer the FujiNet serves, with a small part written again for each; this Manual's author wrote the H89's part.

They are the largest C programs in this Manual, about 4,000 lines each, and the following three chapters list them in full. This chapter explains what they have in common: how a game finds a table and talks to its server, how the shared code meets the H89, and how to build them.

THE GAME SYSTEM

THE SERVERS

Each game has its own server, a small Go program that keeps every table in memory: Texas Hold'em at `th.carr-designs.com`, Battleship at `battleship.carr-designs.com`, Fujitzee at `fujitzee.carr-designs.com`. A server runs the game. It deals, rolls, keeps score, moves its robot players, and advances the game every couple of seconds by the clock; the clients only show what the server says and send what the player chooses. So a game cannot be cheated from the client, and an H89 at 2 MHz keeps up with a modern PC.

THE CALLS

Everything is an HTTP GET to the server, built the same way by all three games:

```
N: + server + path + ?table=ai2&player=thom + &bin=1
```

PATH	DOES
`tables`	lists the server's tables, with how many players each has
`state`	seats the player at the table if need be, and returns the game as it stands
`move/ca`	Texas Hold'em: a move (fold, check, call, bet, raise, all-in)
`ready`	Battleship, Fujitzee: ready to start
`place/...`	Battleship: where the five ships are
`attack/n`	Battleship: fire at square n
`roll/01101`	Fujitzee: roll again the dice marked 1
`score/n`	Fujitzee: score the dice in category n
`leave`	leave the table

There is no "join" call: any request that names a table and a player seats that player. A move's reply is the new state, so it takes the place of the next poll.

BINARY REPLIES

With `bin=1`, the servers answer not in JSON but in a fixed layout of bytes, the same as the game's C structure on a little-endian computer like the Z80. The client reads the reply straight into its structure with `network_read()`, and has no parsing to do at all — which saves an 8-bit computer a great deal of time and memory. Strings come in fixed lengths, padded with zeros, and in lower case. Fujitzee checks at compile time that its structure is the size the server sends:

```
typedef char game_wire_size_check[(sizeof(Game) == 599) ? 1 : -1];
```

POLLING

No server can call the H89, so every client polls: it asks for `state` every second or two, and draws what changed. Texas Hold'em polls back to back and repaints only if a checksum of the reply has changed; Battleship and Fujitzee poll about once a second.

NAMES, AND FINDING A TABLE

On other computers a **lobby** program lists the game servers and starts the game; the H89 has no lobby program yet, so each game asks its own server for its tables. All three keep the player's name in the app key the lobby uses (creator 0001H, application 01H, key 00H, Chapter 8), and the last table in another, so a game can go back to it.

HOW THE CODE IS ARRANGED

Each game's source is a shared part, used on every computer, and a part for each computer:

```
src/*.c, *.h              the game: main loop, logic, screens, server
src/platform-specific/*.h what each computer must supply
src/h89/                  the H89's part
mekkogx/                  the shared build system
```

The interface the H89's part supplies is small: graphics (the screen, text, cards, boards, dice), input (keys), sound, a timer, and in Texas Hold'em the network and app key calls. Everything in `src/h89/` is wrapped in `#ifdef __H89__`, which both H89 builds define.

THE H19 CONSOLE

The shared code thinks in characters on a grid, which suits the H19. Each game keeps a picture in memory of what the terminal shows, one byte per cell — the H19's own character number, with the top bit for reverse video — and sends only the cells that change, since the H19 is a terminal on a 9600-baud line and can take fewer than a thousand characters a second. Texas Hold'em and Fujitzee draw into a second picture and compare whole rows before sending, which lets them clear the screen with `ESC E` when that is cheaper; Battleship compares cell by cell.

The renderer tracks what the terminal is doing — cursor position, reverse video, graphics mode — and sends `ESC Y`, `ESC p/q` and `ESC F/G` only when they must change. Graphics mode remaps only the characters 5EH to 7EH, so text below 5EH never forces a change. The cards, ships, dice and frames are drawn entirely in the H19's graphics characters (the Appendix shows them).

Under HDOS the console is put in raw character mode, with no echo and no case mapping (the graphics characters are sent as lower-case letters); under CP/M the games call the BIOS's console routines directly. The book's `h89con.c` (Chapter 6) is the same code, made smaller.

THE KEYBOARD

The H19 sends its arrow and function keys as escape sequences, back to back at 9600 baud. The games put the keypad in shifted mode (`ESC t`), so its arrows send `ESC A` to `ESC D`, and take an ESC with nothing behind it within 20 ms as the ESC key itself.

TIME AND SOUND

The shared code counts sixtieths of a second; the H89's clock counts 2 ms ticks (Chapter 2), so a "frame" is 8, 8 and 9 ticks in turn. The only sound an H89 makes is the H19's bell, rung when you sit down, when it is your turn and when a game ends.

FUJINET-LIB IN THE GAMES

The games use only a handful of fujinet-lib's functions:

- `network_open()` (mode 4 or 12), `network_read()` and `network_close()`, for every request;
- `fuji_set_appkey_details()`, `fuji_read_appkey()` and `fuji_write_appkey()`, for the name and preferences.

Battleship and Fujitzee are built with `CUSTOM_FUJINET_CALLS`, which routes these through three functions in `src/h89/fujinet.c`. Battleship needs it because its shared code includes its own copy of fujinet-lib's headers, which declare the app key calls as functions where the H89's library has macros; Fujitzee uses it to draw the screen before each call, so "refreshing" shows while the request is out.

BUILDING THE GAMES

Each game builds with its own Makefile, given the H89 platform and fujinet-lib (Chapter 6):

```
make FUJINET_LIB=$HOME/fujinet-lib PLATFORMS=h89-hdos h89-hdos
make FUJINET_LIB=$HOME/fujinet-lib PLATFORMS=h89-cpm  h89-cpm
```

The programs (`texas.abs`, `fbs.abs`, `fujitzee.abs`, and `.com` for CP/M) and 100K H17 disk images holding them are left in `r2r/h89-hdos/` and `r2r/h89-cpm/`.

The games link with no standard I/O and no file handles (they talk to the console themselves), and with `CRT_ENABLE_COMMANDLINE=0`, which also keeps clear of z88dk's HDOS command-line fault. They take z88dk's default HDOS load address, 4480H, with the stack below it: wasteful, as Chapter 6 says, but harmless.

PROGRAM	SIZE	ENDS AT	NEEDS
TEXAS.ABS	25K	AA3DH	HDOS, 48K or more (not tried in 48K)
FBS.ABS	26K	AA9BH	HDOS, 48K or more (not tried in 48K)
FUJITZEE.ABS	34K	CE46H	HDOS on a full-memory H89
TEXAS.COM	25K	669EH	CP/M
FBS.COM	26K	66FCH	CP/M
FUJITZEE.COM	34K	8AA7H	CP/M

Fujitzee is the largest because it keeps three pictures of the screen, 2,000 bytes each. Its Makefile checks that it ends below EDB0H, where HDOS's resident part begins on a full machine.

NOTE: Texas Hold'em and Fujitzee each have a smoke test, `make h89-smoke`, which boots the game in FujiNet Go H89 and checks its first screen.

NOTE: Fujitzee's H89 port was not yet committed to its repository when this Manual was made: its listing is of the working tree, as it then stood.

LICENCES

The games are free software, under the GNU General Public License, version 3: Texas Hold'em's repository carries the licence; Battleship's and Fujitzee's say so in every source file's heading.

TEXAS HOLD'EM

GENERAL

Texas Hold'em is the poker game of the card rooms: up to eight players at a table, each starting with 1000 chips, blinds of 5 and 10. Each player is dealt two cards face down; five more are dealt face up in the middle, three (the **flop**), then one (the **turn**), then one more (the **river**), with a round of betting before each and after the last. The best five-card hand from your two and the five in the middle wins the pot.

The server's tables include "AI rooms" with two, four or six robot players, so there is always a game. The program is TEXAS.ABS for HDOS and TEXAS.COM for CP/M.

PLAYING

( ) Run TEXAS. The first time, it asks for your name (2 to 8 letters), and keeps it in the lobby's app key, where every FujiNet game will find it.

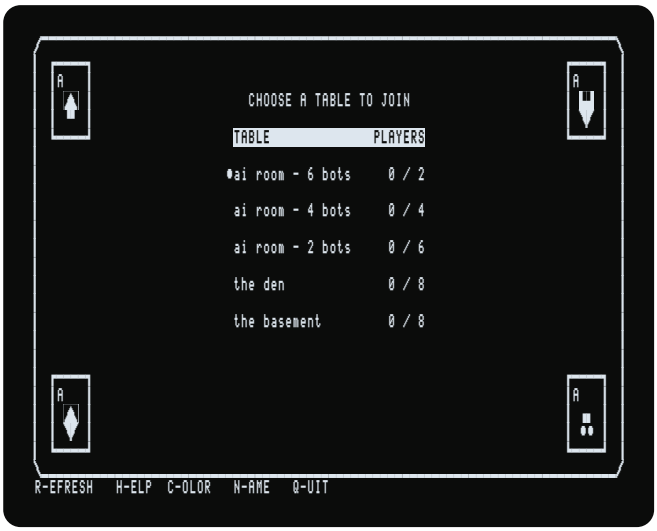
( ) Choose a table with the arrow keys (or - and =) and RETURN. The list shows how many of each table's human seats are taken; tables with robots have fewer human seats.

( ) When it is your turn, the bottom line offers your moves and a countdown. Left and right choose a move; up and down choose between the two sizes of bet or raise; RETURN sends it.

( ) ESC (or any of the H19's function keys) opens the menu: Q leaves the table, H shows the help, C changes how red cards are drawn.

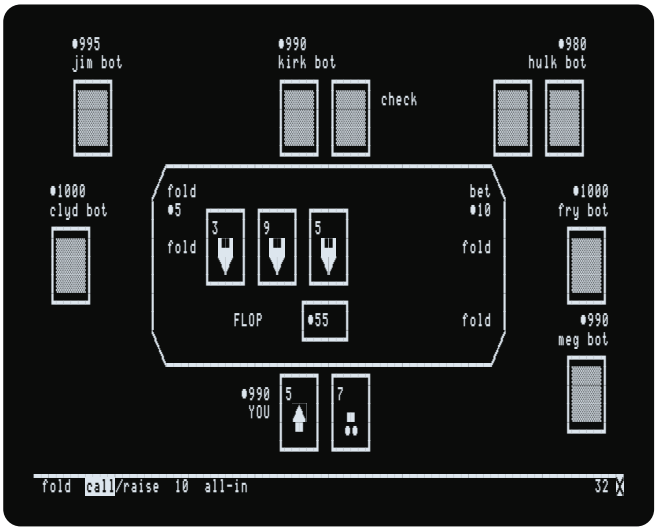


PICTORIAL 17-1
The first screen: your name, kept in the lobby's app key for every game. The suits are pictures two characters square.



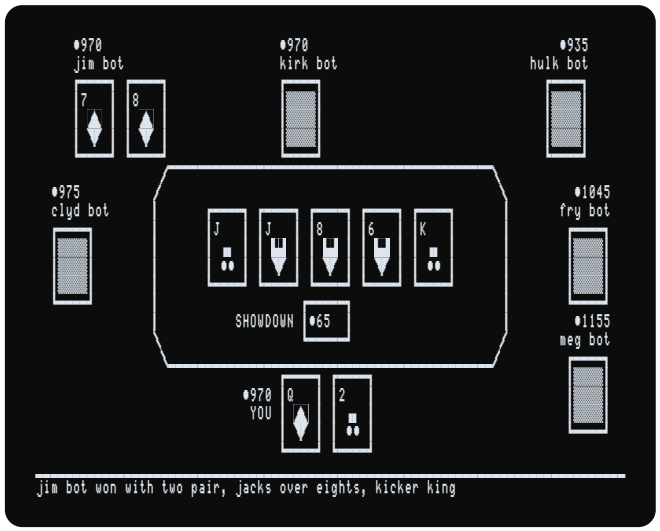
PICTORIAL 17-2

Choosing a table. The names come from the server, in lower case; a table with six robots has two human seats.



PICTORIAL 17-3

A hand at the six-robot table, on the flop, your turn: CALL is chosen, 32 seconds left on the clock at the right of the bottom line.



PICTORIAL 17-4

The showdown: every hand is turned up, and the bottom line gives the server's verdict.

HOW IT WORKS

THE STATE

Every poll is `state?table=ai6&player=thom&bin=1` (or a move, `move/ca`, in its place). The reply is the `Game` structure, read straight into memory:

OFFSET	BYTES	HOLDS
0	81	The last hand's result, as text.
81	1	The round: 0 waiting, 1 before the flop, 2 flop, 3 turn, 4 river, 5 showdown.
82	2	The pot.
84	1	Whose turn it is; players are numbered from you, so 0 is you.
85	1	Seconds left to move (only on your turn).
86	1	1 if you are only watching (the table is full).
87	11	The cards in the middle, as `askh2d` (ace of spades, king of hearts, two of diamonds).
98	66	How many moves you may make, and up to five: a code and a name each.
164	1	How many players.
165	33 each	Each player: name, status, bet, last move, purse, and hand (`????` until the showdown).

Texas Hold'em polls as fast as the server answers, and adds up the reply's bytes, each weighted by its place; it repaints only when the sum changes. The H19 would be sent nothing anyway for an unchanged table (the console layer sends only changed cells), but repainting

also redraws the highlight that sweeps to the active player, which would flicker.

DRAWING THE CARDS

The H19 has no card suits in its character set, so each card's suit is a picture two characters square, made of the graphics set's triangles and quarter blocks, under the rank. A card is six characters by five, with corners and edges from the graphics set; a face-down card is checkerboard. The H19 is monochrome, so the "colour" setting draws hearts and diamonds in reverse video instead of red. The table is an oval of diagonals; you are always at the bottom, in the middle.

The title is TEXAS HOLD'EM in letters made of quarter blocks, two dots by two per character, drawn from a table (`logo.h`) that a small Python program made from a picture.

MOVES

The server says which moves are allowed and computes every amount. The H89's bottom line has room for five stops beside the clock, so CALL and RAISE share one, with the amount beside it. When your clock runs out, the program sends the chosen CHECK or CALL itself, so the server never folds you, and it does not poll the server while it waits for you.

THE FILES

FILE	HOLDS
`src/main.c`	The globals and the main loop; the repaint check.
`src/stateclient.c`	Builds each request and reads the reply into the state.
`src/screens.c`	Help, name, welcome, table list, the game screen, the menu.
`src/gamelogic.c`	Seats, pot, betting, dealing, the move menu and its countdown.
`src/misc.c`, `misc.h`	Pauses, keys, preferences (app key BEBEH/1/0); the structures.
`src/platform-specific/*.h`	What the H89's part must supply.
`src/h89/console.c`	The H19: the screen in memory, changed cells only; the OS console.
`src/h89/graphics.c`	The table, cards, chips, logo and status line.
`src/h89/input.c`	The H19's keys, escape sequences decoded.
`src/h89/network.c`	A request through fujinet-lib.
`src/h89/appkey.c`	App keys through fujinet-lib.
`src/h89/sound.c`, `util.c`	The bell; frames, the move timer, leaving.
`src/h89/vars.c`, `vars.h`, `logo.h`	The layout, keys, and the title.

LISTING

SRC/MAIN.C — 165 LINES

```
/**
 * @brief   5-card-stud
 * @author  Eric Carr, Thomas Cherryhomes, (insert names here)
 * @license gpl v. 3
 * @verbose main
 */

#ifdef _CMOC_VERSION_
#include "coco/coco_bool.h"
#else
#include <stdlib.h>
#include <stdint.h>
```

```
#include <stdbool.h>
#include <string.h>
#endif /* _CMOC_VERSION_ */

#include "platform-specific/graphics.h"
#include "platform-specific/util.h"
#include "platform-specific/input.h"
#include "misc.h"
#include "platform-specific/network.h"
#include "platform-specific/appkey.h"
#include "platform-specific/sound.h"

#include "stateclient.h"
#include "gamelogic.h"
#include "screens.h"

// Store default server endpoint in case lobby did not set app key
char serverEndpoint[50] = "https://th.carr-designs.com/";
//char serverEndpoint[50] = "http://127.0.0.1:8080/"; // local dev; "N: for apple, but not C64"

// Empty at boot so the table selection screen is always shown; the room list
// comes from the server's /tables endpoint
char query[QUERY_LEN] = "";
char playerName[12] = "";

//GameState state;
// On the ColecoVision this is a macro for the cartridge's reply window, not an
// object -- see src/misc.h.
#ifndef BUILD_COLECO
ClientState clientState;
#endif


// State helper vars
#ifdef __WATCOMC__
int inputKey;
#else
char inputKey;
#endif
unsigned char prevPlayerCount, prevRound, currentCard, cardIndex, cursorX, cursorY, waitCount,
    wasViewing;
signed char inputDirX, inputDirY;
uint16_t prevPot, maxJifs;
bool noAnim, doAnim, finalFlip, inputTrigger;

unsigned char playerX[8], playerY[8], moveLoc[5];
signed char playerBetX[8], playerBetY[8], playerDir[8];

// Common local scope temp variables
unsigned char h, i, j, k, x, y, xx;
char tempBuffer[TEMP_BUFFER_LEN];

char prefs[4];

char *hand, *requestedMove;


#ifdef _CMOC_VERSION_
int main(void)
#else
void main(void)
#endif /* _CMOC_VERSION_ */
{
    // uint8_t x, result;
    // x=0;
    // x=x+2;

  //unsigned char i;while (1) {if (kbhit()) {inputKey = cgetc(); printf("\nKEY: %u",
      inputKey);}for(i=0;i<10;i++){pause(2);printf(".");}}
  loadPrefs();
  initGraphics();
  initSound();

// Checking MSX C compiler bug
//    x=10;
//    result=(x)5)+10;
//    itoa(result,tempBuffer,10);

//    cputs(tempBuffer);
//    cgetc();

#ifdef USE_PLATFORM_SPECIFIC_INPUT
  initPlatformKeyboardInput();
#endif

  showWelcomeScreen();
  showTableSelectionScreen();

  // Main in-game loop
  while (true) {

    // Get latest state and draw on screen, then prompt player for move if their turn
    if (getStateFromServer()) {
#if defined(BUILD_COLECO) || defined(BUILD_NES) || defined(BUILD_SMS) || defined(__H89__)
      /*
        The ColecoVision has no back buffer (1K of RAM), so showGameScreen()
        repaints the table straight to VRAM. Every server poll would otherwise
        redraw the whole table even when nothing changed -- invisible on a real
        display (identical bytes rewritten in place) but a waste, and it makes a
        redraw always be in flight, which tears on capture. Skip the repaint
        when the fetched state is byte-for-byte what is already on screen.

        state is the reply window here (see src/misc.h), valid until the next
        fuji_* call -- which is exactly now, right after the fetch. A one-word
        sum is enough: a rare collision just defers a repaint to the next
        differing poll. requestPlayerMove() still runs, so our own turn (where
        the countdown does change the state every second) always draws.

        The NES draws through cc65's vblank ring buffer, a few dozen tiles a
        frame, so an unchanged repaint there costs a backlog the next real
        change has to queue behind. Its state is an ordinary object in WRAM.
        So is the Master System's; its VDP takes a table's worth of writes
        easily, but there is no point shipping an unchanged one every poll.

        The H89's H19 terminal is on a 9600 baud line. Its renderer sends only
        the cells that differ, so an unchanged repaint costs no bytes, but it
        still re-sweeps the active player's highlight; skip it.
      */
      {
        static unsigned int lastDrawnSum = 1; // != any first real sum
        unsigned int sum = 0;
        unsigned int n;                       // Game is 429 bytes: not a char
        unsigned char *sp = &clientState.firstByte;
        for (n = 0; n < sizeof(Game); n++)
          sum += sp[n] ^ (unsigned char)n;    // position-weighted vs byte swaps
        if (sum != lastDrawnSum) {
          lastDrawnSum = sum;
          showGameScreen();
        }
      }
#else
      showGameScreen();
#endif
      requestPlayerMove();
    } else {
      // Wait a bit to avoid hammering the server if getting bad responses
      pause(30);
    }

    // Handle other key presses
    readCommonInput();

    switch(inputKey) {
      case KEY_ESCAPE: // Esc
      case KEY_ESCAPE_ALT: // Esc Alt
        showInGameMenuScreen();
        break;
    }


  }
#ifdef _CMOC_VERSION_
  return 0;
#endif
}
```

SRC/GAMELOGIC.C — 875 LINES

```
#ifdef _CMOC_VERSION_
#include <cmoc.h>
#include <coco.h>
//#define true 1
//#define false 0
//typedef BOOL bool;
#else
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#endif /* _CMOC_VERSION_ */

#include "platform-specific/graphics.h"
#include "platform-specific/sound.h"
#include "platform-specific/util.h"
#include "gamelogic.h"
#include "misc.h"
#include "stateclient.h"
#include "screens.h"
#include "platform-specific/appkey.h"

extern unsigned char redrawGameScreen;
#ifndef POT_Y_MODIFIER
#define POT_Y_MODIFIER 0
#endif

#ifndef STATUS_TIMER_WIDTH
```

```
#define STATUS_TIMER_WIDTH 2
#endif

#ifndef PLAYER_MOVE_START_X
#define PLAYER_MOVE_START_X 1
#endif

#ifndef LEFT_JUSTIFY_PLAYER_PURSE
#define LEFT_JUSTIFY_PLAYER_PURSE 0
#endif

/* 32 columns only fit four, clipping RAISE/CHECK to RAIS/CHEC. */
#ifndef MOVE_FIELD_W
#if WIDTH >= 40
#define MOVE_FIELD_W 5
#else
#define MOVE_FIELD_W 4
#endif
#endif

// Texas Hold'em community card board position (5 cards, 2 columns each)
#ifndef COMMUNITY_X
#define COMMUNITY_X (WIDTH/2-5)
#endif
#ifndef COMMUNITY_Y
#define COMMUNITY_Y 9
#endif

// Card pitch: on the tile platforms a card is 3 cells wide and the next one
// overlaps its right border. The H89's cards are 6 wide with a gap between.
#ifndef HOLE_CARD_STEP
#define HOLE_CARD_STEP 2
#endif
#ifndef COMMUNITY_CARD_STEP
#define COMMUNITY_CARD_STEP 2
#endif

// Where a right-side seat's name and purse end, counted from its card's
// left column: the second column of a 3-wide card, or the H89's right edge.
#ifndef NAME_RIGHT_X
#define NAME_RIGHT_X 1
#endif

void progressAnim(unsigned char y) {
  for(i=0;i<3;++i) {
    pause(10);
    drawChip(WIDTH/2-2+i*2,y);
    drawBuffer();
  }
}

void drawPot() {

  if (redrawGameScreen) {
    /* One wider than 5 Card Stud's: its interior held only three digits. */
    drawBox(WIDTH/2-4,11+POT_Y_MODIFIER,5,1);
    drawChip(WIDTH/2-3,12+POT_Y_MODIFIER);
  }
  itoa(state.pot, tempBuffer, 10);
  /* padded, so a shrinking pot leaves no stale digit */
  for (k=(unsigned char)strlen(tempBuffer); k<4; k++)
    tempBuffer[k]=' ';
  tempBuffer[4]=0;
  drawText(WIDTH/2-2,12+POT_Y_MODIFIER, tempBuffer);
}

// Texas Hold'em street indicator, drawn on the pot row just left of the pot
// box - the one spot that is clear of player names/purses on every platform.
// Labels are padded to a fixed width so each one fully overwrites the previous.
#if WIDTH >= 40
#define STREET_X (WIDTH/2-13)
static const char* streetNames[6] = {
  "        ", "PRE-FLOP ", "FLOP    ", "TURN    ", "RIVER   ", "SHOWDOWN "
};
#else
#define STREET_X 0
/* 32 columns: an eighth player puts a status field at columns 6-9, so the
   wording is cut to five there and only there. One table holds both forms -
   rows 0-5 spelled out, rows 6-11 cut - to save a second static on the
   tightest build. */
static const char streetNames[12][10] = {
  "        ", "PRE-FLOP ", "FLOP    ", "TURN    ", "RIVER   ", "SHOWDOWN ",
  "    ",     "PRE  ",     "FLOP ",     "TURN ",     "RIVER",     "SHOW "
};
#endif

void drawStreetLabel() {
  if (state.round > 5 || state.playerCount < 2)
    return;
#if defined(BUILD_COLECO) || defined(BUILD_NES) || defined(BUILD_SMS)
  // The shared spot sits on the left seat's cards on a 32-column table, and
  // the pot now lives below the board. Row 8 is the clear gap between the top
  // seats (cards end row 7) and the board (row 9).
  drawText(WIDTH/2-4, 8, streetNames[state.round]);
#elif WIDTH >= 40
  drawText(STREET_X, 12+POT_Y_MODIFIER, streetNames[state.round]);
#else
  drawText(STREET_X, 12+POT_Y_MODIFIER,
           streetNames[state.round + (state.playerCount > 7 ? 6 : 0)]);
#endif
}

void resetStateIfNewGame() {
  if (state.round >= prevRound)
    return;

  // Reset status bar and vars for a new game
  if (prevRound != 99) {

   // @SetStatusBarHeight 1
   clearStatusBar();
  } else {

    // Force empty screen if coming from another screen.
    // This is mainly to avoid color glitches on c64
    // when setting the color memory - which is not (YET?) double buffered -
    // before the screen is drawn. There may be a better solution.
    drawBuffer();
  }

  currentCard=0;
  cardIndex=0;
  cursorY=246;
  cursorX=128;
  prevPot=0;
  if (!redrawGameScreen) {
    redrawGameScreen=1;
    resetScreen();
  }

  // If the round is already past 1, we are joining a game in progress. Skip animation this
  +     update
  if (state.round>1)
    noAnim=1;
}


void drawNamePurse() {

  for (i=0;i<(state.playerCount);i++) {
    // Print name, left or right justified based on direction
    y = playerY[i]-1;
    x = playerX[i];
    if (playerDir[i]<0)
      x+=NAME_RIGHT_X;

    if (i>0 || state.viewing) {
      xx=x;

      // Reverse print right side players
      if (playerDir[i]<0)
        xx-=(unsigned char)strlen(state.players[i].name)-1;

      drawText(xx, y, state.players[i].name);

      if (state.activePlayer!=i) {
        hideLine(xx, y+1, (unsigned char)strlen(state.players[i].name));
      } else {
        cursorX=xx;
        cursorY=y;
      }
    } else {
      // Draw YOU player name
      #if WIDTH>=40
        drawText(x-5, playerY[i]+2, (const char *)" YOU");
      #else
        // Squeezed for horizontal space, so shift above cards
        // drawText(x+3, playerY[i]-1, (const char *)" YOU");
      #endif
    }

    // Print purse (chip count)
    x++;
    y--;

    // Override Purse position for YOU player
    if (i==0) {
      #if WIDTH>=40
        x-=2;
        y+=3;
      #else
        x+=6*(LEFT_JUSTIFY_PLAYER_PURSE==0);
        y+=1;
      #endif
    }

```

```
    itoa(state.players[i].purse, tempBuffer, 10);

    if (playerDir[i]<0 || i==LEFT_JUSTIFY_PLAYER_PURSE) {
      x-=(unsigned char)strlen(tempBuffer);
      drawText(x-2,y," "); // Cover case when chip count drops from 100 to 99
    } else {
      drawText(x+strlen(tempBuffer),y," "); // Cover case when chip count drops from 100 to 99
    }

    drawText(x,y, tempBuffer);
    drawChip(x-1,y);

  }
}


void drawBets() {
  if (state.round <1 || state.round>4)
    return;

  for (i=state.playerCount-1;i<255;i--) {
    y = playerY[i]+playerBetY[i]+1;

    // Draw bet amount
    if (state.players[i].bet>0) {
      x= playerX[i]+playerBetX[i]+1;

      itoa(state.players[i].bet, tempBuffer, 10);
      if (playerDir[i]<0)
        x-=(unsigned char)strlen(tempBuffer)+1;

      drawText(x, y, tempBuffer);
      drawChip(x-1,y );
    }

    // Fixed-width field so a shorter move fully overwrites a longer one
#if WIDTH < 40
    /* No room beside seat 7 on 32 columns, and it only exists at seven. */
    if (i==0 && state.playerCount > 6) continue;
#endif
    x= playerX[i]+playerBetX[i];
    y--;

    k=(unsigned char)strlen(state.players[i].move);
    if (k)MOVE_FIELD_W) k=MOVE_FIELD_W;
    memcpy(tempBuffer, "     ", MOVE_FIELD_W);
    tempBuffer[MOVE_FIELD_W]=0;
    if (playerDir[i]<0) {
      x-=MOVE_FIELD_W;
      memcpy(tempBuffer+MOVE_FIELD_W-k, state.players[i].move, k);
    } else {
      memcpy(tempBuffer, state.players[i].move, k);
    }

    drawText(x, y, tempBuffer);

  }
}

void checkFinalFlip() {
  if (prevRound<state.round && state.round == 5)
    drawCards(true);
}

// Texas Hold'em card rendering. Each player has 2 hole cards; up to 5 shared
// community cards sit in the middle of the table. Masking is server-side:
// opponents' hands arrive as "????" until the showdown reveals them, and a
// folded hand is just "??" (drawn as a single face-down card).
void drawCards(bool finalFlip) {
  static unsigned char cc;

  if (state.round<1)
    return;

  // Hole cards: animate the deal only when they first appear this hand
  doAnim = !noAnim && !cardIndex && !finalFlip;
  if (doAnim)
    disableDoubleBuffer();

  for (j=0;j<2;j++) {
    for (h=1;h<=state.playerCount;h++) {
      i = h % state.playerCount;
      hand = state.players[i].hand;
      if (strlen(hand)>j*2+1) {
        /* A one-card hand is a fold: the renderer wipes the empty slot. */
        drawCard(playerX[i]+(j*HOLE_CARD_STEP)*playerDir[i], playerY[i],
                 (unsigned char)(strlen(hand) ) 3 ? FULL_CARD
                                                  : (FULL_CARD | CARD_CLEAR_NEXT)),
                 hand+j*2, false);

        if (doAnim) {
          soundDealCard();
          pause(5);
        }
      }
    }
  }
  cardIndex=1;

  // Community cards: animate each newly revealed street
  cc = (unsigned char)strlen(state.community))>1;
  for (j=0;j<cc;j++) {
    doAnim = !noAnim && j>=currentCard;
    if (doAnim) {
      disableDoubleBuffer();
      pause(10);
    }

    drawCard(COMMUNITY_X+j*COMMUNITY_CARD_STEP, COMMUNITY_Y, FULL_CARD, state.community+j*2,
      +       false);

    if (doAnim)
      soundDealCard();
  }
  currentCard=cc;

  // Showdown: flip each surviving opponent's revealed hand with a beat between
  if (finalFlip) {
    drawBuffer();
    disableDoubleBuffer();

    for (h=1;h<(state.playerCount;h++) {
      // Don't flip a player that doesn't have a visible hand
      if (state.players[h].status != 1 || state.players[h].hand[0]=='?')
        continue;

      for (j=0;j<2;j++) {
        drawCard(playerX[h]+(j*HOLE_CARD_STEP)*playerDir[h], playerY[h], FULL_CARD,
          +          state.players[h].hand+j*2, false);
      }

      soundDealCard();

      pause(35);
    }
  }

  drawBuffer();
  enableDoubleBuffer();
  noAnim=false;
}

void checkIfSpectatorStatusChanged() {
  // Held in a local rather than corrected in place: on the ColecoVision the
  // state is the cartridge's reply window, which is ROM, so a store here would
  // vanish and the correction below would never take (see src/misc.h).
  unsigned char viewing = state.viewing;

  if (viewing == wasViewing)
    return;

  // Temp hack due to server not always sending correct viewing flag when not full
  if (state.playerCount<8)
    viewing=0;

  wasViewing = viewing;

  if (viewing) {
    drawStatusText("TABLE FULL: WATCHING AS A SPECTATOR");
    drawBuffer();
    pause(80);
  } else if (
    state.players[0].status == 0 ||
    (state.round == 1 && state.players[0].status == 1 && state.activePlayer != 0 )
    ) {
    /* Display intro text if player is joining the table on
     * the opening round or sitting down to wait for the next round.
     * Otherwise, they are re-joining due to connection error, so we do not delay
     */

    centerStatusText("YOU SIT DOWN AT THE TABLE");
    drawBuffer();

    soundJoinGame();

    pause(50);
  }
}

void checkIfPlayerCountChanged() {
  if (state.playerCount == prevPlayerCount)
    return;

  // Seats move below, so whatever they drew at the old ones must be cleared
  redrawGameScreen = 1;

```

```
  // Handle if player joins mid game
  if (state.playerCount)1 && prevPlayerCount ) 0) {
    if (state.playerCount ( prevPlayerCount) {

      drawStatusText("A PLAYER LEFT THE TABLE");
      drawBuffer();

      soundPlayerLeft();

      // for j=8 to 2 step -2
      //   sound 1,255-j*j,10,j:pause 2:sound:pause 8
      // next
    ) else (
      strcpy(state.lastResult, "A NEW PLAYER JOINS THE TABLE");
      drawStatusText(state.lastResult);
      drawBuffer();

      soundPlayerJoin();

    )

    pause(40);
    if (state.round ) 1)
      noAnim = true;
  )

  prevPlayerCount = state.playerCount;

  // Don't shuffle player locations until multple players exist
  if (state.playerCount(2)
    return;

  i=0;
  k=(state.playerCount-1)*8;
  for (j=(state.playerCount-2)*8;j(k;j++) {
    h=playerCountIndex[j];
    playerX[i] = playerXMaster[h];
    playerY[i] = playerYMaster[h];
    playerDir[i] = playerDirMaster[h];
    playerBetX[i] = playerBetXMaster[h];
    playerBetY[i] = playerBetYMaster[h];
    i++;
  )
)

// The move countdown, ticked down by the move-selection loop. It is a variable
// of ours rather than state.moveTime -- the server's value only seeds it --
// because on the ColecoVision the state is the cartridge's reply window, which
// is ROM (see src/misc.h): counting down in place would go nowhere, and the
// loop would never time out and auto-send the highlighted move.
unsigned char moveTimeLeft;

void drawStatusTimeLeft() {
  drawStatusTimer();
  tempBuffer[0]=' ';
  itoa(moveTimeLeft, tempBuffer+1, 10);
  drawStatusTextAt( (unsigned char)(WIDTH-strlen(tempBuffer)-STATUS_TIMER_WIDTH), tempBuffer);
  drawBuffer();
)

void highlightActivePlayer() {
 if (state.activePlayer)0 && state.playerCount)1) {
   i=(unsigned char)strlen(state.players[state.activePlayer].name);
    for (j=2;j(=i;j++) {
      drawLine(cursorX, cursorY+1, j);
      drawBuffer();
    )
  )
)

void animateChipsToPotOnRoundEnd() {
  if (state.round (= prevRound !! state.pot == 0)
    return;

  // A new street begins: request a full clean redraw of the table after the
  // chip animation. The incremental bet/move erasing below uses fixed widths
  // and can leave text fragments (varying-length moves, chip columns) that
  // used to be harmless but now sit next to the community board.
  // Not done for the showdown transition (round 5), where the final card
  // flip must stay on screen on SINGLE_BUFFER platforms. (Double-buffered
  // platforms fully redraw every frame anyway; this matters for the CoCo.)
  if (state.round ( 5)
    redrawGameScreen=1;

  clearStatusBar();
  pause(50);

  // Hide moves
  #if WIDTH)=40
  for (i=0;i(state.playerCount;i++) {
    y = playerY[i]+playerBetY[i];
    x = playerX[i]+playerBetX[i];

    if (playerDir[i](0)
        x-=5;

    drawText(x, y, "    ");
  )

  #endif

  drawBuffer();
  pause(30);

  // Don't animate bets if the pot hasn't changed
  if (prevPot == state.pot)
    return;

  prevPot = state.pot;
  disableDoubleBuffer();

  // Clear bets off screen
  for (i=0;i(state.playerCount;i++) {
    y = playerY[i]+playerBetY[i]+1;
    x= playerX[i]+playerBetX[i];

    if (playerDir[i](0)
      x-=3;

    drawText(x, y, "   ");

    soundTakeChip(i);

  )

  drawPot();
  drawBuffer();
  enableDoubleBuffer();

  pause(45);
)


void drawGameStatus() {
  if (state.activePlayer == 0)
    return;

  if (state.activePlayer)0) {
    strcpy(tempBuffer,"WAITING ON ");
    strcat(tempBuffer, state.players[state.activePlayer].name);
    drawStatusText(tempBuffer);

  ) else if (state.activePlayer( 0 && (state.round == 5 !! state.round == 0)) {
    // End of (or in between) games
    drawStatusText(state.lastResult);


    if (state.round==5 && prevRound != state.round) {
      drawBuffer();

      soundGameDone();

      pause(30);
    )
  )


  // Waiting for a player to join - show animation
  if (state.round == 0) {
    waitCount = (waitCount + 1) % 3;
    drawStatusTextAt(26+waitCount," .  ");
    pause(40);
  )

)

/* The server's five moves will not fit a 40-column status bar beside the
   countdown clock, so call and raise share one label and one amount. Folding
   them together is display only: each stop still sends its own move code. */

#define MV_OTHER 0
#define MV_FOLD  1
#define MV_CALL  2     /* call or check */
#define MV_RAISE 3     /* raise or bet  */
#define MV_ALLIN 4

static unsigned char moveLen[5];    /* underline width per cursor stop    */
static unsigned char moveRef[5];    /* validMoves index per cursor stop   */
static unsigned char raiseIdx[5];   /* validMoves index of each raise     */
static unsigned char stopCount, raiseStop, raiseCount, raiseSel, callRef;
static unsigned char numCol, numWidth, menuX, menuEnd;

static unsigned char moveClass(const char *n) {
  switch (n[0]) {
    case 'F': case 'f': return MV_FOLD;
    case 'C': case 'c': return MV_CALL;
```

```
    case 'R': case 'r':
    case 'B': case 'b': return MV_RAISE;
    case 'A': case 'a': return MV_ALLIN;
  }
  return MV_OTHER;
}

/* Offset of the trailing amount within a name, or 0 if it carries none. */
static unsigned char amountAt(const char *n) {
  static unsigned char p, sp;
  sp = 0;
  for (p = 0; n[p]; p++)
    if (n[p] == ' ')
      sp = p;
  if (sp && n[sp + 1] >= '0' && n[sp + 1] <= '9')
    return (unsigned char)(sp + 1);
  return 0;
}

/* Clamped so the menu can never reach the clock's columns. */
static unsigned char emit(const char *s, unsigned char len) {
  if ((unsigned char)(menuX + len) > (unsigned char)(menuEnd + 1))
    len = (menuX > menuEnd) ? 0 : (unsigned char)(menuEnd + 1 - menuX);
  if (len) {
    memcpy(tempBuffer, s, len);
    tempBuffer[len] = 0;
    drawStatusTextAt(menuX, tempBuffer);
  }
  menuX = (unsigned char)(menuX + len);
  return len;
}

static void addStop(unsigned char col, unsigned char len, unsigned char ref) {
  moveLoc[stopCount] = col;
  moveLen[stopCount] = len;
  moveRef[stopCount] = ref;
  stopCount++;
}

/* The raise while its half is selected, otherwise the call. */
void drawMoveNumber() {
  static unsigned char src, a, n;
  if (!numWidth)
    return;
  src = (raiseStop != 255 && cursorX == raiseStop) ? raiseIdx[raiseSel] : callRef;
  a = amountAt(state.validMoves[src].name);
  n = 0;
  if (a)
    while (state.validMoves[src].name[a + n]) {
      tempBuffer[n] = state.validMoves[src].name[a + n];
      n++;
    }
  while (n < numWidth)
    tempBuffer[n++] = ' ';
  tempBuffer[n] = 0;
  drawStatusTextAt(numCol, tempBuffer);
}

void layoutMoveMenu() {
  static unsigned char i, n, gap, fold, allin, start, tot;
  static unsigned char cls[5], vlen[5], amt[5];

  menuEnd = (unsigned char)(WIDTH - 5 - STATUS_TIMER_WIDTH);
  stopCount = 0;
  raiseStop = 255;
  raiseCount = 0;
  raiseSel = 0;
  numWidth = 0;
  callRef = 255;
  fold = 255;
  allin = 255;
  menuX = PLAYER_MOVE_START_X;

  for (i = 0; i < state.validMoveCount; i++) {
    cls[i] = moveClass(state.validMoves[i].name);
    amt[i] = amountAt(state.validMoves[i].name);
    n = (unsigned char)strlen(state.validMoves[i].name);
    vlen[i] = amt[i] ? (unsigned char)(amt[i] - 1) : n;
    if (cls[i] == MV_FOLD) {
      if (fold == 255) fold = i;
    } else if (cls[i] == MV_CALL) {
      if (callRef == 255) callRef = i;
      if (amt[i] && (unsigned char)(n - amt[i]) > numWidth)
        numWidth = (unsigned char)(n - amt[i]);
    } else if (cls[i] == MV_RAISE) {
      raiseIdx[raiseCount++] = i;
      if (amt[i] && (unsigned char)(n - amt[i]) > numWidth)
        numWidth = (unsigned char)(n - amt[i]);
    } else if (cls[i] == MV_ALLIN) {
      if (allin == 255) allin = i;
    }
  }

  /* Nothing to fold together: one entry per move, still clamped. */
  if (callRef == 255 || !raiseCount) {
    numWidth = 0;
    for (i = 0; i < state.validMoveCount; i++) {
      start = menuX;
      n = emit(state.validMoves[i].name,
               (unsigned char)strlen(state.validMoves[i].name));
      addStop(start, n, i);
      menuX = (unsigned char)(menuX + 2);
    }
    return;
  }

  tot = (unsigned char)(PLAYER_MOVE_START_X + vlen[callRef] + 1
                        + vlen[raiseIdx[0]] + numWidth);
  gap = 2;
  n = 2;
  if (fold != 255) n++;
  if (allin != 255) { n++; tot = (unsigned char)(tot + vlen[allin]); }
  if (fold != 255) tot = (unsigned char)(tot + vlen[fold]);
  if ((unsigned char)(tot + 2 * (n - 1) - 1) > menuEnd)
    gap = 1;

  if (fold != 255) {
    start = menuX;
    n = emit(state.validMoves[fold].name, vlen[fold]);
    addStop(start, n, fold);
    menuX = (unsigned char)(menuX + gap);
  }

  start = menuX;
  n = emit(state.validMoves[callRef].name, vlen[callRef]);
  addStop(start, n, callRef);

  emit("/", 1);

  start = menuX;
  n = emit(state.validMoves[raiseIdx[0]].name, vlen[raiseIdx[0]]);
  raiseStop = stopCount;
  addStop(start, n, raiseIdx[0]);
  menuX = (unsigned char)(menuX + gap);

  numCol = menuX;
  if ((unsigned char)(numCol + numWidth) > (unsigned char)(menuEnd + 1))
    numWidth = (numCol > menuEnd) ? 0 : (unsigned char)(menuEnd + 1 - numCol);
  menuX = (unsigned char)(menuX + numWidth);

  if (allin != 255) {
    menuX = (unsigned char)(menuX + gap);
    start = menuX;
    n = emit(state.validMoves[allin].name, vlen[allin]);
    addStop(start, n, allin);
  }
}


void requestPlayerMove() {
  requestedMove=NULL;

  if (state.viewing || state.activePlayer != 0)
    return;

  // Draw the moves on the status bar
  clearStatusBar();
  layoutMoveMenu();

  // The shared amount depends on where the cursor lands, so paint it after
  cursorX = stopCount > 1;
  drawMoveNumber();

  // Prepare the countdown timer
  moveTimeLeft = state.moveTime;
  drawStatusTimeLeft();

  drawBuffer();
  disableDoubleBuffer();

  // Zoom in the cursor
  i=moveLen[cursorX];
  h=moveLoc[cursorX];



// Animate the line. If needed, can add #if around instead
// of checking render_full_cards
//if (always_render_full_cards) {
  drawLine(0,HEIGHT-1, h+h+i);
  pause(5);
  for (j=h;j>0;--j) {
    hideLine(h-j,HEIGHT-1,1);
    hideLine(h+i+j-1,HEIGHT-1,1);
    pause(2);
  }
```

```
//)

  drawLine(h,HEIGHT-1,i);
  soundMyTurn();


  maxJifs = 60;
  maxJifs *=state.moveTime;
  waitCount=0;
  moveTimeLeft = state.moveTime;
  clearCommonInput();
  resetTimer();

  // Move selection loop
  while (moveTimeLeft)0 && !inputTrigger) {
    waitvsync();

    // Tick counter once per second
    if (++waitCount)2) {
      waitCount=0;
      i = (unsigned char)((maxJifs-getTime())/60);
      if (i!= moveTimeLeft) {
        moveTimeLeft =i;
        drawStatusTimeLeft();
        soundTick();

      }
    }

    // Move cursor, retricting to bounds
    readCommonInput();

    if (inputDirX !=0 ) {
      cursorX+=inputDirX;
      if (cursorX(stopCount) {
        hideLine(moveLoc[cursorX-inputDirX],HEIGHT-1,moveLen[cursorX-inputDirX]);
        drawLine(moveLoc[cursorX],HEIGHT-1,moveLen[cursorX]);
        drawMoveNumber();

        soundCursor();

      } else {
        cursorX-=inputDirX;

        soundCursorInvalid();

      }
      getTime();
    }

    // Up is -1, and picks the larger raise
    if (inputDirY != 0) {
      if (raiseStop != 255 && cursorX == raiseStop &&
          (unsigned char)(raiseSel - inputDirY) ( raiseCount) {
        raiseSel = (unsigned char)(raiseSel - inputDirY);
        moveRef[raiseStop] = raiseIdx[raiseSel];
        drawMoveNumber();

        soundCursor();

      } else {

        soundCursorInvalid();

      }
      getTime();
    }

    // Pressed Esc
    switch (inputKey) {
      case KEY_ESCAPE:
      case KEY_ESCAPE_ALT:
        showInGameMenuScreen();
        return;
    }
  }
  clearStatusBar();
  enableDoubleBuffer();

  // Request the highlighted move
  if (cursorX(255) {
    requestedMove = state.validMoves[moveRef[cursorX]].move;
    clearStatusBar();
    menuX = moveLoc[cursorX];
    menuEnd = (unsigned char)(WIDTH - 1);
    emit(state.validMoves[moveRef[cursorX]].name,
         (unsigned char)strlen(state.validMoves[moveRef[cursorX]].name));
    drawBuffer();

    soundSelectMove();


    // For some unknown reason, if I don't delay for a bit here,
    // the selectMove sound plays twice in the AppleWin emulator.
    pause(30);

  }

}

void clearGameState() {
  // Reset some variables
  prevRound = 99;
  prevPlayerCount = 0;
  wasViewing = 255;
  waitCount = -1;
}
```

SRC/GAMELOGIC.H — 31 LINES

```
#ifndef GAMELOGIC_H
#define GAMELOGIC_H

#include "platform-specific/graphics.h"

void showInGameMenuScreen();
void progressAnim(unsigned char y);
void drawPot();
void layoutMoveMenu();
void drawMoveNumber();
void drawStreetLabel();
void resetStateIfNewGame();
void drawNamePurse();
void drawBets();
#if defined(__WATCOMC__) || defined(HOST_UI_TEST)
#include <stdbool.h>
void drawCards(bool finalFlip);
#else
void drawCards();
#endif /* __WATCOMC__*/
void checkIfSpectatorStatusChanged();
void checkIfPlayerCountChanged();
void checkFinalFlip();
void drawStatusTimeLeft();
void highlightActivePlayer();
void animateChipsToPotOnRoundEnd();
void drawGameStatus();
void requestPlayerMove();
void clearGameState();

#endif /*GAMELOGIC_H*/
```

SRC/MISC.C — 176 LINES

```
/*
 * Include CC65 style Joystick defines for Adam - There is a probably a better way to do this.
 */

//#ifdef __WATCOMC__
//#define USE_PLATFORM_SPECIFIC_INPUT 1
//#endif

#ifdef BUILD_MSX
#include "msx/joystick.h"
#endif

#ifdef BUILD_COLECO
#include "coleco/joystick.h"
#endif

#ifdef BUILD_SMS
#include "sms/joystick.h"
#endif

#ifdef _CMOC_VERSION_
#include "coco/joystick.h"
#else
#ifdef __ADAM__
#include "adam/joystick.h"
#else
#ifdef __CC65__
#include <joystick.h>
#endif /* __CC65__ */
#endif /* __ADAM__ */
#include <stdio.h>
#include <string.h>
#include <conio.h>
#endif /* _CMOC_VERSION_ */
#include "platform-specific/graphics.h"
#include "platform-specific/input.h"
#include "platform-specific/sound.h"
#include "platform-specific/appkey.h"
#include "misc.h"

```

```
#ifdef _CMOC_VERSION_
#include <cmoc.h>
#include <coco.h>
#define true 1
#define false 0
char cgetc(void);
unsigned char kbhit(void);
#else
#include <stdbool.h>
#include <stdint.h>
#endif /* _CMOC_VERSION_ */
unsigned char _lastJoy, _joy;

void pause(unsigned char frames) {
  while (frames--)
    waitvsync();
}

void clearCommonInput() {
    inputTrigger=0;
    inputKey=0;
    inputDirX=0;
    inputDirY=0;
    _lastJoy=0;
    _joy=0;

#ifndef USE_PLATFORM_SPECIFIC_INPUT
  while (kbhit()) {
    cgetc();
  }
#else
  //while (!getPlatformKey());
#endif
}

void readCommonInput() {
  inputTrigger=0;
  inputKey=0;
  inputDirX=0;
  inputDirY=0;

  _joy = readJoystick();

  if (_joy != _lastJoy) {
    _lastJoy=_joy;

    if (JOY_LEFT(_joy))
      inputDirX = -1;
    else if (JOY_RIGHT(_joy))
      inputDirX =1;

    if (JOY_UP(_joy))
      inputDirY = -1;
    else if (JOY_DOWN(_joy))
      inputDirY = 1;

    if (JOY_BTN_1(_joy))
       inputTrigger=true;

    return;
  }


#ifndef USE_PLATFORM_SPECIFIC_INPUT
  if (!kbhit())
    return;

  inputKey = cgetc();
#else
  inputKey = getPlatformKey();
#endif
  if (!inputKey )
    return;

  switch (inputKey) {
    case KEY_LEFT_ARROW:
    case KEY_LEFT_ARROW_2:
    case KEY_LEFT_ARROW_3:
      inputDirX=-1;
      break;
    case KEY_RIGHT_ARROW:
    case KEY_RIGHT_ARROW_2:
    case KEY_RIGHT_ARROW_3:
      inputDirX=1;
      break;
    case KEY_UP_ARROW:
    case KEY_UP_ARROW_2:
    case KEY_UP_ARROW_3:
      inputDirY=-1;
      break;
    case KEY_DOWN_ARROW:
    case KEY_DOWN_ARROW_2:
    case KEY_DOWN_ARROW_3:
      inputDirY=1;
      break;
    case KEY_RETURN:
    case KEY_SPACE:
      inputTrigger=true;
      break;
    // case 'S':
    // case 's':
    //   prefs[PREF_SOUND] = toggleSound() ? 1 : 2;
    //   savePrefs();
    //   break;
  }
}

void applyPrefs() {

}

void loadPrefs() {

  read_appkey(AK_CREATOR_ID, AK_APP_ID, AK_KEY_PREFS, tempBuffer);

  if (strlen(tempBuffer)==0) {
    // Default all prefs to 1
    memset(prefs,1,3);
    // Platforms can set a different default color mode in their vars.h
    // (pref value is colorMode+1)
    #ifdef DEFAULT_COLOR_PREF
    prefs[PREF_COLOR] = DEFAULT_COLOR_PREF;
    #endif
  } else {
    strncpy(prefs, tempBuffer,3);
  }

  //setSound(prefs[PREF_SOUND]==1);
  setColorMode(prefs[PREF_COLOR]-1);
}

void savePrefs() {
  prefs[3] = 0; // Terminate prefs end as a string
  write_appkey(AK_CREATOR_ID, AK_APP_ID, AK_KEY_PREFS, prefs);
}

```

SRC/MISC.H 217 LINES

```
#ifndef MISC_H
#define MISC_H

/* To test the game:
 * VICE -
 *  1. In VICE, point DRIVE 11 to a local folder
 *  2. Edit /support/c64/emulator/fuji_mock_network.py to point to that same folder and run it
 *  3. Uncomment the USE_EMULATOR define below. */

// #define USE_EMULATOR

/*
 * Include CC65 style Joystick defines for Adam - There is a probably a better way to do this.
 */

#ifdef BUILD_COLECO
// conio.h must be parsed before the "#define state" below: it drags in
// z88dk's X11 headers, which declare struct members named "state".
#include <conio.h>
#include "coleco/joystick.h"
#else
#ifdef _CMOC_VERSION_
#include "coco/joystick.h"
#else
#ifdef __ADAM__
// conio.h must be parsed before the "#define state" below: it drags in
// z88dk's X11 headers, which declare struct members named "state".
#include <conio.h>
#include "adam/joystick.h"
#else
#ifndef __DOS__
#include <conio.h>
#endif /* __DOS__ */
#endif /* __ADAM__ */
#endif /* _CMOC_VERSION_ */
#endif /* BUILD_COLECO */
#include "platform-specific/graphics.h"
#include "platform-specific/input.h"
#ifdef _CMOC_VERSION_
#include <cmoc.h>
#include <coco.h>
#else
#include <stdbool.h>
#include <stdint.h>
#endif /* _CMOC_VERSION_ */
// FujiNet AppKey settings. These should not be changed
```

```
#define AK_LOBBY_CREATOR_ID 1     // FUJINET Lobby
#define AK_LOBBY_APP_ID 1         // Lobby Enabled Game
#define AK_LOBBY_KEY_USERNAME 0   // Lobby Username key
#define AK_LOBBY_KEY_SERVER 8     // Texas Hold'em registered as Lobby appkey 8

// Texas Hold'em client (own creator id - do not reuse 5 Card Stud's 0xE41C)
#define AK_CREATOR_ID 0xBEBE      // Texas Hold'em creator id
#define AK_APP_ID 0x01            // Texas Hold'em App ID
#define AK_KEY_PREFS 0            // Preferences

#define PREF_HELP 0  // 1/2 seen help screen no/yes
#define PREF_COLOR 1 // 1/2 - Color mode (platform specific)
//#define PREF_SOUND 2 // 1/2 - Sound Enabled yes/no

#ifdef __CC65__
#define _Packed
#endif

#ifdef _CMOC_VERSION_
#define _Packed
#endif

#ifdef BUILD_MSX
#define _Packed
#endif

#if defined(__ADAM__) || defined(BUILD_COLECO)
#define _Packed
#endif

#ifdef BUILD_SMS
#define _Packed
#endif

#ifdef __H89__
#define _Packed
#endif

typedef _Packed struct {
  char table    [9];
  char name     [21];
  char players  [6];
} Table;

typedef _Packed struct {
  char name   [9];
  uint8_t     status;
  uint16_t    bet;
  char move   [8];
  uint16_t    purse;
  char hand   [11];
} Player;

typedef _Packed struct {
  char move     [3];
  char name     [10];
} ValidMove;

typedef _Packed struct {
  char lastResult[81];
  uint8_t round;
  uint16_t pot;
  int8_t activePlayer;
  uint8_t moveTime;
  uint8_t viewing;
  char community[11];  // Texas Hold'em community cards, e.g. "askh2d" (up to 5)
  uint8_t validMoveCount;
  ValidMove validMoves[5];
  uint8_t playerCount;
  Player players[8];
} Game;

typedef _Packed struct {
  uint8_t count;
  Table table[10];
} Tables;

typedef union {
  uint8_t firstByte;
  Game game;
  Tables tables;
} ClientState;


#ifdef __WATCOMC__
extern int inputKey;
#else
extern char inputKey;
#endif
extern unsigned char prevPlayerCount, prevRound, currentCard, cardIndex, cursorX, cursorY,
    waitCount, wasViewing;
extern signed char inputDirX, inputDirY;

extern uint16_t prevPot, maxJifs;
#ifdef _CMOC_VERSION_
extern unsigned char noAnim, doAnim, finalFlip, inputTrigger;
#else
extern bool noAnim, doAnim, finalFlip, inputTrigger;
#endif /* _CMOC_VERSION_ */
/*
  Scratch buffer sizes. The ColecoVision has 1K of RAM in total, so these are
  cut to what each one actually has to hold rather than to a round number.
  tempBuffer cannot go below MAX_APPKEY_LEN+1: screens.c reads app keys into it.
*/
#ifdef BUILD_COLECO
#define TEMP_BUFFER_LEN 68
// query worst case: "?table=" + 8-char table id + "&player=" + an 11-char
// lobby username + NUL = 35. (5 Card Stud shipped 24 here; that was a latent
// overflow, not a target.)
#define QUERY_LEN       36
#define URL_BUFFER_LEN  96
#else
#define TEMP_BUFFER_LEN 128
#define QUERY_LEN       50
#define URL_BUFFER_LEN  128
#endif

extern char tempBuffer[TEMP_BUFFER_LEN];
extern char query[QUERY_LEN];
extern char playerName[12];
extern char serverEndpoint[50];

#ifdef BUILD_COLECO
/*
  The ColecoVision has 1K of RAM and this union is 429 bytes of it, so the
  state is never copied down: it is read in place out of the cartridge's reply
  window, which is what that window is 1K and addressable for. See
  src/coleco/vars.h and src/coleco/network.c.

  It is therefore READ-ONLY -- a store here goes nowhere, the cartridge cannot
  even see a write cycle -- and it is valid only until the next fuji_* call of
  any kind, because every transaction repaints the window.
*/
#define clientState (*(ClientState *) COLECO_REPLY_WINDOW)
#else
extern ClientState clientState;
#endif
//extern GameState state;
#define state clientState.game


// Common local scope temp variables
extern unsigned char h, i, j, k, x, y, xx;
extern unsigned char playerX[8], playerY[8], moveLoc[5];
extern signed char playerBetX[8], playerBetY[8], playerDir[8];
extern char *hand, *requestedMove;
extern char prefs[4];


// Screen specific player/bet coordinates. These are declared by each platform's
// own vars.h as well; the ColecoVision and NES make them const so they land
// in ROM rather than spending 96 bytes of RAM on tables nobody writes.
#if !defined(BUILD_COLECO) && !defined(BUILD_NES)
extern unsigned char playerXMaster[] ;
extern unsigned char playerYMaster[] ;
extern char playerDirMaster[] ;
extern char playerBetXMaster[];
extern char playerBetYMaster[] ;

// Simple hard coded arrangment of players around the table based on player count.
// These refer to index positions in the Master arrays above
// Downside is new players will cause existing player positions to move.
extern char playerCountIndex[] ;
#endif

void pause(unsigned char frames);
void clearCommonInput();
void readCommonInput();
void loadPrefs();
void savePrefs();

#endif /* MISC_H */
```

SRC/SCREENS.C — 626 LINES

```
#ifdef _CMOC_VERSION_
#include "coco/coco_bool.h"
#include <cmoc.h>
#include <coco.h>
unsigned char kbhit(void);
char cgetc(void);
#else
#include <stdlib.h>
#ifdef __WATCOMC__
//#define USE_PLATFORM_SPECIFIC_INPUT 1
```

```
#else
#include <conio.h>
#endif /* __WATCOMC__ */
#include <string.h>
#endif /* _CMOC_VERSION_ */
#include "stateclient.h"
#include "screens.h"
#include "misc.h"
#include "gamelogic.h"

#include "platform-specific/graphics.h"
#include "platform-specific/input.h"
#include "platform-specific/appkey.h"
#include "platform-specific/util.h"
#include "platform-specific/sound.h"

#define PLAYER_NAME_MAX 8

#ifdef HIRES_Y
unsigned char y_bump = 1;
#define BUMP_LINE y_bump+=4;
#define CLEAR_BUMP y_bump=1;
#else
#define BUMP_LINE y++;
#define CLEAR_BUMP
#endif

#define TABLE_LEFT (WIDTH/2-13)

// For limited height scenarios, set this (e.g. to 3) in platform specific vars.h
#ifndef TABLE_TOP
#define TABLE_TOP 5
#endif

// For limited height scenarios, set this to 1 to remove space between table names in platform
+    specific vars.h
#ifndef TABLE_ROW_HEIGHT
#define TABLE_ROW_HEIGHT 2
#endif

// For limited height scenarios, set this to 0 or 1 in platform specific vars.h
// Also, "#define SKIP_FIRST_BUMP" in vars.h to avoid extra space at top
#ifndef HOW_TO_PLAY_ROW_START
#define HOW_TO_PLAY_ROW_START 3
#endif

unsigned char redrawGameScreen = 0;

/// @brief Convenience function to draw text centered at row Y
void centerText(unsigned char y, const char * text) {
#ifdef HIRES_Y
  drawTextAt((unsigned char)(WIDTH/2-strlen(text)/2), y*8+y_bump, text);
#else
  drawText((unsigned char)(WIDTH/2-strlen(text)/2), y, text);
#endif
}

/// @brief Convenience function to draw status text centered
void centerStatusText(const char * text) {
  clearStatusBar();
  drawStatusTextAt((unsigned char)((WIDTH-strlen(text))>>1),text);
}

/// @brief Convenience function to reset screen and draw border
void resetScreenWithBorder() {
  resetScreen();
  drawBorder();
}

/// @brief Shows information about the game
void showHelpScreen() {
  static unsigned char y;
  enableDoubleBuffer();
  centerStatusText("LOADING INSTRUCTIONS..");
  drawBuffer();

  // This COULD be retrieved from the server, especially if
  // this client were game agnostic.
  resetScreenWithBorder();

  centerText(HOW_TO_PLAY_ROW_START,"HOW TO PLAY TEXAS HOLD'EM");
  y=HOW_TO_PLAY_ROW_START+1;

  #ifndef SKIP_FIRST_BUMP
  BUMP_LINE
  #endif
                  //12345678901234567890123456789012
  y++;centerText(y, "2 CARDS EACH, 5 IN THE MIDDLE.");
  y++;centerText(y, "BEST 5-CARD HAND WINS THE POT.");
  y+=2;

  centerText(y, "MOVES");

  BUMP_LINE       //12345678901234567890123456789012
  y++;centerText(y, "FOLD  - QUIT THE HAND     ");BUMP_LINE
  y++;centerText(y, "CHECK - FREE PASS         ");BUMP_LINE
  y++;centerText(y, "CALL  - MATCH THE BET     ");BUMP_LINE
  y++;centerText(y, "RAISE - INCREASE THE BET  ");BUMP_LINE
  y++;centerText(y, "ALL-IN- BET ALL YOUR CHIPS");

/* Lynx has no room for these; every supported platform does. */
#if HEIGHT >= 20
  y++;            //12345678901234567890123456789012
  y++;centerText(y, "LEFT/RIGHT PICKS THE MOVE");
  y++;centerText(y, "UP/DOWN SETS THE AMOUNT  ");
#endif

  CLEAR_BUMP

#ifdef BUILD_COLECO
  centerStatusText("PRESS FIRE TO CONTINUE");
#elif defined(BUILD_NES)
  centerStatusText("PRESS A TO CONTINUE");
#elif defined(BUILD_SMS)
  centerStatusText("PRESS 1 TO CONTINUE");
#else
  centerStatusText("PRESS A KEY TO CONTINUE");
#endif

  drawBuffer();

  clearCommonInput();
#ifdef USE_PLATFORM_SPECIFIC_INPUT
  // getPlatformKey() is non-blocking on these platforms
  while (!getPlatformKey());
#else
  cgetc();
#endif
  resetScreen();
  drawBuffer();
  redrawGameScreen=1;
}

/// @brief Action called in Welcome Screen to check if a server name is stored in an app key
void welcomeActionVerifyServerDetails() {
  // Read server endpoint into server and query
  read_appkey(AK_LOBBY_CREATOR_ID,  AK_LOBBY_APP_ID, AK_LOBBY_KEY_SERVER, tempBuffer);

  i=(unsigned char)strlen(tempBuffer);
  if (i) {

    // Split "server?query" into separate variables
    while(--i) {
      if (tempBuffer[i]=='?') {
        tempBuffer[i]=0;
        strcpy(serverEndpoint, tempBuffer);
        break;
      }
    }

    // No '?' found: the whole appkey is the server endpoint
    if (i==0)
      strcpy(serverEndpoint, tempBuffer);
  }

  // Keep the endpoint from the appkey but always let the player pick a room
  // from the server's table list at launch (query empty -> selection screen)
  query[0]=0;
}




bool inputFieldCycle(uint8_t x, uint8_t y, uint8_t max, char* buffer) {
  static uint8_t done, curx, lastY;

  // Initialize first call to input box
  //i=strlen(buffer);
  if (done == 1 || lastY != y) {
    done=0;
    lastY=y;
    curx = (unsigned char)strlen((char *)buffer);
    drawText(x,y, (const char *)buffer);
    drawChip(x+curx,y);
  }
   // curx=i;
   // done=0;
  // }


  // Process any waiting keystrokes
  #ifdef USE_PLATFORM_SPECIFIC_INPUT
  inputKey = getPlatformKey();
  if (inputKey) {
    done=0;
    disableDoubleBuffer();
```

```
  #else
  if (kbhit()) {
    done=0;
    disableDoubleBuffer();
    inputKey = cgetc();
  #endif

    if ((inputKey == KEY_RETURN ) && curx>1) {
      done=1;
      // remove cursor
      drawText(x+1+i,y," ");
    } else if (inputKey == KEY_BACKSPACE && curx>0) {
      buffer[--curx]=0;
      drawText(x+1+curx,y," ");
    } else if (
      curx < max && ((curx>0 && inputKey == KEY_SPACE) || (inputKey>= 97 && inputKey <=122)
          ||(inputKey>= 48 && inputKey <=57) || (inputKey>= 65 && inputKey <=90))    // 0-9 A-Z
    ) {
       buffer[curx]=(unsigned char)inputKey;
      buffer[++curx]=0;
    }

    drawText(x,y, (const char *)buffer);
    drawChip(x+curx,y);

    return done==1;
  }

  return false;

}

void showPlayerNameScreen() {
  enableDoubleBuffer();
  resetScreenWithBorder();
  clearStatusBar();
  drawLogo();

  drawBuffer();
  disableDoubleBuffer();

  centerText(HEIGHT/2, "ENTER YOUR NAME");
  drawBox(WIDTH/2-PLAYER_NAME_MAX/2-1,HEIGHT/2+2,PLAYER_NAME_MAX+1,1);
  drawText(WIDTH/2-PLAYER_NAME_MAX/2,HEIGHT/2+3, playerName);

  i=(unsigned char)strlen(playerName);

  clearCommonInput();
#ifdef USE_PLATFORM_NAME_ENTRY
  // No keyboard on this platform: the name is typed on screen instead.
  platformNameEntry(WIDTH/2-PLAYER_NAME_MAX/2, HEIGHT/2+3, PLAYER_NAME_MAX, playerName);
#else
  while (!inputFieldCycle(WIDTH/2-PLAYER_NAME_MAX/2, HEIGHT/2+3, PLAYER_NAME_MAX, playerName))
    ;
#endif

  enableDoubleBuffer();
  for (y=HEIGHT/2;y<HEIGHT/2+5;++y)
    centerText(y, "                ");

  drawBuffer();

  write_appkey(AK_LOBBY_CREATOR_ID,  AK_LOBBY_APP_ID, AK_LOBBY_KEY_USERNAME, playerName);
}

/// @brief Action called in Welcome Screen to verify player has a name
void welcomeActionVerifyPlayerName() {
  // Read player's name from app key
  read_appkey(AK_LOBBY_CREATOR_ID,  AK_LOBBY_APP_ID, AK_LOBBY_KEY_USERNAME, tempBuffer);

  tempBuffer[12]=0;
  if (tempBuffer[0] != 0)
    strcpy(playerName,tempBuffer);

  // Capture username if player didn't come in from the lobby
  if (strlen(playerName) == 0)
    showPlayerNameScreen();
}

/// @brief Shows the Welcome Screen with Logo. Asks player's name
void showWelcomeScreen() {
  resetScreenWithBorder();
  drawLogo();

  welcomeActionVerifyPlayerName();
  welcomeActionVerifyServerDetails();

  strcpy(tempBuffer, "WELCOME ");
  strcat(tempBuffer, playerName);
  centerText(HEIGHT/2,tempBuffer);
  drawBuffer();
  pause(45);


  // If first run, show the help screen
  if (prefs[PREF_HELP]!=2) {
    prefs[PREF_HELP]=2;
    savePrefs();
    //showHelpScreen();

  }
  pause(30);
}

/// @brief Action in Table Selection Screen, joins the selected server
void tableActionJoinServer() {
  // Reset the game state
  clearGameState();

  strcat(query, "&player=");
  strcat(query, playerName);

  // Replace space with + for pshowWelcomeScreenlayer name
  i=(unsigned char)strlen(query);
  while(--i)
    if (query[i]==' ')
      query[i]='+';

  redrawGameScreen=1;
}

/// @brief Shows a screen to select a table to join
void showTableSelectionScreen() {
  static uint8_t shownChip;
  static unsigned char tableIndex=0;
  static uint8_t skipApiCall;
  static Table* table;
  skipApiCall=0;
  // An empty query means a table needs to be selected
  while (strlen(query)==0) {

    // SkipApiCall will be true if toggling color, to redraw

    if (!skipApiCall) {
      // Show the status immediately before retrival
      centerStatusText("REFRESHING TABLE LIST..");
      drawBuffer();
    }

    resetScreenWithBorder();

    centerText(3, "CHOOSE A TABLE TO JOIN");
    drawText(TABLE_LEFT,TABLE_TOP, "TABLE");
    drawText(TABLE_LEFT+19,TABLE_TOP, "PLAYERS");
    drawLine(TABLE_LEFT,TABLE_TOP+1,26);

    drawBuffer();
    waitvsync();

    if (!skipApiCall){
      apiCall("tables");
    }

    skipApiCall=0;

    if (clientState.tables.count>0) {
      for(i=0;i<clientState.tables.count;++i) {
        table = &clientState.tables.table[i];
        drawText(TABLE_LEFT,TABLE_TOP+2+i*TABLE_ROW_HEIGHT, table->name);
        drawText((unsigned char)(TABLE_LEFT+26-strlen(table->players)),
            TABLE_TOP+2+i*TABLE_ROW_HEIGHT, table->players);

        if (table->players[0]>'0') {
          drawText((unsigned char)(TABLE_LEFT+24-strlen(table->players)),
              TABLE_TOP+2+i*TABLE_ROW_HEIGHT, "*");
        }
      }
    } else {
      centerText(12, "NO TABLES ARE AVAILABLE");
    }

    //drawStatusText(" R+EFRESH  H+ELP  C+OLOR  S+OUND  Q+UIT");
#if WIDTH>=40
    drawStatusText("R-EFRESH   H-ELP  C-OLOR   N-AME   Q-UIT");
#elif defined(BUILD_COLECO)
    // Keypad, not letters: src/coleco/input.c maps 1-5 onto these shortcuts.
    //            12345678901234567890123456789012
    drawStatusText("1REFRSH 2HELP 3COLOR 4NAME 5QUIT");
#elif defined(BUILD_NES)
    // Buttons, not letters: src/nes/input.c maps them onto these shortcuts.
    //            12345678901234567890123456789012
    drawStatusText("A-JOIN  START-HELP  SELECT-NAME");
#elif defined(BUILD_SMS)
    // Buttons, not letters: src/sms/input.c maps them onto these shortcuts.
    //            12345678901234567890123456789012
    drawStatusText("1-JOIN <NAME >REFRESH PAUSE-HELP");
```

```
#else             //12345678901234567890123456789012
    drawStatusText("R-EFRESH   H-ELP   N-AME   Q-UIT");
#endif
    drawBuffer();
    disableDoubleBuffer();
    shownChip=0;

    clearCommonInput();
    while (!inputTrigger || !clientState.tables.count) {
      readCommonInput();
      //if (kbhit())
        //inputKey = cgetc();
      if (inputKey == 'h' || inputKey == 'H') {
        showHelpScreen();
        break;
      } else if (inputKey == 'r' || inputKey =='R') {
        break;
      } else if (inputKey == 'c' || inputKey =='C') {
        prefs[PREF_COLOR] = cycleNextColor()+1;
        savePrefs();
        enableDoubleBuffer();
        skipApiCall=1;
        break;
        } else if (inputKey == 'n' || inputKey =='N') {
        showPlayerNameScreen();
        break;
      } else if (inputKey == 'q' || inputKey =='Q') {
        quit();
      } /*else if (inputKey != 0) {
        itoa(inputKey, tempBuffer, 10);
        drawStatusText(tempBuffer);
      } */

      if (clientState.tables.count)0 && (!shownChip || inputDirY)) {

        drawText(TABLE_LEFT-1,TABLE_TOP+2+tableIndex*TABLE_ROW_HEIGHT," ");
        tableIndex+=inputDirY;
        if (tableIndex==255)
          tableIndex=clientState.tables.count-1;
        else if (tableIndex)=clientState.tables.count)
          tableIndex=0;

        drawChip(TABLE_LEFT-1,TABLE_TOP+2+tableIndex*TABLE_ROW_HEIGHT);

        soundCursor();
        shownChip=1;
      }
    }

    enableDoubleBuffer();

    if (inputTrigger) {
      soundSelectMove();

      // Clear screen and write server name
      resetScreenWithBorder();
      clearStatusBar();
      centerText(HEIGHT/2, clientState.tables.table[tableIndex].name);

      strcpy(query, "?table=");
      strcat(query, clientState.tables.table[tableIndex].table);
      strcpy(tempBuffer, serverEndpoint);
      strcat(tempBuffer, query);

      //  Update server app key in case of reboot
      write_appkey(AK_LOBBY_CREATOR_ID,  AK_LOBBY_APP_ID, AK_LOBBY_KEY_SERVER, tempBuffer);

    }

  }

  centerText(HEIGHT-7, "CONNECTING TO SERVER");
  drawBuffer();

  progressAnim(HEIGHT-5);

  tableActionJoinServer();
}

/// @brief Shows main game play screen (table and cards)
void showGameScreen() {
  /*

typedef struct {
  char name   [9];
  uint8_t     status;
  uint16_t    bet;
  char move   [8];
  uint16_t    purse;
  char hand   [11];
} Player;

typedef struct {
  char lastResult[81];
  uint8_t round;
  uint16_t pot;
  int8_t activePlayer;
  uint8_t moveTime;
  uint8_t viewing;
  char community[11];
  uint8_t validMoveCount;
  ValidMove validMoves[5];
  uint8_t playerCount;
  Player players[8];
} Game;
*/
  // printf("\r\nround=%u, pot=%u, ap=%s, m=%u,v=%u",state.round, state.pot,
  +     state.activePlayer, state.moveTime, state.viewing);
  // printf("\r\nvalidMoves=%u, playerCount=%u\r\n",state.validMoveCount, state.playerCount);

  // for(i=0;i(state.validMoveCount;++i) {
  //   printf(" %s=%s,",state.validMoves[i].move, state.validMoves[i].name);
  // }
  // printf("\r\n");
  // for(i=0;i(state.playerCount;++i) {
  //   printf("\r\n%s=%u, bet=%u !%s! p=%u, H=%s",state.players[i].name,
  +     state.players[i].status, state.players[i].bet, state.players[i].move,
  +     state.players[i].purse, state.players[i].hand);
  // }

  // printf("\r\n");
  // drawBuffer();
  // cgetc();

#ifndef SINGLE_BUFFER
  redrawGameScreen=1;
#endif

  checkIfSpectatorStatusChanged();
  checkIfPlayerCountChanged();

  // Animate chips to pot before drawing the new state
  animateChipsToPotOnRoundEnd();

  checkFinalFlip();

  if (redrawGameScreen) {
    resetScreen();
  }

  resetStateIfNewGame();

  drawPot();
  drawStreetLabel();

  if (state.playerCount)1) {
    drawNamePurse();
    drawBets();
#ifdef SINGLE_BUFFER
    if (state.round ( 5) {
      drawCards(false);
    }
#else
    drawCards(false);
#endif
  }

  drawGameStatus();
  drawBuffer();
  highlightActivePlayer();

#ifdef SINGLE_BUFFER
  redrawGameScreen=0;
#endif
  prevRound = state.round;
}

/// @brief shows in-game menu
void showInGameMenuScreen() {

  hideLine(0,0,0);
  i=1;
  while (i) {
    enableDoubleBuffer();
    clearStatusBar();
    resetScreenWithBorder();
    drawBuffer();

    x = WIDTH/2-9;
    y = HEIGHT/2-3;

    drawBox(x-3,y-2,22,9);
#ifdef BUILD_COLECO
    drawText(x,y,    "  5: QUIT TABLE");
    drawText(x,y+=2, "  2: HOW TO PLAY");
    drawText(x,y+=2, "  3: COLOR TOGGLE");
```

```
    drawText(x,y+=2, "  *: KEEP PLAYING");
#elif defined(BUILD_NES)
    drawText(x,y,    "SEL+START: QUIT TABLE");
    drawText(x,y+=2, "START: HOW TO PLAY");
    drawText(x,y+=2, "B: KEEP PLAYING");
#elif defined(BUILD_SMS)
    drawText(x,y,    "  2+1: QUIT TABLE");
    drawText(x,y+=2, "PAUSE: HOW TO PLAY");
    drawText(x,y+=2, "    2: KEEP PLAYING");
#else
    drawText(x,y,    "  Q: QUIT TABLE");
    drawText(x,y+=2, "  H: HOW TO PLAY");
    drawText(x,y+=2, "  C: COLOR TOGGLE");
    //drawText(x,y+=2, "  S: SOUND TOGGLE");
    drawText(x,y+=2, "ESC: KEEP PLAYING");
#endif
    drawBuffer();

    clearCommonInput();

    i=1;
    while (i==1) {
      readCommonInput();
      switch (inputKey) {
        case 'c':
        case 'C':
            prefs[PREF_COLOR] = cycleNextColor()+1;
            savePrefs();
            i=2;
            break;
        case 'h':
        case 'H':
          showHelpScreen();
        case KEY_ESCAPE:
        case KEY_ESCAPE_ALT:
          i=0;
          break;
        case 'q':
        case 'Q':
          resetScreenWithBorder();
          centerText(10, "PLEASE WAIT");
          drawBuffer();

          // Inform server player is leaving
          apiCall("leave");
          progressAnim(12);

          //  Clear server app key in case of reboot
          write_appkey(AK_LOBBY_CREATOR_ID,  AK_LOBBY_APP_ID, AK_LOBBY_KEY_SERVER, "");

          // Clear query so a new table will be selected
          clearCommonInput();
          strcpy(query,"");
          showTableSelectionScreen();
          return;
      }
    }
  }

  // Show game screen again before returning
  redrawGameScreen=1;
  resetScreen();
  drawBuffer();
  showGameScreen();
  clearCommonInput();
}
```

SRC/SCREENS.H — 44 LINES

```
#ifndef SCREENS_H
#define SCREENS_H

#ifndef _CMOC_VERSION_
#include <stdint.h>
#endif

/// @brief Convenience function to reset screen and draw border
void resetScreenWithBorder();

/// @brief Shows information about the game
void showHelpScreen();

/// @brief Action called in Welcome Screen to check if a server name is stored in an app key
void welcomeActionVerifyServerDetails();

/// @brief Action called in Welcome Screen to verify player has a name
void welcomeActionVerifyPlayerName();

/// @brief Shows the Welcome Screen with Logo. Asks player's name
void showWelcomeScreen();

/// @brief Action in Table Selection Screen, joins the selected server
void tableActionJoinServer();

/// @brief Shows a screen to select a table to join
void showTableSelectionScreen();

/// @brief Shows main game play screen (table and cards)
void showGameScreen();

/// @brief shows in-game menu
void showInGameMenuScreen();

/// @brief Allow the player to modify their name
void showPlayerNameScreen();

/// @brief Handles available key strokes for the defined input box (player name and chat).
 +    Returns true if user hits enter
bool inputFieldCycle(uint8_t x, uint8_t y, uint8_t max, char* buffer);

/// @brief Centers status text in the status bar
void centerStatusText(const char * text);

#endif /*SCREENS_H*/
```

SRC/STATECLIENT.C — 61 LINES

```
#ifdef _CMOC_VERSION_
#include <cmoc.h>
#define true 1
#define false 0
#else
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
#endif /* CMOC_VERSION */
#define POKE(addr,val)     (*(unsigned char*) (addr) = ((unsigned char)(val)))
#define POKEW(addr,val)    (*(unsigned*) (addr) = ((unsigned)(val)))
#define PEEK(addr)         (*(unsigned char*) (addr))
#define PEEKW(addr)        (*(unsigned*) (addr))
#include "stateclient.h"
#include "misc.h"
#include "platform-specific/network.h"
#ifndef _CMOC_VERSION_
#include <conio.h>
#endif

char urlBuffer[URL_BUFFER_LEN];

unsigned char apiCall(const char *path) {
#ifndef __CBM__
  if (serverEndpoint[0] != 'N') {
    strcpy(urlBuffer,"N:");
  } else {
    strcpy(urlBuffer,"");
  }
#else
  memset(urlBuffer,0,sizeof(urlBuffer));
#endif /* ! __CBM__ */

  strcat(urlBuffer, serverEndpoint);
  strcat(urlBuffer, path);
  strcat(urlBuffer, query);

  // Set Binary mode
  strcat(urlBuffer, query[0] ? "&bin=1" QUERY_SUFFIX : "?bin=1" QUERY_SUFFIX);
  //printf(urlBuffer);
  //printf("\r\n");

  // sizeof(Game), not sizeof(clientState.game): on the ColecoVision clientState
  // is a macro dereferencing a cast of the cartridge's reply window, and
  // sccz80 will not take sizeof() of that expression.
  return getResponse(urlBuffer, &clientState.firstByte, sizeof(Game));

}

unsigned char getStateFromServer()
{
  if (requestedMove) {
    strcpy(tempBuffer, "move/");
    strcat(tempBuffer, requestedMove);
    requestedMove=NULL;
  } else {
    strcpy(tempBuffer, "state");
  }

  return apiCall(tempBuffer);
}
```

SRC/STATECLIENT.H — 15 LINES

```
#ifndef STATECLIENT_H
#define STATECLIENT_H
```

```
 3
 4 #ifdef _CMOC_VERSION_
 5 #include <cmoc.h>
 6 #include <coco.h>
 7 #else
 8 #include <stdbool.h>
 9 #endif
10
11 void updateState(unsigned char isTables);
12 unsigned char getStateFromServer();
13 unsigned char apiCall(const char *path);
14
15 #endif /* STRECLIENT_H */
```

SRC/PLATFORM-SPECIFIC/APPKEY.H 58 LINES

```
 1 /**
 2  * @brief   Cbm App Key Functions
 3  * @author  Oscar Fowler, Thomas Cherryhomes, Eric Carr
 4  * @license gpl v. 3
 5  */
 6
 7 #ifndef APPKEY_H
 8 #define APPKEY_H
 9
10 #define MAX_APPKEY_LEN 64
11
12 #define APPKEY_READ  0
13 #define APPKEY_WRITE 1
14
15 /**
16  * @brief Convenience type for handling an appkey
17  */
18 typedef union {
19     struct
20     {
21         unsigned int creator;
22         unsigned char app;
23         unsigned char key;
24         unsigned char mode;
25         unsigned char reserved;
26     } open;
27     struct
28     {
29         unsigned char length;
30         unsigned char reserved;
31         unsigned char value[MAX_APPKEY_LEN];
32
33     } read;
34     struct
35     {
36        char value[MAX_APPKEY_LEN+32];
37     } write;
38 } appkeyblock;
39
40 /**
41  * @brief Read a key into appkey var appkey
42  * @param creator_id Key creator ID: ($0000-$FFFF)
43  * @param app_id The App ID ($00-$FF)
44  * @param key_id The Key ID ($00-$FF)
45  * @brief string output (64 chars max)
46  */
47 void read_appkey(unsigned int creator_id, unsigned char app_id, unsigned char key_id, char
 +     *data);
48
49 /**
50  * @brief write a key from appkey var appkey
51  * @param creator_id Key creator ID: ($0000-$FFFF)
52  * @param app_id The App ID ($00-$FF)
53  * @param key_id The Key ID ($00-$FF)
54  * @param Data string to write (64 chars max)
55  */
56 void write_appkey(unsigned int creator_id, unsigned char app_id, unsigned char key_id, const
 +     char *data);
57
58 #endif /* APPKEY_H */
```

SRC/PLATFORM-SPECIFIC/GRAPHICS.H 59 LINES

```
 1 /*
 2  Platform specific graphics commands
 3 */
 4
 5 #ifndef GRAPHICS_H
 6 #define GRAPHICS_H
 7
 8 #define PARTIAL_LEFT 1
 9 #define PARTIAL_RIGHT 2
10 #define FULL_CARD 3
11 /* OR'd in when this card is all that is left of a hand, so platforms with no
12    double buffer clear the slot the dropped card used to occupy. */
13 #define CARD_CLEAR_NEXT 4
14
15 #ifdef _CMOC_VERSION_
16 #include "../coco/coco_bool.h"
17 #else
18 #include <stdint.h>
19 #include <stdbool.h>
20 #endif /* _CMOC_VERSION_  */
21 #include "../misc.h"
22
23 extern bool always_render_full_cards;
24
25 void enableDoubleBuffer();
26 void disableDoubleBuffer();
27 void drawBuffer();
28
29 // Call to clear the screen to an empty table
30 void resetScreen();
31
32 void clearStatusBar();
33 void drawStatusTextAt(unsigned char x, const char* s);
34 void drawStatusText(const char* s);
35 void drawStatusTimer();
36
37
38 // For optional HI-RES Y enabled platforms
39 void drawTextAt(unsigned char x, unsigned char y, const char* s);
40
41 void drawText(unsigned char x, unsigned char y, const char* s);
42 void drawCard(unsigned char x, unsigned char y, unsigned char partial, const char* s, unsigned
 +     char isHidden);
43 void drawChip(unsigned char x, unsigned char y);
44 void drawBlank(unsigned char x, unsigned char y);
45 void drawLine(unsigned char x, unsigned char y, unsigned char w);
46 void hideLine(unsigned char x, unsigned char y, unsigned char w);
47 void drawBox(unsigned char x, unsigned char y, unsigned char w, unsigned char h);
48 void drawBorder();
49 void drawLogo();
50
51 void initGraphics();
52 void resetGraphics();
53 void waitvsync();
54
55 uint8_t cycleNextColor();
56 void setColorMode();
57 extern unsigned char colorMode;
58
59 #endif /* GRAPHICS_H */
```

SRC/PLATFORM-SPECIFIC/INPUT.H 35 LINES

```
 1 /*
 2   Platform specific input.
 3 */
 4 #ifndef INPUT_H
 5 #define INPUT_H
 6
 7 // Include platform specific defines before the input include
 8 #include "../c64/vars.h"
 9 #include "../apple2/vars.h"
10 #include "../adam/vars.h"
11 #include "../coco/vars.h"
12 #include "../msdos/vars.h"
13 #include "../plus4/vars.h"
14 #include "../msx/vars.h"
15 #include "../coleco/vars.h"
16 #include "../nes/vars.h"
17 #include "../sms/vars.h"
18 #include "../h89/vars.h"
19
20 // Platform specific implementations
21 unsigned char readJoystick();
22
23 #ifdef USE_PLATFORM_SPECIFIC_INPUT
24 void initPlatformKeyboardInput(void);
25 int getPlatformKey(void);
26 #endif
27
28 // Platforms with no keyboard at all type on screen instead of through
29 // inputFieldCycle(); see src/coleco/osk.c.
30 #ifdef USE_PLATFORM_NAME_ENTRY
31 void platformNameEntry(unsigned char x, unsigned char y, unsigned char max,
32                        char *buffer);
33 #endif
34
35 #endif /* INPUT_H */
```

SRC/PLATFORM-SPECIFIC/NETWORK.H 9 LINES

```
 1 /*
 2   Network functionality
```

```
 3 */
 4 #ifndef NETWORK_H
 5 #define NETWORK_H
 6
 7 uint8_t getResponse(char *url, unsigned char *buffer, uint16_t max_length);
 8
 9 #endif /* NETWORK_H */
```

SRC/PLATFORM-SPECIFIC/SOUND.H 36 LINES

```
 1 /*
 2   Platform specific sound functions
 3 */
 4 #ifndef SOUND_H
 5 #define SOUND_H
 6
 7 #ifdef _CMOC_VERSION_
 8 #include <cmoc.h>
 9 #include <coco.h>
10 #else
11 #include <stdbool.h>
12 #include <stdint.h>
13 #endif /* _CMOC_VERSION_ */
14
15 void initSound();
16
17
18 /* Will bring back sound toggle if there is a platform with no controllable sound volume
19 bool toggleSound();
20 void setSound();
21 */
22
23 void soundJoinGame();
24 void soundMyTurn();
25 void soundGameDone();
26 void soundDealCard();
27 void soundTick();
28 void soundPlayerJoin();
29 void soundPlayerLeft();
30 void soundSelectMove();
31 void soundCursor();
32 void soundCursorInvalid();
33 void soundTakeChip(uint16_t counter);
34
35
36 #endif /* SOUND_H */
```

SRC/PLATFORM-SPECIFIC/UTIL.H 16 LINES

```
 1 #ifndef UTIL_H
 2 #define UTIL_H
 3
 4
 5 /*
 6   Platform specific utilities that don't fit in any category
 7 */
 8
 9
10 void resetTimer();
11 int getTime();
12 void quit();
13
14
15 #endif /* UTIL_H */
16
```

SRC/H89/APPKEY.C 35 LINES

```
 1 #ifdef __H89__
 2
 3 /**
 4  * @brief   Heathkit H89 App Key Functions
 5  * @author  Thomas Cherryhomes
 6  * @email   thom dot cherryhomes at gmail dot com
 7  * @license gpl v. 3, see LICENSE for details
 8  */
 9
10 #include <stdint.h>
11 #include <string.h>
12 #include <fujinet-fuji.h>
13 #include "../platform-specific/appkey.h"
14
15 void read_appkey(unsigned int creator_id, unsigned char app_id, unsigned char key_id, char
 +     *data)
16 {
17     uint16_t read = 0;
18
19     fuji_set_appkey_details(creator_id, app_id, DEFAULT);
20     if (!fuji_read_appkey(key_id, &read, (uint8_t *) data))
21         read = 0;
22
23     // Terminate as a string. Callers hand us the shared tempBuffer scratch
24     // global and then strlen() it, so this has to happen even on failure or
25     // they read back whatever the previous screen left behind.
26     data[read] = 0;
27 }
28
29 void write_appkey(unsigned int creator_id, unsigned char app_id, unsigned char key_id, const
 +     char *data)
30 {
31     fuji_set_appkey_details(creator_id, app_id, DEFAULT);
32     fuji_write_appkey(key_id, strlen(data), (uint8_t *) data);
33 }
34
35 #endif /* __H89__ */
```

SRC/H89/CONSOLE.C 492 LINES

```
 1 #ifdef __H89__
 2
 3 /**
 4  * @brief   H19 terminal screen - Heathkit H89
 5  * @author  Thomas Cherryhomes
 6  * @email   thom dot cherryhomes at gmail dot com
 7  * @license gpl v. 3, see LICENSE for details
 8  *
 9  * The console is the H19 on the H89's 8250 at 9600 baud -- under a thousand
10  * characters a second -- and the shared game code repaints the whole table on
11  * every server poll. So drawing goes into a back buffer (buf), a second buffer
12  * (scr) mirrors what the terminal shows, and flush() sends only the cells
13  * that differ. Console bytes go through the OS raw: HDOS's .SCOUT/.SCIN with
14  * the console driver in raw character mode, or the CP/M BIOS's
15  * CONOUT/CONST/CONIN straight from the jump table (the BDOS console calls
16  * expand, filter and hold back characters). The OS layer is
17  * fujinet-battleship's src/h89/console.c.
18  */
19
20 #include <string.h>
21 #include "console.h"
22
23 #define ESC 0x1B
24 #define CUR_UNKNOWN 0xFF
25
26 // Re-send up to this many unchanged cells rather than address past them
27 // (ESC Y row col is four bytes)
28 #define GAP_MAX 3
29
30 // Erase the rest of a row (ESC K, two bytes plus maybe an address) once at
31 // least this many cells there would otherwise have to be sent as spaces
32 #define EOL_MIN 4
33
34 // Clear the screen (ESC E) once at least this many cells are to be blanked
35 #define CLEAR_MIN 600
36
37 static uint8_t buf[SCR_W * SCR_H];   // what is being drawn
38 static uint8_t scr[SCR_W * SCR_H];   // what the terminal shows
39 static uint8_t dirty[SCR_H];         // rows of buf touched since the last flush
40 static uint8_t changed[SCR_H];       // rows flush() found differing
41
42 // Row starts: sccz80 multiplies through a library call, and put() runs a
43 // couple of thousand times a screen on a 2 MHz Z80
44 static const uint16_t rowOff[SCR_H] = {
45     0 * SCR_W, 1 * SCR_W, 2 * SCR_W, 3 * SCR_W, 4 * SCR_W,
46     5 * SCR_W, 6 * SCR_W, 7 * SCR_W, 8 * SCR_W, 9 * SCR_W,
47     10 * SCR_W, 11 * SCR_W, 12 * SCR_W, 13 * SCR_W, 14 * SCR_W,
48     15 * SCR_W, 16 * SCR_W, 17 * SCR_W, 18 * SCR_W, 19 * SCR_W,
49     20 * SCR_W, 21 * SCR_W, 22 * SCR_W, 23 * SCR_W, 24 * SCR_W,
50 };
51
52 // What the terminal is doing now: cursor, reverse video, graphics mode.
53 // CUR_UNKNOWN forces the next cell to set it.
54 static uint8_t curX, curY, curRev, curGfx;
55
56 /*
57  * Raw console I/O
58  */
59
60 #ifdef __HDOS__
61
62 // HDOS system calls: RST 38H, then the function byte. HDOS is 8080 code and
63 // leaves IX and IY alone.
64 void conOut(uint8_t c) __z88dk_fastcall __naked
65 {
66 #asm
67     ld      a,l
68     rst     38h
69     defb    2           ; .SCOUT
70     ret
71 #endasm
72 }
73
74 int conIn(void) __naked
75 {
```

```
#asm
    rst     38h
    defb    1           ; .SCIN - carry set when nothing is waiting
    ld      hl,-1
    ret     c
    ld      l,a
    ld      h,0
    ret
#endasm
}

// .CONSL: A = index of the console byte, B = the bits to set, C = the mask of
// bits to change
uint8_t h89ConslIndex, h89ConslBits, h89ConslMask;

static void consl(void) __naked
{
#asm
    ld      a,(_h89ConslBits)
    ld      b,a
    ld      a,(_h89ConslMask)
    ld      c,a
    ld      a,(_h89ConslIndex)
    rst     38h
    defb    6           ; .CONSL
    ret
#endasm
}

static void conslSet(uint8_t index, uint8_t bits, uint8_t mask)
{
    h89ConslIndex = index;
    h89ConslBits = bits;
    h89ConslMask = mask;
    consl();
}

// The five console cells (S.CSLMD..S.CONFL) live at 040.326A in HDOS 2.0's
// system values (HOSDEF: S.VAL at 040.277A)
#define S_CSLMD ((volatile uint8_t *)0x20D6)
#define I_CSLMD 0
#define I_CONTY 1
#define I_CUSOR 2
#define I_CONWI 3
#define I_CONFL 4

#define CSL_ECH 0x80 // suppress echo
#define CSL_RAW 0x04 // raw mode I/O (no CTL-C/S/O/P processing)
#define CSL_WRP 0x02 // wrap lines at width
#define CSL_CHR 0x01 // character mode
#define CTP_MLI 0x20 // map lower case to upper on input
#define CTP_MLO 0x10 // map lower case to upper on output
#define CTP_BKM 0x02 // map backspace to rubout on input
#define CO_FLG 0x01  // CTL-O (discard output) flag
#define CS_FLG 0x80  // CTL-S (hold output) flag

static uint8_t savedConsole[5];

static void osInit(void)
{
    memcpy(savedConsole, (void *)S_CSLMD, sizeof(savedConsole));

    // Raw character mode with no echo, no wrap and no width (HDOS counts
    // escape sequences as printing columns), and no case mapping: the
    // graphics characters are sent as lower case letters.
    conslSet(I_CSLMD, CSL_ECH | CSL_RAW | CSL_CHR, CSL_ECH | CSL_RAW | CSL_WRP | CSL_CHR);
    conslSet(I_CONTY, 0, CTP_MLI | CTP_MLO | CTP_BKM);
    conslSet(I_CONWI, 255, 0xFF);
    conslSet(I_CONFL, 0, CO_FLG | CS_FLG);
}

static void osExit(void)
{
    conslSet(I_CSLMD, savedConsole[I_CSLMD], 0xFF);
    conslSet(I_CONTY, savedConsole[I_CONTY], 0xFF);
    conslSet(I_CONWI, savedConsole[I_CONWI], 0xFF);
    conslSet(I_CUSOR, 0, 0xFF); // ESC z homed the cursor
}

#else /* CP/M */

// The BIOS jump table: (0001) points at its WBOOT entry, base+3. CONST is at
// +3 from there, CONIN +6, CONOUT +9. The BIOS may use any register, so IX
// and IY are kept for the C side.
void conOut(uint8_t c) __z88dk_fastcall __naked
{
#asm
    EXTERN  l_dcal
    push    ix
    push    iy
    ld      c,1
    ld      hl,(1)
    ld      de,9
    add     hl,de
    call    l_dcal      ; CONOUT
    pop     iy
    pop     ix
    ret
#endasm
}

int conIn(void) __naked
{
#asm
    EXTERN  l_dcal
    push    ix
    push    iy
    ld      hl,(1)
    ld      de,3
    add     hl,de
    call    l_dcal      ; CONST: A = FF when a character is waiting
    or      a
    ld      hl,-1
    jr      z,conIn_done
    ld      hl,(1)
    ld      de,6
    add     hl,de
    call    l_dcal      ; CONIN
    ld      l,a
    ld      h,0
conIn_done:
    pop     iy
    pop     ix
    ret
#endasm
}

static void osInit(void)
{
}

static void osExit(void)
{
}

#endif /* __HDOS__ */

static void esc(uint8_t c)
{
    conOut(ESC);
    conOut(c);
}

static void forget(void)
{
    curX = curY = curRev = curGfx = CUR_UNKNOWN;
}

static void moveTo(uint8_t x, uint8_t y)
{
    if (x != curX || y != curY)
    {
        esc('Y');
        conOut(32 + y);
        conOut(32 + x);
        curX = x;
        curY = y;
    }
}

/// @brief Graphics mode a cell needs: 1 on, 0 off, 2 either
static uint8_t gfxNeed(uint8_t code)
{
    code &= 0x7F;
    if (code < 0x20 || code == 0x7F)
        return 1;
    // Graphics mode only remaps 0x5E-0x7E: everything below prints the same
    // either way
    return code >= 0x5E ? 0 : 2;
}

/// @brief Send a cell at the cursor, setting the modes it needs
static void sendCell(uint8_t code)
{
    static uint8_t g, need;

    if ((code & REV) != curRev)
    {
        curRev = code & REV;
        esc(curRev ? 'p' : 'q');
    }

    need = gfxNeed(code);
    if (need != 2 && need != curGfx)
    {
        curGfx = need;
```

```
        esc(need ? 'F' : 'G');
    }

    g = code & 0x7F;
    if (need == 1)
    {
        // Graphics characters: ROM 0x00-0x1E are ` through *, 0x1F is _,
        // the bullet is ^
        if (g == 0x7F)
            g = '^';
        else if (g == 0x1F)
            g = '_';
        else
            g += 0x60;
    }

    conOut(g);

    // In discard-at-end-of-line mode the cursor stays on the last column
    curX = (curX == SCR_W - 1) ? CUR_UNKNOWN : curX + 1;
}

/// @brief Can the cells from the cursor up to x be re-sent with no mode
/// change? They already show on screen, so this only moves the cursor.
static bool cheapGap(const uint8_t *row, uint8_t x)
{
    static uint8_t cx, need;

    if (curX == CUR_UNKNOWN || curX >= x || x - curX > GAP_MAX)
        return false;

    for (cx = curX; cx < x; cx++)
    {
        if ((row[cx] & REV) != curRev)
            return false;
        need = gfxNeed(row[cx]);
        if (need != 2 && need != curGfx)
            return false;
    }
    return true;
}

static void clearTerminal(void)
{
    // Home first: ESC E on the 25th line clears only that line
    curX = curY = CUR_UNKNOWN;
    moveTo(0, 0);
    esc('E');
    memset(scr, ' ', SCR_W * STATUS_ROW);
}

static void flushRow(uint8_t y)
{
    static uint8_t *b, *s;
    static uint8_t x, last, stale;

    b = buf + rowOff[y];
    s = scr + rowOff[y];

    // The row's last non-blank cell, and how many cells past it still show
    // something on screen
    last = SCR_W;
    while (last && b[last - 1] == ' ')
        last--;
    stale = 0;
    for (x = last; x < SCR_W; x++)
        if (s[x] != ' ')
            stale++;
    if (stale < EOL_MIN)
        last = SCR_W;

    for (x = 0; x < last; x++)
    {
        if (b[x] == s[x])
            continue;

        if (y == curY && cheapGap(b, x))
        {
            while (curX < x)
                sendCell(b[curX]);
        }
        else
            moveTo(x, y);

        sendCell(b[x]);
        s[x] = b[x];
    }

    if (last < SCR_W)
    {
        moveTo(last, y);
        esc('K');
        memset(s + last, ' ', SCR_W - last);
    }
}

/// @brief Cells of the changed rows that are to go blank
static uint16_t blanksToSend(void)
{
    static uint8_t y, n;
    static uint8_t *b, *s;
    static uint16_t blanks;

    blanks = 0;
    for (y = 0; y < STATUS_ROW; y++)
    {
        if (!changed[y])
            continue;
        b = buf + rowOff[y];
        s = scr + rowOff[y];
        for (n = SCR_W; n; n--)
        {
            if (*b == ' ' && *s != ' ')
                blanks++;
            b++;
            s++;
        }
    }
    return blanks;
}

void flush(void)
{
    static uint8_t y, rows;

    // Which rows really differ (the shared code repaints the whole table
    // every poll, so most dirty rows come out the same)
    rows = 0;
    for (y = 0; y < SCR_H; y++)
    {
        changed[y] = dirty[y] && memcmp(buf + rowOff[y], scr + rowOff[y], SCR_W);
        dirty[y] = 0;
        rows += changed[y];
    }
    if (!rows)
        return;

    // A screen change: clear and draw rather than overwrite with spaces
    if (rows >= STATUS_ROW / 2 && blanksToSend() >= CLEAR_MIN)
    {
        clearTerminal();
        for (y = 0; y < STATUS_ROW; y++)
            changed[y] = memcmp(buf + rowOff[y], scr + rowOff[y], SCR_W) != 0;
    }

    for (y = 0; y < SCR_H; y++)
        if (changed[y])
            flushRow(y);
}

/*
 * Screen
 */

void termInit(void)
{
    osInit();

    esc('G');      // graphics mode off
    esc('q');      // reverse video off
    esc('x');      // cursor off
    conOut('5');
    esc('w');      // discard (no wrap) at the end of a line
    esc('t');      // keypad shifted: the unshifted keypad sends ESC A-D
    esc('x');      // the 25th line, for the status bar
    conOut('1');
    forget();
    curGfx = 0;
    curRev = 0;

    clearTerminal();
    moveTo(0, STATUS_ROW);
    esc('K');
    memset(scr + SCR_W * STATUS_ROW, ' ', SCR_W);
    memset(buf, ' ', sizeof(buf));
    memset(dirty, 0, sizeof(dirty));
}

void termExit(void)
{
    static uint16_t t;

    // Back to the power-up configuration the DIP switches set: modes,
    // cursor, keypad, 25th line off, and a clear screen. The terminal needs
    // a moment.
    esc('z');
```

```
448     t = TICKS;
449     while ((uint16_t)(TICKS - t) < 100)
450         ;
451     osExit();
452 }
453
454 void termBell(void)
455 {
456     conOut(0x07);
457 }
458
459 void put(uint8_t x, uint8_t y, uint8_t code)
460 {
461     if (x >= SCR_W || y >= SCR_H)
462         return;
463
464     buf[rowOff[y] + x] = code;
465     dirty[y] = 1;
466 }
467
468 void fill(uint8_t x, uint8_t y, uint8_t w, uint8_t code)
469 {
470     if (x >= SCR_W || y >= SCR_H)
471         return;
472     if (w > SCR_W - x)
473         w = SCR_W - x;
474
475     memset(buf + rowOff[y] + x, code, w);
476     dirty[y] = 1;
477 }
478
479 void blankRows(uint8_t y, uint8_t n)
480 {
481     memset(buf + rowOff[y], ' ', (uint16_t)n * SCR_W);
482     memset(dirty + y, 1, n);
483 }
484
485 uint8_t peek(uint8_t x, uint8_t y)
486 {
487     if (x >= SCR_W || y >= SCR_H)
488         return ' ';
489     return buf[rowOff[y] + x];
490 }
491
492 #endif /* __H89__ */
```

SRC/H89/CONSOLE.H — 100 LINES

```
  1 #ifndef H89_CONSOLE_H
  2 #define H89_CONSOLE_H
  3
  4 /*
  5   H19 terminal screen for the H89 port.
  6
  7   The screen is drawn by cell codes, one byte each: bit 7 is reverse video and
  8   the low seven bits are the H19's own character ROM index (444-29) --
  9   0x00-0x1F the graphics characters, 0x7F the graphics bullet, 0x20-0x7E
 10   ASCII -- the same form the terminal keeps its screen in.
 11
 12   Drawing goes into a back buffer; flush() sends the terminal only the cells
 13   that differ from what it already shows, with as few cursor-address (ESC Y),
 14   reverse-video (ESC p/q) and graphics-mode (ESC F/G) changes as it can, and
 15   erases (ESC K, ESC E) where they are cheaper than spaces. The console runs at
 16   9600 baud, and the shared game code repaints the whole table every poll.
 17
 18   Row 24 is the H19's 25th line, which termInit() turns on for the status bar.
 19 */
 20
 21 #include <stdint.h>
 22 #include <stdbool.h>
 23
 24 #define SCR_W 80
 25 #define SCR_H 25
 26 #define STATUS_ROW 24
 27
 28 #define REV 0x80
 29
 30 // H19 graphics characters: ROM index, shown in graphics mode (ESC F) for the
 31 // character in the comment
 32 #define G_VLINE      0x00 // `  vertical line
 33 #define G_HLINE      0x01 // a  horizontal line
 34 #define G_CROSS      0x02 // b  cross
 35 #define G_CORNER_UR  0x03 // c  upper right corner
 36 #define G_CORNER_LR  0x04 // d  lower right corner
 37 #define G_CORNER_LL  0x05 // e  lower left corner
 38 #define G_CORNER_UL  0x06 // f  upper left corner
 39 #define G_PLUSMINUS  0x07 // g
 40 #define G_ARROW_R    0x08 // h  right arrow
 41 #define G_CHECKER    0x09 // i  checkerboard
 42 #define G_DIVIDE     0x0A // j
 43 #define G_ARROW_D    0x0B // k  down arrow
 44 #define G_QUAD_LR    0x0C // l  lower right quadrant
 45 #define G_QUAD_LL    0x0D // m  lower left quadrant
 46 #define G_QUAD_UL    0x0E // n  upper left quadrant
 47 #define G_QUAD_UR    0x0F // o  upper right quadrant
 48 #define G_HALF_TOP   0x10 // p  upper half
 49 #define G_HALF_RIGHT 0x11 // q  right half
 50 #define G_TRI_UL     0x12 // r  upper left triangle
 51 #define G_TEE_DOWN   0x13 // s  T, stem down
 52 #define G_TEE_LEFT   0x14 // t  T, stem left
 53 #define G_TEE_UP     0x15 // u  T, stem up
 54 #define G_TEE_RIGHT  0x16 // v  T, stem right
 55 #define G_X          0x17 // w  diagonal cross
 56 #define G_SLASH      0x18 // x  /
 57 #define G_BACKSLASH  0x19 // y  backslash
 58 #define G_BAR_TOP    0x1A // z  top line
 59 #define G_BAR_BOTTOM 0x1B // {  bottom line
 60 #define G_BAR_LEFT   0x1C // |  left bar
 61 #define G_BAR_RIGHT  0x1D // }  right bar
 62 #define G_PILCROW    0x1E // ~
 63 #define G_TRI_UR     0x1F // _  upper right triangle
 64 #define G_BULLET     0x7F // ^  bullet
 65
 66 /// @brief Set up the terminal (and HDOS's console driver) for the game
 67 void termInit(void);
 68
 69 /// @brief Put the terminal (and HDOS's console driver) back as they were
 70 void termExit(void);
 71
 72 /// @brief Set one cell of the back buffer
 73 void put(uint8_t x, uint8_t y, uint8_t code);
 74
 75 /// @brief Set a run of w cells of the back buffer to one code
 76 void fill(uint8_t x, uint8_t y, uint8_t w, uint8_t code);
 77
 78 /// @brief Blank n whole rows of the back buffer from row y
 79 void blankRows(uint8_t y, uint8_t n);
 80
 81 /// @brief The back buffer's cell at x,y (a space off screen)
 82 uint8_t peek(uint8_t x, uint8_t y);
 83
 84 /// @brief Send the terminal whatever differs from the back buffer
 85 void flush(void);
 86
 87 /// @brief Ring the terminal's bell
 88 void termBell(void);
 89
 90 /// @brief Raw console byte, or -1 when none is waiting
 91 int conIn(void);
 92
 93 /// @brief The H89's 2 ms clock tick counter (HDOS .TICCNT / the CP/M BIOS's)
 94 #ifdef __HDOS__
 95 #define TICKS (*(volatile uint16_t *)0x201B)
 96 #else
 97 #define TICKS (*(volatile uint16_t *)0x000B)
 98 #endif
 99
100 #endif /* H89_CONSOLE_H */
```

SRC/H89/GRAPHICS.C — 404 LINES

```
  1 #ifdef __H89__
  2
  3 /**
  4  * @brief   Graphics - Heathkit H89 (H19 terminal, 80x24 + 25th line)
  5  * @author  Thomas Cherryhomes
  6  * @email   thom dot cherryhomes at gmail dot com
  7  * @license gpl v. 3, see LICENSE for details
  8  *
  9  * Everything is drawn with the H19's own characters: its graphics set (line
 10  * drawing, diagonals, triangles, quadrant blocks, the checkerboard, the
 11  * bullet) and reverse video. The ROM has no card suits, so each suit is a
 12  * 2x2 picture made of triangles and quadrants:
 13  *
 14  *   spades â□¢â□£   hearts â□□â□□   diamonds â□¢â□£   clubs â□□â□□
 15  *          â□□â□□           â□¥â□¤             â□¥â□¤          â□□â□□
 16  *
 17  * Cards are 6x5 -- rank top left, suit below -- and backs are checkerboard.
 18  * The table is an oval rail of diagonals and edge lines (resetScreen), the
 19  * other screens a frame of the same with an ace in each corner (drawBorder).
 20  * Chips are bullets. The status bar is the 25th line, where the selected
 21  * move is shown in reverse video; drawLine/hideLine, which are underlines
 22  * elsewhere, reverse the text above them here (the active player's name, the
 23  * table list's header). The H19 is monochrome, so the colour toggle draws
 24  * hearts and diamonds in reverse video instead of red.
 25  *
 26  * Drawing goes into console.c's back buffer and flush() sends only what
 27  * changed. Unbuffered (disableDoubleBuffer), each call flushes when it is
 28  * done, so dealing and flipping show card by card.
 29  */
 30
 31 #include <stdint.h>
 32 #include <stdbool.h>
 33 #include <string.h>
```

```
#include "console.h"
#include "logo.h"
#include "../platform-specific/graphics.h"

#define CARD_W 6
#define CARD_H 5

// The table rail: an oval from row RAIL_TOP to RAIL_BOTTOM, column RAIL_LEFT
// to RAIL_RIGHT, its corners cut by two diagonal steps
#define RAIL_TOP 7
#define RAIL_BOTTOM 17
#define RAIL_LEFT 16
#define RAIL_RIGHT 63

bool always_render_full_cards = 0;
unsigned char colorMode = 0;

static bool buffered;

// Suit pictures, upper left, upper right, lower left, lower right
static const uint8_t suitSpade[4] = {REV | G_TRI_UL, REV | G_TRI_UR, G_QUAD_UR, G_QUAD_UL};
static const uint8_t suitHeart[4] = {REV | G_QUAD_UR, REV | G_QUAD_UL, G_TRI_UR, G_TRI_UL};
static const uint8_t suitDiamond[4] = {REV | G_TRI_UL, REV | G_TRI_UR, G_TRI_UR, G_TRI_UL};
static const uint8_t suitClub[4] = {G_QUAD_LR, G_QUAD_LL, G_BULLET, G_BULLET};

// XOR'd into every cell of the card being drawn: REV for a "red" card
static uint8_t cardRev;

/// @brief Unbuffered, send what this call drew
static void done(void)
{
    if (!buffered)
        flush();
}

static void blankTable(void)
{
    blankRows(0, STATUS_ROW);
}

void enableDoubleBuffer()
{
    buffered = true;
}

void disableDoubleBuffer()
{
    flush();
    buffered = false;
}

void drawBuffer()
{
    flush();
}

// Call to clear the screen to an empty table. Never flushed here: it is
// always followed by drawBorder() or a full table.
void resetScreen()
{
    static uint8_t y;

    blankTable();

    // The rail
    put(RAIL_LEFT + 1, RAIL_TOP, G_SLASH);
    fill(RAIL_LEFT + 2, RAIL_TOP, RAIL_RIGHT - RAIL_LEFT - 3, G_BAR_TOP);
    put(RAIL_RIGHT - 1, RAIL_TOP, G_BACKSLASH);
    put(RAIL_LEFT, RAIL_TOP + 1, G_SLASH);
    put(RAIL_RIGHT, RAIL_TOP + 1, G_BACKSLASH);
    for (y = RAIL_TOP + 2; y < RAIL_BOTTOM - 1; y++)
    {
        put(RAIL_LEFT, y, G_BAR_LEFT);
        put(RAIL_RIGHT, y, G_BAR_RIGHT);
    }
    put(RAIL_LEFT, RAIL_BOTTOM - 1, G_BACKSLASH);
    put(RAIL_RIGHT, RAIL_BOTTOM - 1, G_SLASH);
    put(RAIL_LEFT + 1, RAIL_BOTTOM, G_BACKSLASH);
    fill(RAIL_LEFT + 2, RAIL_BOTTOM, RAIL_RIGHT - RAIL_LEFT - 3, G_BAR_BOTTOM);
    put(RAIL_RIGHT - 1, RAIL_BOTTOM, G_SLASH);

    // A rule above the status line
    fill(0, STATUS_ROW - 1, SCR_W, G_BAR_BOTTOM);
}

void clearStatusBar()
{
    fill(0, STATUS_ROW, SCR_W, ' ');
    done();
}

void drawStatusTextAt(unsigned char x, const char* s)
{
    static uint8_t c;

    // Keep the reverse video the move cursor laid on these cells
    while ((c = (uint8_t)*s++) && x < SCR_W)
    {
        if (c >= 0x20 && c < 0x7F)
        {
            put(x, STATUS_ROW, c | (peek(x, STATUS_ROW) & REV));
            x++;
        }
    }
    done();
}

void drawStatusText(const char* s)
{
    fill(0, STATUS_ROW, SCR_W, ' ');
    drawStatusTextAt(0, s);
}

void drawStatusTimer()
{
    // An hourglass after the countdown
    put(SCR_W - 1, STATUS_ROW, REV | G_X);
    done();
}

void drawText(unsigned char x, unsigned char y, const char* s)
{
    static uint8_t c;

    while ((c = (uint8_t)*s++) && x < SCR_W)
    {
        if (c >= 0x20 && c < 0x7F)
            put(x++, y, c);
    }
    done();
}

static void cardCell(uint8_t x, uint8_t y, uint8_t code)
{
    put(x, y, code ^ cardRev);
}

void drawCard(unsigned char x, unsigned char y, unsigned char partial, const char* s, unsigned
    char isHidden)
{
    static const uint8_t *suit;
    static uint8_t i, r;

    // Every Hold'em caller passes FULL_CARD, and the table is repainted in
    // full each time, so CARD_CLEAR_NEXT's slot is already blank
    (void)partial;
    (void)isHidden;

    cardRev = 0;
    switch (s[1])
    {
    case 's':
        suit = suitSpade;
        break;
    case 'h':
        suit = suitHeart;
        cardRev = colorMode ? REV : 0;
        break;
    case 'd':
        suit = suitDiamond;
        cardRev = colorMode ? REV : 0;
        break;
    case 'c':
        suit = suitClub;
        break;
    default:
        suit = 0; // face down
        break;
    }

    // Border
    cardCell(x, y, G_CORNER_UL);
    cardCell(x + CARD_W - 1, y, G_CORNER_UR);
    cardCell(x, y + CARD_H - 1, G_CORNER_LL);
    cardCell(x + CARD_W - 1, y + CARD_H - 1, G_CORNER_LR);
    for (i = 1; i < CARD_W - 1; i++)
    {
        cardCell(x + i, y, G_HLINE);
        cardCell(x + i, y + CARD_H - 1, G_HLINE);
    }
    for (r = 1; r < CARD_H - 1; r++)
    {
        cardCell(x, y + r, G_VLINE);
        cardCell(x + CARD_W - 1, y + r, G_VLINE);
    }

```

```
    if (!suit)
    {
        for (r = 1; r < CARD_H - 1; r++)
            for (i = 1; i < CARD_W - 1; i++)
                cardCell(x + i, y + r, G_CHECKER);
        done();
        return;
    }

    // Rank, top left: 't' is 10
    for (i = 1; i < CARD_W - 1; i++)
        cardCell(x + i, y + 1, ' ');
    switch (s[0])
    {
    case 't':
        cardCell(x + 1, y + 1, '1');
        cardCell(x + 2, y + 1, '0');
        break;
    case 'j':
    case 'q':
    case 'k':
    case 'a':
        cardCell(x + 1, y + 1, s[0] - 'a' + 'A');
        break;
    default:
        cardCell(x + 1, y + 1, s[0]);
        break;
    }

    // Suit, centred below
    for (r = 2; r < CARD_H - 1; r++)
    {
        cardCell(x + 1, y + r, ' ');
        cardCell(x + 4, y + r, ' ');
    }
    cardCell(x + 2, y + 2, suit[0]);
    cardCell(x + 3, y + 2, suit[1]);
    cardCell(x + 2, y + 3, suit[2]);
    cardCell(x + 3, y + 3, suit[3]);

    done();
}

void drawChip(unsigned char x, unsigned char y)
{
    put(x, y, G_BULLET);
    done();
}

void drawBlank(unsigned char x, unsigned char y)
{
    put(x, y, ' ');
    done();
}

/// @brief Reverse (or not) the w cells above row y, or row y itself on the
/// status line: where the other platforms underline, the H19 reverses
static void reverseRun(uint8_t x, uint8_t y, uint8_t w, bool on)
{
    static uint8_t c;

    if (y != STATUS_ROW)
    {
        if (!y)
            return;
        y--;
    }

    while (w-- && x < SCR_W)
    {
        c = peek(x, y);
        put(x, y, on ? (c | REV) : (c & ~REV));
        x++;
    }
    done();
}

void drawLine(unsigned char x, unsigned char y, unsigned char w)
{
    reverseRun(x, y, w, true);
}

void hideLine(unsigned char x, unsigned char y, unsigned char w)
{
    reverseRun(x, y, w, false);
}

void drawBox(unsigned char x, unsigned char y, unsigned char w, unsigned char h)
{
    static uint8_t r;

    put(x, y, G_CORNER_UL);
    fill(x + 1, y, w, G_HLINE);
    put(x + w + 1, y, G_CORNER_UR);
    for (r = 1; r <= h; r++)
    {
        put(x, y + r, G_VLINE);
        fill(x + 1, y + r, w, ' ');
        put(x + w + 1, y + r, G_VLINE);
    }
    put(x, y + h + 1, G_CORNER_LL);
    fill(x + 1, y + h + 1, w, G_HLINE);
    put(x + w + 1, y + h + 1, G_CORNER_LR);
    done();
}

void drawBorder()
{
    static uint8_t r;
    static bool wasBuffered;

    // Whatever resetScreen() drew goes: the frame replaces the table. Batch
    // the lot so an unbuffered caller never sees the rail go out and back.
    wasBuffered = buffered;
    buffered = true;

    blankTable();

    // A frame round the screen, its corners cut by diagonals like the rail's
    put(0, 0, G_SLASH);
    fill(1, 0, SCR_W - 2, G_BAR_TOP);
    put(SCR_W - 1, 0, G_BACKSLASH);
    for (r = 1; r < STATUS_ROW - 1; r++)
    {
        put(0, r, G_BAR_LEFT);
        put(SCR_W - 1, r, G_BAR_RIGHT);
    }
    put(0, STATUS_ROW - 1, G_BACKSLASH);
    fill(1, STATUS_ROW - 1, SCR_W - 2, G_BAR_BOTTOM);
    put(SCR_W - 1, STATUS_ROW - 1, G_SLASH);

    // An ace of each suit in the corners
    drawCard(2, 1, FULL_CARD, "as", 0);
    drawCard(SCR_W - 2 - CARD_W, 1, FULL_CARD, "ah", 0);
    drawCard(2, STATUS_ROW - 1 - CARD_H, FULL_CARD, "ad", 0);
    drawCard(SCR_W - 2 - CARD_W, STATUS_ROW - 1 - CARD_H, FULL_CARD, "ac", 0);

    buffered = wasBuffered;
    done();
}

void drawLogo()
{
    static const uint8_t *p;
    static uint8_t ly, lx, n;

    // TEXAS / HOLD'EM in quadrant-block letters (src/h89/mklogo.py)
    p = logo;
    while (*p != 0xFF)
    {
        ly = *p++;
        lx = *p++;
        n = *p++;
        while (n--)
            put(lx++, ly, *p++);
    }

    // and a rule with the maker's name under it
    fill(WIDTH / 2 - 17, 10, 10, G_HLINE);
    drawText(WIDTH / 2 - 6, 10, "F U J I N E T");
    fill(WIDTH / 2 + 8, 10, 10, G_HLINE);
    done();
}

void initGraphics()
{
    termInit();
}

void resetGraphics()
{
    termExit();
}

unsigned char cycleNextColor()
{
    colorMode = !colorMode;
    return colorMode;
}

void setColorMode(unsigned char mode)
{
    colorMode = mode ? 1 : 0;
}

#endif /* __H89__ */
```

SRC/H89/INPUT.C 148 LINES

```
#ifdef __H89__

/**
 * @brief   Input - Heathkit H89 (H19 keyboard)
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * The H19 sends its keypad and function keys as escape sequences: with the
 * keypad shifted (termInit sends ESC t) the arrows are ESC A-D, f1-f5 are
 * ESC S-W, and BLUE/RED/GRAY are ESC P-R. Those arrive back to back at 9600
 * baud, so an ESC with nothing behind it within a few ticks is the ESC key
 * itself. Both OSes buffer the keyboard on its interrupt (HDOS's console
 * driver; the CP/M BIOS's 40-byte RST 3 buffer), so nothing is lost between
 * polls. (fujinet-battleship's src/h89/input.c, behind getPlatformKey().)
 */

#include <stdint.h>
#include <string.h>
#include "vars.h"
#include "console.h"

#define ESC_WAIT_TICKS 10 // 20 ms

// Decoded keys not yet returned
static uint8_t pending[4];
static uint8_t pendingCount;

static void push(uint8_t k)
{
    if (pendingCount < sizeof(pending))
        pending[pendingCount++] = k;
}

/// @brief Plain keys: DELETE erases like BACKSPACE, LINE FEED enters
static uint8_t plain(uint8_t c)
{
    if (c == 0x7F)
        return KEY_BACKSPACE;
    if (c == 0x0A)
        return KEY_RETURN;
    return c;
}

/// @brief Wait briefly for the byte after an ESC; -1 if none comes
static int escFollower(void)
{
    static uint16_t t;
    static int c;

    t = TICKS;
    do
    {
        c = conIn();
        if (c >= 0)
            return c;
    } while ((uint16_t)(TICKS - t) < ESC_WAIT_TICKS);

    return -1;
}

static void decode(uint8_t c)
{
    static int n;

    if (c != KEY_ESCAPE)
    {
        push(plain(c));
        return;
    }

    n = escFollower();
    switch (n)
    {
    case 'A':
        push(KEY_UP_ARROW);
        break;
    case 'B':
        push(KEY_DOWN_ARROW);
        break;
    case 'C':
        push(KEY_RIGHT_ARROW);
        break;
    case 'D':
        push(KEY_LEFT_ARROW);
        break;
    case 'P': // BLUE
    case 'Q': // RED
    case 'R': // GRAY
    case 'S': // f1
    case 'T': // f2
    case 'U': // f3
    case 'V': // f4
    case 'W': // f5
        // The function keys open the menu too
        push(KEY_ESCAPE);
        break;
    case 'H': // HOME
    case 'J': // ERASE
    case 'E': // SHIFT ERASE
    case 'L': // IL
    case 'M': // DL
    case 'N': // DC
    case '@': // IC
        // The keypad's other keys do nothing here
        break;
    default:
        // A lone ESC, or ESC then an ordinary key
        push(KEY_ESCAPE);
        if (n >= 0 && n != KEY_ESCAPE)
            push(plain((uint8_t)n));
        else if (n == KEY_ESCAPE)
            push(KEY_ESCAPE);
        break;
    }
}

void initPlatformKeyboardInput(void)
{
    pendingCount = 0;
}

int getPlatformKey(void)
{
    static int c;
    static uint8_t k;

    while (!pendingCount)
    {
        c = conIn();
        if (c < 0)
            return 0;
        decode((uint8_t)c);
    }

    k = pending[0];
    pendingCount--;
    memmove(pending, pending + 1, pendingCount);
    return k;
}

unsigned char readJoystick(void)
{
    // Keyboard only: the arrows and RETURN/SPACE come through getPlatformKey()
    return 0;
}

#endif /* __H89__ */
```

SRC/H89/NETWORK.C 36 LINES

```
#ifdef __H89__

/**
 * @brief   Network functions - Heathkit H89 (H89 FujiNet card)
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 */

#include <stdint.h>
#include <fujinet-network.h>

static uint8_t initialized = 0;

uint8_t getResponse(char *url, unsigned char *buffer, uint16_t max_len)
{
    int16_t count;

    if (!initialized) {
        if (network_init() != FN_ERR_OK)
            return 0;
        initialized = 1;
    }

    // Don't read from a channel that never opened - network_read() would
    // otherwise poll status on a dead unit and hand back whatever it got.
    if (network_open(url, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE) != FN_ERR_OK)
        return 0;

    count = network_read(url, buffer, max_len);
    network_close(url);
```

```

    return count > 0;
}

#endif /* __H89__ */
```

SRC/H89/SOUND.C 43 LINES

```
#ifdef __H89__

/**
 * @brief   Sound - Heathkit H89
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * The only sound an H89 makes is the H19's bell, so it marks the moments
 * that need the player's attention -- sitting down, their turn, a hand's
 * result -- and the rest stay quiet.
 */

#include <stdint.h>
#include "console.h"

void initSound() {}

void soundJoinGame()
{
    termBell();
}

void soundMyTurn()
{
    termBell();
}

void soundGameDone()
{
    termBell();
}

void soundDealCard() {}
void soundTick() {}
void soundPlayerJoin() {}
void soundPlayerLeft() {}
void soundSelectMove() {}
void soundCursor() {}
void soundCursorInvalid() {}
void soundTakeChip(uint16_t counter) { (void)counter; }

#endif /* __H89__ */
```

SRC/H89/UTIL.C 57 LINES

```
#ifdef __H89__

/**
 * @brief   Utility Functions - Heathkit H89
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * The H89's clock interrupts every 2 ms, and both HDOS and the CP/M BIOS
 * count the ticks in a word (console.h's TICKS). The shared code counts in
 * 60ths of a second, so a frame here is 25/3 ticks: 8, 8, then 9.
 */

#include <stdint.h>
#include <stdlib.h>
#include "console.h"
#include "../platform-specific/graphics.h"
#include "../platform-specific/util.h"

static uint16_t lastFrame, timerBase;
static uint8_t framePhase;

void waitvsync(void)
{
    static uint8_t len;

    // At least a frame since the last one returned. Drawing over the 9600
    // baud console can take longer than that; the game just runs at the
    // pace of the terminal then.
    len = (framePhase == 2) ? 9 : 8;
    while ((uint16_t)(TICKS - lastFrame) < len)
        ;
    lastFrame = TICKS;
    framePhase = (framePhase == 2) ? 0 : framePhase + 1;
}

void resetTimer(void)
{
    timerBase = TICKS;
}

int getTime(void)
{
    // 60ths of a second since resetTimer(); the tick word wraps after 131 s,
    // longer than any move clock
    return (int)(((uint32_t)(uint16_t)(TICKS - timerBase) * 3) / 25);
}

void quit(void)
{
    // There is no H89 lobby to load: back to HDOS or CP/M, with the
    // terminal as it was
    resetGraphics();
    exit(0);
}

#endif /* __H89__ */
```

SRC/H89/VARS.C 31 LINES

```
/**
 * @brief   Global variables for the Heathkit H89
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 */

#ifdef __H89__

// 80x24 seat layout around the oval rail (src/h89/graphics.c, rows 7-17,
// columns 16-63). Cards are 6x5 with a column between the two hole cards, so
// a hand is 13 wide; a right-side seat's X is its right card, the other card
// 7 to the left. Top seats (cards rows 2-6) and the bottom-centre seat (YOU,
// rows 18-22) sit outside the rail, the side seats left and right of it.
// Each seat's move and bet lie on the felt just inside the rail in front of
// it, except the top-centre seat's, which sit beside its cards: the community
// board fills the felt below it. The pot and street label are below the
// board.
unsigned char playerXMaster[] = { 33, 2, 2, 5, 33, 69, 72, 72 };
unsigned char playerYMaster[] = { 18, 17, 10, 2, 2, 2, 10, 17 };

char playerDirMaster[] = { 1, 1, 1, 1, 1, -1, -1, -1 };
char playerBetXMaster[] = { 12, 16, 16, 13, 14, -7, -10, -10 };
char playerBetYMaster[] = { -3, -2, 1, 6, 1, 6, 1, -2 };

//                                  2               3               4
char playerCountIndex[] = {0,4,0,0,0,0,0,0, 0,2,6,0,0,0,0,0, 0,2,4,6,0,0,0,0,
// 5               6               7               8
    0,2,3,5,6,0,0,0, 0,2,3,4,5,6,0,0,  0,2,3,4,5,6,7,0, 0,1,2,3,4,5,6,7};

#endif /* __H89__ */
```

SRC/H89/VARS.H 101 LINES

```
#ifdef __H89__

#ifndef KEYMAP_H
#define KEYMAP_H

// Screen dimensions for platform: the H19 terminal's 80x24, plus its 25th
// line (row 24 = HEIGHT-1), which src/h89/console.c turns on for the status
// bar. Not SINGLE_BUFFER: the table is repainted in full every poll, into a
// back buffer, and only the cells that differ go down the 9600 baud line.
#define WIDTH 80
#define HEIGHT 25

// Cards are 6x5 (src/h89/graphics.c), side by side with a column between.
// The community board is centred at rows 9-13, so the pot goes below it.
#define HOLE_CARD_STEP 7
#define COMMUNITY_CARD_STEP 7
#define COMMUNITY_X 23
#define POT_Y_MODIFIER 3

// A right-side seat's name and purse end at its card's right edge
#define NAME_RIGHT_X 5

// The table list: double spaced like the other platforms' while it fits
// between the header and the frame's bottom edge (row 23), one table a row
// past eight. (Expanded where screens.c lays the list out, which can see
// clientState.)
#define TABLE_TOP 5
#define TABLE_ROW_HEIGHT (clientState.tables.count > 8 ? 1 : 2)

#define QUERY_SUFFIX ""

// The H19 sends its keypad and function keys as escape sequences, so
// src/h89/input.c decodes the keyboard itself, without blocking.
#define USE_PLATFORM_SPECIFIC_INPUT 1
```

```
35
36 /**
37  * Platform specific key map for common input (H19 keyboard)
38  */
39
40 // src/h89/input.c turns the keypad's ESC A-D (keypad shifted mode, set by
41 // termInit) into the MSX's cursor codes, which stay below 0x80 because
42 // inputKey is a plain char. The _2 codes are unreachable placeholders --
43 // readCommonInput() switches on all twelve -- and the _3 keys are spares for
44 // a keyboard without the keypad.
45 #define KEY_LEFT_ARROW      0x1D
46 #define KEY_LEFT_ARROW_2    0xF1
47 #define KEY_LEFT_ARROW_3    0x2C // ,
48
49 #define KEY_RIGHT_ARROW     0x1C
50 #define KEY_RIGHT_ARROW_2   0xF2
51 #define KEY_RIGHT_ARROW_3   0x2E // .
52
53 #define KEY_UP_ARROW        0x1E
54 #define KEY_UP_ARROW_2      0xF3
55 #define KEY_UP_ARROW_3      0x2D // -
56
57 #define KEY_DOWN_ARROW      0x1F
58 #define KEY_DOWN_ARROW_2    0xF4
59 #define KEY_DOWN_ARROW_3    0x3D // =
60
61 #define KEY_RETURN      0x0D
62 #define KEY_ESCAPE      0x1B
63 #define KEY_ESCAPE_ALT  0x03 // CTRL-C
64 #define KEY_SPACE       0x20
65 #define KEY_BACKSPACE   0x08
66
67 /* Keyboard only: readJoystick() always returns 0 */
68 #define JOY_UP(v)    (0)
69 #define JOY_DOWN(v)  (0)
70 #define JOY_LEFT(v)  (0)
71 #define JOY_RIGHT(v) (0)
72 #define JOY_BTN_1(v) (0)
73
74 /*
75   Mapping for converting incoming ALT letters to a standard case
76 */
77 #define LINE_ENDING 0x0A
78 #define ALT_LETTER_START 0x0
79 #define ALT_LETTER_END 0x0
80 #define ALT_LETTER_AND 0x0
81
82 /*
83  Screen related variables
84 */
85
86 // Screen specific player/bet coordinates (src/h89/vars.c)
87 extern unsigned char playerXMaster[];
88 extern unsigned char playerYMaster[];
89
90 extern char playerDirMaster[];
91 extern char playerBetXMaster[];
92 extern char playerBetYMaster[];
93
94 // Simple hard coded arrangment of players around the table based on player count.
95 // These refer to index positions in the Master arrays above
96 // Downside is new players will cause existing player positions to move.
97 extern char playerCountIndex[];
98
99 #endif /* KEYMAP_H */
100
101 #endif /* __H89__ */
```

SRC/H89/LOGO.H — 38 LINES

```
1  /* Generated by src/h89/mklogo.py -- do not edit. */
2  /* Title logo as runs of H19 cell codes: row, column, length, codes;
3     0xFF ends it. */
4  #ifndef H89_LOGO_H
5  #define H89_LOGO_H
6
7  static const unsigned char logo[] = {
8      2, 25, 29,
9          0x10, 0x10, 0xA0, 0x10, 0x10, 0x20, 0xA0, 0x10, 0x10, 0x10, 0x10, 0x20,
10         0xA0, 0x9F, 0x20, 0x92, 0xA0, 0x20, 0x20, 0x92, 0xA0, 0x9F, 0x20, 0x20,
11         0x8E, 0x10, 0x10, 0x10, 0x10,
12     3, 25, 29,
13         0x20, 0x20, 0xA0, 0x20, 0x20, 0x20, 0xA0, 0x10, 0x10, 0x10, 0x20, 0x20,
14         0x20, 0xA0, 0xA0, 0xA0, 0xA0, 0x20, 0x20, 0x92, 0x90, 0x90, 0x90, 0x9F, 0x20,
15         0x0F, 0x10, 0x10, 0x10, 0x8F,
16     4, 25, 29,
17         0x20, 0x20, 0xA0, 0x20, 0x20, 0x20, 0xA0, 0x90, 0x90, 0x90, 0x90, 0x20,
18         0xA0, 0x12, 0x20, 0x1F, 0xA0, 0x20, 0xA0, 0x20, 0x20, 0x20, 0xA0, 0x20,
19         0x90, 0x90, 0x90, 0x90, 0x8C,
20     6, 21, 37,
21         0xA0, 0x20, 0x20, 0x20, 0xA0, 0x20, 0x8E, 0x10, 0x10, 0x10, 0x8F, 0x20,
22         0xA0, 0x20, 0x20, 0x20, 0x20, 0x20, 0xA0, 0x10, 0x10, 0x10, 0x8F, 0x20,
23         0xA0, 0x20, 0xA0, 0x10, 0x10, 0x10, 0x10, 0x20, 0xA0, 0x9F, 0x20, 0x92,
24         0xA0,
25     7, 21, 37,
26         0xA0, 0x10, 0x10, 0x10, 0xA0, 0x20, 0xA0, 0x20, 0x20, 0x20, 0xA0, 0x20,
27         0xA0, 0x20, 0x20, 0x20, 0x20, 0x20, 0xA0, 0x20, 0x20, 0x20, 0xA0, 0x20,
28         0x0F, 0x20, 0xA0, 0x10, 0x10, 0x10, 0x20, 0x20, 0xA0, 0x1F, 0x90, 0x12,
29         0xA0,
30     8, 21, 37,
31         0xA0, 0x20, 0x20, 0x20, 0xA0, 0x20, 0x8D, 0x90, 0x90, 0x90, 0x8C, 0x20,
32         0xA0, 0x90, 0x90, 0x90, 0x90, 0x20, 0xA0, 0x90, 0x90, 0x90, 0x8C, 0x20,
33         0x20, 0x20, 0xA0, 0x90, 0x90, 0x90, 0x90, 0x20, 0xA0, 0x20, 0x20, 0x20,
34         0xA0,
35     0xFF
36 };
37
38 #endif /* H89_LOGO_H */
```

BATTLESHIP

GENERAL

Battleship is the old pencil-and-paper game for two to four players. Each hides five ships on a ten-by-ten sea — of five, four, three, three and two squares — and the players take turns firing at a square. On the FujiNet's version a shot goes at **every** other player's sea at once, and the last fleet afloat wins. The program is FBS.ABS for HDOS and FBS.COM for CP/M.

PLAYING

( ) Run FBS, give your name if it asks, and choose a table. The "AI" tables have robot players.

( ) In the lobby, press SPACE when you are ready. When everyone is, a countdown starts.

( ) Place your ships: each appears at a random place; the arrow keys move it, R turns it, and RETURN puts it there.

( ) On your turn a cursor blinks on every opponent's sea at the same square: move it, and RETURN fires. Hits, misses and sunk ships show on each sea and in each player's "drawer" of ships.

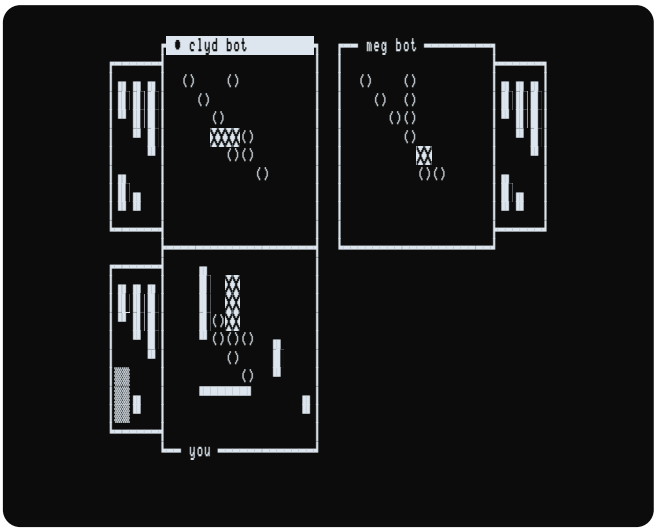
( ) ESC opens the menu: Q leaves, H tells how to play, S turns the bell off.



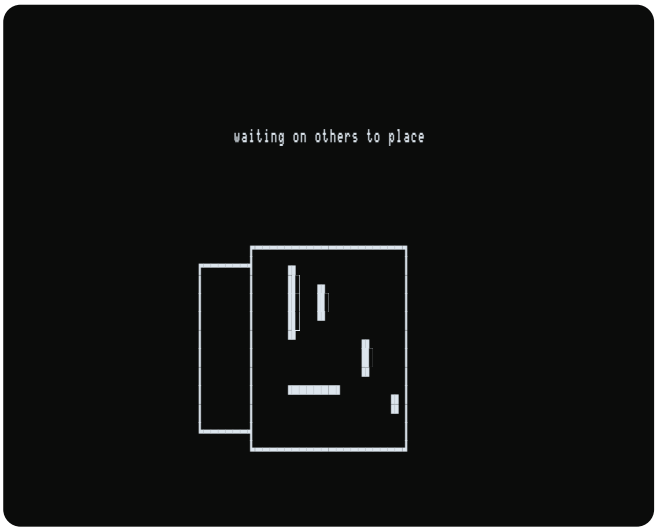
PICTORIAL 18-1
Choosing a game to join.



PICTORIAL 18-2
The lobby: one player and three robots, all ready, and the countdown.



PICTORIAL 18-4
A game in progress, against two robots: misses are `()`, hits reversed crosses, and the badge of the player whose turn it is is in reverse video.



PICTORIAL 18-3
Your five ships placed, half a square thick, waiting for the others.

HOW IT WORKS

THE STATE

Battleship asks for `state` about once a second, with `&bin=1&v=2`. The reply is the `Game` structure; in the lobby the same bytes are read as a `Lobby` (the table's name and who is ready):

OFFSET	BYTES	HOLDS
0	1	How many players (you are 0).
1	33	The prompt to show.
34	1	The game's status: 0 lobby, 1 place ships, 10 start, 11 miss, 12 hit, 13 sunk, 99 over.
35	1	Your status: playing, defeated, watching, ready, or place your ships.
36	1	Whose turn.
37	1	Seconds left to move.
38	1	The square last fired at.
39	10	Your ships (and, at the end, the winner's): square + 100 if down.
49	115 each	Each player: name, status, the sea (100 squares: untried, hit, miss), and which ships are left.

Ships are placed with `place/a,b,c,d,e`, each the square of its top left plus 100 if it runs down the sea; a shot is `attack/n`, n from 0 to 99.

DRAWING THE SEAS

H19 characters are about twice as tall as they are wide, so each square of a sea is two characters wide, which makes it nearly square. Water is blank, a miss is `()`, and a hit two reversed diagonal crosses. A ship's hull is half a

square thick, drawn with half blocks and quarter blocks so it never touches its neighbours; its bow and stern are quarter blocks too. A shot is a little animation: `><`, `\/`, `XX`, reversed, checkerboard, a flash.

Battleship keeps one picture of the screen in memory and sends each changed cell at once, without the second picture the other two games use; it was the first of the three to be ported, and its console and keyboard code became theirs.

FUJINET-LIB

Battleship's shared code includes its own copy of fujinet-lib's headers, which declare the app key calls as functions; on the H89 they are macros, so calls through those declarations would link to nothing. So the game is built with `CUSTOM_FUJINET_CALLS`, and three small functions in `src/h89/fujinet.c`, which include the library's real headers, make the calls.

THE FILES

FILE	HOLDS
`src/main.c`	The globals, the ship sizes, the main loop and its polling.
`src/stateclient.c`	Requests, moves, the state.
`src/screens.c`	Help, name, welcome, table list, the menu.
`src/gamelogic.c`	The lobby, placing ships, the seas, shots, the end.
`src/misc.c`, `misc.h`	Keys, preferences, app keys; the structures.
`src/standard_lib.h`	The standard headers, per compiler; the H89's `kbhit()` and `cgetc()`.
`src/platform-specific/*.h`	What the H89's part must supply.
`src/h89/console.c`	The H19: one picture of the screen; the OS console.
`src/h89/graphics.c`	Seas, ships, the cursor, drawers, name badges.
`src/h89/input.c`	The H19's keys.
`src/h89/fujinet.c`	The three fujinet-lib calls.
`src/h89/sound.c`, `util.c`, `vars.h`	The bell; frames, timer, random numbers; layout and keys.

LISTING

SRC/MAIN.C — 147 LINES

```
/**
 * @brief   Fuji Battleship
 * @author  Eric Carr, Thomas Cherryhomes, (insert names here)
 * @license gpl v. 3
 * @verbose main
 */

/*****************************************************************
 *
 * Do NOT include standard library headers (e.g. conio, std*).
 * Instead, add to standard_lib.h, which gets included in misc.h
 *
 *****************************************************************/

#include "misc.h"
#include "stateclient.h"
#include "gamelogic.h"
#include "screens.h"


// Store default public server endpoint in case lobby did not set app key
char serverEndpoint[50] = "https://battleship.carr-designs.com/";
//char serverEndpoint[50] = "http://127.0.0.1:8080/";

// For local dev testing, instead of changing the endpoint above,
// set 1st byte in the e41c0500 appkey to 0xff, which will cause the below endpoint to be used
const char localServer[] = "http://127.0.0.1:8080/";

// Zero-initialized by the C runtime (BSS) - an explicit = "" would cost the
// z88dk ROM a stored DATA image of the zeros
char query[QUERY_LEN]; //"?table=dev7";//&player=ERICAPL2";
char playerName[12];

// On the ColecoVision this is a macro for the FujiNet cartridge's reply
// window, not an object -- see src/misc.h.
#ifndef BUILD_COLECO
ClientState clientState;
#endif
GameState state;
PrefsStruct prefs;

// Common local scope temp variables
char tempBuffer[TEMP_BUFFER_LEN];
const uint8_t shipSize[5] = {5, 4, 3, 3, 2}; // Standard ship sizes

// extern void toneFinder();
```

```
47
48 void main(void)
49 {
50     uint8_t failedApiCalls = 0;
51         char ch;
52     // Testing
53     // toneFinder();
54     // printf("Press keys\n");while(1) {while (!kbhit());failedApiCalls = cgetc();printf("%d
 +         0x%x\n", failedApiCalls, failedApiCalls);} // Read Key
55
56     loadPrefs();
57     initGraphics();
58     initSound();
59
60     // soundCursor();
61     // cgetc();
62
63     // soundSelect();
64     // cgetc();
65
66     // soundJoinGame();
67     // cgetc();
68
69     // soundMyTurn();
70     // cgetc();
71
72     // soundGameDone();
73     // cgetc();
74
75     // soundTick();
76     // cgetc();
77
78     // soundPlaceShip();
79     // cgetc();
80
81     // soundAttack(); // jak stukniÄ□cie, nie wybuch, za wysoki
82     // cgetc();
83
84     // soundInvalid();
85     // cgetc();
86
87     // soundHit(); // dwa szumy, za wysokie
88     // cgetc();
89
90     showWelcomeScreen();
91     showTableSelectionScreen();
92
93     // Main event loop - process state from server and input from keyboard/joystick
94     state.apiCallWait = 0;
95
96     while (true)
97     {
98
99         // Poll the server every so often.
100        if (!state.apiCallWait--)
101        {
102
103            // Housekeeping - allows platform specific housekeeping, like stopping
 +              Attract/screensaver mode in Atari
104            housekeeping();
105
106            // Poll the server
107            switch (getStateFromServer())
108            {
109            case STATE_UPDATE_ERROR:
110                // ERROR - Wait a bit to avoid hammering the server if getting bad responses
111                // Wait max 4 seconds (since 4*60=240 fits in a single byte)
112                if (failedApiCalls < 4)
113                {
114                    failedApiCalls++;
115                }
116                state.apiCallWait = 60 * failedApiCalls;
117
118                // After consequitive failures, let the player know we are experiencing
 +                  technical difficulties
119                if (failedApiCalls > 1)
120                {
121                    drawConnectionIcon(true);
122                    pause(30);
123                    drawConnectionIcon(false);
124                    pause(30);
125                    drawConnectionIcon(true);
126                }
127                break;
128
129            case STATE_UPDATE_CHANGE:
130
131                // Clear connection failure message
132                if (failedApiCalls > 1)
133                {
134                    drawConnectionIcon(false);
135                }
136                failedApiCalls = 0;
137                processStateChange();
138
139                // Poll again in a bit
140                state.apiCallWait = 59;
141                break;
142            }
143        }
144
145        processInput();
146    }
147 }
```

SRC/GAMELOGIC.C — 863 LINES

```
1  /*****************************************************************
2   *
3   * Do NOT include standard library headers (e.g. conio, std*).
4   * Instead, add to standard_lib.h, which gets included in misc.h
5   *
6   *****************************************************************/
7
8  #include "misc.h"
9  #include "gamelogic.h"
10 #include "stateclient.h"
11 #include "screens.h"
12
13 #ifndef TIMER_WIDTH
14 #define TIMER_WIDTH 1
15 #endif
16
17 // Platforms whose input maps the rotate shortcut onto another physical key
18 // (e.g. the ColecoVision keypad) override this in their vars.h
19 #ifndef ROTATE_PROMPT_TEXT
20 #define ROTATE_PROMPT_TEXT "press R to rotate"
21 #endif
22 #ifndef READY_PROMPT_TEXT
23 #define READY_PROMPT_TEXT "press TRIGGER/SPACE when ready"
24 #endif
25
26 uint8_t posX = 0, posY = 0, inputField_done, validX;
27 uint8_t shipPlacements[5] = {0, 0, 0, 0, 0};
28 uint8_t shipPlaceIndex = 0;
29 char moveBuffer[32];
30
31 void progressAnim(uint8_t y)
32 {
33     static uint8_t i;
34
35     for (i = 0; i < 3; ++i)
36     {
37         pause(10);
38         drawIcon(WIDTH / 2 - 2 + i * 2, y, ICON_MARK);
39     }
40 }
41
42 // Capture a player's server gamefield into the local shadow (see the
43 // GAMEFIELD_BYTES note in misc.h for why this is packed on the ColecoVision)
44 static void packGamefield(uint8_t p)
45 {
46 #ifdef BUILD_COLECO
47     uint8_t *src = clientState.game.players[p].gamefield;
48     uint8_t i;
49
50     memset(&state.gamefield[p], 0, GAMEFIELD_BYTES);
51     for (i = 0; i < 100; i++)
52         if (src[i])
53             state.gamefield[p][i >> 3] |= 1 << (i & 7);
54 #else
55     memcpy(&state.gamefield[p], &clientState.game.players[p].gamefield, 100);
56 #endif
57 }
58
59 void processStateChange()
60 {
61     static uint8_t i;
62
63     switch (clientState.game.status)
64     {
65     case STATUS_LOBBY:
66         renderLobby();
67         break;
68     default:
69         renderGameboard();
70         break;
71     }
72
73     state.prevStatus = clientState.game.status;
74     state.prevPlayerStatus = clientState.game.playerStatus;
75     state.prevPlayerCount = clientState.game.playerCount;
76     state.prevActivePlayer = clientState.game.activePlayer;
77     state.prevAttackPos = clientState.game.lastAttackPos;
78
```

```
    if (clientState.game.status >= STATUS_GAMESTART)
    {
        for(i=0;i<clientState.game.playerCount;i++)
        {
            packGamefield(i);
        }
    }
}

#define READY_LEFT WIDTH / 2 - 8

void renderLobby()
{
    static uint8_t c = 0;
    uint8_t i, len;

    if (clientState.game.status != state.prevStatus || state.drawBoard)
    {
        state.drawBoard = false;

        // Clear gamefield
        memset(state.gamefield, 0, sizeof(state.gamefield));

        resetScreen();

        // Round 0 - Ready Up screen
        drawLogo();
        centerText(6, clientState.lobby.serverName);

        drawLine(READY_LEFT, 7, 16);
        centerTextAlt(HEIGHT - 4, "press " ESCAPE " for menu");
        centerTextAlt(HEIGHT - 1, READY_PROMPT_TEXT);

        // Reset ship placement ahead of next screen
        memset(shipPlacements, 0, sizeof(shipPlacements));
        shipPlaceIndex = 0;
    }

    centerTextWide(HEIGHT - 8, clientState.lobby.prompt);

    if (clientState.lobby.prompt[0] == 's')
    {
        if (state.countdownStarted)
        {
            soundTick();
        }
        else
        {
            soundJoinGame();
            state.countdownStarted = true;
        }
    }
    else
    {
        state.countdownStarted = false;
    }

    // Show players that are ready to start
    c++;
    for (i = 0; i < 9; i++)
    {
        if (i < clientState.lobby.playerCount)
        {
            drawText(READY_LEFT, 8 + i, clientState.lobby.players[i].name);
            len = (uint8_t)strlen(clientState.lobby.players[i].name);
            if (len < 8)
            {
                drawSpace(READY_LEFT + len, 8 + i, 8 - len);
            }
            drawText(READY_LEFT, 8 + i, clientState.lobby.players[i].name);
            if (clientState.lobby.players[i].ready)
            {
                drawTextAlt(READY_LEFT + 11, 8 + i, "ready");
            }
            else
            {
                drawSpace(READY_LEFT + 11, 8 + i, 5);
                drawIcon(READY_LEFT + 11 + ((c + i * 2) % 5), 8 + i, ICON_MARK);
            }
        }
        else if (i < state.prevPlayerCount)
        {
            drawSpace(READY_LEFT, 8 + i, 16);
        }
    }

    // pause(10);
}

void handleShipPlacement()
{
    uint8_t i, x, y, dir, pos, size, change, blink, maxW, maxH, canPlace;

    // Use tempBuffer to track occupied cells while placing ships
    memset(tempBuffer, 0, sizeof(tempBuffer));

    for (i = 0; i < shipPlaceIndex; i++)
    {
        pos = shipPlacements[i];
        placeShip(shipSize[i], pos);
    }

    clearCommonInput();
    while (shipPlaceIndex < 5)
    {
        size = shipSize[shipPlaceIndex];
        // Randomly select new location

        while (1)
        {
            pos = getRandomNumber(200);
            if (testShip(size, pos))
                break;
        }

        dir = pos / 100;
        x = pos % 10;
        y = (pos % 100) / 10;
        blink = 0;
        clearCommonInput();

        while (true)
        {
            // Draw ship at current position

            maxW = (11 - (dir == 0 ? size : 1));
            maxH = (11 - (dir == 1 ? size : 1));
            x = (x % maxW);
            y = (y % maxH);

            waitvsync();
            if (blink == 0 || blink == 16)
            {
                if (!blink)
                {
                    // Clear existing ship
                    drawShip(0, size, pos, DRAWSHIP_HIDE);
                }

                // New ship position
                pos = y * 10 + x + dir * 100;
                drawShip(0, size, pos, blink);

                // Redraw other ships in case they were overwritten
                for (i = 0; i < shipPlaceIndex; i++)
                {
                    drawShip(0, shipSize[i], shipPlacements[i], DRAWSHIP_SHOW);
                }
                if (change)
                {
                    soundCursor();
                }
                change = 0;
            }

            blink = (blink + 1) & 31;

            readCommonInput();
            // Move cursor
            if (input.dirX)
            {
                x += maxW + input.dirX;
                change = 1;
                blink = 0;
            }
            else if (input.dirY)
            {
                y += maxH + input.dirY;
                change = 1;
                blink = 0;
            }

            // Rotate ship
            if (input.key == 'r' || input.key == 'R')
            {
                dir = (dir + 1) % 2;
                change = 1;
                blink = 0;
            }

            // Confirm placement
            if (input.trigger)
            {
                if (testShip(size, pos))
                {
                    placeShip(size, pos);
```

```
                    shipPlacements[shipPlaceIndex] = pos;
                    shipPlaceIndex++;
                    if (shipPlaceIndex < 5)
                    {
                        soundPlaceShip();
                    }
                    else
                    {
                        // Clear input for next ship
                        soundSelect();
                    }

                    break;
                }
                else
                {
                    soundInvalid();
                }
            }

            switch (input.key)
            {
                case KEY_ESCAPE:    // Esc
                case KEY_ESCAPE_ALT: // Esc Alt
                    showInGameMenuScreen();
                    return;
                    break;
            }
        }
    }

    // Now send to server
    strcpy(moveBuffer, "place/");
    for (i = 0; i < 5; i++)
    {
        itoa(shipPlacements[i], &moveBuffer[strlen(moveBuffer)], 10);
        if (i < 4)
            strcat(moveBuffer, ",");
    }

    sendMove(moveBuffer);
}

void renderGameboard()
{
#define LEGEND_X WIDTH / 2 + 8
    static bool redraw, fullWidth;
    uint8_t i, j, jj, x, y, dir, pos, size, playedSound, skipAnim = false;

    // Redraw the entire board when placing ships back to round 0 (ready up)
    redraw = clientState.game.status != state.prevStatus && (clientState.game.status ==
        STATE_INVALID || clientState.game.status == STATUS_PLACE_SHIPS || state.prevStatus ==
        STATUS_PLACE_SHIPS);

    // Clear screen and draw initial backdrop
    if (redraw || state.drawBoard)
    {
        state.drawBoard = false;
        redraw = true;
        skipAnim = true;
        resetScreen();
        drawBoard(clientState.game.status == STATUS_PLACE_SHIPS ? 1 :
            clientState.game.playerCount);

        if (clientState.game.status == STATUS_PLACE_SHIPS)
        {
            centerText(5, "place your five ships");
            centerTextAlt(7, ROTATE_PROMPT_TEXT);
        }
        if (clientState.game.status >= STATUS_GAMESTART)
        {
            // Draw this player's ships
            if (clientState.game.playerStatus != PLAYER_STATUS_VIEWING)
            {
                for (i = 0; i < 5; i++)
                {
                    drawShip(0, shipSize[i], clientState.game.myShips[i], DRAWSHIP_SHOW);
                }
            }

            // Draw gamefield
            for (i = 0; i < clientState.game.playerCount; i++)
            {
                drawGamefield(i, clientState.game.players[i].gamefield);
            }
        }
    }

    if (clientState.game.status >= STATUS_GAMESTART)
    {
        if (clientState.game.activePlayer == state.prevActivePlayer || clientState.game.status
            == STATUS_GAMESTART)
        {
            skipAnim = true;
        }

        playedSound = skipAnim;

        // Render gamefield updates
        if (clientState.game.status > STATUS_GAMESTART)
        {

            // Animate other player's attack
            if (!skipAnim && state.prevActivePlayer != 0)
            {
                for (j = 10; j < 16; j++)
                {
                    for (i = 0; i < clientState.game.playerCount; i++)
                    {
                        if (i != state.prevActivePlayer &&
                            clientState.game.players[i].playerStatus == PLAYER_STATUS_DEFAULT
                            && GAMEFIELD_IS_EMPTY(i, clientState.game.lastAttackPos))
                            drawGamefieldUpdate(i, clientState.game.players[i].gamefield,
                                clientState.game.lastAttackPos, j);
                    }
                    pause(5);
                }
            }

            // Animate/render hit/miss
            for (j = (!skipAnim && clientState.game.status == STATUS_HIT) * 6; j < 255; --j)
            {
                for (i = 0; i < clientState.game.playerCount; i++)
                {
                    if (i != state.prevActivePlayer && GAMEFIELD_IS_EMPTY(i,
                        clientState.game.lastAttackPos))
                        drawGamefieldUpdate(i, clientState.game.players[i].gamefield,
                            clientState.game.lastAttackPos, j & 1);
                }
                if (!playedSound)
                {
                    if (clientState.game.status > STATUS_MISS)
                    {
                        soundHit();
                    }
                    else
                    {
                        soundMiss();
                    }
                    playedSound = 1;
                }

                pause(4);
            }

            for (i = 0; i < clientState.game.playerCount; i++)
            {
                // Draw ships left indicators on legend
                for (j = 0; j < 5; j++)
                {
                    // Animate a ship being sunk
                    if (!skipAnim && state.shipsLeft[i][j] !=
                        clientState.game.players[i].shipsLeft[j])
                    {
                        for (jj = 0; jj < 5; jj++)
                        {
                            pause(4);
                            drawLegendShip(i, j, shipSize[j], jj & 1);
                        }

                        soundSink();
                    }
                    else
                    {
                        drawLegendShip(i, j, shipSize[j], clientState.game.players[i].shipsLeft[j]
                            );
                    }
                }

                // Track current ships left so we can animate when a ship gets sunk
                memcpy(state.shipsLeft[i], clientState.game.players[i].shipsLeft, 5);
            }

            if (clientState.game.status != STATUS_GAMEOVER || redraw || clientState.game.status !=
                state.prevStatus)
            {
                // Clear active player
                for (i = 0; i < clientState.game.playerCount; i++)
                {
                    // Draw player name
                    if (i != clientState.game.activePlayer)
                        drawPlayerName(i, i == 0 && clientState.game.playerStatus !=
                            PLAYER_STATUS_VIEWING ? "you" : (const char
                            *)clientState.game.players[i].name, false);
                }
```

```

            // Indicate active player if not game over
            if (clientState.game.status != STATUS_GAMEOVER)
            {
                // Show active player
                if (clientState.game.activePlayer >= 0)
                {
                    drawPlayerName(clientState.game.activePlayer, clientState.game.activePlayer
                        == 0 && clientState.game.playerStatus != PLAYER_STATUS_VIEWING ? "you"
                        : (const char *)clientState.game.players[clientState.game.activePlayer]
                        .name, true);
                }

                // Blink active player
                if (clientState.game.activePlayer > 0)
                {

                    pause(15);
                    drawPlayerName(clientState.game.activePlayer,
                        clientState.game.players[clientState.game.activePlayer].name, false);

                    pause(15);
                    drawPlayerName(clientState.game.activePlayer,
                        clientState.game.players[clientState.game.activePlayer].name, true);
                    pause(15);
                }
            }
        }
    }

    // Display the gameover message and play a sound if the state just changed
    if (clientState.game.status == STATUS_GAMEOVER && (redraw || clientState.game.status !=
        state.prevStatus))
    {
        // Reveal winning player's ships (if not this player)
        waitvsync();
        if (clientState.game.activePlayer > 0)
        {
            for (i = 0; i < 5; i++)
            {
                drawShip(clientState.game.activePlayer, shipSize[i], clientState.game.myShips[5
                    + i], DRAWSHIP_SHOW);
            }

            drawGamefield(clientState.game.activePlayer, clientState.game.players[clientState.g
                ame.activePlayer].gamefield);
        }

        drawEndgameMessage(clientState.game.prompt);

        if (clientState.game.status != state.prevStatus)
        {
            soundGameDone();

            // Wait for user to press button to close game result
            clearCommonInput();
            while (!input.trigger)
            {
                waitvsync();
                readCommonInput();
            }
        }
    }

    // Draw ships that have been placed already
    if (clientState.game.status == STATUS_PLACE_SHIPS)
    {
        if (clientState.game.playerStatus != PLAYER_STATUS_VIEWING)
        {
            if (clientState.game.playerStatus == PLAYER_STATUS_PLACE_SHIPS)
            {
                handleShipPlacement();
            }
            else
            {
                // Draw already placed ships from game state
                for (i = 0; i < 5; i++)
                {
                    drawShip(0, shipSize[i], clientState.game.myShips[i], DRAWSHIP_SHOW);
                }
            }
        }
        if (clientState.game.status == STATUS_PLACE_SHIPS)
        {
            centerTextWide(5, clientState.game.prompt);
            centerTextAlt(7, "                  ");
        }
    }
    pause(30);
    // cgetc();
    // pause(60);
}


void placeShip(uint8_t shipSize, uint8_t pos)
{
    uint8_t i;
    drawShip(0, shipSize, pos, DRAWSHIP_SHOW);
    for (i = 0; i < shipSize; i++)
    {
        tempBuffer[pos % 100] = 1;
        pos += (pos >= 100) ? 10 : 1;
    }
}

/// @brief Returns true if free to place ship at position
bool testShip(uint8_t shipSize, uint8_t pos)
{
    uint8_t i;
    for (i = 0; i < shipSize; i++)
    {
        // if existing ship || outside Y bounds || crossed H bounds, return false
        if (tempBuffer[pos % 100] || pos > 199 || (i > 0 && pos <= 100 && pos % 10 == 0))
            return false;

        pos += (pos >= 100) ? 10 : 1;
    }
    return true;
}

void processInput()
{
    waitvsync();
    readCommonInput();

    if (state.waitingOnEndGameContinue)
    {
        if (input.trigger)
        {
            state.waitingOnEndGameContinue = false;
            //  clearRenderState();
        }
    }
    else if (clientState.game.playerStatus != PLAYER_STATUS_VIEWING)
    {
        // Toggle readiness if waiting to start game. The server echoes the
        // updated lobby state back from "ready", so render from that truth
        // rather than writing the toggle into clientState first - clientState
        // is not writable on every platform (the ColecoVision reads it out of
        // the cartridge's reply window).
        if (clientState.game.status == STATUS_LOBBY && input.trigger)
        {
            if (apiCall("ready") == API_CALL_SUCCESS && clientState.game.status ==
                STATUS_LOBBY)
            {
                if (clientState.lobby.players[0].ready)
                    soundSelect();
                else
                    soundInvalid();

                renderLobby();
            }

            // Poll again right away in case the reply was not the lobby state
            state.apiCallWait = 0;
            clearCommonInput();
            return;
        }

        // Wait on this player to attack
        if (clientState.game.activePlayer == 0 && clientState.game.status >= STATUS_GAMESTART)
        {
            waitOnPlayerMove();
        }
    }

    switch (input.key)
    {
    case KEY_ESCAPE:    // Esc
    case KEY_ESCAPE_ALT: // Esc Alt
        showInGameMenuScreen();
        break;
    }
}

void waitOnPlayerMove()
{
    bool foundValidLocation;
    uint8_t waitCount, frames, lastFrame, i, j, moved, attackPos;
    // The countdown tracker. Hoisted out of clientState: writing the ticking
    // value back into it never worked on platforms where the state is
    // read-only (the ColecoVision's cartridge reply window) - the write went
    // nowhere, the loop condition never expired, and the move was never
    // auto-resolved.
    uint8_t timeLeft;
    uint16_t jifsPerSecond, maxJifs;

```

```
    resetTimer();

    // Determine max jiffies for PAL and NTS
    jifsPerSecond = getJiffiesPerSecond();
    timeLeft = clientState.game.moveTime;
    maxJifs = jifsPerSecond * timeLeft;
    waitCount = 0;
    moved = frames = 9;

    // Move selection loop
    while (timeLeft > 0)
    {
        frames = (frames + 1) % 30;
        i = frames / 10;
        waitvsync();
        if (moved || i != lastFrame)
        {


            // Always show the cursor (do not blink) when moving it around.
            // Otherwise, it appears to skip around
            if (moved)
                lastFrame = 1; // Show cursor frame 1
            else
                lastFrame = i; // Show 0,1,2 depending on frame

            // Draw cursor
            for (i = 1; i < clientState.game.playerCount; i++)
            {
                if (clientState.game.players[i].playerStatus == PLAYER_STATUS_DEFAULT)
                    drawGamefieldCursor(i, posX, posY, clientState.game.players[i].gamefield,
                        lastFrame);
            }
        }

        if (moved)
        {
            // Play "my turn" sound on first move
            if (moved == 9)
                soundMyTurn();
            else
                soundCursor();

            moved = 0;
        }

        // Handle trigger press
        if (input.trigger)
        {

            attackPos = posY * 10 + posX;

            // Check if at least one enemy cell is valid to attack
            for (i = 1; i < clientState.game.playerCount; i++)
            {
                if (clientState.game.players[i].playerStatus == PLAYER_STATUS_DEFAULT &&
                    GAMEFIELD_IS_EMPTY(i, attackPos))
                    break;
            }

            if (i == clientState.game.playerCount)
            {
                // Invalid location
                soundInvalid();
            }
            else
            {
                // Attack!
                soundAttack();

                // Animate attack / clear cursor
                for (j = 10; j < 16; j++)
                {
                    for (i = 1; i < clientState.game.playerCount; i++)
                    {
                        if (clientState.game.players[i].playerStatus == PLAYER_STATUS_DEFAULT)
                            drawGamefieldUpdate(i, clientState.game.players[i].gamefield,
                                attackPos, GAMEFIELD_IS_EMPTY(i, attackPos) ?  j : 0);
                    }
                    pause(5);
                }

                // Send command to score this value
                strcpy(moveBuffer, "attack/");
                itoa(attackPos, moveBuffer + strlen(moveBuffer), 10);
                sendMove(moveBuffer);

                // Clear timer
                drawSpace(WIDTH - TIMER_WIDTH - 2, HEIGHT - 1, 2 + TIMER_WIDTH);
                return;
            }
        }

        // Update cursor
        if (input.dirX || input.dirY)
        {
            for (i = 1; i < clientState.game.playerCount; i++)
            {
                if (clientState.game.players[i].playerStatus == PLAYER_STATUS_DEFAULT)
                    drawGamefieldCursor(i, posX, posY, clientState.game.players[i].gamefield,
                        0);
            }
            posX = (posX + 10 + input.dirX) % 10;
            posY = (posY + 10 + input.dirY) % 10;
            moved = 1;
        }

        // Tick counter once per second
        if (++waitCount > 5)
        {
            waitCount = 0;
            i = (uint8_t)((maxJifs - getTime()) / jifsPerSecond);
            if (i <= 20 && i != timeLeft)
            {
                timeLeft = i;
                if (i < 10)
                    tempBuffer[0] = ' ';

                itoa(i, tempBuffer + (i < 10), 10);
                drawTextAlt(WIDTH - TIMER_WIDTH - 2, HEIGHT - 1, tempBuffer);
                drawClock();
                soundTick();
            }
        }

        // Pressed Esc
        switch (input.key)
        {
        case KEY_ESCAPE:
        case KEY_ESCAPE_ALT:
            for (i = 1; i < clientState.game.playerCount; i++)
            {
                if (clientState.game.players[i].playerStatus == PLAYER_STATUS_DEFAULT)
                    drawGamefieldCursor(i, posX, posY, clientState.game.players[i].gamefield,
                        0);
            }
            showInGameMenuScreen();
            return;
        }

        // Read input for next iteration
        readCommonInput();
    }

    // Timed out
}

uint8_t prevCursorPos;
// Invalidate state variables that will trigger re-rendering of screen items on the next cycle
void clearRenderState()
{
    state.prevPlayerCount = 0;
    state.drawBoard = true;
}

/// @brief Convenience function to draw text centered at row Y
void centerText(uint8_t y, const char *text)
{
    drawText(WIDTH / 2 - (uint8_t)strlen(text) / 2, y, text);
}

/// @brief Convenience function to draw text centered at row Y, blanking out the rest of the row
void centerTextWide(uint8_t y, const char *text)
{
    uint8_t i, x;
    i = (uint8_t)strlen(text);
    x = WIDTH / 2 - i / 2;

    drawSpace(0, y, x);
    drawText(x, y, text);
    drawSpace(x + i, y, WIDTH - x - i);
}

/// @brief Convenience function to draw text centered at row Y in alternate color
void centerTextAlt(uint8_t y, const char *text)
{
    drawTextAlt(WIDTH / 2 - (uint8_t)strlen(text) / 2, y, text);
}

/// @brief Convenience function to draw status text centered
void centerStatusText(const char *text)
{
    drawTextAlt((WIDTH - (uint8_t)strlen(text)) >> 1, HEIGHT - 1, text);
}
```

```
794 /// @brief Init/reset the input field for display
795 void resetInputField()
796 {
797     inputField_done = 1;
798     disableKeySounds();
799 }
800
801 /// @brief Handles available key strokes for the defined input box (player name and chat).
  +     Returns true if user hits enter
802 bool inputFieldCycle(uint8_t x, uint8_t y, uint8_t max, char *buffer)
803 {
804     // These must persist between calls (curx tracks the cursor across the
805     // keystroke cycles); as stack locals they only worked by accident on cc65
806     static uint8_t curx, lastY;
807
808     // Initialize first call to input box
809     if (inputField_done == 1 || lastY != y)
810     {
811         inputField_done = 0;
812         lastY = y;
813         curx = (uint8_t)strlen(buffer);
814         drawTextAlt(x, y, buffer);
815         drawIcon(x + curx, y, ICON_TEXT_CURSOR);
816         enableKeySounds();
817     }
818
819     // Process any waiting keystrokes
820     if (kbhit())
821     {
822         inputField_done = 0;
823
824         input.key = cgetc();
825
826         // Debugging - See what key was pressed
827         // itoa(input.key, tempBuffer, 10);drawText(0,0, tempBuffer);
828
829         if (input.key == KEY_RETURN && curx > 1)
830         {
831             inputField_done = 1;
832             // remove cursor
833             drawBlank(x + curx, y);
834         }
835         else if ((input.key == KEY_BACKSPACE || input.key == KEY_LEFT_ARROW) && curx > 0)
836         {
837             buffer[--curx] = 0;
838             drawText(x + 1 + curx, y, " ");
839         }
840         else if (
841             curx < max && ((curx > 0 && input.key == KEY_SPACEBAR) || (input.key >= 48 &&
  +                 input.key <= 57) || (input.key >= 65 && input.key <= 90) || (input.key >= 97 &&
  +                 input.key <= 122)) // 0-9 A-Z a-z
842         )
843         {
844
845             if (input.key >= 65 && input.key <= 90)
846                 input.key += 32;
847
848             buffer[curx] = input.key;
849             buffer[++curx] = 0;
850         }
851
852         drawTextAlt(x, y, buffer);
853
854         if (inputField_done)
855             disableKeySounds();
856         else
857             drawIcon(x + curx, y, ICON_TEXT_CURSOR);
858
859         return inputField_done;
860     }
861
862     return false;
863 }
```

SRC/GAMELOGIC.H — 28 LINES

```
 1 #ifndef GAMELOGIC_H
 2 #define GAMELOGIC_H
 3
 4 void processStateChange();
 5 void renderLobby();
 6 void renderGameboard();
 7 void handleAnimation();
 8
 9 void processInput();
10
11 void clearRenderState();
12
13 void centerText(uint8_t y, const char *text);
14 void centerTextAlt(uint8_t y, const char *text);
15 void centerTextWide(uint8_t y, const char *text);
16 void centerStatusText(const char *text);
17
18 void resetInputField();
19 bool inputFieldCycle(uint8_t x, uint8_t y, uint8_t max, char *buffer);
20
21 void waitOnPlayerMove();
22
23 void progressAnim(uint8_t y);
24
25 void placeShip(uint8_t shipSize, uint8_t pos);
26 bool testShip(uint8_t shipSize, uint8_t pos);
27
28 #endif /*GAMELOGIC_H*/
```

SRC/MISC.C — 199 LINES

```
 1 /*****************************************************************
 2  *
 3  * Do NOT include standard library headers (e.g. conio, std*).
 4  * Instead, add to standard_lib.h, which gets included in misc.h
 5  *
 6  *****************************************************************/
 7
 8 #include "misc.h"
 9
10 InputStruct input;
11 uint8_t _lastJoy, _joy, _joySameCount = 10;
12 bool _buttonReleased = true;
13 #ifdef __APPLE2__
14 uint8_t _lastKey = 0;  // Track last key pressed to prevent repeated input (Apple II only)
15 #endif
16
17 #ifndef JOY_BTN_2_MASK
18 #define JOY_BTN_2_MASK JOY_BTN_1_MASK
19 #endif
20
21
22 #ifdef CUSTOM_FUJINET_CALLS
23 // Optional: This would be implemented in platform-specific code for emulators, etc
24 uint16_t custom_read_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, char
  +     *destination);
25 void custom_write_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, uint16_t count,
  +     char *data);
26 #endif
27
28 void pause(uint8_t frames)
29 {
30     while (frames--)
31         waitvsync();
32 }
33
34 void clearCommonInput()
35 {
36     input.trigger = input.key = input.dirY = input.dirX = _lastJoy = _joy = _buttonReleased =
  +         0;
37 #ifdef __APPLE2__
38     _lastKey = 0;
39 #endif
40     while (kbhit())
41         cgetc();
42 }
43
44 void readCommonInput()
45 {
46     input.trigger = input.key = input.dirX = input.dirY = 0;
47
48     _joy = readJoystick();
49
50     // Simulate the keyboard delay for joystick input, by checking previous joystick value
51     // There is special logic so that "shifting into a diagnal" still results in a single X and
  +         Y move
52     // e.g. RIGHT, RIGHT+UP, UP moves the cursor the same as a single RIGHT+UP, rathar than TWO
  +         to the right and ONE up
53     if (_joy != _lastJoy)
54     {
55         if (JOY_LEFT(_joy) && (_lastJoy == 99 || !JOY_LEFT(_lastJoy)))
56             input.dirX = -1;
57         else if (JOY_RIGHT(_joy) && (_lastJoy == 99 || !JOY_RIGHT(_lastJoy)))
58             input.dirX = 1;
59
60         if (JOY_UP(_joy) && (_lastJoy == 99 || !JOY_UP(_lastJoy)))
61             input.dirY = -1;
62         else if (JOY_DOWN(_joy) && (_lastJoy == 99 || !JOY_DOWN(_lastJoy)))
63             input.dirY = 1;
64
65         // Reset the delay if no movement is detected
66         if (_joy == 0)
67             _joySameCount = 12;
68
69         _lastJoy = _joy;
70
71         // Trigger button press only if it was previously unpressed
72         if (JOY_BTN_1(_joy) || JOY_BTN_2(_joy))
```

```
        {
            if (_buttonReleased)
            {
                input.trigger = true;
                _buttonReleased = false;
            }
        }
        else
        {
            _buttonReleased = true;
        }

        return;
    }
    else if (_joy != 0)
    {
        // Pressing in same direction, decrement wait counter
        if (!_joySameCount--)
        {
            _joySameCount = 0;

            // Set lastJoy to 99 to trigger immediate direction change on next read
            _lastJoy = 99;
        }
    }

    if (!kbhit())
    {
#ifdef __APPLE2__
        // No key pressed, reset last key (Apple II only)
        _lastKey = 0;
#endif
        return;
    }

    input.key = cgetc();

#ifdef __APPLE2__
    // Only process if this is a different key than last time (Apple II only)
    // This prevents the same key from being processed multiple times
    // But allow processing if last key was 0 (no key was pressed before)
    if (input.key == _lastKey && _lastKey != 0)
    {
        // Same key as last time, ignore it
        input.key = 0;
        return;
    }

    // New key pressed, save it
    _lastKey = input.key;
#endif

    switch (input.key)
    {
    case KEY_LEFT_ARROW:
    case KEY_LEFT_ARROW_2:
    case KEY_LEFT_ARROW_3:
        input.dirX = -1;
        break;
    case KEY_RIGHT_ARROW:
    case KEY_RIGHT_ARROW_2:
    case KEY_RIGHT_ARROW_3:
        input.dirX = 1;
        break;
    case KEY_UP_ARROW:
    case KEY_UP_ARROW_2:
    case KEY_UP_ARROW_3:
        input.dirY = -1;
        break;
    case KEY_DOWN_ARROW:
    case KEY_DOWN_ARROW_2:
    case KEY_DOWN_ARROW_3:
        input.dirY = 1;
        break;
    case KEY_SPACEBAR:
    case KEY_RETURN:
        input.trigger = true;
        break;
    }
}

void loadPrefs()
{
    memset(&prefs, 0, sizeof(prefs));
    if (read_appkey(AK_CREATOR_ID, AK_APP_ID, AK_KEY_PREFS, tempBuffer))
    {
        memcpy(&prefs, tempBuffer, sizeof(prefs));

        if (prefs.debugFlag == 0xff)
        {
            strcpy(serverEndpoint, localServer);
        }
    }
}

void savePrefs()
{
    write_appkey(AK_CREATOR_ID, AK_APP_ID, AK_KEY_PREFS, sizeof(prefs), (char *)&prefs);
}


uint16_t read_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, char *destination)
{
    uint16_t read = 0;

    #ifdef CUSTOM_FUJINET_CALLS
        read = custom_read_appkey(creator_id, app_id, key_id, destination);
    #else
        fuji_set_appkey_details(creator_id, app_id, DEFAULT);
        if (!fuji_read_appkey(key_id, &read, (uint8_t *)destination))
            read = 0;
    #endif

    // Add string terminator after the data ends in case it is being interpreted as a string
    destination[read] = 0;
    return read;
}

void write_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, uint16_t count, char
    *data)
{
    #ifdef CUSTOM_FUJINET_CALLS
        custom_write_appkey(creator_id, app_id, key_id, count, data);
    #else
        fuji_set_appkey_details(creator_id, app_id, DEFAULT);
        fuji_write_appkey(key_id, count, (uint8_t *)data);
    #endif
}
```

SRC/MISC.H 251 LINES

```
/*
 * Common include file for project.
*/

/*****************************************************************
 *
 * Do NOT include standard library headers (e.g. conio, std*).
 * Instead, add to standard_lib.h, which gets included in this file
 *
 *****************************************************************/

#ifndef MISC_H
#define MISC_H

#include "standard_lib.h"
#include "platform-specific/graphics.h"
#include "platform-specific/util.h"
#include "platform-specific/input.h"
#include "platform-specific/sound.h"
#include "platform-specific/vars.h"

// Client version string to send to server
#define API_CLIENT_VERSION "2"

// FujiNet AppKey settings. These should not be changed
#define AK_LOBBY_CREATOR_ID 1   // FUJINET Lobby
#define AK_LOBBY_APP_ID 1       // Lobby Enabled Game
#define AK_LOBBY_KEY_USERNAME 0 // Lobby Username key
#define AK_LOBBY_KEY_SERVER 5   // Battleship registered as Lobby appkey 5 at
    https://github.com/FujiNetWIFI/fujinet-firmware/wiki/SIO-Command-$DC-Open-App-Key

// Battleship
#define AK_CREATOR_ID 0xE41C // Eric Carr's creator id
#define AK_APP_ID 5          // Battleship App ID
#define AK_KEY_PREFS 0       // Preferences

#define PLAYER_MAX 4

#define FUJITZEE_SCORE 14

#define STATUS_LOBBY 0
#define STATUS_PLACE_SHIPS 1
#define STATUS_GAMESTART 10
#define STATUS_MISS 11     // Attack did not hit any ships
#define STATUS_HIT 12      // A ship was hit
#define STATUS_SUNK 13     // Also implies a ship was hit
#define STATUS_GAMEOVER 99 // Also implies a ship was hit

// Set at start to force full redraw
#define STATE_INVALID 200

// Player status
#define PLAYER_STATUS_DEFAULT 0
#define PLAYER_STATUS_DEFEATED 1
```

```
#define PLAYER_STATUS_VIEWING 2
#define PLAYER_STATUS_READY 3
#define PLAYER_STATUS_PLACE_SHIPS 10

#define FIELD_ATTACK 1
#define FIELD_MISS 2

#define LEGEND_SHIP_DESTROYED 0
#define LEGEND_SHIP_INTACT 1

#define DRAWSHIP_SHOW 0
#define DRAWSHIP_HIDE 1

typedef struct
{
    char table[9];
    char name[21];
    char players[6];
} Table;

typedef struct
{
    char name[9];
    uint8_t playerStatus;
    uint8_t gamefield[100];
    uint8_t shipsLeft[5];
} Player;

typedef struct
{
    char name[9];
    uint8_t ready;
} LobbyPlayer;

typedef struct
{
    uint8_t count;
    Table table[10];
} Tables;

typedef struct
{
    uint8_t playerCount;
    char prompt[33];
    uint8_t status;
    uint8_t playerStatus;
    int8_t activePlayer;
    uint8_t moveTime;
    uint8_t lastAttackPos;
    // First 5 are my ships, last 5 are winner, sent at game over
    uint8_t myShips[10];
    Player players[PLAYER_MAX];
} Game;

typedef struct
{
    uint8_t playerCount;
    char prompt[33];
    uint8_t status;
    uint8_t playerStatus;
    int8_t activePlayer;
    uint8_t moveTime;
    char serverName[21];
    LobbyPlayer players[PLAYER_MAX];
} Lobby;

typedef union
{
    uint8_t firstByte;
    Game game;
    Lobby lobby;
    Tables tables;
} ClientState;

#ifdef BUILD_COLECO
/*
  The ColecoVision has 1K of RAM and this union is 509 bytes of it, so the
  state is never copied down: it is read in place out of the FujiNet
  cartridge's reply window, which is what that window is 1K and addressable
  for. See src/coleco/vars.h and src/coleco/fujinet.c.

  It is therefore READ-ONLY -- a store here goes nowhere, the cartridge cannot
  even see a write cycle -- and it is valid only until the next fuji_* call of
  any kind, because every transaction repaints the window.
*/
#define clientState (*(ClientState *) COLECO_REPLY_WINDOW)
#else
extern ClientState clientState;
#endif

/*
  The gamefield shadow only ever answers "was this cell empty before" (the
  shoot animation trigger), so on the RAM-starved ColecoVision it is packed to
  one bit per cell - 13 bytes per player instead of 100. Everywhere else it
  stays byte-per-cell. gamelogic.c's packGamefield() fills it and
  GAMEFIELD_IS_EMPTY() is the only reader.
*/
#ifdef BUILD_COLECO
#define GAMEFIELD_BYTES 13
#define GAMEFIELD_IS_EMPTY(p, pos) (!(state.gamefield[p][(pos) >> 3] & (1 << ((pos)&7))))
#else
#define GAMEFIELD_BYTES 100
#define GAMEFIELD_IS_EMPTY(p, pos) (state.gamefield[p][pos] == 0)
#endif

typedef struct
{

    // Internal game state
    uint8_t prevPlayerCount;
    uint8_t prevStatus;
    uint8_t prevPlayerStatus;
    uint8_t apiCallWait;

    int8_t prevActivePlayer;
    int8_t prevAttackPos;

    bool countdownStarted;
    bool waitingOnEndGameContinue;
    bool drawBoard;
    bool inGame;

    // Track gamefield state - used to know when to fire shoot animation
    uint8_t gamefield[PLAYER_MAX][GAMEFIELD_BYTES];

    // Track ships left - used to know when to fire sink animation
    uint8_t shipsLeft[PLAYER_MAX][5];
} GameState;

typedef struct
{
    uint16_t key;
    bool trigger;
    int8_t dirX;
    int8_t dirY;
} InputStruct;

typedef struct
{
    uint8_t debugFlag; // 0xFF to use localhost instead of server
    bool seenHelp;
    uint8_t disableSound;
    uint8_t colorMode;
    uint8_t reserved[20]; // Reserve blank space for future
} PrefsStruct;

/*
  Scratch buffer sizes. The ColecoVision has 1K of RAM in total, so these are
  cut to what each one actually has to hold rather than to a round number.
  On the ColecoVision tempBuffer also doubles as stateclient.c's url build
  buffer, so its floor is the full url: "n:" + endpoint(49) + the longest path
  ("place/199,..." = 25) + query(35) + "&bin=1&v=2" = 122 (which also covers
  its other floors, the 100-cell ship-placement occupancy map and
  MAX_APPKEY_LEN+1). query is "?table=" + 8 + "&player=" + 11.
*/
#ifdef BUILD_COLECO
#define TEMP_BUFFER_LEN 124
#define QUERY_LEN 36
#define URL_BUFFER_LEN 124
#else
#define TEMP_BUFFER_LEN 128
#define QUERY_LEN 50
#define URL_BUFFER_LEN 160
#endif

extern char tempBuffer[TEMP_BUFFER_LEN];
extern char serverEndpoint[50];
extern const char localServer[];
extern char query[QUERY_LEN];
extern char playerName[12];
extern const uint8_t shipSize[5];

#ifdef USE_PLATFORM_NAME_ENTRY
// Keyboard-less platforms implement name entry themselves (e.g. the
// ColecoVision on-screen keyboard in src/coleco/osk.c)
void platformNameEntry(uint8_t x, uint8_t y, uint8_t max, char *buffer);
#endif

extern GameState state;
extern InputStruct input;
extern PrefsStruct prefs;

// Common local scope temp variables

void pause(uint8_t frames);
void clearCommonInput();
```

```
240 void readCommonInput();
241 void loadPrefs();
242 void savePrefs();
243
244 /// @brief Helper method to write to an appkey
245 void write_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, uint16_t count, char
  +     *data);
246
247 /// @brief Helper method to read from an appkey.
248 /// NULL will be appended to data in case this is a string, though the length returned will not
  +     consider the NULL.
249 uint16_t read_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, char *destination);
250
251 #endif /* MISC_H */
```

SRC/SCREENS.C 543 LINES

```
  1 /*********************************************************************
  2  *
  3  * Do NOT include standard library headers (e.g. conio, std*).
  4  * Instead, add to standard_lib.h, which gets included in misc.h
  5  *
  6  *********************************************************************/
  7
  8 #include "misc.h"
  9 #include "stateclient.h"
 10 #include "screens.h"
 11 #include "gamelogic.h"
 12
 13 #define PLAYER_NAME_MAX 8
 14 #define PLAYER_BOX_TOP 13
 15 #define REMOVE_PLAYER_KEY '/'
 16 #define INGAME_MENU_X WIDTH / 2 - 8
 17
 18 // Key labels shown to the player. Platforms whose input maps these shortcuts
 19 // onto other physical keys (e.g. the ColecoVision keypad) override them in
 20 // their vars.h so the labels match what the player actually presses.
 21 #ifndef TABLE_STATUS_TEXT
 22 #define TABLE_STATUS_TEXT "Refresh    Help     Name    Quit"
 23 #endif
 24 #ifndef MENU_QUIT_TEXT
 25 #define MENU_QUIT_TEXT "  Q: quit game"
 26 #endif
 27 #ifndef MENU_HELP_TEXT
 28 #define MENU_HELP_TEXT "  H: how to play"
 29 #endif
 30 // The ON variant carries a trailing space so toggling from OFF erases the F
 31 #ifndef MENU_SOUND_ON_TEXT
 32 #define MENU_SOUND_ON_TEXT "  S: sound ON "
 33 #endif
 34 #ifndef MENU_SOUND_OFF_TEXT
 35 #define MENU_SOUND_OFF_TEXT "  S: sound OFF"
 36 #endif
 37
 38 #ifndef MENU_CLOSE_TEXT
 39 #define MENU_CLOSE_TEXT "press TRIGGER/SPACE to close"
 40 #endif
 41
 42 // Logo vertical position - set in [platform]/vars.h to override if needed
 43 #ifndef LOGO_Y
 44 #define LOGO_Y 1
 45 #endif
 46
 47 bool inBorderedScreen = false, prevBorderedScreen = false, savedScreen = false;
 48
 49 bool saveScreen()
 50 {
 51     prevBorderedScreen = inBorderedScreen;
 52     return savedScreen = saveScreenBuffer();
 53 }
 54
 55 bool restoreScreen()
 56 {
 57     if (savedScreen)
 58     {
 59         savedScreen = false;
 60         inBorderedScreen = prevBorderedScreen;
 61         restoreScreenBuffer();
 62         return true;
 63     }
 64     return false;
 65 }
 66
 67 /// @brief Shows information about the game
 68 void showHelpScreen()
 69 {
 70     uint8_t y;
 71 #define X WIDTH / 2 - 16
 72
 73     resetScreen();
 74
 75     drawTextAlt(WIDTH / 2 - 14, 1, "how to play FUJI BATTLESHIP");
 76     drawLine(WIDTH / 2 - 16, 2, 31);
 77     y = 3;
 78
 79     //                1234567890123456789012345678901
 80     y++;
 81     drawTextAlt(X, y, "BATTLESHIP is a classic game of");
 82     y++;
 83     drawTextAlt(X, y, "sea battle for up to 4 players!");
 84
 85     y += 3;
 86     centerTextAlt(y, "GAME PLAY");
 87
 88     y++;
 89     y++;
 90     drawTextAlt(X, y, "player starts with FIVE ships,");
 91     y++;
 92     drawTextAlt(X, y, "taking turns attacking until");
 93     y++;
 94     drawTextAlt(X, y, "only ONE player remains.");
 95     y++;
 96     y++;
 97     drawTextAlt(X, y, "when you attack a location, ALL");
 98     y++;
 99     drawTextAlt(X, y, "ENEMY players are attacked.");
100
101     y++;
102     y++;
103     y++;
104     drawTextAlt(X, y, "use your strategy and luck to");
105     y++;
106     drawTextAlt(X, y, "be the last player standing!");
107
108     centerStatusText("press any key to close");
109
110     clearCommonInput();
111     cgetc();
112 }
113
114 /// @brief Action called in Welcome Screen to check if a server name is stored in an app key
115 void welcomeActionVerifyServerDetails()
116 {
117     uint8_t i;
118     // Read server endpoint into server and query
119     read_appkey(AK_LOBBY_CREATOR_ID, AK_LOBBY_APP_ID, AK_LOBBY_KEY_SERVER, tempBuffer);
120
121     i = (uint8_t)strlen(tempBuffer);
122     if (i)
123     {
124         // Split "server?query" into separate variables
125         while (--i)
126         {
127             if (tempBuffer[i] == '?')
128             {
129                 strcpy(query, tempBuffer + i);
130                 tempBuffer[i] = 0;
131                 strcpy(serverEndpoint, tempBuffer);
132                 break;
133             }
134         }
135     }
136 }
137
138 void drawLogo()
139 {
140     drawBox(WIDTH / 2 - 8, 0, 15, 1);
141     drawTextAlt(WIDTH / 2 - 7, LOGO_Y, "FUJI battleship");
142
143 }
144
145 void showPlayerNameScreen()
146 {
147     uint8_t i, y;
148     resetScreen();
149     drawLogo();
150
151     centerText(13, "ENTER YOUR NAME");
152     drawBox(WIDTH / 2 - PLAYER_NAME_MAX / 2 - 1, 16, PLAYER_NAME_MAX + 1, 1);
153     drawText(WIDTH / 2 - PLAYER_NAME_MAX / 2, 17, playerName);
154
155     i = (uint8_t)strlen(playerName);
156
157     clearCommonInput();
158 #ifdef USE_PLATFORM_NAME_ENTRY
159     platformNameEntry(WIDTH / 2 - PLAYER_NAME_MAX / 2, 17, PLAYER_NAME_MAX, playerName);
160 #else
161     while (!inputFieldCycle(WIDTH / 2 - PLAYER_NAME_MAX / 2, 17, PLAYER_NAME_MAX, playerName))
162         ;
163 #endif
164
165     for (y = 13; y < 19; ++y)
166         centerText(y, "                ");
167
168     write_appkey(AK_LOBBY_CREATOR_ID, AK_LOBBY_APP_ID, AK_LOBBY_KEY_USERNAME,
```

```
 +        strlen(playerName), playerName);
169 }
170
171 /// @brief Action called in Welcome Screen to verify player has a name
172 void welcomeActionVerifyPlayerName()
173 {
174     char *ptr;
175     // Read player's name from app key
176     read_appkey(AK_LOBBY_CREATOR_ID, AK_LOBBY_APP_ID, AK_LOBBY_KEY_USERNAME, tempBuffer);
177     tempBuffer[8] = 0;
178     strcpy(playerName, tempBuffer);
179
180     // Capture username if player didn't come in from the lobby
181     if (!playerName[0])
182     {
183         showPlayerNameScreen();
184     }
185
186     // Convert to lowercase
187     ptr = playerName;
188     while (*ptr)
189     {
190         if (*ptr >= 65 && *ptr <= 90)
191             *ptr += 32;
192         ptr++;
193     }
194 }
195
196 /// @brief Shows the Welcome Screen with Logo. Asks player's name
197 void showWelcomeScreen()
198 {
199     // Retrieve the main player's name
200     welcomeActionVerifyPlayerName();
201
202     // Parse server url from app key if present
203     welcomeActionVerifyServerDetails();
204
205     // If first run, show the help screen
206     if (!prefs.seenHelp)
207     {
208         prefs.seenHelp = true;
209         savePrefs();
210         showHelpScreen();
211     }
212
213     pause(30);
214 }
215
216 #define TWID 26
217 #define RMAR (WIDTH / 2 + TWID / 2)
218 #define LMAR (WIDTH / 2 - TWID / 2)
219 /// @brief Shows a screen to select a table to join
220 void showTableSelectionScreen()
221 {
222     // tableCount is the local truth for the table list: clientState is not
223     // writable on every platform (the ColecoVision reads it out of cartridge
224     // ROM), so the "no tables yet" state lives here, not in clientState.
225     uint8_t shownCursor, tableIndex, blinkCursor, i, j, tableCount = 0;
226     Table *table;
227     state.inGame = tableIndex = blinkCursor = 0;
228
229     resetScreen();
230
231     // An empty query means a table needs to be selected
232     while (strlen(query) == 0)
233     {
234         // Show names of local player(s)
235
236         strcpy(tempBuffer, "HELLO ");
237         strcat(tempBuffer, playerName);
238         centerTextAlt(20, tempBuffer);
239
240         if (tableCount > 0)
241         {
242             for (i = 0; i < tableCount; ++i)
243             {
244                 drawSpace(LMAR, 9 + i * 2, TWID);
245             }
246         }
247         waitvsync();
248         centerText(12, "      refreshing game list..      ");
249
250         drawLogo();
251
252         centerText(4, "choose a game to join");
253         drawText(LMAR, 7, "game");
254         drawText(RMAR - 7, 7, "players");
255         drawLine(LMAR, 8, TWID);
256
257         // waitvsync();
258
259         tableCount = apiCall("tables") == API_CALL_SUCCESS ? clientState.tables.count : 0;
260         if (tableIndex >= tableCount)
261             tableIndex = 0;
262
263         if (tableCount > 0)
264         {
265             drawSpace(LMAR, 12, TWID);
266             for (i = 0; i < tableCount; ++i)
267             {
268                 table = &clientState.tables.table[i];
269                 j = 9 + i * 2;
270                 drawTextAlt(LMAR, j, table->name);
271                 drawTextAlt(RMAR - 5, j, table->players);
272
273                 if (table->players[0] > '0')
274                 {
275                     drawIcon(RMAR - 7, j, ICON_PLAYER);
276                 }
277                 // cgetc();
278             }
279         }
280         else
281         {
282             centerText(12, "no servers are available");
283         }
284
285         centerStatusText(TABLE_STATUS_TEXT);
286
287 #ifdef COLOR_TOGGLE
288         if (prefs.color)
289         {
290             drawLine(6, HEIGHT, 1);  // R
291             drawLine(15, HEIGHT, 1); // H
292             drawLine(21, HEIGHT, 1); // P
293             drawLine(30, HEIGHT, 1); // Q
294         }
295 #endif
296
297         shownCursor = !tableCount;
298
299         clearCommonInput();
300         while (!input.trigger || !tableCount)
301         {
302
303             if (tableCount)
304             {
305                 drawIcon(LMAR - 2, 9 + tableIndex * 2, blinkCursor < 50 ? ICON_MARK :
 +                    ICON_MARK_ALT);
306             }
307
308             waitvsync();
309             blinkCursor = (blinkCursor + 1) % 60;
310             readCommonInput();
311
312             if (input.key == 'h' || input.key == 'H')
313             {
314                 saveScreen();
315                 showHelpScreen();
316                 if (!restoreScreen())
317                 {
318                     resetScreen();
319                     break;
320                 }
321             }
322             else if (input.key == 'r' || input.key == 'R')
323             {
324                 drawBlank(LMAR - 2, 9 + tableIndex * 2);
325                 break;
326             }
327             else if (input.key == 'c' || input.key == 'C')
328             {
329                 cycleNextColor();
330                 savePrefs();
331 #ifdef COLOR_CYCLE_REQUIRES_REDRAW
332                 break;
333 #endif
334             }
335             else if (input.key == 's' || input.key == 'S')
336             {
337                 prefs.disableSound = !prefs.disableSound;
338                 soundCursor();
339                 savePrefs();
340                 // savePrefs() ran a FujiNet transaction; on platforms where
341                 // the table list lives in the cartridge's reply window it is
342                 // stale now, so refresh rather than keep navigating it
343                 break;
344             }
345             else if (input.key == 'n' || input.key == 'N')
346             {
347                 showPlayerNameScreen();
348                 resetScreen();
349                 break;
350             }
351             else if (input.key == 'q' || input.key == 'Q')
352             {
```

```
            quit();
            // quit() only returns on failure, after FujiNet transactions
            // of its own - same stale-window refresh as above
            break;
        } /*else if (input.key != 0) {
            itoa(input.key, tempBuffer, 10);
            drawStatusText(tempBuffer);
        }*/

        if (!shownCursor || (tableCount > 0 && input.dirY))
        {

            // Visually unselect old table
            table = &clientState.tables.table[tableIndex];
            j = table->name[0]; // Reference table so cmoc 0.1.96 optimizer does not corrupt memory
            j = 9 + tableIndex * 2;

            drawBlank(LMAR - 2, j);
            drawTextAlt(LMAR, j, table->name);
            drawTextAlt(RMAR - 5, j, table->players);

            // Move table index to new table
            tableIndex = (input.dirY + tableIndex + tableCount) % tableCount;

            // Visually select new table

            table = &clientState.tables.table[tableIndex];
            j = table->name[0]; // Reference table so cmoc 0.1.96 optimizer does not corrupt memory
            j = 9 + tableIndex * 2;

            drawIcon(LMAR - 2, j, ICON_MARK);
            drawText(LMAR, j, table->name);
            drawText(RMAR - 5, j, table->players);

            soundCursor();

            // Housekeeping - allows platform specific housekeeping, like stopping Attract/screensaver mode in Atari
            housekeeping();

            shownCursor = 1;
        }
    }

    if (input.trigger)
    {
        soundSelect();

        // Clear screen and write server name
        resetScreen();
        centerText(15, clientState.tables.table[tableIndex].name);

        strcpy(query, "?table=");
        strcat(query, clientState.tables.table[tableIndex].table);

        // Combine server endpoint and query for final base url
        strcpy(tempBuffer, serverEndpoint);
        strcat(tempBuffer, query);

        //  Update server app key in case of reboot
        write_appkey(AK_LOBBY_CREATOR_ID, AK_LOBBY_APP_ID, AK_LOBBY_KEY_SERVER, strlen(tempBuffer), tempBuffer);
    }
}

centerTextAlt(17, "connecting to server");
progressAnim(19);

// Append player name to query
strcat(query, "&player=");
strcat(query, playerName);

// Replace space with + for pshowWelcomScreenlayer name
i = (uint8_t)strlen(query);
while (--i)
    if (query[i] == ' ')
        query[i] = '+';

// Reset the game state
clearRenderState();
state.waitingOnEndGameContinue = false;

// Clear gamefield
memset(state.gamefield, 0, sizeof(state.gamefield));

// Join table
apiCall("state");

// Reduce wait count for an immediate call
state.apiCallWait = 0;

    state.inGame = true;
}

/// @brief shows in-game menu
void showInGameMenuScreen()
{
    static uint8_t y, i, wait;

    saveScreen();
    state.inGame = false;
    i = wait = 1;
    while (i)
    {

        resetScreen();
        y = HEIGHT / 2 - 3;
        drawTextAlt(INGAME_MENU_X, y, MENU_QUIT_TEXT);
        drawTextAlt(INGAME_MENU_X, y += 2, MENU_HELP_TEXT);
        if (prefs.colorMode)
            drawTextAlt(INGAME_MENU_X, y += 2, "  C: color mode");

        drawTextAlt(INGAME_MENU_X, y += 2, prefs.disableSound ? MENU_SOUND_OFF_TEXT : MENU_SOUND_ON_TEXT);

        drawBox(INGAME_MENU_X - 2, HEIGHT / 2 - 5, 19, y - (HEIGHT / 2 - 5) + 1);

        centerTextAlt(HEIGHT - 2, MENU_CLOSE_TEXT);

        // centerTextAlt(y + 6, tempBuffer);
        clearCommonInput();
        i = 1;
        while (i == 1)
        {
            waitvsync();
            readCommonInput();
            if (input.trigger)
            {
                i = 0;
                break;
            }
            switch (input.key)
            {
            case 's':
            case 'S':
                prefs.disableSound = !prefs.disableSound;
                drawTextAlt(INGAME_MENU_X, y, prefs.disableSound ? MENU_SOUND_OFF_TEXT : MENU_SOUND_ON_TEXT);
                soundSelect();
                savePrefs();
                break;
            case 'c':
            case 'C':
                cycleNextColor();
                savePrefs();
#ifdef COLOR_CYCLE_REQUIRES_REDRAW
                i = 2;
#endif
                break;

            case 'h':
            case 'H':
                // Back to the menu afterwards (i = 2 redraws it), not on
                // into 'q': reading the rules must not leave the table
                showHelpScreen();
                i = 2;
                break;

            case 'q':
            case 'Q':
                resetScreen();
                centerText(10, "please wait");

                //  Clear server app key in case of reboot
                write_appkey(AK_LOBBY_CREATOR_ID, AK_LOBBY_APP_ID, AK_LOBBY_KEY_SERVER, 0, (char *)"");

                // Inform server player is leaving
                apiCall("leave");

                progressAnim(12);

                // Clear query so a new table will be selected
                strcpy(query, "");
                showTableSelectionScreen();

                return;
            }
        }
    }

    // Show game screen again before returning

    clearCommonInput();
```

```
532
533     state.inGame = true;
534     if ((!state.drawBoard && !restoreScreen()) || state.drawBoard)
535     {
536         // Re-fetch before redrawing rather than rendering clientState here:
537         // the menu's savePrefs()/appkey writes ran FujiNet transactions, and
538         // on platforms where the state lives in the cartridge's reply window
539         // it is stale now. The main loop redraws on the next poll.
540         clearRenderState();
541         state.apiCallWait = 0;
542     }
543 }
```

SRC/SCREENS.H — 50 LINES

```
 1 /*****************************************************************
 2  *
 3  * Do NOT include standard library headers (e.g. conio, std*).
 4  * Instead, add to standard_lib.h, which gets included in misc.h
 5  *
 6  *****************************************************************/
 7
 8 #ifndef SCREENS_H
 9 #define SCREENS_H
10
11 extern uint8_t inputField[20];
12
13 /// @brief Save screen to memory for quick recall - returns true if successful
14 bool saveScreen();
15
16 /// @brief Restore screen - returns true if successful
17 bool restoreScreen();
18
19 /// @brief Clear screen
20 void resetScreen();
21
22 /// @brief Shows information about the game
23 void showHelpScreen();
24
25 /// @brief Action called in Welcome Screen to check if a server name is stored in an app key
26 void welcomeActionVerifyServerDetails();
27
28 /// @brief Action called in Welcome Screen to verify player has a name
29 void welcomeActionVerifyPlayerName();
30
31 /// @brief Shows the Welcome Screen with Logo. Asks player's name
32 void showWelcomeScreen();
33
34 /// @brief Shows a screen to select a table to join
35 void showTableSelectionScreen();
36
37 /// @brief Shows main game play screen (table and cards)
38 void showGameScreen();
39
40 /// @brief shows in-game menu
41 void showInGameMenuScreen();
42
43 /// @brief Allow the player to modify their name
44 void showPlayerNameScreen();
45
46 /// @brief Draw the title
47 void drawLogo();
48
49 #endif /*SCREENS_H*/
50
```

SRC/STATECLIENT.C — 94 LINES

```
 1 /*****************************************************************
 2  *
 3  * Do NOT include standard library headers (e.g. conio, std*).
 4  * Instead, add to standard_lib.h, which gets included in misc.h
 5  *
 6  *****************************************************************/
 7
 8 #include "misc.h"
 9 #include "stateclient.h"
10 #include "fujinet-network.h"
11
12 // Internal to this file
13 #ifdef BUILD_COLECO
14 // 1K machine: the URL is staged in tempBuffer instead of its own buffer. Safe
15 // because no apiCall() path is ever tempBuffer itself (getStateFromServer()
16 // passes the move buffer or a literal), and nothing that lives in tempBuffer
17 // is needed across an API call.
18 #define url tempBuffer
19 #else
20 static char url[URL_BUFFER_LEN];
21 #endif
22 char *requestedMove;
23
24 #ifdef CUSTOM_FUJINET_CALLS
25 // Optional: This would be implemented in platform-specific code for emulators, etc
26 int16_t custom_network_call(char *url, uint8_t *buffer, uint16_t max_len);
27 #endif
28
29 /*
30  * @brief Makes an Api call, returning true if valid payload received
31  * Returns API_CALL_*:
32  *  1 - successfully received a payload
33  *  2 - async - received payload, still in process, call for more data
34  *  0 - error - aborted
35  */
36 uint8_t apiCall(const char *path)
37 {
38     static int16_t read;
39
40     strcpy(url, "n:");
41     strcat(url, serverEndpoint);
42     strcat(url, path);
43     strcat(url, query);
44     strcat(url, query[0] ? "&bin=1&v=" API_CLIENT_VERSION : "?bin=1&v=" API_CLIENT_VERSION);
45
46     // Allow platform-specific override (e.g. for mocking network calls in
47     // emulator, or the ColecoVision's read-into-cartridge-window transport).
48     // sizeof(Game), not sizeof(clientState.game): on the ColecoVision
49     // clientState is a macro for a cast-and-dereference of the reply window,
50     // and sccz80 will not take sizeof() of that expression.
51 #ifdef CUSTOM_FUJINET_CALLS
52     read = custom_network_call(url, &clientState.firstByte, sizeof(Game));
53 #else
54     if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE))
55     {
56         return API_CALL_ERROR;
57     }
58
59     read = network_read(url, &clientState.firstByte, sizeof(Game));
60     network_close(url);
61 #endif
62
63     // On error clientState is unspecified (and on the ColecoVision it is not
64     // writable at all) - callers must trust the return code, not the state.
65     if (read <= 0)
66     {
67         return API_CALL_ERROR;
68     }
69
70     return API_CALL_SUCCESS;
71 }
72
73 void sendMove(char *move)
74 {
75     if (move != NULL)
76         state.apiCallWait = 0;
77
78     requestedMove = move;
79 }
80
81 uint8_t getStateFromServer()
82 {
83     // The path is handed to apiCall() as-is - never staged in tempBuffer,
84     // which doubles as the url build buffer on the ColecoVision
85     const char *path = "state";
86
87     if (requestedMove)
88     {
89         path = requestedMove;
90         requestedMove = NULL;
91     }
92
93     return apiCall(path);
94 }
```

SRC/STATECLIENT.H — 24 LINES

```
 1 /*****************************************************************
 2  *
 3  * Do NOT include standard library headers (e.g. conio, std*).
 4  * Instead, add to standard_lib.h, which gets included in misc.h
 5  *
 6  *****************************************************************/
 7
 8 #ifndef STATECLIENT_H
 9 #define STATECLIENT_H
10
11 #define API_CALL_ERROR (0)
12 #define API_CALL_SUCCESS (1)
13 #define API_CALL_PENDING (2)
14
15 #define STATE_UPDATE_ERROR (0)
16 #define STATE_UPDATE_CHANGE (1)
17 #define STATE_UPDATE_NOCHANGE (2)
18
```

```
19 void updateState(bool isTables);
20 uint8_t getStateFromServer();
21 uint8_t apiCall(const char *path );
22 void sendMove(char* move);
23
24 #endif /* STATECLIENT_H */
```

SRC/STANDARD_LIB.H 84 LINES

```
 1 #ifndef STANDARD_LIB_H
 2 #define STANDARD_LIB_H
 3
 4 /*
 5  * All standard library includes should be added here for use by
 6  * platform agnostic code in the "src" directory.
 7  *
 8  * This creates one place to manage platform/compiler specific standard headers.
 9  *
10  * This ONLY applies to code in the immediate "src" directory.
11  * Files in "/src/[platform]" may reference misc.h, this file, or headers directly as needed.
12  */
13
14 // Include FujiNet Fuji header, which sets std int and bool types (uint8_t, bool, etc)
15 #include "fujinet-fuji.h"
16
17 #ifdef _CMOC_VERSION_
18 #include <coco.h>
19
20 /* (Non blocking) Return a character if there's a key waiting, otherwise 0.
21  */
22 unsigned char kbhit (void);
23
24 /* (Blocking) Return a character from the keyboard. If there is no key waiting,
25  * the function waits until the user does press a key.
26  */
27 char cgetc (void);
28
29 #elif defined(__ADAM__)
30 // z88dk's conio.h maps cgetc() onto getk(), which is a dead stub on the
31 // Adam target - kbhit/cgetc are implemented over EOS in src/adam/input.c
32 #include <stdlib.h>
33 #include <stdio.h>
34 #include <string.h>
35
36 /* (Non blocking) Return non-zero if a key is waiting. */
37 unsigned char kbhit (void);
38
39 /* (Blocking) Return a character from the keyboard.
40  * Must be unsigned: sccz80 chars are signed, and the Adam's arrow/smart
41  * keys are 0x81-0xA8 - a plain char return would sign-extend them and no
42  * KEY_* case in the input switch would ever match. */
43 unsigned char cgetc (void);
44
45 #elif defined(BUILD_COLECO) || defined(BUILD_NES) || defined(BUILD_SMS)
46 // No keyboard at all: src/coleco/input.c synthesizes key codes from the
47 // twelve-key keypad and the second fire button, src/nes/input.c and
48 // src/sms/input.c from the joypad's buttons. unsigned for the same
49 // sign-extension reason as the Adam above.
50 #include <stdlib.h>
51 #include <stdio.h>
52 #include <string.h>
53
54 /* (Non blocking) Return non-zero if a synthesized key is waiting. */
55 unsigned char kbhit (void);
56
57 /* (Blocking) Return a synthesized key code. */
58 unsigned char cgetc (void);
59
60 #elif defined(__H89__)
61 // The H19 terminal: src/h89/input.c reads the console through HDOS SCIN or
62 // the CP/M BIOS and decodes the keypad's ESC A-D into arrow codes 0xA0-0xA3.
63 // z88dk's conio.h would bring getk(), which on CP/M drains the type-ahead and
64 // keeps only the last byte. unsigned for the same sign-extension reason as
65 // the Adam above.
66 #include <stdlib.h>
67 #include <stdio.h>
68 #include <string.h>
69
70 /* (Non blocking) Return non-zero if a key is waiting. */
71 unsigned char kbhit (void);
72
73 /* (Blocking) Return a key code. */
74 unsigned char cgetc (void);
75
76 #else
77 // Standard libraries
78 #include <conio.h>
79 #include <stdlib.h>
80 #include <stdio.h>
81 #include <string.h>
82 #endif
83
84 #endif /* STANDARD_LIB_H */
```

SRC/PLATFORM-SPECIFIC/GRAPHICS.H 103 LINES

```
 1 /*
 2  Platform specific graphics commands
 3 */
 4
 5 #ifndef GRAPHICS_H
 6 #define GRAPHICS_H
 7 #include "../standard_lib.h"
 8
 9 // Call to clear the screen, passing true to preserve borders
10 void resetScreen();
11
12 uint8_t cycleNextColor();
13
14 /// @brief Draw a string of text, default color
15 void drawText(uint8_t x, uint8_t y, const char *s);
16
17 /// @brief Draw a string of text. Draw alternate color for CAPITALIZED lettes.
18 void drawTextAlt(uint8_t x, uint8_t y, const char *s);
19
20 /// @brief Draw a single character - used for icons
21 void drawIcon(uint8_t x, uint8_t y, uint8_t icon);
22
23 /// @brief Draw (or erase) a ship at given size and position.
24 /// @param quadrant player index (0-3) for positioning purpose
25 /// @param size ship size (2-5)
26 /// @param pos ship position 0-99, add 100 for vertical
27 /// @param hide 0=show, 1=hide
28 void drawShip(uint8_t quadrant, uint8_t size, uint8_t pos, bool hide);
29
30 /// @brief Draw a ship either in whole or destroyed, in the legend (side trays)
31 /// @param player player index (0-3) for positioning purpose
32 /// @param index ship index (0-4)
33 /// @param size ship size (2-5)
34 /// @param status ship status: 0=destroyed, 1=intact
35 void drawLegendShip(uint8_t player, uint8_t index, uint8_t size, uint8_t status);
36
37 /// @brief Draw the player's name by their gamefield
38 /// @param player player index (0-3) for positioning purpose
39 /// @param name player name
40 /// @param active whether this is the currently active player
41 void drawPlayerName(uint8_t player, const char *name, bool active);
42
43 /// @brief Draw end game message
44 /// @param message
45 void drawEndgameMessage(const char *message);
46
47 /// @brief Draw the gamefield for a given player/quadrant
48 /// @param quadrant player index (0-3). Starting bottom left and moving clockwise
49 /// @param field pointer to 100 byte gamefield array from server
50 void drawGamefield(uint8_t quadrant, uint8_t *field);
51
52 /// @brief Draw/update a single cell (attackPos) for the the specified gamefield
53 /// @param quadrant [0-3] player index
54 /// @param gamefield pointer to 100 byte gamefield array from server
55 /// @param attackPos [0-99] position of cell to update
56 /// @param anim [0/1,10-15] : [0/1] toggle between two "hit" sprites for animation, [10-15]
 +     attack animation
57 void drawGamefieldUpdate(uint8_t quadrant, uint8_t *gamefield, uint8_t attackPos, uint8_t
 +     anim);
58
59 /// @brief  Draw the cursor at the specified position on the specified gamefield
60 /// @param quadrant     [0-3] player index
61 /// @param x    [0-9] cursor x position
62 /// @param y    [0-9] cursor y position
63 /// @param gamefield pointer to 100 byte gamefield array from server
64 /// @param blink [0-2] used to cycle between different cursor sprites
65 void drawGamefieldCursor(uint8_t quadrant, uint8_t x, uint8_t y, uint8_t *gamefield, uint8_t
 +     blink);
66
67 /// @brief Draw the clock icon at the bottom right of the screen
68 void drawClock();
69
70 /// @brief Draw or erase the connection status icon at the bottom left of the screen
71 void drawConnectionIcon(bool show);
72
73 /// @brief Draw a single blank space character at given position
74 void drawBlank(uint8_t x, uint8_t y);
75
76 /// @brief Draw W blank space characters at given position
77 void drawSpace(uint8_t x, uint8_t y, uint8_t w);
78
79 /// @brief Draw a horizontal line of width W characters at location
80 void drawLine(uint8_t x, uint8_t y, uint8_t w);
81
82 /// @brief Draw a box at given location
83 void drawBox(uint8_t x, uint8_t y, uint8_t w, uint8_t h);
84
85 /// @brief Draw the main game board layout for the specified number of players
```

```
86 /// The player count dictates the general layout
87 void drawBoard(uint8_t playerCount);
88
89 /// @brief Call to save screen buffer for later restore. Returns false if screen buffer not
 +     supported
90 bool saveScreenBuffer();
91
92 /// @brief Call to restore screen buffer
93 void restoreScreenBuffer();
94
95 /// @brief Initialize graphics mode
96 void initGraphics();
97
98 /// @brief Reset graphics mode to default state
99 void resetGraphics();
100
101 /// @brief Wait for vertical sync
102 void waitvsync();
103 #endif /* GRAPHICS_H */
```

SRC/PLATFORM-SPECIFIC/INPUT.H 14 LINES

```
1 /*
2   Platform specific input.
3 */
4 #ifndef INPUT_H
5 #define INPUT_H
6
7 // Include platform specific vars
8 #include "../standard_lib.h"
9
10 // Platform specific implementations
11 uint8_t readJoystick();
12
13 #endif /* INPUT_H */
14
```

SRC/PLATFORM-SPECIFIC/SOUND.H 27 LINES

```
1 /*
2   Platform specific sound functions
3 */
4 #ifndef SOUND_H
5 #define SOUND_H
6
7 void initSound();
8
9 void disableKeySounds();
10 void enableKeySounds();
11 void soundCursor();
12 void soundSelect();
13 void soundStop();
14 void soundJoinGame();
15 void soundMyTurn();
16 void soundGameDone();
17 void soundTick();
18 void soundPlaceShip();
19 void soundAttack();
20 void soundInvalid();
21 void soundHit();
22 void soundSink();
23 void soundMiss();
24
25 void pause(uint8_t frames);
26
27 #endif /* SOUND_H */
```

SRC/PLATFORM-SPECIFIC/UTIL.H 16 LINES

```
1 #ifndef UTIL_H
2 #define UTIL_H
3
4 /*
5   Platform specific utilities that don't fit in any category
6 */
7
8
9 void resetTimer();
10 uint16_t getTime();
11 void quit();
12 void housekeeping();
13 uint8_t getJiffiesPerSecond();
14 uint8_t getRandomNumber(uint8_t maxExclusive);
15
16 #endif /* UTIL_H */
```

SRC/PLATFORM-SPECIFIC/VARS.H 33 LINES

```
1 /*
2   Platform specific vars.
3 */
4 #ifndef VARS_H
5 #define VARS_H
6
7 // Include platform specific vars - this is defined in the Makefile as "../$(PLATFORM)/vars.h"
8 // Watcom / wine has issues with \" in the define, so hacking this for now
9 // zcc (Adam/ColecoVision/Master System/H89) gets the same treatment to avoid the \" quoting
 +     through defoogi.
10 // Note z88dk defines __COLECO__ for the Adam subtype too, so the ColecoVision
11 // build is keyed on BUILD_COLECO instead.
12 #if __MSDOS__
13 #include "../msdos/vars.h"
14 #elif defined(__ADAM__)
15 #include "../adam/vars.h"
16 #elif defined(BUILD_COLECO)
17 #include "../coleco/vars.h"
18 #elif defined(BUILD_SMS)
19 #include "../sms/vars.h"
20 #elif defined(__H89__)
21 #include "../h89/vars.h"
22 #else
23 #include PLATFORM_VARS
24 #endif
25
26 // Default vars (covers most platforms) - after the platform include so a
27 // platform's vars.h can override them
28 #ifndef ESCAPE
29 #define ESCAPE "ESCAPE"
30 #define ESC "ESC"
31 #endif
32
33 #endif /* VARS_H */
```

SRC/H89/CONSOLE.C 331 LINES

```
1 #ifdef __H89__
2
3 /**
4  * @brief   H19 terminal screen - Heathkit H89
5  * @author  Thomas Cherryhomes
6  * @email   thom dot cherryhomes at gmail dot com
7  * @license gpl v. 3, see LICENSE for details
8  *
9  * The console is the H19 on the H89's 8250 at 9600 baud -- under a thousand
10  * characters a second -- so the screen is a shadow of what the terminal
11  * shows, and only cells that change are sent. Console bytes go through the
12  * OS raw: HDOS's .SCOUT/.SCIN with the console driver in raw character mode,
13  * or the CP/M BIOS's CONOUT/CONST/CONIN straight from the jump table (the
14  * BDOS console calls expand, filter and hold back characters).
15  */
16
17 #include <string.h>
18 #include "console.h"
19
20 #define ESC 0x1B
21 #define CUR_UNKNOWN 0xFF
22
23 static uint8_t scr[SCR_W * SCR_H];
24 static uint8_t scrBak[SCR_W * SCR_H];
25
26 // What the terminal is doing now: cursor, reverse video, graphics mode.
27 // CUR_UNKNOWN forces the next cell to set it.
28 static uint8_t curX, curY, curRev, curGfx;
29
30 /*
31  * Raw console I/O
32  */
33
34 #ifdef __HDOS__
35
36 // HDOS system calls: RST 38H, then the function byte. HDOS is 8080 code and
37 // leaves IX and IY alone.
38 void conOut(uint8_t c) __z88dk_fastcall __naked
39 {
40 #asm
41     ld      a,l
42     rst     38h
43     defb    2           ; .SCOUT
44     ret
45 #endasm
46 }
47
48 int conIn(void) __naked
49 {
50 #asm
51     rst     38h
52     defb    1           ; .SCIN - carry set when nothing is waiting
53     ld      hl,-1
54     ret     c
55     ld      l,a
56     ld      h,0
57     ret
```

```
#endasm
}

// .CONSL: A = index of the console byte, B = the bits to set, C = the mask of
// bits to change
uint8_t h89ConslIndex, h89ConslBits, h89ConslMask;

static void consl(void) __naked
{
#asm
    ld      a,(_h89ConslBits)
    ld      b,a
    ld      a,(_h89ConslMask)
    ld      c,a
    ld      a,(_h89ConslIndex)
    rst     38h
    defb    6           ; .CONSL
    ret
#endasm
}

static void conslSet(uint8_t index, uint8_t bits, uint8_t mask)
{
    h89ConslIndex = index;
    h89ConslBits = bits;
    h89ConslMask = mask;
    consl();
}

// The five console cells (S.CSLMD..S.CONFL) live at 040.326A in HDOS 2.0's
// system values (HOSDEF: S.VAL at 040.277A)
#define S_CSLMD ((volatile uint8_t *)0x20D6)
#define I_CSLMD 0
#define I_CONTY 1
#define I_CUSOR 2
#define I_CONWI 3
#define I_CONFL 4

#define CSL_ECH 0x80 // suppress echo
#define CSL_RAW 0x04 // raw mode I/O (no CTL-C/S/O/P processing)
#define CSL_WRP 0x02 // wrap lines at width
#define CSL_CHR 0x01 // character mode
#define CTP_MLI 0x20 // map lower case to upper on input
#define CTP_MLO 0x10 // map lower case to upper on output
#define CTP_BKM 0x02 // map backspace to rubout on input
#define CO_FLG 0x01  // CTL-O (discard output) flag
#define CS_FLG 0x80  // CTL-S (hold output) flag

static uint8_t savedConsole[5];

static void osInit(void)
{
    memcpy(savedConsole, (void *)S_CSLMD, sizeof(savedConsole));

    // Raw character mode with no echo, no wrap and no width (HDOS counts
    // escape sequences as printing columns), and no case mapping: the
    // graphics characters are sent as lower case letters.
    conslSet(I_CSLMD, CSL_ECH | CSL_RAW | CSL_CHR, CSL_ECH | CSL_RAW | CSL_WRP | CSL_CHR);
    conslSet(I_CONTY, 0, CTP_MLI | CTP_MLO | CTP_BKM);
    conslSet(I_CONWI, 255, 0xFF);
    conslSet(I_CONFL, 0, CO_FLG | CS_FLG);
}

static void osExit(void)
{
    conslSet(I_CSLMD, savedConsole[I_CSLMD], 0xFF);
    conslSet(I_CONTY, savedConsole[I_CONTY], 0xFF);
    conslSet(I_CONWI, savedConsole[I_CONWI], 0xFF);
    conslSet(I_CUSOR, 0, 0xFF); // ESC z homed the cursor
}

#else /* CP/M */

// The BIOS jump table: (0001) points at its WBOOT entry, base+3. CONST is at
// +3 from there, CONIN +6, CONOUT +9. The BIOS may use any register, so IX
// and IY are kept for the C side.
void conOut(uint8_t c) __z88dk_fastcall __naked
{
#asm
    EXTERN  l_dcal
    push    ix
    push    iy
    ld      c,1
    ld      hl,(1)
    ld      de,9
    add     hl,de
    call    l_dcal      ; CONOUT
    pop     iy
    pop     ix
    ret
#endasm
}

int conIn(void) __naked
{
#asm
    EXTERN  l_dcal
    push    ix
    push    iy
    ld      hl,(1)
    ld      de,3
    add     hl,de
    call    l_dcal      ; CONST: A = FF when a character is waiting
    or      a
    ld      hl,-1
    jr      z,conIn_done
    ld      hl,(1)
    ld      de,6
    add     hl,de
    call    l_dcal      ; CONIN
    ld      l,a
    ld      h,0
conIn_done:
    pop     iy
    pop     ix
    ret
#endasm
}

static void osInit(void)
{
}

static void osExit(void)
{
}

#endif /* __HDOS__ */

static void esc(uint8_t c)
{
    conOut(ESC);
    conOut(c);
}

static void forget(void)
{
    curX = curY = curRev = curGfx = CUR_UNKNOWN;
}

/*
 * Screen
 */

void termInit(void)
{
    osInit();

    esc('G');      // graphics mode off
    esc('q');      // reverse video off
    esc('x');      // cursor off
    conOut('5');
    esc('w');      // discard (no wrap) at the end of a line
    esc('t');      // keypad shifted: the unshifted keypad sends ESC A-D
    forget();
    termClear();
}

void termExit(void)
{
    static uint16_t t;

    // Back to the power-up configuration the DIP switches set: modes,
    // cursor, keypad, and a clear screen. The terminal needs a moment.
    esc('z');
    t = TICKS;
    while ((uint16_t)(TICKS - t) < 100)
        ;
    osExit();
}

void termClear(void)
{
    esc('E');
    memset(scr, ' ', sizeof(scr));
    curX = curY = 0;
}

void termBell(void)
{
    conOut(0x07);
}

/// @brief Send a cell to the terminal at x,y
static void emit(uint8_t x, uint8_t y, uint8_t code)
{
```

```
    static uint8_t g;

    if (x != curX || y != curY)
    {
        esc('Y');
        conOut(32 + y);
        conOut(32 + x);
    }

    if ((code & REV) != curRev)
    {
        curRev = code & REV;
        esc(curRev ? 'p' : 'q');
    }

    g = code & 0x7F;
    if (g < 0x20 || g == 0x7F)
    {
        // Graphics characters: ROM 0x00-0x1E are ` through *, 0x1F is _,
        // the bullet is ^
        if (curGfx != 1)
        {
            curGfx = 1;
            esc('F');
        }
        if (g == 0x7F)
            g = '^';
        else if (g == 0x1F)
            g = '_';
        else
            g += 0x60;
    }
    else if (g >= 0x5E)
    {
        // Graphics mode only remaps 0x5E-0x7E: everything below prints the
        // same either way, so the mode is left alone for it
        if (curGfx != 0)
        {
            curGfx = 0;
            esc('G');
        }
    }

    conOut(g);

    // In discard-at-end-of-line mode the cursor stays on the last column
    curY = y;
    curX = (x == SCR_W - 1) ? CUR_UNKNOWN : x + 1;
}

void put(uint8_t x, uint8_t y, uint8_t code)
{
    static uint16_t off;

    if (x >= SCR_W || y >= SCR_H)
        return;

    off = (uint16_t)y * SCR_W + x;
    if (scr[off] == code)
        return;

    scr[off] = code;
    emit(x, y, code);
}

void fill(uint8_t x, uint8_t y, uint8_t w, uint8_t code)
{
    while (w--)
        put(x++, y, code);
}

void termSave(void)
{
    memcpy(scrBak, scr, sizeof(scr));
}

void termRestore(void)
{
    static uint8_t x, y;
    static uint8_t *p;

    p = scrBak;
    for (y = 0; y < SCR_H; y++)
        for (x = 0; x < SCR_W; x++)
            put(x, y, *p++);
}

#endif /* __H89__ */
```

SRC/H89/CONSOLE.H — 92 LINES

```
#ifndef H89_CONSOLE_H

#define H89_CONSOLE_H

/*
  H19 terminal screen for the H89 port.

  The screen is drawn by cell codes, one byte each: bit 7 is reverse video and
  the low seven bits are the H19's own character ROM index (444-29) --
  0x00-0x1F the graphics characters, 0x7F the graphics bullet, 0x20-0x7E
  ASCII -- the same form the terminal keeps its screen in. console.c keeps a
  shadow of what the terminal shows and sends only the cells that change, with
  as few cursor-address (ESC Y), reverse-video (ESC p/q) and graphics-mode
  (ESC F/G) changes as it can: the console runs at 9600 baud.
*/

#include <stdint.h>
#include <stdbool.h>

#define SCR_W 80
#define SCR_H 24

#define REV 0x80

// H19 graphics characters: ROM index, shown in graphics mode (ESC F) for the
// character in the comment
#define G_VLINE      0x00 // `  vertical line
#define G_HLINE      0x01 // a  horizontal line
#define G_CROSS      0x02 // b  cross
#define G_CORNER_UR  0x03 // c  upper right corner
#define G_CORNER_LR  0x04 // d  lower right corner
#define G_CORNER_LL  0x05 // e  lower left corner
#define G_CORNER_UL  0x06 // f  upper left corner
#define G_PLUSMINUS  0x07 // g
#define G_ARROW_R    0x08 // h  right arrow
#define G_CHECKER    0x09 // i  checkerboard
#define G_DIVIDE     0x0A // j
#define G_ARROW_D    0x0B // k  down arrow
#define G_QUAD_LR    0x0C // l  lower right quadrant
#define G_QUAD_LL    0x0D // m  lower left quadrant
#define G_QUAD_UL    0x0E // n  upper left quadrant
#define G_QUAD_UR    0x0F // o  upper right quadrant
#define G_HALF_TOP   0x10 // p  upper half
#define G_HALF_RIGHT 0x11 // q  right half
#define G_TRI_UL     0x12 // r  upper left triangle
#define G_TEE_DOWN   0x13 // s  T, stem down
#define G_TEE_LEFT   0x14 // t  T, stem left
#define G_TEE_UP     0x15 // u  T, stem up
#define G_TEE_RIGHT  0x16 // v  T, stem right
#define G_X          0x17 // w  diagonal cross
#define G_SLASH      0x18 // x  /
#define G_BACKSLASH  0x19 // y  backslash
#define G_BAR_TOP    0x1A // z  top line
#define G_BAR_BOTTOM 0x1B // {  bottom line
#define G_BAR_LEFT   0x1C // |  left bar
#define G_BAR_RIGHT  0x1D // }  right bar
#define G_PILCROW    0x1E // ~
#define G_TRI_UR     0x1F // _  upper right triangle
#define G_BULLET     0x7F // ^  bullet

/// @brief Set up the terminal (and HDOS's console driver) for the game
void termInit(void);

/// @brief Put the terminal (and HDOS's console driver) back as they were
void termExit(void);

/// @brief Clear the screen
void termClear(void);

/// @brief Set one cell, sending it only if it changed
void put(uint8_t x, uint8_t y, uint8_t code);

/// @brief Set a run of w cells to one code
void fill(uint8_t x, uint8_t y, uint8_t w, uint8_t code);

/// @brief Copy the screen aside / bring it back
void termSave(void);
void termRestore(void);

/// @brief Ring the terminal's bell
void termBell(void);

/// @brief Raw console byte, or -1 when none is waiting
int conIn(void);

/// @brief The H89's 2 ms clock tick counter (HDOS .TICCNT / the CP/M BIOS's)
#ifdef __HDOS__
#define TICKS (*(volatile uint16_t *)0x201B)
#else
#define TICKS (*(volatile uint16_t *)0x000B)
#endif

#endif /* H89_CONSOLE_H */
```

SRC/H89/FUJINET.C 51 LINES

```
#ifdef __H89__

/**
 * @brief   FujiNet calls - Heathkit H89 (CUSTOM_FUJINET_CALLS)
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * Everything that talks to the H89 FujiNet card through fujinet-lib is in
 * this one file, because it has to see the LIBRARY's (fujinet-fuji.h): the
 * h89 build of the lib implements the appkey calls as macros over its bus
 * call, so the prototypes in src/fujinet-fuji.h (which src/misc.h pulls in)
 * link against nothing. This file therefore includes none of the game's
 * headers, and the shared code reaches the card through the
 * CUSTOM_FUJINET_CALLS hooks instead. (src/sms/fujinet.c, less the lobby.)
 */

#include <stdbool.h>
#include <stdint.h>
#include <fujinet-fuji.h>
#include <fujinet-network.h>

int16_t custom_network_call(char *url, uint8_t *buffer, uint16_t max_len)
{
    int16_t read;

    if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE) != FN_ERR_OK)
        return -1;

    read = network_read(url, buffer, max_len);
    network_close(url);
    return read;
}

uint16_t custom_read_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, char
 +    *destination)
{
    uint16_t read = 0;

    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    if (!fuji_read_appkey(key_id, &read, (uint8_t *)destination))
        read = 0;
    return read;
}

void custom_write_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, uint16_t count,
 +    char *data)
{
    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    fuji_write_appkey(key_id, count, (uint8_t *)data);
}

#endif /* __H89__ */
```

SRC/H89/GRAPHICS.C 505 LINES

```
#ifdef __H89__

/**
 * @brief   Graphics - Heathkit H89 (H19 terminal, 80x24)
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * Everything is drawn with the H19's own characters: its graphics set (line
 * drawing, quadrant blocks, the diagonal cross, the checkerboard) and reverse
 * video. The layout is the Adam's 32x24 one (src/adam/graphics.c) with every
 * board cell two characters wide -- the H19's dots are about twice as tall as
 * they are wide, so two characters make a cell close to square:
 *
 *   row 0      name badges of the top boards
 *   rows 1-10  top fields
 *   row 11     divider shared by the top and bottom boards
 *   rows 12-21 bottom fields
 *   row 22     name badges of the bottom boards
 *   row 23     status: connection, timer, clock
 *
 * Fields are 20 columns wide, at column 18 (left boards) and 42 (right
 * boards), or 30 when there are only one or two players. The fleet drawer
 * (the ships-left legend) sits outside the field, three hull columns wide.
 *
 * The screen cell codes and the shadow that keeps the 9600 baud console down
 * to the cells that change are in console.c.
 */

#include "../misc.h"
#include "console.h"

#define FIELD_W 20    // columns in a field (10 cells, two characters each)
#define SHIFT_1V1 12  // field shift right when the boards stack in one column
#define DRAWER_LEFT 7  // left drawer interior starts this far left of the field
#define DRAWER_RIGHT 21 // right drawer interior starts this far right of it

/**
 * @brief Top left of each player's field. Players run clockwise from the
 * bottom left.
 */
static const uint8_t quadrantX[4] = {18, 18, 42, 42};
static const uint8_t quadrantY[4] = {12, 1, 1, 12};

/**
 * @brief Ship positions in the fleet drawer (hull column, row from the field top)
 */
static const uint8_t legendShipOffset[10] = {
    2, 1,
    1, 1,
    0, 1,
    0, 6,
    1, 7,
};

static uint8_t fieldShift = 0, playerCount = 0;

/*
 * A field cell is a pair of screen cells: two cell codes, left and right.
 * (Flat arrays: sccz80 balks at nested initializers.)
 */
static const uint8_t WATER[2] = (' ', ' ');
static const uint8_t MISS[2] = ('(', ')'); // a centred circle
static const uint8_t HIT[2] = (REV | G_X, REV | G_X);
static const uint8_t HIT2[2] = (G_CHECKER, G_CHECKER);
static const uint8_t SUNK[2] = (G_CHECKER, G_CHECKER);

// Hulls are half a cell thick both ways: vertical ones take the middle half
// of the two columns, horizontal ones the half of the row toward the middle
// of the screen -- the lower half on the top boards (a reversed upper half is
// the lower half), the upper half on the bottom ones -- so they never touch a
// name badge, and keep clear of the divider.
static const uint8_t HULL_H_LOW[6] = {
    G_QUAD_LR, REV | G_HALF_TOP,        // bow
    REV | G_HALF_TOP, REV | G_HALF_TOP, // middle
    REV | G_HALF_TOP, G_QUAD_LL,        // stern
};
static const uint8_t HULL_H_HIGH[6] = {
    G_QUAD_UR, G_HALF_TOP,
    G_HALF_TOP, G_HALF_TOP,
    G_HALF_TOP, G_QUAD_UL,
};
static const uint8_t HULL_V_BOW[2] = {G_QUAD_LR, G_QUAD_LL};
static const uint8_t HULL_V_MID[2] = {G_HALF_RIGHT, REV | G_HALF_RIGHT};
static const uint8_t HULL_V_STERN[2] = {G_QUAD_UR, G_QUAD_UL};

// The attack, frames 10-15: closing in, the strike, the flash
static const uint8_t ATTACK[12] = {
    '>', '<',
    G_BACKSLASH, G_SLASH,
    G_X, G_X,
    REV | G_X, REV | G_X,
    G_CHECKER, G_CHECKER,
    REV | ' ', REV | ' ',
};

static void putPair(uint8_t x, uint8_t y, const uint8_t *p)
{
    put(x, y, p[0]);
    put(x + 1, y, p[1]);
}

static uint8_t fieldLeft(uint8_t q)
{
    return quadrantX[q] + fieldShift;
}

/// @brief Does this player's drawer sit right of the field?
static bool drawerRight(uint8_t q)
{
    return q > 1 || (playerCount < 3 && q > 0);
}

/// @brief Is there a board above or below this one, sharing its divider?
static bool hasPartner(uint8_t q)
{
    static const uint8_t partner[4] = {1, 0, 3, 2};

    return partner[q] < playerCount;
}

uint8_t cycleNextColor()
{
    return 0;
}

void initGraphics()
```

```
{
    termInit();
}

void resetGraphics()
{
    termExit();
}

void resetScreen()
{
    termClear();
}

bool saveScreenBuffer()
{
    termSave();
    return true;
}

void restoreScreenBuffer()
{
    termRestore();
}

/// @brief Text, as is; with alt, the capitals in reverse video
static void text(uint8_t x, uint8_t y, const char *s, bool alt)
{
    static uint8_t c;

    if (y >= HEIGHT)
        y = HEIGHT - 1;
    if (x >= WIDTH) // centered text wider than the screen
        x = 0;

    while ((c = *s++) && x < WIDTH)
    {
        if (c < 0x20 || c > 0x7E)
            c = '?';
        else if (alt && c >= 'A' && c <= 'Z')
            c |= REV;
        put(x++, y, c);
    }
}

void drawText(uint8_t x, uint8_t y, const char *s)
{
    text(x, y, s, false);
}

void drawTextAlt(uint8_t x, uint8_t y, const char *s)
{
    text(x, y, s, true);
}

void drawIcon(uint8_t x, uint8_t y, uint8_t icon)
{
    put(x, y, icon);
}

void drawBlank(uint8_t x, uint8_t y)
{
    put(x, y, ' ');
}

void drawSpace(uint8_t x, uint8_t y, uint8_t w)
{
    fill(x, y, w, ' ');
}

void drawLine(uint8_t x, uint8_t y, uint8_t w)
{
    fill(x, y, w, G_HLINE);
}

void drawBox(uint8_t x, uint8_t y, uint8_t w, uint8_t h)
{
    static uint8_t i;

    put(x, y, G_CORNER_UL);
    fill(x + 1, y, w, G_HLINE);
    put(x + w + 1, y, G_CORNER_UR);

    for (i = 1; i <= h; i++)
    {
        put(x, y + i, G_VLINE);
        put(x + w + 1, y + i, G_VLINE);
    }

    put(x, y + h + 1, G_CORNER_LL);
    fill(x + 1, y + h + 1, w, G_HLINE);
    put(x + w + 1, y + h + 1, G_CORNER_LR);
}

void drawClock()
{
    put(WIDTH - 1, HEIGHT - 1, G_BULLET);
}

void drawConnectionIcon(bool show)
{
    put(0, HEIGHT - 1, show ? REV | '<' : ' ');
    put(1, HEIGHT - 1, show ? REV | '>' : ' ');
}

void drawPlayerName(uint8_t i, const char *name, bool active)
{
    static uint8_t x, y, by, fy, ox, ix, drawX, drawEdge, r, n, c, fillCode, right;

    x = fieldLeft(i);
    y = quadrantY[i];
    right = drawerRight(i);

    if (right)
    {
        ox = x - 1;
        ix = x + FIELD_W;
        drawX = x + DRAWER_RIGHT;
        drawEdge = drawX + 6;
    }
    else
    {
        ix = x - 1;
        ox = x + FIELD_W;
        drawX = x - DRAWER_LEFT;
        drawEdge = drawX - 1;
    }

    if (i == 1 || i == 2)
    {
        // Top board: badge above the field, divider below it
        by = y - 1;
        fy = y + 10;
        put(x - 1, by, G_CORNER_UL);
        put(x + FIELD_W, by, G_CORNER_UR);
    }
    else
    {
        // Bottom board: divider above the field, badge below it
        by = y + 10;
        fy = y - 1;
        put(x - 1, by, G_CORNER_LL);
        put(x + FIELD_W, by, G_CORNER_LR);
    }

    // Field edges. The drawer's top and bottom edges meet the inner one.
    for (r = 0; r < 10; r++)
    {
        put(ox, y + r, G_VLINE);
        put(ix, y + r, (r == 0 || r == 9) ? (right ? G_TEE_RIGHT : G_TEE_LEFT) : G_VLINE);
    }

    // Divider: a T where a board sits on its other side too
    fill(x, fy, FIELD_W, G_HLINE);
    if (hasPartner(i))
    {
        put(x - 1, fy, G_TEE_RIGHT);
        put(x + FIELD_W, fy, G_TEE_LEFT);
    }
    else if (i == 1 || i == 2)
    {
        put(x - 1, fy, G_CORNER_LL);
        put(x + FIELD_W, fy, G_CORNER_LR);
    }
    else
    {
        put(x - 1, fy, G_CORNER_UL);
        put(x + FIELD_W, fy, G_CORNER_UR);
    }

    // Fleet drawer
    fill(drawX, y, 6, G_HLINE);
    fill(drawX, y + 9, 6, G_HLINE);
    put(drawEdge, y, right ? G_CORNER_UR : G_CORNER_UL);
    put(drawEdge, y + 9, right ? G_CORNER_LR : G_CORNER_LL);
    for (r = 1; r < 9; r++)
        put(drawEdge, y + r, G_VLINE);

    // Badge: "-- name ------" or, for the active player, the whole bar in
    // reverse video with the bullet
    fillCode = active ? REV | ' ' : G_HLINE;
    n = 0;
    put(x + n++, by, fillCode);
    if (*name)
    {
        put(x + n++, by, active ? REV | G_BULLET : G_HLINE);
```

```
        put(x + n++, by, active ? REV | ' ' : ' ');
        while ((c = *name++) && n < FIELD_W - 2)
        {
            if (c < 0x20 || c > 0x7E)
                c = '?';
            put(x + n++, by, active ? REV | c : c);
        }
        put(x + n++, by, active ? REV | ' ' : ' ');
    }
    while (n < FIELD_W)
        put(x + n++, by, fillCode);
}

void drawBoard(uint8_t currentPlayerCount)
{
    static uint8_t i, r, x, y, drawX;

    playerCount = currentPlayerCount;
    fieldShift = playerCount > 2 ? 0 : SHIFT_1V1;

    for (i = 0; i < playerCount; i++)
    {
        // Frame, drawer and an empty badge
        drawPlayerName(i, "", false);

        // Water
        x = fieldLeft(i);
        y = quadrantY[i];
        drawX = drawerRight(i) ? x + DRAWER_RIGHT : x - DRAWER_LEFT;
        for (r = 0; r < 10; r++)
            fill(x, y + r, FIELD_W, ' ');
        for (r = 1; r < 9; r++)
            fill(drawX, y + r, 6, ' ');
    }
}

/// @brief A ship's hull from its bow at x,y on quadrant's board
static void drawShipInternal(uint8_t quadrant, uint8_t x, uint8_t y, uint8_t size, uint8_t
    delta)
{
    static uint8_t i;
    static const uint8_t *p;

    for (i = 0; i < size; i++)
    {
        if (delta)
        {
            p = i == 0 ? HULL_V_BOW : i == size - 1 ? HULL_V_STERN : HULL_V_MID;
            putPair(x, y++, p);
        }
        else
        {
            p = (quadrant == 1 || quadrant == 2) ? HULL_H_LOW : HULL_H_HIGH;
            if (i == size - 1)
                p += 4;
            else if (i)
                p += 2;
            putPair(x, y, p);
            x += 2;
        }
    }
}

void drawShip(uint8_t quadrant, uint8_t size, uint8_t pos, bool hide)
{
    static uint8_t i, delta, x, y;

    delta = 0;
    if (pos > 99)
    {
        delta = 1; // vertical
        pos -= 100;
    }

    x = fieldLeft(quadrant) + (pos % 10) * 2;
    y = quadrantY[quadrant] + pos / 10;

    if (hide)
    {
        for (i = 0; i < size; i++)
        {
            putPair(x, y, WATER);
            if (delta)
                y++;
            else
                x += 2;
        }
        return;
    }

    drawShipInternal(quadrant, x, y, size, delta);
}

void drawLegendShip(uint8_t player, uint8_t index, uint8_t size, uint8_t status)
{
    static uint8_t i, x, y;

    x = fieldLeft(player) + legendShipOffset[index * 2] * 2;
    y = quadrantY[player] + legendShipOffset[index * 2 + 1];

    if (drawerRight(player))
        x += DRAWER_RIGHT;
    else
        x -= DRAWER_LEFT;

    if (status)
    {
        drawShipInternal(player, x, y, size, 1);
    }
    else
    {
        for (i = 0; i < size; i++)
            putPair(x, y + i, SUNK);
    }
}

void drawGamefield(uint8_t quadrant, uint8_t *field)
{
    static uint8_t x, y, cx;

    y = quadrantY[quadrant];
    for (cx = 0; cx < 100; cx++)
    {
        if (*field)
        {
            x = fieldLeft(quadrant) + (cx % 10) * 2;
            putPair(x, y + cx / 10, *field == FIELD_ATTACK ? HIT : MISS);
        }
        field++;
    }
}

void drawGamefieldUpdate(uint8_t quadrant, uint8_t *gamefield, uint8_t attackPos, uint8_t anim)
{
    static uint8_t c, x, y;

    x = fieldLeft(quadrant) + (attackPos % 10) * 2;
    y = quadrantY[quadrant] + attackPos / 10;

    if (anim > 9)
    {
        putPair(x, y, ATTACK + (anim - 10) * 2);
        return;
    }

    c = gamefield[attackPos];
    if (c == FIELD_ATTACK)
        putPair(x, y, anim ? HIT2 : HIT);
    else if (c == FIELD_MISS)
        putPair(x, y, MISS);
}

void drawGamefieldCursor(uint8_t quadrant, uint8_t x, uint8_t y, uint8_t *gamefield, uint8_t
    blink)
{
    static uint8_t c, sx, sy;
    static const uint8_t *p;

    sx = fieldLeft(quadrant) + x * 2;
    sy = quadrantY[quadrant] + y;

    c = gamefield[y * 10 + x];
    p = c == FIELD_ATTACK ? HIT : c == FIELD_MISS ? MISS : WATER;

    if (!blink)
    {
        putPair(sx, sy, p);
    }
    else if (blink == 2 && !c)
    {
        // Open water pulses: block, box, nothing
        put(sx, sy, '[');
        put(sx + 1, sy, ']');
    }
    else
    {
        // The cell turned over
        put(sx, sy, p[0] ^ REV);
        put(sx + 1, sy, p[1] ^ REV);
    }
}

void drawEndgameMessage(const char *message)
{
    static uint8_t x;
```

```
    x = (WIDTH - (uint8_t)strlen(message)) / 2;

    fill(0, HEIGHT - 2, WIDTH, G_HLINE);
    fill(0, HEIGHT - 1, WIDTH, ' ');
    drawText(x, HEIGHT - 1, message);
}

#endif /* __H89__ */
```

SRC/H89/INPUT.C 149 LINES

```
#ifdef __H89__

/**
 * @brief   Input - Heathkit H89 (H19 keyboard)
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * The H19 sends its keypad and function keys as escape sequences: with the
 * keypad shifted (termInit sends ESC t) the arrows are ESC A-D, f1-f5 are
 * ESC S-W, and BLUE/RED/GRAY are ESC P-R. Those arrive back to back at 9600
 * baud, so an ESC with nothing behind it within a few ticks is the ESC key
 * itself. Both OSes buffer the keyboard on its interrupt (HDOS's console
 * driver; the CP/M BIOS's 40-byte RST 3 buffer), so nothing is lost between
 * polls.
 */

#include "../misc.h"
#include "console.h"

#define ESC_WAIT_TICKS 10 // 20 ms

// Decoded keys not yet returned
static uint8_t pending[4];
static uint8_t pendingCount;

static void push(uint8_t k)
{
    if (pendingCount < sizeof(pending))
        pending[pendingCount++] = k;
}

/// @brief Plain keys: DELETE erases like BACKSPACE, LINE FEED enters
static uint8_t plain(uint8_t c)
{
    if (c == 0x7F)
        return KEY_BACKSPACE;
    if (c == 0x0A)
        return KEY_RETURN;
    return c;
}

/// @brief Wait briefly for the byte after an ESC; -1 if none comes
static int escFollower(void)
{
    static uint16_t t;
    static int c;

    t = TICKS;
    do
    {
        c = conIn();
        if (c >= 0)
            return c;
    } while ((uint16_t)(TICKS - t) < ESC_WAIT_TICKS);

    return -1;
}

static void decode(uint8_t c)
{
    static int n;

    if (c != KEY_ESCAPE)
    {
        push(plain(c));
        return;
    }

    n = escFollower();
    switch (n)
    {
    case 'A':
        push(KEY_UP_ARROW);
        break;
    case 'B':
        push(KEY_DOWN_ARROW);
        break;
    case 'C':
        push(KEY_RIGHT_ARROW);
        break;
    case 'D':
        push(KEY_LEFT_ARROW);
        break;
    case 'P': // BLUE
    case 'Q': // RED
    case 'R': // GRAY
    case 'S': // f1
    case 'T': // f2
    case 'U': // f3
    case 'V': // f4
    case 'W': // f5
        // The function keys open the menu too
        push(KEY_ESCAPE);
        break;
    case 'H': // HOME
    case 'J': // ERASE
    case 'E': // SHIFT ERASE
    case 'L': // IL
    case 'M': // DL
    case 'N': // DC
    case '@': // IC
        // The keypad's other keys do nothing here
        break;
    default:
        // A lone ESC, or ESC then an ordinary key
        push(KEY_ESCAPE);
        if (n >= 0 && n != KEY_ESCAPE)
            push(plain((uint8_t)n));
        else if (n == KEY_ESCAPE)
            push(KEY_ESCAPE);
        break;
    }
}

unsigned char kbhit(void)
{
    static int c;

    while (!pendingCount)
    {
        c = conIn();
        if (c < 0)
            return 0;
        decode((uint8_t)c);
    }
    return 1;
}

unsigned char cgetc(void)
{
    static uint8_t k;

    while (!kbhit())
        ;

    k = pending[0];
    pendingCount--;
    memmove(pending, pending + 1, pendingCount);
    return k;
}

uint8_t readJoystick(void)
{
    // Keyboard only: the arrows and RETURN/SPACE come through cgetc()
    return 0;
}

#endif /* __H89__ */
```

SRC/H89/SOUND.C 60 LINES

```
#ifdef __H89__

/**
 * @brief   Sound - Heathkit H89
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * The only sound an H89 makes is the H19's bell, so it marks the moments
 * that need the player's attention and the rest stay quiet.
 */

#include "../misc.h"
#include "console.h"

static void bell(void)
{
    if (!prefs.disableSound)
        termBell();
}

```

```
22 void initSound() {}
23
24 void soundJoinGame()
25 {
26     bell();
27 }
28
29 void soundMyTurn()
30 {
31     bell();
32 }
33
34 void soundGameDone()
35 {
36     bell();
37 }
38
39 void soundInvalid()
40 {
41     bell();
42 }
43
44 void soundSink()
45 {
46     bell();
47 }
48
49 void soundCursor() {}
50 void soundPlaceShip() {}
51 void soundTick() {}
52 void soundSelect() {}
53 void soundMiss() {}
54 void soundAttack() {}
55 void soundHit() {}
56 void soundStop() {}
57 void disableKeySounds() {}
58 void enableKeySounds() {}
59
60 #endif /* __H89__ */
```

SRC/H89/UTIL.C 76 LINES

```
 1 #ifdef __H89__
 2
 3 /**
 4  * @brief   Utility Functions - Heathkit H89
 5  * @author  Thomas Cherryhomes
 6  * @email   thom dot cherryhomes at gmail dot com
 7  * @license gpl v. 3, see LICENSE for details
 8  *
 9  * The H89's clock interrupts every 2 ms, and both HDOS and the CP/M BIOS
10  * count the ticks in a word (console.h's TICKS). A frame here is 10 ticks:
11  * 50 a second.
12  */
13
14 #include <stdbool.h>
15 #include <stdint.h>
16 #include <stdlib.h>
17 #include "vars.h"
18 #include "console.h"
19 #include "../platform-specific/graphics.h"
20 #include "../platform-specific/util.h"
21
22 #define TICKS_PER_FRAME 10
23
24 static uint16_t lastFrame, timerBase;
25 static bool seeded;
26
27 void waitvsync(void)
28 {
29     // At least a frame since the last one returned. Drawing over the 9600
30     // baud console can take longer than that; the game just runs at the
31     // pace of the terminal then.
32     while ((uint16_t)(TICKS - lastFrame) < TICKS_PER_FRAME)
33         ;
34     lastFrame = TICKS;
35 }
36
37 void resetTimer(void)
38 {
39     timerBase = TICKS;
40 }
41
42 uint16_t getTime(void)
43 {
44     return (uint16_t)(TICKS - timerBase) / TICKS_PER_FRAME;
45 }
46
47 uint8_t getJiffiesPerSecond(void)
48 {
49     return 500 / TICKS_PER_FRAME;
50 }
51
52 uint8_t getRandomNumber(uint8_t maxExclusive)
53 {
54     if (!seeded)
55     {
56         // Seeded at the first request, which comes only after some
57         // human-paced input
58         seeded = true;
59         srand(TICKS);
60     }
61     return (uint8_t)(rand() % maxExclusive);
62 }
63
64 void housekeeping(void)
65 {
66 }
67
68 void quit(void)
69 {
70     // There is no H89 lobby to load: back to HDOS or CP/M, with the
71     // terminal as it was
72     resetGraphics();
73     exit(0);
74 }
75
76 #endif /* __H89__ */
```

SRC/H89/VARS.H 74 LINES

```
 1 #ifdef __H89__
 2
 3 #ifndef KEYMAP_H
 4 #define KEYMAP_H
 5
 6 // Screen dimensions for platform: the H19 terminal's 80x24 (its 25th line is
 7 // left alone). Every board cell is two characters wide (src/h89/graphics.c).
 8 #define WIDTH 80
 9 #define HEIGHT 24
10
11 #define GAMEOVER_PROMPT_Y HEIGHT - 2
12
13 /*
14   Icons are screen cell codes (src/h89/console.h): bit 7 is reverse video and
15   the low seven bits are the H19's own character ROM index -- 0x00-0x1F the
16   graphics characters, 0x7F the graphics bullet, 0x20-0x7E ASCII.
17 */
18 #define ICON_TEXT_CURSOR 0xA0 // reverse space: a block cursor
19 #define ICON_PLAYER 0x7F      // bullet
20 #define ICON_MARK 0x08        // right arrow
21 #define ICON_MARK_ALT 0x20
22 #define ICON_ACTIVE_PLAYER 0x7F
23
24 /**
25  * Platform specific key map for common input (H19 keyboard)
26  */
27
28 // src/h89/input.c turns the keypad's ESC A-D (keypad shifted mode, set by
29 // termInit) into 0xA0-0xA3. The _2 codes are unreachable placeholders --
30 // readCommonInput() switches on all twelve -- and the _3 keys are the Adam's
31 // spares for a keyboard without the keypad.
32 #define KEY_UP_ARROW       0xA0
33 #define KEY_UP_ARROW_2     0xF1
34 #define KEY_UP_ARROW_3     0x2D // -
35
36 #define KEY_DOWN_ARROW     0xA1
37 #define KEY_DOWN_ARROW_2   0xF2
38 #define KEY_DOWN_ARROW_3   0x3D // =
39
40 #define KEY_RIGHT_ARROW    0xA2
41 #define KEY_RIGHT_ARROW_2  0xF3
42 #define KEY_RIGHT_ARROW_3  0x2E // .
43
44 #define KEY_LEFT_ARROW     0xA3
45 #define KEY_LEFT_ARROW_2   0xF4
46 #define KEY_LEFT_ARROW_3   0x2C // ,
47
48 #define KEY_RETURN 0x0D
49 #define KEY_ESCAPE 0x1B
50 #define KEY_ESCAPE_ALT 0x03 // CTRL-C
51 #define KEY_SPACEBAR 0x20
52 #define KEY_BACKSPACE 0x08
53
54 /* Keyboard only: readJoystick() always returns 0 */
55 #define JOY_UP(v) ((v) & 1)
56 #define JOY_DOWN(v) ((v) & 2)
57 #define JOY_LEFT(v) ((v) & 4)
58 #define JOY_RIGHT(v) ((v) & 8)
59 #define JOY_BTN_1(v) ((v) & 0)
60 #define JOY_BTN_2(v) ((v) & 0)
61
62 // Countdown can reach two digits
63 #define TIMER_WIDTH 2
```

```
64
65 // What the shared screens call the keys: the H19's is labelled ESC, and its
66 // f1-f5 and colour keys open the menu too (src/h89/input.c)
67 #define ESCAPE "ESC"
68 #define ESC "ESC"
69 #define READY_PROMPT_TEXT "press SPACE when ready"
70 #define MENU_CLOSE_TEXT "press SPACE to close"
71
72 #endif /* KEYMAP_H */
73
74 #endif /* __H89__ */
```

FUJITZEE

GENERAL

Fujitzee is a dice game for up to six, in thirteen rounds. In each turn you roll five dice, may roll any of them again twice more, and then must score the result in one of thirteen boxes: the ones to the sixes, a set of three or four, a full house, short and long runs, a "count" of everything, and a Fujitzee — five of a kind, 50 points. Each box takes one score in a game; the highest total wins. The program is FUJITZEE.ABS for HDOS and FUJITZEE.COM for CP/M.

PLAYING

( ) Run FUJITZEE, give your name if it asks, and choose a table. P on the table list adds up to four players who share the H89, taking turns at the keyboard.

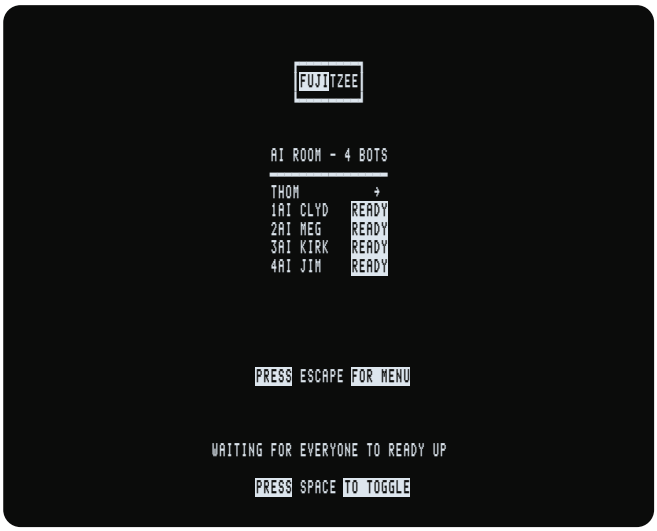
( ) Press SPACE when you are ready.

( ) On your turn the server rolls for you. Left and right move along the dice; RETURN keeps a die (or lets it go); RETURN on ROLL! rolls the rest. You have two more rolls.

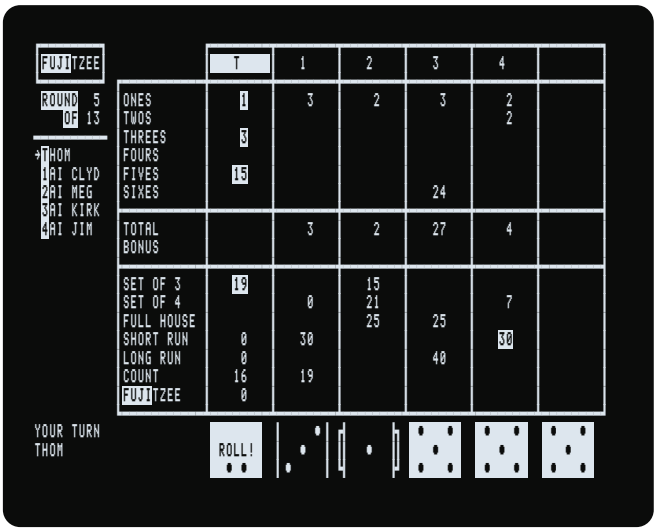
( ) Then move up into the score column, choose a box with up and down (only those you may use are offered), and press RETURN.



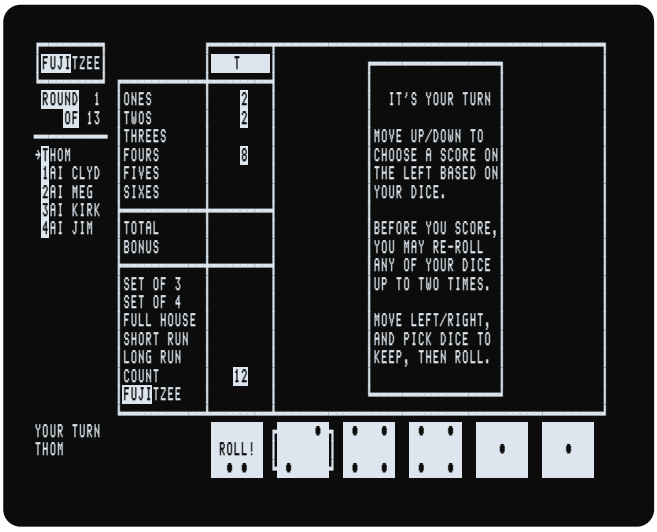
PICTORIAL 19-1
Choosing a game to join: a die in each corner, and the frame joins itself to the title's box.



PICTORIAL 19-2
The lobby: the robots are ready (in reverse video); you are not, yet.



PICTORIAL 19-4
Round 5, your turn, two dice kept (the hollow ones); the scores you could take are in reverse video.



PICTORIAL 19-3
Your first turn, with the help box over the robots' columns.

HOW IT WORKS

THE STATE

Fujitzee polls about once a second. Its `Game` structure is 599 bytes, which the program checks at compile time:

OFFSET	BYTES	HOLDS
0	1	How many players (players first, then those watching).
1	21	The table's name.
22	41	The prompt to show.
63	1	The round, 1 to 13; 0 in the lobby, 99 when it is over.
64	1	Rolls left.
65	1	Whose turn.
66	1	Seconds left to move.
67	1	1 if you are only watching.
68	6	The dice, as digits: `35126`.
74	6	Which were kept: `01101`.
80	15	For each box, the score it would give, or –1 if it may not be used.
95	42 each	Each player: name, a one-letter alias, and sixteen scores (-1 for empty).

A roll is `roll/01101` (1 to roll that die again), and a score `score/n`. With players sharing the H89, each has a request of its own.

DRAWING THE BOARD

The board is the 40-column one made 80 wide: the boxes' names in full, then a column eight wide for each player. The lines are drawn by direction, so where two meet the right corner, T or cross appears by itself. The dice are

seven characters by three, which looks square on the H19: a die still to roll is a solid block in reverse video with dark pips; a kept one is hollow, with bright pips between two bars.

Fujitzee keeps three pictures of the screen in memory — the one being drawn, the one on the terminal, and one saved under the menu and help — 6,000 bytes in all, which is why it needs a full-memory H89 under HDOS. It sends the screen before each FujiNet request, so that "refreshing" shows while the request is out.

THE FILES

FILE	HOLDS
`src/main.c`	The globals, the main loop: poll, animate, keys.
`src/stateclient.c`	Requests (one per player sharing the H89), moves, the state.
`src/screens.c`	Help, players, name, welcome, table list, the menu.
`src/gamelogic.c`	The board, the dice and their animation, turns, scoring.
`src/misc.c`, `misc.h`	Keys, preferences (app key E41CH/3/0); the structures.
`src/platform-specific/*.h`	What the H89's part must supply.
`src/h89/console.c`	The H19: three pictures of the screen; the OS console.
`src/h89/graphics.c`	The board, dice, cursor, tab and frames.
`src/h89/input.c`	The H19's keys; sends the screen first.
`src/h89/fujinet.c`	The fujinet-lib calls, sending the screen first.
`src/h89/sound.c`, `util.c`, `vars.h`	The bell; frames, the move clock, the size check; layout.

NOTE: Fujitzee's H89 port was not yet committed to its repository when this Manual was made. This listing is of its working tree, as it then stood (branch `add-h89`, on 3fe25a6, with 11 files changed).

LISTING

SRC/MAIN.C 118 LINES

```
/**
 * @brief   Fujitzee
 * @author  Eric Carr, Thomas Cherryhomes, (insert names here)
 * @license gpl v. 3
 * @verbose main
 */


#include <stdlib.h>
#include <stdint.h>
#include <stdbool.h>
#include <string.h>

#include "platform-specific/graphics.h"
#include "platform-specific/util.h"
#include "platform-specific/input.h"
#include "misc.h"
#include "platform-specific/sound.h"

#include "stateclient.h"
#include "gamelogic.h"
#include "screens.h"

#include "fujinet-fuji.h"

// Store default public server endpoint in case lobby did not set app key
char serverEndpoint[50] = "https://fujitzee.carr-designs.com/";

// For local dev testing, instead of changing the endpoint above,
// set 3rd byte in the e41c0300 appkey to 0xff, which will cause the below endpoing to be used
char localServer[] = "http://127.0.0.1:8080/";

ClientState clientState;
GameState state;
PrefsStruct prefs;

// State helper vars
uint16_t maxJifs;

// Common local scope temp variables
unsigned char h, i, j, k, x, y;
char tempBuffer[128];

extern void toneFinder();

void main(void)
{
  uint8_t failedApiCalls=0;

  // Testing
  //toneFinder();
  //printf("Press keys\n");while(1) {while (!kbhit());failedApiCalls = cgetc();printf("%d
      0x%x\n", failedApiCalls, failedApiCalls);} // Read Key

#ifdef COCO3
  loadPrefs(); // color mode must be known before initGraphics prompts
```

```
#endif

  initGraphics();
  initSound();

  showWelcomeScreen();
  showTableSelectionScreen();

  // Main event loop - process state from server and input from keyboard/joystick
  state.apiCallWait=0;

  while (true) {

    // Poll the server every so often.
    if (!state.apiCallWait--) {

      // Housekeeping - allows platform specific housekeeping, like stopping
          Attract/screensaver mode in Atari
      housekeeping();

      // Poll the server
      switch (getStateFromServer()) {
        case STATE_UPDATE_ERROR:
          // ERROR - Wait a bit to avoid hammering the server if getting bad responses
          // Wait max 4 seconds (since 4*60=240 fits in a single byte)
          if (failedApiCalls<4) {
            failedApiCalls++;
          }
          state.apiCallWait=60*failedApiCalls;

          // After consequitive failures, let the player know we are experiencing technical
              difficulties
          if (failedApiCalls>1) {
            drawConnectionIcon(0,HEIGHT-1);
            pause(30);
            drawText(0, HEIGHT-1, "  ");
            pause(30);
            drawConnectionIcon(0,HEIGHT-1);
          }
          break;

        case STATE_UPDATE_CHANGE:

          // Clear connection failure message
          if (failedApiCalls>1) {
            drawText(0, HEIGHT-1, "  ");
          }
          failedApiCalls=0;
          processStateChange();

          // Poll again in a bit
          state.apiCallWait = 59;
          break;
      }
    }

    // Animation and input
    if (failedApiCalls==0) {
      handleAnimation();
    }

    processInput();
  }

}
```

SRC/GAMELOGIC.C 996 LINES

```
#include <stdlib.h>
#include <string.h>
#include "platform-specific/graphics.h"
#include "platform-specific/sound.h"
#include "platform-specific/util.h"
#include "gamelogic.h"
#include "misc.h"
#include "stateclient.h"
#include "screens.h"
#if !defined(__WATCOMC__) && !defined(__ADAM__) && !defined(BUILD_SMS) && !defined(__H89__)
#include <peekpoke.h>
#endif

#define PLAYER_LIST_Y_OFFSET 5
#define BOTTOM_PANEL_Y HEIGHT-BOTTOM_HEIGHT
#ifndef HELP_X
#define HELP_X WIDTH-18
#endif

// Board layout. A platform whose board is not the 40 column one (the H89's
// 80 column H19) sets these in its vars.h. The defaults are the original
// expressions, unparenthesised, so the other platforms compile exactly as
// they did.
#ifndef SCORE_COL_X
#define SCORE_COL_X(i)   SCORES_X+6+i*4      // a player's 3 wide score field
#endif
#ifndef INITIAL_X
#define INITIAL_X(i)     SCORES_X+3+i*4      // player i's initial (i from 1)
#endif
#ifndef DIE_X
#define DIE_X(i)         WIDTH-20+4*i        // die i (from 0)
#endif
#ifndef KEPT_DIE_X
#define KEPT_DIE_X(p)    WIDTH-24+p*4        // die at cursor position p (from 1)
#endif
#ifndef DICE_CURSOR_X
#define DICE_CURSOR_X(p) 4*p-(p==0)+WIDTH-24 // cursor position p (0 is roll)
#endif
#ifndef HELP_CLEAR_X
#define HELP_CLEAR_X     SCORES_X+10         // what the in-game help covers
#define HELP_CLEAR_W     WIDTH-SCORES_X-10
#endif
#ifndef CENTER_SERVER_TEXT
#define CENTER_SERVER_TEXT centerTextAlt     // server names are mixed case
#endif

uint8_t scoreY[] =   {3,4,5,6,7,8,10,11,13,14,15,16,17,18,19,21};
char* scores[]={"one","two","three","four","five","six","total","bonus","set 3","set
    4","house","s run","1 run","count"};
uint8_t cursorPos, prevCursorPos, spectators, inputField_done, validX;
bool currentlyShowingHelp = 0;

void progressAnim(unsigned char y) {
  static uint8_t i;

  for(i=0;i<3;++i) {
    pause(10);
    drawIcon(WIDTH/2-2+i*2,y, ICON_MARK);
  }
}

void processStateChange() {

  renderBoardNamesMessages();
  handleAnimation();

  state.prevPlayerCount=clientState.game.playerCount;
  state.prevActivePlayer = clientState.game.activePlayer;
  state.prevRollsLeft = clientState.game.rollsLeft;
  state.prevRound = clientState.game.round;
}

void clearScores(uint8_t x) {
  for(j=0;j<15;j++)  {
    drawSpace(x, scoreY[j], 3);
  }
}

#define READY_LEFT WIDTH/2-8

void renderBoardNamesMessages() {
  static bool redraw, fullWidth;
  static uint8_t scoreCursorX, scoreCursorY, c, len,maxScoreY, newScoreFound, ignoreNewScore,
      mostRecentPlayer, i4;
  static Player *player;
  static int16_t score;

  // Could move this to a define to save space if runtime check is not needed
  fullWidth = WIDTH>=40;

  // If player is waiting on end game screen, auto-ready up if the game is starting

  // Redraw the entire board when going back to round 0 (ready up)
  redraw = clientState.game.round < state.prevRound || clientState.game.round == 1 &&
      state.prevRound==0;

  if (state.waitingOnEndGameContinue) {
    if (clientState.game.round == 1) {
      state.waitingOnEndGameContinue =false;
      // Force a redraw since the player waited on the end game screen until start
      redraw = true;
      clearRenderState();
    } else {

      if (state.countdownStarted)
        centerTextWide(HEIGHT-3,clientState.game.prompt);

      if (!state.countdownStarted && clientState.game.prompt[0]== 's') {
          soundJoinGame();
          if (clientState.game.players[0].scores[0] < 1) {
            apiCallForAll("ready");
          }

          state.countdownStarted = true;
        } else if (state.countdownStarted && clientState.game.prompt[0]!= 's') {
          state.countdownStarted = false;
        }
```

```
    return;
   }
  }


  if (redraw || state.drawBoard) {
   resetScreenNoBorder();
   redraw=true;

   if (clientState.game.round>0) {
     // Round 1+
     drawBoard();
     if (fullWidth) {
       drawLogo(0,0);
       drawTextAlt(1,4,"players");
#if defined(__WATCOMC__) || defined(COCO3)
       /* FUJITZEE-box bottom-right corner and score-box top-left corner
        * land on the same cell on msdos / CoCo 3. Replace it with a thick
        * cross so both boxes' edges visually connect through the joint. */
       drawIcon(9, 2, 0x50);
#endif
     }

   } else {
     // Round 0 - Ready Up screen
     drawLogo(WIDTH/2-5,1);
     CENTER_SERVER_TEXT(6, clientState.game.serverName);

     drawLine(READY_LEFT,7,16);
     centerTextAlt(18,"press " ESCAPE " for menu");
     centerTextAlt(HEIGHT-1,"press " TRIGGER_TEXT " to toggle");
   }


   state.drawBoard = false;
  }


  // Draw player names
  spectators=0;
  if (clientState.game.round>0) {
   for(i=1;i<=PLAYER_MAX;i++) {
     y=i+PLAYER_LIST_Y_OFFSET;
     x=INITIAL_X(i);

     if (i<=clientState.game.playerCount) {

       player = &clientState.game.players[i-1];
       c= player->name[player->alias];
       len = (uint8_t)strlen(player->name);

       if (player->scores[0]==-2) {
         // Draw spec icon in front of name
         if (fullWidth)
           drawIcon(0,y, ICON_SPEC);

         spectators++;
         // Clear initial/scoreboard for this player index if they were not previously viewing
         if (i<7 && !state.isViewing[i]) {
           drawBlank(x,1);
           clearScores(x-1);

           state.isViewing[i]=true;
         }
       } else {
         state.isViewing[i]=false;
         // Clear player indicator
         if (clientState.game.round>0 && clientState.game.activePlayer != i-1) {
           if (fullWidth)
             drawBlank(0,y);
           else if (!currentlyShowingHelp)
             drawBlank(x-1,1);
         }

         // Player initials across top of screen
         if (!currentlyShowingHelp)
           drawChar(x,1,c,1);
       }

       // Draw player's name, highlighting the alias as green
       if (fullWidth) {
         for (j=0;j<player->alias;j++) {
           drawChar(1+j,y,player->name[j], 0);
         }
         drawChar(1+j,y,player->name[j], 1);
         drawText(2+player->alias,y,player->name+player->alias+1);
         if (len<8) {
           drawSpace(1+len, y, 8-len);
         }
       }

     } else if (i<=state.prevPlayerCount || state.prevPlayerCount==0) {
       // Blank out entries for this player
       if (fullWidth)
         drawSpace(0,y,9);

       // Blank scoreboard
       if (i<7) {
         drawBlank(x,1);
         clearScores(x-1);
       }
     }
   }
  }

  // Round 0 (waiting to start) checks, or going into round 1
  if (clientState.game.round ==0) { //) || (clientState.game.round == 1 && state.prevRound==0))
     {
   // Display "waiting for players" prompt if changed in ready mode
   if (clientState.game.round==0) {
       centerTextWide(HEIGHT-3,clientState.game.prompt);

     if (clientState.game.prompt[0]== 's') {
       if (state.countdownStarted)
         soundTick();
       else {
         soundJoinGame();
         state.countdownStarted = true;
       }
     } else {
       state.countdownStarted = false;
     }
   }

   // Show players that are ready to start
   c++;
   for(i=0;i<9;i++) {
     if (i<clientState.game.playerCount) {
       drawText(READY_LEFT, 8+i, clientState.game.players[i].name);
       len = (uint8_t)strlen(clientState.game.players[i].name);
       if (len<8) {
         drawSpace(READY_LEFT+len, 8+i, 8-len);
       }
       drawText(READY_LEFT, 8+i, clientState.game.players[i].name);
       if (clientState.game.players[i].scores[0]) {
         drawTextAlt(READY_LEFT+11,8+i,"ready");
         //drawIcon(READY_LEFT+10,8+i, ICON_MARK);
       } else {
         drawSpace(READY_LEFT+11, 8+i, 5);
         drawIcon(READY_LEFT+12+(c%3),8+i, ICON_MARK);
       }
     } else if (i< state.prevPlayerCount) {
       drawSpace(READY_LEFT, 8+i, 16);
     }
   }

   // Exit early as below text is for rounds > 0
   return;
  }


  // Scoreboard and prompt - refresh if the active player changed, or round changed
  if (clientState.game.activePlayer != state.prevActivePlayer || clientState.game.round !=
     state.prevRound || clientState.game.playerCount != state.prevPlayerCount) {

   // Remove score cursorPos
   if (cursorPos>10 && cursorPos<99) {
     drawBlank(validX,scoreY[cursorPos-10]);
     cursorPos = 99;
   }

   if (clientState.game.round == 99) {
     clearBelowBoard();

     // Only specify scores header if there are scores (game not aborted early)
     if (clientState.game.players[0].scores[15]>0) {
       drawText(SCORES_X,21,"score");
     }
     setHighlight(-1,0,0);
   }


   // Update scores on-screen
   newScoreFound = 0;
   scoreCursorY=0;

   // Skip ahead to drawing the second round if the player hasn't changed
   ignoreNewScore= clientState.game.activePlayer == state.prevActivePlayer || redraw;


   // In case this client is lagging behind (multiple players scored since)
   // only animate the most recent player's score
   mostRecentPlayer = (clientState.game.activePlayer+clientState.game.playerCount-1) %
       clientState.game.playerCount;
   while (clientState.game.players[mostRecentPlayer].scores[0]==-2) {
     mostRecentPlayer = (mostRecentPlayer+clientState.game.playerCount-1) %
```

```
+         clientState.game.playerCount;
298   }
299
300
301   for (i=clientState.game.playerCount-1;i<255;i--) {
302     // Skip spectators or going beyond 6 players
303     if (i>5 || clientState.game.players[i].scores[0]==-2 || (currentlyShowingHelp && i>0))
304       continue;
305     h=SCORE_COL_X(i);
306     i4=h+3;
307     maxScoreY= clientState.game.round<99 ? 15 : 16;
308     for (j=0;j<maxScoreY;j++) {
309       score = clientState.game.players[i].scores[j];
310       if (score>-1) {
311         itoa(score, tempBuffer, 10);
312         len=(uint8_t)strlen(tempBuffer);
313         if (len<3) {
314           drawSpace(h,scoreY[j],len);
315         }
316         if (j!=6 && j!=7 && j!=15 && !newScoreFound && !ignoreNewScore && i!=
+             state.localPlayer[state.currentLocalPlayer].index && i==mostRecentPlayer &&
+             !state.renderedScore[i*16+j]) {
317           drawTextAlt(i4-len,scoreY[j],tempBuffer);
318           if (scoreCursorY==0 ) {
319             scoreCursorY=scoreY[j];
320             scoreCursorX=h;
321           }
322           newScoreFound=1;
323         } else {
324           drawText(i4-len,scoreY[j],tempBuffer);
325         }
326         state.renderedScore[i*16+j]=true;
327       } else if (ignoreNewScore) {
328         // Draw blank (just in case there was something there from a previous player)
329         drawSpace(h,scoreY[j],3);
330       }
331     }
332   }
333
334   // Animate arrow showing newly added score
335   if (!redraw && clientState.game.round<99 && scoreCursorY>0 && newScoreFound) {
336     drawIcon(scoreCursorX,scoreCursorY, ICON_CURSOR);
337     pause(25);
338     drawIcon(scoreCursorX,scoreCursorY, ICON_CURSOR_ALT);
339     soundScore();
340     pause(5);
341     drawIcon(scoreCursorX,scoreCursorY, ICON_CURSOR);
342     pause(30);
343     drawBlank(scoreCursorX,scoreCursorY);
344   }
345
346
347    // Clear old turn indicator
348   if (fullWidth && state.prevActivePlayer>-1)
349     drawBlank(0,state.prevActivePlayer+PLAYER_LIST_Y_OFFSET+1);
350
351   // Draw active player indicator
352   if (clientState.game.activePlayer>-1) {
353     if (fullWidth)
354       drawIcon(0,clientState.game.activePlayer+PLAYER_LIST_Y_OFFSET+1, ICON_MARK);
355     else
356       drawIcon(SCORE_COL_X(clientState.game.activePlayer),1, ICON_MARK);
357
358     setHighlight(clientState.game.activePlayer, state.localPlayerIsActive, 0);
359   } else {
360     setHighlight(-1,0,0);
361   }
362
363
364   // Display current round
365   if (clientState.game.round != state.prevRound) {
366     if (clientState.game.round>0 && clientState.game.round< 14) {
367       // Display static round details
368       itoa(clientState.game.round, tempBuffer, 10);
369       // A ternary, not 8-(round)9): z88dk's sccz80 (Adam, SMS) drops the
370       // 0/1 of a > comparison that feeds arithmetic, and draws round n at x-n
371       if (fullWidth) {
372         drawTextAlt(1,3,"round ");
373         drawTextAlt(1,4,"   of ");
374         drawText(7,4,"13");
375         drawText(clientState.game.round>9 ? 7 : 8,3,tempBuffer);
376       } else {
377         drawText(clientState.game.round>9 ? 2 : 3,1,tempBuffer);
378         drawTextAlt(4,1,"/");
379         drawText(5,1,"13");
380
381       }
382     } else if (fullWidth) {
383       drawSpace(1,3,8);
384       drawTextAlt(1,4,"players ");
385     }
386   }
387
388
389     if (clientState.game.round < 99) {
390       // Normal game messaging
391       if (!state.localPlayerIsActive)
392         pause(30);
393
394       clearBelowBoard();
395       if (spectators>0) {
396         itoa(spectators, tempBuffer, 10);
397
398
399         if (fullWidth) {
400           drawIcon(0,HEIGHT-1, ICON_SPEC);
401           strcat(tempBuffer," watching");
402         } else {
403           strcat(tempBuffer," spec");
404           if (spectators>1)
405             strcat(tempBuffer,"s");
406         }
407         drawTextAlt(1+fullWidth,HEIGHT-1, tempBuffer);
408       }
409
410       // Visual override - when playing multiple local players, put player's name + your turn
411       if (state.localPlayerIsActive) {
412         if (fullWidth) {
413         drawText(0,BOTTOM_PANEL_Y,"your turn");
414         drawText(0,BOTTOM_PANEL_Y+1,clientState.game.players[clientState.game.activePlayer].nam
+             e);
415         } else {
416           drawText(1,BOTTOM_PANEL_Y,"your");
417           drawText(1,BOTTOM_PANEL_Y+1,"turn");
418           //drawText(0,BOTTOM_PANEL_Y+2,clientState.game.players[clientState.game.activePlayer]
+             .name);
419         }
420       } else {
421         if (fullWidth) {
422           drawText(0,BOTTOM_PANEL_Y, clientState.game.prompt);
423         } else {
424           drawText(1,BOTTOM_PANEL_Y,"waiting on");
425           drawText(1,BOTTOM_PANEL_Y+1,clientState.game.players[clientState.game.activePlayer].n
+              ame);
426
427         }
428       }
429
430       pause(30);
431
432     } else {
433        // Handle end of game
434
435       // Clear active indicator
436       if (fullWidth && state.prevActivePlayer>-1) {
437         drawBlank(0,state.prevActivePlayer+PLAYER_LIST_Y_OFFSET+1);
438       }
439
440       centerText(GAMEOVER_PROMPT_Y, clientState.game.prompt);
441
442       if (clientState.game.round != state.prevRound) {
443         soundGameDone();
444
445         centerTextAlt(HEIGHT-1, "please wait..");
446         pause(240);
447         centerTextAlt(HEIGHT-1,"press " TRIGGER_TEXT " to continue");
448         state.waitingOnEndGameContinue = true;
449         state.countdownStarted = false;
450       }
451       clearCommonInput();
452     }
453
454     if (state.localPlayerIsActive && !clientState.game.viewing) {
455       setHighlight(clientState.game.activePlayer, true, 2);
456       pause(5);
457       soundMyTurn();
458       pause(5);
459       setHighlight(clientState.game.activePlayer, true, 0);
460     }
461   }
462 }
463
464 void handleAnimation() {
465   static bool isThisPlayer;
466   static uint8_t highScoreIndex, isFujitzee, i4, keptCountChanged;
467   static int16_t score;
468
469   waitvsync();
470
471   // Don't touch anything until player continues
472   if (state.waitingOnEndGameContinue)
473     return;
474
475   isThisPlayer = !clientState.game.viewing && state.localPlayerIsActive;
476
477   // Setup the player input details if this is a new roll
```

```
478  if (clientState.game.rollsLeft != state.prevRollsLeft || clientState.game.activePlayer !=
 +     state.prevActivePlayer )  {
479    state.rollFrames=ROLL_FRAMES;
480
481    if (clientState.game.rollsLeft==2) {
482      strcpy(state.prevKept,"11111");
483      strcpy(state.prevDice,clientState.game.dice);
484    }
485
486    if (isThisPlayer) {
487      state.playerMadeMove=false;
488      prevCursorPos=5;
489      cursorPos=1;
490      validX=SCORE_COL_X(clientState.game.activePlayer);
491
492
493      // If no rolls are remaining, default the cursorPos on the highest score
494      if (clientState.game.rollsLeft==0) {
495        highScoreIndex = 0;
496        for (j=1;j<15;j++) {
497          if (clientState.game.validScores[j]>clientState.game.validScores[highScoreIndex])
498            highScoreIndex = j;
499        }
500        cursorPos=10+highScoreIndex;
501      }
502    }
503  }
504
505  if (clientState.game.activePlayer(0 || !state.rollFrames)
506    return;
507
508  state.rollFrames--;
509
510  // Check for Fujitzee roll (all 5 dice match each other)
511  // To display special animation
512  if (state.rollFrames==0) {
513    strcpy(state.prevDice,clientState.game.dice);
514    for (i=1;i<5;i++) {
515      if (clientState.game.dice[i]!=clientState.game.dice[0])
516        break;
517    }
518    isFujitzee=i==5;
519  } else {
520    isFujitzee=0;
521  }
522
523  // Draw the dice, randomly displaying the ones that are currently being rolled
524  if (state.rollFrames % ROLL_SOUND_MOD ==0)
525    soundRollDice();
526
527
528  keptCountChanged=state.rollFrames == ROLL_FRAMES-1 && !isThisPlayer && strcmp(state.prevKept,
 +     clientState.game.keepRoll);
529
530  for(j=0;j<=isFujitzee;j++) {
531
532    // Play fujitzee sound when rolled, then clear flag so dice return to white
533    if (isFujitzee && j) {
534      soundFujitzee();
535      isFujitzee=0;
536    }
537    for(i=0;i<5;i++) {
538      i4=DIE_X(i);
539      if (state.rollFrames && clientState.game.keepRoll[i]=='1' ) {
540        // Draw a random die after the first frame
541        drawDie(i4,HEIGHT-4,state.rollFrames < ROLL_FRAMES-1 ? rand()%6+1 :
 +            state.prevDice[i]-48,0,isFujitzee);
542      } else {
543        // Draw the kept die
544        drawDie(i4,HEIGHT-4,clientState.game.dice[i]-48,(state.rollFrames ||
 +            clientState.game.rollsLeft)0) && clientState.game.keepRoll[i]=='0', isFujitzee);
545      }
546    }
547
548    soundStop();
549
550    // Pause a bit on the first render of a roll to show kept dice
551    if (keptCountChanged) {
552      strcpy(state.prevKept,clientState.game.keepRoll);
553      pause(30);
554    }
555  }
556
557
558  // If the rolling has stopped and this player is playing
559  if (isThisPlayer && !state.rollFrames) {
560
561    // Display the potential scores for this player
562    for (j=0;j<15;j++) {
563      score= clientState.game.validScores[j];
564      if (score)=0) {
565        if (score)0) {
566          itoa(score, tempBuffer, 10);
567          drawSpace(validX,scoreY[j],3-strlen(tempBuffer));
568        } else {
569          strcpy(tempBuffer,"  ");
570        }
571
572        drawTextAlt(validX+3-strlen(tempBuffer),scoreY[j],tempBuffer);
573      }
574    }
575
576    // Draw "Rolls" die if this player's turn and there are rolls left
577    drawDie(ROLL_X,HEIGHT-4,clientState.game.rollsLeft+13,0,0);
578
579  #ifdef __WATCOMC__
580      setHighlight(clientState.game.activePlayer, true, 0);
581  #endif
582    }
583  }
584
585  void processInput() {
586    readCommonInput();
587
588    if (state.waitingOnEndGameContinue) {
589      if (input.trigger) {
590        state.waitingOnEndGameContinue = false;
591        clearBelowBoard();
592        clearRenderState();
593      }
594    } else if (!clientState.game.viewing) {
595      // Toggle readiness if waiting to start game
596      if (clientState.game.round == 0 && input.trigger) {
597        j=!clientState.game.players[0].scores[0];
598        for(i=0;i<prefs.localPlayerCount;i++) {
599          clientState.game.players[state.localPlayer[i].index].scores[0] = j;
600        }
601        renderBoardNamesMessages();
602        if (clientState.game.players[0].scores[0])
603          soundScore();
604        else
605          soundScoreCursor();
606
607        apiCallForAll("ready");
608        return;
609      }
610
611      // Wait on this player to make roll decisions
612      if (!state.rollFrames && state.localPlayerIsActive && !state.playerMadeMove &&
 +         clientState.game.round)0) {
613        waitOnPlayerMove();
614      }
615    }
616
617    switch(input.key) {
618        case KEY_ESCAPE: // Esc
619        case KEY_ESCAPE_ALT: // Esc Alt
620          showInGameMenuScreen();
621          break;
622      }
623  }
624
625  void hideInGameHelp() {
626    if (!currentlyShowingHelp)
627      return;
628
629    if (!restoreScreen()) {
630      state.drawBoard = true;
631     // clearRenderState();
632     // setHighlight(-1,0,0);
633     // processStateChange();
634    }
635    currentlyShowingHelp = false;
636  }
637
638  void showInGameHelp() {
639    if (clientState.game.activePlayer!=0)
640      return;
641
642    currentlyShowingHelp=true;
643    saveScreen();
644
645    for(y=1;y<20 ;y++) {
646      drawSpace(HELP_CLEAR_X,y,HELP_CLEAR_W);
647    }
648
649    drawBox(HELP_X-1,1,17, 17);
650
651    drawText(HELP_X+2, 3,"it's your turn");
652                                                    //123456789012345678
653    drawText(HELP_X, 5,"move up/down to");
654    drawText(HELP_X, 6,"choose a score on");
655    drawText(HELP_X, 7,"the left based on");
656    drawText(HELP_X, 8,"your dice.");
657
658    drawText(HELP_X,10,"before you score,");
```

```
  drawText(HELP_X,11,"you may re-roll ");
  drawText(HELP_X,12,"any of your dice");
  drawText(HELP_X,13,"up to two times.");

  drawText(HELP_X,15,"move left/right,");
  drawText(HELP_X,16,"and pick dice to");
  drawText(HELP_X,17,"keep, then roll.");
}

void waitOnPlayerMove() {
  static bool foundValidLocation;
  static uint8_t waitCount, frames;
  static uint16_t jifsPerSecond;
  resetTimer();

  // Determine max jiffies for PAL and NTS
  jifsPerSecond=getJiffiesPerSecond();
  maxJifs = jifsPerSecond*clientState.game.moveTime;
  waitCount=frames=0;

  // If this is the player's first time playing, show instructions
  if (!prefs.hasPlayed) {
    prefs.hasPlayed=true;
    showInGameHelp();
    savePrefs();
  }

  // Move selection loop
  while (clientState.game.moveTime>0) {
    frames=(frames+1) % 30;
    waitvsync();

    // Handle trigger press
    if (input.trigger) {

      if (cursorPos>=10) {
        // Select score
        i=cursorPos-10;

        state.renderedScore[clientState.game.activePlayer*16+i]=true;

        // Cursor select animation
        drawIcon(validX,scoreY[cursorPos-10],ICON_CURSOR_ALT);soundScore();

        // First, hide all score options but the chosen score
        for (j=0;j<15;j++) {
          if (clientState.game.validScores[j]>0) {
            if (j != i)
              drawSpace(validX,scoreY[j],3);
          }
        }

        pause(5);drawIcon(validX,scoreY[cursorPos-10],ICON_CURSOR);; pause(10);

        // Send command to score this value
        strcpy(tempBuffer, "score/");
        itoa(i, tempBuffer+6, 10);
        sendMove(tempBuffer);

        hideInGameHelp();
        clearBelowBoard();
        state.playerMadeMove = true;

        return;
      } else if (cursorPos>0) {
        // Toggle kept state of die
        i = clientState.game.keepRoll[cursorPos-1]= clientState.game.keepRoll[cursorPos-1]=='1'
          ? '0' : '1';
        drawDie(KEPT_DIE_X(cursorPos),HEIGHT-4,clientState.game.dice[cursorPos-1]-48,i == '0',
            0);
        if (i=='0')
          soundKeep();
        else
          soundRelease();

        drawDie(ROLL_X,HEIGHT-4,13+ (strcmp(clientState.game.keepRoll,"00000") ?
            clientState.game.rollsLeft : 3),0,0);
      } else {
        // Request another roll, assuming at least one dice is not kept
        if (strcmp(clientState.game.keepRoll,"00000")) {
          // Highlight the roll die in green
          drawDie(ROLL_X,HEIGHT-4,clientState.game.rollsLeft+13,0,1);
          soundRollButton();

          strcpy(tempBuffer, "roll/");
          strcat(tempBuffer, clientState.game.keepRoll);
          sendMove(tempBuffer);

          state.playerMadeMove = true;
          hideDiceCursor(DICE_CURSOR_X(prevCursorPos));
          // Clear clock
          drawSpace(TIMER_X,BOTTOM_PANEL_Y,2);
          drawSpace(TIMER_X+TIMER_NUM_OFFSET_X-2,BOTTOM_PANEL_Y+TIMER_NUM_OFFSET_Y,2);
          return;
        } else {
          // If all 5 dice are kept, there is nothing to roll.
          // Flash all dice, then move cursor up
          for (j=1;j<255;j--) {
            drawDie(ROLL_X,HEIGHT-4,16,0,j);
            for(i=0;i<6;i++) {
              drawDie(DIE_X(i),HEIGHT-4,clientState.game.dice[i]-48,1,j);
            }
            soundRelease();
            pause(6);
          }

          input.dirY=-1;
          input.dirX=0;
        }
      }
    }

    // Update horizontal cursorPos location - dice: 0 = roll, 1-5 equal select dice at that
        index
    if (input.dirX !=0 && clientState.game.rollsLeft) {
      // If cursorPos was selecting a score, bring it back down
      if (cursorPos>9)
        cursorPos = 1;

      cursorPos+=input.dirX;

      // Bounds check
      if (cursorPos>5)
        cursorPos= cursorPos==255?5:0;

    } else if (input.dirY == 1 && cursorPos<10) {
      // If pressing down on dice cursor, navigate directly to roll button.
      cursorPos=0;

      // Update vertical cursorPos location - scores: 10-24 represent the possible scoring
          locations
    }else if (input.dirY != 0) {
      // If cursorPos was selecting dice, bring it up
      if (cursorPos<10)
        cursorPos=25;

      while (1) {
        cursorPos+=input.dirY;

        // Bounds checks
        foundValidLocation=true;
        if (cursorPos<10 || (cursorPos>24 && clientState.game.rollsLeft==0))
          cursorPos = prevCursorPos;
        else if (cursorPos>24)
          cursorPos=prevCursorPos <10 ? prevCursorPos : 1;
        else {
          // Skip over invalid scores
          if (clientState.game.validScores[cursorPos-10]<0)
            foundValidLocation=false;
        }

        // Break once we are at a valid location
        if (foundValidLocation)
          break;
      }
    }

    waitvsync();

    // Draw cursorPos
    if (cursorPos != prevCursorPos) {

      // Hide cursorPos
      if (prevCursorPos < 6)
        hideDiceCursor(DICE_CURSOR_X(prevCursorPos));
      else {
        h=prevCursorPos-10;
        drawBlank(validX,scoreY[h]);
        if (clientState.game.validScores[h]==0) {
          drawBlank(validX+2,scoreY[h]);
        }
      }

      // Draw cursorPos
      if (cursorPos < 6) {
        drawDiceCursor(DICE_CURSOR_X(cursorPos));
        soundCursor();
      } else {
        h=cursorPos-10;
        drawIcon(validX,scoreY[h],ICON_CURSOR);
        if (clientState.game.validScores[h]==0) {
          drawTextAlt(validX+2,scoreY[h],"0");
        }
        soundScoreCursor();
      }
      prevCursorPos = cursorPos;
```

```
840
841
842     } else if (cursorPos>9){
843       drawIcon(validX,scoreY[cursorPos-10],(frames<2)? ICON_CURSOR_BLIP : ICON_CURSOR);
844     }
845
846     // Tick counter once per second
847     if (++waitCount>5) {
848       waitCount=0;
849       i = (maxJifs-getTime())/jifsPerSecond;
850       if (i<=15 && i != clientState.game.moveTime) {
851         clientState.game.moveTime = i;
852         if (i<10)
853           tempBuffer[0]=' ';
854
855         itoa(i, tempBuffer+(i<10), 10);
856         drawTextAlt(TIMER_X+TIMER_NUM_OFFSET_X-strlen(tempBuffer),
 +             BOTTOM_PANEL_Y+TIMER_NUM_OFFSET_Y, tempBuffer);
857         drawClock(TIMER_X,BOTTOM_PANEL_Y);
858         soundTick();
859       }
860     }
861
862     // Pressed Esc
863     switch (input.key) {
864       case 'h':case 'H':
865         if (currentlyShowingHelp)
866           hideInGameHelp();
867         else
868           showInGameHelp();
869         break;
870       case KEY_ESCAPE:
871       case KEY_ESCAPE_ALT:
872         hideInGameHelp();
873         showInGameMenuScreen();
874         return;
875     }
876
877     // Read input for next iteration
878     readCommonInput();
879   }
880
881   // Timed out, so hide all scores
882   hideInGameHelp();
883   clearBelowBoard();
884
885   centerText(BOTTOM_PANEL_Y+1,"you timed out. auto scoring.");
886   state.playerMadeMove=1;
887   i=0;
888   for (j=0;j<15;j++) {
889     if (clientState.game.validScores[j]>-1) {
890       if (!i) {
891         itoa(clientState.game.validScores[j], tempBuffer, 10);
892         drawTextAlt(validX+3-strlen(tempBuffer),scoreY[j],tempBuffer);
893         i=1;
894       } else {
895         drawSpace(validX,scoreY[j],3);
896       }
897     }
898   }
899
900   pause(60);
901 }
902
903 // Invalidate state variables that will trigger re-rendering of screen items on the next cycle
904 void clearRenderState() {
905   state.prevActivePlayer = state.prevRound = 99;
906   state.prevPlayerCount = 0;
907   memset(state.renderedScore, 0, 16*6);
908   state.drawBoard=true;
909   setHighlight(-1,0,0);
910 }
911
912 /// @brief Convenience function to draw text centered at row Y
913 void centerText(unsigned char y, char * text) {
914   drawText(WIDTH/2-strlen(text)/2, y, text);
915 }
916
917 /// @brief Convenience function to draw text centered at row Y, blanking out the rest of the
 +    row
918 void centerTextWide(unsigned char y, char * text) {
919   i = strlen(text);
920   x = WIDTH/2-i/2;
921
922   drawSpace(0,y, x);
923   drawText(x, y, text);
924   drawSpace(x+i,y, WIDTH-x-i);
925 }
926
927 /// @brief Convenience function to draw text centered at row Y in alternate color
928 void centerTextAlt(unsigned char y, char * text) {
929   drawTextAlt(WIDTH/2-strlen(text)/2, y, text);
930 }
931
932 /// @brief Convenience function to draw status text centered
933 void centerStatusText(char * text) {
934   drawTextAlt((WIDTH-strlen(text))>>1,HEIGHT-1,text);
935 }
936
937 /// @brief Init/reset the input field for display
938 void resetInputField() {
939   inputField_done = 1;
940   disableKeySounds();
941 }
942
943 /// @brief Handles available key strokes for the defined input box (player name and chat).
 +    Returns true if user hits enter
944 bool inputFieldCycle(uint8_t x, uint8_t y, uint8_t max, char* buffer) {
945   static uint8_t curx, lastY;
946
947   // Initialize first call to input box
948   if (inputField_done == 1 || lastY != y) {
949     inputField_done=0;
950     lastY=y;
951     curx = strlen(buffer);
952     drawTextAlt(x,y, buffer);
953     drawIcon(x+curx,y, ICON_TEXT_CURSOR);
954     enableKeySounds();
955   }
956
957   // Process any waiting keystrokes
958   if (kbhit()) {
959     inputField_done=0;
960
961     input.key = cgetc();
962
963     // Debugging - See what key was pressed
964     //itoa(input.key, tempBuffer, 10);drawText(0,0, tempBuffer);
965
966     if (input.key == KEY_RETURN && curx>1) {
967       inputField_done=1;
968       // remove cursor
969       drawBlank(x+curx,y);
970     } else if ((input.key == KEY_BACKSPACE || input.key == KEY_LEFT_ARROW) && curx>0) {
971       buffer[--curx]=0;
972       drawText(x+1+curx,y," ");
973     } else if (
974       curx < max && ((curx)0 && input.key == KEY_SPACEBAR) || (input.key)= 48 && input.key
 +          (=57) || (input.key)= 65 && input.key (=90) || (input.key)= 97 && input.key (=122))
 +          // 0-9 A-Z a-z
975     ) {
976
977       if (input.key)=65 && input.key(=90)
978         input.key+=32;
979
980       buffer[curx]=input.key;
981       buffer[++curx]=0;
982     }
983
984     drawTextAlt(x,y, buffer);
985
986     if (inputField_done)
987       disableKeySounds();
988     else
989       drawIcon(x+curx,y, ICON_TEXT_CURSOR);
990
991     return inputField_done;
992   }
993
994   return false;
995
996 }
```

SRC/GAMELOGIC.H 29 LINES

```
1 #ifndef GAMELOGIC_H
2 #define GAMELOGIC_H
3
4 #include "platform-specific/graphics.h"
5
6 void processStateChange();
7 void processInput();
8 void handleAnimation();
9
10
11 void clearRenderState();
12
13 void centerText(unsigned char y, char * text);
14 void centerTextAlt(unsigned char y, char * text);
15 void centerTextWide(unsigned char y, char * text);
16 void centerStatusText(char * text);
17
18 void resetInputField();
19 bool inputFieldCycle(uint8_t x, uint8_t y, uint8_t max, char* buffer);
20
```

```
21 void renderBoardNamesMessages();
22
23 void waitOnPlayerMove();
24
25
26 void progressAnim(unsigned char y);
27
28
29 #endif /*GAMELOGIC_H*/
```

SRC/MISC.C 177 LINES

```
 1 #if !defined(__WATCOMC__) && !defined(__ADAM__) && !defined(BUILD_SMS) && !defined(__H89__)
 2 #include <joystick.h>
 3 #endif
 4 #include <string.h>
 5 #include "platform-specific/graphics.h"
 6 #include "platform-specific/input.h"
 7 #include "platform-specific/sound.h"
 8 #include "misc.h"
 9 #ifdef CUSTOM_FUJINET_CALLS
10 // Implemented in platform-specific code, for a platform whose fujinet-lib
11 // headers cannot be included alongside the game's (see src/nes/fujinet.c)
12 uint16_t custom_read_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, char
 +     *destination);
13 void custom_write_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, uint16_t count,
 +     char *data);
14 #else
15 #include "fujinet-fuji.h"
16 #endif
17
18 #include <stdbool.h>
19 #include <stdint.h>
20
21 #ifdef __WATCOMC__
22 #include <conio.h>      /* Watcom kbhit() */
23 #define JOY_LEFT(v)   0
24 #define JOY_RIGHT(v)  0
25 #define JOY_UP(v)     0
26 #define JOY_DOWN(v)   0
27 #define JOY_BTN_1(v)  0
28 #define JOY_BTN_2(v)  0
29 extern char cgetc(void);
30 #endif
31
32 InputStruct input;
33 unsigned char _lastJoy, _joy, _joySameCount=10;
34 bool _buttonReleased=true;
35
36 #ifndef JOY_BTN_2_MASK
37 #define JOY_BTN_2_MASK JOY_BTN_1_MASK
38 #endif
39
40 void pause(unsigned char frames) {
41   while(frames--)
42     waitvsync();
43 }
44
45 void clearCommonInput() {
46   input.trigger=input.key=input.dirY=input.dirX=_lastJoy=_joy=_buttonReleased=0;
47   while (kbhit())
48     cgetc();
49 }
50
51 void readCommonInput() {
52   input.trigger=input.key=input.dirX=input.dirY=0;
53
54   _joy = readJoystick();
55
56   if (_joy != _lastJoy) {
57     if (_lastJoy!=99)
58       _joySameCount=12;
59
60     _lastJoy=_joy;
61
62     if (JOY_LEFT(_joy))
63       input.dirX = -1;
64     else if (JOY_RIGHT(_joy))
65       input.dirX =1;
66
67     if (JOY_UP(_joy))
68       input.dirY = -1;
69     else if (JOY_DOWN(_joy))
70       input.dirY = 1;
71
72     // Trigger button press only if it was previously unpressed
73     if (JOY_BTN_1(_joy) || JOY_BTN_2(_joy)) {
74       if (_buttonReleased) {
75         input.trigger=true;
76         _buttonReleased=false;
77       }
78     } else {
79       _buttonReleased = true;
80     }
81
82     return;
83   } else if (_joy!=0) {
84     if (!_joySameCount--) {
85       _joySameCount=0;
86       _lastJoy=99;
87     }
88   }
89
90   input.key=0;
91
92   if (!kbhit())
93     return;
94
95   input.key = cgetc();
96
97   switch (input.key) {
98     case KEY_LEFT_ARROW:
99     case KEY_LEFT_ARROW_2:
100    case KEY_LEFT_ARROW_3:
101      input.dirX=-1;
102      break;
103    case KEY_RIGHT_ARROW:
104    case KEY_RIGHT_ARROW_2:
105    case KEY_RIGHT_ARROW_3:
106      input.dirX=1;
107      break;
108    case KEY_UP_ARROW:
109    case KEY_UP_ARROW_2:
110    case KEY_UP_ARROW_3:
111      input.dirY=-1;
112      break;
113    case KEY_DOWN_ARROW:
114    case KEY_DOWN_ARROW_2:
115    case KEY_DOWN_ARROW_3:
116      input.dirY=1;
117      break;
118    case KEY_SPACEBAR:
119    case KEY_RETURN:
120      input.trigger=true;
121      break;
122  }
123 }
124
125 void loadPrefs() {
126   read_appkey(AK_CREATOR_ID, AK_APP_ID, AK_KEY_PREFS, tempBuffer);
127
128   if (strlen(tempBuffer))0) {
129     memcpy(&prefs, tempBuffer, sizeof(prefs));
130
131     if (prefs.debugFlag == 0xff) {
132       strcpy(serverEndpoint, localServer);
133     }
134   }
135
136   // Ensure local player count is at least one (older appkeys may have this set to 0)
137 #ifdef SINGLE_LOCAL_PLAYER
138   prefs.localPlayerCount=1;
139 #else
140   if (prefs.localPlayerCount==0 || prefs.localPlayerCount)4) {
141     prefs.localPlayerCount=1;
142   }
143 #endif
144
145   setColorMode(prefs.color);
146 }
147
148 void savePrefs() {
149   write_appkey(AK_CREATOR_ID, AK_APP_ID, AK_KEY_PREFS, sizeof(prefs), (char*)&prefs);
150 }
151
152
153 uint16_t read_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, char* destination) {
154   uint16_t read=0;
155
156 #ifdef CUSTOM_FUJINET_CALLS
157   read = custom_read_appkey(creator_id, app_id, key_id, destination);
158 #else
159   fuji_set_appkey_details(creator_id, app_id, DEFAULT);
160   if (!fuji_read_appkey(key_id, &read, (uint8_t*)destination))
161     read=0;
162 #endif
163
164   // Add string terminator after the data ends in case it is being interpreted as a string
165   destination[read] = 0;
166   return read;
167 }
168
169 void write_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id,  uint16_t count, char
 +     *data)
170 {
```

```
#ifdef CUSTOM_FUJINET_CALLS
  custom_write_appkey(creator_id, app_id, key_id, count, data);
#else
  fuji_set_appkey_details(creator_id, app_id, DEFAULT);
  fuji_write_appkey(key_id, count, (uint8_t*)data);
#endif
}
```

SRC/MISC.H 184 LINES

```
#ifndef MISC_H
#define MISC_H

#if defined(__ADAM__) || defined(BUILD_SMS) || defined(__H89__)
/* z88dk's conio.h maps cgetc() onto getk(), a dead stub on the Adam
 * target - kbhit/cgetc are implemented over EOS in src/adam/input.c.
 * cgetc must return unsigned: sccz80 chars are signed and the Adam's
 * arrow/SmartKeys codes are 0x81-0xA8 - a plain char return would
 * sign-extend them and no KEY_* case in the input switch would match.
 * The Master System is z88dk too, with no keyboard: src/sms/input.c
 * reports its buttons as these key codes. So is the Heathkit H89, whose
 * src/h89/input.c decodes the H19's escape sequences itself. */
unsigned char kbhit(void);
unsigned char cgetc(void);
#elif !defined(__WATCOMC__)
#include <joystick.h>
#include <conio.h>
#endif
#include "platform-specific/graphics.h"
#include "platform-specific/input.h"
#include <stdbool.h>
#include <stdint.h>

// FujiNet AppKey settings. These should not be changed
#define AK_LOBBY_CREATOR_ID 1     // FUJINET Lobby
#define AK_LOBBY_APP_ID 1         // Lobby Enabled Game
#define AK_LOBBY_KEY_USERNAME 0   // Lobby Username key
#define AK_LOBBY_KEY_SERVER 3     // Fujzee registered as Lobby appkey 3 at
+    https://github.com/FujiNetWIFI/fujinet-firmware/wiki/SIO-Command-$DC-Open-App-Key

// Fujitzee
#define AK_CREATOR_ID 0xE41C      // Eric Carr's creator id
#define AK_APP_ID 3               // Fujzee App ID
#define AK_KEY_PREFS 0            // Preferences

#define PLAYER_MAX 12

#define FUJITZEE_SCORE 14

#if defined(__WATCOMC__) || defined(FUJITZEE_PACK_STRUCTS)
/* Watcom defaults to 2-byte struct alignment in 16-bit, which inserts
 * padding before int16_t fields. The server sends bytes assuming the
 * cc65/CMOC tightly-packed layout, so any padding here shifts every
 * field after the pad and corrupts the parsed state (e.g. the local
 * player's name shows up missing its first letter). Force 1-byte
 * packing for the Game/Player/Tables structs so the binary layout
 * matches what the server sends.
 *
 * Any toolchain whose default alignment is wider than 1 byte needs this;
 * define FUJITZEE_PACK_STRUCTS in its build to opt in. (m68k-amigaos-gcc
 * is one: it aligns int16_t to 2 bytes, which pads Game.players[] from
 * offset 95 to 96 and reads every player record one byte out of phase.)
 * Both GCC and Clang honour #pragma pack. */
#pragma pack(push, 1)
#endif

typedef struct {
  char table    [9];
  char name     [21];
  char players  [6];
} Table;

typedef struct {
  char name[9];
  uint8_t alias;
  int16_t scores[16];
} Player;

typedef struct {
  uint8_t count;
  Table table[10];
} Tables;

typedef struct {
  uint8_t playerCount;
  char serverName [21];
  char prompt     [41];
  uint8_t round;
  uint8_t rollsLeft;
  int8_t activePlayer;
  uint8_t moveTime;
  uint8_t viewing;
  char dice     [6];
  char keepRoll [6];
  int8_t validScores[15];
  Player players[PLAYER_MAX];
} Game;

#if defined(__WATCOMC__) || defined(FUJITZEE_PACK_STRUCTS)
#pragma pack(pop)
#endif

typedef union {
  uint8_t firstByte;
  Game game;
  Tables tables;
} ClientState;

extern ClientState clientState;

typedef struct {
  char query[50]; //?table=12345678&pov=12345678&player=12345678
  uint8_t index;
} LocalPlayerState;

typedef struct {

  // Internal game state
  uint8_t rollFrames;

  uint8_t prevRollsLeft;
  uint8_t prevPlayerCount;
  uint8_t prevRound;

  uint8_t apiCallWait;

  int8_t prevActivePlayer;

  bool playerMadeMove;

  bool countdownStarted;
  bool waitingOnEndGameContinue;
  bool drawBoard;
  bool isViewing[PLAYER_MAX];

  int8_t currentLocalPlayer;
  bool localPlayerIsActive;
  LocalPlayerState localPlayer[4];
  bool renderedScore[16*6];
  bool inGame;
  char prevKept[6];
  char prevDice[6];

} GameState;

typedef struct {
  unsigned char key;
  bool trigger;
  int8_t dirX;
  int8_t dirY;
} InputStruct;


typedef struct {
  char name[9];
} LocalPlayer;

typedef struct {
  bool seenHelp;
  uint8_t color;
  uint8_t debugFlag; // 0xFF to use localhost instead of server
  uint8_t localPlayerCount;
  LocalPlayer localPlayer[4];
  uint8_t disableSound;
  bool hasPlayed;
} PrefsStruct;


extern uint16_t maxJifs;
extern char tempBuffer[128];
extern char serverEndpoint[50];
extern char localServer[];

extern GameState state;
extern InputStruct input;
extern PrefsStruct prefs;

// Common local scope temp variables
extern unsigned char h, i, j, k, x, y;

void pause(unsigned char frames);
void clearCommonInput();
void readCommonInput();
void loadPrefs();
void savePrefs();
```

```
175
176
177 /// @brief Helper method to write to an appkey
178 void write_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id,  uint16_t count, char
  +     *data);
179
180 /// @brief Helper method to read from an appkey.
181 /// NULL will be appended to data in case this is a string, though the length returned will not
  +     consider the NULL.
182 uint16_t read_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, char* destination);
183
184 #endif /* MISC_H */
```

SRC/SCREENS.C 804 LINES

```
  1 #include <stdlib.h>
  2 #include "stateclient.h"
  3 #include "screens.h"
  4 #include "misc.h"
  5 #include "gamelogic.h"
  6
  7 #include "platform-specific/graphics.h"
  8 #include "platform-specific/input.h"
  9 #include "platform-specific/util.h"
 10 #include "platform-specific/sound.h"
 11 #include <string.h>
 12
 13 #define PLAYER_NAME_MAX 8
 14 #define PLAYER_BOX_TOP 13
 15 #define REMOVE_PLAYER_KEY '/'
 16 #define INGAME_MENU_X WIDTH/2-8
 17
 18 // The table list draws the selected table in the normal style and the others
 19 // in the alternate one. A platform whose alternate style is reverse video
 20 // (the H89) turns that around.
 21 #ifndef LOBBY_SEL_TEXT
 22 #define LOBBY_SEL_TEXT drawText
 23 #define LOBBY_UNSEL_TEXT drawTextAlt
 24 #endif
 25
 26 // Server names are mixed case, which the alternate style splits up on some
 27 // platforms (see gamelogic.c)
 28 #ifndef CENTER_SERVER_TEXT
 29 #define CENTER_SERVER_TEXT centerTextAlt
 30 #endif
 31
 32 char query[20] = "";
 33 bool inBorderedScreen=false, prevBorderedScreen=false, savedScreen=false;
 34
 35
 36 bool saveScreen() {
 37   prevBorderedScreen=inBorderedScreen;
 38   return savedScreen = saveScreenBuffer();
 39 }
 40
 41 bool restoreScreen() {
 42   if (savedScreen) {
 43     savedScreen=false;
 44     inBorderedScreen=prevBorderedScreen;
 45     restoreScreenBuffer();
 46     return true;
 47   }
 48   return false;
 49 }
 50
 51 void resetScreenNoBorder() {
 52   inBorderedScreen=false;
 53   resetScreen(inBorderedScreen);
 54 }
 55
 56 /// @brief Convenience function to reset screen and draw border
 57 void resetScreenWithBorder() {
 58
 59   resetScreen(inBorderedScreen);
 60
 61   if (!inBorderedScreen) {
 62     // Draw dice corners
 63     drawDie(0,0,1, 0,0);
 64     //drawDie(0,HEIGHT/2-2,"5", 0);
 65     drawDie(0,HEIGHT-3,3, 0,0);
 66
 67     drawDie(WIDTH-3,0,2, 0,0);
 68     //drawDie(WIDTH-3,HEIGHT/2-2,"6", 0);
 69     drawDie(WIDTH-3,HEIGHT-3,4, 0,0);
 70     inBorderedScreen=true;
 71   }
 72 }
 73
 74 // Draws a help line and advances the row; keeps showHelpScreen's code compact.
 75 static unsigned char hy;
 76 // HELP_LEFT moves the 40 column help text right, to centre it on a wider
 77 // screen.
 78 #ifdef HELP_LEFT
 79 static void helpLn(unsigned char x, char *s) { drawTextAlt(x + HELP_LEFT, ++hy, s); }
 80 #else
 81 static void helpLn(unsigned char x, char *s) { drawTextAlt(x, ++hy, s); }
 82 #endif
 83
 84 /// @brief Shows information about the game
 85 void showHelpScreen() {
 86   resetScreenWithBorder();
 87
 88   drawTextAlt(WIDTH/2-10,1,"how to play fujiTZEE");
 89   drawLine(WIDTH/2-10,2,20);
 90   hy=3;
 91
 92 #ifdef ONLINE_HELP
 93   hy+=8;centerTextAlt(hy, "ONLINE HELP FOR cOcO");
 94   hy+=2;centerText(hy, "is coming soon..");
 95 #else
 96
 97   #if WIDTH < 40
 98               //12345678901234567890123456789012
 99   helpLn(2, "TAKE TURNS ROLLING DICE");
100   helpLn(2, "UP TO three TIMES A turn");
101   helpLn(2, "TO MAKE COMBOS LIKE A set,");
102   helpLn(2, "full house OR run.");
103   hy++;
104   helpLn(2, "score A COMBO TO EARN");
105   helpLn(2, "POINTS EACH turn.");
106   hy++;
107   helpLn(2, "PLAY 13 ROUNDS UNTIL THE");
108   helpLn(2, "SCORESHEET IS FILLED.");
109   helpLn(2, "TOP score WINS!");
110   hy+=2;
111   centerTextAlt(hy, "turn steps");
112   hy++;
113   helpLn(6, "1. keep ANY DICE");
114   helpLn(6, "2. roll FOR NEW DICE");
115   helpLn(6, "3. REPEAT 1-2");
116   helpLn(6, "4. CHOOSE A score");
117   centerStatusText("any KEY FOR score INFO");
118   #else
119                       //12345678901234567890123456789012345678901234567890
120   helpLn(3, "TAKE TURNS ROLLING DICE UP TO three");
121   helpLn(3, "TIMES PER turn TO GET COMBINATIONS");
122   helpLn(3, "LIKE A set, full house, OR run.");
123   hy++;
124   helpLn(3, "score YOUR COMBO TO EARN POINTS.");
125   hy++;
126   helpLn(3, "PLAY CONTINUES FOR 13 ROUNDS UNTIL");
127   helpLn(3, "THE SCORESHEET IS FILLED.");
128   hy++;
129   helpLn(3, "THE PLAYER WITH THE highest TOTAL");
130   helpLn(3, "SCORE WINS THE GAME!");
131   hy+=2;
132   centerTextAlt(hy, "turn steps");
133   hy++;
134   helpLn(7, "1. SELECT ANY DICE TO keep");
135   helpLn(7, "2. roll TO GET NEW DICE");
136   helpLn(7, "3. YOU MAY REPEAT STEPS 1-2");
137   helpLn(7, "4. CHOOSE YOUR score");
138   centerStatusText("PRESS ANY KEY FOR score INFO");
139   #endif
140
141   clearCommonInput();
142   cgetc();
143
144   resetScreenWithBorder();
145   drawTextAlt(WIDTH/2-9,1,"scoring your rolls");
146   drawLine(WIDTH/2-9,2,18);
147
148   #if WIDTH < 40
149   hy=4;centerTextAlt(hy, "upper scores");
150   hy+=2;drawText(2,hy, "add dice matching the");
151   hy++;drawText(2,hy, "number rolled.");
152   hy+=2;centerTextAlt(hy, "upper bonus");
153   hy++;
154   helpLn(2, "63+ IN UPPER SCORES A");
155   helpLn(2, "bonus OF 35 POINTS.");
156   hy+=2;centerTextAlt(hy, "bottom scores");
157   hy++;
158   helpLn(2, "set   - x OF A KIND");
159   helpLn(2, "house - FULL HOUSE");
160   helpLn(2, "run   - RUN, EG 12345");
161   helpLn(2, "count - ADD ALL DICE");
162   helpLn(2, "      - 5 OF A KIND");
163   drawFujitzee(2,hy);
164   #else
165   hy=4;centerTextAlt(hy, "upper scores");
166 #ifdef HELP_LEFT
167   hy+=2;drawText(5+HELP_LEFT,hy, "total all dice that match the");
168   hy++;drawText(5+HELP_LEFT,hy, "number");
169 #else
170   hy+=2;drawText(5,hy, "total all dice that match the");
```

```
  hy++;drawText(5,hy, "number");
#endif
  hy+=2;centerTextAlt(hy, "upper bonus");
  hy++;
  helpLn(5, "IF UPPER TOTAL IS 63 OR HIGHER");
  helpLn(5, "YOU SCORE A bonus 35 POINTS");
  hy+=3;centerTextAlt(hy, "bottom scores");
  hy++;
  helpLn(4, "set   - x OF A KIND - ADD ALL DICE");
  helpLn(4, "house - FULL HOUSE - SET OF 2 AND 3");
  helpLn(4, "run   - RUN OF x - EXAMPLE 12345");
  helpLn(4, "count - SCORE TOTAL OF ALL DICE");
  helpLn(4, "      - 5 OF A KIND");
#ifdef HELP_LEFT
  drawFujitzee(4+HELP_LEFT,hy);
#else
  drawFujitzee(4,hy);
#endif
  #endif
#endif

  centerStatusText("press any key to close");

  clearCommonInput();
  cgetc();
}

/// @brief Action called in Welcome Screen to check if a server name is stored in an app key
void welcomeActionVerifyServerDetails() {
  // Read server endpoint into server and query
  read_appkey(AK_LOBBY_CREATOR_ID,  AK_LOBBY_APP_ID, AK_LOBBY_KEY_SERVER, tempBuffer);

  i=strlen(tempBuffer);
  if (i) {

    // Split "server?query" into separate variables
    while(--i) {
      if (tempBuffer[i]=='?') {
        strcpy(query, tempBuffer+i);
        tempBuffer[i]=0;
        strcpy(serverEndpoint, tempBuffer);
        break;
      }
    }
  }
}

void drawLogo(uint8_t x, uint8_t y)
{
  drawBox(x,y,8,1);
  drawTextAlt(x+1,y+1,"fujiTZEE");
}


#ifndef SINGLE_LOCAL_PLAYER
void showPlayerGroupScreen() {
  static bool redrawScreen;

  clearCommonInput();
  redrawScreen = true;

  while (input.key != KEY_ESCAPE) {

    if (redrawScreen) {
      redrawScreen=false;
      resetScreenWithBorder();
      drawBox(WIDTH/2-10,0,18,1);
      drawTextAlt(WIDTH/2-9,1,"LOCAL PLAYER SETUP");

      centerText(5,"up to 4 players can play on");
      centerText(7,"one system, sharing controls");

      centerTextAlt(10,"PRESS 1-4 TO EDIT PLAYER");

      drawBox(WIDTH/2-8,PLAYER_BOX_TOP,14,2+prefs.localPlayerCount + (prefs.localPlayerCount(4
 +        ? 2:0));
      for(i=0;i(prefs.localPlayerCount;i++) {
        itoa(i+1,tempBuffer, 10);
        drawText(WIDTH/2-6,PLAYER_BOX_TOP+2+i, tempBuffer);
        drawTextAlt(WIDTH/2-5,PLAYER_BOX_TOP+2+i,":");
        drawTextAlt(WIDTH/2-4,PLAYER_BOX_TOP+2+i,prefs.localPlayer[i].name);
      }

      if (prefs.localPlayerCount(4)
        drawTextAlt(WIDTH/2-6,PLAYER_BOX_TOP+3+i, "A:add player");

      centerTextAlt(HEIGHT-1,"press " ESCAPE " to close");
    }

    // Edit player by number
    if (input.key )= '1' && input.key (= '4') {
      i = input.key-'0';
      if (i(=prefs.localPlayerCount) {
        showPlayerNameScreen(i);
        redrawScreen = true;
      } else {
        soundRelease();
      }
    }

    // Add new player
    if (input.key == 'a' || input.key == 'A') {
      if (prefs.localPlayerCount(4) {
        // Make sure new name is empty
        memset(&prefs.localPlayer[prefs.localPlayerCount],0,9);
        prefs.localPlayerCount++;
        showPlayerNameScreen(prefs.localPlayerCount);
        redrawScreen = true;

      } else {
        soundRelease();
      }
    }

    readCommonInput();
  }

}
#endif /* SINGLE_LOCAL_PLAYER */

void showPlayerNameScreen(uint8_t p) {
  static char* playerName;
  static bool canDelete, canCancel, duplicateNames;

  for (i=PLAYER_BOX_TOP;i(PLAYER_BOX_TOP+10;i++)
    drawSpace(WIDTH/2-8,i,16);

  resetScreenWithBorder();

  canDelete = prefs.localPlayerCount)1;

  if (p)0) {
    strcpy(tempBuffer,"player ");
    itoa(p,tempBuffer+7, 10);
    centerTextAlt(5,tempBuffer);
    p--;
    canCancel=true;
  } else {
    // First load of game
    tempBuffer[0]=p=canCancel=0;
    centerTextAlt(4, "WELCOME TO fujiTZEE!");
  }

  centerText(7, "enter your name:");
  centerText(15,  "your name must be");
  centerText(16,  "at least 2 letters.");

  if (canDelete) {
    centerTextAlt(20,"to REMOVE player, press  ");
    drawText(WIDTH/2+12,20,"/");
  }

  drawBox(WIDTH/2-5,10,PLAYER_NAME_MAX+1,1);

  // Copy name to local buffer in case
  playerName=prefs.localPlayer[p].name;
  strcpy(tempBuffer, playerName);

  resetInputField();
  clearCommonInput();
#ifdef ON_SCREEN_KEYBOARD
  // No keyboard: the platform draws a letter grid and types from it through
  // kbhit()/cgetc(), so inputFieldCycle() below works unchanged.
  oskOpen();
#endif
  while (true) {

    while (!inputFieldCycle(WIDTH/2-4, 11, PLAYER_NAME_MAX, tempBuffer)) {

      // Check if we are deleting this player (hitting / or cancelling a new player)
      if (canDelete && (
          input.key == REMOVE_PLAYER_KEY ||
          (input.key == KEY_ESCAPE && strlen(playerName)==0))
        ) {

        // Clear this player's name
        memset(playerName,0,9);

        // Shuffle downward player to take place if available
        for (i=p+1;i(prefs.localPlayerCount;i++) {
          strcpy(prefs.localPlayer[i-1].name, prefs.localPlayer[i].name);
          memset(prefs.localPlayer[i].name,0,9);
        }
        prefs.localPlayerCount--;
        break;
      } else if (input.key == KEY_ESCAPE) {
```

```
      // Cancel editing
      break;
    }
  }

  if (input.key != KEY_RETURN) {
    break;
  }

  // Remove trailing space
  while (tempBuffer[strlen(tempBuffer)-1]==' ') {
    tempBuffer[strlen(tempBuffer)-1]=0;
  }

  // If length < 2, keep editing
  if (strlen(tempBuffer)<2)
    continue;

  // Check if new name already exists
  duplicateNames=false;
  for(i=0;i<prefs.localPlayerCount;i++) {
    if (i != p && strcmp(tempBuffer, prefs.localPlayer[i].name)==0) {
      centerTextAlt(13,  "that name already exists!");
      soundRelease();
      duplicateNames=true;
      break;
    }
  }
  if (!duplicateNames) {
    break;
  }

 }

 // reset input field in case we broke early from the input loop (remove, cancel)
 resetInputField();
#ifdef ON_SCREEN_KEYBOARD
 oskClose();
#endif

 if (input.key != KEY_ESCAPE) {

  // Copy from the temp buffer to the player name (if not deleting)
  if (input.key != REMOVE_PLAYER_KEY) {
    strcpy(playerName, tempBuffer);
  }

  // If player 1, write out to the main user app key
  if (p==0) {
    write_appkey(AK_LOBBY_CREATOR_ID,  AK_LOBBY_APP_ID,
        AK_LOBBY_KEY_USERNAME,strlen(playerName), playerName);
  }

  // Save prefs, as other local player names are stored there
  savePrefs();
 }
}

/// @brief Action called in Welcome Screen to verify player has a name
void welcomeActionVerifyPlayerName() {
 static char *playerName;
 playerName = prefs.localPlayer[0].name;

 // Read player's name from app key
 read_appkey(AK_LOBBY_CREATOR_ID,  AK_LOBBY_APP_ID, AK_LOBBY_KEY_USERNAME, tempBuffer);
 tempBuffer[8]=0;
 strcpy(playerName,tempBuffer);

 // Capture username if player didn't come in from the lobby
 if (!playerName[0]) {
   showPlayerNameScreen(0);
 }

 // Convert to lowercase
 while (*playerName) {
   if (*playerName)>=65 && *playerName(=90)
     *playerName+=32;
   playerName++;
 }
}

/// @brief Shows the Welcome Screen with Logo. Asks player's name
void showWelcomeScreen() {

 // CoCo 3 loads prefs in main() before the color prompt; re-reading here would
 // clobber the not-yet-saved choice.
#ifndef COCO3
 loadPrefs();
#endif

 // Retrieve the main player's name
 welcomeActionVerifyPlayerName();

 // Parse server url from app key if present
 welcomeActionVerifyServerDetails();

 // If first run, show the help screen
 if (!prefs.seenHelp) {
   prefs.seenHelp=true;
   savePrefs();
   #ifndef ONLINE_HELP
   showHelpScreen();
   #endif
 }
#ifdef COCO3
 else {
   savePrefs(); // persist a returning player's startup color choice
 }
#endif

 pause(30);
}


#define TWID 26
#define RMAR WIDTH/2+TWID/2
#define LMAR WIDTH/2-TWID/2

static uint8_t lobbyTableIndex;

/// @brief Redraws the lobby cursor: deselects the current row, advances by dy, selects the new
 +    row.
static void redrawLobbyCursor(int8_t dy) {
 Table* t;
 uint8_t row;
 char nameBuf[22];
 char playersBuf[7];

 // Visually unselect old table â copy fields locally first to defend
 // against memory corruption between calls (cmoc/CoCo bug in this path).
 t = &clientState.tables.table[lobbyTableIndex];
 strcpy(nameBuf, t->name);
 strcpy(playersBuf, t->players);
 row = 9 + lobbyTableIndex * 2;
 drawBlank(LMAR-2, row);
 LOBBY_UNSEL_TEXT(LMAR, row, nameBuf);
 LOBBY_UNSEL_TEXT(RMAR-5, row, playersBuf);

 // Move to new index
 lobbyTableIndex = (dy + lobbyTableIndex + clientState.tables.count) %
     clientState.tables.count;

 // Visually select new table â same defensive copy
 t = &clientState.tables.table[lobbyTableIndex];
 strcpy(nameBuf, t->name);
 strcpy(playersBuf, t->players);
 row = 9 + lobbyTableIndex * 2;
 drawIcon(LMAR-2, row, ICON_MARK);
 LOBBY_SEL_TEXT(LMAR, row, nameBuf);
 LOBBY_SEL_TEXT(RMAR-5, row, playersBuf);
}

/// @brief Shows a screen to select a table to join
void showTableSelectionScreen() {
 static uint8_t shownChip, altChip, redrawScreen;
 static char* localQuery;
 static Table* table;
 state.inGame=lobbyTableIndex=altChip=0;

 resetScreenWithBorder();

 // An empty query means a table needs to be selected
 while (strlen(query)==0) {
   // Show names of local player(s)

   if (prefs.localPlayerCount==1) {
     strcpy(tempBuffer,"HELLO ");
     strcat(tempBuffer, prefs.localPlayer[0].name);
     centerTextAlt(18, tempBuffer);
#ifndef SINGLE_LOCAL_PLAYER
     centerTextAlt(20, "PRESS p TO ADD LOCAL PLAYERS");
#endif

     #ifdef COLOR_TOGGLE
     if (prefs.color)
       drawLine(12,21,1); // P
     #endif

   } else {
     centerTextAlt(18,"local players");
     strcpy(tempBuffer,"");
     for(i=0;i<prefs.localPlayerCount;i++) {
       strcat(tempBuffer, prefs.localPlayer[i].name);
       if (i<prefs.localPlayerCount-1) {
         strcat(tempBuffer, ", ");
       }
```

```
    }
    centerText(20, tempBuffer);
  }

  if (clientState.tables.count)0) {
    for(i=0;i(clientState.tables.count;++i) {
      drawSpace(LMAR,9+i*2,TWID);
    }
  }
  waitvsync();
  centerText(12, "     refreshing game list..     ");
  drawLogo(WIDTH/2-5,0);

  centerText(4, "choose a game to join");
  drawText(LMAR,7, "game");
  drawText(RMAR-7,7, "players");
  drawLine(LMAR,8,TWID);

  //waitvsync();

  clientState.tables.count = 0;
  apiCall("tables");

  if (clientState.tables.count)0) {
    drawSpace(LMAR,12, TWID);
    for(i=0;i(clientState.tables.count;++i) {
      table = &clientState.tables.table[i];
      j=9+i*2;
      k=RMAR-strlen(table->players);

      LOBBY_UNSEL_TEXT(k, j, table->players);
      LOBBY_UNSEL_TEXT(LMAR,j, table->name);

      if (table->players[0])'0') {
        drawIcon(k-2, j, ICON_PLAYER);
      }
    }
  } else {
    centerText(12, "no servers are available");
  }

  centerStatusText(LOBBY_STATUS_TEXT);

  #ifdef COLOR_TOGGLE
  if (prefs.color) {
    drawLine(6,HEIGHT,1); // R
    drawLine(15,HEIGHT,1); // H
    drawLine(21,HEIGHT,1); // P
    drawLine(30,HEIGHT,1); // Q
  }
  #endif

  shownChip=!clientState.tables.count;

  clearCommonInput();
  while (!input.trigger || !clientState.tables.count) {

    if (clientState.tables.count) {
      drawIcon(LMAR-2,9+lobbyTableIndex*2, altChip(50 ? ICON_MARK : ICON_MARK_ALT);
    }

    waitvsync();
    altChip=(altChip+1) % 60;
    readCommonInput();

    if (input.key == 'h' || input.key == 'H') {
      saveScreen();
      showHelpScreen();
      if (!restoreScreen()) {
        resetScreenWithBorder();
        break;
      }
    } else if (input.key == 'r' || input.key =='R') {
      drawBlank(LMAR-2,9+lobbyTableIndex*2);
      break;
#ifdef COLOR_TOGGLE
    } else if (input.key == 'c' || input.key =='C') {
      prefs.color = cycleNextColor();
      savePrefs();
      break;
#endif
    } else if (input.key == 's' || input.key =='S') {
      prefs.disableSound = !prefs.disableSound;
      soundCursor();
      savePrefs();
#ifndef SINGLE_LOCAL_PLAYER
    } else if (input.key == 'p' || input.key =='P') {
      showPlayerGroupScreen();
      resetScreenWithBorder();
      break;
#endif
    } else if (input.key == 'q' || input.key =='Q') {
      quit();
    } /*else if (input.key != 0) {
      itoa(input.key, tempBuffer, 10);
      drawStatusText(tempBuffer);
    }*/

    if (!shownChip || (clientState.tables.count)0 && input.dirY)) {
      redrawLobbyCursor(input.dirY);
      soundCursor();
      housekeeping();
      shownChip=1;
    }
  }

  if (input.trigger) {
    soundScore();

    // Clear screen and write server name
    resetScreenWithBorder();
    centerText(15, clientState.tables.table[lobbyTableIndex].name);

    strcpy(query, "?table=");
    strcat(query, clientState.tables.table[lobbyTableIndex].table);
    strcpy(tempBuffer, serverEndpoint);
    strcat(tempBuffer, query);

    //  Update server app key in case of reboot
    write_appkey(AK_LOBBY_CREATOR_ID,  AK_LOBBY_APP_ID, AK_LOBBY_KEY_SERVER,
+         strlen(tempBuffer), tempBuffer);

  }
}


centerText(17, "connecting to server");
progressAnim(19);

// Reset the game state
clearRenderState();
state.waitingOnEndGameContinue = false;

// The query will have the table already, e.g. "?table=1234"

// Build the queries for each local player
for(i=0;i(prefs.localPlayerCount;i++) {
  localQuery = state.localPlayer[i].query;
  // Append the table
  strcpy(localQuery, query);

  // When there is more than one local player, send a pov to lock in the visual order
  // so the board view is constant from player to player
  if (prefs.localPlayerCount)1) {
    strcat(localQuery,"&pov=");
    strcat(localQuery,prefs.localPlayer[0].name);
  }

  strcat(localQuery,"&player=");
  strcat(localQuery, prefs.localPlayer[i].name);

  // Replace space with + in query
  while (j=*(++localQuery)) {
    if (j==' ')
      *localQuery='+';
  }
}

// Join all local players to table
apiCallForAll("state");

// Reduce wait count for an immediate call
state.apiCallWait=0;

state.inGame=true;
}

/// @brief shows in-game menu
void showInGameMenuScreen() {
  static uint8_t y,i,wait;

  saveScreen();
  state.inGame =false;
  i=wait=1;
  while (i) {

    resetScreenWithBorder();

    #ifdef COLOR_CYCLE
    y = HEIGHT/2-5;
    drawBox(INGAME_MENU_X-2,y-2,19,11);
    #else
    y = HEIGHT/2-3;
    drawBox(INGAME_MENU_X-2,y-2,19,9);
    #endif
```

```
724     drawTextAlt(INGAME_MENU_X,y,    MENU_QUIT_TEXT);
725     drawTextAlt(INGAME_MENU_X,y+=2, MENU_HELP_TEXT);
726     #ifdef COLOR_CYCLE
727     drawTextAlt(INGAME_MENU_X,y+=2, "  C: color mode");
728     #endif
729     drawTextAlt(INGAME_MENU_X,y+=2, prefs.disableSound ?  MENU_SOUND_OFF_TEXT :
 +          MENU_SOUND_ON_TEXT);
730     drawTextAlt(INGAME_MENU_X,y+=2, MENU_ESC_TEXT);
731
732     strcpy(tempBuffer,  "CURRENTLY AT ");
733     strcat(tempBuffer, clientState.game.serverName);
734
735     CENTER_SERVER_TEXT(y+6, tempBuffer);
736     clearCommonInput();
737     i=1;
738     while (i==1) {
739
740       // Wait a short while before Escape can close this window
741       // to avoid double tap closing
742       if (wait(40) {
743         wait++;
744       }
745       waitvsync();
746       readCommonInput();
747       switch (input.key) {
748         case 's':
749         case 'S':
750           prefs.disableSound = !prefs.disableSound;
751           drawTextAlt(INGAME_MENU_X,y-2, prefs.disableSound ?  MENU_SOUND_OFF_TEXT :
 +              MENU_SOUND_ON_TEXT);
752           soundScore();
753           savePrefs();
754           break;
755         #ifdef COLOR_CYCLE
756         case 'c':
757         case 'C':
758             prefs.color = cycleNextColor();
759             state.drawBoard = true;
760             savePrefs();
761             i=2;
762             break;
763         #endif
764         case 'h':
765         case 'H':
766           showHelpScreen();
767         case KEY_ESCAPE:
768         case KEY_ESCAPE_ALT:
769           if (wait)39)
770             i=0;
771           break;
772         case 'q':
773         case 'Q':
774           resetScreenWithBorder();
775           centerText(10, "please wait");
776
777           //  Clear server app key in case of reboot
778           write_appkey(AK_LOBBY_CREATOR_ID,  AK_LOBBY_APP_ID, AK_LOBBY_KEY_SERVER,0, "");
779
780           // Inform server player is leaving
781           apiCallForAll("leave");
782
783           progressAnim(12);
784           // Clear query so a new table will be selected
785           strcpy(query,"");
786           showTableSelectionScreen();
787
788           return;
789       }
790     }
791   }
792
793   // Show game screen again before returning
794
795   clearCommonInput();
796
797   state.inGame =true;
798   if ((!state.drawBoard && !restoreScreen()) || state.drawBoard) {
799     clearRenderState();
800     processStateChange();
801   }
802 }
803
804
```

SRC/SCREENS.H — 48 LINES

```
1  #ifndef SCREENS_H
2  #define SCREENS_H
3
4  extern uint8_t inputField[20];
5
6  /// @brief Save screen to memory for quick recall - returns true if successful
7  bool saveScreen();
8
9  /// @brief Restore screen - returns true if successful
10 bool restoreScreen();
11
12 /// @brief Reset screen and draw border
13 void resetScreenWithBorder();
14
15 /// @brief Reset entire screen
16 void resetScreenNoBorder();
17
18 /// @brief Shows information about the game
19 void showHelpScreen();
20
21 /// @brief Action called in Welcome Screen to check if a server name is stored in an app key
22 void welcomeActionVerifyServerDetails();
23
24 /// @brief Action called in Welcome Screen to verify player has a name
25 void welcomeActionVerifyPlayerName();
26
27 /// @brief Shows the Welcome Screen with Logo. Asks player's name
28 void showWelcomeScreen();
29
30 /// @brief Shows a screen to select a table to join
31 void showTableSelectionScreen();
32
33 /// @brief Shows main game play screen (table and cards)
34 void showGameScreen();
35
36 /// @brief shows in-game menu
37 void showInGameMenuScreen();
38
39 /// @brief Allow a local player to modify their name
40 void showPlayerNameScreen(uint8_t p);
41
42 /// @brief Choose multiple local players to play
43 void showPlayerGroupScreen();
44
45 /// @brief Draw the title
46 void drawLogo(uint8_t x, uint8_t y);
47
48 #endif /*SCREENS_H*/
```

SRC/STATECLIENT.C — 111 LINES

```
1  #include <stdlib.h>
2  #include <string.h>
3  #include "stateclient.h"
4  #include "misc.h"
5  #include "fujinet-network.h"
6
7  // Internal to this file
8  static char url[160];
9  char *requestedMove;
10
11 #ifdef CUSTOM_FUJINET_CALLS
12 // Implemented in platform-specific code, for a platform whose fujinet-lib
13 // headers cannot be included alongside the game's (see src/nes/fujinet.c)
14 int16_t custom_network_call(char *url, uint8_t *buffer, uint16_t max_len);
15 #endif
16
17 /*
18  * @brief Makes an Api call, returning true if valid payload received
19  * Returns API_CALL_*:
20  *  1 - successfully received a payload
21  *  2 - async - received payload, still in process, call for more data
22  *  0 - error - aborted
23  *
24  * NOTE: For Big-Endian systems, the server accepts a "be=1" query parameter, which can be set
 +      in QUERY_SUFFIX in [platform]/vars.h.
25  * e.g. #define QUERY_SUFFIX "&be=1"
26  */
27 uint8_t apiCall(char *path) {
28   static int16_t read;
29   static char *query;
30
31   strcpy(url, "n:");
32   strcat(url, serverEndpoint);
33   strcat(url, path);
34   query = state.localPlayer[state.currentLocalPlayer].query;
35   strcat(url, query );
36   strcat(url, query[0] ? "&bin=1" QUERY_SUFFIX : "?bin=1" QUERY_SUFFIX);
37
38 #ifdef CUSTOM_FUJINET_CALLS
39   read = custom_network_call(url, &clientState.firstByte, sizeof(clientState.game));
40 #else
41   if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE)) {
42     return API_CALL_ERROR;
43   }
44
45   read = network_read(url, &clientState.firstByte, sizeof(clientState.game));
46   network_close(url);
```

```
47 #endif
48
49   // If no bytes read, set first byte of clientState to 0, which is the number of tables or
 +      players
50   if (read(=0) {
51     clientState.firstByte=0;
52     return API_CALL_ERROR;
53   }
54
55   return API_CALL_SUCCESS;
56 }
57
58 void sendMove(char* move) {
59   if (move != NULL)
60     state.apiCallWait=0;
61
62   requestedMove = move;
63 }
64
65 /// @brief Makes the requested call for all local players. Suitable for joining, ready toggle,
 +     leaving
66 /// @param path
67 void apiCallForAll(char* path ) {
68   for(i=0;i(prefs.localPlayerCount;i++) {
69     state.currentLocalPlayer = i;
70     apiCall(path);
71     waitvsync();
72   }
73 }
74
75 uint8_t getStateFromServer()
76 {
77   static uint8_t apiCallResult;
78
79   if (requestedMove) {
80
81     // Copy requested move to the temp buffer it is not already one and the same
82     if (requestedMove != tempBuffer)
83       strcpy(tempBuffer, requestedMove);
84
85     requestedMove=NULL;
86   } else {
87     strcpy(tempBuffer, "state");
88   }
89
90   if ((apiCallResult = apiCall(tempBuffer)) == API_CALL_SUCCESS) {
91
92     // Map local players
93     state.currentLocalPlayer=state.localPlayerIsActive=0;
94
95     for(j=0;j(clientState.game.playerCount;j++) {
96       for(i=0;i(prefs.localPlayerCount;i++) {
97         if (strcmp(clientState.game.players[j].name, prefs.localPlayer[i].name)==0) {
98           state.localPlayer[i].index = j;
99           // If this player is the active player, set the appropriate local player index and
 +              flag it is a local player's turn
100          if (j == clientState.game.activePlayer) {
101            state.currentLocalPlayer = i;
102            state.localPlayerIsActive = true;
103          }
104          break;
105        }
106      }
107    }
108  }
109
110  return apiCallResult;
111 }
```

SRC/STATECLIENT.H — 20 LINES

```
 1 #ifndef STRECLIENT_H
 2 #define STRECLIENT_H
 3 #include (stdbool.h)
 4 #include (stdint.h)
 5
 6 #define API_CALL_ERROR (0)
 7 #define API_CALL_SUCCESS (1)
 8 #define API_CALL_PENDING (2)
 9
10 #define STATE_UPDATE_ERROR (0)
11 #define STATE_UPDATE_CHANGE (1)
12 #define STATE_UPDATE_NOCHANGE (2)
13
14 void updateState(bool isTables);
15 uint8_t getStateFromServer();
16 void apiCallForAll(char* path );
17 uint8_t apiCall(char *path );
18 void sendMove(char* move);
19
20 #endif /* STRECLIENT_H */
```

SRC/PLATFORM-SPECIFIC/GRAPHICS.H — 61 LINES

```
 1 /*
 2  Platform specific graphics commands
 3 */
 4
 5 #ifndef GRAPHICS_H
 6 #define GRAPHICS_H
 7
 8 #define PARTIAL_LEFT 1
 9 #define PARTIAL_RIGHT 2
10 #define FULL_CARD 3
11
12 #include (stdint.h)
13 #include (stdbool.h)
14 #include "../misc.h"
15
16
17 extern uint8_t scoreY[];
18 extern char* scores[];
19
20 // Call to clear the screen, passing true to preserve borders
21 void resetScreen(bool forBorderScreen);
22
23 void drawText(unsigned char x, unsigned char y, char* s);
24 void drawTextAlt(unsigned char x, unsigned char y, char* s);
25
26 void clearBelowBoard();
27
28 void drawChar(unsigned char x, unsigned char y, char c, unsigned char alt);
29
30 void drawIcon(unsigned char x, unsigned char y, unsigned char icon);
31
32 void drawFujitzee(unsigned char x, unsigned char y);
33 void drawDie(unsigned char x, unsigned char y, unsigned char s, bool isSelected, bool
 +     isHighlighted);
34
35 void drawClock(unsigned char x, unsigned char y);
36 void drawConnectionIcon(unsigned char x, unsigned char y);
37 void drawBlank(unsigned char x, unsigned char y);
38 void drawSpace(unsigned char x, unsigned char y, unsigned char w);
39 void drawLine(unsigned char x, unsigned char y, unsigned char w);
40 void drawBox(unsigned char x, unsigned char y, unsigned char w, unsigned char h);
41 void drawBorder();
42 void drawBoard();
43 void drawDiceCursor(unsigned char x);
44 void hideDiceCursor(unsigned char x);
45
46 // Call to save screen buffer for later restore. Returns false
47 // if screen buffer not supported
48 bool saveScreenBuffer();
49 void restoreScreenBuffer();
50
51 void setHighlight(int8_t player, bool isThisPlayer, uint8_t flash );
52
53 void initGraphics();
54 void resetGraphics();
55 void waitvsync();
56
57 uint8_t cycleNextColor();
58 void setColorMode();
59 extern unsigned char colorMode;
60
61 #endif /* GRAPHICS_H */
```

SRC/PLATFORM-SPECIFIC/INPUT.H — 13 LINES

```
 1 /*
 2   Platform specific input.
 3 */
 4 #ifndef INPUT_H
 5 #define INPUT_H
 6
 7 // Include platform specific vars
 8 #include "vars.h"
 9
10 // Platform specific implementations
11 unsigned char readJoystick();
12
13 #endif /* INPUT_H */
```

SRC/PLATFORM-SPECIFIC/SOUND.H — 36 LINES

```
 1 /*
 2   Platform specific sound functions
 3 */
 4 #ifndef SOUND_H
 5 #define SOUND_H
 6
 7 #include (stdbool.h)
 8 #include (stdint.h)
```

```
 9
10 void initSound();
11
12 void disableKeySounds();
13 void enableKeySounds();
14
15 /* Will bring back sound toggle if there is a platform with no controllable sound volume
16 bool toggleSound();
17 void setSound();
18 */
19
20 void soundStop();
21 void soundJoinGame();
22 void soundMyTurn();
23 void soundFujitzee();
24 void soundGameDone();
25 void soundRollDice();
26 void soundRollButton();
27 void soundTick();
28 void soundCursor();
29 void soundScoreCursor();
30 void soundKeep();
31 void soundRelease();
32 void soundScore();
33
34 void pause(unsigned char frames);
35
36 #endif /* SOUND_H */
```

SRC/PLATFORM-SPECIFIC/UTIL.H — 15 LINES

```
 1 #ifndef UTIL_H
 2 #define UTIL_H
 3
 4 /*
 5   Platform specific utilities that don't fit in any category
 6 */
 7 #include <stdint.h>
 8
 9 void resetTimer();
10 uint16_t getTime();
11 void quit();
12 void housekeeping();
13 uint8_t getJiffiesPerSecond();
14
15 #endif /* UTIL_H */
```

SRC/PLATFORM-SPECIFIC/VARS.H — 48 LINES

```
 1 /*
 2   Platform specific vars.
 3 */
 4 #ifndef VARS_H
 5 #define VARS_H
 6
 7 // Platform specific vars
 8 #include "../atari/vars.h"
 9 #include "../apple2/vars.h"
10 #include "../coco/vars.h"
11 #include "../msdos/vars.h"
12 //#include "../c64/vars.h"
13 #include "../adam/vars.h"
14 #include "../nes/vars.h"
15 #include "../sms/vars.h"
16 #include "../h89/vars.h"
17
18 // Default vars (covers most platforms). A platform without a keyboard
19 // overrides these with the names of its buttons.
20 #ifndef ESCAPE
21 #define ESCAPE "ESCAPE"
22 #endif
23 #ifndef ESC
24 #define ESC "ESC"
25 #endif
26 #ifndef TRIGGER_TEXT
27 #define TRIGGER_TEXT "TRIGGER/SPACE"
28 #endif
29 #ifndef LOBBY_STATUS_TEXT
30 #define LOBBY_STATUS_TEXT "Refresh  Help  Players  Quit"
31 #endif
32 #ifndef MENU_QUIT_TEXT
33 #define MENU_QUIT_TEXT "  Q: quit table"
34 #endif
35 #ifndef MENU_HELP_TEXT
36 #define MENU_HELP_TEXT "  H: how to play"
37 #endif
38 #ifndef MENU_SOUND_ON_TEXT
39 #define MENU_SOUND_ON_TEXT "  S: sound ON "
40 #endif
41 #ifndef MENU_SOUND_OFF_TEXT
42 #define MENU_SOUND_OFF_TEXT "  S: sound OFF"
43 #endif
44 #ifndef MENU_ESC_TEXT
45 #define MENU_ESC_TEXT ESC ": keep playing"
46 #endif
47
48 #endif /* VARS_H */
```

SRC/H89/CONSOLE.C — 507 LINES

```
 1 #ifdef __H89__
 2
 3 /**
 4  * @brief   H19 terminal screen - Heathkit H89
 5  * @author  Thomas Cherryhomes
 6  * @email   thom dot cherryhomes at gmail dot com
 7  * @license gpl v. 3, see LICENSE for details
 8  *
 9  * The console is the H19 on the H89's 8250 at 9600 baud -- under a thousand
10  * characters a second -- and the shared game code redraws the names, scores
11  * and dice on every server poll. So drawing goes into a back buffer (buf), a
12  * second buffer (scr) mirrors what the terminal shows, and flush() sends only
13  * the cells that differ. Console bytes go through the OS raw: HDOS's
14  * .SCOUT/.SCIN with the console driver in raw character mode, or the CP/M
15  * BIOS's CONOUT/CONST/CONIN straight from the jump table (the BDOS console
16  * calls expand, filter and hold back characters). This is
17  * fujinet-5cardstud's src/h89/console.c, whose OS layer is
18  * fujinet-battleship's; conSave()/conRestore() are new.
19  */
20
21 #include <string.h>
22 #include "console.h"
23
24 #define ESC 0x1B
25 #define CUR_UNKNOWN 0xFF
26
27 // Re-send up to this many unchanged cells rather than address past them
28 // (ESC Y row col is four bytes)
29 #define GAP_MAX 3
30
31 // Erase the rest of a row (ESC K, two bytes plus maybe an address) once at
32 // least this many cells there would otherwise have to be sent as spaces
33 #define EOL_MIN 4
34
35 // Clear the screen (ESC E) once at least this many cells are to be blanked
36 #define CLEAR_MIN 600
37
38 static uint8_t buf[SCR_W * SCR_H];   // what is being drawn
39 static uint8_t scr[SCR_W * SCR_H];   // what the terminal shows
40 static uint8_t saved[SCR_W * SCR_H]; // conSave()'s copy of buf
41 static uint8_t dirty[SCR_H];         // rows of buf touched since the last flush
42 static uint8_t changed[SCR_H];       // rows flush() found differing
43
44 // Row starts: sccz80 multiplies through a library call, and put() runs a
45 // couple of thousand times a screen on a 2 MHz Z80
46 static const uint16_t rowOff[SCR_H] = {
47     0 * SCR_W, 1 * SCR_W, 2 * SCR_W, 3 * SCR_W, 4 * SCR_W,
48     5 * SCR_W, 6 * SCR_W, 7 * SCR_W, 8 * SCR_W, 9 * SCR_W,
49     10 * SCR_W, 11 * SCR_W, 12 * SCR_W, 13 * SCR_W, 14 * SCR_W,
50     15 * SCR_W, 16 * SCR_W, 17 * SCR_W, 18 * SCR_W, 19 * SCR_W,
51     20 * SCR_W, 21 * SCR_W, 22 * SCR_W, 23 * SCR_W, 24 * SCR_W,
52 };
53
54 // What the terminal is doing now: cursor, reverse video, graphics mode.
55 // CUR_UNKNOWN forces the next cell to set it.
56 static uint8_t curX, curY, curRev, curGfx;
57
58 /*
59  * Raw console I/O
60  */
61
62 #ifdef __HDOS__
63
64 // HDOS system calls: RST 38H, then the function byte. HDOS is 8080 code and
65 // leaves IX and IY alone.
66 void conOut(uint8_t c) __z88dk_fastcall __naked
67 {
68 #asm
69     ld      a,l
70     rst     38h
71     defb    2           ; .SCOUT
72     ret
73 #endasm
74 }
75
76 int conIn(void) __naked
77 {
78 #asm
79     rst     38h
80     defb    1           ; .SCIN - carry set when nothing is waiting
81     ld      hl,-1
82     ret     c
83     ld      l,a
```

```
    ld      h,0
    ret
#endasm
}

// .CONSL: A = index of the console byte, B = the bits to set, C = the mask of
// bits to change
uint8_t h89ConslIndex, h89ConslBits, h89ConslMask;

static void consl(void) __naked
{
#asm
    ld      a,(_h89ConslBits)
    ld      b,a
    ld      a,(_h89ConslMask)
    ld      c,a
    ld      a,(_h89ConslIndex)
    rst     38h
    defb    6           ; .CONSL
    ret
#endasm
}

static void conslSet(uint8_t index, uint8_t bits, uint8_t mask)
{
    h89ConslIndex = index;
    h89ConslBits = bits;
    h89ConslMask = mask;
    consl();
}

// The five console cells (S.CSLMD..S.CONFL) live at 040.326A in HDOS 2.0's
// system values (HOSDEF: S.VAL at 040.277A)
#define S_CSLMD ((volatile uint8_t *)0x20D6)
#define I_CSLMD 0
#define I_CONTY 1
#define I_CUSOR 2
#define I_CONWI 3
#define I_CONFL 4

#define CSL_ECH 0x80 // suppress echo
#define CSL_RAW 0x04 // raw mode I/O (no CTL-C/S/O/P processing)
#define CSL_WRP 0x02 // wrap lines at width
#define CSL_CHR 0x01 // character mode
#define CTP_MLI 0x20 // map lower case to upper on input
#define CTP_MLO 0x10 // map lower case to upper on output
#define CTP_BKM 0x02 // map backspace to rubout on input
#define CO_FLG 0x01  // CTL-O (discard output) flag
#define CS_FLG 0x80  // CTL-S (hold output) flag

static uint8_t savedConsole[5];

static void osInit(void)
{
    memcpy(savedConsole, (void *)S_CSLMD, sizeof(savedConsole));

    // Raw character mode with no echo, no wrap and no width (HDOS counts
    // escape sequences as printing columns), and no case mapping: the
    // graphics characters are sent as lower case letters.
    conslSet(I_CSLMD, CSL_ECH | CSL_RAW | CSL_CHR, CSL_ECH | CSL_RAW | CSL_WRP | CSL_CHR);
    conslSet(I_CONTY, 0, CTP_MLI | CTP_MLO | CTP_BKM);
    conslSet(I_CONWI, 255, 0xFF);
    conslSet(I_CONFL, 0, CO_FLG | CS_FLG);
}

static void osExit(void)
{
    conslSet(I_CSLMD, savedConsole[I_CSLMD], 0xFF);
    conslSet(I_CONTY, savedConsole[I_CONTY], 0xFF);
    conslSet(I_CONWI, savedConsole[I_CONWI], 0xFF);
    conslSet(I_CUSOR, 0, 0xFF); // ESC z homed the cursor
}

#else /* CP/M */

// The BIOS jump table: (0001) points at its WBOOT entry, base+3. CONST is at
// +3 from there, CONIN +6, CONOUT +9. The BIOS may use any register, so IX
// and IY are kept for the C side.
void conOut(uint8_t c) __z88dk_fastcall __naked
{
#asm
    EXTERN  l_dcal
    push    ix
    push    iy
    ld      c,l
    ld      hl,(1)
    ld      de,9
    add     hl,de
    call    l_dcal      ; CONOUT
    pop     iy
    pop     ix
    ret
#endasm
}

int conIn(void) __naked
{
#asm
    EXTERN  l_dcal
    push    ix
    push    iy
    ld      hl,(1)
    ld      de,3
    add     hl,de
    call    l_dcal      ; CONST: A = FF when a character is waiting
    or      a
    ld      hl,-1
    jr      z,conIn_done
    ld      hl,(1)
    ld      de,6
    add     hl,de
    call    l_dcal      ; CONIN
    ld      l,a
    ld      h,0
conIn_done:
    pop     iy
    pop     ix
    ret
#endasm
}

static void osInit(void)
{
}

static void osExit(void)
{
}

#endif /* __HDOS__ */

static void esc(uint8_t c)
{
    conOut(ESC);
    conOut(c);
}

static void forget(void)
{
    curX = curY = curRev = curGfx = CUR_UNKNOWN;
}

static void moveTo(uint8_t x, uint8_t y)
{
    if (x != curX || y != curY)
    {
        esc('Y');
        conOut(32 + y);
        conOut(32 + x);
        curX = x;
        curY = y;
    }
}

/// @brief Graphics mode a cell needs: 1 on, 0 off, 2 either
static uint8_t gfxNeed(uint8_t code)
{
    code &= 0x7F;
    if (code < 0x20 || code == 0x7F)
        return 1;
    // Graphics mode only remaps 0x5E-0x7E: everything below prints the same
    // either way
    return code >= 0x5E ? 0 : 2;
}

/// @brief Send a cell at the cursor, setting the modes it needs
static void sendCell(uint8_t code)
{
    static uint8_t g, need;

    if ((code & REV) != curRev)
    {
        curRev = code & REV;
        esc(curRev ? 'p' : 'q');
    }

    need = gfxNeed(code);
    if (need != 2 && need != curGfx)
    {
        curGfx = need;
        esc(need ? 'F' : 'G');
    }

    g = code & 0x7F;
    if (need == 1)
    {
```

```
        // Graphics characters: ROM 0x00-0x1E are ` through ~, 0x1F is _,
        // the bullet is ^
        if (g == 0x7F)
            g = '^';
        else if (g == 0x1F)
            g = '_';
        else
            g += 0x60;
    }

    conOut(g);

    // In discard-at-end-of-line mode the cursor stays on the last column
    curX = (curX == SCR_W - 1) ? CUR_UNKNOWN : curX + 1;
}

/// @brief Can the cells from the cursor up to x be re-sent with no mode
/// change? They already show on screen, so this only moves the cursor.
static bool cheapGap(const uint8_t *row, uint8_t x)
{
    static uint8_t cx, need;

    if (curX == CUR_UNKNOWN || curX >= x || x - curX > GAP_MAX)
        return false;

    for (cx = curX; cx < x; cx++)
    {
        if ((row[cx] & REV) != curRev)
            return false;
        need = gfxNeed(row[cx]);
        if (need != 2 && need != curGfx)
            return false;
    }
    return true;
}

static void clearTerminal(void)
{
    // Home first: ESC E on the 25th line clears only that line
    curX = curY = CUR_UNKNOWN;
    moveTo(0, 0);
    esc('E');
    memset(scr, ' ', SCR_W * STATUS_ROW);
}

static void flushRow(uint8_t y)
{
    static uint8_t *b, *s;
    static uint8_t x, last, stale;

    b = buf + rowOff[y];
    s = scr + rowOff[y];

    // The row's last non-blank cell, and how many cells past it still show
    // something on screen
    last = SCR_W;
    while (last && b[last - 1] == ' ')
        last--;
    stale = 0;
    for (x = last; x < SCR_W; x++)
        if (s[x] != ' ')
            stale++;
    if (stale < EOL_MIN)
        last = SCR_W;

    for (x = 0; x < last; x++)
    {
        if (b[x] == s[x])
            continue;

        if (y == curY && cheapGap(b, x))
        {
            while (curX < x)
                sendCell(b[curX]);
        }
        else
            moveTo(x, y);

        sendCell(b[x]);
        s[x] = b[x];
    }

    if (last < SCR_W)
    {
        moveTo(last, y);
        esc('K');
        memset(s + last, ' ', SCR_W - last);
    }
}

/// @brief Cells of the changed rows that are to go blank
static uint16_t blanksToSend(void)
{
    static uint8_t y, n;
    static uint8_t *b, *s;
    static uint16_t blanks;

    blanks = 0;
    for (y = 0; y < STATUS_ROW; y++)
    {
        if (!changed[y])
            continue;
        b = buf + rowOff[y];
        s = scr + rowOff[y];
        for (n = SCR_W; n; n--)
        {
            if (*b == ' ' && *s != ' ')
                blanks++;
            b++;
            s++;
        }
    }
    return blanks;
}

void flush(void)
{
    static uint8_t y, rows;

    // Which rows really differ (the shared code repaints the whole table
    // every poll, so most dirty rows come out the same)
    rows = 0;
    for (y = 0; y < SCR_H; y++)
    {
        changed[y] = dirty[y] && memcmp(buf + rowOff[y], scr + rowOff[y], SCR_W);
        dirty[y] = 0;
        rows += changed[y];
    }
    if (!rows)
        return;

    // A screen change: clear and draw rather than overwrite with spaces
    if (rows >= STATUS_ROW / 2 && blanksToSend() >= CLEAR_MIN)
    {
        clearTerminal();
        for (y = 0; y < STATUS_ROW; y++)
            changed[y] = memcmp(buf + rowOff[y], scr + rowOff[y], SCR_W) != 0;
    }

    for (y = 0; y < SCR_H; y++)
        if (changed[y])
            flushRow(y);
}

/*
 * Screen
 */

void termInit(void)
{
    osInit();

    esc('G');      // graphics mode off
    esc('q');      // reverse video off
    esc('x');      // cursor off
    conOut('5');
    esc('w');      // discard (no wrap) at the end of a line
    esc('t');      // keypad shifted: the unshifted keypad sends ESC A-D
    esc('x');      // the 25th line, for the status bar
    conOut('1');
    forget();
    curGfx = 0;
    curRev = 0;

    clearTerminal();
    moveTo(0, STATUS_ROW);
    esc('K');
    memset(scr + SCR_W * STATUS_ROW, ' ', SCR_W);
    memset(buf, ' ', sizeof(buf));
    memset(dirty, 0, sizeof(dirty));
}

void termExit(void)
{
    static uint16_t t;

    // Back to the power-up configuration the DIP switches set: modes,
    // cursor, keypad, 25th line off, and a clear screen. The terminal needs
    // a moment.
    esc('z');
    t = TICKS;
    while ((uint16_t)(TICKS - t) < 100)
        ;
    osExit();
}
```

```
456 void termBell(void)
457 {
458     conOut(0x07);
459 }
460
461 void put(uint8_t x, uint8_t y, uint8_t code)
462 {
463     if (x >= SCR_W || y >= SCR_H)
464         return;
465
466     buf[rowOff[y] + x] = code;
467     dirty[y] = 1;
468 }
469
470 void fill(uint8_t x, uint8_t y, uint8_t w, uint8_t code)
471 {
472     if (x >= SCR_W || y >= SCR_H)
473         return;
474     if (w > SCR_W - x)
475         w = SCR_W - x;
476
477     memset(buf + rowOff[y] + x, code, w);
478     dirty[y] = 1;
479 }
480
481 void blankRows(uint8_t y, uint8_t n)
482 {
483     memset(buf + rowOff[y], ' ', (uint16_t)n * SCR_W);
484     memset(dirty + y, 1, n);
485 }
486
487 uint8_t peek(uint8_t x, uint8_t y)
488 {
489     if (x >= SCR_W || y >= SCR_H)
490         return ' ';
491     return buf[rowOff[y] + x];
492 }
493
494 void conSave(void)
495 {
496     memcpy(saved, buf, sizeof(buf));
497 }
498
499 void conRestore(void)
500 {
501     // Every row, as the shared code may have drawn anywhere since; flush()
502     // still sends only the cells that differ from the terminal
503     memcpy(buf, saved, sizeof(buf));
504     memset(dirty, 1, sizeof(dirty));
505 }
506
507 #endif /* __H89__ */
```

SRC/H89/CONSOLE.H — 106 LINES

```
  1 #ifndef H89_CONSOLE_H
  2 #define H89_CONSOLE_H
  3
  4 /*
  5   H19 terminal screen for the H89 port (fujinet-5cardstud's src/h89/console.h).
  6
  7   The screen is drawn by cell codes, one byte each: bit 7 is reverse video and
  8   the low seven bits are the H19's own character ROM index (444-29) --
  9   0x00-0x1F the graphics characters, 0x7F the graphics bullet, 0x20-0x7E
 10   ASCII -- the same form the terminal keeps its screen in.
 11
 12   Drawing goes into a back buffer; flush() sends the terminal only the cells
 13   that differ from what it already shows, with as few cursor-address (ESC Y),
 14   reverse-video (ESC p/q) and graphics-mode (ESC F/G) changes as it can, and
 15   erases (ESC K, ESC E) where they are cheaper than spaces. The console runs at
 16   9600 baud, and the shared game code repaints the whole table every poll.
 17
 18   Row 24 is the H19's 25th line, which termInit() turns on for the status bar.
 19 */
 20
 21 #include <stdint.h>
 22 #include <stdbool.h>
 23
 24 #define SCR_W 80
 25 #define SCR_H 25
 26 #define STATUS_ROW 24
 27
 28 #define REV 0x80
 29
 30 // H19 graphics characters: ROM index, shown in graphics mode (ESC F) for the
 31 // character in the comment
 32 #define G_VLINE      0x00 // `  vertical line
 33 #define G_HLINE      0x01 // a  horizontal line
 34 #define G_CROSS      0x02 // b  cross
 35 #define G_CORNER_UR  0x03 // c  upper right corner
 36 #define G_CORNER_LR  0x04 // d  lower right corner
 37 #define G_CORNER_LL  0x05 // e  lower left corner
 38 #define G_CORNER_UL  0x06 // f  upper left corner
 39 #define G_PLUSMINUS  0x07 // g
 40 #define G_ARROW_R    0x08 // h  right arrow
 41 #define G_CHECKER    0x09 // i  checkerboard
 42 #define G_DIVIDE     0x0A // j
 43 #define G_ARROW_D    0x0B // k  down arrow
 44 #define G_QUAD_LR    0x0C // l  lower right quadrant
 45 #define G_QUAD_LL    0x0D // m  lower left quadrant
 46 #define G_QUAD_UL    0x0E // n  upper left quadrant
 47 #define G_QUAD_UR    0x0F // o  upper right quadrant
 48 #define G_HALF_TOP   0x10 // p  upper half
 49 #define G_HALF_RIGHT 0x11 // q  right half
 50 #define G_TRI_UL     0x12 // r  upper left triangle
 51 #define G_TEE_DOWN   0x13 // s  T, stem down
 52 #define G_TEE_LEFT   0x14 // t  T, stem left
 53 #define G_TEE_UP     0x15 // u  T, stem up
 54 #define G_TEE_RIGHT  0x16 // v  T, stem right
 55 #define G_X          0x17 // w  diagonal cross
 56 #define G_SLASH      0x18 // x  /
 57 #define G_BACKSLASH  0x19 // y  backslash
 58 #define G_BAR_TOP    0x1A // z  top line
 59 #define G_BAR_BOTTOM 0x1B // {  bottom line
 60 #define G_BAR_LEFT   0x1C // |  left bar
 61 #define G_BAR_RIGHT  0x1D // }  right bar
 62 #define G_PILCROW    0x1E // ~
 63 #define G_TRI_UR     0x1F // _  upper right triangle
 64 #define G_BULLET     0x7F // ^  bullet
 65
 66 /// @brief Set up the terminal (and HDOS's console driver) for the game
 67 void termInit(void);
 68
 69 /// @brief Put the terminal (and HDOS's console driver) back as they were
 70 void termExit(void);
 71
 72 /// @brief Set one cell of the back buffer
 73 void put(uint8_t x, uint8_t y, uint8_t code);
 74
 75 /// @brief Set a run of w cells of the back buffer to one code
 76 void fill(uint8_t x, uint8_t y, uint8_t w, uint8_t code);
 77
 78 /// @brief Blank n whole rows of the back buffer from row y
 79 void blankRows(uint8_t y, uint8_t n);
 80
 81 /// @brief The back buffer's cell at x,y (a space off screen)
 82 uint8_t peek(uint8_t x, uint8_t y);
 83
 84 /// @brief Send the terminal whatever differs from the back buffer
 85 void flush(void);
 86
 87 /// @brief Copy the back buffer aside
 88 void conSave(void);
 89
 90 /// @brief Put back the copy conSave() took (sent at the next flush)
 91 void conRestore(void);
 92
 93 /// @brief Ring the terminal's bell
 94 void termBell(void);
 95
 96 /// @brief Raw console byte, or -1 when none is waiting
 97 int conIn(void);
 98
 99 /// @brief The H89's 2 ms clock tick counter (HDOS .TICCNT / the CP/M BIOS's)
100 #ifdef __HDOS__
101 #define TICKS (*(volatile uint16_t *)0x201B)
102 #else
103 #define TICKS (*(volatile uint16_t *)0x000B)
104 #endif
105
106 #endif /* H89_CONSOLE_H */
```

SRC/H89/FUJINET.C — 53 LINES

```
 1 #ifdef __H89__
 2
 3 /**
 4  * @brief   FujiNet calls - Heathkit H89 (CUSTOM_FUJINET_CALLS)
 5  * @author  Thomas Cherryhomes
 6  * @email   thom dot cherryhomes at gmail dot com
 7  * @license gpl v. 3, see LICENSE for details
 8  *
 9  * The shared code's network and AppKey calls, through fujinet-lib's h89 bus
10  * (the H89 FujiNet card's 82C55 at 0xD0). They are routed here only so the
11  * screen can be sent first: a call blocks for as long as the request takes,
12  * and what was drawn just before it -- "refreshing game list..",
13  * "connecting to server" -- has to be on the terminal meanwhile.
14  */
15
16 #include <stdint.h>
17 #include <fujinet-fuji.h>
18 #include <fujinet-network.h>
19 #include "console.h"
20
```

```
int16_t custom_network_call(char *url, uint8_t *buffer, uint16_t max_len)
{
    static int16_t read;

    flush();
    if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE) != FN_ERR_OK)
        return -1;

    read = network_read(url, buffer, max_len);
    network_close(url);
    return read;
}

uint16_t custom_read_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, char
 +    *destination)
{
    static uint16_t read;

    flush();
    read = 0;
    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    if (!fuji_read_appkey(key_id, &read, (uint8_t *)destination))
        read = 0;
    return read;
}

void custom_write_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, uint16_t count,
 +    char *data)
{
    flush();
    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    fuji_write_appkey(key_id, count, (uint8_t *)data);
}

#endif /* __H89__ */
```

SRC/H89/GRAPHICS.C 666 LINES

```
#ifdef __H89__

/**
 * @brief   Graphics - Heathkit H89 (H19 terminal, 80x24 + 25th line)
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * Everything is drawn with the H19's own characters: its graphics set (line
 * drawing, edge bars, the bullet, the checkerboard, arrows) and reverse
 * video, which is the alternate text style here. Text is shown in capitals,
 * as the game's fonts do on the other platforms.
 *
 * The board (drawBoard) is the 40 column one widened to 80 -- see
 * src/h89/vars.h for the columns. Lines are drawn by direction (line()), so
 * where two meet -- the board's rules and dividers, a box laid over the
 * frame of the bordered screens -- the cell becomes the right corner, T or
 * cross by itself.
 *
 * Dice are 7x3, which is square on the H19's tall cells: a free die is a
 * solid reverse video block with dark pips, a kept one is hollow -- bright
 * pips between edge bars. The roll button is the same size, and the dice
 * cursor is a bracket in the gap columns either side. The active player's
 * column header is a reverse video tab, kept through every redraw of the
 * initial in it by cell(), which all drawing goes through.
 *
 * Drawing goes into console.c's back buffer, and goes to the terminal only
 * at waitvsync() (src/h89/util.c), when input is read (src/h89/input.c) and
 * before every FujiNet call (src/h89/fujinet.c) -- at 9600 baud, only what
 * changed.
 */

#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include "console.h"
#include "../platform-specific/graphics.h"

// Line directions, for line()
#define LN_U 1
#define LN_D 2
#define LN_L 4
#define LN_R 8

// The board: the score name box's left border, the player column dividers
// (the first is the name box's right border), and the rules
#define NAMES_LEFT   11
#define DIV_FIRST    23
#define DIV_STEP     9
#define DIV_LAST     77
#define COL_W        8  // a player column's inside
#define RULE_TOP     0
#define RULE_HEAD    2
#define RULE_UPPER   9
#define RULE_LOWER   12
#define RULE_BOTTOM  20
#define INITIAL_ROW  1

#define DIE_W 7
#define DIE_H 3

// The bordered screens: a die in each corner, the frame between them
#define CORNER_RIGHT (WIDTH - DIE_W)
#define CORNER_BOTTOM (HEIGHT - 1 - DIE_H)

unsigned char colorMode = 0;

// The score names, in scoreY[] order (the FUJITZEE row is drawFujitzee's)
static const char *const scoreName[14] = {
    "ONES", "TWOS", "THREES", "FOURS", "FIVES", "SIXES", "TOTAL", "BONUS",
    "SET OF 3", "SET OF 4", "FULL HOUSE", "SHORT RUN", "LONG RUN", "COUNT"};

// Pips on a 3x3 grid, bit r*3+c, shown in columns 1, 3 and 5
static const uint16_t pipMask[6] = {
    0x010, // 1: centre
    0x044, // 2: top right, bottom left
    0x054, // 3
    0x145, // 4: the corners
    0x155, // 5
    0x16D, // 6: two columns of three
};

// The line drawing character for each combination of directions
static const uint8_t lineGlyph[16] = {
    ' ',        G_VLINE,    G_VLINE,    G_VLINE,
    G_HLINE,    G_CORNER_LR, G_CORNER_UR, G_TEE_LEFT,
    G_HLINE,    G_CORNER_LL, G_CORNER_UL, G_TEE_RIGHT,
    G_HLINE,    G_TEE_UP,   G_TEE_DOWN, G_CROSS};

// The active player's column (-1 none), whether its scores are flashed, and
// whether the board is up at all
static int8_t tab = -1;
static bool flashed, boardShown;

// The dice cursor
static bool cursorShown;
static uint8_t cursorX;

// What saveScreenBuffer() keeps beside the screen
static int8_t savedTab;
static bool savedBoardShown, savedCursorShown;
static uint8_t savedCursorX;

/*
 * Cells
 */

static uint8_t tabLeft(int8_t p)
{
    return DIV_FIRST + 1 + DIV_STEP * (uint8_t)p;
}

/// @brief Set a cell, in reverse if it is in the active player's tab
static void cell(uint8_t x, uint8_t y, uint8_t code)
{
    static uint8_t l;

    if (y == INITIAL_ROW && tab >= 0 && boardShown)
    {
        l = tabLeft(tab);
        if (x >= l && x < l + COL_W)
            code |= REV;
    }
    put(x, y, code);
}

// The directions of each graphics character (screen codes 0x00-0x1F), 0 for
// those that are not line drawing
static const uint8_t glyphBits[32] = {
    LN_U | LN_D,              // G_VLINE
    LN_L | LN_R,              // G_HLINE
    LN_U | LN_D | LN_L | LN_R, // G_CROSS
    LN_D | LN_L,              // G_CORNER_UR
    LN_U | LN_L,              // G_CORNER_LR
    LN_U | LN_R,              // G_CORNER_LL
    LN_D | LN_R,              // G_CORNER_UL
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    LN_D | LN_L | LN_R,       // G_TEE_DOWN
    LN_U | LN_D | LN_L,       // G_TEE_LEFT
    LN_U | LN_L | LN_R,       // G_TEE_UP
    LN_U | LN_D | LN_R,       // G_TEE_RIGHT
    0, 0, 0, 0, 0, 0, 0, 0};

/// @brief Draw a piece of line, joining any line already in the cell
static void line(uint8_t x, uint8_t y, uint8_t bits)
{
    static uint8_t c;
```

```
148
149     c = peek(x, y);
150     if (c < 0x20)
151         bits |= glyphBits[c];
152     put(x, y, lineGlyph[bits]);
153 }
154
155 /*
156   Lines are drawn as a set of straight segments at once, a row at a time:
157   every segment through a cell adds its directions, so corners and junctions
158   come out whole. (A line end stored on its own would be a plain line, and
159   read back as one.) Only what was on screen before -- the frame under a box
160   -- is joined cell by cell.
161 */
162 #define SEG_MAX 8
163
164 typedef struct
165 {
166     uint8_t from, to, at; // across: x from-to on row at; down: y from-to in column at
167 } Seg;
168
169 static Seg acrossSeg[SEG_MAX], downSeg[SEG_MAX];
170 static uint8_t acrossCount, downCount;
171 static uint8_t rowBits[WIDTH];
172 static uint8_t segLeft = WIDTH - 1, segRight; // the columns the segments span
173
174 static void segSpan(uint8_t x0, uint8_t x1)
175 {
176     if (x0 < segLeft)
177         segLeft = x0;
178     if (x1 > segRight)
179         segRight = x1;
180 }
181
182 static void across(uint8_t x0, uint8_t x1, uint8_t y)
183 {
184     if (x1 >= WIDTH)
185         x1 = WIDTH - 1;
186     acrossSeg[acrossCount].from = x0;
187     acrossSeg[acrossCount].to = x1;
188     acrossSeg[acrossCount].at = y;
189     acrossCount++;
190     segSpan(x0, x1);
191 }
192
193 static void down(uint8_t x, uint8_t y0, uint8_t y1)
194 {
195     if (x >= WIDTH)
196         return;
197     downSeg[downCount].from = y0;
198     downSeg[downCount].to = y1;
199     downSeg[downCount].at = x;
200     downCount++;
201     segSpan(x, x);
202 }
203
204 /// @brief Draw the segments given since the last call, rows y0-y1
205 static void drawSegs(uint8_t y0, uint8_t y1)
206 {
207     static uint8_t x, y, i, n;
208     static Seg *g;
209
210     n = segRight - segLeft + 1;
211     for (y = y0; y <= y1; y++)
212     {
213         memset(rowBits + segLeft, 0, n);
214         for (i = 0; i < acrossCount; i++)
215         {
216             g = &acrossSeg[i];
217             if (g->at != y)
218                 continue;
219             for (x = g->from; x <= g->to; x++)
220                 rowBits[x] |= (x > g->from ? LN_L : 0) | (x < g->to ? LN_R : 0);
221         }
222         for (i = 0; i < downCount; i++)
223         {
224             g = &downSeg[i];
225             if (y < g->from || y > g->to)
226                 continue;
227             rowBits[g->at] |= (y > g->from ? LN_U : 0) | (y < g->to ? LN_D : 0);
228         }
229         for (x = segLeft; x <= segRight; x++)
230             if (rowBits[x])
231                 line(x, y, rowBits[x]);
232     }
233     acrossCount = downCount = 0;
234     segLeft = WIDTH - 1;
235     segRight = 0;
236 }
237
238 /*
239  * Text
240  */
241
242 /// @brief The screen code for a character of text: capitals, and a space
243 /// for anything the terminal can't show
244 static uint8_t textCode(uint8_t c)
245 {
246     if (c >= 'a' && c <= 'z')
247         return c - 32;
248     if (c < 0x20 || c > 0x7E)
249         return ' ';
250     return c;
251 }
252
253 /// @brief Is a letter or digit of alternate text in the alternate style?
254 /// Capitals are shown plain, the rest in reverse video.
255 static bool altStyle(uint8_t c)
256 {
257     return (c >= 'a' && c <= 'z') || (c >= '0' && c <= '9');
258 }
259
260 void drawText(unsigned char x, unsigned char y, char *s)
261 {
262     static uint8_t c;
263
264     while ((c = (uint8_t)*s++) && x < WIDTH)
265         cell(x++, y, textCode(c));
266 }
267
268 void drawTextAlt(unsigned char x, unsigned char y, char *s)
269 {
270     static uint8_t c;
271     static bool rev, prevRev;
272
273     // A space is reversed only between two reversed letters or digits, so
274     // headings become one bar, words keep their gaps and padding stays
275     // blank; punctuation goes with what it follows ("1.", "SET," but
276     // "FILLED.")
277     prevRev = false;
278     while ((c = (uint8_t)*s++) && x < WIDTH)
279     {
280         if (c == ' ')
281             rev = prevRev && altStyle((uint8_t)*s);
282         else if ((c >= 'A' && c <= 'Z') || altStyle(c))
283             rev = altStyle(c);
284         else
285             rev = prevRev;
286         cell(x++, y, textCode(c) | (rev ? REV : 0));
287         prevRev = rev;
288     }
289 }
290
291 void drawTextRev(unsigned char x, unsigned char y, char *s)
292 {
293     static uint8_t c;
294
295     while ((c = (uint8_t)*s++) && x < WIDTH)
296         cell(x++, y, textCode(c) | REV);
297 }
298
299 void drawChar(unsigned char x, unsigned char y, char c, unsigned char alt)
300 {
301     // The initials over the player columns are plain: the tab marks the
302     // active one
303     if (y == INITIAL_ROW && boardShown)
304         alt = 0;
305     cell(x, y, textCode((uint8_t)c) | (alt ? REV : 0));
306 }
307
308 void drawIcon(unsigned char x, unsigned char y, unsigned char icon)
309 {
310     cell(x, y, icon);
311 }
312
313 void drawBlank(unsigned char x, unsigned char y)
314 {
315     cell(x, y, ' ');
316 }
317
318 void drawSpace(unsigned char x, unsigned char y, unsigned char w)
319 {
320     while (w--)
321         cell(x++, y, ' ');
322 }
323
324 void drawLine(unsigned char x, unsigned char y, unsigned char w)
325 {
326     while (w--)
327         line(x++, y, LN_L | LN_R);
328 }
329
330 void drawBox(unsigned char x, unsigned char y, unsigned char w, unsigned char h)
331 {
332     static uint8_t r;
333
```

```
    // The in-game help clears the columns it covers but not the board's top
    // and bottom rules: make those plain there, rather than T pieces
    // leading nowhere. (Hiding the help restores the whole screen.)
    if (boardShown && x == HELP_X - 1 && y == 1)
    {
        fill(HELP_CLEAR_X, RULE_TOP, HELP_CLEAR_W, G_HLINE);
        fill(HELP_CLEAR_X, RULE_BOTTOM, HELP_CLEAR_W, G_HLINE);
        put(HELP_CLEAR_X - 1, RULE_HEAD, G_TEE_LEFT);
        put(HELP_CLEAR_X - 1, RULE_UPPER, G_TEE_LEFT);
        put(HELP_CLEAR_X - 1, RULE_LOWER, G_TEE_LEFT);
        put(DIV_LAST, RULE_HEAD, G_VLINE);
        put(DIV_LAST, RULE_UPPER, G_VLINE);
        put(DIV_LAST, RULE_LOWER, G_VLINE);
    }

    across(x, x + w + 1, y);
    across(x, x + w + 1, y + h + 1);
    down(x, y, y + h + 1);
    down(x + w + 1, y, y + h + 1);
    drawSegs(y, y + h + 1);
    for (r = 1; r <= h; r++)
        fill(x + 1, y + r, w, ' ');
}

void drawClock(unsigned char x, unsigned char y)
{
    // An hourglass
    cell(x, y, REV | G_X);
}

void drawConnectionIcon(unsigned char x, unsigned char y)
{
    cell(x, y, REV | G_X);
    cell(x + 1, y, REV | '?');
}

void drawFujitzee(unsigned char x, unsigned char y)
{
    // On the board, the last score name; anywhere else (the help screen) it
    // stands in front of " - 5 OF A KIND" drawn at x, so it ends before it
    if (x != SCORES_X)
        x -= 3;
    drawTextAlt(x, y, "fujiTZEE");
}

/*
 * Dice
 */

/// @brief A 7x3 die (1-6), roll button (14-15 for 1-2 rolls left, 16 when
/// all the dice are kept) or blank (13)
static void dieAt(uint8_t x, uint8_t y, uint8_t s, bool isSelected, bool isHighlighted)
{
    static uint8_t r, c, code, flip, rowPips, bit;
    static uint16_t mask;

    flip = isHighlighted ? REV : 0;
    if (s <= 6)
        mask = pipMask[s - 1];

    for (r = 0; r < DIE_H; r++)
    {
        // This row's three pip places, in bits 1, 3 and 5 for columns 1, 3, 5
        rowPips = (uint8_t)((mask & 7) << 1);
        rowPips = (rowPips & 2) | ((rowPips & 4) << 1) | ((rowPips & 8) << 2);
        mask >>= 3;
        bit = 1;
        for (c = 0; c < DIE_W; c++, bit <<= 1)
        {
            if (s <= 6)
            {
                if (rowPips & bit)
                    code = G_BULLET;
                else if (isSelected && c == 0)
                    code = G_BAR_LEFT;
                else if (isSelected && c == DIE_W - 1)
                    code = G_BAR_RIGHT;
                else
                    code = ' ';
                // Free dice are solid; kept ones hollow
                if (!isSelected)
                    code |= REV;
            }
            else if (s == 13)
                code = ' ';
            else if (r == 1)
            {
                // " ROLL! " -- plain on a checkerboard when nothing is left
                // to roll
                code = (uint8_t)" ROLL! "[c] | (s == 16 ? 0 : REV);
            }
            else if (s == 16)
                code = G_CHECKER;
            else if (r == 2 && ((s == 14 && c == 3) || (s == 15 && (c == 2 || c == 4))))
                code = REV | G_BULLET; // a dot for each roll left
            else
                code = REV | ' ';

            cell(x + c, y + r, code ^ flip);
        }
    }
}

/// @brief The frame between the bordered screens' corner dice
static void drawFrameLines(void)
{
    across(DIE_W, CORNER_RIGHT - 1, 0);
    across(DIE_W, CORNER_RIGHT - 1, CORNER_BOTTOM + DIE_H - 1);
    down(0, DIE_H, CORNER_BOTTOM - 1);
    down(WIDTH - 1, DIE_H, CORNER_BOTTOM - 1);
    drawSegs(0, CORNER_BOTTOM + DIE_H - 1);
}

/// @brief The bordered screens' corner dice and the frame between them
static void drawFrame(void)
{
    dieAt(0, 0, 1, 0, 0);
    dieAt(CORNER_RIGHT, 0, 2, 0, 0);
    dieAt(0, CORNER_BOTTOM, 3, 0, 0);
    dieAt(CORNER_RIGHT, CORNER_BOTTOM, 4, 0, 0);
    drawFrameLines();
}

void drawBorder()
{
    drawFrame();
}

void drawDie(unsigned char x, unsigned char y, unsigned char s, bool isSelected, bool
 +    isHighlighted)
{
    // The shared code's flash loop passes the string terminator after the
    // fifth die
    if (!s || s > 16)
        return;

    // The bordered screens' 3x3 corner dice (at WIDTH-3 and HEIGHT-3) are
    // 7x3 here, the bottom ones above the status line, with a frame between
    // them (drawn with the first, the top left one)
    if ((x == 0 || x == WIDTH - 3) && (y == 0 || y == HEIGHT - 3))
    {
        dieAt(x ? CORNER_RIGHT : 0, y ? CORNER_BOTTOM : 0, s, 0, 0);
        if (!x && !y)
            drawFrameLines();
        return;
    }

    dieAt(x, y, s, isSelected, isHighlighted);
}

static void cursorCells(uint8_t x, uint8_t l, uint8_t m, uint8_t r)
{
    static uint8_t y;

    y = HEIGHT - 4;
    put(x - 1, y, l);
    put(x - 1, y + 1, m);
    put(x - 1, y + 2, r);
}

void drawDiceCursor(unsigned char x)
{
    hideDiceCursor(x);

    // [ ] in the gap columns either side of the die
    cursorCells(x, G_CORNER_UL, G_VLINE, G_CORNER_LL);
    cursorCells(x + DIE_W + 1, G_CORNER_UR, G_VLINE, G_CORNER_LR);
    cursorX = x;
    cursorShown = true;
}

void hideDiceCursor(unsigned char x)
{
    // Where it was drawn, whatever the shared code asks
    (void)x;
    if (!cursorShown)
        return;
    cursorCells(cursorX, ' ', ' ', ' ');
    cursorCells(cursorX + DIE_W + 1, ' ', ' ', ' ');
    cursorShown = false;
}

/*
 * Board
 */
```

```
519 /// @brief Turn the active player's tab on or off
520 static void paintTab(int8_t p, bool on)
521 {
522     static uint8_t x, l, c;
523 
524     l = tabLeft(p);
525     for (x = l; x < l + COL_W; x++)
526     {
527         c = peek(x, INITIAL_ROW);
528         put(x, INITIAL_ROW, on ? (c | REV) : (c & ~REV));
529     }
530 }
531 
532 /// @brief Reverse (or put back) a player's score column
533 static void flashColumn(int8_t p)
534 {
535     static uint8_t x, y, l;
536 
537     l = tabLeft(p);
538     for (y = RULE_HEAD + 1; y < RULE_BOTTOM; y++)
539     {
540         if (y == RULE_UPPER || y == RULE_LOWER)
541             continue;
542         for (x = l; x < l + COL_W; x++)
543             put(x, y, peek(x, y) ^ REV);
544     }
545 }
546 
547 void drawBoard()
548 {
549     static uint8_t x, i;
550 
551     // The rules across, the name box and dividers down through them, and a
552     // rule under the round, over the players
553     across(DIV_FIRST, DIV_LAST, RULE_TOP);
554     across(NAMES_LEFT, DIV_LAST, RULE_HEAD);
555     across(NAMES_LEFT, DIV_LAST, RULE_UPPER);
556     across(NAMES_LEFT, DIV_LAST, RULE_LOWER);
557     across(NAMES_LEFT, DIV_LAST, RULE_BOTTOM);
558     across(0, 9, 5);
559     down(NAMES_LEFT, RULE_HEAD, RULE_BOTTOM);
560     for (x = DIV_FIRST; x <= DIV_LAST; x += DIV_STEP)
561         down(x, RULE_TOP, RULE_BOTTOM);
562     drawSegs(RULE_TOP, RULE_BOTTOM);
563 
564     for (i = 0; i < 14; i++)
565         drawText(SCORES_X, scoreY[i], (char *)scoreName[i]);
566     drawFujitzee(SCORES_X, scoreY[14]);
567 
568     boardShown = true;
569     if (tab >= 0)
570         paintTab(tab, true);
571 }
572 
573 void setHighlight(int8_t player, bool isThisPlayer, uint8_t flash)
574 {
575     (void)isThisPlayer;
576 
577     if (flashed)
578     {
579         flashColumn(tab);
580         flashed = false;
581     }
582 
583     // Six columns on the board
584     if (player > 5)
585         player = -1;
586 
587     if (player != tab)
588     {
589         if (tab >= 0 && boardShown)
590             paintTab(tab, false);
591         tab = player;
592         if (tab >= 0 && boardShown)
593             paintTab(tab, true);
594     }
595 
596     // Your turn: the whole column, for a moment
597     if (flash == 2 && tab >= 0 && boardShown)
598     {
599         flashColumn(tab);
600         flashed = true;
601     }
602 }
603 
604 void clearBelowBoard()
605 {
606     blankRows(HEIGHT - 4, 4);
607     cursorShown = false;
608 }
609 
610 /*
611  * Screen
612  */
613 
614 void resetScreen(bool forBorderScreen)
615 {
616     blankRows(0, HEIGHT);
617     tab = -1;
618     flashed = boardShown = cursorShown = false;
619 
620     if (forBorderScreen)
621         drawFrame();
622 }
623 
624 bool saveScreenBuffer()
625 {
626     // The tab is in the screen; a flash is always over by the time the
627     // shared code saves it
628     conSave();
629     savedTab = tab;
630     savedBoardShown = boardShown;
631     savedCursorShown = cursorShown;
632     savedCursorX = cursorX;
633     return true;
634 }
635 
636 void restoreScreenBuffer()
637 {
638     conRestore();
639     tab = savedTab;
640     boardShown = savedBoardShown;
641     cursorShown = savedCursorShown;
642     cursorX = savedCursorX;
643     flashed = false;
644 }
645 
646 void initGraphics()
647 {
648     termInit();
649     resetScreen(false);
650 }
651 
652 void resetGraphics()
653 {
654     termExit();
655 }
656 
657 uint8_t cycleNextColor()
658 {
659     return 0;
660 }
661 
662 void setColorMode()
663 {
664 }
665 
666 #endif /* __H89__ */
```

SRC/H89/INPUT.C 162 LINES

```
1 #ifdef __H89__
2 
3 /**
4  * @brief   Input - Heathkit H89 (H19 keyboard)
5  * @author  Thomas Cherryhomes
6  * @email   thom dot cherryhomes at gmail dot com
7  * @license gpl v. 3, see LICENSE for details
8  *
9  * The H19 sends its keypad and function keys as escape sequences: with the
10 * keypad shifted (termInit sends ESC t) the arrows are ESC A-D, f1-f5 are
11 * ESC S-W, and BLUE/RED/GRAY are ESC P-R. Those arrive back to back at 9600
12 * baud, so an ESC with nothing behind it within a few ticks is the ESC key
13 * itself. Both OSes buffer the keyboard on its interrupt, so nothing is lost
14 * between polls. The decoder is fujinet-5cardstud's src/h89/input.c, behind
15 * the kbhit()/cgetc() the shared code reads (src/misc.h).
16 *
17 * Reading the keyboard is also when the screen goes to the terminal: the
18 * shared code polls kbhit() in every input loop and blocks in cgetc().
19 */
20 
21 #include <stdint.h>
22 #include <string.h>
23 #include "console.h"
24 #include "../misc.h"
25 
26 #define ESC_WAIT_TICKS 10 // 20 ms
27 
28 // Decoded keys not yet returned
29 static uint8_t pending[4];
30 static uint8_t pendingCount;
31 
32 static void push(uint8_t k)
33 {
34     if (pendingCount < sizeof(pending))
```

```
        pending[pendingCount++] = k;
}

/// @brief Plain keys: DELETE erases like BACKSPACE, LINE FEED enters
static uint8_t plain(uint8_t c)
{
    if (c == 0x7F)
        return KEY_BACKSPACE;
    if (c == 0x0A)
        return KEY_RETURN;
    return c;
}

/// @brief Wait briefly for the byte after an ESC; -1 if none comes
static int escFollower(void)
{
    static uint16_t t;
    static int c;

    t = TICKS;
    do
    {
        c = conIn();
        if (c >= 0)
            return c;
    } while ((uint16_t)(TICKS - t) < ESC_WAIT_TICKS);

    return -1;
}

static void decode(uint8_t c)
{
    static int n;

    if (c != KEY_ESCAPE)
    {
        push(plain(c));
        return;
    }

    n = escFollower();
    switch (n)
    {
    case 'A':
        push(KEY_UP_ARROW);
        break;
    case 'B':
        push(KEY_DOWN_ARROW);
        break;
    case 'C':
        push(KEY_RIGHT_ARROW);
        break;
    case 'D':
        push(KEY_LEFT_ARROW);
        break;
    case 'P': // BLUE
    case 'Q': // RED
    case 'R': // GRAY
    case 'S': // f1
    case 'T': // f2
    case 'U': // f3
    case 'V': // f4
    case 'W': // f5
        // The function keys open the menu too
        push(KEY_ESCAPE);
        break;
    case 'H': // HOME
    case 'J': // ERASE
    case 'E': // SHIFT ERASE
    case 'L': // IL
    case 'M': // DL
    case 'N': // DC
    case '@': // IC
        // The keypad's other keys do nothing here
        break;
    default:
        // A lone ESC, or ESC then an ordinary key
        push(KEY_ESCAPE);
        if (n >= 0 && n != KEY_ESCAPE)
            push(plain((uint8_t)n));
        else if (n == KEY_ESCAPE)
            push(KEY_ESCAPE);
        break;
    }
}

/// @brief Decode whatever the terminal has sent
static void poll(void)
{
    static int c;

    while (pendingCount < sizeof(pending) - 1)
    {
        c = conIn();
        if (c < 0)
            return;
        decode((uint8_t)c);
    }
}

unsigned char kbhit(void)
{
    flush();
    poll();
    return pendingCount != 0;
}

unsigned char cgetc(void)
{
    static uint8_t k;

    flush();
    while (!pendingCount)
        poll();

    k = pending[0];
    pendingCount--;
    memmove(pending, pending + 1, pendingCount);
    return k;
}

unsigned char readJoystick()
{
    // Keyboard only: the arrows and RETURN/SPACE come through kbhit()/cgetc()
    return 0;
}

#endif /* __H89__ */
```

SRC/H89/SOUND.C 62 LINES

```
#ifdef __H89__

/**
 * @brief   Sound - Heathkit H89
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * The only sound an H89 makes is the H19's bell, so it marks the moments
 * that need the player's attention -- sitting down at a table, their turn, a
 * Fujitzee, the end of the game -- and the rest stay quiet.
 */

#include <stdint.h>
#include <stdbool.h>
#include "console.h"
#include "../misc.h"
#include "../platform-specific/sound.h"

static void bell(void)
{
    if (!prefs.disableSound)
        termBell();
}

void initSound() {}
void disableKeySounds() {}
void enableKeySounds() {}
void soundStop() {}

void soundJoinGame()
{
    bell();
}

void soundMyTurn()
{
    bell();
}

void soundFujitzee()
{
    bell();
    pause(10);
    bell();
}

void soundGameDone()
{
    bell();
}

void soundRollDice() {}
void soundRollButton() {}
```

```
void soundTick() {}
void soundCursor() {}
void soundScoreCursor() {}
void soundKeep() {}
void soundRelease() {}
void soundScore() {}

#endif /* __H89__ */
```

SRC/H89/UTIL.C 74 LINES

```
#ifdef __H89__

/**
 * @brief   Utility Functions - Heathkit H89
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * The H89's clock interrupts every 2 ms, and both HDOS and the CP/M BIOS
 * count the ticks in a word (console.h's TICKS). waitvsync() paces frames at
 * 60 a second, as the shared code expects of pause(): 25/3 ticks, so 8, 8,
 * then 9. The move clock counts in 50ths of a second (TICKS/10), which needs
 * no long arithmetic.
 */

#include <stdint.h>
#include <stdlib.h>
#include "console.h"
#include "../misc.h"
#include "../platform-specific/graphics.h"
#include "../platform-specific/util.h"

/* The server sends the packed wire layout straight into clientState.game,
 * so sccz80 must not pad Game. Fails the build if it ever does (in which
 * case define FUJITZEE_PACK_STRUCTS - see misc.h). */
typedef char game_wire_size_check[(sizeof(Game) == 599) ? 1 : -1];

static uint16_t lastFrame, timerBase;
static uint8_t framePhase;

void waitvsync()
{
    static uint8_t len;

    // Send what was drawn, then wait out the rest of the frame. Drawing over
    // the 9600 baud console can take longer than a frame; the game just runs
    // at the pace of the terminal then.
    flush();
    len = (framePhase == 2) ? 9 : 8;
    while ((uint16_t)(TICKS - lastFrame) < len)
        ;
    lastFrame = TICKS;
    framePhase = (framePhase == 2) ? 0 : framePhase + 1;
}

void resetTimer()
{
    timerBase = TICKS;
}

uint16_t getTime()
{
    // The tick word wraps after 131 s, longer than any move clock
    return (uint16_t)(TICKS - timerBase) / 10;
}

uint8_t getJiffiesPerSecond()
{
    return 50;
}

void housekeeping()
{
}

void quit()
{
    // There is no H89 lobby to load: back to HDOS or CP/M, with the
    // terminal as it was
    resetGraphics();
    exit(0);
}

#endif /* __H89__ */
```

SRC/H89/VARS.H 116 LINES

```
#ifdef __H89__

#ifndef KEYMAP_H
#define KEYMAP_H

/*
  Heathkit H89: the H19 terminal's 80x24, plus its 25th line as row 24
  (HEIGHT-1, the status line), which src/h89/console.c turns on.

  The board is the 40 column one widened to 80: the score names in full in a
  box at columns 11-23, then six player columns eight wide (24-31, 33-40, ...
  69-76) between dividers at 23, 32, ... 77. The shared code's three wide
  score field sits in the middle of each (SCORE_COL_X), with the score cursor
  at its left and the initial over its centre. The dice are 7x3 -- the H19's
  cells are about two and a half times as tall as they are wide, so that is
  square -- every 9 columns, the roll button under the first player column.
  The layout is drawn in src/h89/graphics.c; the macros below only place what
  the shared code draws.

  Every macro is parenthesised and free of comparisons: sccz80 drops the 0/1
  of a ) that feeds arithmetic.
*/

#define WIDTH 80
#define HEIGHT 25

#define ROLL_SOUND_MOD 4 // How often to play roll sound
#define ROLL_FRAMES 5    // How many roll frames to play (each takes ~0.3 s at 9600 baud)
#define BOTTOM_HEIGHT 4  // How high the bottom panel is
#define SCORES_X 12      // X start of the score names
#define GAMEOVER_PROMPT_Y (HEIGHT-3)
#define ROLL_X 24
#define TIMER_X 13
#define TIMER_NUM_OFFSET_X 0
#define TIMER_NUM_OFFSET_Y 0

// Board layout hooks (src/gamelogic.c)
#define SCORE_COL_X(i)   (26+9*(i))
#define INITIAL_X(i)     (SCORE_COL_X((i)-1)+1)
#define DIE_X(i)         (33+9*(i))
#define DICE_CURSOR_X(p) (24+9*(p))
#define KEPT_DIE_X(p)    DICE_CURSOR_X(p)

// The in-game help: a box over player columns 2-6 (only player 1 sees it)
#define HELP_X           46
#define HELP_CLEAR_X     33
#define HELP_CLEAR_W     44

// The 40 column help screens, centred (src/screens.c)
#define HELP_LEFT        ((WIDTH-40)/2)

// The alternate style is reverse video here: the selected table is the
// reversed one, and mixed case server names are drawn plain
#define LOBBY_SEL_TEXT     drawTextRev
#define LOBBY_UNSEL_TEXT   drawText
#define CENTER_SERVER_TEXT centerText

/// @brief Draw text all in reverse video (src/h89/graphics.c)
void drawTextRev(unsigned char x, unsigned char y, char *s);

#define QUERY_SUFFIX "" // Z80 is little endian - no &be=1 needed

#define TRIGGER_TEXT "SPACE"

// Icons: H19 screen codes (src/h89/console.h) -- the character ROM index,
// 0x00-0x1F and 0x7F the graphics characters, with bit 7 for reverse video
#define ICON_TEXT_CURSOR  0xA0 // reverse space
#define ICON_MARK         0x08 // right arrow
#define ICON_MARK_ALT     0x20 // (blinks)
#define ICON_PLAYER       0x7F // bullet
#define ICON_SPEC         0x09 // checkerboard
#define ICON_CURSOR       0x08 // right arrow
#define ICON_CURSOR_ALT   0x88 // reverse right arrow
#define ICON_CURSOR_BLIP  0x20 // (blinks)

/**
 * Platform specific key map for common input (H19 keyboard)
 *
 * src/h89/input.c turns the keypad's ESC ? D (keypad shifted mode, set by
 * termInit) into the arrow codes below. readCommonInput() switches on all
 * twelve: the _2 codes are unreachable placeholders, and the _3 keys are
 * spares for typing without the keypad.
 */
#define KEY_LEFT_ARROW      0x1D
#define KEY_LEFT_ARROW_2    0xF1
#define KEY_LEFT_ARROW_3    0x2C // ,

#define KEY_RIGHT_ARROW     0x1C
#define KEY_RIGHT_ARROW_2   0xF2
#define KEY_RIGHT_ARROW_3   0x2E // .

#define KEY_UP_ARROW        0x1E
#define KEY_UP_ARROW_2      0xF3
#define KEY_UP_ARROW_3      0x2D // -

#define KEY_DOWN_ARROW      0x1F
```

```
 97 #define KEY_DOWN_ARROW_2    0xF4
 98 #define KEY_DOWN_ARROW_3    0x3D // =
 99
100 #define KEY_RETURN      0x0D
101 #define KEY_ESCAPE      0x1B
102 #define KEY_ESCAPE_ALT   0x03 // CTRL-C
103 #define KEY_SPACEBAR     0x20
104 #define KEY_BACKSPACE    0x08
105
106 // Keyboard only: readJoystick() always returns 0
107 #define JOY_UP(v)    ((v) & 0)
108 #define JOY_DOWN(v)  ((v) & 0)
109 #define JOY_LEFT(v)  ((v) & 0)
110 #define JOY_RIGHT(v) ((v) & 0)
111 #define JOY_BTN_1(v) ((v) & 0)
112 #define JOY_BTN_2(v) ((v) & 0)
113
114 #endif /* KEYMAP_H */
115
116 #endif /* __H89__ */
```

APPENDIX

NETWORK ERROR CODES

The fourth byte of a network STATUS reply (Chapter 10) is one of these. 1 means all is well; 136 means the data has all been read. fujinet-lib returns the same numbers.

CODE	NAME	GIVEN WHEN
1	SUCCESS	OK
131	WRITE_ONLY	READ on HTTP channel in a header-set/post-data mode; also CLIPBOARD/Calendar/Mailbox
132	INVALID_COMMAND	bad HTTP channel mode; write not allowed in this HTTP method/mode
135	READ_ONLY	WRITE in HTTP GET_HEADERS mode
136	END_OF_FILE	no more data: FS file/dir consumed, HTTP body done, TCP peer disconnected, all headers read
138	GENERAL_TIMEOUT	HTTP 408; TNFS mount failure
144	GENERAL	fatal/unclassified: unknown scheme at OPEN, unmapped errno, unmapped HTTP status incl. client -1 for empty POST/PUT, HTML/XML parse failure
146	NOT_IMPLEMENTED	HTTP aux1 not supported, HTTP 405, COLLECT_HEADERS on non-GET, SD LOCK/UNLOCK
151	FILE_EXISTS	TNFS file exists
162	NO_SPACE_ON_DEVICE	TNFS no space
165	INVALID_DEVICESPEC	OPEN spec is not a URL (no scheme); RENAME without comma
166	INVALID_POINT	defined, not produced by any code in this tree
167	ACCESS_DENIED	HTTP 423/451; TNFS read-only/denied
170	FILE_NOT_FOUND	HTTP 404/410; TNFS not found
200	CONNECTION_REFUSED	ECONNREFUSED
201	NETWORK_UNREACHABLE	ENETUNREACH
202	SOCKET_TIMEOUT	ETIMEDOUT; TCP short read/write
203	NETWORK_DOWN	ENETDOWN
204	CONNECTION_RESET	ECONNRESET; accepted client gone
205	CONNECTION_ALREADY_IN_PROGRESS	EINPROGRESS
206	ADDRESS_IN_USE	EADDRINUSE, e.g. TCP listen/UDP bind on a busy port
207	NOT_CONNECTED	no channel open on the unit
208	SERVER_NOT_RUNNING	ACCEPT/CLOSE_CLIENT on a channel that is not a listener
209	NO_CONNECTION_WAITING	ACCEPT with no pending client
210	SERVICE_NOT_AVAILABLE	used by FTP, GMAIL, IMAPS, Calendar adapters
211	CONNECTION_ABORTED	defined, not produced by any code in this tree
212	INVALID_USERNAME_OR_PASSWORD	HTTP 401/402/403/407
213	COULD_NOT_PARSE_JSON	JSON PARSE failed; sticks in STATUS
214	CLIENT_GENERAL	other HTTP 4xx
215	SERVER_GENERAL	HTTP 5xx
255	COULD_NOT_ALLOCATE_BUFFERS	SD adapter only

COMMAND QUICK REFERENCE

Every command the H89's FujiNet answers, by device. FIELDS gives the size of each field in order (1, a byte; 2, a word; 4, a long); PAYLOAD and REPLY give their length in bytes, or ACK for a reply with no data. The page is the command's card.

THE FUJINET DEVICE 70H

COMMAND	NO.	FIELDS	PAY-LOAD	REPLY	PAGE
FUJI_BASE64_DECODE_COMPUTE	CBH	--	--	ACK	8-16
FUJI_BASE64_DECODE_INPUT	CCH	2	len	ACK	8-16
FUJI_BASE64_DECODE_LENGTH	CAH	--	--	4	8-16
FUJI_BASE64_DECODE_OUTPUT	C9H	2	--	len	8-16
FUJI_BASE64_ENCODE_COMPUTE	CFH	--	--	ACK	8-15
FUJI_BASE64_ENCODE_INPUT	D0H	2	len	ACK	8-15
FUJI_BASE64_ENCODE_LENGTH	CEH	--	--	4	8-15
FUJI_BASE64_ENCODE_OUTPUT	CDH	2	--	len	8-15
FUJI_CLOSE_APPKEY	DBH	--	--	ACK	8-14
FUJI_CLOSE_DIRECTORY	F5H	--	--	ACK	8-8
FUJI_CONFIG_BOOT	D9H	1	--	ACK	8-12
FUJI_COPY_FILE	D8H	1,1	srcldst	ACK	8-11
FUJI_DEVICE_READY	00H	--	--	512	8-21
FUJI_GENERATE_GUID	BBH	--	--	37	8-20
FUJI_GET_ADAPTERCONFIG	E8H	--	--	140	8-2
FUJI_GET_ADAPTERCONFIG_EXTENDED	C4H	--	--	240	8-2
FUJI_GET_DEVICE_FULLPATH	DAH	1	--	256	8-10
FUJI_GET_DIRECTORY_POSITION	E5H	--	--	2	8-8
FUJI_GET_HOST_PREFIX	E0H	1	--	256	8-6
FUJI_GET_SCAN_RESULT	FCH	1	--	34	8-3
FUJI_GET_SSID	FEH	--	--	97	8-4
FUJI_GET_WIFISTATUS	FAH	--	--	1	8-2
FUJI_GET_WIFI_ENABLED	EAH	--	--	1	8-3
FUJI_HASH_CLEAR	C2H	--	--	ACK	8-18
FUJI_HASH_COMPUTE	C7H	1	--	ACK	8-17
FUJI_HASH_COMPUTE_NO_CLEAR	C3H	1	--	ACK	8-17
FUJI_HASH_INPUT	C8H	2	len	ACK	8-17
FUJI_HASH_LENGTH	C6H	1	--	1	8-17
FUJI_HASH_OUTPUT	C5H	1	--	length	8-17
FUJI_MOUNT_ALL	D7H	--	--	ACK	8-11
FUJI_MOUNT_HOST	F9H	1	--	ACK	8-5
FUJI_MOUNT_IMAGE	F8H	1,1	--	ACK	8-10
FUJI_NEW_DISK	E7H	--	262	NAK	8-11
FUJI_OPEN_APPKEY	DCH	--	6	ACK	8-13
FUJI_OPEN_DIRECTORY	F7H	1	up to 256	ACK	8-7
FUJI_QRCODE_ENCODE	BDH	1,1,1	--	ACK	8-19
FUJI_QRCODE_INPUT	BCH	2	len	ACK	8-19
FUJI_QRCODE_LENGTH	BEH	1	--	4	8-19
FUJI_QRCODE_OUTPUT	BFH	2	--	len	8-19
FUJI_RANDOM_NUMBER	D3H	--	--	4	8-20
FUJI_READ_APPKEY	DDH	--	--	2+n	8-14
FUJI_READ_DEVICE_SLOTS	F2H	--	--	304	8-9
FUJI_READ_DIR_ENTRY	F6H	1,1	--	field 1	8-7
FUJI_READ_HOST_SLOTS	F4H	--	--	256	8-5
FUJI_RESET	FFH	--	--	none	8-21
FUJI_SCAN_NETWORKS	FDH	--	--	1	8-3
FUJI_SET_BOOT_MODE	D6H	1	--	ACK	8-12
FUJI_SET_DEVICE_FULLPATH	E2H	1,1,1	up to 256	ACK	8-9
FUJI_SET_DIRECTORY_POSITION	E4H	2	--	ACK	8-8
FUJI_SET_HOST_PREFIX	E1H	1	up to 256	ACK	8-6
FUJI_SET_SSID	FBH	1	97	ACK	8-4
FUJI_STATUS	53H	1	--	4	8-21
FUJI_UNMOUNT_HOST	E6H	1	--	ACK	8-6
FUJI_UNMOUNT_IMAGE	E9H	1	--	ACK	8-10
FUJI_WRITE_APPKEY	DEH	--	<= 64	ACK	8-13
FUJI_WRITE_DEVICE_SLOTS	F1H	--	304	ACK	8-9
FUJI_WRITE_HOST_SLOTS	F3H	--	256	ACK	8-5

NETWORK CHANNELS 71H-78H

COMMAND	NO.	FIELDS	PAY-LOAD	REPLY	PAGE
NET_ACCEPT	41H	--	--	ACK	10-14
NET_CHDIR	2CH	--	path	ACK	10-20
NET_CLOSE	43H	--	--	ACK	10-3
NET_CLOSE_CLIENT	63H	--	--	ACK	10-15
NET_DELETE	21H	1,1	spec	ACK	10-19
NET_GETCWD	30H	--	--	prefix	10-21
NET_GET_REMOTE	72H	--	--	256	10-16
NET_LOCK	23H	1,1	spec	ACK	10-20
NET_MKDIR	2AH	1,1	spec	ACK	10-20
NET_OPEN	4FH	1,1	spec	ACK	10-2
NET_PARSE	50H	--	--	ACK	10-7
NET_PASSWORD	FEH	--	text	ACK	10-22
NET_QUERY	51H	--	pointer	ACK	10-8
NET_READ	52H	2	--	<= field	10-5
NET_RENAME	20H	1,1	spec,new	ACK	10-19
NET_RMDIR	2BH	1,1	spec	ACK	10-20
NET_SEEK	25H	4	--	ACK	10-18
NET_SET_DESTINATION	44H	--	N:host:port	ACK	10-17
NET_SET_EOL	4CH	--	1-2	ACK	10-22
NET_SET_HTTP_MODE	4DH	1,1	--	ACK	10-10
NET_SET_INT_RATE	5AH	1	--	ACK	10-23
NET_SET_PARAMETER	FBH	1,1	--	ACK	10-9
NET_SET_PARSER	FCH	1,1	--	ACK	10-7
NET_STATUS	53H	--	--	4	10-4
NET_TELL	26H	--	--	4	10-19
NET_TRANSLATION	54H	1,1	--	ACK	10-22
NET_UNLOCK	24H	1,1	spec	ACK	10-20
NET_USERNAME	FDH	--	text	ACK	10-22
NET_WRITE	57H	2	field	ACK	10-6

DISK DRIVES 31H-38H

COMMAND	NO.	FIELDS	PAY-LOAD	REPLY	PAGE
DISK_FORMAT	21H	--	--	128	9-3
DISK_FORMAT_MEDIUM	22H	--	--	128	--
DISK_PERCOM_READ	4EH	--	--	12	9-3
DISK_PERCOM_WRITE	4FH	--	12	ACK	9-3
DISK_PUT	50H	4	512	ACK	9-2
DISK_READ	52H	4	--	512	9-1
DISK_STATUS	53H	1	--	4	9-2
DISK_WRITE	57H	4	512	ACK	9-2

CLOCK 45H

COMMAND	NO.	FIELDS	PAY-LOAD	REPLY	PAGE
APETIME_GETTIME	93H	1	--	6	12-1
APETIME_GETTZTIME	9AH	1	--	6	12-4
APETIME_GETTZ_LEN	4CH	--	--	1	12-3
APETIME_GET_GENERAL	47H	--	--	text	12-3
APETIME_GET_ISO_LOCAL	49H	1	--	25	12-3
APETIME_GET_ISO_UTC	5AH	1	--	25	12-3
APETIME_GET_PRODOS	50H	1	--	4	12-2
APETIME_GET_SIMPLE_HUNDREDTHS	4DH	1	--	8	12-2
APETIME_GET_SOS	53H	1	--	19	12-2
APETIME_SETTZ	99H	1	text	ACK	12-4
APETIME_SETTZ_ALT	74H	1	text	ACK	12-5
APETIME_SETTZ_ALT2	54H	1	--	7	12-1

PRINTER 40H

COMMAND	NO.	FIELDS	PAY-LOAD	REPLY	PAGE
PRINTER_PUT	50H	--	text	ACK	12-6
PRINTER_STATUS	53H	1	--	4	12-7
PRINTER_WRITE	57H	1	text	ACK	12-6

MODEM 50H

COMMAND	NO.	FIELDS	PAY-LOAD	REPLY	PAGE
MODEM_AUTOANSWER	4FH	1	--	none	12-11
MODEM_BAUDRATELOCK	4EH	1,1	--	none	12-11
MODEM_CONFIGURE	42H	1	--	none	12-11
MODEM_CONTROL	41H	1	--	none	12-11
MODEM_LISTEN	4CH	2	--	none	12-10
MODEM_SET_DUMP	44H	1	--	none	12-11
MODEM_STATUS	53H	1	--	2	12-9
MODEM_STREAM	58H	1	--	9	12-10
MODEM_UNLISTEN	4DH	--	--	none	12-11
MODEM_WRITE	57H	1	bytes	none	12-8

THE H19'S ESCAPE SEQUENCES

The H19 in its own Heath mode answers these. Each is the ESC character (1BH) followed by the characters shown. In ANSI mode (after ESC <) it follows the ANSI standard instead, until ESC [? 2 h.

SEND	TO
ESC H	Move the cursor home (top left)
ESC Y r c	Move the cursor to row r, column c, each sent plus 32 (20H)
ESC A B C D	Move the cursor up, down, right, left
ESC I	Reverse line feed (up, scrolling down at the top)
ESC j k	Save the cursor's place; go back to it
ESC n	Report the cursor's place (as ESC Y r c)
ESC E	Clear the screen
ESC J	Erase to the end of the screen
ESC b	Erase from the start of the screen
ESC K	Erase to the end of the line
ESC o	Erase from the start of the line
ESC l	Erase the whole line
ESC L M	Insert a line; delete a line
ESC N	Delete a character
ESC @ O	Insert-character mode on; off
ESC p q	Reverse video on; off
ESC F G	Graphics characters on; off (see below)
ESC x n	Set mode n: 1 the 25th line on, 2 no key click, 3 hold screen, 4 block cursor, 5 cursor off, 6 keypad shifted, 7 alternate keypad, 8 auto LF after CR, 9 auto CR after LF
ESC y n	Reset mode n (the same numbers)
ESC t u	Keypad shifted mode on; off
ESC = >	Alternate keypad mode on; off
ESC v w	Wrap at the end of a line; discard instead
ESC { }	Keyboard on; off
ESC z	Reset to the power-up state
ESC <	ANSI mode
ESC Z	Identify (the H19 answers ESC / K)

THE H19'S GRAPHICS CHARACTERS

After ESC F, the H19 shows the characters from ^ (5EH) to ~ (7EH) as these 33 graphics, drawn here from its character generator ROM (444-29), until ESC G.

KEY	SHOWS	KEY	SHOWS
^	bullet	o	upper right quadrant
_	upper right triangle	p	upper half
`	vertical line	q	right half
a	horizontal line	r	upper left triangle
b	cross	s	T, stem down
c	upper right corner	t	T, stem left
d	lower right corner	u	T, stem up
e	lower left corner	v	T, stem right
f	upper left corner	w	diagonal cross
g	plus or minus	x	slash
h	right arrow	y	backslash
i	checkerboard	z	top line
j	divide	{	bottom line
k	down arrow	|	left bar
l	lower right quadrant	}	right bar
m	lower left quadrant	~	paragraph mark
n	upper left quadrant		

CP/M 2.2: THE BDOS AND THE BIOS

A CP/M program calls the BDOS with the function number in C and an argument in E or DE, then `CALL 5`. A byte comes back in A, a word in HL.

NO.	FUNCTION	NO.	FUNCTION
0	System reset	20	Read sequential
1	Console input	21	Write sequential
2	Console output (E)	22	Make file
3	Reader input	23	Rename file
4	Punch output	24	Login vector
5	List output	25	Current disk
6	Direct console I/O (E = FFH: input)	26	Set DMA address
7	Get I/O byte	27	Allocation vector
8	Set I/O byte	28	Write protect disk
9	Print string to $	29	Read-only vector
10	Read console buffer	30	Set file attributes
11	Console status	31	Disk parameters
12	Version number	32	Get/set user number
13	Reset disk system	33	Read random
14	Select disk	34	Write random
15	Open file	35	File size
16	Close file	36	Set random record
17	Search for first	37	Reset drive
18	Search for next	40	Write random, zero fill
19	Delete file		

The BIOS is a table of jumps; the word at 0001H holds the address of its second entry, WBOOT. The entries, 3 bytes apart from there: WBOOT, CONST (A = FFH if a key waits), CONIN, CONOUT (C), LIST, PUNCH, READER, HOME, SELDSK, SETTRK, SETSEC, SETDMA, READ, WRITE, LISTST, SECTRAN. The book's `h89con.c` calls CONST, CONIN and CONOUT directly. Heath's BIOS keeps the 2 ms tick count at 000BH.

FIRMWARE BEHAVIOUR AND ERRATA

Everything the making of this Manual found the FujiNet's software doing that a programmer would not expect: the firmware (by device), fujinet-lib, z88dk, and fujinet-pc where it differs from the card. The entries that matter in practice are WARNINGs on their cards; this is the complete list, each with the line of source it comes from, so it can be checked against later versions. `errata.md`, beside the book's source, holds the same list.

FUJINET DEVICE (70H)

F1 `FUJI_RESET`. No reply at all: the base handler table runs first (fn-h89/lib/device/rs232/rs232Fuji.cpp:215) and never ACKs; rs232Fuji's case FUJI_RESET -> fujicmd_reset(), which would ACK first (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:405-411), is unreachable (fn-h89/lib/device/rs232/rs232Fuji.cpp:228-230)

F2 `FUJI_DEVICE_READY`. Debug/test command left in the RS232 switch; one of only two commands reached through rs232Fuji's own switch.

F3 `FUJI_STATUS`. Always 4 zero bytes. rs232_status() (reqtype 1 = per-drive mount times, 8 x time_t) is dead code: base table handles FUJI_STATUS before the rs232 switch . The switch's paramCount<1 NAK is also unreachable. (fn-h89/lib/device/rs232/rs232Fuji.cpp:66-90, 220-227)

F4 `FUJI_GET_ADAPTERCONFIG`. fn_version is truncated to 14 characters (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:1232)

F5 `FUJI_GET_ADAPTERCONFIG_EXTENDED`. String fields are filled even when not connected (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:1250-1259)

F6 `FUJI_SCAN_NETWORKS`. ESP32 scan is blocking and temporarily disconnects an associated station (fn-h89/lib/hardware/fnWiFi.cpp:370-405)

F7 `FUJI_SET_SSID`. Failure sends two NAK frames: fujicore_net_set_ssid_success() NAKs and fujicmd_net_set_ssid_success() NAKs again (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:569). fujinet-lib resyncs on the next SLIP_END (fujinet-lib/bus/h89/port_slip.c:56-68) and so reads the stray NAK as the reply to the NEXT command: replies run one behind. (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:540)

F8 `FUJI_SET_SSID`. ACK does not mean associated: ESP32 connect() returns esp_wifi_connect()'s immediate status ; a wrong password is ACKed and saved. (fn-h89/lib/hardware/fnWiFi.cpp:149-204)

F9 `FUJI_GET_SSID`. Returns the saved WiFi passphrase in plain text (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:431-433, 444)

F10 `FUJI_WRITE_HOST_SLOTS`. Buffer is not zeroed : a payload shorter than 256 bytes fills the remaining slots with stack garbage. (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:1444)

F11 `FUJI_MOUNT_HOST`. Once hostMounted is set the mount is not retried even if the server went away. (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:459)

F12 `FUJI_UNMOUNT_HOST`. On the SD host the disks are unmounted and their slots cleared BEFORE the NAK (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:1553-1568)

F13 `FUJI_UNMOUNT_HOST`. MOUNT_ALL and COPY_FILE mount hosts without setting hostMounted, so UNMOUNT_HOST after them NAKs.

F14 `FUJI_SET_HOST_PREFIX`. COPY_FILE breaks once a prefix longer than "/" is set (see FUJI_COPY_FILE).

F15 `FUJI_GET_HOST_PREFIX`. prefix[] is never zeroed ; strlcpy writes only up to the NUL (fn-h89/lib/fuji/fujiHost.cpp:300-306): the tail of the reply is garbage. (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:1396)

F16 `FUJI_OPEN_DIRECTORY`. MOUNT_HOST first: dir_open fails on an unmounted host.

F17 `FUJI_OPEN_DIRECTORY`. With a path < 253 chars the pattern pointer is never null; an empty pattern is passed as "" (fnFsTNFS.cpp:296 then indexes realpat[-1])

F18 `FUJI_READ_DIR_ENTRY`. addtl 0xC0 (block mode, group size 0) divides by zero at fn-h89/lib/device/fujiDevice/fujiDevice.cpp:887 -> SIGFPE on fujinet-pc, panic/reboot on ESP32.

F19 `FUJI_READ_DIR_ENTRY`. With 0x80 keep maxlen >= 14: set_additional_direntry_details does uint8_t maxlen -= 12 , smaller values wrap and the reply is cut mid-header. (fn-h89/lib/device/rs232/rs232Fuji.cpp:199)

F20 `FUJI_READ_DIR_ENTRY`. TRUNC flag and slash rules disagree by one for directories: a dir name of maxlen-13 chars gets TRUNC although only the '/' was dropped; one of maxlen-14 chars gets neither '/' nor TRUNC

(fn-h89/lib/device/rs232/rs232Fuji.cpp:199-205 vs fn-h89/lib/device/fujiDevice/fujiDevice.cpp:1004-1020)
F21 FUJI_READ_DIR_ENTRY. Plain .img files report mediatype 0 (unknown), not 1 (fn-h89/lib/media/rs232/diskType.cpp:64-95)
F22 FUJI_READ_DEVICE_SLOTS. Empty slots report mode 0x01 . (fujiDisk default, fn-h89/lib/fuji/fujiDisk.cpp:9)
F23 FUJI_READ_DEVICE_SLOTS. mode reflects SET_DEVICE_FULLPATH / WRITE_DEVICE_SLOTS, not the mode passed to MOUNT_IMAGE . (MOUNT_IMAGE never stores it, fn-h89/lib/device/fujiDevice/fujiDevice.cpp:592-635)
F24 FUJI_WRITE_DEVICE_SLOTS. Filenames in the payload are ignored: fujiDisk::reset(fname, ...) never copies fname . Paths can only be set with SET_DEVICE_FULLPATH. (fn-h89/lib/fuji/fujiDisk.cpp:15-25)
F25 FUJI_WRITE_DEVICE_SLOTS. hostSlot 0xFF drops the slot from fnconfig.ini (path gone after restart) while the RAM filename stays and READ_DEVICE_SLOTS still shows it (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:221-222)
F26 FUJI_WRITE_DEVICE_SLOTS. No host-slot validation: a value 8..254 with an existing filename makes MOUNT_ALL use _fnHosts[] out of bounds (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:239,248)
F27 FUJI_WRITE_DEVICE_SLOTS. Buffer not zeroed : a short payload gives the remaining slots garbage host/mode. (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:1619)
F28 FUJI_SET_DEVICE_FULLPATH. Missing field aborts the firmware: the rs232 switch's paramCount<3 guard is unreachable. (fn-h89/lib/device/rs232/rs232Fuji.cpp:334-342)
F29 FUJI_MOUNT_IMAGE. ROM-type extensions push the whole file before replying: FujiNet sends its own command frames to device 0xFF (NET_OPEN 0x4F with 5-byte payload id+u32 size, NET_WRITE 0x57 chunks, NET_CLOSE 0x43 [+0x01 abort]) and waits for ACKs (fn-h89/lib/device/rs232/disk.cpp:28-110, 305-315; fn-h89/lib/bus/rs232/rs232.h:178-186). On the H89 a .BIN file is therefore not a disk image. (.ROM .BIN .INT .ITV .CHF .NES .SMS .A78 .A52 .ST2, fn-h89/lib/media/rs232/diskType.cpp:86-93)
F30 FUJI_MOUNT_IMAGE. Needs filename+host from SET_DEVICE_FULLPATH (or saved config) and a mounted host; it does not mount the host itself.
F31 FUJI_MOUNT_IMAGE. Repeated MOUNT_IMAGE with a prefix set re-applies the prefix to the already-prefixed filename.
F32 FUJI_MOUNT_ALL. Out-of-range host_slot 8..254 (possible via WRITE_DEVICE_SLOTS) indexes _fnHosts out of bounds (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:239,248)
F33 FUJI_NEW_DISK. Leaves a 0-byte file behind and a dangling _fnDisks[slot].fileh (closed handle never set to nullptr, fn-h89/lib/device/rs232/rs232Fuji.cpp:150,159); fileh != nullptr is what image rotation tests (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:324,349)
F34 FUJI_NEW_DISK. Reached only through rs232Fuji's switch (not in the base table).
F35 FUJI_SET_BOOT_MODE. Mode 0 puts the RS232 MS-DOS boot disk (360 KB FAT12, fn-h89/data/webui/device_specific/BUILD_RS232/autorun.img) on drive 0x31 even though setup() deliberately skips it on PINMAP_H89 (fn-h89/lib/device/rs232/rs232Fuji.cpp:38-43)
F36 FUJI_SET_BOOT_MODE. Always ACKs: invalid mode, missing image (only autorun.img ships for BUILD_RS232) or TNFS failure mount nothing silently (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:678-702)
F37 FUJI_CONFIG_BOOT. Despite the name it never changes boot_config or the saved config; boot_config is only written, never read, on BUILD_RS232.
F38 FUJI_COPY_FILE. Fails whenever a host prefix longer than "/" is set: each path is opened with fullpathlen = path length + 1 (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:1126-1132), so prefix+path never fits and fnfile_open returns null (fn-h89/lib/fuji/fujiHost.cpp:245-254)
F39 FUJI_COPY_FILE. The trailing-'/' rule only works if the LAST payload byte is '/': a NUL-terminated or padded spec disables it.
F40 FUJI_COPY_FILE. fread/fwrite results are not checked: a short copy still ACKs (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:1148-1152)
F41 FUJI_COPY_FILE. Missing field aborts: the rs232 switch's guard is unreachable. (fn-h89/lib/device/rs232/rs232Fuji.cpp:375-383)
F42 FUJI_OPEN_APPKEY. Any mode other than 0xFF is accepted; mode 2 still reads only 64 bytes . (MAX_APPKEY_LEN, fn-h89/lib/device/fujiDevice/AppKeyMixin.h:6)
F43 FUJI_READ_APPKEY. Never NAKs: no SD, no OPEN, OPEN in write mode, or missing file all return ACK with length 0 (fn-h89/lib/device/fujiDevice/AppKeyMixin.cpp:84 empty buffer; failure paths fn-h89/lib/device/fujiDevice/AppKeyMixin.cpp:87-108)
F44 FUJI_WRITE_APPKEY. An empty/missing payload writes a 0-byte key (truncating an existing one) and ACKs (fn-h89/lib/device/fujiDevice/AppKeyMixin.cpp:189, 139, 172-175)
F45 FUJI_BASE64_ENCODE_INPUT. transaction_get's result is ignored : no payload appends len zero bytes and ACKs. (fn-h89/lib/device/fujiDevice/Base64Mixin.cpp:21)
F46 FUJI_BASE64_ENCODE_INPUT. Encode and decode share ONE buffer and there is no clear command: unread output from a previous operation is prepended to the next input. (global base64.base64_buffer, fn-h89/lib/encoding/base64.h:62)
F47 FUJI_BASE64_ENCODE_COMPUTE. Output uses the standard alphabet with '=' padding AND inserts '\n' after every 72 characters plus a final '\n' ; ENCODE_LENGTH counts them. (fn-h89/lib/encoding/base64.cpp:39-42, 61-62)
F48 FUJI_BASE64_DECODE_INPUT. Payload result ignored : missing payload appends zeros and ACKs. (fn-h89/lib/device/fujiDevice/Base64Mixin.cpp:109)
F49 FUJI_BASE64_DECODE_INPUT. Encode and decode share ONE buffer and there is no clear command: unread output from a previous operation is prepended to the next input. (global base64.base64_buffer, fn-h89/lib/encoding/base64.h:62)
F50 FUJI_BASE64_DECODE_COMPUTE. Encode and decode share ONE buffer and there is no clear command: unread output from a previous operation is prepended to the next input. (global base64.base64_buffer, fn-h89/lib/encoding/base64.h:62)
F51 FUJI_HASH_INPUT. Payload result ignored : missing payload hashes len zero bytes. (fn-h89/lib/device/fujiDevice/HashMixin.cpp:23)
F52 FUJI_HASH_COMPUTE. Unknown algorithm still ACKs, leaves an empty digest (LENGTH 0, OUTPUT 0 bytes) and still clears the data.
F53 FUJI_HASH_LENGTH. Reply is ONE byte, unlike the u32 Base64/QR lengths.
F54 FUJI_QRCODE_INPUT. Payload result ignored (fn-h89/lib/device/fujiDevice/QRMixin.cpp:22)
F55 FUJI_QRCODE_ENCODE. All three fields are required: a two-field packet aborts the firmware.
F56 FUJI_QRCODE_LENGTH. After OUTPUT has consumed the whole code, LENGTH returns 0 and cannot re-render.
F90 FUJI_SET_DEVICE_FULLPATH (emptying a slot). An empty path sets the slot's host slot to FFH but leaves the old path, which FUJI_GET_DEVICE_FULLPATH still returns: the slot is not emptied. And a request that itself gives host slot FFH (what FUJI_READ_DEVICE_SLOTS reports for an empty slot) is refused with NAK, even with an empty path. Only FUJI_UNMOUNT_IMAGE empties a slot. (fn-h89/lib/device/fujiDevice/fujiDevice.cpp:1311-1324, fn-h89/lib/fuji/fujiDisk.cpp:3-12; observed: build/runs/libtest-fujidev-cpm, libtestfix-fujidev-hdos)
F91 FUJI_COPY_FILE (time). The ACK comes only when the copy is finished. A 161,280-byte image from tnfs.fujinet.online took from 14.4 to 18.2 s in six runs, four of them more than the 15 s that fujinet-lib and the book's library wait for a reply (see L21). (observed: build/runs/libtest{,fix,slow}-disk-{hdos,cpm}/tap.jsonl)

NETWORK CHANNELS (71H-78H)

N1 NET_OPEN. Two fields are mandatory even when aux2 is 0; one field (or a single u16) crashes the firmware (NDevice.cpp:136-137, FujiBusPacket.h:75-77)
N2 NET_OPEN. On TNFS:// and SD:// an aux1 other than 4/6/7/8/9/12 reaches abort() - firmware reboot (fn-h89/lib/network-protocol/TNFS.cpp:54-56, SD.cpp:56-58)
N3 NET_OPEN. A '*' in a non-directory URL (e.g. in an HTTP query string) is turned into NUL and truncates the URL (utils.cpp:953)
N4 NET_OPEN. A URL ending in '.' loses the dot (utils.cpp:957-960)
N5 NET_OPEN. Terminate the spec with NUL or CR LF; a lone CR stays inside the URL (NDevice.cpp:27-29)
N6 NET_OPEN. HTTP aux1 4 and 8 URL-encode the path (so do not pre-encode); 5/9/12/13/14 send it verbatim (HTTP.cpp:310-316)
N7 NET_OPEN. UDP OPEN without a port NAKs but leaves STATUS err = 1 (error never set) (UDP.cpp:50-54, NDevice.cpp:182)
N8 NET_CLOSE. HTTP POST (aux1 13/14) and DELETE (5/9) are never sent if no STATUS/READ happened before CLOSE: resultCode 0 means 'no transaction' and close just succeeds (HTTP.cpp:553-564)
N9 NET_READ. TCP: READ with nothing pre-buffered asks the socket for exactly 'length'; a short read NAKs with err 202 and the bytes that did arrive are thrown away . Always STATUS first (it pre-reads into the buffer, Protocol.cpp:169-175) and ask for no more than bytes-waiting (TCP.cpp:123-139)
N10 NET_READ. HTTP body with nothing buffered: client read short -> NAK and the partial bytes are lost (HTTP.cpp:508-514, FS.cpp:303-312)
N11 NET_READ. UDP with empty buffer pads the reply with zero bytes up to 'length' (UDP.cpp:102,115-118)
N12 NET_READ. HTTP GET_HEADERS mode: each READ consumes one header and copies exactly 'length' bytes from it (over-read when length > header); a READ after the last header indexes past the vector - undefined, may crash (HTTP.cpp:490-495)
N13 NET_READ. If NET_SET_PARSER was sent while no channel was open, READ dereferences a null protocol - crash (NDevice.cpp:543-561, NParser.cpp:5)
N14 NET_WRITE. Payload shorter than the length field: zero bytes are appended up to 'length' and written ; longer: the excess is ignored (NDevice.cpp:276-281, rs232.cpp:77-81)
N15 NET_WRITE. A failed write leaves its bytes in the transmit buffer; they are prepended to the next WRITE and retried at CLOSE (FS.cpp:351-356, TCP.cpp:166-196, Protocol.cpp:127-128). NET_SET_HTTP_MODE clears the buffer (HTTP.cpp:69-70)
N16 NET_WRITE. Write translation looks for the full native EOL (CR LF unless NET_SET_EOL changed it); a lone CR is not translated (Protocol.cpp:231, rs232.h:165)
N17 NET_WRITE. A STATUS on an HTTP PUT channel before CLOSE fires the PUT early with the data so far (empty body -> client returns -1 ->

err 144); CLOSE then PUTs again (HTTP.cpp:430-436, 553-554, fn-h89/lib/http/mgHttpClient.cpp:982-983)
N18 `NET_STATUS`. No channel: 00 00 00 CF (207 NOT_CONNECTED) (NDevice.cpp:309-314); after a failed OPEN or CLOSE: 00 00 00 <reason> (NParser.h:31-51)
N19 `NET_STATUS`. UDP under-reports: STATUS pre-reads the waiting datagram into the receive buffer, then computes avail from udp.available() - the NEXT datagram - so a single pending datagram shows avail 0 . The datagram is still returned by a READ (Protocol.cpp:169-175, NDevice.cpp:317-319, UDP.h:72, fn-h89/lib/tcpip/fnUDP.cpp:286-314)
N20 `NET_STATUS`. err 136 appears as soon as an FS file or HTTP body is fully consumed, including right after a JSON PARSE (body consumed) and before QUERY fills the buffer (FS.cpp:398, HTTP.cpp:443-444)
N21 `NET_STATUS`. After a failed PARSE err stays 213 (JSON) or 144 (HTML/XML) on every STATUS (NParser.cpp:58-59)
N22 `NET_STATUS`. HTTP GET_HEADERS mode: STATUS (and the background interrupt poll via available()) fires the request if not done yet (HTTP.cpp:456-458, 959-961)
N23 `NET_PARSE`. On a stream that stays connected (e.g. TCP) PARSE does not return until the peer disconnects (fnjson.cpp:364); the H89 library times out after 15 s of silence (fujinet-lib/bus/h89/fuji_bus_call.c:14)
N24 `NET_QUERY`. Querying a value that is, or contains, an empty array [] crashes the firmware: the array loop dereferences a null child . Any query whose result includes [] - including the empty query on a document containing [] - triggers it (fnjson.cpp:301-307)
N25 `NET_QUERY`. Flag 0x04 with a string holding '<' but no later '>' loops forever (fnjson.cpp:135-144)
N26 `NET_QUERY`. HTML/XML: the RS232 handler keeps a trailing NUL and any bytes after it in the selector string - send exactly the selector bytes (rs232Network.cpp:9; fnhtml.cpp:80-112 compares/parses the whole std::string)
N27 `NET_QUERY`. NET_SET_PARSER while no channel was open, then QUERY -> null protocol dereference crash (JSONParser.cpp:23)
N28 `NET_SET_PARAMETER`. NET_SET_PARSER builds a fresh parser and resets flags and line ending: send SET_PARAMETER after SET_PARSER (NDevice.cpp:543-569, JSONParser.h:14-17)
N29 `NET_SET_PARAMETER`. param 1 with parser NONE after SET_PARSER-before-OPEN dereferences a null protocol (NParser.cpp:68-72)
N30 `NET_SET_PARSER`. Does not check for an open channel: with none it creates a parser bound to a null protocol, and a later READ, QUERY, SEEK/TELL (mode 0) or SET_PARAMETER 1 (mode 0) crashes the firmware (NDevice.cpp:548-561, NParser.cpp:5,33,70, JSONParser.cpp:23)
N31 `NET_SET_PARSER`. OPEN resets the parser to NONE : send SET_PARSER after OPEN (NDevice.cpp:154,188)
N32 `NET_SET_PARSER`. Write HTTP POST data before switching to JSON - writes NAK afterwards (JSONParser.cpp:3-6)
N33 `NET_TRANSLATION`. OR semantics: override 1 with aux2 2 gives mode 3 (CR LF) (NDevice.cpp:174)
N34 `NET_SET_EOL`. A NUL terminator in the payload becomes part of the line ending
N35 `NET_SET_INT_RATE`. rate 0: poll_interrupt divides by timerRate*2 - integer division by zero, firmware crash once a channel is open on that unit (NDevice.cpp:861,867); poll_interrupt runs on the H89 card too (rs232.cpp:173-183)
N36 `NET_SET_INT_RATE`. On the H89 card the data-ready signal goes nowhere: ESP32 calls setRI(), a no-op on PPIChannel (rs232.cpp:186-187, PPIChannel.h:48); fujinet-pc drives DTR/RTS only on a real serial port with serial_proceed configured, never under BoIP (rs232.cpp:184-199)
N37 `NET_SEEK`. The seek is performed twice - on HTTP that is two Range requests (NDevice.cpp:672-676)
N38 `NET_TELL`. No channel: the 4 bytes are uninitialised stack contents - u32le_t has no constructor (NDevice.cpp:697-704, fn-h89/include/global_types.h:69-71)
N39 `NET_TELL`. TNFS/SD: the seek(0,SEEK_CUR) clears the receive buffer, so bytes STATUS pre-read (translation read-ahead) are discarded and the position reported is past them (TNFS.cpp:333-341, SD.cpp:338-346)
N40 `NET_TELL`. HTTP reports bytes fetched from the server so far (HTTP.cpp:510-512, 811-812)
N41 `NET_TELL`. Parser NONE bound to a null protocol (SET_PARSER before OPEN) -> crash (NParser.cpp:33)
N42 `NET_CHDIR`. CR/LF are NOT stripped (unlike OPEN): a CR LF-terminated payload puts CR LF into the prefix and into every later relative OPEN (NDevice.cpp:420)
N43 `NET_CHDIR`. A packet with no payload clears the prefix (transaction_get failure ignored, NDevice.cpp:419)
N44 `NET_USERNAME`. If the user name is empty the password is not passed either (NetworkProtocolFactory.cpp:125-129)
N45 `NET_PASSWORD`. Ignored unless a user name is also set (NetworkProtocolFactory.cpp:125)
N46 `NET_RENAME`. Kills whatever channel was open on the unit - buffered writes/HTTP PUT lost, no close (NDevice.cpp:498,738-739)
N47 `NET_RENAME`. Failure reason is lost: STATUS afterwards says 207 (NDevice.cpp:309-314)
N48 `NET_RENAME`. TNFS without a comma ACKs and does nothing (TNFS.cpp:231-232)
N49 `NET_RENAME`. FTP: ACK, nothing done (FTP.h:37); GDRIVE has no override - only parses, ACKs (FS.cpp:463-486, GDRIVE.h:41-43)
N50 `NET_RENAME`. HTTP: WebDAV MOVE with Destination set to a server path, not a full URL (HTTP.cpp:885-900)
N51 `NET_DELETE`. FTP: ACK, nothing deleted (FTP.h:44)
N52 `NET_DELETE`. channel on the unit destroyed; error reason lost
N53 `NET_LOCK`. Silent no-op ACK on FTP (FTP.h:65), NFS (NFS.cpp:360-363) and every protocol without an override - HTTP, SMB, S3, GDRIVE, ONEDRIVE (FS.h:124)
N54 `NET_UNLOCK`. Silent no-op ACK where not implemented (FS.h:131; NFS.cpp:365-368; FTP has no unlock override so the FS default applies)
N55 `NET_MKDIR`. FTP: ACK, nothing created (FTP.h:51)
N56 `NET_MKDIR`. TNFS MKDIR/RMDIR never unmount their session (TNFS.cpp:260-284)
N57 `NET_RMDIR`. FTP: ACK, nothing removed (FTP.h:58)
N58 `NET_ACCEPT`. Poll STATUS first: on a listener with no client 'connected' = 1 means a client is waiting (TCP.cpp:225-229)
N59 `NET_CLOSE_CLIENT`. Return value inverted: NAK = client closed, ACK = nothing was closed (TCP.cpp:347,354,370)
N60 `NET_SET_HTTP_MODE`. Any response bytes still buffered are discarded by the mode switch (HTTP.cpp:69)
N61 `NET_GET_REMOTE`. Overwrites the first ~22 bytes of an unread datagram, and writes past the string's allocation when the receive buffer is empty/small; then sends 256 bytes read from that storage (NDevice.cpp:834-835)
N62 `NET_GET_REMOTE`. Before any datagram arrived the address is fnUDP's initial remote_ip/remote_port (fn-h89/lib/tcpip/fnUDP.cpp:349-357)
N63 `NET_SET_DESTINATION`. ESP32 extracts the host with substr(first_colon+1, last_colon-2), correct only when the first colon is at index 1: 'N:host:port' works, 'N1:host:port' yields 'host:' and sends fail (UDP.cpp:211-212)
N64 `NET_SET_DESTINATION`. fujinet-pc builds the string with strlen on a payload that is not NUL-terminated - send a trailing NUL (UDP.cpp:197-200, NDevice.cpp:847)
N65 `NET_SET_DESTINATION`. Any STATUS after a datagram arrives replaces the destination with that datagram's sender (UDP.cpp:169-183)
N90 `NET_QUERY (ASCII flag)`. With query flag 10H (ASCII output) a JSON string's LF line ending is turned into a space, which network_json_query() does not strip: "FujiNet " comes back for "FujiNet". Numbers are not affected. (fn-h89/lib/fnjson/fnjson.cpp:228, fn-h89/lib/fntext/fn_sanitize.cpp:658-660)
N91 `NET_READ on HTTP (correction)`. A READ larger than what is buffered returns what is there (FS.cpp caps the request to available()); it NAKs only when nothing is available. (fn-h89/lib/network-protocol/FS.cpp:294-297)
N92 `NET_GET_REMOTE`. Overwrites a buffered datagram (the next READ returned '127.'); before any datagram it answers 255.255.255.255:0. (observed on fujinet-pc; fn-h89/lib/device/NDevice/NDevice.cpp:834-835)

THE BUS

B1 `framing`. On 0xC0 the bus collects bytes until a second 0xC0 has been read; that span is the frame . A frame must therefore be exactly C0 <body> C0: a doubled leading C0 (C0 C0 <body> C0) produces an empty 2-byte frame that is rejected silently, the body bytes are then read as raw modem data, and the trailing C0 opens a bogus frame (derived from rs232.cpp:99-116 + 337-359 and FujiBusPacket.cpp:150-151) (fn-h89/lib/bus/rs232/rs232.cpp:337-366)
B2 `framing`. Frame rejected if: encoded span < 8 bytes or does not start and end with 0xC0 ; decoded < 6 bytes (157-158); header length != decoded length (164-165); checksum mismatch (170-176); descriptor chain or fields run past the end (193-194, 212-213) (FujiBusPacket.cpp:150-153)
B3 `params`. param(n) returns _params.at(n).value as uint32 regardless of the field's wire size ; devices never check the field width, so a u8, u16 or u32 field all work where a number is expected (fn-h89/lib/bus/rs232/FujiBusPacket.h:75-77)
B4 `params`. A missing field makes .at() throw std::out_of_range. ESP32 H89 build has C++ exceptions off (fn-h89/sdkconfig.fujinet-heathkit-h89:422) so it aborts (reboot); fujinet-pc terminates on the uncaught exception. Guarded: disk R/P/W (disk.cpp:385-388), disk STATUS (395-396), clock alt flag (rs232Clock.h:14-17), printer STATUS (printer.cpp:241). Unguarded: modem SET_DUMP, LISTEN, BAUDRATELOCK (2 fields), AUTOANSWER, STATUS (modem.cpp:1632, 1635, 1641, 1644, 1648)
B5 `replies`. Data is attached only when ack is true; a NAK never carries data, even when the handler supplied some (rs232.cpp:390-394)
B6 `replies`. Exactly one reply per accepted frame except: bad frame (silence), printer disabled , modem disabled (silence, modem.cpp:1612-1615), modem ACK/NAK (no bytes), and ROM mount pushes (extra unsolicited frames to device 0xFF before the Fuji reply, disk.cpp:28-110) (silence, printer.cpp:226-229)
B7 `replies`. transaction_success asserts the state is NO_GET/DID_GET ; assertions are enabled on the ESP32 H89 build (sdkconfig.fujinet-heathkit-h89:414), so a success after an error aborts (see modem LISTEN 0) (rs232.cpp:55)
B8 `errors`. Read timeout inside a frame (gap longer than the channel read timeout): frame is incomplete, parse fails, silence (rs232.cpp:355-359, 365). The bytes that arrive after the gap are then read as raw modem data until the next C0 (rs232.cpp:99-107)

B9 `errors`. The bus does not consult device_active / enable state before dispatch ; each device decides (rs232.cpp:126-128)

DISK DRIVES (31H-38H)

D1 `DISK_READ`. Sector numbers are 0-based although diskTypeImg.cpp:16 says '1-based' (diskTypeImg.cpp:17-20)
D2 `DISK_READ`. A trailing partial 512-byte block of an IMG is unreachable: sector count = size/512 truncated (diskTypeImg.cpp:162)
D3 `DISK_READ`. An IMD sector with a data error or deleted mark is still returned with ACK; only DISK_STATUS shows it (diskTypeIMD.cpp:173-183)
D4 `DISK_WRITE`. verify flag (WRITE vs PUT) is ignored by every media type (diskTypeImg.cpp:63-105; diskTypeIMD.cpp:193)
D5 `DISK_WRITE`. IMD out-of-range sector: sector_size()=0 -> zero-length get succeeds -> write() NAKs (diskTypeIMD.cpp:245-250; rs232.cpp:71-72)
D6 `DISK_PUT`. Identical to DISK_WRITE; the only difference is ignored downstream (verify=false, disk.cpp:392)
D7 `DISK_STATUS`. Answers even with nothing mounted: 00 FF E0 00 (disk.cpp:206-217)
D8 `DISK_STATUS`. IMG never reports double density: _disk_sector_size stays at its 128 default because MediaTypeImg::mount never sets it (diskTypeImg.cpp:157-166, 111-112). Normal IMG reply: 00 FF E0 00 (diskType.h:57)
D9 `DISK_STATUS`. ROM media: 00 00 00 00 (diskTypeROM.cpp:48-51)
D10 `DISK_STATUS`. After unmount the stale media object still answers (disk.cpp:212-213)
D11 `DISK_STATUS`. Was mapped onto WRITE before 7ab84ae64 (commit message); fixed at HEAD (disk.cpp:394-397)
D12 `DISK_FORMAT`. FORMAT does not format: IMG just ACKs (diskTypeImg.cpp:130-142)
D13 `DISK_FORMAT`. Reply length is _disk_sector_size = 128, not 512 (diskTypeImg.cpp:139; diskType.h:57)
D14 `DISK_FORMAT`. No device_active check: after unmount an IMG drive still ACKs (disk.cpp:226)
D15 `DISK_FORMAT_MEDIUM`. Alias of DISK_FORMAT
D16 `DISK_PERCOM_READ`. IMG: all zeros, derive_percom_block is empty - unless a DISK_PERCOM_WRITE stored something (fn-h89/lib/media/rs232/diskType.cpp:39-42)
D17 `DISK_PERCOM_READ`. IMD: filled at mount: tracks=max cyl+1, step 1, spt=max, sides=heads-1, density, sector_size=largest sector, drive_present 0xFF (diskTypeIMD.cpp:66-77)
D18 `DISK_PERCOM_READ`. Layout from fn-h89/lib/media/rs232/diskType.h:62-75
D19 `DISK_PERCOM_WRITE`. ACKs even with no payload: transaction_get result is ignored (disk.cpp:269-273)

CLOCK (45H)

C1 `APETIME_GETTIME`. Year byte is tm_year-100, i.e. two-digit year from 2000 (fujiClock.cpp:124)
C2 `APETIME_GETTZTIME`. With no alternate TZ set it falls back to the system TZ (fujiClock.h:30-33)
C3 `APETIME_SETTZ_ALT2`. Despite the name this is the SIMPLE time GETTER on RS232: the read table claims 0x54 before the write table (fujiClock.cpp:350-352, 385-390, 406-409)
C4 `APETIME_GET_SOS`. NUL included because rs232Clock does not override string_needs_null() (fujiClock.h:76; fujiClock.cpp:264-268)
C5 `APETIME_SETTZ`. ACK is sent BEFORE validation ; an invalid TZ (e.g. payload starting with NUL, or a control char) is silently ignored but still ACKed (fujiClock.cpp:212)
C6 `APETIME_SETTZ`. Any printable string is stored; POSIX validity is not checked
C7 `APETIME_SETTZ_ALT`. ACK before validation, invalid TZ silently dropped (fujiClock.cpp:212, 317-321)
C8 `APETIME_SETTZ_ALT`. Persists across reboots (writes config)

PRINTER (40H)

R1 `PRINTER_WRITE`. Buffer overrun: memcpy(provideBuffer(), data, len) with no bound into uint8_t buffer[320] - payloads > 320 bytes overwrite the emulator object (printer.cpp:46; fn-h89/lib/printer-emulator/printer_emulator.h:41, 72)
R2 `PRINTER_WRITE`. Length truncated to 8 bits: process(uint8_t linelen, ...) gets len mod 256, so 256 bytes print nothing (still ACK) and 300 bytes print the first 44 (printer.cpp:48; printer_emulator.h:68)
R3 `PRINTER_WRITE`. Default type FILE_TRIM writes up to and including the first 0x9B (ATASCII EOL) and drops the rest of the record . setEOL('\r') does not affect the file printer (fn-h89/lib/printer-emulator/file_printer.cpp:20-27; fn-h89/include/atascii.h:12)
R4 `PRINTER_WRITE`. FILE_ASCII drops CR and LF (keeps only 32..122 except 96, after &0x7F) so lines run together on the H89 (file_printer.cpp:28-58)
R5 `PRINTER_WRITE`. Printer disabled in config -> no reply at all (silence), not a NAK (printer.cpp:226-229); default enabled (fn-h89/lib/config/fnConfig.h:487)
R6 `PRINTER_PUT`. Alias of PRINTER_WRITE
R7 `PRINTER_STATUS`. Byte 1 is an uninitialized member (printer.h:27; printer.cpp:96; object created with new at main.cpp:360)
R90 `output`. fujinet-pc writes the printer's output to the file paper in its SD directory. (fn-h89/lib/printer-emulator/printer_emulator.cpp:7, 83-89)

MODEM (50H)

M1 `MODEM_CONTROL`. No reply bytes at all
M2 `MODEM_CONFIGURE`. Always 9600 regardless of the request
M3 `MODEM_SET_DUMP`. Missing field aborts the firmware
M4 `MODEM_LISTEN`. port 0: transaction_error() then transaction_success() -> assert(state) fails -> abort on the ESP32 (assertions on) (modem.cpp:372-380; rs232.cpp:55; sdkconfig.fujinet-heathkit-h89:414)
M5 `MODEM_BAUDRATELOCK`. Two fields required
M6 `MODEM_STATUS`. Reply is 2 raw bytes, NOT a SLIP frame (rs232.cpp:381-386)
M7 `MODEM_WRITE`. cmdOutput is forced false for the whole command and every at_cmd_println() returns early when it is false (modem.cpp:468, 484, 504, 525): OK/ERROR/help/IP/echo text is never sent for commands given this way (modem.cpp:105, 113)
M8 `MODEM_WRITE`. Numeric results (ATV0) still emit CR LF but the digits go to an unsent _packetData buffer (modem.cpp:456-461, 1665-1683)
M9 `MODEM_WRITE`. No payload -> returns without action (modem.cpp:100-101)
M10 `MODEM_STREAM`. fujinet-pc on a TTY: the HOST LINK baud changes to modemBaud (e.g. 9600 after MODEM_CONFIGURE) (rs232.cpp:323-335); PPI: setBaudrate is a no-op (PPIChannel.h:39); BoIP: skipped (rs232.cpp:333)
M11 `MODEM_STREAM`. Raw 9-byte reply, not a frame
M90 `ATDT / ATA`. After ATDT the far end's first data reaches the H89 before CONNECT (about a second ahead); after ATA a bare CR LF comes first, then the caller's data, then CONNECT. (observed on fujinet-pc; fn-h89/lib/device/rs232/modem.cpp:1039-1040)
M91 `bare AT commands`. AT commands sent as bytes outside a frame are acted on about 500 ms after the last byte (the BoIP channel's read timeout on fujinet-pc). (observed on fujinet-pc)

CP/M DEVICE (5AH)

X1 `CPM_INIT`. Not registered, and ATCPM dereferences the null CPM pointer (modem.cpp:1256-1257)

FUJINET-LIB

L1 `descriptor flags on the wire`. fuji_bus_call puts its FUJI_FIELD_DATA (08H) and FUJI_FIELD_REPLY (10H) flags in the packet's descriptor byte. The firmware ignores bits 3-6, so no harm, but the bytes differ from other FujiBus clients'. (fujinet-lib/bus/h89/fuji_bus_call.c:201)
L2 `port_putc`. No timeout: with the FujiNet not running, a C program hangs in its first call. (fujinet-lib/bus/h89/portio.c:160-173)
L3 `FUJI_DIR_ENTRY_IS_DIR / _IS_TRUNC`. Use is_dir/is_trunc, which the H89's FujiDirEntry does not have (it has flags): code using them does not compile. Test flags & 1 and flags & 2. (fujinet-lib/include/fujinet-fuji.h (FUJI_DIR_ENTRY_IS_*; FujiDirEntry for FUJI_PLATFORM_H89))
L4 `fixed 256-byte payloads`. fuji_open_directory, fuji_set_host_prefix, fuji_set_device_filename and fuji_copy_file always send 256 bytes from the caller's string, so whatever follows its NUL is sent too (and the firmware may read it as a pattern or part of a path). Clear a 256-byte buffer first. (fujinet-lib/include/fujinet-fuji.h (MAX_FILENAME_LEN in the FUJICALL_..._D macros))
L5 `network_json_query`. Always sends 256 bytes of query and reads the answer into the caller's buffer with no limit. (fujinet-lib/common/network_json_query.c:12-30)
L6 `network_bus_read`. Returns the length asked for on any ACK, even when fewer bytes came. (fujinet-lib/bus/h89/network.c:6-9)
L7 `fuji_get_wifi_enabled`. Ignores whether the call succeeded and returns the last value it got. (fujinet-lib/common/fuji_get_wifi_enabled.c)
L8 `clock_get_time`. Writes a NUL one byte past the reply: the buffer must be one byte longer than the format's reply. (fujinet-lib/common/fuji_clock.c:51)
L9 `network_udp_set_destination`. Sends the devicespec without a NUL and with the unit prefix (N7:), which the firmware reads wrongly; the datagram does not arrive. Use fuji_bus_call with 'N:host:port' and a NUL. (fujinet-lib/common/network_udp_set_destination.c)
L10 `network_read_nb`. Busy-loops forever while STATUS says nothing waiting and no error, e.g. on UDP (whose STATUS under-reports) or an idle server. (fujinet-lib/common/network_read_nb.c)
L11 `fuji_get_time, fuji_enable_device, fuji_disable_device`. Send commands the H89's FujiNet does not have: always false. (fujinet-lib/include/fujinet-fuji.h:268-269, 649)
L12 `unlinkable`. fn_error, network_proceed, network_has_proceed, network_get_dstats, fuji_enable_udpstream, fuji_get_device_enabled_status and _fuji_status are declared but not built for the H89. (fujinet-lib/include/fujinet-network.h:49, 341-343; fujinet-fuji.h:239, 272, 300)
L13 `fuji_get_host_slots / fuji_get_device_slots`. The count must be 8: the reply is always all eight slots, and a smaller buffer makes the call fail. (fujinet-lib/bus/h89/port_slip.c:87-90)
L14 `network_unit`. No range check: "N0:" addresses the FujiNet device (70H), so a network command arrives there as some other command (NET_SET_PARAMETER, FBH, as SET_SSID); "N9:" goes to 79H, "N10:" to unit 1. (fujinet-lib/common/network_unit.c:3-12)
L15 `network_unit_status`. Returns 0 on success (the macro is !fuji_bus_call). (fujinet-lib/include/fujinet-network.h:492)
L16 `fuji_open_directory_filter`. With a NULL pattern (which the header allows) it reads address 0 and sends what it finds as the pattern; with an

empty one it sends the caller's memory after the path. (fujinet-lib/common/fuji_open_directory_filter.c:16-19)
L17 `fuji_hash_data / fuji_hash_calculate`. Hex output has no NUL: the buffer needs fuji_hash_size() + 1. (fujinet-lib/common/fuji_hash.c:28)
L18 `network_json_strip_newlines`. Writes buffer[buflen], one past the data. (fujinet-lib/common/network_json.c:17)
L19 `qrcode_create`. Returns negative NETWORK_ERROR codes (-162, -215), not FN_ERR values. (fujinet-lib/common/qrcode_create.c:12-27)
L20 `port_getbuf_slip_dual`. Does not read the closing END of a reply that fills its buffer exactly, which includes every bare ACK and NAK; the next read starts with a stray C0H. Harmless for fujinet-lib's own calls (they resynchronise on END), but a program mixing in raw reads (the modem) must drop it first. (fujinet-lib/bus/h89/port_slip.c)
L21 `reply timeout`. fuji_bus_call() waits a fixed 15 s (TIMEOUT_SLOW) for a reply and has no way to drop a reply that arrives later. When FUJI_COPY_FILE took longer, fuji_copy_file() returned false and the late ACK was taken as the reply to the next request (a STATUS), after which every reply was one behind and the test program reported the FujiNet BROKEN. With TIMEOUT_SLOW at 60 s the copy test passed. Proposed: a longer timeout for slow commands, and a drain before each request. (fujinet-lib/bus/h89/fuji_bus_call.c:14; observed: build/runs/libtest-disk-hdos/tap.jsonl)
L22 `port_init (9e6d4dd)`. Took EN (PC2) high to mean the card was set up, and skipped the mode word. After a reset the 82C55A's bus hold can make PC2 read 1 while port A is not in Mode 2, so on a real H89 under HDOS the first call waited forever. CP/M's BIOS writes a mode word at boot, which hid it there, and FujiNet Go H89 before 51f411d read PC2 as 0 after a reset. Fixed in d08f584 with the mark in port B (Chapter 2). fujinet-h89's card_on (FN.DVD, FNDISK) before 4cb0024 and FNON before 51f411d made the same test. (fujinet-lib/bus/h89/portio.c:91-107 (9e6d4dd))

FUJINET-LIB'S TEST PROGRAMS

T1 `network: line ending`. BASIC_LINE_ENDING is "\n" for the H89, but the FujiNet's line ending on this bus is CR LF. The echo test opens with OPEN_TRANS_LF, so its LF comes back as CR LF, and the test, reading only as many bytes as it sent, compares CR with LF and stops: only 5 of the network program's 82 tests run. Fix: give the H89 "\r\n", as __WATCOMC__ has; then all 82 pass under both systems. (fujinet-lib/testing/tests/network.c:16-25, 485)
T2 `disk: directory entry offset`. DIRENTRY_FILENAME_OFFSET is 13 for the H89, but on this bus a FUJI_READ_DIR_ENTRY header is 12 bytes (as for MS-DOS), so the name is compared from its second letter and the copied lobby.dsk is never found. Fixed (the H89 given 12, as MS-DOS has) and with TIMEOUT_SLOW raised (L21), the disk test passes under both systems. (fujinet-lib/testing/tests/constants.h:60-66; fdsk.c:197)
T3 `fujidev: restoring a slot`. The test puts back an empty drive slot with FUJI_SET_DEVICE_FULLPATH, an empty path and the host slot FFH it read. The firmware refuses FFH, and even with 0 it leaves the test's name in the slot (F90), so the read-back fails and drive slot 0 is left named /test_filename. Fix: put back an empty slot with fuji_unmount_disk_image(). Fixed so, all 88 of the fujidev program's tests pass under both systems. (fujinet-lib/testing/tests/fuji.c:250)

Z88DK (HDOS)

Z1 `load address`. The HDOS target's default CRT_ORG_CODE is 4480H: the split-octal address 042.200 (2280H) read as the plain octal number 042200. Programs work but waste 8.6K. Proposed fix: CRT_ORG_CODE = $2280 with a REGISTER_SP that is not inside the program. (z88dk/lib/target/hdos/classic/hdos_crt0.asm (TAR__register_sp $447F, CRT_ORG_CODE $4480))
Z2 `command line`. With startup=3 the start-up code pushes the entry SP, moves the stack (crt_init_sp, crt_init_atexit), then pops from the NEW stack, so the command line is split with a wrong length and argv[] is damaged. Proposed fix: replace the push/pop with ld hl,(__restore_sp_onexit+1). The book builds with CRT_ENABLE_COMMANDLINE=0 and hdos_args(). (z88dk/lib/target/hdos/classic/hdos_crt0.asm (start: push hl ... pop hl))
Z3 `hdos.def`. The HDOS system values (S_SYSM, S_USRM, S_DATE ...) are given 2000H too high, e.g. S_SYSM $40D0 for 20D0H: the same split-octal misreading. (z88dk/lib/target/hdos/def/hdos.def:47-52)
Z5 `unsigned long %`. In ISS.C, t % 60UL (and % 60, (t / 60UL) % 60UL, (t / 3600UL) % 24UL) gave wrong results for t = 1760270400: the seconds came out 36, which is the remainder of t's low 16 bits. Division of the same long was right. The book subtracts whole days and works in 16 bits instead. Seen under HDOS with sccz80's default options; not narrowed down further. (observed: build/runs/iss-c-hdos (before the fix); src/c/iss/iss.c print_stamp())

Z88DK

Z4 `-l with a dotted library name`. zcc -lfujinet.h89-hdos does not find fujinet.h89-hdos.lib (the dot stops .lib being added); give the name in full: -lfujinet.h89-hdos.lib. (observed with z88dk v2.4-1231)

FUJINET-PC (THE BOOK'S STAND-IN FOR THE CARD)

P1 `directory details`. READ_DIR_ENTRY with 80H returns blank details (size 0, 1969-12-31, in the host's time zone) for files below the top of the SD directory, and network directory listings show size 0: dir_open looks files up in the wrong place. The card is not affected. (fn-h89/lib/filesystem/fnFsSD.cpp:306 (PC build))
P2 `WiFi`. A stand-in: always connected to 'Dummy Cafe'; a scan finds 'Dummy Cafe 0'-'15'; SET_SSID always succeeds. (fn-h89/lib/hardware/fnWiFi (PC))
P3 `shutdown with a TCP channel open`. fujinet-pc crashed (segmentation fault) while shutting down, in NetworkProtocolTCP's destructor, each time it was stopped with a TCP channel still open: a connection, after fujinet-lib's network test had stopped early, and a listening channel, after the corrected test had finished. Only at shutdown; the card never shuts down this way. (observed: build/runs/libtest-network-{hdos,cpm}/fujinet.log, libtestfix-network-hdos/fujinet.log)

GLOSSARY

ACK. The reply that means “done”: 06H in the reply’s command byte.
Channel. One of the eight network connections the FujiNet keeps, devices 71H to 78H.
Checksum. An 8-bit sum with end-around carry over a packet, in byte 4 of its header.
Descriptor. Byte 5 of a packet’s header: how many fields follow, and their size.
Device. Something the FujiNet makes look like a peripheral: itself (70H), a disk drive, a network channel, the clock, the printer, the modem.
Devicespec. The text that tells a network channel what to open: `N:HTTP://host/path`.
END. C0H: the byte that starts and ends a SLIP frame.
ESC. In SLIP, DBH, which begins an escape pair. (On the H19, 1BH, which begins an escape sequence.)
Field. A small number sent in a request’s header: a slot, a length, a mode.
fujinet-lib. The FujiNet project’s C library, one function per command.
fujinet-pc. The FujiNet firmware built to run on a personal computer.
FujiBus. The FujiNet’s request-and-reply protocol over a byte stream (Chapter 3).
Host slot. One of eight places the FujiNet keeps the name of a server of disk images.
JSON Pointer. A path through a JSON document, such as `/people/0/name` (RFC 6901).
NAK. The reply that means “refused”: 15H.
Payload. The data that follows a request’s fields.
Request block. In the book’s library, the two lines (FNREQ, FNBUF) that describe one request.
SLIP. Serial Line Internet Protocol: the framing FujiBus uses.
Split octal. Heath’s way of writing a 16-bit address: each byte in octal, as 042.200.
TNFS. The Tiny Network File System, made for 8-bit computers.
Translation. Changing line endings between the network’s and the computer’s (Chapter 10).

FUJINET

for the H89

FUJINET PROJECT • FUJINET.ONLINE

THE WORLD'S FRIENDLIEST NETWORK ADAPTERS FOR VINTAGE COMPUTERS

PRINTED WHEREVER YOU ARE